Database Management Systems

Theory · Design · SQL · Implementation · Modern Database Systems

Prepared By - Dr Mohammed Mahmudur Rahman and Yeakob Ali

Database Systems · Laboratory Courses · Self-Study · Viva and Interview Preparation

Contents

Preface . . . . . **x**

How to Use This Book . . . . . **xi**

Learning Outcomes . . . . . **xii**

1 Introduction to Database Systems **1**

1.1 What is Data? 1
1.2 What is a Database? 3
1.3 What is a DBMS? 4
1.4 Database System vs DBMS vs Database 6
1.5 Components of a Database System 7
1.6 Database Users 8
1.7 Database Administrator 9
1.8 Database Applications 10
1.9 Advantages of DBMS 10
1.10 Limitations of DBMS 11

2 File Systems and DBMS **16**

2.1 Traditional File Processing System 16
2.2 Problems with File Systems 17
2.3 File System vs DBMS 18

3 Database Architecture **23**

3.1 One-Tier Architecture 23
3.2 Two-Tier Architecture 23
3.3 Three-Tier Architecture 24
3.4 Three-Schema Architecture 26
3.5 Data Independence 27

4 Data Models **32**

4.1 What a Data Model Provides 32
4.2 Hierarchical Model 32
4.3 Network Model 33
4.4 Relational Model 34
4.5 Object-Oriented Model 34
4.6 Object-Relational Model 34
4.7 Semi-Structured Model 35
4.8 Comparison of Data Models 35
4.9 Why the Relational Model Became Dominant 35

5 The Relational Model **40**

5.1 Relation 40
5.2 Tuple 41
5.3 Attribute 41
5.4 Domain 41

5.5 Schema . . . 41
5.6 Instance . . . 42
5.7 Relational Integrity Rules . . . 42
5.8 Advantages and Limitations of the Relational Model . . . 43

6 Keys and Constraints . . . 47
6.1 Super Key . . . 47
6.2 Candidate Key . . . 47
6.3 Primary Key . . . 48
6.4 Alternate Key . . . 49
6.5 Composite Key . . . 49
6.6 Surrogate Key . . . 49
6.7 Foreign Key . . . 50
6.8 Key Summary Table . . . 51
6.9 A Complete Worked Example in SQL . . . 52

7 Entity-Relationship Model . . . 58
7.1 Basic Concepts . . . 58
7.2 Types of Attributes . . . 60
7.3 Weak Entity . . . 61
7.4 Degree of Relationship . . . 62
7.5 A Complete ER Example . . . 63

8 Advanced ER (EER) Modeling . . . 68
8.1 Specialization . . . 68
8.2 Generalization . . . 68
8.3 Inheritance . . . 69
8.4 Constraints on Specialization . . . 69
8.5 Specialization Hierarchy and Lattice . . . 70
8.6 Category (Union Type) . . . 70
8.7 Aggregation . . . 71
8.8 Real-Life EER Example: A Bank . . . 72

9 ER to Relational Mapping . . . 77
9.1 Step 1 — Strong Entity Sets . . . 77
9.2 Step 2 — Weak Entity Sets . . . 78
9.3 Step 3 — Binary 1:1 Relationships . . . 78
9.4 Step 4 — Binary 1:N Relationships . . . 78
9.5 Step 5 — Binary M:N Relationships . . . 79
9.6 Step 6 — Multivalued Attributes . . . 80
9.7 Step 7 — Specialization and Generalization . . . 80
9.8 Mapping Higher-Degree Relationships and Aggregation . . . 81
9.9 Complete Worked Example . . . 82

10 SQL Fundamentals . . . 87
10.1 SQL Syntax Rules . . . 87
10.2 The Five Command Categories . . . 87
10.3 DDL — Data Definition Language . . . 88
10.4 DML — Data Manipulation Language . . . 89
10.5 DQL — Data Query Language . . . 90
10.6 DCL — Data Control Language . . . 90
10.7 TCL — Transaction Control Language . . . 90
10.8 A Complete First Session . . . 91

11 SQL Querying . . . 96

11.1 Creating the Sample Database . . . 96
11.2 Populating the Sample Database . . . 98
11.3 The Logical Order of Evaluation . . . 99
11.4 SELECT and Column Selection . . . 99
11.5 WHERE — Filtering Rows . . . 100
11.6 BETWEEN, IN, LIKE and IS NULL . . . 101
11.7 DISTINCT . . . 101
11.8 ORDER BY . . . 101
11.9 LIMIT and Pagination . . . 102
11.10 Aliases . . . 102
11.11 GROUP BY — Aggregation per Group . . . 102
11.12 HAVING — Filtering Groups . . . 103
11.13 Twelve Worked Queries . . . 103

12 SQL Functions . . . 110
12.1 Aggregate Functions . . . 110
12.2 String Functions . . . 111
12.3 Numeric Functions . . . 111
12.4 Date and Time Functions . . . 112
12.5 Conditional and NULL Functions . . . 113

13 SQL Joins . . . 118
13.1 Why Joins Exist . . . 118
13.2 INNER JOIN . . . 118
13.3 LEFT OUTER JOIN . . . 119
13.4 RIGHT OUTER JOIN . . . 120
13.5 FULL OUTER JOIN . . . 120
13.6 CROSS JOIN . . . 120
13.7 SELF JOIN . . . 120
13.8 NATURAL JOIN and USING . . . 121
13.9 Multi-Table Joins . . . 121
13.10 Join Comparison . . . 122

14 Subqueries and Set Operations . . . 127
14.1 Where a Subquery May Appear . . . 127
14.2 Single-Row Subqueries . . . 127
14.3 Multi-Row Subqueries: IN, ANY, ALL . . . 128
14.4 Correlated Subqueries . . . 129
14.5 EXISTS and NOT EXISTS . . . 129
14.6 Derived Tables and Common Table Expressions . . . 130
14.7 Subqueries in DML . . . 130
14.8 Set Operations . . . 131

15 Views, Triggers and Stored Procedures . . . 136
15.1 Views . . . 136
15.2 Triggers . . . 137
15.3 Stored Procedures . . . 139
15.4 User-Defined Functions . . . 140

16 Integrity Constraints . . . 145
16.1 Domain Constraints . . . 145
16.2 Entity Integrity . . . 146
16.3 Referential Integrity . . . 146
16.4 Semantic and Business Constraints . . . 147
16.5 When Constraints Are Checked . . . 147

16.6 A Fully Constrained Table . . . 148
16.7 Common Constraint Errors and Their Meaning . . . 148

17 Functional Dependencies . . . 153
17.1 Definition . . . 153
17.2 Trivial and Non-Trivial Dependencies . . . 153
17.3 Full, Partial and Transitive Dependencies . . . 154
17.4 Armstrong's Axioms . . . 154
17.5 Attribute Closure . . . 155
17.6 Canonical Cover . . . 156
17.7 Equivalence of FD Sets . . . 156

18 Database Anomalies . . . 161
18.1 The Badly Designed Relation . . . 161
18.2 Insertion Anomaly . . . 162
18.3 Update Anomaly . . . 162
18.4 Deletion Anomaly . . . 162
18.5 Why All Three Have One Cause . . . 162
18.6 Redundancy and Its Costs . . . 163

19 Normalization . . . 167
19.1 First Normal Form (1NF) . . . 167
19.2 Second Normal Form (2NF) . . . 168
19.3 Third Normal Form (3NF) . . . 168
19.4 Boyce–Codd Normal Form (BCNF) . . . 169
19.5 Lossless-Join Decomposition . . . 170
19.6 Dependency Preservation . . . 171
19.7 3NF Synthesis Algorithm . . . 171
19.8 Higher Normal Forms . . . 172
19.9 Denormalization . . . 172

20 Relational Algebra . . . 178
20.1 The Operators . . . 178
20.2 Selection (σ) . . . 178
20.3 Projection (π) . . . 179
20.4 Rename (ρ) . . . 179
20.5 Set Operations . . . 180
20.6 Cartesian Product and Joins . . . 180
20.7 Division ($\div$) . . . 180
20.8 Extended Operators . . . 181
20.9 Worked Expressions . . . 182

21 Relational Calculus . . . 186
21.1 Tuple Relational Calculus (TRC) . . . 186
21.2 Domain Relational Calculus (DRC) . . . 187
21.3 Safety of Expressions . . . 187
21.4 Equivalence of Algebra and Calculus . . . 188
21.5 Translating Between the Three . . . 188

22 Storage and File Organization . . . 193
22.1 The Storage Hierarchy . . . 193
22.2 Magnetic Disk Structure . . . 194
22.3 Records and Blocks . . . 195
22.4 File Organization . . . 195

23 Indexing . . . 201

23.1 The Idea . . . 201
23.2 Dense and Sparse Indexes . . . 202
23.3 Multilevel Indexes . . . 202
23.4 B-Trees and B+ Trees . . . 203
23.5 Hash Indexes . . . 204
23.6 Indexes in Practice . . . 205

24 RAID . . . 210
24.1 The Two Techniques . . . 210
24.2 Mean Time to Failure . . . 210
24.3 RAID 0 — Striping . . . 211
24.4 RAID 1 — Mirroring . . . 211
24.5 RAID 2, 3 and 4 (Historical) . . . 211
24.6 RAID 5 — Distributed Parity . . . 212
24.7 RAID 6 — Double Parity . . . 213
24.8 RAID 10 (1 + 0) — Striped Mirrors . . . 213
24.9 Comparison and Selection . . . 213
24.10 Hardware versus Software RAID . . . 214
24.11 What RAID Does Not Do . . . 214

25 Query Processing . . . 219
25.1 The Stages . . . 219
25.2 Measuring Cost . . . 220
25.3 Algorithms for Selection . . . 220
25.4 Sorting: External Merge Sort . . . 221
25.5 Join Algorithms . . . 222
25.6 Other Operations . . . 223
25.7 Evaluation Strategies . . . 223

26 Query Optimization . . . 227
26.1 Why Optimization Matters . . . 227
26.2 Equivalence Rules . . . 227
26.3 Heuristic Optimization . . . 228
26.4 Cost-Based Optimization . . . 229
26.5 Why Estimates Go Wrong . . . 230
26.6 Reading an Execution Plan . . . 230
26.7 Helping the Optimizer . . . 230

27 Transactions . . . 235
27.1 The Canonical Example . . . 235
27.2 The ACID Properties . . . 235
27.3 Transaction States . . . 236
27.4 Schedules . . . 237
27.5 Why Allow Concurrency at All? . . . 237
27.6 Transactions in SQL . . . 237

28 Concurrency Control . . . 241
28.1 The Lost Update Problem . . . 241
28.2 The Dirty Read Problem (Uncommitted Dependency) . . . 241
28.3 The Non-Repeatable Read Problem . . . 242
28.4 The Phantom Read Problem . . . 242
28.5 The Incorrect Summary Problem . . . 242
28.6 SQL Isolation Levels . . . 243
28.7 Preventing the Lost Update in Practice . . . 244
28.8 Families of Concurrency Control . . . 244

29 Serializability . . . 249
29.1 Notation . . . 249
29.2 Conflicting Operations . . . 249
29.3 Conflict Serializability . . . 249
29.4 Testing with a Precedence Graph . . . 250
29.5 View Serializability . . . 251
29.6 Recoverability . . . 251

30 Locking Protocols . . . 256
30.1 Lock Modes . . . 256
30.2 Why Locks Alone Are Not Enough . . . 256
30.3 Two-Phase Locking (2PL) . . . 257
30.4 Strict and Rigorous 2PL . . . 257
30.5 Lock Granularity . . . 257
30.6 Timestamp-Ordering Protocol . . . 258
30.7 Optimistic (Validation-Based) Concurrency Control . . . 259
30.8 Multiversion Concurrency Control (MVCC) . . . 259

31 Deadlock . . . 264
31.1 Example . . . 264
31.2 The Four Necessary Conditions (Coffman Conditions) . . . 264
31.3 Deadlock Prevention . . . 265
31.4 Deadlock Detection . . . 266
31.5 Recovery from Deadlock . . . 266
31.6 Starvation versus Deadlock . . . 267
31.7 Practical Guidance for Developers . . . 267

32 Database Recovery . . . 271
32.1 Types of Failure . . . 271
32.2 Buffer Management Policies . . . 271
32.3 Log-Based Recovery . . . 272
32.4 Deferred and Immediate Update . . . 273
32.5 Checkpoints . . . 273
32.6 ARIES . . . 274
32.7 Shadow Paging . . . 275
32.8 Backup and Media Recovery . . . 275

33 Database Security . . . 280
33.1 Goals and Threats . . . 280
33.2 Authentication . . . 280
33.3 Authorization, Privileges and Access Control . . . 281
33.4 Encryption . . . 283
33.5 Auditing . . . 283
33.6 SQL Injection . . . 283

34 Distributed Databases . . . 289
34.1 Why Distribute? . . . 289
34.2 Types of Distributed System . . . 289
34.3 Distribution Transparency . . . 290
34.4 Data Fragmentation . . . 290
34.5 Replication . . . 291
34.6 Distributed Query Processing . . . 291
34.7 Distributed Transactions and Two-Phase Commit . . . 292
34.8 Advantages and Limitations . . . 293

35 NoSQL Databases . . . 297
35.1 Why NoSQL Emerged . . . 297
35.2 The Four NoSQL Data Models . . . 297
35.3 Relational versus NoSQL . . . 299
35.4 The CAP Theorem . . . 300
35.5 BASE and Eventual Consistency . . . 301

36 Data Warehousing . . . 305
36.1 Definitions . . . 305
36.2 OLTP versus OLAP . . . 305
36.3 Warehouse Architecture . . . 306
36.4 ETL — Extract, Transform, Load . . . 306
36.5 Data Marts . . . 307
36.6 Fact and Dimension Tables . . . 307
36.7 Star Schema . . . 308
36.8 Snowflake Schema . . . 309

37 OLAP . . . 313
37.1 The Data Cube . . . 313
37.2 Roll-Up . . . 314
37.3 Drill-Down . . . 314
37.4 Slice . . . 314
37.5 Dice . . . 315
37.6 Pivot . . . 315
37.7 SQL Extensions for OLAP . . . 316
37.8 OLAP Server Architectures . . . 316

38 Modern Database Systems . . . 320
38.1 Scalability: Vertical and Horizontal . . . 320
38.2 Database Replication . . . 320
38.3 Sharding . . . 321
38.4 High Availability . . . 321
38.5 Cloud Databases . . . 322
38.6 NewSQL and Distributed SQL . . . 322
38.7 In-Memory Databases . . . 323
38.8 Graph Databases . . . 323
38.9 Spatial Databases . . . 323
38.10 Temporal Databases . . . 324
38.11 Big-Data Databases . . . 324
38.12 Summary Map . . . 325

39 Database Performance and Tuning . . . 329
39.1 The Tuning Method . . . 329
39.2 Query Performance . . . 329
39.3 Index Tuning . . . 330
39.4 Query Plan Analysis . . . 331
39.5 Buffer Management . . . 331
39.6 Caching . . . 331
39.7 Connection Pooling . . . 332
39.8 Partitioning . . . 332
39.9 Replication and Sharding for Performance . . . 333
39.10 Performance Monitoring . . . 333

40 Complete Database Project: University Management System . . . 338
40.1 Requirements Analysis . . . 338

40.2 Entities, Attributes and Relationships . . . 339
40.3 ER Diagram . . . 340
40.4 Relational Schema . . . 340
40.5 Normalization Check . . . 341
40.6 SQL Table Creation (MySQL 8) . . . 342
40.7 Sample Data . . . 345
40.8 Supporting Function: Grade Points . . . 347
40.9 Stored Procedures . . . 347
40.10 Triggers . . . 348
40.11 Views . . . 349
40.12 Indexes . . . 350
40.13 Queries . . . 350
40.14 Transactions . . . 352
40.15 Security . . . 352
40.16 Deployment and Operations Summary . . . 353

A Database Design Case Studies . . . 357
A.1 Banking Management System . . . 357
A.2 Hospital Management System . . . 361
A.3 E-Commerce System . . . 364
A.4 Library Management System . . . 367
A.5 Comparing the Five Case Studies . . . 370

B Glossary . . . 372

C SQL Cheat Sheet . . . 377
C.1 DDL — Data Definition . . . 377
C.2 DML — Data Manipulation . . . 377
C.3 DQL — Querying . . . 377
C.4 Joins . . . 378
C.5 Subqueries and Set Operations . . . 378
C.6 Functions . . . 378
C.7 Constraints . . . 378
C.8 Views, Triggers, Procedures, Functions . . . 379
C.9 Indexes . . . 379
C.10 DCL and TCL . . . 379

D Important Formulas and Concepts . . . 380
D.1 Functional Dependencies . . . 380
D.2 Normalization Rules . . . 380
D.3 Relational Algebra . . . 380
D.4 Storage, Indexing and RAID . . . 381
D.5 Query Cost . . . 381
D.6 Transactions and Serializability . . . 381

E Viva Questions . . . 382

F Interview Questions . . . 386
F.1 Basic . . . 386
F.2 Intermediate . . . 386
F.3 Advanced . . . 387
F.4 SQL . . . 388
F.5 Database Design . . . 388
F.6 Transactions and Concurrency . . . 389
F.7 Performance . . . 389

F.8 Modern Databases . . . 390

G Final Examination Question Bank . . . 392

G.1 Multiple Choice Questions (50) . . . 392
G.2 Short Questions (50) . . . 393
G.3 Descriptive Questions (30) . . . 395
G.4 SQL Problems (20) with Solutions . . . 395
G.5 ER Diagram Problems (10) . . . 398
G.6 Normalization Problems (10) with Answers . . . 398
G.7 Relational Algebra Problems (10) with Answers . . . 399
G.8 Transaction and Serializability Problems (10) with Answers . . . 399
G.9 Query Optimization Problems (10) with Answers . . . 399
G.10 RAID Problems (10) with Answers . . . 400

H Practical Lab Exercises . . . 401

H.1 Setup: Creating and Populating university_db . . . 401
H.2 Basic Queries (1–20) . . . 404
H.3 Intermediate Queries (21–40) . . . 405
H.4 Advanced Queries (41–60) . . . 407
H.5 Lab Assessment Checklist . . . 410

References . . . 412

Preface

Almost every piece of software a student uses in a single day is, underneath, a database application. A university portal that shows a semester result, a mobile banking app that transfers money in two seconds, an e-commerce site that remembers a cart from last week, a hospital system that retrieves a patient's prescription history — all of them exist because someone designed a database carefully and because a Database Management System is quietly enforcing correctness behind the scenes.

This book was written for the undergraduate student who has to learn all of that in one semester, and who then has to defend it in a viva, reproduce it in a written examination, and use it in a job interview. It therefore tries to do three things at once: explain the **theory** with the precision an examiner expects, show the **implementation** with SQL that actually runs, and connect both to the **real systems** the student already uses.

The approach in every section is deliberately repetitive in structure, because that structure is what makes difficult material learnable:

```
Basic concept  ->  Intuition  ->  Formal definition  ->  Figure
       ->  Example  ->  SQL / Algorithm  ->  Real-life application
       ->  Advantages and limitations  ->  Solved problems  ->  Exercises
```

Nothing advanced is introduced before its prerequisites. Normalization is not discussed before functional dependencies; functional dependencies are not discussed before keys; keys are not discussed before the relational model. A reader who starts at Chapter 1 and moves forward will never meet an undefined term.

A second commitment of this book is **consistency**. From Chapter 11 onward, all SQL examples use a single sample database, `university_db`, with eight tables. The same rows appear in the query chapters, the normalization chapters, the indexing chapters, the transaction chapters and the final project. A student can therefore run every query in this book against one database created once.

The SQL dialect used throughout is **MySQL 8**, which is the most common teaching dialect in undergraduate laboratories. Where PostgreSQL, Oracle or SQL Server differ in a way that matters, the difference is labelled explicitly. Where a feature belongs to the SQL standard rather than to a vendor, that is stated too, because examiners frequently ask students to distinguish between database *theory*, DBMS *implementation*, and vendor-specific *features*.

How to Use This Book

For a semester course

Read the chapters in order. Chapters 1 to 9 build the conceptual and design foundation, Chapters 10 to 16 build SQL skill, Chapters 17 to 21 cover design theory and formal query languages, Chapters 22 to 26 cover physical storage and query processing, Chapters 27 to 32 cover transactions and recovery, and Chapters 33 to 40 cover security, distribution, modern systems and a complete project.

For laboratory work

Create the sample database once from the script in Chapter 11, keep it, and run every SQL example against it. The practical lab section at the end of the book contains sixty graded queries that use the same schema.

For examination preparation

Each chapter ends with a fixed set of study aids in the same order every time:

Section	What it gives you
Chapter Summary	Everything examinable in the chapter, compressed
Key Terms	The vocabulary an examiner expects you to define
Important Formulas / Notation	Symbols and formulas used in the chapter
Common Mistakes	The errors students actually make in scripts
Solved Problems	Full step-by-step model answers
Review Questions	Ten conceptual questions
Multiple Choice Questions	Ten MCQs with an answer key
Short Questions	Ten short-answer questions
Long / Descriptive Questions	Five essay-type questions
Practical Exercises	Five hands-on tasks

For viva and interview preparation

Use the chapter summaries and key terms first, then the appendices: a SQL cheat sheet, a glossary, one hundred viva questions with short answers, and one hundred interview questions organised from basic to advanced.

A note on the figures

The figures in this book are drawn in a fixed-width text style so they can be reproduced exactly on a whiteboard or in an examination script. This is intentional. An examiner does not expect a decorated diagram; an examiner expects a diagram whose boxes, arrows and labels are correct. Every figure is numbered as **Figure C.N**, where C is the chapter number, and is followed by an explanation of what it shows.

Learning Outcomes

After working through this book, a student should be able to:

1. Explain what data, information, metadata, a database, a DBMS and a database system are, and distinguish clearly between them.
2. Describe the drawbacks of traditional file processing and justify the use of a DBMS.
3. Explain one-tier, two-tier, three-tier and three-schema architectures, and explain logical and physical data independence.
4. Compare the hierarchical, network, relational, object-oriented, object-relational and semi-structured data models.
5. Define relation, tuple, attribute, domain, schema and instance precisely, and apply relational integrity rules.
6. Identify super keys, candidate keys, primary keys, alternate keys, composite keys, surrogate keys and foreign keys in a given schema.
7. Design an ER or EER diagram for a real-world requirement, including weak entities, specialization, generalization and aggregation.
8. Convert an ER/EER diagram into a correct relational schema.
9. Write DDL, DML, DQL, DCL and TCL statements, including joins, subqueries, set operations, views, triggers, stored procedures and functions.
10. Determine functional dependencies, compute attribute closures, find candidate keys, and normalize a relation up to BCNF with lossless, dependency-preserving decomposition.
11. Express queries in relational algebra and relational calculus, and translate between them and SQL.
12. Explain storage hierarchy, file organization, indexing structures including B+ trees and hashing, and RAID levels with their trade-offs.
13. Describe query processing and optimization, and estimate the relative cost of alternative execution plans.
14. Explain ACID properties, analyse schedules for conflict and view serializability, apply locking protocols, and detect and handle deadlock.
15. Explain log-based recovery, write-ahead logging, checkpointing and shadow paging.
16. Apply database security concepts including authentication, authorization, privileges, roles, encryption, auditing and safe parameterized queries.
17. Explain distributed databases, fragmentation, replication, NoSQL data models, the CAP theorem, data warehousing and OLAP.
18. Design and implement a complete database application from requirements to working SQL.

CHAPTER 1

Introduction to Database Systems

Every database course begins with a deceptively simple question: *what exactly is being stored?* Students usually answer "data", and they are right, but the word hides four different ideas — data, information, knowledge and metadata — that a database professional must be able to separate. This chapter builds the vocabulary of the subject from that point up to the definition of a complete database system, and it introduces the people who build and operate such systems.

1.1 What is Data?

Intuition first

Imagine a piece of paper on which someone has written:

```
101   Rahim   CSE   3.75
```

Is this useful? Only partly. We can read four items, but we do not know what they mean. They could be a student record, a product record, or a cricket score. These four items are **data**: they are raw, unprocessed facts that have not yet been given meaning by context.

Now suppose the same four items appear under column headings:

```
StudentID | Name  | Department | CGPA
----------|-------|------------|-----
101       | Rahim | CSE        | 3.75
```

Now the facts have meaning. We can say: *"Student 101, whose name is Rahim, studies in the CSE department and has a CGPA of 3.75."* This is **information** — data that has been processed, organized or given context so that it answers a question.

If we look at a thousand such rows and observe that *students who take the Database Systems course in their fifth semester have an average CGPA that is 0.2 higher than those who take it in the seventh*, we have produced **knowledge** — a pattern or rule derived from information that supports decision-making.

Finally, the row of headings itself — the fact that `StudentID` is an integer, that it is the primary key, that `CGPA` is a decimal number between 0.00 and 4.00 — is **metadata**: data that describes other data.

Formal definitions

Data is a collection of raw, unorganized facts, figures, symbols or measurements that have no inherent meaning until they are interpreted.

Information is data that has been processed, structured or contextualized so that it is meaningful and useful to a recipient.

Knowledge is the understanding, pattern or rule obtained by analysing and interpreting information, generally used to make decisions or predictions.

Metadata is data about data: it describes the structure, type, constraints, origin, meaning and permitted use of the actual data.

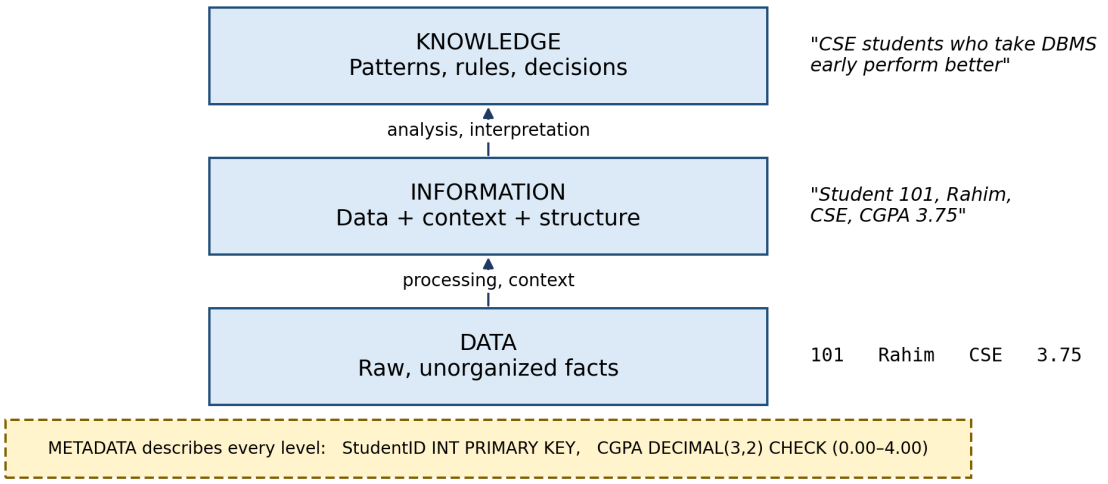


Figure 1.1: The data–information–knowledge hierarchy

Each level in Figure 1.1 adds value to the level below it. A DBMS stores data, presents information, and — crucially — stores its own metadata inside a **data dictionary** or **system catalog** so that it knows how to interpret what it has stored.

Comparison table

Aspect	Data	Information	Knowledge	Metadata
Definition	Raw facts	Processed, contextual data	Derived patterns and rules	Data describing data
Meaning	None by itself	Meaningful	Actionable	Descriptive
Example	`3.75`	CGPA of Rahim is 3.75	Students with CGPA > 3.5 rarely fail	`CGPA DECIMAL(3,2)`
Produced by	Measurement, entry	Processing, organizing	Analysis, experience	Schema definition
Stored in	Tables	Reports, query results	Models, business rules	System catalog

Real-life example: a university record

Consider what Noakhali Science and Technology University must store about one student:

Item	Raw data	Becomes information when
Student ID	`ASH1901001`	linked to a person and a session
Name	`Rahim Uddin`	attached to the ID
Department	`CSE`	mapped to a department offering courses
Semester	`5`	interpreted against the academic calendar
Course	`CSE 3201`	resolved to “Database Management Systems, 3 credits”
Grade	`A`	converted to grade point 3.75 and combined into a CGPA

Individually these are meaningless tokens. Combined by the database, they answer questions such as *“What is Rahim’s CGPA after the fifth semester?”*, *“How many CSE students scored A in CSE 3201?”* and *“Which students are eligible for the dean’s award?”*. The transformation from the first column to the second is exactly what a database system exists to perform, reliably and repeatedly, for tens of thousands of rows.

> Examination tip: the distinction "data has no meaning; information has context" earns marks. Adding the sentence "metadata is stored in the DBMS data dictionary" usually earns one more.

1.2 What is a Database?

Definition

A **database** is a shared, integrated, persistent and self-describing collection of logically related data, organized to model some part of the real world, and stored in a way that permits efficient retrieval and modification by multiple users and applications.

Every phrase in that definition is doing work:

- **Logically related** — the data is about one universe of discourse (a university, a bank), not an arbitrary pile of files.
- **Shared** — many users and applications use the same data instead of each keeping a private copy.
- **Integrated** — related facts are stored once and connected, so redundancy is controlled.
- **Persistent** — the data outlives the program that created it; it survives power loss and restarts.
- **Self-describing** — the database stores its own description (metadata) alongside the data.

Characteristics of a database

1. **Represents a mini-world.** A university database models students, courses, teachers and results — nothing else.
2. **Controlled redundancy.** A department name is stored once in `Department`, not repeated in every student row.
3. **Data independence.** Applications do not break when physical storage changes.
4. **Multi-user access.** Hundreds of users may read and write concurrently without corrupting each other's work.
5. **Integrity.** Rules such as "CGPA must lie between 0.00 and 4.00" are enforced by the system, not by each programmer.
6. **Security.** Different users see different parts of the data.
7. **Recoverability.** After a crash, the database can be restored to a consistent state.

Components of a database

Component	Contents	Example
User data	The actual stored facts	Student rows, course rows
Metadata	Description of structures and constraints	Table definitions, data types, keys
Indexes	Access structures for fast lookup	B+ tree on `student.name`
Transaction log	Record of every change	Redo/undo log entries
Stored procedures and views	Saved logic and derived tables	`vw_cse_students`, `sp_enrol_student`

Databases in the real world

Sector	What is stored	Typical query the database must answer
University	Students, courses, teachers, results, fees	"Print the transcript of student ASH1901001"
Bank	Customers, accounts, transactions, loans	"Transfer 1,000 taka from A to B, atomically"
Hospital	Patients, doctors, prescriptions, tests, beds	"Show all medicines given to patient P-204 this week"

Sector	What is stored	Typical query the database must answer
E-commerce	Products, stock, carts, orders, payments	"Is this item still in stock while the customer pays?"
Airline	Flights, seats, passengers, fares	"Reserve seat 14A only if nobody else has just taken it"
Government	Citizens, national IDs, land records, taxes	"Verify whether NID 1990… matches this fingerprint record"
Social media	Users, posts, comments, likes, relationships	"Show posts from people this user follows, newest first"
Telecom	Subscribers, call records, packages, billing	"Bill 4.2 million call detail records for this month"

Two observations follow from this table. First, the *shape* of the data differs a great deal — a bank's data is small but demands absolute correctness, while a telecom operator's data is enormous but tolerates approximate analytics. Second, the *guarantees* differ: an airline seat reservation must never be sold twice, whereas a like-count on a social network may lag by a second without harm. These differences explain why several kinds of database systems exist, a theme returned to in Chapters 35 and 38.

1.3 What is a DBMS?

Definition

A **Database Management System (DBMS)** is a software system that enables users to define, create, maintain, query and control access to a database, while hiding the details of physical storage and guaranteeing consistency, security and recoverability.

The database is the *data*; the DBMS is the *software*. MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, SQLite, MongoDB and Redis are DBMSs. The collection of tables holding a university's records is a database.

Purpose of a DBMS

Without a DBMS, every application programmer would have to write code for file layout, searching, locking, crash recovery, access control and backup. A DBMS centralizes all of that work once, correctly, behind a declarative interface. The programmer writes *what* is wanted:

```
SELECT name FROM student WHERE dept_id = 'CSE' AND cgpa > 3.50;
```

and the DBMS decides *how* to obtain it — whether to scan the table, use an index, or use both.

Major functions of a DBMS

1. **Data definition.** Create, alter and drop tables, views, indexes and constraints (the DDL).
2. **Data manipulation.** Insert, update, delete and retrieve rows (the DML and DQL).
3. **Storage management.** Decide how rows are placed in pages, blocks and files.
4. **Query processing and optimization.** Parse a query, generate alternative plans, and choose a cheap one.
5. **Transaction management.** Guarantee atomicity, consistency, isolation and durability.
6. **Concurrency control.** Allow many users at once without letting them corrupt each other's updates.
7. **Recovery management.** Restore a consistent state after a crash using logs and checkpoints.
8. **Security and authorization.** Authenticate users, grant and revoke privileges, audit activity.
9. **Integrity enforcement.** Reject data that violates declared rules.
10. **Backup, restore and utilities.** Import, export, reorganize, monitor.

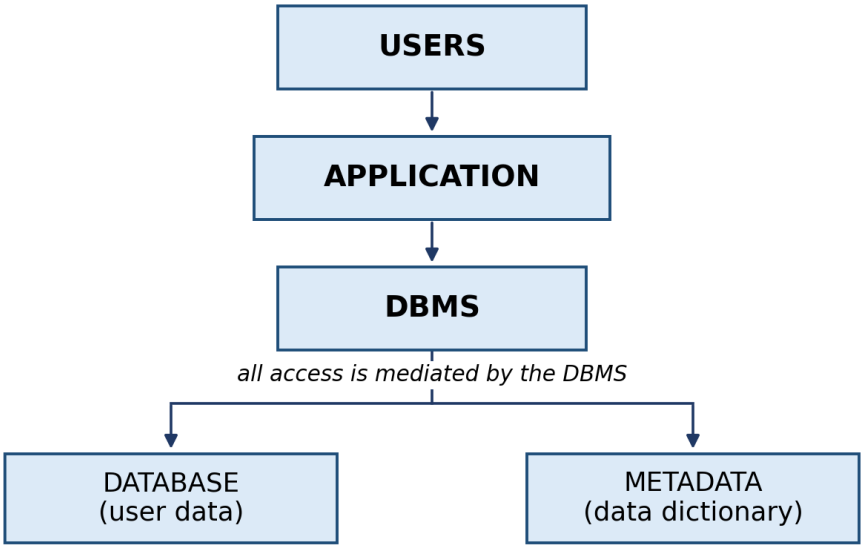


Figure 1.2: Position of the DBMS between users and stored data

The figure repays careful reading, because the meaning of each arrow is examinable.

- **Users** are people or programs: a student checking a result, a clerk entering marks, a batch job computing CGPA.
- **Application** is the software the user actually touches — a web portal, a mobile app, a reporting tool. It translates a click into a database request.
- **DBMS** receives that request in SQL, checks the user's privileges, verifies the request against the metadata, chooses an execution strategy, performs the reads and writes, enforces integrity and isolation, and returns the result.
- **Database** holds the actual user data on disk.
- **Metadata** holds the description of that data. The DBMS consults it on every single request; without metadata the bytes on disk are unreadable.

No arrow goes directly from the application to the database. That is the central discipline of the whole subject: **all access is mediated by the DBMS**, which is why security, integrity and concurrency can be guaranteed at all.

Components of a DBMS

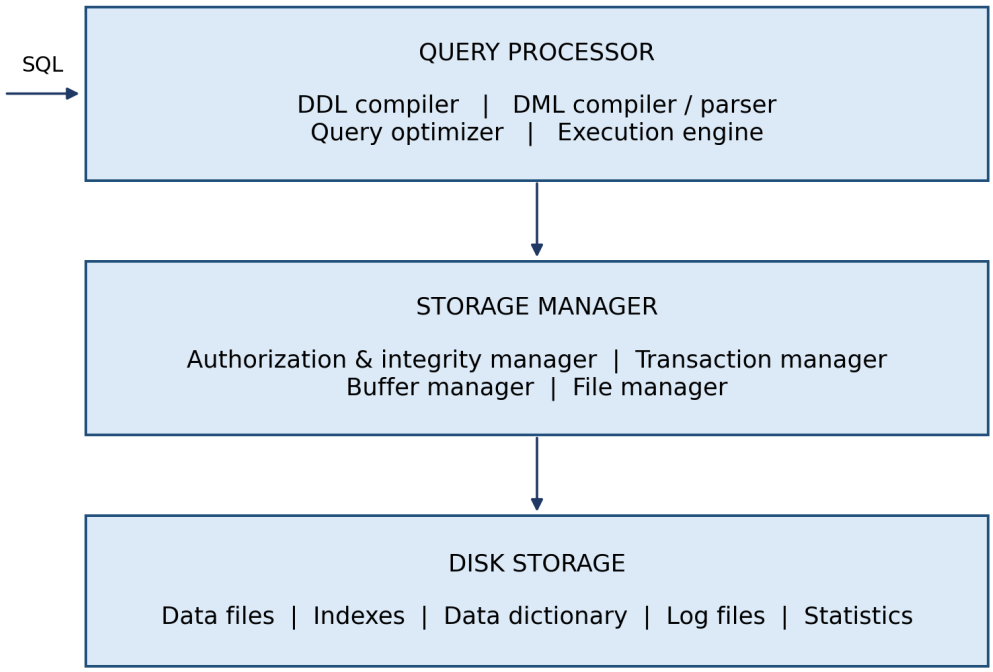


Figure 1.3: Internal components of a DBMS

Component	Responsibility
DDL compiler	Translates schema definitions into entries in the data dictionary
DML compiler / parser	Checks syntax and semantics of queries, produces an internal form
Query optimizer	Chooses the cheapest execution plan among many equivalent ones
Execution engine	Runs the chosen plan, calling the storage manager for pages
Authorization manager	Verifies that the user is permitted to perform the operation
Integrity manager	Enforces constraints such as `CHECK` and referential integrity
Transaction manager	Preserves ACID properties, coordinates commit and rollback
Buffer manager	Moves pages between disk and main memory, decides what to evict
File manager	Maps logical pages to physical disk blocks
Data dictionary	Stores metadata: tables, columns, types, keys, users, privileges
Log file	Stores a record of every change for recovery

1.4 Database System vs DBMS vs Database

Students lose marks constantly by using these three words interchangeably. The distinction is mechanical once seen in a table.

Term	Meaning	Nature	Example
Data	Raw facts	Values	Student marks: `A`, `3.75`
Database	Organized, integrated collection of related data	Stored content	The university database
DBMS	Software that manages the database	Software	MySQL 8, PostgreSQL 16, Oracle 19c

Term	Meaning	Nature	Example
Database System	Database + DBMS + hardware + users + applications + procedures	Complete environment	The university information system

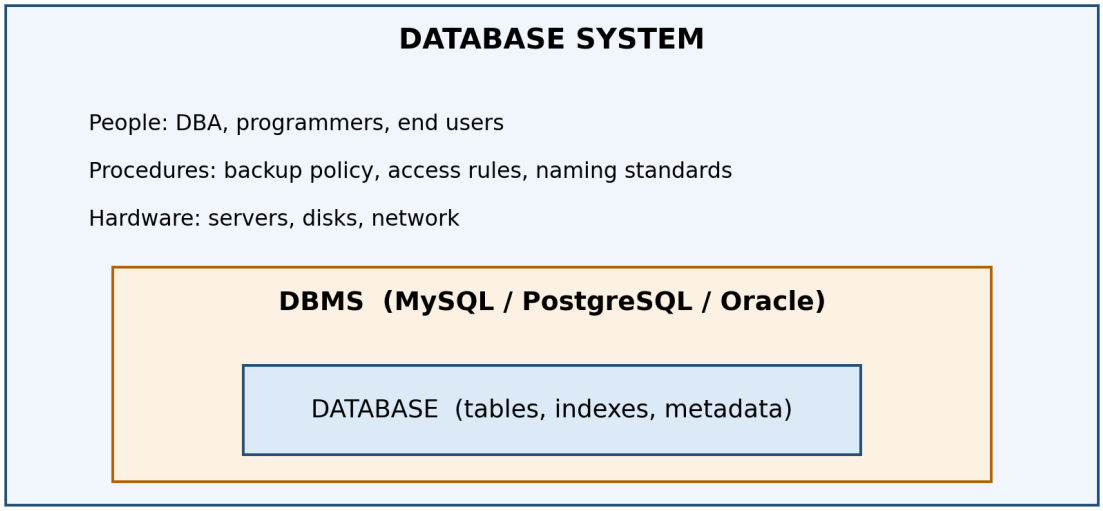


Figure 1.4: A database system contains the DBMS, which contains access to the database

A useful one-line answer for a viva: *"The database is what is stored, the DBMS is the software that stores it, and the database system is everything — hardware, software, data, procedures and people — taken together."*

1.5 Components of a Database System

A complete database system has five components. Examiners frequently ask for all five with an example of each.

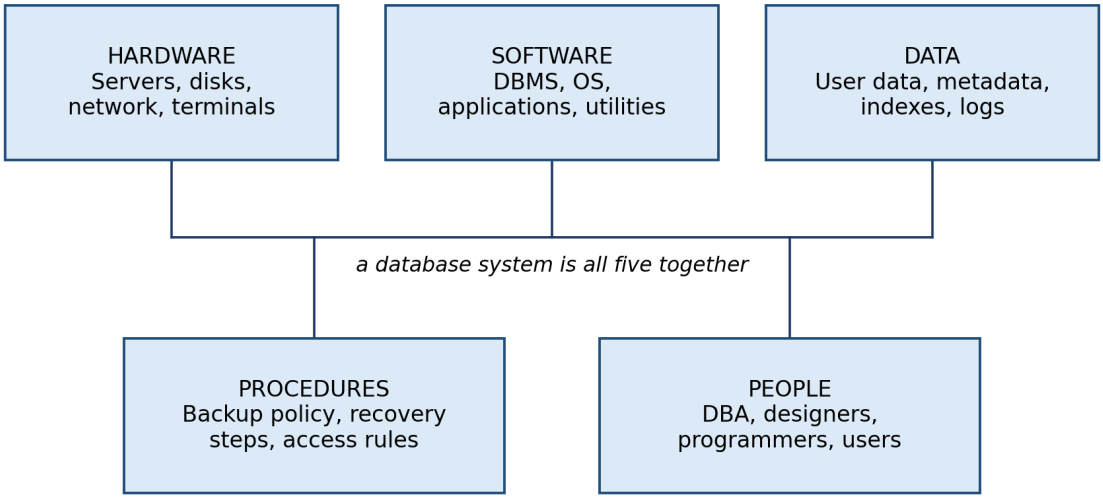


Figure 1.5: The five components of a database system

1. **Hardware** — the physical devices: database servers, storage (HDD, SSD, RAID arrays), memory, network links and client terminals. Hardware determines the ceiling on performance: a query that needs a million random disk reads will be slow regardless of how well it is written.
2. **Software** — the DBMS itself, the operating system beneath it, the network software, the application programs above it, and utilities for backup, monitoring and migration.
3. **Data** — user data, metadata, indexes and logs. This is the only component that is irreplaceable: hardware can be bought again and software reinstalled, but lost data is lost permanently unless it was backed up.
4. **Procedures** — the documented rules for using the system: how backups are taken and verified, how a new user is granted access, how a failure is handled, what naming conventions apply. Procedures are the component students forget.
5. **People** — database administrators, designers, application programmers, system analysts, end users and data analysts.

1.6 Database Users

User	Role	Typical activity	Interacts through
Database Administrator (DBA)	Owns and operates the database	Grants privileges, tunes, backs up, recovers	Admin tools, SQL, shell
Database designer	Decides the structure	Builds ER models, defines tables and constraints	Modelling tools, DDL
Application programmer	Writes software that uses the database	Embeds SQL in Java/Python/PHP	Programming language + driver
Sophisticated / power user	Writes complex ad-hoc queries	Analytical SQL, reporting	SQL client
Naive / end user	Uses ready-made application screens	Checks a result, books a ticket	Web or mobile application
System analyst	Gathers requirements	Turns business needs into specifications	Documents, models

User	Role	Typical activity	Interacts through
Data analyst / scientist	Extracts insight	Aggregation, trend analysis, dashboards	SQL, BI tools

A single example makes the separation concrete. When a student checks a result on a university portal, the **end user** is the student; the **application programmer** wrote the portal; the **database designer** decided that `Result` has a composite key of `(student_id, course_id, semester)`; the **DBA** made sure the portal's account may read `Result` but not update it; and the **system analyst** originally wrote down the rule that a student may retake a course only twice.

1.7 Database Administrator

The DBA is the custodian of the database. Responsibilities fall into seven groups.

1. **Security and user management.** Creating accounts, defining roles, granting the minimum privileges each role needs, revoking access when a person leaves, and reviewing who can read sensitive columns.
2. **Backup.** Scheduling full and incremental backups, storing copies off-site, and — the part that is often neglected — periodically *restoring* a backup on a test server to prove that it works.
3. **Recovery.** Restoring the database after a media failure or a logical error, applying log files to roll forward to a chosen point in time.
4. **Performance monitoring and tuning.** Watching slow-query logs, examining execution plans, adding or removing indexes, adjusting buffer pool size, and detecting locking contention.
5. **Storage management.** Sizing tablespaces and files, reclaiming space, planning growth, configuring RAID.
6. **Integrity and standards.** Ensuring constraints are declared in the database rather than left to application code, and enforcing naming conventions.
7. **Upgrades and migration.** Testing and applying DBMS version upgrades, moving schemas between servers.

Real-life DBA scenario

At 09:10 on a results-publication morning, the university portal becomes unusable. Students see a spinning loader; the help desk receives two hundred calls in ten minutes.

The DBA's actual sequence of work is instructive:

1. **Check availability first.** Is the server up, is the disk full, is the network reachable? The DBA finds the server up and disk at 62 %.
2. **Check active sessions.** The DBA lists running queries and sees 400 sessions all executing the same transcript query, each taking 9 seconds.
3. **Diagnose.** The execution plan shows a full table scan of `enrollment`, which now holds 1.2 million rows, because the query filters on `student_id` and no index exists on that column — the column was added last semester and nobody indexed it.
4. **Act.** The DBA creates the index during a short maintenance pause:

```
CREATE INDEX idx_enrollment_student ON enrollment(student_id);
```

5. **Verify.** Re-running the query now takes 12 milliseconds instead of 9 seconds; sessions drain within a minute.
6. **Prevent recurrence.** The DBA adds a rule to the review checklist: every foreign key column must be indexed before release, and the slow-query log threshold is lowered so that the next such query

is noticed in testing rather than in production.

Notice that only step 4 is technical trivia. The rest is method — and method is what distinguishes a DBA from someone who merely knows SQL.

1.8 Database Applications

Domain	Representative system	Core tables	Dominant requirement
Banking	Core banking, ATM, mobile financial services	`customer`, `account`, `transaction`, `loan`	Absolute correctness; no lost or duplicated money
Education	Student information system, LMS	`student`, `course`, `enrollment`, `result`	Accuracy of results; peak load on publication day
Healthcare	Hospital management, lab systems	`patient`, `doctor`, `appointment`, `prescription`	Privacy, auditability, availability
E-commerce	Storefront, inventory, payments	`product`, `stock`, `cart`, `order`, `payment`	Concurrency on limited stock; fast reads
Transportation	Airline, railway, ride-hailing	`flight`, `seat`, `booking`, `trip`	No double booking; real-time availability
Social media	Feeds, messaging, graphs	`user`, `post`, `follows`, `comment`	Massive scale; relaxed consistency acceptable
Government	NID, land records, taxation	`citizen`, `land_parcel`, `tax_return`	Long-term retention, legal integrity
Telecommunication	Billing, call records, prepaid balance	`subscriber`, `cdr`, `package`, `invoice`	Very high insert rate; accurate billing

Read the last column down the page. The *same* technology — a relational DBMS — serves all of these domains, but the *emphasis* differs. Banking pushes the transaction subsystem (Chapters 27 to 32). E-commerce pushes concurrency control (Chapters 28 to 31). Social media pushes scalability, which is why such systems often adopt NoSQL stores (Chapter 35). Telecom pushes both insert throughput and analytical reporting, which leads to data warehousing (Chapter 36).

1.9 Advantages of DBMS

1. **Controlled redundancy.** In a file-based system the same address might appear in four files. A database stores it once and refers to it, saving space and — far more importantly — removing the possibility that the four copies disagree.
2. **Data consistency.** Because a fact is stored once, updating it updates it everywhere.
3. **Data sharing.** Multiple applications and users work on one integrated set of data. The admission module and the examination module see the same student.
4. **Data integrity.** Constraints (`NOT NULL`, `CHECK`, `FOREIGN KEY`) are declared once in the schema and enforced for every application, including ad-hoc SQL typed by a human.
5. **Security.** Privileges are granted per user, per table, and even per column; sensitive data can be encrypted and access audited.
6. **Backup and recovery.** The DBMS provides automatic logging and recovery; a programmer does not implement crash recovery by hand.
7. **Concurrent access.** Thousands of users may read and write simultaneously; the DBMS serializes conflicting operations so the result is as if they had run one after another.
8. **Atomicity of updates.** A transfer that debits one account and credits another either happens completely or not at all.

9. **Data independence.** Storage structures and indexes may change without rewriting applications.
10. **Reduced application development time.** Searching, sorting, joining, locking and recovering are provided; the programmer writes business logic.
11. **Enforcement of standards.** Naming, formats and documentation can be standardized centrally.

Illustration: consistency

Suppose a student's phone number is stored in an admission file, a library file, a hostel file and an examination file. The student changes the number. In a file system, four programs must update four files; if one program is not run, the library will keep calling the old number forever. In a database, `student.phone` is one column in one row. One `UPDATE` corrects the fact for the entire university:

```
UPDATE student
SET    phone = '+8801712345678'
WHERE  student_id = 'ASH1901001';
```

1.10 Limitations of DBMS

1. **Cost.** Commercial licences (Oracle, SQL Server) are expensive; even open-source systems require capable servers and skilled staff.
2. **Complexity.** A DBMS is one of the most complex pieces of software in existence. Using it well requires real study — the length of this book is itself evidence.
3. **Hardware requirements.** Memory for buffer pools, fast storage for logs and redundant disks for safety all cost money.
4. **Administration overhead.** Backup, tuning, patching and user management are continuing work, not one-time setup.
5. **Performance overhead for simple needs.** For a single-user program that reads a small configuration list, a plain text or JSON file is faster and simpler. A DBMS pays a price for generality — parsing, optimization, locking and logging are not free.
6. **Single point of failure.** Centralizing all data means that if the database server is down, every dependent application is down. This is mitigated by replication and high availability (Chapter 38) but never fully eliminated.
7. **Security risk concentration.** One compromised administrator account can expose everything, precisely because everything is in one place.
8. **Vendor lock-in.** Stored procedures and proprietary extensions written for one DBMS are expensive to port to another.

> The honest summary: a DBMS is the right choice whenever data is shared, must stay correct, and must outlive any single program. It is the wrong choice for a small, private, throwaway dataset used by one program on one machine.

Chapter Summary

- **Data** are raw facts; **information** is data given context; **knowledge** is patterns derived from information; **metadata** is data describing data and lives in the DBMS data dictionary.
- A **database** is a shared, integrated, persistent, self-describing collection of logically related data that models a mini-world.
- A **DBMS** is the software that defines, stores, retrieves, protects and recovers that data, offering a declarative interface so users state *what* they want, not *how* to get it.
- A **database system** is the whole environment: hardware, software, data, procedures and people.
- Major DBMS functions: data definition, manipulation, storage management, query optimization,

transaction management, concurrency control, recovery, security, integrity and utilities.

- Internally a DBMS consists of a query processor (DDL compiler, parser, optimizer, execution engine) and a storage manager (authorization, integrity, transaction, buffer and file managers) sitting above data files, indexes, the data dictionary and the log.
- Users include the DBA, designers, application programmers, sophisticated users, naive end users, system analysts and data analysts.
- The DBA is responsible for security, backup, recovery, performance, storage, integrity and upgrades.
- Advantages of a DBMS centre on controlled redundancy, consistency, sharing, integrity, security, recovery, concurrency and data independence.
- Limitations centre on cost, complexity, hardware demands, administration effort, overhead for trivial tasks, centralization risk and vendor lock-in.

Key Terms

Data · Information · Knowledge · Metadata · Data dictionary · System catalog · Database · Mini-world · DBMS · Database system · Persistence · Self-describing · Controlled redundancy · Data independence · Query processor · Query optimizer · Storage manager · Buffer manager · Transaction manager · Authorization manager · DBA · Naive user · Sophisticated user · Data sharing · Integrity constraint · Recovery · Concurrent access · Single point of failure · Vendor lock-in

Common Mistakes

1. **Writing "database" when "DBMS" is meant.** "MySQL is a database" is wrong; MySQL is a DBMS that manages databases.
2. **Claiming a DBMS removes redundancy entirely.** It *controls* redundancy; some redundancy (indexes, replicas, denormalized warehouses) is deliberate and useful.
3. **Confusing information with knowledge.** A report showing CGPA is information; the rule inferred from many reports is knowledge.
4. **Forgetting procedures and people** when listing the components of a database system. Five components are expected, not three.
5. **Listing only advantages.** Examiners look for balanced answers; a DBMS has genuine costs.
6. **Saying metadata is stored outside the database.** In a modern DBMS, metadata is stored *inside* it, in system tables.
7. **Assuming applications can bypass the DBMS.** In a properly designed system every access is mediated by the DBMS.

Solved Problems

Problem 1.1 — Classify each item as data, information, knowledge or metadata: (a) `88`, (b) "Rahim scored 88 in CSE 3201", (c) "Students who attend more than 80 % of classes score on average 12 marks higher", (d) `marks TINYINT UNSIGNED CHECK (marks BETWEEN 0 AND 100)`.

Solution. (a) Data — a bare number with no context. (b) Information — the number has been attached to a person and a course. (c) Knowledge — a pattern derived by analysing many records, usable for decisions such as attendance policy. (d) Metadata — it describes the type and permitted range of the `marks` column and is stored in the data dictionary.

Problem 1.2 — A college keeps student addresses in the admission office file, the library file and the hostel file. A student moves house. List every problem that can arise, and state which DBMS advantage solves each.

Solution.

Problem	Explanation	DBMS advantage that solves it
Wasted space	The same address occupies three copies	Controlled redundancy
Inconsistency	If only two files are updated, the third is wrong	Data consistency
Extra effort	Three separate update programs must be run	Data sharing / single update point
Audit difficulty	Nobody can say which copy is authoritative	Integrity and integration

In a database the address is one column in `student`; one `UPDATE` corrects it for all offices at once.

Problem 1.3 — Explain, with reference to Figure 1.2, why an application should not be allowed to open the database files directly.

Solution. Direct access bypasses the DBMS, and with it: (i) authorization checks, so any user could read any data; (ii) integrity enforcement, so invalid rows could be written; (iii) concurrency control, so two programs writing the same page would corrupt it; (iv) logging, so a crash would leave the database unrecoverable; and (v) the metadata layer, so the application would have to hard-code physical layout, destroying data independence. Every guarantee the DBMS offers depends on its being the only path to the data.

Problem 1.4 — A startup stores a list of 40 country names used to fill a dropdown in a desktop tool used by one person offline. Should it use a DBMS? Justify.

Solution. No. The data is tiny, private, read-only and used by a single program with no concurrency, no integrity relationships and no recovery requirement. A plain file (CSV or JSON) is simpler, faster to load and easier to deploy. A DBMS would add installation, administration and licensing cost with no corresponding benefit. The decision would change if the list had to be shared across applications, edited by several people, audited, or linked to other data.

Problem 1.5 — State the five components of a database system and give one university example of each.

Solution. **Hardware** — the university's database server and its RAID disk array. **Software** — MySQL 8 plus the Linux operating system and the results portal. **Data** — the `student`, `course`, `enrollment` and `result` tables together with their indexes and the data dictionary. **Procedures** — the documented rule that a full backup runs nightly at 01:00 and is restore-tested monthly. **People** — the DBA, the portal developers, the examination controller's clerks and the students.

Problem 1.6 — A DBA is told "the database is slow". List, in order, the first five things the DBA should check, and justify the order.

Solution. (1) Availability and resources — server up, disk not full, memory not exhausted; a resource problem makes all other analysis meaningless. (2) Current sessions and their queries — is the slowness caused by one heavy query or by many? (3) Execution plans of the slow queries — is a scan being used where an index should be? (4) Locking and blocking — are sessions waiting on each other rather than doing work? (5) Recent changes — new code, new data volume, a dropped index. The order goes from cheap and global to expensive and specific, so that the largest cause is found with the least work.

Problem 1.7 — Distinguish between "data independence" and "data integrity" with one example each.

Solution. **Data independence** is the ability to change one level of the system without changing the level above it: adding an index on `student.name` speeds up queries but no application code changes. **Data integrity** is the correctness of the stored data with respect to declared rules: a `CHECK (cgpa BETWEEN 0 AND 4)` constraint prevents a CGPA of 7.5 from ever being stored. The first is about *insulation from change*; the second is about *correctness of content*.

Review Questions

1. Define data, information, knowledge and metadata, and give one example of each from a hospital system.
2. What does it mean to say that a database is "self-describing"?
3. List and explain the major functions of a DBMS.
4. Draw and explain the diagram showing users, applications, the DBMS, the database and metadata.
5. Distinguish between a database, a DBMS and a database system.
6. Describe the internal components of a DBMS and the responsibility of each.
7. Identify the six categories of database user and state what each one does.
8. List seven responsibilities of a database administrator with an example of each.
9. Explain five advantages of a DBMS over keeping data in ordinary files.
10. Under what circumstances is a DBMS *not* the right choice? Justify your answer.

Multiple Choice Questions

1. Data that describes the structure and constraints of other data is called (a) information (b) knowledge (c) metadata (d) a schema instance.
2. Which of these is a DBMS, not a database? (a) The university records (b) PostgreSQL (c) The `student` table (d) A transcript.
3. The component of a DBMS that chooses among equivalent execution plans is the (a) parser (b) buffer manager (c) query optimizer (d) authorization manager.
4. "The database survives program termination and power loss" describes (a) integrity (b) persistence (c) isolation (d) independence.
5. Which is **not** one of the five components of a database system? (a) Hardware (b) Procedures (c) People (d) Compiler.
6. Moving pages between disk and main memory is the job of the (a) file manager (b) buffer manager (c) transaction manager (d) DDL compiler.
7. A user who only clicks buttons in a ready-made application is a (a) sophisticated user (b) naive user (c) DBA (d) system analyst.
8. Which advantage most directly prevents two copies of the same fact from disagreeing? (a) Security (b) Controlled redundancy (c) Recovery (d) Concurrency.
9. Which is a genuine limitation of a DBMS? (a) It cannot enforce constraints (b) It creates a single point of failure (c) It prevents concurrent access (d) It cannot store metadata.
10. The data dictionary of a modern DBMS is stored (a) in a separate text file (b) in the application code (c) inside the database itself (d) only in memory.

Answers: 1-(c) 2-(b) 3-(c) 4-(b) 5-(d) 6-(b) 7-(b) 8-(b) 9-(b) 10-(c)

Short Questions

1. Why is metadata essential to a DBMS?
2. Give one sentence distinguishing information from knowledge.
3. What is a mini-world in database terminology?
4. Name any four functions of a DBMS.
5. What is the role of the authorization manager?
6. Why must a backup be restore-tested?
7. What is meant by "declarative" access to data?
8. Give one example where relaxed consistency is acceptable.

9. What is vendor lock-in?
10. Why is "single point of failure" a limitation of centralizing data?

Long / Descriptive Questions

1. Explain the data–information–knowledge–metadata hierarchy with a complete university example, and discuss where each level is stored in a DBMS.
2. Draw the internal architecture of a DBMS and explain the responsibility of every component in the query processor and the storage manager.
3. "A DBMS is not always the right answer." Discuss the advantages and limitations of a DBMS, and state clear criteria for when a database should be used.
4. Describe in detail the responsibilities of a database administrator, illustrating each with a realistic scenario from a university or bank.
5. Compare the database requirements of a banking system, a social media platform and a telecom billing system, and explain why they lead to different technology choices.

Practical Exercises

1. Install MySQL 8 (or PostgreSQL) locally. Verify the installation and record the version string returned by `SELECT VERSION();`.
2. Create a database named `practice_db`, list all databases on the server, and then inspect the metadata: list the tables in `information_schema` and describe what three of those tables contain.
3. Create a small table `student(student_id, name, dept, cgpa)`, insert five rows, and write one query whose result is *information* rather than raw data (for example, the average CGPA per department).
4. Find where your DBMS stores its data files and its log files on disk. Record the paths and the current sizes, and explain why the log is stored separately from the data.
5. Write a one-page note classifying an application you use daily (banking app, ride-hailing app, campus portal) into the five components of a database system, and list five entities its database must store.

CHAPTER 2

File Systems and DBMS

Chapter 1 claimed that a DBMS offers consistency, integrity, security, concurrency and recovery. Those claims mean little until one has seen what happens without them. This chapter reconstructs the way organizations managed data before database systems existed, shows precisely how each problem arises, and then compares the two approaches.

2.1 Traditional File Processing System

Before the 1970s — and in many small organizations for long after — each department wrote its own programs and kept its own files.

Consider a university in that era. The admission office needs student particulars, so it creates a student file and writes programs to add and print records. The examination office needs marks, so it creates its own result file, which also contains the student's name and department because printing a mark sheet without a name is useless. The teachers' section keeps a teacher file. The registrar keeps a course file.

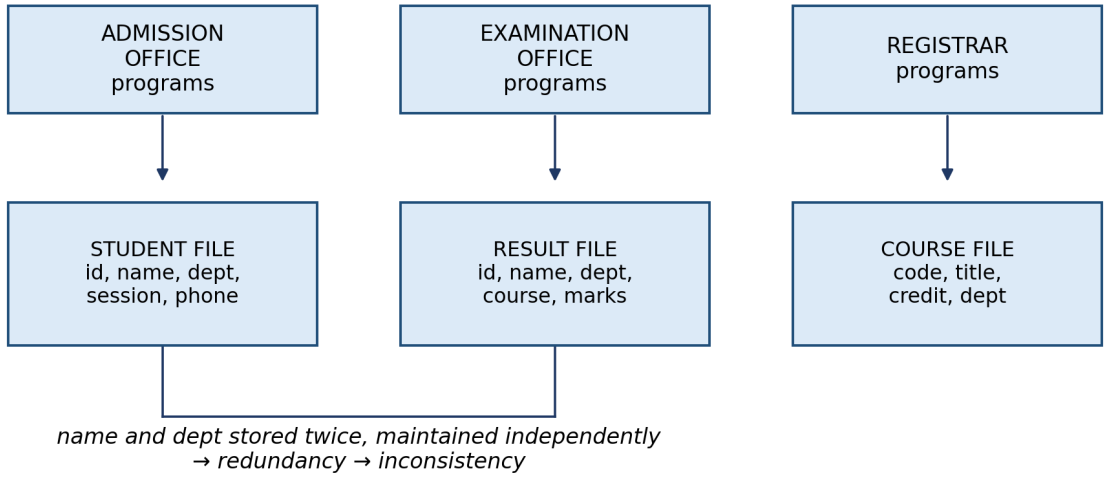


Figure 2.1: Departmental file processing in a university

The four files are typically:

```
Student File   : student_id, name, department, session, address, phone
Course File    : course_code, title, credit_hours, department
Teacher File   : teacher_id, name, designation, department, phone
Result File    : student_id, name, department, course_code, marks, grade
```

Each file has its own format, its own access programs, and its own copy of whatever fields its department happens to need. Nothing connects them except the discipline of the people who maintain them. That last sentence is the source of every problem in the next section.

2.2 Problems with File Systems

1. Data redundancy

The same fact is stored in several files. `name` and `department` appear in both the student file and the result file; `department` also appears in the course and teacher files.

Consequence: wasted storage, and more seriously, multiple versions of one truth.

2. Data inconsistency

A student transfers from CSE to EEE. The admission office updates the student file. The examination office does not update the result file, because nobody told it to.

```
STUDENT FILE : ASH1901001  Rahim Uddin  EEE
RESULT FILE  : ASH1901001  Rahim Uddin  CSE   CSE 3201  78
```

The database now asserts two contradictory facts. Any report is now suspect, and there is no automatic way to decide which copy is correct. This is **data inconsistency**, and it is the direct consequence of uncontrolled redundancy.

3. Data isolation

Data is scattered across files in different formats — one fixed-width, one comma-separated, one binary. Writing a program that answers *"list CSE students who scored above 70 in a course taught by a professor of the same department"* requires reading three incompatible files and joining them manually in application code. Each new question requires a new program.

4. Difficulty of access

File systems support only the queries that someone has already written a program for. If the vice-chancellor asks an unanticipated question at 4 p.m. — *"how many students from Noakhali district have CGPA above 3.5?"* — the answer requires a programmer, not a query. In a DBMS this is one `SELECT`.

5. Integrity problems

Rules such as "marks must lie between 0 and 100" or "a result row must refer to an existing student" live inside application programs. Every program must re-implement them. A new program written in a hurry, or a manual edit of the file, can insert `marks = 950` or a result for a student who does not exist. There is no central enforcement.

6. Security problems

File-level permissions are all-or-nothing. A clerk who must update a phone number needs write access to the student file — and therefore can also change CGPA. There is no way to grant access to one column, or to one subset of rows.

7. Atomicity problems

Consider a fee transfer written against two files: deduct 5,000 from the scholarship fund file and credit 5,000 to the student's account file. If the machine fails after the first write:

```
    Step 1: scholarship fund  -5000     <-- written to disk
    *** power failure ***
    Step 2: student account   +5000     <-- never happened
```

The money has vanished. A file system has no concept of "both or neither"; that concept is the **atomicity** property of a transaction, provided by a DBMS (Chapter 27).

8. Concurrent access anomalies

Two clerks open the same student record to update different fields. Both read the record, both modify their own copy in memory, and both write it back. The second write overwrites the first, silently — the **lost update** problem (Chapter 28).

```
    Clerk A reads  (phone=X, address=Y)
    Clerk B reads  (phone=X, address=Y)
    Clerk A writes (phone=NEW, address=Y)
    Clerk B writes (phone=X,   address=NEW)     <-- A's phone update is lost
```

File-based programs either ignore this or lock the entire file, which destroys throughput.

9. Backup and recovery problems

Recovery from a crash means restoring last night's tape and asking users to re-enter a day's work — if they remember it. There is no log of individual changes, so there is no way to roll forward to the moment before the failure, and no way to undo a half-finished update.

10. Program–data dependence

If the address field is widened from 50 to 100 characters, every program that reads the file must be recompiled, because the record layout is hard-coded in the program. This is the absence of **data independence** — the subject of Section 3.5.

2.3 File System vs DBMS

Basis	File Processing System	Database Management System
Data storage	Separate, independent files per application	One integrated database shared by all applications
Redundancy	High and uncontrolled	Controlled and minimized
Consistency	Not guaranteed; copies drift apart	Enforced, because a fact is stored once
Data relationships	Not represented; joins done in program code	Represented explicitly by keys and foreign keys
Integrity constraints	Coded separately in every program	Declared once in the schema, enforced for all access
Query capability	Only pre-written programs	Ad-hoc declarative queries (SQL)
Data independence	None; layout hard-coded in programs	Logical and physical data independence
Concurrency control	Absent or coarse file locking	Fine-grained locking, isolation levels, serializability
Atomicity	Not supported	Transactions with commit and rollback
Recovery	Manual restore from backup; work is lost	Log-based recovery to a consistent state
Security	File-level permissions only	User, role, table, column and row-level privileges
Metadata	Embedded in program code	Stored in the data dictionary
Backup	Whole-file copies	Full, incremental and point-in-time recovery
Development effort	High: each program reimplements storage logic	Low: DBMS provides searching, joining, locking, recovery
Cost and complexity	Low initial cost, simple	Higher cost, needs skilled administration
Suitable for	Small, single-user, isolated datasets	Shared, large, concurrent, mission-critical data

Important nuance for a viva: file systems are not obsolete. A DBMS itself stores its data *in files* managed by the operating system's file system. The comparison above is between file-*based application design* and database-based application design, not between files and databases as physical media.

Chapter Summary

- In traditional file processing each department keeps its own files and programs; nothing connects them except human discipline.
- The characteristic problems are redundancy, inconsistency, isolation, difficult ad-hoc access, weak integrity, coarse security, no atomicity, concurrency anomalies, poor recovery and program–data dependence.
- A DBMS solves each of these by integrating data, declaring constraints centrally, offering a declarative query language, and providing transactions, concurrency control, recovery and fine-grained security.
- The price is cost, complexity and administration; file-based storage remains reasonable for small, private, single-user datasets.

Key Terms

File processing system · Data redundancy · Data inconsistency · Data isolation · Ad-hoc query · Integrity problem · Atomicity problem · Lost update · Program–data dependence · Data integration · Data dictionary

Common Mistakes

1. Saying a DBMS "removes all redundancy" — it controls it.
2. Confusing *inconsistency* (copies disagree) with *redundancy* (copies exist). Redundancy is the cause; inconsistency is the effect.
3. Claiming file systems cannot store large data — they can; what they cannot do is *manage* shared, concurrent, constrained data.
4. Forgetting atomicity and concurrency when listing file system problems; these carry the most marks.
5. Writing that a DBMS does not use files. It does — it manages them itself.

Solved Problems

Problem 2.1 — A hospital stores patient name and phone in both a registration file and a billing file. A patient changes phone number and only registration is updated. Name the problem, explain the mechanism, and state how a DBMS prevents it.

Solution. The problem is **data inconsistency** caused by **uncontrolled redundancy**. Because the phone number exists twice and the two copies are maintained by different programs, an update applied to one copy leaves the other stale; the system now holds two contradictory values with no rule to decide which is authoritative. A DBMS stores the phone number once, in `patient.phone`; the billing module reads it through a join or a view rather than keeping a copy, so one `UPDATE` corrects every consumer simultaneously.

Problem 2.2 — Show, using a step-by-step schedule, how a lost update occurs in a file system, and state which DBMS component prevents it.

Solution.

Time	Clerk A	Clerk B	Stored value
t1	read balance = 5000		5000
t2		read balance = 5000	5000
t3	compute 5000 + 1000		5000
t4		compute 5000 − 500	5000
t5	write 6000		6000
t6		write 4500	4500

A's deposit of 1,000 has disappeared; the final value should have been 5,500. The DBMS **concurrency control** subsystem (locking or timestamp ordering, Chapters 28–30) prevents this by not allowing B to read the item until A's transaction has finished writing it.

Problem 2.3 — A file-based payroll program enforces "salary ≥ minimum wage" in its own code. Explain two ways the rule can still be violated, and the DBMS solution.

Solution. (i) A second program, written later by another team, writes to the same file without the check. (ii) A person edits the file directly with a text editor or a bulk-import utility that bypasses the program. In both cases the rule is not part of the data, only part of one program. A DBMS declares the rule as a `CHECK (salary >= 8000)` constraint in the schema; the constraint is verified by the DBMS on every insert or update no matter which program, tool or human performs it.

Problem 2.4 — Give one scenario where a file-based approach is preferable to a DBMS and justify it.

Solution. A logging utility on an embedded device that appends sensor readings to a local file, is read by one program, never shared, never queried ad hoc, and is discarded after upload. The DBMS would add memory footprint, startup cost and administration for no benefit; sequential appends to a file are faster and simpler, and there is no concurrency, integrity or security requirement to satisfy.

Problem 2.5 — Which file-system problem does each DBMS feature primarily address? (a) transaction log, (b) foreign key, (c) view, (d) query optimizer, (e) data dictionary.

Solution. (a) Atomicity and recovery problems. (b) Integrity problems, specifically dangling references between files. (c) Security problems — a view exposes only permitted rows and columns — and to a lesser extent logical data independence. (d) Difficulty of ad-hoc access — the optimizer makes unanticipated queries fast enough to be practical. (e) Program–data dependence, because layout information lives in the database, not in program source.

Review Questions

1. Describe how a university stored data before DBMSs, with a diagram.
2. Explain data redundancy and data inconsistency and the relationship between them.
3. What is data isolation and why does it make reporting difficult?
4. Explain the atomicity problem with a two-step money-transfer example.
5. Describe the lost update problem in a file-based system.
6. Why is security coarse in a file system?
7. What is program–data dependence? Give an example.
8. Compare file systems and DBMSs on any eight criteria.
9. Why is recovery difficult without a transaction log?
10. Is a DBMS always better than a file system? Justify.

Multiple Choice Questions

1. Storing a student's department in two files is an example of (a) inconsistency (b) redundancy (c) isolation (d) anomaly.
2. Two clerks overwriting each other's updates is the (a) dirty read (b) phantom read (c) lost update (d) deadlock problem.
3. "Both operations must succeed or neither" describes (a) consistency (b) atomicity (c) durability (d) isolation.
4. Hard-coding record layout in program source causes (a) data independence (b) program–data dependence (c) redundancy (d) isolation.
5. A DBMS enforces "marks between 0 and 100" through (a) a view (b) an index (c) a CHECK constraint (d) a trigger only.
6. Which is **not** a file-system problem? (a) Difficult ad-hoc access (b) Declarative query support (c) Weak security (d) Poor recovery.
7. Ad-hoc queries in a file system require (a) SQL (b) a new program (c) a view (d) an index.
8. Data stored in incompatible formats across files causes (a) data isolation (b) atomicity failure (c) deadlock (d) normalization.
9. Which DBMS feature most directly replaces manual joining in application code? (a) Trigger (b) Foreign key with SQL JOIN (c) Buffer manager (d) Log file.
10. A DBMS internally stores its data (a) in registers (b) in operating system files (c) only in RAM (d) in program code.

Answers: 1-(b) 2-(c) 3-(b) 4-(b) 5-(c) 6-(b) 7-(b) 8-(a) 9-(b) 10-(b)

Short Questions

1. Define data redundancy.
2. Give one example of data inconsistency.
3. What is an ad-hoc query?
4. Why can a file system not guarantee atomicity?
5. Name two concurrency problems in file systems.
6. What is stored in a transaction log?
7. Why is column-level security impossible with ordinary file permissions?
8. State one advantage of file processing over a DBMS.
9. What does "integrated data" mean?
10. Which DBMS subsystem prevents lost updates?

Long / Descriptive Questions

1. Describe the traditional file processing environment with a diagram and explain, with examples, at least eight problems it creates.
2. Compare file processing systems and DBMSs across at least twelve criteria in a table, and discuss three criteria in detail.
3. Explain the atomicity and concurrency problems of file systems using step-by-step schedules, and describe how a DBMS solves each.
4. "Redundancy is the root of most file-system problems." Justify this statement using inconsistency, storage, update effort and auditability.
5. Discuss the circumstances under which an organization should *not* migrate from files to a DBMS.

Practical Exercises

1. Create two CSV files, `student.csv` and `result.csv`, both containing student name and department. Update a department in one file only, then write a short script that detects the mismatch. Report how many rows disagree.
2. Write a program that appends to a shared file from two processes simultaneously and observe interleaving or lost data.
3. Load the same two CSV files into a database using `LOAD DATA INFILE` (MySQL) or `COPY` (PostgreSQL), define a foreign key between them, and attempt to insert a result for a non-existent student. Record the error message.
4. Add a `CHECK (marks BETWEEN 0 AND 100)` constraint, then attempt an invalid insert both through SQL and through a bulk import. Confirm that both are rejected.
5. Write a one-page comparison of the effort required to answer three ad-hoc questions (a) by writing programs over CSV files and (b) by writing SQL against tables.

CHAPTER 3

Database Architecture

Architecture answers the question: *where does each part of the work happen?* This chapter covers the deployment architectures (one-, two- and three-tier), the logical architecture that ANSI/SPARC defined (the three-schema architecture), and the property those architectures exist to provide — data independence.

3.1 One-Tier Architecture

In a **one-tier** (single-tier) architecture, the database, the DBMS and the user interface all reside on the same machine, and the user works directly with the DBMS.

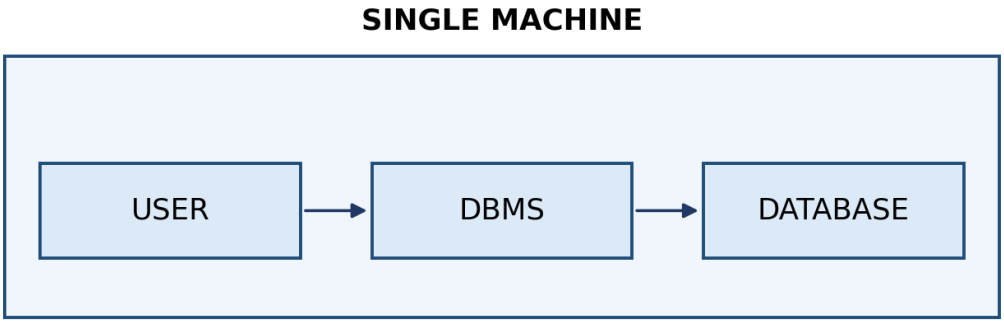


Figure 3.1: One-tier architecture

The user issues SQL directly; there is no network and no separate application server.

- **Examples:** MySQL command-line client on a developer's laptop; SQLite embedded in a mobile app; Microsoft Access used by one person.
- **Advantages:** simplest possible setup, no network latency, no configuration, ideal for learning, development and embedded use.
- **Disadvantages:** no sharing, no remote access, no central control, security limited to the machine's own login, and the data dies with the machine.
- **Use when:** developing, teaching, prototyping, or embedding a database inside a single application.

3.2 Two-Tier Architecture

In a **two-tier** (client–server) architecture, the application runs on the client machine and communicates directly with a database server over a network, usually through a driver such as ODBC or JDBC.

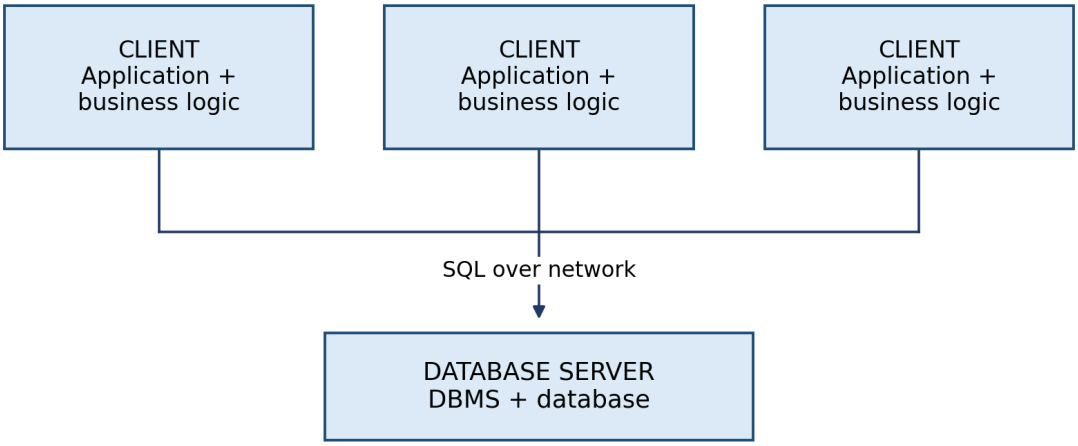


Figure 3.2: Two-tier client–server architecture

Example

A desktop inventory application installed in ten shop counters, each connecting through JDBC to a MySQL server in the back office. Business logic — how a discount is computed, what makes a sale valid — lives inside the desktop program.

Advantages	Disadvantages
Simple to build and understand	Business logic is duplicated on every client
Direct, fast communication with the server	Any logic change requires reinstalling every client
Good for small user populations (tens of users)	Each client holds a database connection: poor scalability
Server centralizes the data	Database credentials sit on client machines: security risk
Low infrastructure cost	Clients must be on the same network or VPN

3.3 Three-Tier Architecture

A **three-tier** architecture inserts an application server between the client and the database. The client handles presentation only; the application server holds business logic; the database server holds data.

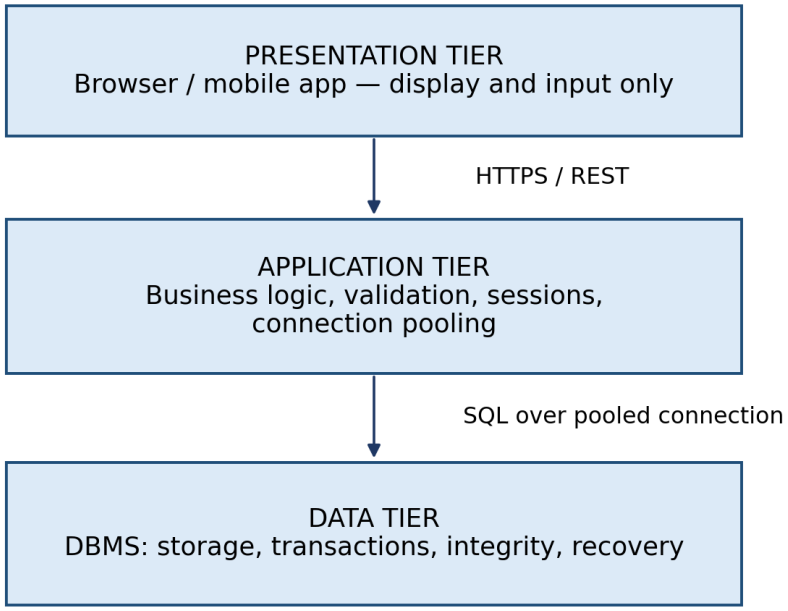


Figure 3.3: Three-tier architecture

E-commerce example

A customer taps **Place order** in a mobile app. The app sends an HTTPS request carrying the cart; it holds no database password and knows no SQL. The application server authenticates the session, checks stock, computes the payable amount including delivery charge and VAT, calls the payment gateway, and only then opens a database transaction that inserts into `orders` and `order_items` and decrements `stock`. The database server enforces the foreign keys, the `CHECK (quantity > 0)` constraint and the row locks that stop two customers from buying the last unit.

Advantages	Disadvantages
Business logic is central: change once, everyone gets it	More components to build, deploy and monitor
Clients never hold database credentials	Extra network hop adds some latency
Application server pools connections, so thousands of users share a few dozen database sessions	Higher infrastructure cost
Each tier can be scaled independently	Requires more design discipline
Clear separation of concerns aids maintenance and testing	Debugging spans multiple tiers

> *n*-tier architecture generalizes this idea: caching layers, message queues, microservices and API gateways may be inserted, but the principle — presentation, logic and data are separate concerns — is unchanged.

Basis	One-tier	Two-tier	Three-tier
Components	All on one machine	Client + database server	Client + app server + database server
Business logic	In the DBMS/user	On the client	On the application server
Scalability	None	Limited (tens of users)	High (thousands of users)
Security	Local only	Credentials on clients	Credentials only on app server

Basis	One-tier	Two-tier	Three-tier
Maintenance	Trivial	Update every client	Update one server
Example	SQLite, MySQL CLI	Desktop POS with JDBC	Web or mobile e-commerce

3.4 Three-Schema Architecture

The deployment architectures above describe *machines*. The **three-schema (ANSI/SPARC) architecture** describes *levels of description* and exists for one purpose: to separate the user's view of the data from its logical structure and from its physical storage.

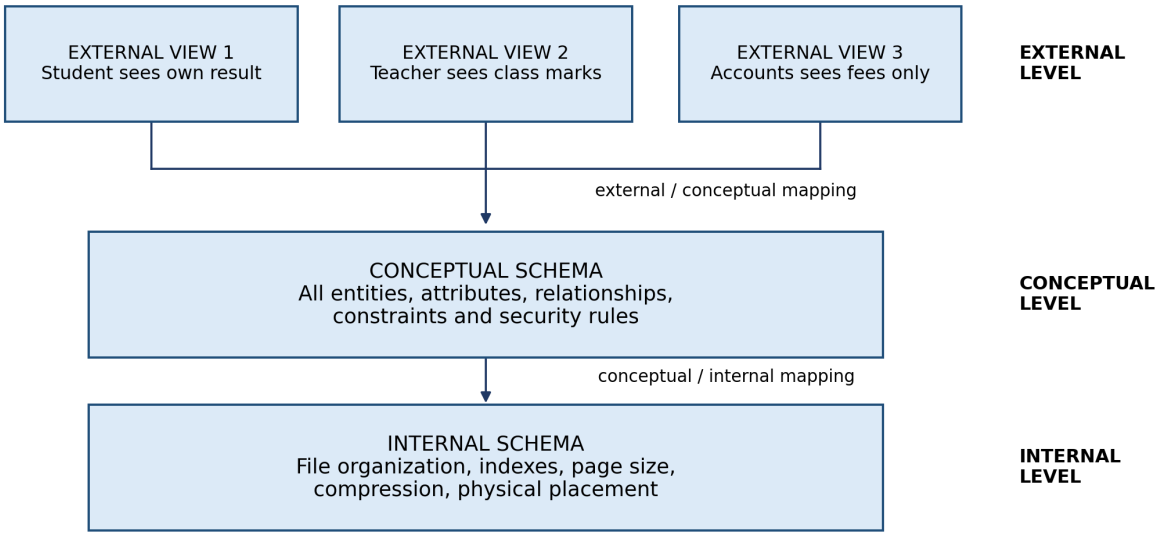


Figure 3.4: The ANSI/SPARC three-schema architecture

External level (view level)

The external level is what an individual user or application sees. There are many external schemas, one per user group, each showing only the relevant part of the database in a convenient form.

A student's view of results:

```
CREATE VIEW vw_my_result AS
SELECT course_code, credit_hours, grade, grade_point
FROM   enrollment e JOIN course c ON e.course_id = c.course_id
WHERE  e.student_id = CURRENT_USER_ID();
```

The student sees neither other students' marks nor the internal `mark_entry_by` audit column.

Conceptual level (logical level)

The conceptual schema is the single, complete, community-wide description of the entire database: every entity, every attribute, every relationship, every integrity constraint and every security rule — but no storage detail whatever. In SQL terms, this is the set of `CREATE TABLE` statements with their constraints.

Internal level (physical level)

The internal schema describes how the data is actually stored: which file organization is used, which indexes exist and of what type, how pages are laid out, what compression is applied, how records are placed on disk.

```sql
CREATE INDEX idx_student_dept ON student(dept_id) USING BTREE;
```

Nothing in the conceptual schema changes when this index is created, and no user view changes — yet queries on department become far faster.

Mappings

Mapping	Connects	Purpose
External/ conceptual	A user view to the conceptual schema	Lets the DBMS translate a query on a view into a query on base tables
Conceptual/ internal	The conceptual schema to stored structures	Lets the DBMS translate a logical request into physical reads

The DBMS performs both translations on every query. The cost of maintaining these mappings is exactly what buys data independence.

3.5 Data Independence

Data independence is the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

Physical data independence

The ability to change the **internal schema** without changing the **conceptual schema** or any application.

Examples of changes that should be invisible:

- Creating or dropping an index.
- Moving a table to a different disk or tablespace.
- Changing from a heap file to a hashed file organization.
- Enabling compression or changing the page size.
- Partitioning a large table by year.

If `enrollment` is partitioned by academic year to speed up reporting, this query is unchanged and still correct:

```sql
SELECT COUNT(*) FROM enrollment WHERE student_id = 'ASH1901001';
```

Physical data independence is comparatively easy to achieve and is well supported by all relational DBMSs.

Logical data independence

The ability to change the **conceptual schema** without changing the **external schemas** or applications.

Examples:

- Adding a new column, such as `student.blood_group`. Existing views and queries that name their columns explicitly are unaffected.
- Adding a new table or a new relationship.
- Splitting a table (for normalization) while preserving the old structure as a view:

```sql
-- Conceptual schema changes: address is moved to its own table.
CREATE TABLE student_address (
    student_id  CHAR(10) PRIMARY KEY,
    street      VARCHAR(100),
```

```
    city        VARCHAR(50),
    postal_code CHAR(4),
    FOREIGN KEY (student_id) REFERENCES student(student_id)
);

-- The old external view is preserved, so applications do not change.
CREATE VIEW vw_student_full AS
SELECT s.student_id, s.name, s.dept_id, a.street, a.city, a.postal_code
FROM   student s LEFT JOIN student_address a USING (student_id);
```

An application that still selects from `vw_student_full` sees exactly what it saw before, although the underlying tables have changed. That is logical data independence in action.

Logical data independence is **harder** to achieve than physical independence, because applications depend on the logical structure directly. Removing a column that a view uses, or changing the meaning of a column, cannot be hidden.

Basis	Physical data independence	Logical data independence
Levels involved	Internal ↔ conceptual	Conceptual ↔ external
Changed without affecting	Conceptual schema and applications	External views and applications
Typical change	Add an index, repartition, change file organization	Add a column, split a table, add a relationship
Difficulty	Easier	Harder
Support in practice	Excellent in all relational DBMSs	Partial; views help considerably

Chapter Summary

- One-tier keeps user, DBMS and database on one machine; two-tier separates client application from database server; three-tier adds an application server holding business logic.
- Three-tier scales better, keeps credentials off clients, centralizes logic and pools connections, at the cost of more components.
- The ANSI/SPARC three-schema architecture defines the external level (many user views), the conceptual level (one complete logical description) and the internal level (physical storage), joined by two mappings.
- Physical data independence means storage changes do not disturb the logical schema; logical data independence means logical changes do not disturb user views.
- Views are the practical tool for delivering both kinds of independence in a relational DBMS.

Key Terms

One-tier · Two-tier · Three-tier · *n*-tier · Client–server · Application server · Connection pooling · ANSI/SPARC · External schema · Conceptual schema · Internal schema · View · External/conceptual mapping · Conceptual/internal mapping · Physical data independence · Logical data independence

Important Notation

```
    External level  = many views          (what each user sees)
    Conceptual level = one logical schema (what exists, logically)
    Internal level   = one physical schema (how it is stored)
```

Common Mistakes

1. Confusing the three-*tier* architecture (machines) with the three-*schema* architecture (levels of description). They are different ideas and are frequently asked in the same paper.
2. Saying there is one external schema. There are many — one per user group.
3. Claiming logical data independence is easier than physical. It is the opposite.
4. Describing an index as a conceptual-level object; it belongs to the internal level.
5. Forgetting that the two mappings are what make independence possible.
6. Writing that three-tier is always better — for a ten-user desktop tool, two-tier is cheaper and simpler.

Solved Problems

Problem 3.1 — For each change, state whether physical or logical data independence is being exercised: (a) creating a hash index; (b) adding column `phone` to `student`; (c) moving the `result` table to an SSD; (d) splitting `employee` into `employee` and `employee_salary`.

Solution. (a) Physical — the internal schema changes only. (b) Logical — the conceptual schema changes; views selecting named columns are unaffected. (c) Physical. (d) Logical — the conceptual schema changes and a view must be provided if applications are to remain unchanged.

Problem 3.2 — A university portal serves 5,000 concurrent students during result publication. Explain why a two-tier design fails and how three-tier solves it.

Solution. In two-tier, each client holds its own database connection, so 5,000 clients mean up to 5,000 database sessions; each session consumes memory and lock-manager structures, and the server saturates long before that number. Credentials also reside on 5,000 machines. In three-tier, students talk to the application server over HTTPS; the application server keeps a pool of perhaps 50 database connections and multiplexes all requests over them, holds the only copy of the credentials, and can be scaled horizontally by adding more application servers behind a load balancer while the database sees a stable, modest session count.

Problem 3.3 — Write a view that provides an external schema in which a teacher sees only the students enrolled in courses that the teacher teaches.

Solution.

```sql
CREATE VIEW vw_my_students AS
SELECT s.student_id, s.name, c.course_code, e.grade
FROM   enrollment e
JOIN   student    s ON s.student_id = e.student_id
JOIN   course     c ON c.course_id  = e.course_id
WHERE  c.instructor_id = CURRENT_INSTRUCTOR_ID();
```

The teacher is granted `SELECT` on the view and no privilege at all on the base tables, so no other student's record is reachable. This is the external level enforcing security while the conceptual schema remains unchanged.

Problem 3.4 — Explain which schema level each of these belongs to: `CREATE TABLE`, `CREATE INDEX`, `CREATE VIEW`, page size, foreign key, user's mark sheet.

Solution. `CREATE TABLE` and foreign key — conceptual level. `CREATE INDEX` and page size — internal level. `CREATE VIEW` and the user's mark sheet — external level.

Problem 3.5 — A DBA partitions a 50-million-row `transaction` table by month. Applications are not modified. Which architectural property made this possible, and which component performed the translation?

Solution. **Physical data independence** made it possible: the internal schema changed while the conceptual schema (the table `transaction` with the same columns and constraints) stayed the same. The **conceptual/internal mapping**, maintained by the DBMS, translates each logical request into reads of the correct partitions, so an unchanged `SELECT` still returns the correct rows.

Problem 3.6 — Give one change that *cannot* be hidden by logical data independence and explain why.

Solution. Dropping a column that a view exposes, or changing its meaning — for instance, redefining `total_marks` from "out of 100" to "out of 50". The external schema depends directly on the existence and semantics of conceptual-level items; a view can rename, filter or combine, but it cannot invent data that no longer exists, nor can it detect that a number's meaning has silently changed. This is precisely why logical data independence is harder to achieve than physical.

Review Questions

1. Compare one-, two- and three-tier architectures with diagrams.
2. Why does a two-tier architecture scale poorly?
3. Explain the role of the application server in a three-tier design.
4. Draw and explain the ANSI/SPARC three-schema architecture.
5. Why are there many external schemas but only one conceptual schema?
6. What are the two mappings, and what does each translate?
7. Define physical data independence with two examples.
8. Define logical data independence with two examples.
9. Why is logical data independence harder to achieve?
10. How do views support both security and data independence?

Multiple Choice Questions

1. The level describing how data is physically stored is the (a) external (b) conceptual (c) internal (d) view level.
2. An index belongs to the (a) external (b) conceptual (c) internal (d) logical schema.
3. Business logic resides on the client in a (a) one-tier (b) two-tier (c) three-tier (d) n-tier architecture.
4. Adding a new column without breaking existing views demonstrates (a) physical (b) logical (c) view (d) storage independence.
5. Connection pooling is typically provided by the (a) client (b) application server (c) disk (d) optimizer.
6. The three-schema architecture was proposed by (a) IEEE (b) ANSI/SPARC (c) ISO only (d) ACM.
7. How many conceptual schemas does a database have? (a) One per user (b) One (c) One per table (d) Two.
8. SQLite embedded in a mobile app is an example of (a) one-tier (b) two-tier (c) three-tier (d) distributed architecture.
9. Which mapping translates a query on a view into a query on base tables? (a) Conceptual/internal (b) External/conceptual (c) Internal/physical (d) Logical/physical.
10. Which is **not** an advantage of three-tier? (a) Central business logic (b) Lower latency than two-tier (c) Better security (d) Independent scaling.

Answers: 1-(c) 2-(c) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(a) 9-(b) 10-(b)

Short Questions

1. What is an external schema?

2. State one difference between three-tier and three-schema architecture.
3. What is connection pooling and why does it matter?
4. Give one example of physical data independence.
5. Give one example of logical data independence.
6. Why should a client application not hold database credentials?
7. Name the two mappings in the ANSI/SPARC architecture.
8. Which schema level contains integrity constraints?
9. Which architecture suits an embedded application?
10. How many external schemas may a database have?

Long / Descriptive Questions

1. Explain one-, two- and three-tier architectures with diagrams, examples and a comparative evaluation.
2. Describe the ANSI/SPARC three-schema architecture in detail, including both mappings, and explain how it achieves data independence.
3. Distinguish physical and logical data independence with at least three examples of each, and explain why one is harder to achieve.
4. Explain how views implement the external level, and discuss their role in security, independence and usability.
5. Design a three-tier deployment for a university results portal expecting 5,000 concurrent users. Describe each tier's responsibility and justify your choices.

Practical Exercises

1. Connect to a database server from a client machine using a driver, and record the connection string. Identify which tier each component occupies.
2. Create a table, then create an index on it. Show that an existing query's result is unchanged while its execution plan changes (`EXPLAIN`). Explain which independence this demonstrates.
3. Create two views over one base table that expose different columns to different roles, grant privileges accordingly, and verify that each role cannot see the other's columns.
4. Add a column to a base table and confirm that an existing view still works. Then drop a column used by the view and record the error.
5. Write a short note mapping a web application you have used onto the three tiers, naming a concrete technology for each tier.

CHAPTER 4

Data Models

A **data model** is a collection of concepts for describing data, relationships among data, semantics and constraints. It is the vocabulary in which a schema is written. Different data models have dominated at different times, and knowing why one replaced another explains most of what follows in this book.

4.1 What a Data Model Provides

Every data model supplies three things:

1. **Structure** — what the building blocks are (records, tables, objects, documents, nodes).
2. **Operations** — how data is retrieved and modified (navigation, algebra, path traversal).
3. **Constraints** — what combinations of values are legal (keys, types, cardinalities).

Data models are usually grouped by level of abstraction:

Category	Purpose	Examples
High-level / conceptual	Close to how users think; used for design	ER model, EER model, UML class diagram
Representational / implementation	What real DBMSs implement	Relational, network, hierarchical, object-relational
Low-level / physical	How bytes are stored and accessed	Records, pointers, B+ trees, hash buckets

The designer's workflow moves down this table: requirements → ER model (conceptual) → relational schema (representational) → indexes and file organization (physical).

4.2 Hierarchical Model

Introduced with IBM's IMS in the late 1960s, the hierarchical model organizes records into a **tree**. Each child record has exactly one parent; the root has none.

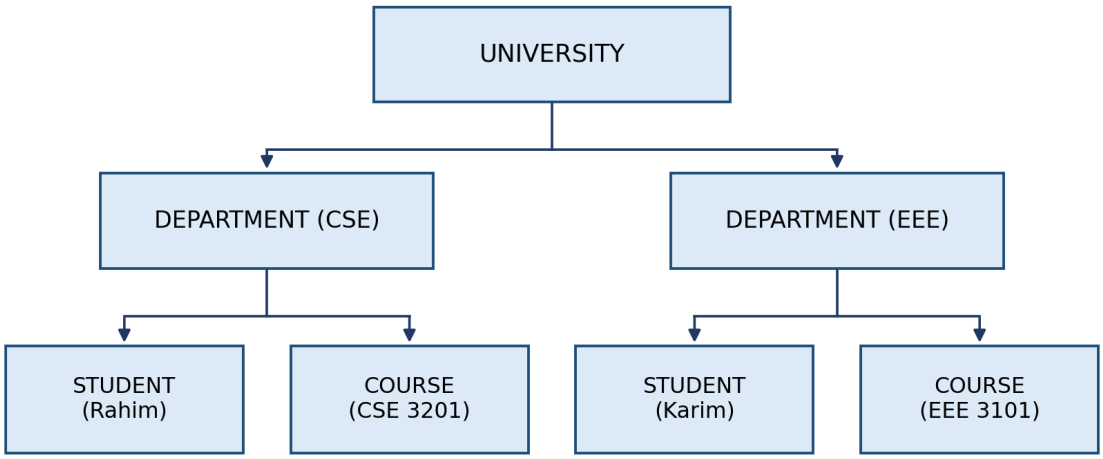


Figure 4.1: Hierarchical model — a university as a tree

- **Access:** navigation from the root downward, using parent–child pointers. The programmer writes *how* to traverse.
- **Advantages:** very fast for queries that follow the tree (get all courses of one department); simple and efficient for one-to-many hierarchies; mature and reliable.
- **Disadvantages:** a child cannot have two parents, so many-to-many relationships must be simulated by duplicating records — reintroducing redundancy. Queries that cut across the hierarchy (list every student taking any course with "database" in the title) are extremely awkward. The structure is rigid: changing the tree means rewriting programs. There is no declarative query language.

4.3 Network Model

Standardized by CODASYL in 1971, the network model generalizes the tree to a **graph**: a record may have several parents, and relationships are represented by named **sets** implemented as pointer chains.

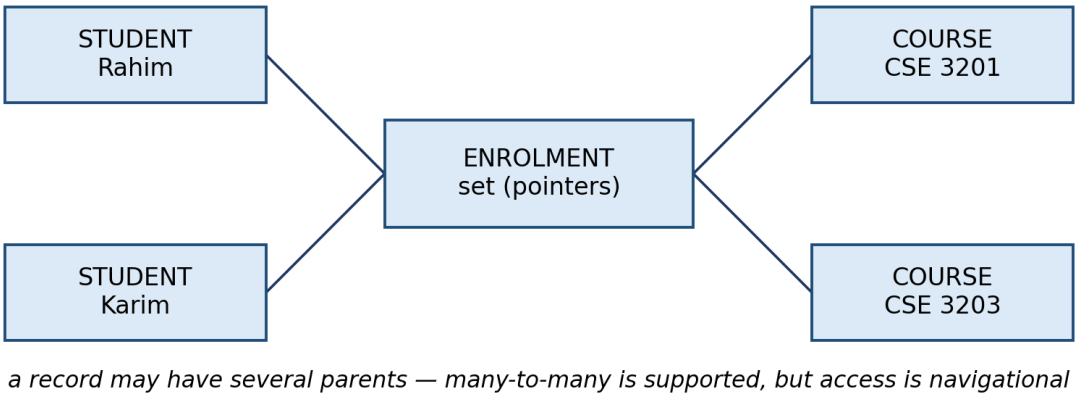


Figure 4.2: Network model — many-to-many becomes possible

- **Advantages:** supports many-to-many directly; navigation along pointers is fast; supports more complex structures than the hierarchical model.
- **Disadvantages:** the pointer structure is complex and must be known by the programmer; queries

are still navigational, not declarative; structural changes are expensive; achieving data independence is nearly impossible because programs encode the access paths.

4.4 Relational Model

Proposed by E. F. Codd in 1970, the relational model represents *all* data as **relations** — mathematical sets of tuples, presented as tables — and represents relationships not by pointers but by **values** (foreign keys matching primary keys).

```
STUDENT                                  DEPARTMENT
+------------+--------+---------+        +---------+----------------------+
| student_id | name   | dept_id |        | dept_id | dept_name            |
+------------+--------+---------+        +---------+----------------------+
| 101        | Rahim  | CSE     |------->| CSE     | Computer Sci. & Engg |
| 102        | Karim  | EEE     |------->| EEE     | Electrical & Elec.   |
+------------+--------+---------+        +---------+----------------------+
```

The link between a student and a department is the *value* `CSE` appearing in both tables. There is no pointer to maintain, no chain to repair, and no traversal order to program.

- **Advantages:** simple, uniform structure; a declarative query language (SQL) in which the user states *what*, not *how*; a rigorous mathematical foundation (set theory and predicate logic) enabling relational algebra, provable query equivalences and therefore automatic optimization; strong data independence; well-developed theory of normalization and integrity.
- **Disadvantages:** deeply nested or graph-like data (a social network's friend-of-friend traversal, a bill of materials) requires repeated joins and can be slow; object–relational impedance mismatch when mapping to object-oriented programs; horizontal scaling across many machines is harder than for some NoSQL models.

4.5 Object-Oriented Model

Data is stored as **objects**, as in object-oriented programming: each object has identity (independent of its attribute values), state (attributes) and behaviour (methods). It supports classes, inheritance, encapsulation and polymorphism, and complex types such as arrays and nested objects.

- **Advantages:** no impedance mismatch with OO programming languages; models complex types (CAD drawings, multimedia, engineering assemblies) naturally; supports user-defined types and methods.
- **Disadvantages:** no universally accepted declarative query standard comparable to SQL; weaker formal foundation for optimization; limited market adoption and tooling; steeper learning curve.

4.6 Object-Relational Model

Modern mainstream systems — PostgreSQL, Oracle, and increasingly MySQL — are **object-relational**: they keep the relational core and add object features such as user-defined types, inheritance, arrays and rich built-in types.

```sql
-- PostgreSQL: a user-defined composite type and an array column
CREATE TYPE address_t AS (street VARCHAR(100), city VARCHAR(50), postcode CHAR(4));

CREATE TABLE applicant (
    applicant_id SERIAL PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    home         address_t,              -- composite type
    languages    VARCHAR(20)[]           -- array attribute
```

```
);
```

- **Advantages:** relational rigour plus richer types; incremental adoption; SQL still works.
- **Disadvantages:** greater complexity; object extensions are largely vendor-specific and reduce portability.

4.7 Semi-Structured Model

Data carries its own structure, which may differ from record to record. There is no fixed schema fixed in advance. XML and JSON are the usual representations; document databases such as MongoDB are built on this model.

```
{ "student_id": "ASH1901001",
  "name": "Rahim Uddin",
  "department": "CSE",
  "courses": [ { "code": "CSE 3201", "grade": "A" },
               { "code": "CSE 3203", "grade": "A-" } ],
  "scholarship": { "type": "merit", "amount": 15000 } }
```

The second student's document might omit `scholarship` entirely and add `guardian`. In a relational table every row must have every column (possibly NULL); here each document carries only what applies to it.

- **Advantages:** flexible and evolvable schema; natural for hierarchical or irregular data; maps directly to web APIs; easy horizontal scaling.
- **Disadvantages:** weak global integrity enforcement; the application must handle structural variation; joins are limited and often discouraged; query optimization is less mature than in relational systems.

4.8 Comparison of Data Models

Basis	Hierarchical	Network	Relational	Object-oriented	Object-relational	Semi-structured
Structure	Tree	Graph	Tables	Objects/classes	Tables + objects	Documents/trees
Relationships	Parent–child pointers	Pointer sets	Foreign key values	Object references	Both	Nesting/references
Many-to-many	Not directly	Yes	Yes (junction table)	Yes	Yes	By nesting
Query language	Navigational	Navigational	Declarative (SQL)	OQL / language APIs	SQL + extensions	JSON/XPath-style APIs
Schema	Rigid, fixed	Rigid, fixed	Fixed but evolvable	Class-based	Fixed + extensible	Flexible / schema-less
Data independence	Very low	Low	High	Medium	High	Medium
Theoretical basis	None formal	None formal	Set theory, predicate logic	OO theory	Mixed	Tree/graph theory
Typical system	IBM IMS	IDMS	MySQL, PostgreSQL	ObjectStore, db4o	PostgreSQL, Oracle	MongoDB, CouchDB
Best for	Fixed hierarchies	Complex fixed networks	General-purpose business data	Complex engineering objects	Mixed workloads	Evolving, web-scale data

4.9 Why the Relational Model Became Dominant

Five reasons, and they are examinable as a set:

1. **Simplicity.** One structure — the table — represents everything. A user who understands a spreadsheet already half understands a database.

2. **Declarative querying.** SQL states the goal; the DBMS chooses the method. In the hierarchical and network models the *programmer* chose the access path, so any change in storage broke programs.
3. **Mathematical foundation.** Because relations are sets and queries are algebraic expressions, two differently written queries can be *proved* equivalent. This is what makes automatic query optimization possible — the optimizer may rewrite a query freely because equivalence is guaranteed by theory.
4. **Data independence.** Relationships are values, not pointers. Adding an index, reorganizing storage or adding a column does not invalidate existing programs.
5. **Design theory.** Functional dependencies and normal forms give objective, checkable criteria for whether a design is good — a tool the earlier models never had.

> The relational model did not win because it was fastest. In 1970 it was considered too slow to be practical; navigational systems beat it easily. It won because the optimizer eventually closed the performance gap while the productivity, correctness and independence advantages remained. This is a recurring lesson in systems: a better abstraction with an adequate implementation usually defeats a worse abstraction with an excellent one.

Chapter Summary

- A data model supplies structure, operations and constraints, and may be conceptual, representational or physical.
- The hierarchical model uses trees (one parent per child); the network model uses graphs with pointer sets; both are navigational.
- The relational model represents data as relations and relationships as matching values, is declarative, and rests on set theory and predicate logic.
- The object-oriented model stores objects with identity, state and behaviour; the object-relational model adds such features to a relational core.
- The semi-structured model (XML/JSON) carries its structure with the data and allows a flexible schema.
- The relational model became dominant because of simplicity, declarative querying, a provable foundation enabling optimization, data independence and normalization theory.

Key Terms

Data model · Conceptual model · Representational model · Physical model · Hierarchical model · Network model · CODASYL · Relational model · Codd · Tuple · Object-oriented model · Object identity · Object-relational model · User-defined type · Semi-structured model · Schema-less · Navigational access · Declarative access · Impedance mismatch

Common Mistakes

1. Saying the hierarchical model cannot represent many-to-many relationships *at all*. It can simulate them, but only by duplicating records.
2. Confusing the ER model with the relational model. The ER model is a *design* notation; the relational model is what the DBMS *implements*.
3. Claiming NoSQL replaced relational databases. It supplements them; most organizations run both.
4. Writing that the relational model stores pointers between tables. It stores *values*; the link is logical.
5. Calling semi-structured data “unstructured”. Unstructured means free text or images; semi-structured data carries structure within itself.

Solved Problems

Problem 4.1 — A student may take many courses and a course may be taken by many students. Show how this is represented in the hierarchical, network and relational models.

Solution. **Hierarchical:** impossible directly, since a course record can have only one parent. The usual workaround is to store the course under each student, duplicating course data once per enrolled student, which creates redundancy and update anomalies. **Network:** represented directly by an `ENROLMENT` set connecting `STUDENT` and `COURSE` owner and member records through pointer chains. **Relational:** represented by a third table whose primary key is the pair of foreign keys:

```
CREATE TABLE enrollment (
    student_id CHAR(10),
    course_id  INT,
    grade      CHAR(2),
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES student(student_id),
    FOREIGN KEY (course_id)  REFERENCES course(course_id)
);
```

Problem 4.2 — Explain why automatic query optimization is feasible in the relational model but not in the network model.

Solution. In the relational model a query is an expression in relational algebra over sets. Algebraic laws (commutativity of joins, pushing selections below joins, and so on) let the optimizer transform the expression into a provably equivalent one and choose the cheapest form. In the network model the query *is* the access path: the program says "find owner, then follow next member pointer". There is no algebra over which to reason and no guarantee that a different traversal returns the same result, so the system cannot safely rewrite anything.

Problem 4.3 — Choose a data model for each: (a) a CAD system storing nested mechanical assemblies; (b) a bank's core ledger; (c) a product catalogue whose attributes differ by category; (d) a social network's friend recommendations. Justify.

Solution. (a) Object-oriented or object-relational — deeply nested complex types with behaviour; relational decomposition would require many joins. (b) Relational — transactions, ACID guarantees and integrity dominate every other consideration. (c) Semi-structured (document) — a book has an author and a page count while a shirt has a size and colour; forcing both into one table produces a wide, sparse, mostly NULL relation. (d) Graph model (Chapters 35 and 38) — the workload is traversal of relationships, where repeated relational self-joins become expensive.

Problem 4.4 — Distinguish "structured", "semi-structured" and "unstructured" data with an example of each.

Solution. **Structured** data conforms to a fixed schema known in advance: rows of a `student` table. **Semi-structured** data carries its own descriptive tags and may vary between records: a JSON document where one student has a scholarship field and another does not. **Unstructured** data has no internal descriptive structure usable by a DBMS: a scanned answer script image or the free text of an essay. A DBMS can index and constrain the first, partially query the second, and mostly only store and full-text search the third.

Problem 4.5 — Give two changes that would break programs in a network database but not in a relational one, and explain why.

Solution. (i) Adding a new record type in the middle of a pointer chain, which changes the traversal sequence programs rely on. (ii) Reorganizing the physical placement of records so that a previously fast set traversal becomes slow, forcing programs to be rewritten for a different access path. In the relational

model neither is visible to a program: relationships are matched by value, so inserting a new table changes nothing for existing queries, and physical reorganization is hidden by the conceptual/internal mapping (physical data independence).

Review Questions

1. Define a data model and state the three things it must provide.
2. Distinguish conceptual, representational and physical data models.
3. Explain the hierarchical model with a diagram and list its limitations.
4. Explain the network model and how it improves on the hierarchical model.
5. Describe the relational model and explain how relationships are represented.
6. What is object identity and why does the object-oriented model need it?
7. What is the object-relational model? Give an example of an object feature in SQL.
8. Describe the semi-structured model and give a JSON example.
9. Compare all six models in a table.
10. Explain five reasons for the dominance of the relational model.

Multiple Choice Questions

1. The hierarchical model organizes data as a (a) graph (b) tree (c) table (d) set of documents.
2. The network model was standardized by (a) ANSI (b) CODASYL (c) IBM (d) ISO.
3. The relational model was proposed by (a) Chen (b) Codd (c) Bachman (d) Date.
4. In the relational model, relationships are represented by (a) pointers (b) sets (c) matching values (d) tree links.
5. Which model best suits data whose attributes vary from record to record? (a) Hierarchical (b) Relational (c) Semi-structured (d) Network.
6. "Impedance mismatch" refers to friction between (a) disk and memory (b) OO programs and relational tables (c) client and server (d) SQL and algebra.
7. Which model has the weakest data independence? (a) Hierarchical (b) Relational (c) Object-relational (d) Semi-structured.
8. Arrays and user-defined types in PostgreSQL are features of the (a) hierarchical (b) network (c) object-relational (d) semi-structured model.
9. Query optimization is possible in the relational model mainly because of its (a) speed (b) mathematical foundation (c) storage format (d) use of SQL keywords.
10. A JSON document database follows the (a) network (b) semi-structured (c) hierarchical (d) object-oriented model.

Answers: 1-(b) 2-(b) 3-(b) 4-(c) 5-(c) 6-(b) 7-(a) 8-(c) 9-(b) 10-(b)

Short Questions

1. What are the three components of a data model?
2. Why can a child have only one parent in the hierarchical model?
3. What is navigational access?
4. Name two advantages of the relational model.
5. What is a user-defined type?
6. Give one example of semi-structured data.
7. What is object identity?
8. Which model suits graph traversal workloads?

9. Why is the ER model called a conceptual model?
10. Name one disadvantage of the relational model.

Long / Descriptive Questions

1. Explain the hierarchical, network and relational models with diagrams, and compare them on structure, relationships, querying and data independence.
2. Discuss the object-oriented and object-relational models, with SQL examples of object features.
3. Explain the semi-structured model in detail and compare it with the relational model for a product catalogue.
4. "The relational model won on abstraction, not on speed." Discuss, giving five reasons for its dominance.
5. Given a hospital's requirements, recommend a data model for each subsystem (patient records, medical imaging, billing, doctor referral network) and justify each choice.

Practical Exercises

1. Represent one university department, its courses and its students in (a) a JSON document and (b) three relational tables. Compare the effort needed to answer "which students take CSE 3201?" in each.
2. In PostgreSQL, create a table with an array column and a composite type; insert and query rows. Record which parts are non-standard SQL.
3. Store the same five records in a relational table and in a MongoDB collection; add a new field to only one record in each and record what happens.
4. Draw a hierarchical tree and a network diagram for a library (library → sections → books → borrowers) and identify which relationship the tree cannot express.
5. Write a two-page comparison of relational and document models for an e-commerce catalogue, covering schema evolution, integrity and query flexibility.

CHAPTER 5

The Relational Model

This chapter defines precisely the six terms every later chapter depends on: relation, tuple, attribute, domain, schema and instance. One table is used throughout so that the definitions can be read off a concrete example.

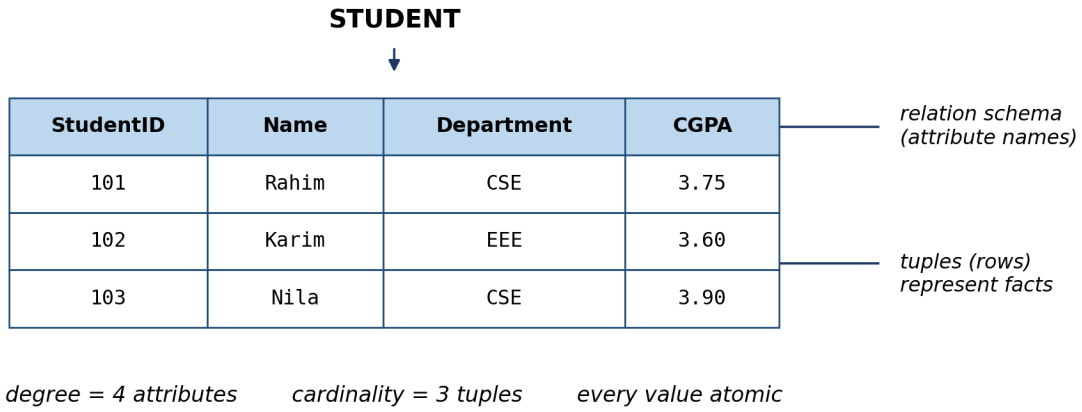


Figure 5.1: The running example — the STUDENT relation

5.1 Relation

A **relation** is a named, two-dimensional table of values in which every row represents one fact about one entity or relationship, and every column represents one property of that entity.

Formally, given domains D_1, D_2, …, D_n, a relation r is a subset of the Cartesian product $D_1 \times D_2 \times \ldots \times D_n$. Because a relation is mathematically a **set** of tuples, four properties follow, and every one of them is examinable:

1. **No duplicate tuples.** A set cannot contain the same element twice; this is why every relation must have a key.
2. **Tuple order is irrelevant.** $\{t_1, t_2\}$ and $\{t_2, t_1\}$ are the same relation. If output order matters, `ORDER BY` must be specified.
3. **Attribute order is irrelevant** in theory, although SQL does assign columns a position.
4. **All values are atomic.** A cell holds one indivisible value, not a list or a nested table. This property is the **first normal form** requirement of Chapter 19.

> A caution students must carry forward: SQL tables are *not* strictly relations. SQL permits duplicate rows unless a key or `DISTINCT` forbids them, and SQL has `NULL`, which the pure relational model does not. The relational model is the theory; SQL is a practical, slightly impure implementation of it.

5.2 Tuple

A **tuple** is one row of a relation: an ordered collection of values, one from each attribute's domain, that together state one complete fact.

```
(101, 'Rahim', 'CSE', 3.75)
```

reads as: *there exists a student whose ID is 101, whose name is Rahim, who belongs to CSE, and whose CGPA is 3.75*. Each tuple is an assertion of truth about the mini-world. The number of tuples in a relation is its **cardinality** — here, 3.

5.3 Attribute

An **attribute** is a named column of a relation representing one property of the entity the relation describes. The number of attributes is the **degree** (or arity) of the relation — here, 4.

Every attribute has a name unique within its relation and an associated domain. `Name` in `STUDENT` and `Name` in `INSTRUCTOR` are different attributes that happen to share a name; when both appear in a query, they are qualified as `student.name` and `instructor.name`.

5.4 Domain

A **domain** is the set of all permitted atomic values for an attribute, together with the operations meaningful on them.

Attribute	Domain description	Permitted	Not permitted
StudentID	Integers 100–999	`101`	`-5`, `'abc'`
Name	Character strings up to 100 characters	`'Rahim'`	a 500-character string
Department	The set {CSE, EEE, ME, CE, BBA}	`'CSE'`	`'XYZ'`
CGPA	Decimal numbers 0.00–4.00	`3.75`	`4.50`

In SQL a domain is approximated by a data type plus constraints:

```
CREATE TABLE student (
    student_id  INT          PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    department  VARCHAR(5)   NOT NULL,
    cgpa        DECIMAL(3,2) CHECK (cgpa BETWEEN 0.00 AND 4.00)
);
```

Two further points matter. First, a domain is **atomic**: `'CSE, EEE'` stored in one cell violates the model. Second, a special marker, **NULL**, means "value unknown or not applicable" and belongs to no domain; comparisons with NULL yield *unknown* rather than true or false, a fact that causes many query bugs (Chapter 11).

5.5 Schema

A **relation schema** is the description of a relation: its name, its attributes, their domains and its constraints. It is written

```
STUDENT(StudentID: integer, Name: string, Department: string, CGPA: decimal)
```

or, more compactly in examination answers,

```
STUDENT(StudentID, Name, Department, CGPA)
```

with the primary key underlined.

A **database schema** is the set of all relation schemas plus the constraints that hold between them:

```
STUDENT(StudentID, Name, Department, CGPA)
COURSE(CourseID, Title, Credit, Department)
ENROLLMENT(StudentID, CourseID, Semester, Grade)
     StudentID references STUDENT
     CourseID  references COURSE
```

The schema is the **intension** of the database — its structure. It is defined once and changes rarely.

5.6 Instance

A **relation instance** (or relation state) is the set of tuples present in a relation at a particular moment; a **database instance** is the set of all relation instances at that moment. This is the **extension** of the database — its content. It changes with every insert, update and delete.

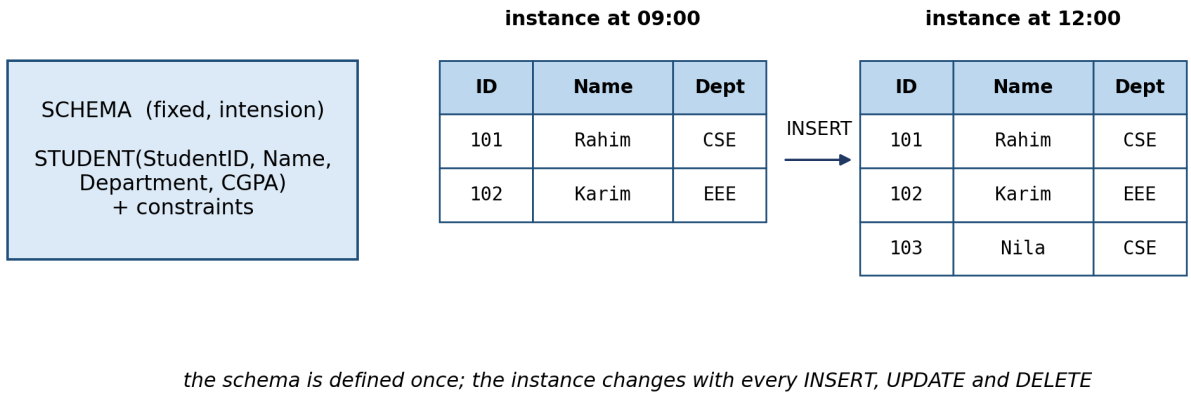


Figure 5.2: Schema versus instance over time

Basis	Schema	Instance
Meaning	Structure and constraints	Actual data at a moment
Also called	Intension	Extension
Changes	Rarely (a design change)	Constantly (every DML statement)
Defined by	DDL (`CREATE TABLE`)	DML (`INSERT`, `UPDATE`, `DELETE`)
Example	`STUDENT(StudentID, Name, …)`	The three rows shown above

A valid instance is one that satisfies every constraint in the schema. The DBMS's job, on every modification, is to reject any instance that would be invalid.

5.7 Relational Integrity Rules

Three rules define what it means for an instance to be valid.

1. **Domain integrity.** Every attribute value must belong to the attribute's domain. `CGPA = 4.50` violates it.
2. **Entity integrity.** No attribute of a primary key may be NULL. If `StudentID` were NULL, the tuple could not be identified or referenced.
3. **Referential integrity.** Every non-NULL foreign key value must match an existing primary key

value in the referenced relation. An `ENROLLMENT` row for `StudentID = 999` when no such student exists is a **dangling reference** and must be rejected.

```sql
-- Violates domain integrity: rejected by the CHECK constraint
INSERT INTO student VALUES (104, 'Sumaiya', 'CSE', 4.50);

-- Violates entity integrity: primary key cannot be NULL
INSERT INTO student VALUES (NULL, 'Sumaiya', 'CSE', 3.40);

-- Violates referential integrity: student 999 does not exist
INSERT INTO enrollment VALUES (999, 3201, '5th', 'A');
```

5.8 Advantages and Limitations of the Relational Model

Advantages	Limitations
Simple, uniform tabular structure	Complex nested or graph data needs many joins
Declarative SQL; user states *what*, not *how*	Impedance mismatch with object-oriented code
Formal basis permits provable optimization	Horizontal scaling across machines is difficult
Strong integrity enforcement	Rigid schema; changes need migration
Well-developed normalization theory	Performance overhead of ACID for very high write rates
Excellent data independence	Not ideal for unstructured or rapidly evolving data

Chapter Summary

- A relation is a set of tuples over named attributes drawn from domains; because it is a set, duplicate tuples are impossible and row order is irrelevant.
- A tuple is a row asserting one fact; an attribute is a named column; a domain is the set of permitted atomic values.
- Degree is the number of attributes; cardinality is the number of tuples.
- A relation schema describes structure (intension); a relation instance is the current data (extension).
- Domain, entity and referential integrity define a valid instance.
- SQL tables deviate from pure relations by permitting duplicates and NULLs.

Key Terms

Relation · Tuple · Attribute · Domain · Atomic value · Degree · Arity · Cardinality · Relation schema · Database schema · Instance · Intension · Extension · NULL · Domain integrity · Entity integrity · Referential integrity · Dangling reference · First normal form

Important Notation

```
STUDENT(StudentID, Name, Department, CGPA)     relation schema; key underlined
r(R)                                           instance r of schema R
degree(R)      = number of attributes
cardinality(r) = number of tuples
r ⊆ D1 × D2 × ... × Dn                         a relation is a subset of a Cartesian
 ↪ product
```

Common Mistakes

1. Using *relation* and *relationship* interchangeably. A relation is a table; a relationship is an association between entities (Chapter 7).
2. Confusing degree with cardinality. Degree counts columns; cardinality counts rows.
3. Believing SQL tables cannot contain duplicate rows. Without a key or `DISTINCT`, they can.
4. Storing a list such as `'CSE, EEE'` in one cell — a violation of atomicity and of 1NF.
5. Treating NULL as zero or as an empty string. It is neither; `NULL = NULL` evaluates to unknown, not true.
6. Confusing schema with instance in examination answers; the words intension and extension distinguish them cleanly.

Solved Problems

Problem 5.1 — For the `STUDENT` relation of Figure 5.1, state the degree, the cardinality, the domain of `CGPA` and one candidate key.

Solution. Degree = 4 (`StudentID`, `Name`, `Department`, `CGPA`). Cardinality = 3. The domain of `CGPA` is the set of decimal numbers with two places between 0.00 and 4.00. `StudentID` is a candidate key, since it is unique for every student and no proper subset of it is unique (it is a single attribute). `Name` is not a key because two students may share a name.

Problem 5.2 — Which relational property does each violate? (a) two identical rows; (b) `Department = 'CSE, EEE'`; (c) `StudentID = NULL`; (d) an enrollment row referring to a non-existent course.

Solution. (a) The set property — a relation cannot contain duplicate tuples; the underlying cause is the absence of a primary key. (b) Atomicity of values, and therefore first normal form. (c) Entity integrity. (d) Referential integrity.

Problem 5.3 — Write SQL that enforces all three integrity rules for `ENROLLMENT(StudentID, CourseID, Semester, Grade)`.

Solution.

```sql
CREATE TABLE enrollment (
    student_id INT          NOT NULL,
    course_id  INT          NOT NULL,
    semester   VARCHAR(10)  NOT NULL,
    grade      CHAR(2)      CHECK (grade IN ('A','A-','B+','B','B-','C+','C','D','F')),
    PRIMARY KEY (student_id, course_id, semester),          -- entity integrity
    FOREIGN KEY (student_id) REFERENCES student(student_id)  -- referential integrity
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (course_id)  REFERENCES course(course_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

The `PRIMARY KEY` gives entity integrity (no NULL, no duplicate enrollment of the same student in the same course in the same semester); the two `FOREIGN KEY` clauses give referential integrity; `CHECK` plus the declared data types give domain integrity.

Problem 5.4 — A relation has degree 5 and cardinality 0. Is it a valid relation? What does it represent?

Solution. Yes. The empty set is a legal subset of any Cartesian product, so an empty relation is a valid relation of degree 5. It represents a table that has been created but contains no tuples — for example, `enrollment` immediately after `CREATE TABLE`, before any student registers.

Problem 5.5 — Given domains $|D_1| = 3$, $|D_2| = 4$, $|D_3| = 2$, how many distinct tuples are possible, and

how many distinct relations exist over these domains?

Solution. The Cartesian product $D_1 \times D_2 \times D_3$ contains $3 \times 4 \times 2 = \mathbf{24}$ distinct tuples. A relation is any subset of that product, so the number of possible relations is $2^{24} = \mathbf{16{,}777{,}216}$. This calculation is worth remembering: it is the standard examination question testing whether a student really understands "a relation is a subset of a Cartesian product".

Problem 5.6 — Explain, with an example, why `SELECT` without `ORDER BY` may return rows in different orders on two executions.

Solution. A relation is a set, so no order is defined; the DBMS is free to return tuples in whatever order its chosen execution plan produces. If the optimizer first uses a full table scan and later, after an index is created, uses an index scan, the two runs may produce different orders even though both results are the same relation. Any application that requires a specific order must state it explicitly:

```
SELECT student_id, name FROM student ORDER BY cgpa DESC, name ASC;
```

Problem 5.7 — Distinguish schema and instance for a bank's `ACCOUNT` relation, and state which SQL commands change each.

Solution. The schema is `ACCOUNT(AccNo, CustomerID, Type, Balance, OpenDate)` with its keys and constraints; it is the intension and is altered only by DDL such as `CREATE TABLE`, `ALTER TABLE` and `DROP TABLE`. The instance is the set of account rows existing at a given moment — for example 42,318 accounts at 10:00 this morning; it is the extension and is changed by DML such as `INSERT`, `UPDATE` and `DELETE`, potentially thousands of times per second.

Review Questions

1. Define relation, tuple, attribute and domain with reference to one example table.
2. State the four properties that follow from a relation being a set.
3. Distinguish degree from cardinality.
4. Distinguish schema from instance, using the terms intension and extension.
5. State and illustrate the three relational integrity rules.
6. Why must every relation have a key?
7. In what three ways does an SQL table differ from a pure relation?
8. Why must attribute values be atomic?
9. What does NULL mean and why is it problematic?
10. List the advantages and limitations of the relational model.

Multiple Choice Questions

1. The number of attributes in a relation is its (a) cardinality (b) degree (c) domain (d) arity of tuples.
2. A relation is formally a subset of (a) a tuple (b) a Cartesian product of domains (c) a schema (d) an instance.
3. Which cannot occur in a pure relation? (a) An empty instance (b) Duplicate tuples (c) A single attribute (d) A composite key.
4. "No primary key attribute may be NULL" is (a) domain (b) entity (c) referential (d) semantic integrity.
5. `Department = 'CSE, EEE'` violates (a) entity integrity (b) atomicity (c) referential integrity (d) cardinality.
6. The current set of rows in a table is its (a) schema (b) intension (c) instance (d) domain.
7. If $|D_1| = 2$ and $|D_2| = 3$, the number of possible tuples is (a) 5 (b) 6 (c) 8 (d) 9.

8. A foreign key referring to a non-existent primary key value is a (a) dangling reference (b) surrogate key (c) partial dependency (d) view.
9. `NULL = NULL` evaluates to (a) TRUE (b) FALSE (c) UNKNOWN (d) an error.
10. Row order in a relation is (a) alphabetical (b) insertion order (c) undefined (d) defined by the primary key.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(b) 6-(c) 7-(b) 8-(a) 9-(c) 10-(c)

Short Questions

1. Define a domain.
2. What is an atomic value?
3. State the entity integrity rule.
4. What is the extension of a database?
5. Give one example of a dangling reference.
6. Why is `ORDER BY` needed in SQL?
7. What is the degree of `COURSE(CourseID, Title, Credit)`?
8. Can a relation have zero tuples?
9. Name one way SQL deviates from the relational model.
10. What does an underline denote in a relation schema?

Long / Descriptive Questions

1. Define relation, tuple, attribute, domain, schema and instance, illustrating each with the `STUDENT` table, and explain the four set-theoretic properties of relations.
2. Explain the three relational integrity rules with SQL examples of statements that violate each.
3. Distinguish schema from instance in detail, with examples of changes to each and the commands that cause them.
4. Discuss the advantages and limitations of the relational model, illustrating each point.
5. Explain how NULL complicates the relational model, giving examples in comparisons, aggregates and joins.

Practical Exercises

1. Create the `student` table with appropriate domain constraints and attempt four inserts that each violate a different rule. Record the exact error messages.
2. Insert duplicate rows into a table without a primary key, then add a primary key and observe the failure. Explain the result in relational terms.
3. Determine the degree and cardinality of every table in your sample database using `information_schema`.
4. Demonstrate that `SELECT` without `ORDER BY` gives no ordering guarantee by comparing execution plans before and after creating an index.
5. Write queries showing three different behaviours of NULL: in a `WHERE` comparison, in `COUNT(column)` versus `COUNT(*)`, and in an `AVG` computation.

CHAPTER 6

Keys and Constraints

A relation is a set, so every tuple must be distinguishable from every other. The mechanism that guarantees this is the **key**. Keys are also the means by which one relation refers to another, and they are the foundation of normalization (Chapter 19), indexing (Chapter 23) and join processing (Chapter 13). This chapter defines all seven kinds of key precisely and shows each one in SQL.

Throughout the chapter we use a single example relation:

```
STUDENT
+------------+---------+-----------------+-------------+-------+---------+
| student_id | reg_no  | email           | nid         | name  | dept_id |
+------------+---------+-----------------+-------------+-------+---------+
| ASH1901001 | 1901001 | rahim@nstu.edu  | 19901234567 | Rahim | CSE     |
| ASH1901002 | 1901002 | karim@nstu.edu  | 19901234568 | Karim | EEE     |
| ASH1901003 | 1901003 | nila@nstu.edu   | 19901234569 | Nila  | CSE     |
+------------+---------+-----------------+-------------+-------+---------+
```

Four attributes here are individually unique: `student_id`, `reg_no`, `email` and `nid`. That single fact generates most of the definitions below.

6.1 Super Key

A **super key** is any set of attributes whose values, taken together, uniquely identify a tuple in a relation.

The definition contains no requirement of minimality. Therefore if `{student_id}` is unique, then so is every superset of it.

```
{student_id}
{student_id, name}
{student_id, name, dept_id}
{student_id, reg_no, email, nid, name, dept_id}      <- the whole relation
{email}
{email, name}
{nid, dept_id}
```

All of these are super keys. A relation of degree n with a single-attribute key has 2^{n-1} super keys containing that attribute — a standard numerical examination question.

> Memory aid: **super key = unique**, nothing more. Minimality is what the *candidate* key adds.

6.2 Candidate Key

A **candidate key** is a *minimal* super key: a super key from which no attribute can be removed without destroying uniqueness.

In `STUDENT` the candidate keys are:

```
{student_id}    {reg_no}    {email}    {nid}
```

`{student_id, name}` is a super key but **not** a candidate key, because dropping `name` still leaves a unique set — it is therefore not minimal.

Two points are frequently examined:

- **Minimal does not mean smallest.** If a relation has candidate keys `{email}` and `{course_id, semester, section}`, both are candidate keys; the three-attribute one is minimal because removing any one of its attributes destroys uniqueness.
- Every relation has **at least one** candidate key, because in the worst case the set of all attributes is a super key and some minimal subset of it is unique.

Finding candidate keys

Given a relation and its functional dependencies (Chapter 17), the procedure is:

1. Compute the closure of candidate attribute sets.
2. A set X is a super key if X^+ contains all attributes.
3. X is a candidate key if it is a super key and no proper subset of X is.

6.3 Primary Key

The **primary key** is the one candidate key chosen by the designer to be the principal means of identifying tuples in that relation.

```sql
CREATE TABLE student (
    student_id CHAR(10)      PRIMARY KEY,
    reg_no     INT           NOT NULL UNIQUE,
    email      VARCHAR(100)  NOT NULL UNIQUE,
    nid        CHAR(11)      UNIQUE,
    name       VARCHAR(100)  NOT NULL,
    dept_id    CHAR(3)       NOT NULL
);
```

Properties enforced automatically by the DBMS:

1. **Unique** — no two tuples may share a primary key value.
2. **Not NULL** — entity integrity; no part of a primary key may be NULL.
3. **One per relation** — exactly one primary key, though it may consist of several attributes.
4. **Stable** — it should never need to change, because foreign keys elsewhere depend on it.
5. **Indexed** — every DBMS automatically creates a unique index on the primary key.

Choosing among candidate keys

Criterion	Prefer	Reason
Stability	`student_id` over `email`	Email changes; the ID does not
Simplicity	Single attribute over composite	Simpler joins and foreign keys
Size	`INT` over long `VARCHAR`	Smaller index, faster comparison
Privacy	`student_id` over `nid`	Do not propagate national ID across tables
Availability	Always present	Cannot be NULL, so must exist for every row

`nid` is unique but is a poor primary key: it is legally sensitive, sometimes unavailable for a new student, and would be copied into every referencing table.

6.4 Alternate Key

An **alternate key** (secondary key) is any candidate key that was *not* chosen as the primary key.

In `STUDENT`, with `student_id` chosen as primary key, the alternate keys are `{reg_no}`, `{email}` and `{nid}`. They remain unique and should still be declared, using `UNIQUE`:

```
ALTER TABLE student ADD CONSTRAINT uq_student_email UNIQUE (email);
```

The distinction between `PRIMARY KEY` and `UNIQUE` is a favourite viva question:

Basis	PRIMARY KEY	UNIQUE
Number per table	Exactly one	Any number
NULL allowed	No	Yes (one NULL in most DBMSs; MySQL allows several)
Purpose	Principal identifier	Enforces an alternate key
Referenced by foreign keys	Usually	Permitted but uncommon
Index created	Clustered by default in some systems	Non-clustered unique index

6.5 Composite Key

A **composite key** is a key consisting of two or more attributes. It is used when no single attribute is unique.

```
CREATE TABLE enrollment (
    student_id CHAR(10),
    course_id  INT,
    semester   VARCHAR(10),
    grade      CHAR(2),
    PRIMARY KEY (student_id, course_id, semester)
);
```

Here `student_id` alone is not unique (a student takes many courses), `course_id` alone is not unique (a course has many students), and even `(student_id, course_id)` is not unique if the student retakes a course in a later semester. The three attributes together are minimal and unique.

> Terminology note: a **compound key** is a composite key each of whose components is itself a foreign key — as in `(student_id, course_id)` above. Many textbooks use the two words interchangeably; if asked, state the distinction and then note that usage varies.

6.6 Surrogate Key

A **surrogate key** is a system-generated artificial identifier with no business meaning, introduced solely to identify tuples.

```
CREATE TABLE enrollment (
    enrollment_id INT AUTO_INCREMENT PRIMARY KEY,   -- surrogate key
    student_id    CHAR(10) NOT NULL,
    course_id     INT      NOT NULL,
    semester      VARCHAR(10) NOT NULL,
    grade         CHAR(2),
    UNIQUE (student_id, course_id, semester),       -- the natural key, still enforced
    FOREIGN KEY (student_id) REFERENCES student(student_id),
```

```
    FOREIGN KEY (course_id)  REFERENCES course(course_id)
);
```

The opposite of a surrogate key is a **natural key**: one drawn from real-world data, such as `email` or `nid`.

Surrogate key	Natural key
Meaningless integer or UUID	Carries business meaning
Never changes	May change (email, phone, name)
Compact and fast to join	May be long text
Requires an extra unique constraint on the real identifier	Self-documenting
Hides duplicates unless the natural key is also constrained	Prevents duplicates inherently

The practical rule used in industry: **use a surrogate primary key, and additionally declare a `UNIQUE` constraint on the natural key.** The surrogate gives stable, compact references; the unique constraint keeps the data honest. Omitting the second half is the classic beginner error — a table with an auto-increment id and no unique constraint will happily store the same student enrolled twice in the same course.

6.7 Foreign Key

A **foreign key** is an attribute or set of attributes in one relation whose values must match a primary (or unique) key value in another relation, or be entirely NULL.

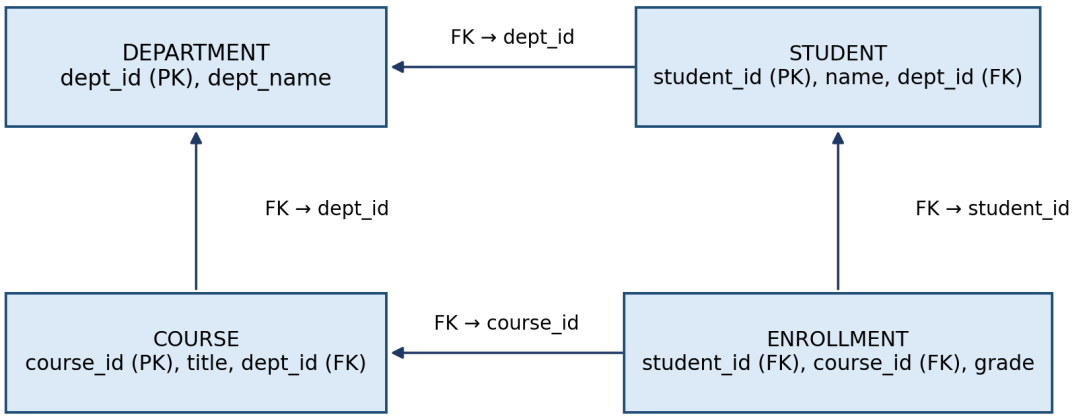


Figure 6.1: Foreign key relationships in the sample database

The referenced relation is the **parent**; the referencing relation is the **child**.

```
CREATE TABLE student (
    student_id CHAR(10) PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    dept_id    CHAR(3),
    CONSTRAINT fk_student_dept FOREIGN KEY (dept_id)
        REFERENCES department(dept_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

Referential actions

When a parent row is deleted or its key updated, the DBMS must decide what happens to matching child rows.

Action	Effect on delete of parent	Typical use
`RESTRICT` / `NO ACTION`	Reject the deletion if children exist	Default and safest; a department with students cannot vanish
`CASCADE`	Delete the children too	`enrollment` rows of a deleted student
`SET NULL`	Set the child's foreign key to NULL	A student whose department is dissolved, kept for records
`SET DEFAULT`	Set the child's foreign key to its default	Reassign orphans to a default category

```
-- A student is removed: their enrollments should go too.
ALTER TABLE enrollment
  ADD CONSTRAINT fk_enr_student FOREIGN KEY (student_id)
      REFERENCES student(student_id) ON DELETE CASCADE;

-- A course must not be deleted while results exist for it.
ALTER TABLE enrollment
  ADD CONSTRAINT fk_enr_course FOREIGN KEY (course_id)
      REFERENCES course(course_id) ON DELETE RESTRICT;
```

Notice that the choice is a *business* decision, not a technical one. Cascading deletes on financial records destroys audit history; restricting deletes on temporary data blocks routine cleanup.

Self-referencing foreign keys

A foreign key may reference the same relation, which is how hierarchies are stored in a table:

```
CREATE TABLE employee (
    emp_id     INT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    manager_id INT,
    FOREIGN KEY (manager_id) REFERENCES employee(emp_id)
);
```

The chief executive's `manager_id` is NULL; everyone else points to their manager. Querying such a table uses a self join (Chapter 13).

6.8 Key Summary Table

Key	Definition	Unique?	Minimal?	NULL allowed?	Count per relation
Super key	Any uniquely identifying attribute set	Yes	No	Not in practice	Many
Candidate key	Minimal super key	Yes	Yes	No	One or more
Primary key	Chosen candidate key	Yes	Yes	No	Exactly one
Alternate key	Unchosen candidate key	Yes	Yes	Usually yes	Zero or more
Composite key	Key of two or more attributes	Yes	Yes	No	Depends
Surrogate key	Artificial generated identifier	Yes	Yes	No	Zero or one
Foreign key	Attribute(s) matching a parent key	No	—	Yes	Zero or more

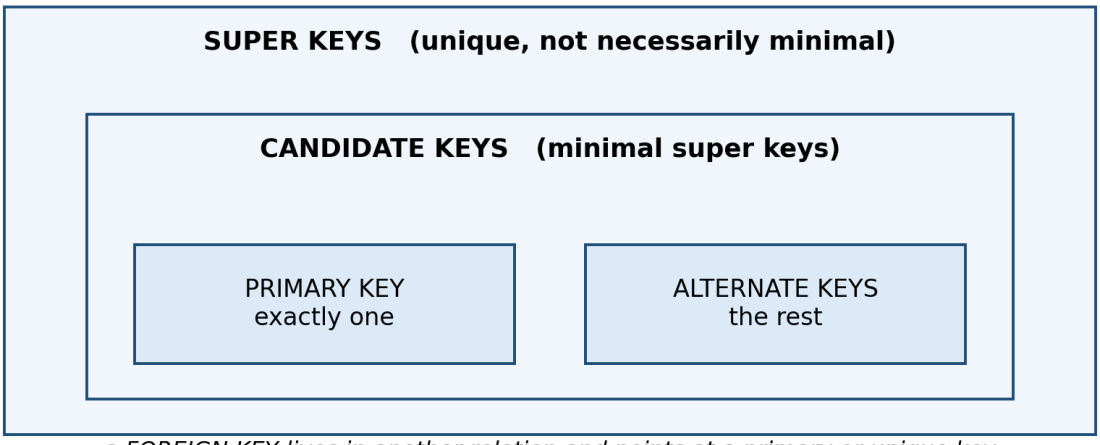


Figure 6.2: Containment of key concepts

6.9 A Complete Worked Example in SQL

The statements below are explained line by line, as required in laboratory examinations.

```sql
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    name       VARCHAR(100),
    department VARCHAR(50)
);
```

Line	Meaning
`CREATE TABLE Student (`	A DDL statement creating a new relation named `Student`; the parenthesis opens the list of attribute definitions
`student_id INT PRIMARY KEY,`	Attribute `student_id` of domain integer; `PRIMARY KEY` makes it unique, non-NULL, and causes a unique index to be created
`name VARCHAR(100),`	Variable-length character string of at most 100 characters; NULL is allowed because `NOT NULL` is absent
`department VARCHAR(50)`	Variable-length string of at most 50 characters; no comma follows because it is the last attribute
`);`	Closes the definition and terminates the statement

That table is correct but weakly constrained. A production version declares every rule the mini-world requires:

```sql
CREATE TABLE Student (
    student_id  CHAR(10)      NOT NULL,
    reg_no      INT           NOT NULL,
    email       VARCHAR(100)  NOT NULL,
    name        VARCHAR(100)  NOT NULL,
    dept_id     CHAR(3)       NOT NULL,
    cgpa        DECIMAL(3,2)  DEFAULT 0.00,
    admitted_on DATE          NOT NULL,

    CONSTRAINT pk_student        PRIMARY KEY (student_id),
    CONSTRAINT uq_student_reg    UNIQUE (reg_no),
    CONSTRAINT uq_student_email  UNIQUE (email),
```

```
    CONSTRAINT ck_student_cgpa   CHECK (cgpa BETWEEN 0.00 AND 4.00),
    CONSTRAINT ck_student_email  CHECK (email LIKE '%@%.%'),
    CONSTRAINT fk_student_dept   FOREIGN KEY (dept_id)
        REFERENCES Department(dept_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

Clause	Kind of constraint	What it guarantees
`NOT NULL`	Domain	The attribute always has a value
`PRIMARY KEY (student_id)`	Entity	Unique and non-NULL identification
`UNIQUE (reg_no)`, `UNIQUE (email)`	Key	Alternate keys remain unique
`CHECK (cgpa BETWEEN 0 AND 4)`	Domain	No impossible CGPA is stored
`DEFAULT 0.00`	Domain	A sensible value when none is supplied
`FOREIGN KEY (dept_id)`	Referential	No student belongs to a non-existent department
`ON UPDATE CASCADE`	Referential action	Renaming a department code updates students automatically
`ON DELETE RESTRICT`	Referential action	A department with students cannot be deleted

Naming constraints (`pk_`, `uq_`, `ck_`, `fk_` prefixes) is not decoration: when a constraint is violated at three in the morning, the error message names the constraint, and a good name tells the on-call engineer immediately what rule was broken.

Chapter Summary

- A **super key** is any uniquely identifying set of attributes; a **candidate key** is a minimal super key; the **primary key** is the chosen candidate key; the remaining candidate keys are **alternate keys**.
- A **composite key** has two or more attributes; a **surrogate key** is an artificial identifier with no business meaning; a **natural key** comes from real data.
- A **foreign key** references a parent relation's primary or unique key and may be NULL; referential actions (`RESTRICT`, `CASCADE`, `SET NULL`, `SET DEFAULT`) determine what happens to children when a parent changes.
- Entity integrity forbids NULL in a primary key; referential integrity forbids dangling foreign key values.
- Best practice: surrogate primary key plus a `UNIQUE` constraint on the natural key, all constraints declared in the schema and explicitly named.

Key Terms

Super key · Candidate key · Primary key · Alternate key · Secondary key · Composite key · Compound key · Surrogate key · Natural key · Foreign key · Parent and child relation · Referential action · CASCADE · RESTRICT · SET NULL · Entity integrity · Referential integrity · UNIQUE constraint · CHECK constraint · Self-referencing foreign key

Important Formulas / Notation

```
Every candidate key is a super key;  not every super key is a candidate key.
If {A} is a unique attribute in a relation of degree n,
the number of super keys containing A is 2^(n-1).
```

```
    Primary key  = chosen candidate key,  UNIQUE + NOT NULL.
```

Common Mistakes

1. Calling every unique attribute set a candidate key. Minimality must be checked.
2. Believing a composite key cannot be minimal. Minimality is about removing attributes, not about count.
3. Allowing a NULL in any part of a primary key.
4. Using a national ID, phone number or email as a primary key — values that change or are sensitive.
5. Adding a surrogate `id` and forgetting the `UNIQUE` constraint on the natural key, so duplicates slip in.
6. Declaring `ON DELETE CASCADE` on financial or audit tables, destroying history with one delete.
7. Assuming a foreign key column is automatically indexed. In MySQL/InnoDB it is; in PostgreSQL and SQL Server it is **not**, and missing indexes on foreign keys are a common cause of slow joins and slow cascading deletes.
8. Declaring constraints in application code rather than in the schema, so a different application can violate them.

Solved Problems

Problem 6.1 — For `EMPLOYEE(emp_id, nid, email, phone, name, dept)` where `emp_id`, `nid` and `email` are each unique and `phone` may repeat, list (a) five super keys, (b) all candidate keys, (c) a reasonable primary key with justification, (d) the alternate keys.

Solution. (a) `{emp_id}`, `{nid}`, `{email}`, `{emp_id, name}`, `{email, dept, phone}` — any set containing a unique attribute. (b) `{emp_id}`, `{nid}`, `{email}` — each is unique and single-attribute, hence minimal. (c) `emp_id`: it is stable, internally generated, compact and not legally sensitive, while `email` changes and `nid` is personal data that should not be copied into every child table. (d) `{nid}` and `{email}`, declared with `UNIQUE`.

Problem 6.2 — A relation has degree 6 and exactly one single-attribute candidate key `A`. How many super keys contain `A`?

Solution. A super key containing `A` is formed by choosing any subset of the remaining 5 attributes. The number of such subsets is $2^5 = \mathbf{32}$. In general, for degree n, the answer is 2^{n-1}.

Problem 6.3 — Decide whether each is a candidate key of `ENROLLMENT(student_id, course_id, semester, grade)`, given that a student may retake a course in a different semester: (a) `{student_id}`; (b) `{student_id, course_id}`; (c) `{student_id, course_id, semester}`; (d) `{student_id, course_id, semester, grade}`.

Solution. (a) Not a super key — a student has many enrollments. (b) Not a super key, because a retake gives two rows with the same pair. (c) A super key and minimal, since removing any one attribute breaks uniqueness — therefore the **candidate key**. (d) A super key but not minimal (`grade` can be removed), therefore not a candidate key.

Problem 6.4 — Write SQL for `Department` and `Student` such that renaming a department code updates students automatically, but deleting a department that still has students is refused. Then demonstrate both behaviours.

Solution.

```sql
CREATE TABLE Department (
    dept_id   CHAR(3) PRIMARY KEY,
    dept_name VARCHAR(80) NOT NULL UNIQUE
);
```

```sql
CREATE TABLE Student (
    student_id CHAR(10) PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    dept_id    CHAR(3) NOT NULL,
    CONSTRAINT fk_student_dept FOREIGN KEY (dept_id)
        REFERENCES Department(dept_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

INSERT INTO Department VALUES ('CSE','Computer Science and Engineering');
INSERT INTO Student    VALUES ('ASH1901001','Rahim','CSE');

UPDATE Department SET dept_id = 'CSI' WHERE dept_id = 'CSE';
-- Student.dept_id is now 'CSI' automatically (CASCADE).

DELETE FROM Department WHERE dept_id = 'CSI';
-- ERROR: cannot delete or update a parent row: a foreign key constraint fails (RESTRICT
    ↪ ).
```

Problem 6.5 — An `orders` table has `order_id INT AUTO_INCREMENT PRIMARY KEY` and columns `customer_id`, `product_id`, `order_date`. Duplicate orders keep appearing. Diagnose and fix.

Solution. The surrogate key guarantees that every *row* is distinct, but it says nothing about the *business* identity of an order. Two identical orders receive two different `order_id` values and both are accepted. The fix is to declare the natural key as well — assuming a customer may order a product only once per day:

```sql
ALTER TABLE orders
  ADD CONSTRAINT uq_orders_natural UNIQUE (customer_id, product_id, order_date);
```

This is the general lesson: a surrogate key never replaces the need to state what makes a row unique in the real world.

Problem 6.6 — Explain the difference between `PRIMARY KEY` and `UNIQUE NOT NULL`. Are they identical?

Solution. Functionally they are very close: both enforce uniqueness and both forbid NULL. The differences are: a table may have only one primary key but many `UNIQUE NOT NULL` constraints; the primary key is the declared principal identifier and is what foreign keys normally reference; in some systems (SQL Server, MySQL/InnoDB) the primary key determines the clustered physical ordering of the table, which `UNIQUE` does not; and tools, documentation and ER diagrams derive the entity identifier from the primary key. So they are equivalent in *enforcement* but not in *meaning* or in *physical effect*.

Problem 6.7 — For an employee hierarchy, write the self-referencing table and a query listing each employee with their manager's name.

Solution.

```sql
CREATE TABLE employee (
    emp_id     INT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    manager_id INT NULL,
    CONSTRAINT fk_emp_mgr FOREIGN KEY (manager_id) REFERENCES employee(emp_id)
);

SELECT e.name AS employee, COALESCE(m.name, 'None') AS manager
FROM   employee e
```

```
LEFT   JOIN employee m ON e.manager_id = m.emp_id
ORDER  BY manager, employee;
```

A `LEFT JOIN` is required so that the chief executive, whose `manager_id` is NULL, still appears; an inner join would silently drop that row.

Problem 6.8 — A designer proposes `(first_name, last_name, date_of_birth)` as the primary key of `patient`. Evaluate.

Solution. It is a poor choice. Uniqueness is not guaranteed — two patients can share a name and birth date. Names change (marriage, correction of spelling), and a changing primary key propagates through every referencing table. The key is wide, so every foreign key and index becomes large and slow. Personal data would be replicated across many tables, worsening privacy exposure. The correct design is a surrogate `patient_id` as primary key, with a unique constraint on a genuinely unique identifier such as a hospital registration number.

Review Questions

1. Define super key, candidate key, primary key and alternate key, with one example of each from a single relation.
2. Why is every candidate key a super key but not conversely?
3. State the properties the DBMS enforces for a primary key.
4. Explain the criteria used to choose a primary key among candidate keys.
5. Define a composite key and explain when it is necessary.
6. Compare surrogate keys and natural keys.
7. Define a foreign key and explain referential integrity.
8. Describe all four referential actions with an example of when each is appropriate.
9. Distinguish `PRIMARY KEY` from `UNIQUE`.
10. What is a self-referencing foreign key? Give an example.

Multiple Choice Questions

1. A minimal super key is a (a) foreign key (b) candidate key (c) composite key (d) surrogate key.
2. How many primary keys may a relation have? (a) Zero (b) Exactly one (c) One per attribute (d) Any number.
3. `{student_id, name}` where `student_id` is unique is a (a) candidate key (b) super key only (c) foreign key (d) alternate key.
4. Which may contain NULL? (a) Primary key (b) Candidate key chosen as primary (c) Foreign key (d) None.
5. If a relation of degree 5 has one single-attribute candidate key, the number of super keys containing it is (a) 5 (b) 16 (c) 31 (d) 32.
6. An auto-increment identifier with no business meaning is a (a) natural key (b) surrogate key (c) foreign key (d) super key.
7. Deleting a parent row and automatically deleting children uses (a) RESTRICT (b) CASCADE (c) SET NULL (d) NO ACTION.
8. Entity integrity is violated when (a) a foreign key is NULL (b) part of a primary key is NULL (c) a CHECK fails (d) a duplicate name exists.
9. A candidate key not chosen as primary key is called (a) super key (b) alternate key (c) composite key (d) foreign key.
10. In PostgreSQL, a foreign key column is (a) always indexed automatically (b) never allowed to be

NULL (c) not indexed automatically (d) always the primary key.

Answers: 1-(b) 2-(b) 3-(b) 4-(c) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(c)

Short Questions

1. Define a super key in one sentence.
2. Why must a primary key be stable?
3. Give an example of a composite primary key.
4. What is a natural key?
5. State the entity integrity rule.
6. What does `ON DELETE SET NULL` do?
7. Can a foreign key reference the same table?
8. Why should constraints be named?
9. Can a table have two candidate keys of different sizes?
10. Why is `email` usually a poor primary key?

Long / Descriptive Questions

1. Define all seven kinds of key with examples from one relation, and draw the containment diagram relating super, candidate, primary and alternate keys.
2. Explain foreign keys and referential integrity in detail, covering all four referential actions with realistic scenarios where each is correct.
3. Compare surrogate and natural keys, and defend the industry practice of using both.
4. Write and explain, line by line, a fully constrained `CREATE TABLE` statement using primary, unique, check, default and foreign key constraints.
5. Discuss the consequences of poor key selection, using examples of changing keys, wide keys and privacy-sensitive keys.

Practical Exercises

1. Create `department`, `student` and `enrollment` with appropriate primary, unique, check and foreign key constraints, all explicitly named.
2. Attempt four inserts, each violating a different constraint, and record the exact error message and the constraint name reported.
3. Demonstrate `ON UPDATE CASCADE` by renaming a department code and showing that student rows follow.
4. Add a surrogate key to `enrollment` and then show that duplicates become possible; add the natural-key `UNIQUE` constraint and show that they are blocked again.
5. Create a self-referencing `employee` table with at least three management levels, and write a query that lists every employee with their manager, including the top-level employee.

CHAPTER 7

Entity-Relationship Model

The Entity-Relationship model, introduced by Peter Chen in 1976, is the standard notation for *conceptual* database design. It lets a designer describe what the mini-world contains before deciding anything about tables, keys or storage. Chapter 9 then converts the ER diagram mechanically into relations.

7.1 Basic Concepts

Entity

An **entity** is a distinguishable object in the mini-world about which data is stored. A particular student, a particular course and a particular teacher are entities. Entities may be physical (a book, a classroom) or conceptual (a course, an enrollment, a loan).

Entity set

An **entity set** is a collection of all entities of the same type sharing the same attributes: all students form the entity set `STUDENT`. In diagrams, an entity set is drawn as a **rectangle**.

Attribute

An **attribute** is a property of an entity set, drawn as an **ellipse** connected to its rectangle. `STUDENT` has `student_id`, `name`, `date_of_birth`, `cgpa`.

Relationship and relationship set

A **relationship** is an association among entities; a **relationship set** is the collection of all relationships of the same type, drawn as a **diamond** connecting the participating entity sets.

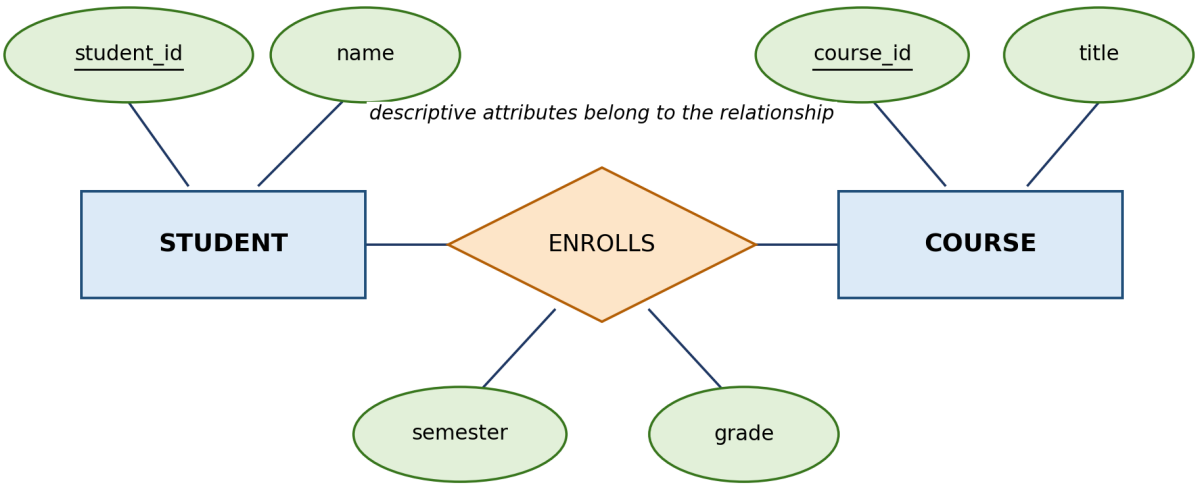


Figure 7.1: The basic ER notation

`ENROLLS` is a relationship set between `STUDENT` and `COURSE`. Its own attribute `semester` (a **descriptive attribute**) belongs to the relationship, not to either entity: a semester is not a property of a student nor of a course, but of the act of one student taking one course.

Cardinality ratio

The **cardinality ratio** states how many entities of one set may be associated with an entity of the other.

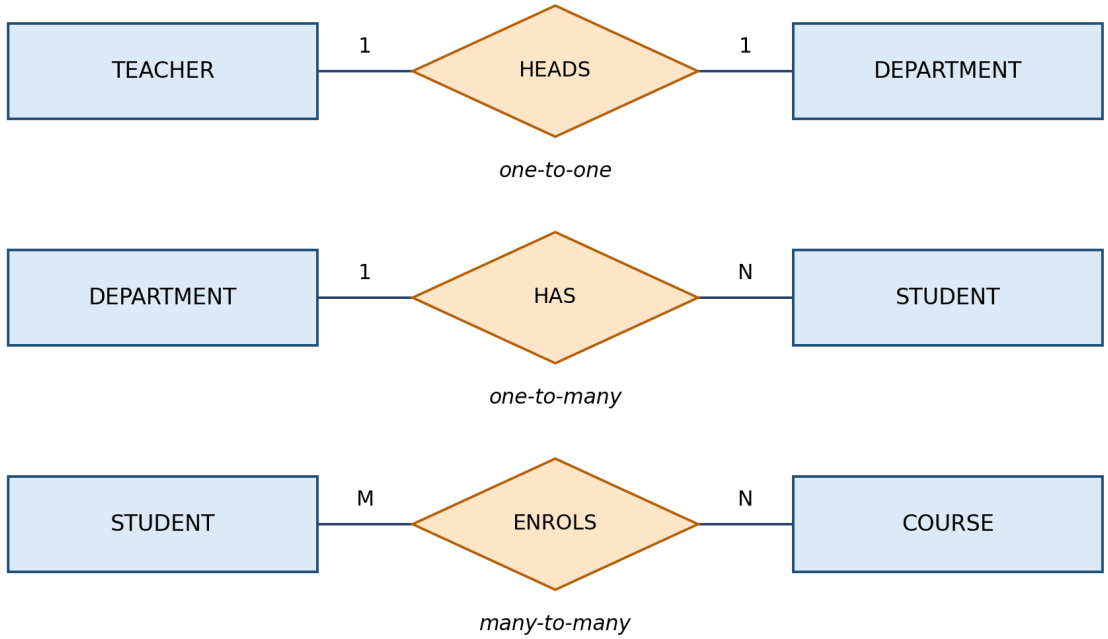


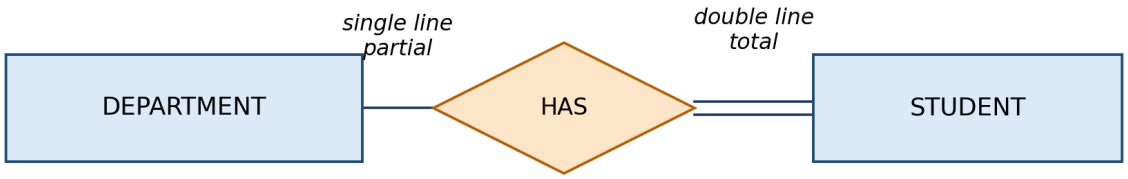
Figure 7.2: The three cardinality ratios

Ratio	Meaning	Example	Mapping to relations (Chapter 9)
1:1	One entity on each side	Teacher heads department	Foreign key in either table, `UNIQUE`
1:N	One on the left, many on the right	Department has students	Foreign key on the "many" side
M:N	Many on both sides	Students enrol in courses	Separate junction table

Participation constraint

Participation states whether *every* entity must take part in the relationship.

- **Total participation** (existence dependency): every entity of the set must participate. Drawn as a **double line**. Every student must belong to some department.
- **Partial participation**: some entities may not participate. Drawn as a single line. Not every teacher heads a department.



a department may exist with no students (partial); every student must belong to a department (total)

Figure 7.3: Total versus partial participation

The combination of cardinality and participation is written as `(min, max)` in some notations: `STUDENT (1,1) — DEPARTMENT` means each student participates at least once and at most once.

7.2 Types of Attributes

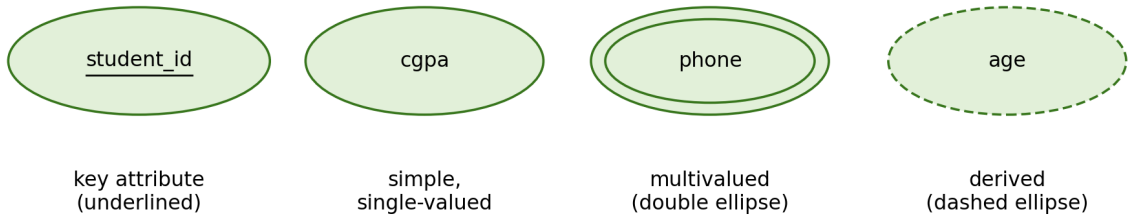


Figure 7.4: Attribute notation

Type	Definition	Example	ER symbol
Simple (atomic)	Cannot be divided further	`cgpa`, `age`	Single ellipse
Composite	Divisible into meaningful sub-parts	`name` → first, middle, last; `address` → street, city, postcode	Ellipse with attached sub-ellipses
Single-valued	Holds exactly one value per entity	`date_of_birth`	Single ellipse
Multivalued	Holds a set of values per entity	`phone`, `skills`	Double ellipse
Derived	Computed from other data, not stored	`age` from `date_of_birth`; `cgpa` from grades	Dashed ellipse
Stored	Physically stored	`date_of_birth`	Single ellipse
Key	Uniquely identifies the entity	`student_id`	Underlined
Complex	Composite and multivalued together	`{addresses: (street, city, postcode)}`	Combined notation

Each type maps to a specific relational construct, which is why the classification matters:

- Composite → one column per leaf sub-attribute (`first_name`, `last_name`), never one column hold-

ing the whole structure.

- Multivalued → a **separate table**, because a cell must be atomic.
- Derived → normally *not* stored; computed by a query or a generated column.

```sql
-- composite name -> separate columns
first_name VARCHAR(40), last_name VARCHAR(40),

-- multivalued phone -> separate table
CREATE TABLE student_phone (
    student_id CHAR(10),
    phone      VARCHAR(20),
    PRIMARY KEY (student_id, phone),
    FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE
);

-- derived age -> computed, not stored
SELECT student_id, TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) AS age FROM student;
```

Storing a derived value is a deliberate denormalization: it is faster to read and dangerous to maintain, because it becomes wrong the moment its inputs change.

7.3 Weak Entity

A **weak entity set** has no key attribute of its own; it is identified only in combination with its **owner** (identifying) entity set, to which it is connected by an **identifying relationship**. Its own attribute that distinguishes it among the owner's children is called a **partial key** or discriminator, drawn with a dashed underline.

Notation: weak entity set = double rectangle; identifying relationship = double diamond; participation is always total on the weak side.

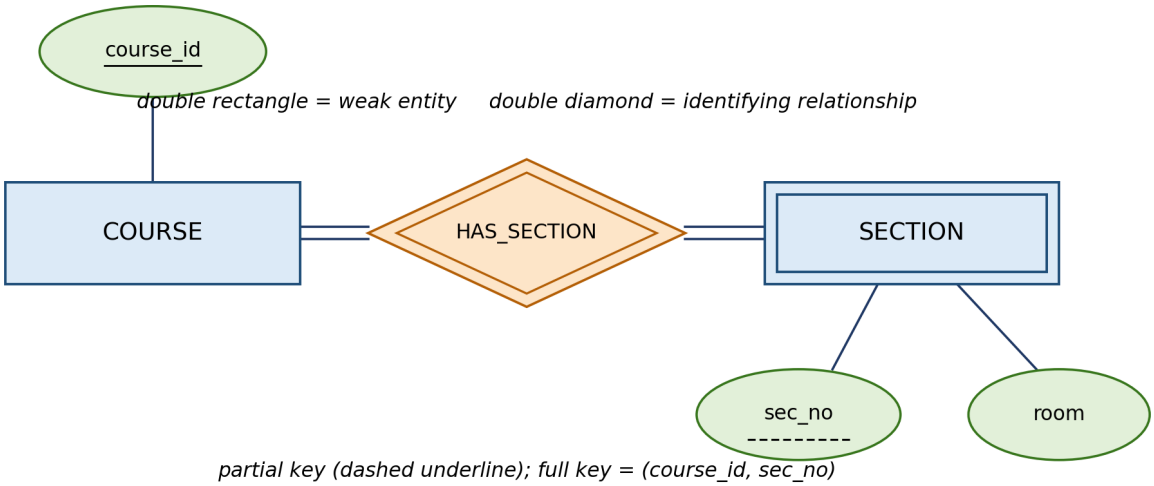


Figure 7.5: Weak entity — a course section

Real-life examples

1. **Course sections.** Section "A" of CSE 3201 and section "A" of EEE 3101 are different sections; `sec_no` alone is meaningless, so the key is `(course_id, sec_no)`.
2. **Order line items.** Line 1 exists in thousands of orders; the key is `(order_id, line_no)`.
3. **Dependents of an employee.** A dependant named "Rahim" is identified only with the employee:

key `(emp_id, dependent_name)`.

4. **Bank transactions in a passbook.** Transaction number 5 exists in every account; key `(account_no, txn_no)`.

The defining test in an examination: *if deleting the owner makes the child meaningless, and the child has no independent identifier, it is a weak entity.* In SQL this becomes a composite primary key whose first part is a foreign key declared `ON DELETE CASCADE`:

```sql
CREATE TABLE section (
    course_id INT,
    sec_no    CHAR(1),
    room      VARCHAR(20),
    PRIMARY KEY (course_id, sec_no),
    FOREIGN KEY (course_id) REFERENCES course(course_id) ON DELETE CASCADE
);
```

Basis	Strong entity	Weak entity
Key	Has its own key attribute	Has only a partial key
Identification	Independent	Depends on owner entity
Existence	Independent	Existence-dependent on owner
Notation	Single rectangle	Double rectangle
Relationship	Ordinary (single diamond)	Identifying (double diamond)
Participation	May be partial	Always total
Maps to	Table with own primary key	Table with composite key including owner's key

7.4 Degree of Relationship

The **degree** of a relationship set is the number of participating entity sets.

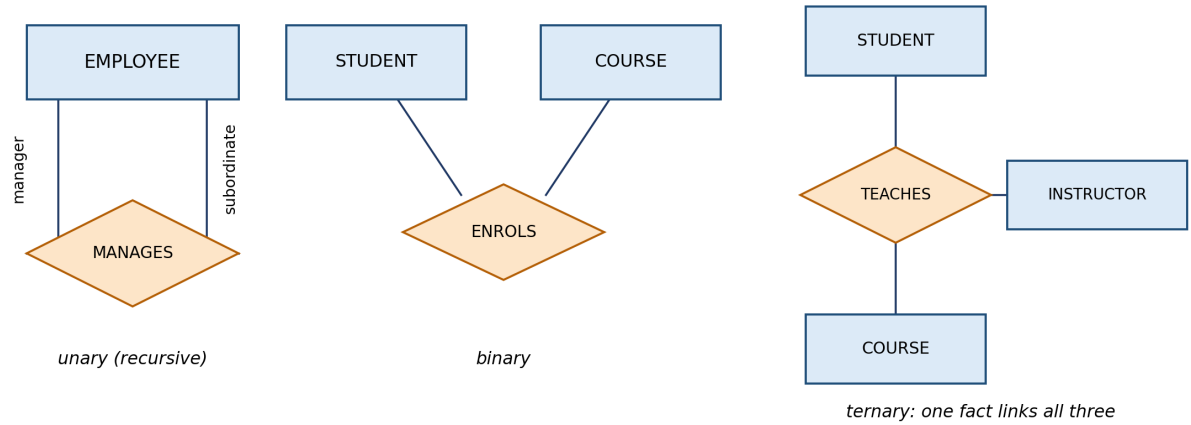


Figure 7.6: Unary, binary and ternary relationships

- **Unary (recursive).** One entity set relates to itself; the two ends carry **role names** (`manager`, `subordinate`) to distinguish them. Examples: employee manages employee, course is a prerequisite of course, person is married to person.
- **Binary.** Two entity sets. Nearly all real designs are built from binary relationships.
- **Ternary.** Three entity sets in one indivisible association. A ternary relationship is *not* the same as three binary relationships: the fact "instructor I taught course C to student S" cannot be recovered from the three separate facts "I teaches C", "S takes C" and "I teaches S". Use a ternary relationship only when the association genuinely needs all three at once.

- **n-ary.** Degree n; rare in practice and usually replaced by an entity set with n binary relationships.

7.5 A Complete ER Example

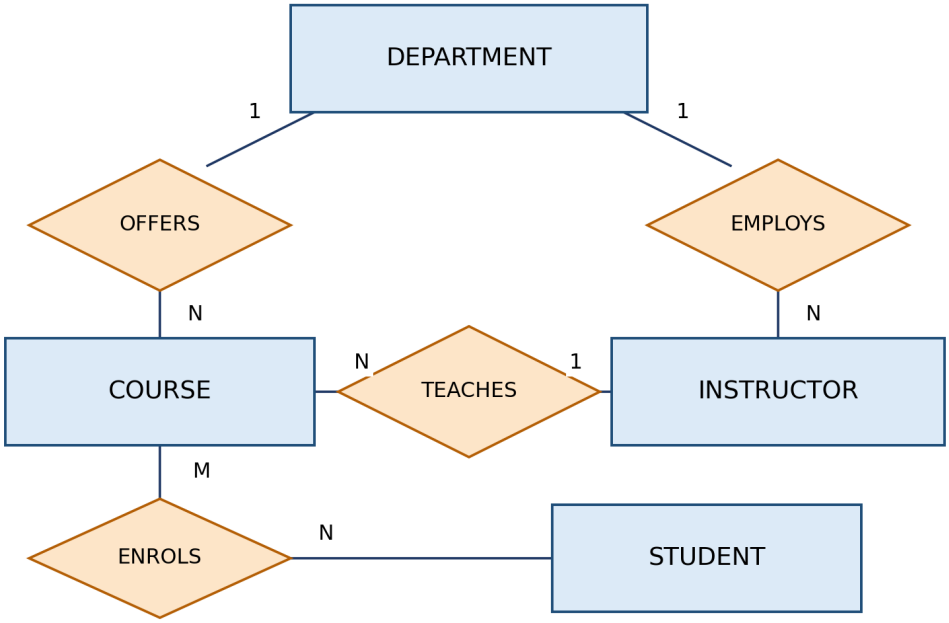


Figure 7.7: University ER diagram (simplified)

Business rules captured by this diagram:

1. A department offers many courses; each course belongs to exactly one department (1:N, total on the course side).
2. A department employs many instructors; each instructor belongs to one department (1:N).
3. A student enrols in many courses and a course enrols many students (M:N), with `semester` and `grade` as descriptive attributes of the relationship.
4. An instructor teaches many courses; in this simplified model each course is taught by one instructor (1:N).

Chapter Summary

- An entity is a distinguishable object; an entity set groups entities of the same type (rectangle); attributes are properties (ellipses); relationships associate entities (diamonds).
- Attributes may be simple or composite, single- or multivalued, stored or derived, and one or more are key attributes.
- Cardinality ratios are 1:1, 1:N and M:N; participation is total (double line) or partial (single line).
- A weak entity has no key of its own, uses a partial key, is identified through an identifying relationship with its owner, and participates totally.
- Relationship degree may be unary (recursive), binary, ternary or n-ary; a ternary relationship is not decomposable into binary ones without losing information.

Key Terms

Entity · Entity set · Strong entity · Weak entity · Owner entity · Identifying relationship · Partial key · Attribute · Simple · Composite · Single-valued · Multivalued · Derived · Key attribute · Relationship · Relationship set · Descriptive attribute · Cardinality ratio · 1:1 · 1:N · M:N · Participation · Total participation · Partial participation · Degree · Unary · Binary · Ternary · Role name

Common Mistakes

1. Modelling a multivalued attribute as a single column containing commas. It must become a separate table.
2. Drawing a relationship between an entity and an attribute. Relationships connect entity sets only.
3. Confusing cardinality (how many) with participation (whether it is compulsory). Both must be stated.
4. Making an entity out of something that is only an attribute — creating a `NAME` entity set, for instance.
5. Replacing a genuine ternary relationship with three binary ones and losing the combined fact.
6. Forgetting that a weak entity always participates totally in its identifying relationship.
7. Storing derived attributes without a plan for keeping them correct.
8. Omitting descriptive attributes of M:N relationships (`grade` belongs to `ENROLLS`, not to `STUDENT` or `COURSE`).

Solved Problems

Problem 7.1 — Identify entity sets, attributes, relationships, cardinalities and participation for: *"A library has many books. Each book has an ISBN, title and publisher. Members borrow books; a member may borrow many books and a book may be borrowed by many members over time. Each borrowing has an issue date and a due date."*

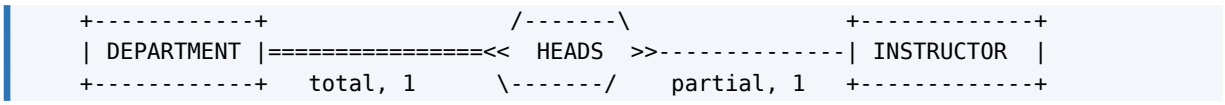
Solution. Entity sets: `BOOK(isbn, title, publisher)`, `MEMBER(member_id, name, phone)`. Relationship set: `BORROWS` between `MEMBER` and `BOOK`, cardinality M:N, with descriptive attributes `issue_date` and `due_date` (these belong to the act of borrowing, not to the book or the member). Participation: partial on both sides — a book may never have been borrowed and a member may never have borrowed anything. Note that if the library holds several physical copies of the same title, `BOOK` should be split into `TITLE` and `COPY`, with `COPY` a weak entity identified by `(isbn, copy_no)`.

Problem 7.2 — Classify each attribute of `EMPLOYEE(emp_id, name(first, last), date_of_birth, age, phone_numbers, salary)`.

Solution. `emp_id` — simple, single-valued, key. `name` — composite (first, last), single-valued. `date_of_birth` — simple, single-valued, stored. `age` — simple, single-valued, **derived** from `date_of_birth`. `phone_numbers` — simple but **multivalued**. `salary` — simple, single-valued. In the relational mapping: `name` becomes `first_name` and `last_name`; `age` is not stored but computed; `phone_numbers` becomes a table `employee_phone(emp_id, phone)`.

Problem 7.3 — Draw the ER fragment for "each department has exactly one head, who must be an instructor of that department; an instructor may head at most one department", and state cardinality and participation for both sides.

Solution.



Cardinality 1:1. Participation: **total** on DEPARTMENT (every department must have a head) and **partial** on INSTRUCTOR (most instructors head nothing). In the relational mapping the foreign key is placed in DEPARTMENT as head_id, declared NOT NULL UNIQUE — NOT NULL capturing total participation and UNIQUE capturing the 1:1 ratio.

Problem 7.4 — Explain why SECTION is a weak entity and give the resulting table.

Solution. A section is identified only as "section A of CSE 3201"; the discriminator sec_no repeats across courses, so SECTION has no key of its own. Deleting the course makes its sections meaningless, so existence is dependent. Hence SECTION is a weak entity, COURSE is its owner, HAS_SECTION is the identifying relationship, sec_no is the partial key, and the full key is (course_id, sec_no):

```
CREATE TABLE section (
    course_id INT,
    sec_no    CHAR(1),
    semester  VARCHAR(10) NOT NULL,
    room      VARCHAR(20),
    PRIMARY KEY (course_id, sec_no, semester),
    FOREIGN KEY (course_id) REFERENCES course(course_id) ON DELETE CASCADE
);
```

Problem 7.5 — Show with an example that a ternary relationship cannot always be replaced by three binary relationships.

Solution. Consider SUPPLIES(supplier, part, project) meaning "supplier S supplies part P to project J". Suppose the true facts are: S1 supplies bolts to J1, and S2 supplies nuts to J1, and S1 also supplies nuts to J2. Decomposing into SUPPLIES_PART(S,P), SUPPLIES_PROJECT(S,J) and USES(P, J) gives: (S1,bolts), (S2,nuts), (S1,nuts); (S1,J1), (S2,J1), (S1,J2); (bolts,J1), (nuts,J1), (nuts,J2). Recombining these now also implies the false fact "S1 supplies nuts to J1", which never happened. The three-way association is genuinely atomic and must be modelled as a ternary relationship — in relational terms, a table with a three-attribute key.

Problem 7.6 — For a hospital, model: patients, doctors, appointments with date and time, and prescriptions issued during an appointment.

Solution. Entity sets PATIENT(patient_id, name, dob), DOCTOR(doctor_id, name, specialization). APPOINTMENT is best modelled as an entity set rather than a relationship, because prescriptions must refer to it: APPOINTMENT(appt_id, appt_datetime) with 1:N relationships from both PATIENT and DOCTOR. PRESCRIPTION is a **weak entity** owned by APPOINTMENT, with partial key line_no and attributes medicine, dose, duration, giving key (appt_id, line_no). Participation: total for PRESCRIPTION in its identifying relationship; partial for patients and doctors, who may have no appointments yet.

Problem 7.7 — When should a relationship be promoted to an entity set?

Solution. Promote when (i) the relationship has many descriptive attributes of its own; (ii) other entity sets must refer to the relationship instance, as prescriptions refer to an appointment — a relationship cannot be referenced, only an entity can; (iii) the relationship itself has a life cycle and status (created, confirmed, cancelled); or (iv) the same pair may be related more than once over time, as with repeated borrowings of the same book by the same member, which requires an identifier for each occurrence.

Review Questions

1. Define entity, entity set, attribute, relationship and relationship set with a diagram.
2. Explain all attribute types with ER symbols and their relational mapping.
3. Describe the three cardinality ratios with examples.

4. Distinguish total from partial participation with a diagram.
5. What is a weak entity? Explain the identifying relationship and the partial key.
6. Compare strong and weak entities on at least six criteria.
7. Explain unary, binary and ternary relationships with examples.
8. Why can a ternary relationship not always be decomposed into binary ones?
9. What is a descriptive attribute? Give an example.
10. When should a relationship be promoted to an entity set?

Multiple Choice Questions

1. In an ER diagram, a relationship set is drawn as a (a) rectangle (b) ellipse (c) diamond (d) double rectangle.
2. A double ellipse denotes a (a) key attribute (b) derived attribute (c) multivalued attribute (d) composite attribute.
3. A weak entity is drawn as a (a) rectangle (b) double rectangle (c) diamond (d) dashed rectangle.
4. `age` computed from `date_of_birth` is a (a) stored (b) derived (c) composite (d) multivalued attribute.
5. Every student must belong to a department. This is (a) partial participation (b) total participation (c) cardinality (d) a weak entity.
6. The key of a weak entity is (a) its partial key alone (b) the owner's key alone (c) owner's key + partial key (d) system-generated only.
7. A relationship between an entity set and itself is (a) binary (b) unary (c) ternary (d) weak.
8. `grade` in `STUDENT ENROLLS COURSE` is an attribute of (a) STUDENT (b) COURSE (c) the relationship (d) neither.
9. An M:N relationship maps to (a) a foreign key on either side (b) a foreign key on the many side (c) a separate junction table (d) a view.
10. The ER model was proposed by (a) Codd (b) Chen (c) Bachman (d) Date.

Answers: 1-(c) 2-(c) 3-(b) 4-(b) 5-(b) 6-(c) 7-(b) 8-(c) 9-(c) 10-(b)

Short Questions

1. What is an entity set?
2. Give one example of a composite attribute.
3. How is a derived attribute drawn?
4. Define cardinality ratio.
5. What does a double line between an entity and a relationship mean?
6. What is a partial key?
7. Give one real example of a weak entity.
8. What is a role name in a recursive relationship?
9. How is a multivalued attribute mapped to tables?
10. What is the degree of a relationship?

Long / Descriptive Questions

1. Explain the complete ER notation with a university example, covering entities, all attribute types, relationships, cardinality and participation.
2. Explain weak entities in detail with two real-life examples and their relational mapping.

3. Discuss relationship degree, with special attention to why ternary relationships cannot generally be decomposed.
4. Design an ER diagram for a hospital management system and explain every design decision.
5. Discuss when a relationship should be modelled as an entity set, with examples.

Practical Exercises

1. Draw an ER diagram for a library system with titles, copies, members, borrowings and reservations. Mark cardinality and participation on every edge.
2. Identify three multivalued and two derived attributes in a hospital scenario and write the SQL tables or expressions that implement them.
3. Draw a recursive relationship for employee–manager and write the SQL table with a self-referencing foreign key.
4. Model a food-delivery system (restaurants, customers, orders, items, delivery agents). Identify at least one weak entity and justify it.
5. Take an existing database you have used and reverse-engineer an ER diagram from its tables, identifying which tables came from M:N relationships.

CHAPTER 8

Advanced ER (EER) Modeling

The basic ER model cannot express the statement *"students and teachers are both persons, and share name, address and date of birth"* without duplicating attributes. The **Enhanced (Extended) Entity-Relationship model** adds specialization, generalization, inheritance, constraints on those hierarchies, categories and aggregation.

8.1 Specialization

Specialization is the top-down process of defining subclasses of an entity set based on some distinguishing characteristic.

Starting from `PERSON`, we observe that some persons are students with a CGPA and an admission session, and some are teachers with a designation and a salary. We therefore define subclasses `STUDENT` and `TEACHER`.

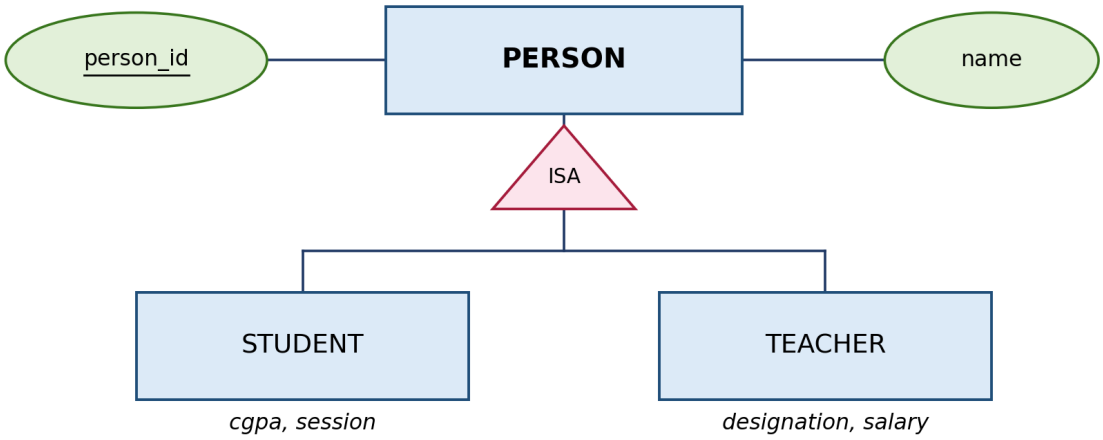


Figure 8.1: Specialization of PERSON

The triangle labelled **ISA** reads "is a": a student *is a* person.

8.2 Generalization

Generalization is the bottom-up inverse: several entity sets with common attributes are combined into a higher-level superclass.

Observing `CAR(reg_no, model, colour, seats)` and `TRUCK(reg_no, model, colour, load_capacity)`, a designer notices the shared attributes and generalizes them into `VEHICLE(reg_no, model, colour)`, leaving only the distinguishing attributes in the subclasses.

Basis	Specialization	Generalization
Direction	Top-down	Bottom-up
Starts from	One superclass	Several entity sets

Basis	Specialization	Generalization
Produces	Subclasses	A superclass
Motivation	Subgroups have distinct attributes or relationships	Several sets share attributes
Emphasis	Differences	Commonality

Both produce the same final structure; the difference is the direction of the designer's reasoning, and examiners expect that distinction stated explicitly.

8.3 Inheritance

A subclass **inherits** all attributes of its superclass and all relationships in which the superclass participates. `STUDENT` inherits `person_id`, `name`, `address` and `dob`, and `person_id` remains the identifier of a student. This is why a subclass never redefines the key: the key comes from the superclass.

Attribute inheritance keeps the design free of duplication. If `address` were repeated in `STUDENT` and `TEACHER`, a person who is both — a graduate student who teaches — would have two addresses that could disagree.

8.4 Constraints on Specialization

Two independent constraints apply to every specialization, and a complete answer must state both.

Disjointness constraint

- **Disjoint (d).** An entity may belong to **at most one** subclass. A vehicle is either a car or a truck, never both.
- **Overlapping (o).** An entity may belong to **several** subclasses simultaneously. A person in a university may be both a student and an employee.

Completeness constraint

- **Total specialization (double line).** Every superclass entity **must** belong to some subclass. Every vehicle must be a car, a truck or a bus.
- **Partial specialization (single line).** A superclass entity **may** belong to no subclass. A person may be neither a student nor a teacher — for example, a visitor.

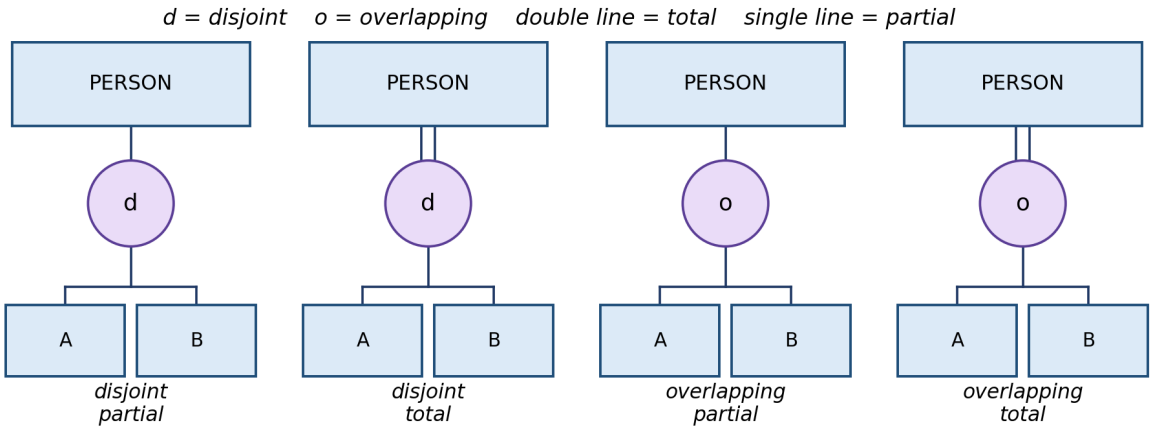


Figure 8.2: The four combinations of constraints

Combination	Meaning	Example
Disjoint + total	Exactly one subclass	`ACCOUNT` is exactly one of savings or current
Disjoint + partial	At most one subclass	`VEHICLE` may be a car, a truck, or unclassified
Overlapping + total	At least one subclass, possibly more	`EMPLOYEE` is at least one of engineer or manager
Overlapping + partial	Any number, including none	`PERSON` may be a student, an employee, both or neither

8.5 Specialization Hierarchy and Lattice

- A **specialization hierarchy** is a tree: every subclass has exactly one superclass (single inheritance).
- A **specialization lattice** allows a subclass with **more than one** superclass (multiple inheritance). `TEACHING_ASSISTANT` may be a subclass of both `STUDENT` and `EMPLOYEE`. Such a subclass is called a **shared subclass**.

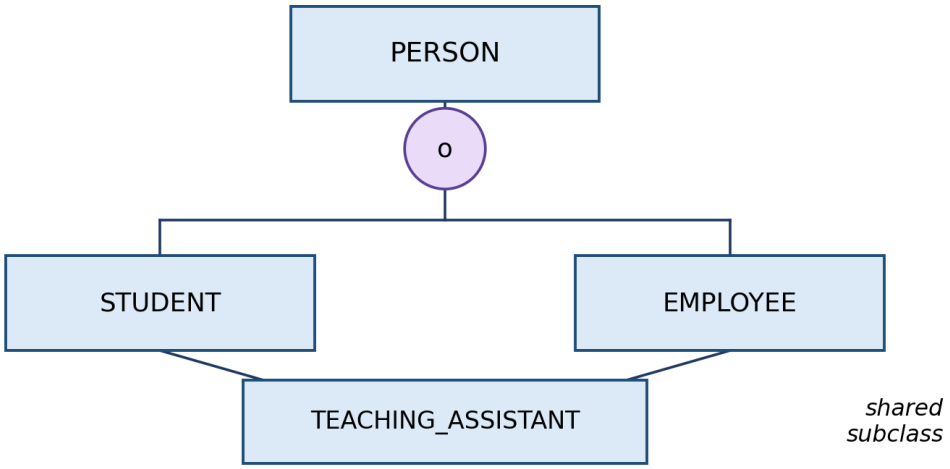


Figure 8.3: A specialization lattice with a shared subclass

8.6 Category (Union Type)

A **category** is a subclass whose members come from the **union** of several *distinct* superclasses. It is drawn with a circle containing a ∪.

The classic example: a vehicle's owner may be a `PERSON` or a `COMPANY`. `OWNER` is a category of the union of two different entity sets. This differs from a shared subclass, where a member belongs to *all* its superclasses at once; a category member belongs to *exactly one* of them.

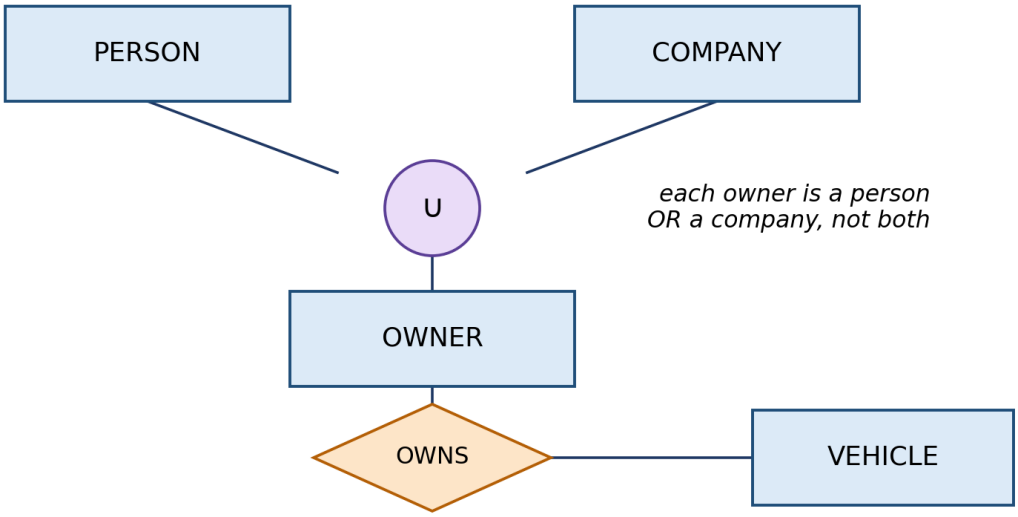


Figure 8.4: A category (union type)

Basis	Shared subclass	Category (union type)
Membership	Belongs to **all** superclasses	Belongs to **one** superclass
Inheritance	Inherits from all	Inherits selectively, from the one it belongs to
Symbol	Multiple ISA triangles	Circle with ∪
Example	Teaching assistant (student *and* employee)	Owner (person *or* company)

8.7 Aggregation

Aggregation treats an entire relationship set as if it were a higher-level entity, so that other relationships can be attached to it. It solves a problem the basic ER model cannot express: *a relationship cannot participate in another relationship.*

Consider: a student works on a project under a guide; and each such work assignment is evaluated by an external examiner. The evaluation refers not to the student, nor to the project, but to the combination.

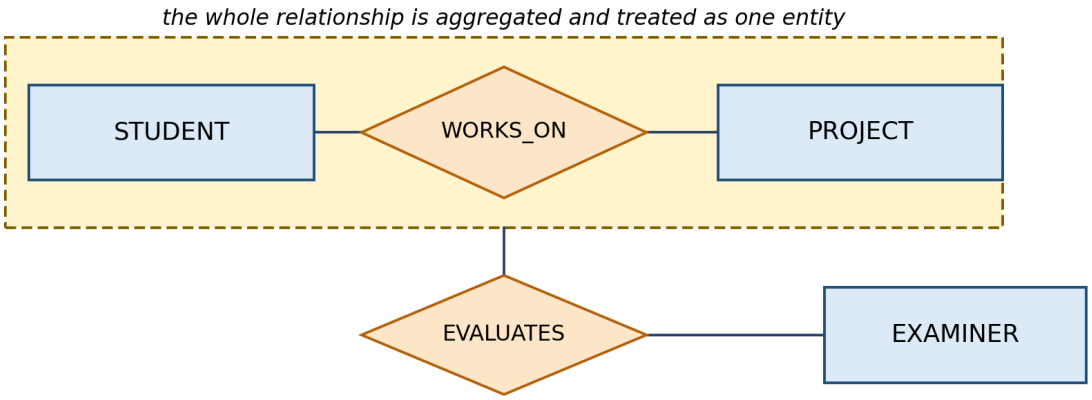


Figure 8.5: Aggregation

The dashed box encloses the relationship `WORKS_ON` and treats it as an entity that `EVALUATES` can relate to.

In the relational mapping, `WORKS_ON` becomes a table with its own key, and `EVALUATES` holds a foreign key to that table:

```sql
CREATE TABLE works_on (
    work_id    INT AUTO_INCREMENT PRIMARY KEY,
    student_id CHAR(10) NOT NULL,
    project_id INT      NOT NULL,
    start_date DATE     NOT NULL,
    UNIQUE (student_id, project_id),
    FOREIGN KEY (student_id) REFERENCES student(student_id),
    FOREIGN KEY (project_id) REFERENCES project(project_id)
);

CREATE TABLE evaluates (
    work_id     INT NOT NULL,
    examiner_id INT NOT NULL,
    eval_date   DATE NOT NULL,
    marks       DECIMAL(5,2) CHECK (marks BETWEEN 0 AND 100),
    PRIMARY KEY (work_id, examiner_id),
    FOREIGN KEY (work_id)     REFERENCES works_on(work_id) ON DELETE CASCADE,
    FOREIGN KEY (examiner_id) REFERENCES examiner(examiner_id)
);
```

8.8 Real-Life EER Example: A Bank

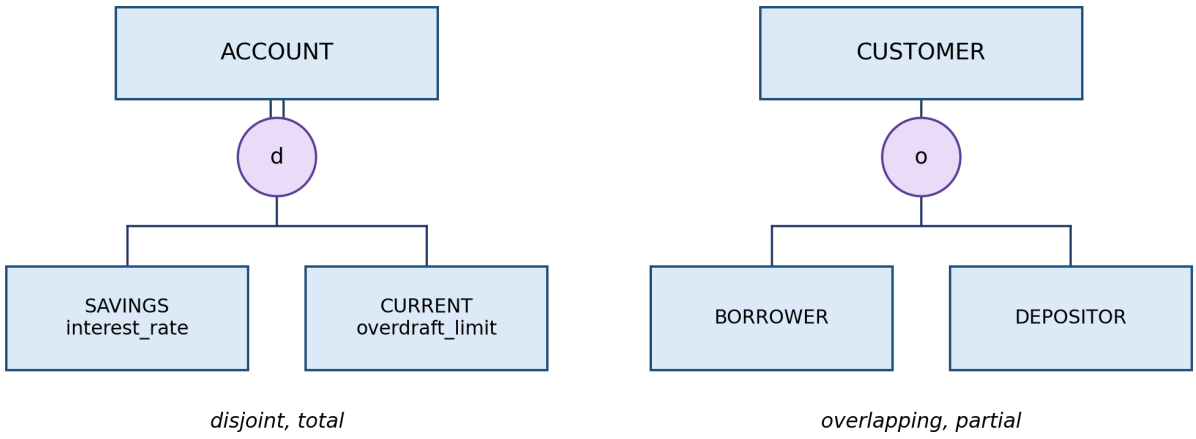


Figure 8.6: EER diagram for a bank

Reading the constraints: every account is **exactly one** of savings or current (disjoint, total). A customer may be a borrower, a depositor, both, or neither — a person who has only applied for a card is still a customer (overlapping, partial). These two lines of notation replace several paragraphs of prose in a requirements document, which is precisely why EER diagrams are worth drawing.

Chapter Summary

- Specialization is top-down subclassing; generalization is bottom-up superclassing; both yield an ISA hierarchy.
- A subclass inherits all attributes and relationships of its superclass, including the key.
- Two independent constraints apply: disjoint versus overlapping, and total versus partial, giving four

combinations.

- A specialization hierarchy allows one superclass per subclass; a lattice allows multiple inheritance through shared subclasses.
- A category (union type) has members drawn from exactly one of several distinct superclasses.
- Aggregation treats a relationship set as an entity so that another relationship may refer to it.

Key Terms

EER model · Superclass · Subclass · Specialization · Generalization · ISA relationship · Attribute inheritance · Relationship inheritance · Disjoint constraint · Overlapping constraint · Total specialization · Partial specialization · Specialization hierarchy · Specialization lattice · Shared subclass · Multiple inheritance · Category · Union type · Aggregation

Common Mistakes

1. Stating only one of the two constraints. Every specialization needs both a disjointness and a completeness constraint.
2. Confusing a shared subclass (belongs to *all* superclasses) with a category (belongs to *one*).
3. Giving a subclass its own primary key; the key is inherited from the superclass.
4. Using aggregation where a simple ternary relationship suffices, or the reverse.
5. Drawing generalization arrows upward and specialization arrows downward as if they were different constructs; the resulting diagram is identical, only the design reasoning differs.
6. Over-specializing: creating subclasses that have no distinct attributes or relationships, which only adds joins with no benefit.

Solved Problems

Problem 8.1 — State the disjointness and completeness constraints for: (a) `ACCOUNT` → savings, current; (b) `PERSON` → student, employee; (c) `VEHICLE` → car, truck, bus; (d) `EMPLOYEE` → permanent, contractual.

Solution. (a) Disjoint and total — an account is exactly one type. (b) Overlapping and partial — a person may be both (a working student) or neither (a visitor). (c) Disjoint and total if the organization owns only these three kinds; disjoint and partial if other vehicle types exist that are not subclassed. (d) Disjoint and total — an employee is exactly one of the two.

Problem 8.2 — Distinguish a shared subclass from a category with an example of each and their SQL representation.

Solution. A **shared subclass** belongs to all its superclasses simultaneously: `TEACHING_ASSISTANT` is both a `STUDENT` and an `EMPLOYEE`, so its table carries foreign keys to both, and its key is drawn from the common superclass `PERSON`. A **category** belongs to exactly one superclass: an `OWNER` is a `PERSON` or a `COMPANY`. In SQL a category is commonly implemented with a surrogate `owner_id`, an `owner_type` discriminator column constrained by `CHECK (owner_type IN ('PERSON','COMPANY'))`, and nullable foreign keys to each superclass with a check ensuring exactly one is non-NULL.

Problem 8.3 — A university has `PERSON`, specialized into `STUDENT` and `EMPLOYEE` (overlapping, partial), and `EMPLOYEE` further specialized into `TEACHER` and `STAFF` (disjoint, total). Draw the lattice and give the resulting tables.

Solution.

```
                PERSON
                  |
                / o \  partial
```

```
                -------
            |            |
         STUDENT       EMPLOYEE
                          |
                        / d \  total
                        =======
                    |            |
                 TEACHER       STAFF
```

```
CREATE TABLE person   (person_id INT PRIMARY KEY, name VARCHAR(100), dob DATE);
CREATE TABLE student  (person_id INT PRIMARY KEY, session VARCHAR(10), cgpa DECIMAL(3,2)
    ↪ ,
                       FOREIGN KEY (person_id) REFERENCES person(person_id));
CREATE TABLE employee (person_id INT PRIMARY KEY, join_date DATE, salary DECIMAL(10,2),
                       emp_type CHAR(1) NOT NULL CHECK (emp_type IN ('T','S')),
                       FOREIGN KEY (person_id) REFERENCES person(person_id));
CREATE TABLE teacher  (person_id INT PRIMARY KEY, designation VARCHAR(50),
                       FOREIGN KEY (person_id) REFERENCES employee(person_id));
CREATE TABLE staff    (person_id INT PRIMARY KEY, section VARCHAR(50),
                       FOREIGN KEY (person_id) REFERENCES employee(person_id));
```

Every subclass reuses `person_id` as its primary key *and* as a foreign key, which is exactly how attribute inheritance is expressed in the relational model.

Problem 8.4 — Why is aggregation needed for "an examiner evaluates a student's work on a project"?

Solution. Evaluation applies to the *pair* (student, project), not to a student alone or a project alone. The basic ER model forbids a relationship from participating in another relationship, so `EVALUATES` cannot be attached to `WORKS_ON` directly. Aggregation packages `WORKS_ON` as a higher-level entity that `EVALUATES` may then relate to. The alternative — a ternary relationship among student, project and examiner — is subtly different: it would allow an examiner to be recorded for a (student, project) pair that has no `WORKS_ON` instance at all, losing the guarantee that an evaluation always refers to real work.

Problem 8.5 — Convert this specialization to tables using all three standard mapping options and compare them: `VEHICLE(reg_no, model)` specialized into `CAR(seats)` and `TRUCK(load_capacity)`, disjoint and total.

Solution.

Option 1 — one table per entity (superclass + subclasses):

```
vehicle(reg_no PK, model)
car(reg_no PK/FK, seats)
truck(reg_no PK/FK, load_capacity)
```

Clean, no NULLs, requires a join to see all attributes.

Option 2 — one table per subclass only (valid because the specialization is total):

```
car(reg_no PK, model, seats)
truck(reg_no PK, model, load_capacity)
```

No joins, but `model` is duplicated in two tables, and a query over all vehicles needs a `UNION`. Only usable when the specialization is total and disjoint.

Option 3 — single table with a discriminator:

```
vehicle(reg_no PK, model, vehicle_type CHECK (vehicle_type IN ('CAR','TRUCK')),
        seats NULL, load_capacity NULL)
```

Fastest to query, but permits NULL-related anomalies (a truck with `seats` filled in) unless additional `CHECK` constraints enforce that the irrelevant columns are NULL.

Problem 8.6 — Give a real EER design for an e-commerce `USER` with overlapping subclasses `BUYER` and `SELLER`, and explain what the overlapping constraint permits.

Solution. `USER(user_id, name, email, phone)` specializes into `BUYER(default_address, loyalty_points)` and `SELLER(shop_name, trade_licence, rating)`, overlapping and partial. Overlapping permits a single account to buy and sell — the normal case on a marketplace — without duplicating the identity record. Partial permits a registered user who has neither bought nor listed anything. The relational mapping keeps `user` as the parent with `buyer` and `seller` as child tables keyed on `user_id`; a person who both buys and sells has one row in each child table and one identity in `user`.

Review Questions

1. Define specialization and generalization and distinguish them.
2. Explain attribute and relationship inheritance in an ISA hierarchy.
3. Describe the disjointness constraint with examples of both values.
4. Describe the completeness constraint with examples of both values.
5. Explain the four combinations of specialization constraints.
6. What is a specialization lattice? Give an example of a shared subclass.
7. Define a category and distinguish it from a shared subclass.
8. What is aggregation and why does the basic ER model require it?
9. Draw and explain an EER diagram for a bank.
10. Compare the three options for mapping a specialization to relations.

Multiple Choice Questions

1. Specialization is a (a) bottom-up (b) top-down (c) lateral (d) recursive process.
2. Generalization emphasizes (a) differences (b) similarities (c) keys (d) constraints.
3. A double line in a specialization denotes (a) disjoint (b) overlapping (c) total (d) partial specialization.
4. "An entity may belong to at most one subclass" is the (a) total (b) partial (c) disjoint (d) overlapping constraint.
5. A subclass with two superclasses is a (a) category (b) shared subclass (c) weak entity (d) aggregate.
6. A category represents the (a) intersection (b) union (c) difference (d) product of superclasses.
7. Treating a relationship set as an entity is (a) generalization (b) aggregation (c) specialization (d) categorization.
8. A subclass's primary key is (a) newly generated (b) inherited from the superclass (c) composite always (d) not required.
9. `PERSON` specialized into `STUDENT` and `EMPLOYEE` where both are possible is (a) disjoint (b) overlapping (c) total (d) a category.
10. Mapping a specialization into subclass tables only is valid when the specialization is (a) partial (b) total (c) overlapping (d) aggregated.

Answers: 1-(b) 2-(b) 3-(c) 4-(c) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What does the ISA triangle mean?

2. State one example of overlapping specialization.
3. What is a shared subclass?
4. Which symbol denotes a category?
5. Why does a subclass not define its own key?
6. Give one example requiring aggregation.
7. What is multiple inheritance in EER?
8. Distinguish total from partial specialization.
9. Name the three ways of mapping a specialization to tables.
10. Why is over-specialization undesirable?

Long / Descriptive Questions

1. Explain specialization, generalization and inheritance in detail with a university example and full notation.
2. Explain all four combinations of disjointness and completeness constraints with real examples and their SQL enforcement.
3. Distinguish specialization hierarchies from lattices, and shared subclasses from categories, with diagrams.
4. Explain aggregation with a complete example and its relational mapping, and compare it with a ternary relationship.
5. Design a complete EER diagram for a banking system covering accounts, customers, loans and employees, and justify every constraint.

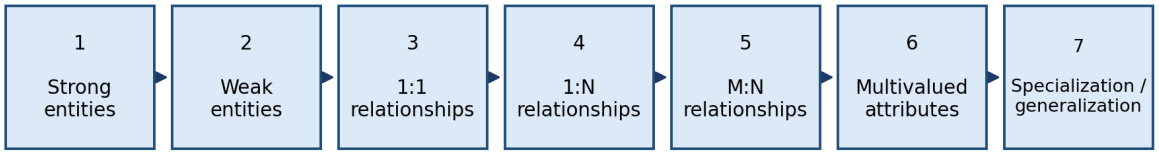
Practical Exercises

1. Draw an EER diagram for a university with `PERSON` specialized into `STUDENT`, `TEACHER` and `STAFF`, marking both constraints on every specialization.
2. Implement that hierarchy with all three mapping options and compare the SQL needed to answer "list the name and role of every person".
3. Model a vehicle registry with an `OWNER` category over `PERSON` and `COMPANY`, and implement it with a discriminator column and check constraints.
4. Model a shared subclass `TEACHING_ASSISTANT` and write a query listing every teaching assistant with both their student and employee attributes.
5. Model "an examiner evaluates a student's project work" using aggregation, create the tables, and insert data that demonstrates why a ternary relationship would permit an invalid evaluation.

CHAPTER 9

ER to Relational Mapping

An ER diagram is a design document; a relational schema is what a DBMS can actually store. The translation between them is not a matter of taste — it is a seven-step algorithm that produces the same tables for everyone who applies it correctly. This chapter presents that algorithm and works it through completely.



each ER construct is translated in a fixed order; later steps depend on keys created by earlier ones

Figure 9.1: The seven steps of ER-to-relational mapping

The order matters. Step 3 needs the primary keys created in step 1; step 5 needs the keys of both participating entity sets. Working out of order is the most common source of mapping errors in examinations.

9.1 Step 1 — Strong Entity Sets

Rule. For every strong (regular) entity set E, create a relation R that includes all of E's **simple** attributes. For a composite attribute, include only its leaf components. Choose one of the entity's key attributes as the primary key of R.

`STUDENT` with attributes `student_id` (key), `name` (composite: first, last), `date_of_birth`, `age` (derived) and `cgpa` becomes:

```
CREATE TABLE student (
    student_id    CHAR(10)     PRIMARY KEY,
    first_name    VARCHAR(40)  NOT NULL,      -- leaf of composite "name"
    last_name     VARCHAR(40)  NOT NULL,      -- leaf of composite "name"
    date_of_birth DATE         NOT NULL,
    cgpa          DECIMAL(3,2) DEFAULT 0.00
);
```

Two decisions are visible here. The composite `name` did not become one column; it became two, because a relational cell must be atomic. The derived `age` became no column at all; it is computed on demand from `date_of_birth`.

9.2 Step 2 — Weak Entity Sets

Rule. For every weak entity set W with owner E, create a relation R containing all simple attributes of W, plus the primary key of E as a foreign key. The primary key of R is the combination of E's primary key and W's partial key. The foreign key is normally declared `ON DELETE CASCADE`, because a weak entity cannot exist without its owner.

`SECTION`, weak, owned by `COURSE`, with partial key `sec_no`:

```
CREATE TABLE section (
    course_id INT,
    sec_no    CHAR(1),
    room      VARCHAR(20),
    PRIMARY KEY (course_id, sec_no),                    -- owner key + partial key
    FOREIGN KEY (course_id) REFERENCES course(course_id)
        ON DELETE CASCADE
);
```

9.3 Step 3 — Binary 1:1 Relationships

Three options exist; the correct choice depends on participation.

Option A — foreign key approach (preferred). Place the primary key of one side into the other side's relation as a foreign key, declared `UNIQUE`. If one side has **total** participation, put the foreign key **there** and declare it `NOT NULL`, because then it is never NULL.

For "each department has exactly one head, who is an instructor" (total on `DEPARTMENT`, partial on `INSTRUCTOR`):

```
ALTER TABLE department
    ADD head_id INT NOT NULL UNIQUE,                    -- NOT NULL = total
    ↪ participation
    ADD CONSTRAINT fk_dept_head FOREIGN KEY (head_id)
        REFERENCES instructor(instructor_id);
```

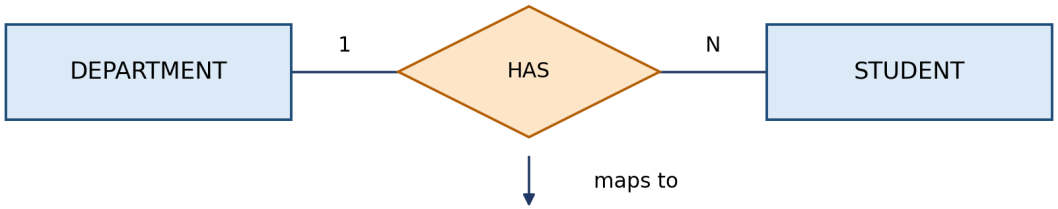
`UNIQUE` is what enforces the 1:1 ratio; without it the design silently becomes 1:N.

Option B — merged relation. If *both* sides have total participation and are never accessed independently, merge them into one relation. Rarely appropriate.

Option C — cross-reference relation. Create a separate relation holding both keys, with one of them declared `UNIQUE`. Useful when both sides are partial and the relationship is sparse, since it avoids a column that is mostly NULL.

9.4 Step 4 — Binary 1:N Relationships

Rule. Place the primary key of the "one" side into the relation of the "many" side as a foreign key. Any descriptive attributes of the relationship also go on the many side. Never create a separate table for a plain 1:N relationship.



dept_id	dept_name
CSE	Computer Sc.
EEE	Electrical

student_id	name	dept_id
ASH1901001	Rahim	CSE
ASH1901002	Karim	EEE

the foreign key always goes on the MANY side

Figure 9.2: Mapping a 1:N relationship — the foreign key goes on the many side

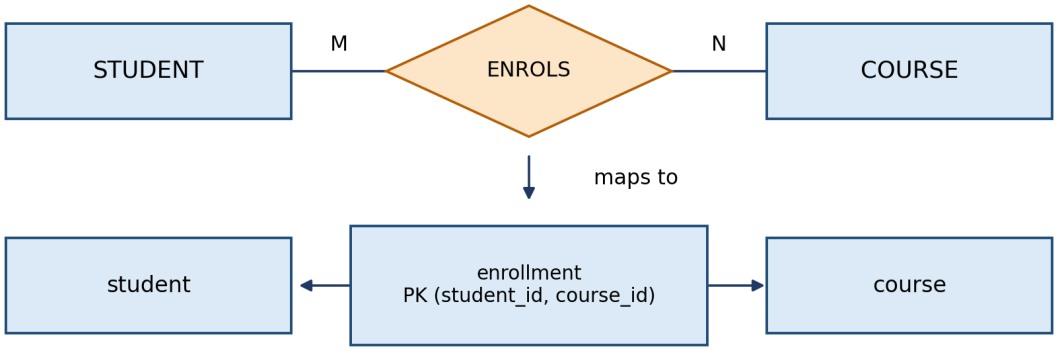
```sql
CREATE TABLE student (
    student_id CHAR(10) PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    dept_id    CHAR(3) NOT NULL,                -- NOT NULL because participation is
    ↪ total
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

If participation on the many side were partial — a student who has not yet been assigned a department — the column would be nullable.

The reason the foreign key cannot go on the "one" side is decisive: a department has many students, so `department.student_id` would have to hold many values in one cell, violating atomicity.

9.5 Step 5 — Binary M:N Relationships

Rule. Create a new relation R — a **junction**, **bridge** or **associative** table — containing the primary keys of both entity sets as foreign keys, plus every descriptive attribute of the relationship. The primary key of R is the combination of the two foreign keys (extended with a discriminator such as `semester` if the same pair may recur).



a third (junction) table whose key is the pair of foreign keys

Figure 9.3: Mapping an M:N relationship to a junction table

```sql
CREATE TABLE enrollment (
    student_id CHAR(10),
    course_id  INT,
    semester   VARCHAR(10),
    grade      CHAR(2),                          -- descriptive attribute of the
    ↪ relationship
    PRIMARY KEY (student_id, course_id, semester),
    FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE,
    FOREIGN KEY (course_id)  REFERENCES course(course_id)  ON DELETE RESTRICT
);
```

An M:N relationship **always** produces a third table. There is no alternative: putting a foreign key on either side would require multiple values in one cell.

9.6 Step 6 — Multivalued Attributes

Rule. For each multivalued attribute A of entity set E, create a relation R containing A and the primary key of E as a foreign key. The primary key of R is the combination of both (unless A is composite, in which case all its leaves participate).

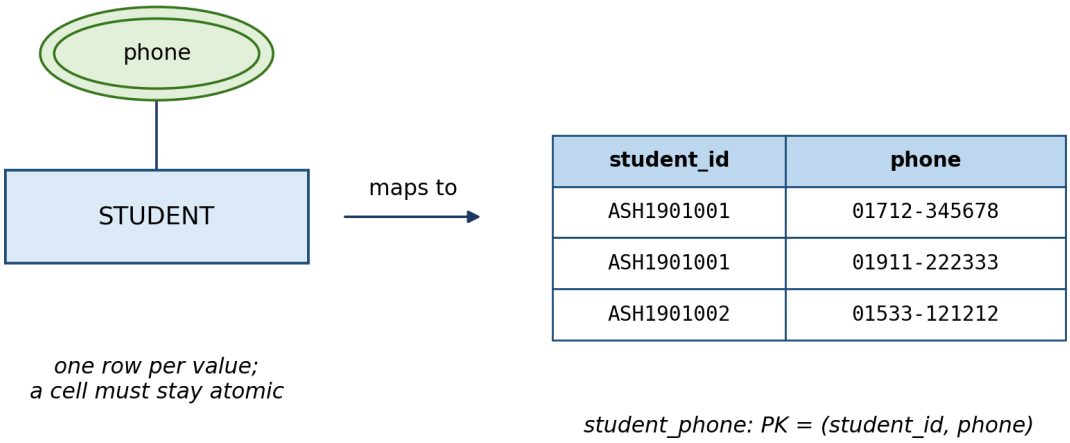


Figure 9.4: Mapping a multivalued attribute to its own relation

```sql
CREATE TABLE student_phone (
    student_id CHAR(10),
    phone      VARCHAR(20),
    PRIMARY KEY (student_id, phone),
    FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE
);
```

Two wrong alternatives are worth naming. Storing `'01712-345678, 01911-222333'` in one column breaks atomicity and makes searching for a number impossible without string manipulation. Creating columns `phone1`, `phone2`, `phone3` fixes an arbitrary maximum and wastes space; adding a fourth number then requires a schema change.

9.7 Step 7 — Specialization and Generalization

Four standard options exist, and choosing among them is a genuine design decision.

Option	Structure	Best when
A. Superclass + subclass tables	One table for the superclass, one per subclass, each keyed on the superclass key	General case; any constraint combination
B. Subclass tables only	One table per subclass, superclass attributes duplicated in each	Specialization is total **and** disjoint
C. Single table with a type column	One table with all attributes and a discriminator	Disjoint, few subclass-specific attributes
D. Single table with multiple boolean flags	One table, one flag per subclass	Overlapping specialization with few attributes

```sql
-- Option A, the general solution
CREATE TABLE person   (person_id INT PRIMARY KEY, name VARCHAR(100), dob DATE);
CREATE TABLE student  (person_id INT PRIMARY KEY, session VARCHAR(10), cgpa DECIMAL(3,2)
    ↪ ,
                       FOREIGN KEY (person_id) REFERENCES person(person_id) ON DELETE
    ↪ CASCADE);
CREATE TABLE teacher  (person_id INT PRIMARY KEY, designation VARCHAR(50), salary
    ↪ DECIMAL(10,2),
                       FOREIGN KEY (person_id) REFERENCES person(person_id) ON DELETE
    ↪ CASCADE);

-- Option C, single table with discriminator
CREATE TABLE person_single (
    person_id   INT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    person_type CHAR(1) NOT NULL CHECK (person_type IN ('S','T')),
    cgpa        DECIMAL(3,2) NULL,
    designation VARCHAR(50)  NULL,
    CHECK ((person_type = 'S' AND designation IS NULL)
        OR (person_type = 'T' AND cgpa IS NULL))         -- keeps the NULLs honest
);
```

Option B is only valid for a total specialization, because an entity belonging to no subclass would have nowhere to live. Option C is fast — no joins — but accumulates nullable columns as subclasses multiply.

9.8 Mapping Higher-Degree Relationships and Aggregation

n-ary relationship. Create a relation containing the primary keys of all participating entity sets plus descriptive attributes; the primary key is generally the combination of all of them (or of all but the "one" side, if one participant has cardinality 1).

```sql
CREATE TABLE teaches (
    instructor_id INT,
    course_id     INT,
    semester      VARCHAR(10),
    PRIMARY KEY (instructor_id, course_id, semester),
    FOREIGN KEY (instructor_id) REFERENCES instructor(instructor_id),
    FOREIGN KEY (course_id)     REFERENCES course(course_id)
);
```

Aggregation. The aggregated relationship becomes a table with its own surrogate key; the outer relationship then references that key. This is the mapping already shown in Section 8.7.

9.9 Complete Worked Example

Requirements: *A department has an id, a name and a building. It employs instructors and enrols students. An instructor has an id, a name, a set of phone numbers, and a designation; exactly one instructor heads each department. A course has an id, code, title and credit, and belongs to one department. Students enrol in courses with a semester and a grade.*

Step 1 — strong entities:

```
department(dept_id, dept_name, building)
instructor(instructor_id, name, designation, dept_id)
student(student_id, name, cgpa, dept_id)
course(course_id, course_code, title, credit, dept_id)
```

Step 2 — weak entities: none in this requirement.

Step 3 — 1:1 (department–head): `department.head_id` as `UNIQUE NOT NULL` foreign key to `instructor`.

Step 4 — 1:N: `instructor.dept_id`, `student.dept_id`, `course.dept_id` — all foreign keys on the many side, already placed in step 1.

Step 5 — M:N (student–course): junction table `enrollment(student_id, course_id, semester, grade)`.

Step 6 — multivalued (`phone`): `instructor_phone(instructor_id, phone)`.

Step 7 — specialization: none here.

Final schema, with keys underlined in examination notation:

```
department(dept_id, dept_name, building, head_id)
instructor(instructor_id, name, designation, dept_id)
instructor_phone(instructor_id, phone)
student(student_id, name, cgpa, dept_id)
course(course_id, course_code, title, credit, dept_id)
enrollment(student_id, course_id, semester, grade)
```

There is one ordering problem left, and it is a real one: `department.head_id` references `instructor`, while `instructor.dept_id` references `department`. Neither table can be created with both constraints in place first. The standard solution is to create both tables, then add one foreign key afterwards:

```
CREATE TABLE department (dept_id CHAR(3) PRIMARY KEY, dept_name VARCHAR(80), building
    ↪ VARCHAR(40));
CREATE TABLE instructor (instructor_id INT PRIMARY KEY, name VARCHAR(100), designation
    ↪ VARCHAR(50),
                         dept_id CHAR(3) NOT NULL,
                         FOREIGN KEY (dept_id) REFERENCES department(dept_id));
ALTER TABLE department ADD head_id INT UNIQUE,
      ADD CONSTRAINT fk_dept_head FOREIGN KEY (head_id) REFERENCES instructor(
    ↪ instructor_id);
```

Chapter Summary

- Strong entities become relations with their simple attributes and their own primary key; composite attributes contribute their leaves; derived attributes contribute nothing.
- Weak entities become relations keyed on (owner key + partial key) with a cascading foreign key.
- 1:1 relationships use a `UNIQUE` foreign key, placed on the total-participation side if there is one.

- 1:N relationships place the foreign key on the many side.
- M:N relationships always require a junction table keyed on both foreign keys.
- Multivalued attributes always require their own relation.
- Specializations can be mapped four ways; the choice depends on the disjointness and completeness constraints.
- Mutually referencing foreign keys are added with `ALTER TABLE` after both tables exist.

Key Terms

Mapping algorithm · Strong entity mapping · Weak entity mapping · Partial key · Junction table · Bridge table · Associative entity · Cross-reference relation · Discriminator column · Surrogate key for aggregation · Circular foreign key reference

Common Mistakes

1. Creating a separate table for a 1:N relationship. It is unnecessary and adds a join to every query.
2. Omitting `UNIQUE` on the foreign key of a 1:1 relationship, which turns it into 1:N.
3. Placing the foreign key of a 1:N relationship on the "one" side.
4. Storing a multivalued attribute as a comma-separated string or as `phone1`, `phone2`, `phone3`.
5. Forgetting the descriptive attributes of an M:N relationship when creating the junction table.
6. Using a subclass-only mapping for a partial specialization, leaving unclassified entities homeless.
7. Not marking a foreign key `NOT NULL` when the ER diagram showed total participation.
8. Trying to create two mutually referencing tables in one pass instead of using `ALTER TABLE`.

Solved Problems

Problem 9.1 — Map: `EMPLOYEE(emp_id, name, skills{multivalued}, address(city, street))` with `DEPENDENT` as a weak entity (partial key `dep_name`).

Solution.

```sql
CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    name   VARCHAR(100) NOT NULL,
    city   VARCHAR(50),                    -- leaf of composite address
    street VARCHAR(100)                    -- leaf of composite address
);
CREATE TABLE employee_skill (
    emp_id INT, skill VARCHAR(50),
    PRIMARY KEY (emp_id, skill),
    FOREIGN KEY (emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE
);
CREATE TABLE dependent (
    emp_id   INT,
    dep_name VARCHAR(60),
    relation VARCHAR(30),
    dob      DATE,
    PRIMARY KEY (emp_id, dep_name),
    FOREIGN KEY (emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE
);
```

Problem 9.2 — A doctor treats many patients; a patient is treated by many doctors; each treatment has a date and a diagnosis. Map it and explain why a junction table is unavoidable.

Solution.

```
CREATE TABLE treatment (
    doctor_id  INT,
    patient_id INT,
    treat_date DATE,
    diagnosis  VARCHAR(200),
    PRIMARY KEY (doctor_id, patient_id, treat_date),
    FOREIGN KEY (doctor_id)  REFERENCES doctor(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES patient(patient_id)
);
```

A foreign key on `doctor` would need to hold every patient that doctor has treated, and a foreign key on `patient` would need every treating doctor — both require multiple values in one cell, which the relational model forbids. Furthermore `diagnosis` belongs to neither entity; it is a property of the association, and only a junction table has a place for it. `treat_date` is included in the key because the same doctor may treat the same patient on several occasions.

Problem 9.3 — Map a 1:1 relationship "each employee is assigned at most one laptop; each laptop is assigned to at most one employee" and justify your placement of the foreign key.

Solution. Both sides are partial, so either placement leaves NULLs. Since laptops are fewer than employees and most employees may have none, put the foreign key on `laptop`:

```
ALTER TABLE laptop
    ADD assigned_to INT NULL UNIQUE,
    ADD CONSTRAINT fk_laptop_emp FOREIGN KEY (assigned_to) REFERENCES employee(emp_id);
```

`UNIQUE` enforces "at most one employee per laptop"; being a single column on `laptop` enforces "at most one laptop per employee". If the relationship were sparse in both directions, option C — a separate `assignment(emp_id UNIQUE, laptop_id UNIQUE)` table — would avoid NULLs entirely.

Problem 9.4 — A ternary relationship `SUPPLIES(supplier, part, project, quantity)` exists. Map it and state the primary key.

Solution.

```
CREATE TABLE supplies (
    supplier_id INT, part_id INT, project_id INT,
    quantity    INT NOT NULL CHECK (quantity > 0),
    PRIMARY KEY (supplier_id, part_id, project_id),
    FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id),
    FOREIGN KEY (part_id)     REFERENCES part(part_id),
    FOREIGN KEY (project_id)  REFERENCES project(project_id)
);
```

The key is all three foreign keys, because the fact being recorded is the three-way combination. If the requirement were "each project obtains a given part from only one supplier", the key would shrink to `(part_id, project_id)` and `supplier_id` would become a non-key attribute.

Problem 9.5 — For `ACCOUNT` specialized into `SAVINGS` and `CURRENT` (disjoint, total), give mappings under options A, B and C, and recommend one.

Solution. **A:** `account(acc_no, balance, open_date)`, `savings(acc_no, interest_rate)`, `current(acc_no, overdraft_limit)`. **B:** `savings(acc_no, balance, open_date, interest_rate)` and `current(acc_no, balance, open_date, overdraft_limit)`; valid because the specialization is total, but "list all accounts" now needs a `UNION`, and nothing prevents the same `acc_no` appearing in both tables unless extra checks are added. **C:** one `account` table with `acc_type CHAR(1)`

`CHECK (acc_type IN ('S','C'))` plus nullable `interest_rate` and `overdraft_limit`. Recommendation: **A** for a bank, because account-level operations (balance enquiry, transfer, audit) are far more frequent than subtype-specific ones, and option A keeps referential integrity simple — other tables such as `transaction` can reference a single `account` table.

Problem 9.6 — Explain why `department.head_id` and `instructor.dept_id` cannot both be created in a single `CREATE TABLE` pass, and give two solutions.

Solution. Each constraint requires its target table to exist already, so whichever table is created first has a foreign key to a table that does not yet exist. Solution 1: create both tables without the circular constraint and add it afterwards with `ALTER TABLE ... ADD CONSTRAINT`. Solution 2 (MySQL): temporarily `SET FOREIGN_KEY_CHECKS = 0`, create both, then re-enable — acceptable for scripted loads but less explicit. Note also that inserting data has the same chicken-and-egg problem: insert the department with a NULL head, insert the instructor, then update the head.

Review Questions

1. State the seven steps of ER-to-relational mapping in order and explain why the order matters.
2. How are composite and derived attributes handled when mapping a strong entity?
3. Describe the mapping rule for weak entities and give an example.
4. Explain all three options for mapping a 1:1 relationship and when each is appropriate.
5. Why must the foreign key of a 1:N relationship go on the many side?
6. Why does an M:N relationship always need a third table?
7. How is a multivalued attribute mapped? Give two incorrect alternatives and say why they fail.
8. Describe the four options for mapping a specialization and their applicability.
9. How is a ternary relationship mapped, and what is its primary key?
10. How are circular foreign key references created?

Multiple Choice Questions

1. A multivalued attribute maps to (a) a column (b) several columns (c) a separate relation (d) a view.
2. In a 1:N relationship the foreign key is placed on the (a) one side (b) many side (c) either side (d) a new table.
3. The primary key of a weak entity's relation is (a) the partial key (b) the owner's key (c) owner's key + partial key (d) a surrogate.
4. An M:N relationship requires (a) no extra table (b) a junction table (c) a UNIQUE foreign key (d) a view.
5. A derived attribute maps to (a) a stored column (b) no column normally (c) a separate table (d) an index.
6. To enforce 1:1 with a foreign key you must add (a) NOT NULL (b) UNIQUE (c) CHECK (d) CASCADE.
7. Mapping a specialization into subclass tables only is valid when the specialization is (a) partial (b) total (c) overlapping (d) aggregated.
8. A composite attribute `address(city, street)` maps to (a) one column (b) two columns (c) a separate table always (d) a view.
9. Total participation of a 1:N relationship is enforced by (a) UNIQUE (b) NOT NULL on the foreign key (c) CASCADE (d) an index.
10. Aggregation is mapped by giving the aggregated relationship (a) no table (b) its own table with a key (c) a view (d) a trigger.

Answers: 1-(c) 2-(b) 3-(c) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What happens to a derived attribute during mapping?
2. What is a junction table?
3. Why is `ON DELETE CASCADE` typical for weak entities?
4. Which constraint turns a 1:N foreign key into 1:1?
5. What is the key of a multivalued attribute's table?
6. Name the four options for mapping specialization.
7. Why can a foreign key not hold multiple values?
8. What is a cross-reference relation?
9. When is a discriminator column needed?
10. How do you break a circular foreign key dependency?

Long / Descriptive Questions

1. Explain the complete ER-to-relational mapping algorithm with an example for every step.
2. Discuss all approaches to mapping 1:1 relationships and specializations, with recommendations.
3. Map a complete hospital ER diagram to a relational schema, showing every step.
4. Explain why M:N relationships and multivalued attributes both require separate relations, relating your answer to the atomicity requirement.
5. Given an EER diagram with a shared subclass and an aggregation, produce the full relational schema with all constraints.

Practical Exercises

1. Draw an ER diagram for a library and map it completely to SQL `CREATE TABLE` statements.
2. Implement the same 1:1 relationship using all three mapping options and compare the SQL needed to query it.
3. Map a specialization using options A and C, load the same data into both, and compare query complexity and storage.
4. Create two tables with circular foreign keys using `ALTER TABLE`, then insert a consistent pair of rows.
5. Take an existing database and identify which of its tables originated from M:N relationships and which from multivalued attributes.

CHAPTER 10

SQL Fundamentals

SQL (Structured Query Language) is the standard language for defining, manipulating, querying and controlling relational data. It was developed at IBM in the 1970s as SEQUEL, standardized by ANSI in 1986 and by ISO in 1987, and revised roughly every few years since — SQL-92, SQL:1999 (triggers, recursion), SQL:2003 (window functions, XML), SQL:2011 (temporal data) and SQL:2016 (JSON).

Two properties define the language. It is **declarative**: a statement describes the desired result, not the procedure for obtaining it. And it is **set-oriented**: statements operate on entire sets of rows at once, not one row at a time.

10.1 SQL Syntax Rules

- Keywords are **case-insensitive** (`SELECT` = `select`), but this book writes them in upper case for readability. Identifiers may or may not be case-sensitive depending on the system and operating system: MySQL table names are case-sensitive on Linux, insensitive on Windows; PostgreSQL folds unquoted identifiers to lower case.
- Statements end with a **semicolon**.
- Strings are enclosed in **single quotes**: `'Rahim'`. Double quotes denote identifiers in standard SQL and PostgreSQL; MySQL uses backticks.
- Comments: `-- to end of line`, or `/* block */`.
- White space and line breaks are free.
- `NULL` is a marker, not a value; test it with `IS NULL`, never with `= NULL`.

10.2 The Five Command Categories

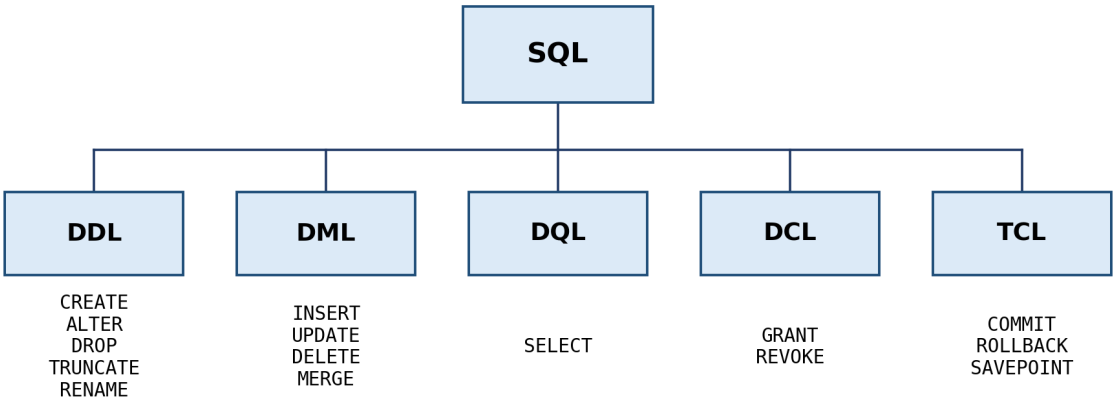


Figure 10.1: The five categories of SQL commands

Category	Full name	Purpose	Commands	Auto-commit?
DDL	Data Definition Language	Define and modify structure	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`	Yes (implicit commit)
DML	Data Manipulation Language	Change data	`INSERT`, `UPDATE`, `DELETE`, `MERGE`	No

Category	Full name	Purpose	Commands	Auto-commit?
DQL	Data Query Language	Retrieve data	`SELECT`	Not applicable
DCL	Data Control Language	Control access	`GRANT`, `REVOKE`	Yes
TCL	Transaction Control Language	Manage transactions	`COMMIT`, `ROLLBACK`, `SAVEPOINT`, `SET TRANSACTION`	—

The auto-commit column is worth memorizing, because it explains a behaviour that surprises every student at least once: a `DROP TABLE` issued by mistake cannot be undone with `ROLLBACK`, whereas a `DELETE` usually can.

10.3 DDL — Data Definition Language

CREATE

```sql
CREATE DATABASE university_db;
USE university_db;                                  -- MySQL;  PostgreSQL: \c university_db

CREATE TABLE department (
    dept_id   CHAR(3)     PRIMARY KEY,
    dept_name VARCHAR(80) NOT NULL UNIQUE,
    building  VARCHAR(40),
    founded   DATE        DEFAULT (CURRENT_DATE)
);
```

Common data types

Category	MySQL	PostgreSQL	Notes
Integer	`INT`, `BIGINT`, `SMALLINT`	same	Use the smallest that fits
Decimal	`DECIMAL(p,s)`	`NUMERIC(p,s)`	Exact; use for money and CGPA
Floating	`FLOAT`, `DOUBLE`	`REAL`, `DOUBLE PRECISION`	Approximate — never for money
Fixed text	`CHAR(n)`	`CHAR(n)`	Padded to n; use for codes
Variable text	`VARCHAR(n)`	`VARCHAR(n)`, `TEXT`	Most common
Date/time	`DATE`, `DATETIME`, `TIMESTAMP`, `TIME`	`DATE`, `TIMESTAMP`, `TIME`	`TIMESTAMP` tracks time zone in PostgreSQL
Boolean	`BOOLEAN` (alias of `TINYINT(1)`)	`BOOLEAN`	MySQL stores 0/1
Binary	`BLOB`	`BYTEA`	Prefer storing a file path
JSON	`JSON`	`JSON`, `JSONB`	`JSONB` is indexable

> A rule worth adopting permanently: **never store money or CGPA in `FLOAT`**. Binary floating point cannot represent 0.1 exactly, so sums drift. Use `DECIMAL(10,2)`.

ALTER

```sql
ALTER TABLE department ADD COLUMN phone VARCHAR(20);
ALTER TABLE department MODIFY COLUMN building VARCHAR(60);      -- MySQL
ALTER TABLE department ALTER COLUMN building TYPE VARCHAR(60);  -- PostgreSQL
ALTER TABLE department DROP COLUMN phone;
ALTER TABLE department RENAME COLUMN founded TO established;
ALTER TABLE department ADD CONSTRAINT uq_dept_name UNIQUE (dept_name);
ALTER TABLE department DROP CONSTRAINT uq_dept_name;            -- MySQL: DROP INDEX
```

DROP and TRUNCATE

```sql
DROP TABLE enrollment;          -- removes structure and data permanently
TRUNCATE TABLE enrollment;      -- removes all rows, keeps the structure
```

Basis	DELETE	TRUNCATE	DROP
Category	DML	DDL	DDL
Removes	Selected rows	All rows	Rows + structure
`WHERE` clause	Yes	No	No
Rollback	Yes (within a transaction)	Generally no	No
Speed on large tables	Slow (row by row, logged)	Very fast (deallocates pages)	Fast
Resets `AUTO_INCREMENT`	No	Yes	Not applicable
Fires triggers	Yes	No	No
Table exists afterwards	Yes	Yes	No

This three-way comparison is one of the most frequently asked questions in both examinations and interviews.

10.4 DML — Data Manipulation Language

INSERT

```sql
-- all columns, in table order
INSERT INTO department VALUES ('CSE', 'Computer Science and Engineering', 'Academic
    ↪ Building 3', '2006-01-01');

-- named columns (preferred: survives schema changes)
INSERT INTO department (dept_id, dept_name, building)
VALUES ('EEE', 'Electrical and Electronic Engineering', 'Academic Building 2');

-- multiple rows in one statement
INSERT INTO department (dept_id, dept_name, building) VALUES
    ('ME',  'Mechanical Engineering', 'Academic Building 1'),
    ('BBA', 'Business Administration', 'Academic Building 4');

-- insert from a query
INSERT INTO archived_student (student_id, name)
SELECT student_id, name FROM student WHERE cgpa IS NULL;
```

UPDATE

```sql
UPDATE student
SET    cgpa = 3.85, name = 'Rahim Uddin'
WHERE  student_id = 'ASH1901001';
```

> The single most destructive mistake in SQL is an `UPDATE` or `DELETE` without a `WHERE` clause; it silently modifies every row. Two habits prevent it: write the `WHERE` clause **first**, and run the statement as a `SELECT` before converting it to an `UPDATE`.

DELETE

```sql
DELETE FROM enrollment WHERE grade = 'F' AND semester = '5th';
```

```sql
DELETE FROM enrollment;                    -- every row: almost always a mistake
```

10.5 DQL — Data Query Language

`SELECT` is the whole of DQL and the subject of Chapters 11 to 14. Its general form:

```sql
SELECT   [DISTINCT] column_list
FROM     table_list
[JOIN    ... ON ...]
[WHERE   row_condition]
[GROUP BY grouping_columns]
[HAVING  group_condition]
[ORDER BY sort_columns [ASC|DESC]]
[LIMIT   n [OFFSET m]];
```

10.6 DCL — Data Control Language

```sql
CREATE USER 'exam_clerk'@'localhost' IDENTIFIED BY 'strong_password';

GRANT SELECT, INSERT, UPDATE ON university_db.enrollment TO 'exam_clerk'@'localhost';
GRANT SELECT ON university_db.student TO 'exam_clerk'@'localhost';

REVOKE UPDATE ON university_db.enrollment FROM 'exam_clerk'@'localhost';

-- column-level privilege
GRANT UPDATE (grade) ON university_db.enrollment TO 'exam_clerk'@'localhost';

FLUSH PRIVILEGES;                          -- MySQL, after direct grant-table changes
```

The clerk can now record grades but can neither delete enrollments nor change a student's CGPA — the principle of least privilege, developed fully in Chapter 33.

10.7 TCL — Transaction Control Language

```sql
START TRANSACTION;                         -- or BEGIN

UPDATE account SET balance = balance - 1000 WHERE acc_no = 'A101';
UPDATE account SET balance = balance + 1000 WHERE acc_no = 'B202';

SAVEPOINT after_transfer;

UPDATE account SET balance = balance - 50 WHERE acc_no = 'A101';   -- service charge

ROLLBACK TO SAVEPOINT after_transfer;      -- undo only the charge
COMMIT;                                    -- make the transfer permanent
```

Command	Effect
`START TRANSACTION` / `BEGIN`	Begins a transaction; changes become provisional
`COMMIT`	Makes all changes since the start permanent and visible to others
`ROLLBACK`	Discards all changes since the start (or since a savepoint)
`SAVEPOINT name`	Marks an intermediate point that can be rolled back to
`SET TRANSACTION ISOLATION LEVEL ...`	Chooses the isolation level (Chapter 28)

10.8 A Complete First Session

Each statement below is followed by its effect, as laboratory examinations require.

```
CREATE DATABASE university_db;                    -- creates an empty database
USE university_db;                                -- selects it as the default

CREATE TABLE department (                         -- DDL: defines structure
    dept_id   CHAR(3) PRIMARY KEY,
    dept_name VARCHAR(80) NOT NULL UNIQUE,
    building  VARCHAR(40)
);
-- Result: Query OK, 0 rows affected.  The table now exists and is empty.

INSERT INTO department VALUES ('CSE','Computer Science and Engineering','AB-3');
-- Result: Query OK, 1 row affected.

SELECT * FROM department;
```

Expected output:

dept_id	dept_name	building
CSE	Computer Science and Engineering	AB-3

```
UPDATE department SET building = 'AB-4' WHERE dept_id = 'CSE';
-- Result: Query OK, 1 row affected.  Rows matched: 1  Changed: 1

DELETE FROM department WHERE dept_id = 'CSE';
-- Result: Query OK, 1 row affected.  The table still exists but is empty.

DROP TABLE department;
-- Result: the table and its definition are gone; ROLLBACK cannot restore it.
```

Real-life interpretation. This sequence is exactly what happens when a university adds a new department to its information system, corrects the building it occupies, removes a department created in error, and finally decommissions an obsolete structure. The distinction between the last two statements is what separates a recoverable mistake from an unrecoverable one.

Chapter Summary

- SQL is a declarative, set-oriented language standardized by ANSI/ISO and extended by each vendor.
- Commands fall into five categories: DDL (structure), DML (data), DQL (`SELECT`), DCL (privileges) and TCL (transactions).
- DDL and DCL statements commit implicitly; DML statements do not, so they can be rolled back.
- `DELETE` removes selected rows and can be rolled back; `TRUNCATE` removes all rows quickly and generally cannot; `DROP` removes the table itself.
- Use `DECIMAL`, never `FLOAT`, for money and grade points.
- `NULL` is tested with `IS NULL`, never with `=`.
- Always write the `WHERE` clause before the `SET` clause.

Key Terms

SQL · SEQUEL · ANSI/ISO standard · Declarative language · Set-oriented · DDL · DML · DQL · DCL · TCL · Implicit commit · `CREATE` · `ALTER` · `DROP` · `TRUNCATE` · `INSERT` · `UPDATE` · `DELETE` · `GRANT` · `REVOKE` · `COMMIT` · `ROLLBACK` · `SAVEPOINT` · Data type · `DECIMAL` · `VARCHAR` · `NULL`

Common Mistakes

1. Omitting `WHERE` in `UPDATE` or `DELETE`.
2. Expecting `ROLLBACK` to undo `DROP` or `TRUNCATE`.
3. Writing `WHERE cgpa = NULL` instead of `WHERE cgpa IS NULL`.
4. Using `FLOAT` for money, producing rounding drift.
5. Using `CHAR` for variable-length text, wasting space through padding.
6. Writing `INSERT INTO t VALUES (...)` without a column list, so the statement breaks when a column is added.
7. Granting `ALL PRIVILEGES` where `SELECT` would do.
8. Assuming identifier case-sensitivity is the same across systems.

Solved Problems

Problem 10.1 — Classify each statement: `CREATE INDEX`, `COMMIT`, `SELECT`, `REVOKE`, `TRUNCATE`, `UPDATE`.

Solution. `CREATE INDEX` — DDL. `COMMIT` — TCL. `SELECT` — DQL. `REVOKE` — DCL. `TRUNCATE` — DDL (this is the one students misclassify as DML, because it appears to delete data). `UPDATE` — DML.

Problem 10.2 — A table has 10 million rows, all to be removed, and the structure is to be kept. Compare `DELETE FROM t` with `TRUNCATE TABLE t`.

Solution. `DELETE` logs every row for rollback, fires row triggers, leaves the `AUTO_INCREMENT` counter unchanged, and may take many minutes while holding locks and expanding the log. `TRUNCATE` deallocates the table's pages, does not log individual rows, does not fire triggers, resets the counter, and completes in well under a second. Therefore use `TRUNCATE` — *unless* the deletion must be reversible or triggers must fire, in which case `DELETE` inside a transaction is the only correct choice.

Problem 10.3 — Write DDL for `course` with a check that credit is between 1 and 4, a default of 3, a unique course code and a foreign key to `department`.

Solution.

```
CREATE TABLE course (
    course_id   INT AUTO_INCREMENT PRIMARY KEY,
    course_code VARCHAR(10)  NOT NULL,
    title       VARCHAR(120) NOT NULL,
    credit      DECIMAL(2,1) NOT NULL DEFAULT 3.0,
    dept_id     CHAR(3)      NOT NULL,
    CONSTRAINT uq_course_code UNIQUE (course_code),
    CONSTRAINT ck_course_credit CHECK (credit BETWEEN 1.0 AND 4.0),
    CONSTRAINT fk_course_dept FOREIGN KEY (dept_id)
        REFERENCES department(dept_id) ON UPDATE CASCADE ON DELETE RESTRICT
);
```

Problem 10.4 — Demonstrate `SAVEPOINT` with a banking example in which only part of a transaction is undone.

Solution.

```
START TRANSACTION;
UPDATE account SET balance = balance - 5000 WHERE acc_no = 'A101';   -- debit
UPDATE account SET balance = balance + 5000 WHERE acc_no = 'B202';   -- credit
SAVEPOINT transfer_done;
UPDATE account SET balance = balance - 25 WHERE acc_no = 'A101';     -- charge
-- the bank waives the charge for this customer
ROLLBACK TO SAVEPOINT transfer_done;
COMMIT;
```

The transfer is committed; the charge is discarded. Note that `ROLLBACK TO SAVEPOINT` does not end the transaction — a `COMMIT` or full `ROLLBACK` is still required.

Problem 10.5 — A clerk must record grades but must never change a student's CGPA or delete an enrollment. Write the DCL.

Solution.

```
CREATE USER 'exam_clerk'@'%' IDENTIFIED BY 'strong_password';
GRANT SELECT ON university_db.student TO 'exam_clerk'@'%';
GRANT SELECT, INSERT ON university_db.enrollment TO 'exam_clerk'@'%';
GRANT UPDATE (grade) ON university_db.enrollment TO 'exam_clerk'@'%';
```

No `DELETE` is granted anywhere, and `UPDATE` is restricted to a single column, so neither `enrollment` rows nor `student.cgpa` can be altered.

Problem 10.6 — Explain why `SELECT * FROM student WHERE cgpa = NULL` returns nothing, and give the correct query.

Solution. `NULL` means "unknown"; any comparison with an unknown value evaluates to *unknown*, not true, and `WHERE` keeps only rows for which the condition is true. Every row therefore fails the test, including those where `cgpa` actually is NULL. The correct form uses the null predicate:

```
SELECT * FROM student WHERE cgpa IS NULL;
```

Problem 10.7 — Which statements below can be rolled back, and why? (a) `INSERT`; (b) `CREATE TABLE`; (c) `UPDATE`; (d) `GRANT`; (e) `TRUNCATE`.

Solution. (a) and (c) are DML and are rolled back normally. (b), (d) and (e) are DDL/DCL; in MySQL and Oracle they cause an implicit commit, so everything before them is committed and they themselves cannot be undone. PostgreSQL is the notable exception — it supports transactional DDL, so `CREATE TABLE` and `TRUNCATE` *can* be rolled back there. Stating this vendor difference earns full marks.

Review Questions

1. What does it mean to call SQL declarative and set-oriented?
2. List the five categories of SQL commands with their member statements.
3. Which categories commit implicitly, and what practical consequence follows?
4. Compare `DELETE`, `TRUNCATE` and `DROP` on at least seven criteria.
5. Why should `DECIMAL` be used instead of `FLOAT` for money?
6. Explain `CHAR` versus `VARCHAR`.
7. Write and explain the general syntax of `SELECT`.
8. Explain `GRANT` and `REVOKE` with column-level examples.
9. Explain `SAVEPOINT` and `ROLLBACK TO SAVEPOINT`.
10. Why must `IS NULL` be used instead of `= NULL`?

Multiple Choice Questions

1. `TRUNCATE` belongs to (a) DML (b) DDL (c) DCL (d) TCL.
2. Which can be rolled back in MySQL? (a) `DROP` (b) `TRUNCATE` (c) `DELETE` (d) `GRANT`.
3. `GRANT` belongs to (a) DDL (b) DML (c) DCL (d) TCL.
4. For an exact monetary value use (a) `FLOAT` (b) `DOUBLE` (c) `DECIMAL` (d) `REAL`.
5. `SELECT` belongs to (a) DDL (b) DQL (c) DCL (d) TCL.
6. Which resets `AUTO_INCREMENT`? (a) `DELETE` (b) `TRUNCATE` (c) `UPDATE` (d) `SELECT`.
7. To test for a missing value use (a) `= NULL` (b) `IS NULL` (c) `== NULL` (d) `NULL()`.
8. `SAVEPOINT` belongs to (a) DDL (b) DML (c) DCL (d) TCL.
9. Which statement changes a table's structure? (a) `UPDATE` (b) `ALTER` (c) `INSERT` (d) `SELECT`.
10. Strings in standard SQL are enclosed in (a) double quotes (b) single quotes (c) backticks (d) brackets.

Answers: 1-(b) 2-(c) 3-(c) 4-(c) 5-(b) 6-(b) 7-(b) 8-(d) 9-(b) 10-(b)

Short Questions

1. Expand DDL, DML, DQL, DCL and TCL.
2. What is an implicit commit?
3. Give one difference between `DELETE` and `TRUNCATE`.
4. What does `DEFAULT` do in a column definition?
5. Why is a named column list preferred in `INSERT`?
6. What is the principle of least privilege?
7. What does `ROLLBACK TO SAVEPOINT` not do?
8. Name two SQL standard versions and one feature each introduced.
9. Which data type suits a fixed three-letter department code?
10. Write a comment in SQL two different ways.

Long / Descriptive Questions

1. Describe the five categories of SQL with syntax, examples, output and real-life interpretation for each.
2. Compare `DELETE`, `TRUNCATE` and `DROP` in depth, including logging, triggers, rollback and performance.
3. Explain SQL data types in detail and give guidance on choosing among them, with examples of poor choices and their consequences.
4. Explain transaction control in SQL with a complete banking example using `SAVEPOINT`.
5. Discuss SQL standardization and vendor differences, with examples from MySQL, PostgreSQL, Oracle and SQL Server.

Practical Exercises

1. Create `university_db` and the `department` table; insert four rows; then alter the table to add a column and a constraint.
2. Demonstrate the difference between `DELETE`, `TRUNCATE` and `DROP` on a copy of a table with an `AUTO_INCREMENT` column, recording the counter value after each.
3. Attempt to roll back a `TRUNCATE` in MySQL and then in PostgreSQL; record and explain the difference.

4. Create a restricted user with column-level `UPDATE` privilege and verify from that account that other operations are refused.
5. Write a transaction with two savepoints and demonstrate partial rollback, verifying the final state with `SELECT`.

CHAPTER 11

SQL Querying

Everything from this chapter to the end of the book uses one database. Create it once, keep it, and every example in the remaining chapters can be run against it.

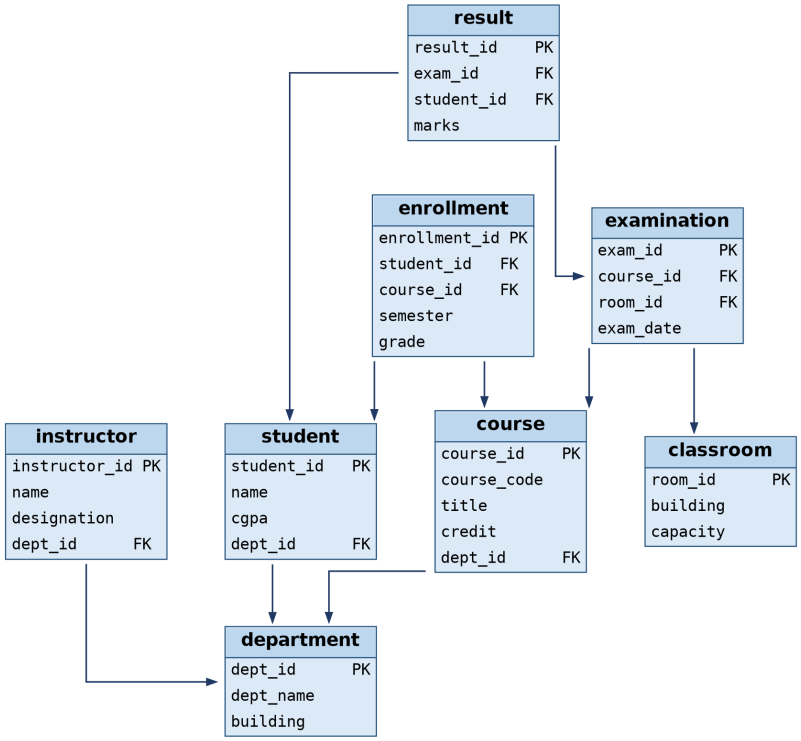


Figure 11.1: The university_db schema used throughout this book

11.1 Creating the Sample Database

```sql
DROP DATABASE IF EXISTS university_db;
CREATE DATABASE university_db;
USE university_db;

CREATE TABLE department (
    dept_id   CHAR(3)      PRIMARY KEY,
    dept_name VARCHAR(80)  NOT NULL UNIQUE,
    building  VARCHAR(40)
);

CREATE TABLE instructor (
    instructor_id INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    designation   VARCHAR(40)  NOT NULL,
    salary        DECIMAL(10,2) CHECK (salary > 0),
    dept_id       CHAR(3)      NOT NULL,
    CONSTRAINT fk_inst_dept FOREIGN KEY (dept_id)
        REFERENCES department(dept_id) ON UPDATE CASCADE ON DELETE RESTRICT
```

```
);

CREATE TABLE student (
    student_id  CHAR(10)     PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(100) UNIQUE,
    dob         DATE,
    cgpa        DECIMAL(3,2) CHECK (cgpa BETWEEN 0.00 AND 4.00),
    dept_id     CHAR(3)      NOT NULL,
    admitted_on DATE         NOT NULL,
    CONSTRAINT fk_stu_dept FOREIGN KEY (dept_id)
        REFERENCES department(dept_id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE course (
    course_id   INT AUTO_INCREMENT PRIMARY KEY,
    course_code VARCHAR(10)  NOT NULL UNIQUE,
    title       VARCHAR(120) NOT NULL,
    credit      DECIMAL(2,1) NOT NULL DEFAULT 3.0 CHECK (credit BETWEEN 0.5 AND 4.0),
    dept_id     CHAR(3)      NOT NULL,
    CONSTRAINT fk_course_dept FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

CREATE TABLE classroom (
    room_id  VARCHAR(10) PRIMARY KEY,
    building VARCHAR(40) NOT NULL,
    capacity INT         NOT NULL CHECK (capacity > 0)
);

CREATE TABLE enrollment (
    enrollment_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id    CHAR(10) NOT NULL,
    course_id     INT      NOT NULL,
    semester      VARCHAR(10) NOT NULL,
    grade         CHAR(2)  NULL,
    CONSTRAINT uq_enrol UNIQUE (student_id, course_id, semester),
    CONSTRAINT fk_enr_stu FOREIGN KEY (student_id)
        REFERENCES student(student_id) ON DELETE CASCADE,
    CONSTRAINT fk_enr_course FOREIGN KEY (course_id)
        REFERENCES course(course_id) ON DELETE RESTRICT
);

CREATE TABLE examination (
    exam_id   INT AUTO_INCREMENT PRIMARY KEY,
    course_id INT         NOT NULL,
    exam_date DATE        NOT NULL,
    room_id   VARCHAR(10) NOT NULL,
    CONSTRAINT fk_exam_course FOREIGN KEY (course_id) REFERENCES course(course_id),
    CONSTRAINT fk_exam_room   FOREIGN KEY (room_id)   REFERENCES classroom(room_id)
);

CREATE TABLE result (
    result_id  INT AUTO_INCREMENT PRIMARY KEY,
    exam_id    INT      NOT NULL,
    student_id CHAR(10) NOT NULL,
    marks      DECIMAL(5,2) CHECK (marks BETWEEN 0 AND 100),
    CONSTRAINT uq_result UNIQUE (exam_id, student_id),
    CONSTRAINT fk_res_exam FOREIGN KEY (exam_id)    REFERENCES examination(exam_id),
```

```
    CONSTRAINT fk_res_stu  FOREIGN KEY (student_id) REFERENCES student(student_id)
);
```

11.2 Populating the Sample Database

```
INSERT INTO department VALUES
 ('CSE','Computer Science and Engineering','Academic Building 3'),
 ('EEE','Electrical and Electronic Engineering','Academic Building 2'),
 ('ME', 'Mechanical Engineering','Academic Building 1'),
 ('BBA','Business Administration','Academic Building 4');

INSERT INTO instructor (name, designation, salary, dept_id) VALUES
 ('Nasrin Akter','Professor',95000,'CSE'),
 ('Imran Hossain','Associate Professor',78000,'CSE'),
 ('Sabina Yasmin','Assistant Professor',62000,'CSE'),
 ('Tanvir Ahmed','Professor',94000,'EEE'),
 ('Farhana Islam','Lecturer',45000,'EEE'),
 ('Mahmudul Hasan','Assistant Professor',60000,'ME'),
 ('Shirin Sultana','Associate Professor',75000,'BBA');

INSERT INTO student VALUES
 ('ASH1901001','Rahim Uddin','rahim@nstu.edu.bd','2001-03-14',3.75,'CSE','2019-01-10'),
 ('ASH1901002','Karim Mia','karim@nstu.edu.bd','2001-07-02',3.60,'EEE','2019-01-10'),
 ('ASH1901003','Nila Rahman','nila@nstu.edu.bd','2002-01-25',3.90,'CSE','2019-01-10'),
 ('ASH1901004','Sumaiya Akter','sumaiya@nstu.edu.bd','2001-11-30',3.20,'CSE','2019-01-10
    ↪ '),
 ('ASH1901005','Jubayer Hossain','jubayer@nstu.edu.bd','2000-09-18',2.85,'ME','
    ↪ 2019-01-10'),
 ('ASH2001001','Tania Sultana','tania@nstu.edu.bd','2002-05-05',3.95,'CSE','2020-01-12')
    ↪ ,
 ('ASH2001002','Rakib Hasan','rakib@nstu.edu.bd','2002-02-19',3.10,'EEE','2020-01-12'),
 ('ASH2001003','Mitu Barua','mitu@nstu.edu.bd','2002-08-08',NULL,'BBA','2020-01-12'),
 ('ASH2001004','Shakib Al Hasan','shakib@nstu.edu.bd','2001-12-01',3.45,'BBA','
    ↪ 2020-01-12'),
 ('ASH2001005','Fahmida Nabi','fahmida@nstu.edu.bd','2002-04-21',3.70,'ME','2020-01-12')
    ↪ ;

INSERT INTO course (course_code, title, credit, dept_id) VALUES
 ('CSE3201','Database Management Systems',3.0,'CSE'),
 ('CSE3203','Operating Systems',3.0,'CSE'),
 ('CSE2101','Data Structures',3.0,'CSE'),
 ('CSE4101','Machine Learning',3.0,'CSE'),
 ('EEE3101','Digital Electronics',3.0,'EEE'),
 ('EEE3103','Signals and Systems',3.0,'EEE'),
 ('ME2101', 'Engineering Mechanics',3.0,'ME'),
 ('BBA1101','Principles of Management',3.0,'BBA');

INSERT INTO classroom VALUES
 ('AB3-401','Academic Building 3',60),
 ('AB3-402','Academic Building 3',45),
 ('AB2-201','Academic Building 2',80),
 ('AB4-101','Academic Building 4',120);

INSERT INTO enrollment (student_id, course_id, semester, grade) VALUES
 ('ASH1901001',1,'5th','A'),   ('ASH1901001',2,'5th','A-'),
 ('ASH1901001',3,'3rd','B+'),  ('ASH1901003',1,'5th','A'),
 ('ASH1901003',2,'5th','A'),   ('ASH1901003',4,'7th','A-'),
```

```
 ('ASH1901004',1,'5th','B'),   ('ASH1901004',3,'3rd','C+'),
 ('ASH2001001',1,'5th','A'),   ('ASH2001001',3,'3rd','A'),
 ('ASH1901002',5,'5th','B+'),  ('ASH1901002',6,'5th','B'),
 ('ASH2001002',5,'5th','C'),   ('ASH1901005',7,'3rd','B-'),
 ('ASH2001005',7,'3rd','A-'),  ('ASH2001004',8,'1st','B+'),
 ('ASH2001003',8,'1st',NULL);

INSERT INTO examination (course_id, exam_date, room_id) VALUES
 (1,'2024-06-10','AB3-401'), (2,'2024-06-13','AB3-401'),
 (3,'2024-06-16','AB3-402'), (5,'2024-06-11','AB2-201'),
 (8,'2024-06-12','AB4-101');

INSERT INTO result (exam_id, student_id, marks) VALUES
 (1,'ASH1901001',82.00), (1,'ASH1901003',88.50), (1,'ASH1901004',67.00),
 (1,'ASH2001001',91.00), (2,'ASH1901001',78.00), (2,'ASH1901003',85.00),
 (3,'ASH1901001',71.50), (3,'ASH1901004',58.00), (3,'ASH2001001',89.00),
 (4,'ASH1901002',74.00), (4,'ASH2001002',52.00), (5,'ASH2001004',76.00);
```

11.3 The Logical Order of Evaluation

A query is *written* `SELECT` first, but it is *evaluated* `FROM` first. Understanding this single fact removes most beginner confusion.



SQL is written SELECT-first but evaluated FROM-first — this is why a SELECT alias cannot be used in WHERE, but can be used in ORDER BY

Figure 11.2: The logical processing order of a SELECT statement

Two consequences follow immediately:

- A column alias defined in `SELECT` **cannot** be used in `WHERE`, because `WHERE` runs before `SELECT` exists.
- The same alias **can** be used in `ORDER BY`, because `ORDER BY` runs after `SELECT`.

```
-- WRONG in standard SQL: the alias does not exist yet when WHERE runs
SELECT name, cgpa * 25 AS percentage FROM student WHERE percentage > 90;

-- Correct: repeat the expression in WHERE
SELECT name, cgpa * 25 AS percentage FROM student WHERE cgpa * 25 > 90;

-- Correct: the alias is available in ORDER BY
SELECT name, cgpa * 25 AS percentage FROM student ORDER BY percentage DESC;
```

11.4 SELECT and Column Selection

Problem. List every student with their name and CGPA.

```sql
SELECT name, cgpa FROM student;
```

Explanation. `FROM student` supplies the rows; `SELECT name, cgpa` keeps two of the seven columns.

Expected output (first rows):

name	cgpa
Rahim Uddin	3.75
Karim Mia	3.60
Nila Rahman	3.90
Sumaiya Akter	3.20

Real-world interpretation. This is the query behind a merit list screen.

> `SELECT *` is convenient in an interactive session and a bad habit in application code: it breaks when a column is added, transfers columns nobody needs, and prevents the optimizer from using index-only access.

11.5 WHERE — Filtering Rows

Problem. List CSE students whose CGPA is above 3.50.

```sql
SELECT student_id, name, cgpa
FROM   student
WHERE  dept_id = 'CSE' AND cgpa > 3.50;
```

student_id	name	cgpa
ASH1901001	Rahim Uddin	3.75
ASH1901003	Nila Rahman	3.90
ASH2001001	Tania Sultana	3.95

Comparison and logical operators

Operator	Meaning	Example
`=`, `<>` or `!=`	Equal, not equal	`dept_id = 'CSE'`
`<`, `>`, `<=`, `>=`	Ordering	`cgpa >= 3.50`
`AND`, `OR`, `NOT`	Logical combination	`cgpa > 3 AND dept_id = 'CSE'`
`BETWEEN a AND b`	Inclusive range	`cgpa BETWEEN 3.00 AND 3.50`
`IN (list)`	Membership	`dept_id IN ('CSE','EEE')`
`LIKE`	Pattern match	`name LIKE 'R%'`
`IS NULL`, `IS NOT NULL`	Null test	`grade IS NULL`

`AND` binds more tightly than `OR`, so parentheses are not optional when both appear:

```sql
-- returns CSE students with high CGPA, PLUS every EEE student
SELECT name FROM student WHERE dept_id = 'CSE' AND cgpa > 3.5 OR dept_id = 'EEE';

-- returns high-CGPA students from either department
SELECT name FROM student WHERE (dept_id = 'CSE' OR dept_id = 'EEE') AND cgpa > 3.5;
```

11.6 BETWEEN, IN, LIKE and IS NULL

```sql
-- BETWEEN is inclusive of both endpoints
SELECT name, cgpa FROM student WHERE cgpa BETWEEN 3.00 AND 3.50;

-- IN is shorthand for a chain of ORs
SELECT name, dept_id FROM student WHERE dept_id IN ('CSE','ME');

-- LIKE:  %  = any sequence of characters,  _ = exactly one character
SELECT name FROM student WHERE name LIKE 'R%';        -- starts with R
SELECT name FROM student WHERE name LIKE '%Akter';    -- ends with Akter
SELECT name FROM student WHERE name LIKE '%Has%';     -- contains Has
SELECT course_code FROM course WHERE course_code LIKE 'CSE3___';  -- CSE3 + 3 chars

-- NULL test
SELECT student_id, name FROM student WHERE cgpa IS NULL;
```

Output of the last query:

student_id	name
ASH2001003	Mitu Barua

Real-world interpretation. A student whose CGPA is NULL has not yet completed a graded semester — a condition the registrar must be able to find, which is impossible with `= NULL`.

11.7 DISTINCT

Problem. Which departments actually have students enrolled?

```sql
SELECT DISTINCT dept_id FROM student;
```

dept_id
CSE
EEE
ME
BBA

`DISTINCT` applies to the **whole row** of the select list, not to one column: `SELECT DISTINCT dept_id, cgpa` removes duplicate *pairs*, which is usually not what a student intends.

11.8 ORDER BY

```sql
SELECT name, cgpa, dept_id
FROM   student
ORDER  BY dept_id ASC, cgpa DESC;
```

Sorting is by department ascending, then by CGPA descending within each department. Points worth knowing:

- `ASC` is the default and may be omitted.
- Sorting may use a column position (`ORDER BY 2 DESC`), though naming the column is clearer and safer.
- NULL ordering differs by vendor: MySQL sorts NULLs first ascending; PostgreSQL sorts them

last, and supports `NULLS FIRST` / `NULLS LAST`.

- Without `ORDER BY`, no order is guaranteed — see Section 5.1.

11.9 LIMIT and Pagination

```sql
SELECT name, cgpa FROM student ORDER BY cgpa DESC LIMIT 3;          -- top 3
SELECT name, cgpa FROM student ORDER BY cgpa DESC LIMIT 3 OFFSET 3; -- next 3
```

name	cgpa
Tania Sultana	3.95
Nila Rahman	3.90
Rahim Uddin	3.75

Vendor note: MySQL and PostgreSQL use `LIMIT`; SQL Server uses `TOP n` or `OFFSET … FETCH NEXT`; Oracle 12c+ uses `FETCH FIRST n ROWS ONLY`.

11.10 Aliases

```sql
SELECT s.name        AS student_name,
       s.cgpa        AS grade_point,
       s.cgpa * 25   AS percentage
FROM   student AS s
WHERE  s.dept_id = 'CSE'
ORDER  BY percentage DESC;
```

Table aliases (`s`) shorten qualified names and are **mandatory** in self joins; column aliases rename output columns. An alias containing spaces must be quoted: `AS "Grade Point"`.

11.11 GROUP BY — Aggregation per Group

`GROUP BY` collapses rows that share a value into one row per group, and aggregate functions summarize each group.

Problem. How many students are in each department, and what is their average CGPA?

```sql
SELECT   dept_id,
         COUNT(*)           AS total_students,
         ROUND(AVG(cgpa),2) AS avg_cgpa,
         MAX(cgpa)          AS best_cgpa
FROM     student
GROUP BY dept_id
ORDER BY avg_cgpa DESC;
```

dept_id	total_students	avg_cgpa	best_cgpa
CSE	4	3.70	3.95
BBA	2	3.45	3.45
EEE	2	3.35	3.60
ME	2	3.28	3.70

Two rules govern `GROUP BY` and are examined constantly:

1. Every column in `SELECT` must either appear in `GROUP BY` or be inside an aggregate function.

(MySQL relaxes this unless `ONLY_FULL_GROUP_BY` is enabled — the relaxation produces unpredictable results and should not be relied on.)

2. `COUNT(*)` counts rows; `COUNT(column)` counts non-NULL values. For BBA, `COUNT(*)` is 2 but `COUNT(cgpa)` is 1, because Mitu Barua's CGPA is NULL. `AVG(cgpa)` likewise ignores the NULL rather than treating it as zero — which is why the BBA average is 3.45 and not 1.725.

11.12 HAVING — Filtering Groups

Problem. List departments having more than two students whose average CGPA exceeds 3.30.

```sql
SELECT   dept_id, COUNT(*) AS n, ROUND(AVG(cgpa),2) AS avg_cgpa
FROM     student
GROUP BY dept_id
HAVING   COUNT(*) > 2 AND AVG(cgpa) > 3.30;
```

dept_id	n	avg_cgpa
CSE	4	3.70

Basis	`WHERE`	`HAVING`
Applies to	Individual rows	Groups
Runs	Before grouping	After grouping
Aggregates allowed	No	Yes
Typical use	`WHERE dept_id = 'CSE'`	`HAVING COUNT(*) > 2`
Can use SELECT aliases	No	In MySQL yes, in standard SQL no

Both may appear in one query, and when they do, the `WHERE` clause should carry every condition it can, because filtering rows before grouping is cheaper than grouping and then discarding:

```sql
SELECT   dept_id, COUNT(*) AS n
FROM     student
WHERE    admitted_on >= '2020-01-01'     -- row filter: runs first, cheapest
GROUP BY dept_id
HAVING   COUNT(*) >= 2;                  -- group filter: runs last
```

11.13 Twelve Worked Queries

Each follows the required pattern: problem, SQL, explanation, output, interpretation.

Q1. Students admitted in 2020.

```sql
SELECT student_id, name, admitted_on FROM student
WHERE  admitted_on BETWEEN '2020-01-01' AND '2020-12-31';
```

Five rows (`ASH2001001`–`ASH2001005`). Date literals are strings in `YYYY-MM-DD` form; `BETWEEN` includes both endpoints. *Used to produce a batch-wise admission register.*

Q2. Students whose name contains "Hasan".

```sql
SELECT name FROM student WHERE name LIKE '%Hasan%';
```

Returns `Rakib Hasan` and `Shakib Al Hasan`. `%` matches any run of characters, so the pattern finds the word anywhere. *Used by a help desk searching by partial name.*

Q3. Courses worth three credits in CSE or EEE.

```sql
SELECT course_code, title, dept_id FROM course
WHERE  credit = 3.0 AND dept_id IN ('CSE','EEE');
```

Six rows. *Used when building a semester course list.*

Q4. Number of enrollments per semester.

```sql
SELECT semester, COUNT(*) AS enrollments
FROM   enrollment GROUP BY semester ORDER BY semester;
```

semester	enrollments
1st	2
3rd	5
5th	9
7th	1

Used for load planning: the fifth semester needs the most classroom capacity.

Q5. Enrollments with no grade recorded.

```sql
SELECT student_id, course_id, semester FROM enrollment WHERE grade IS NULL;
```

One row (`ASH2001003`, course 8). *Used by the examination office to chase missing results.*

Q6. Average marks per examination.

```sql
SELECT   exam_id, COUNT(*) AS appeared, ROUND(AVG(marks),2) AS avg_marks,
         MIN(marks) AS lowest, MAX(marks) AS highest
FROM     result GROUP BY exam_id ORDER BY avg_marks DESC;
```

exam_id	appeared	avg_marks	lowest	highest
1	4	82.13	67.00	91.00
2	2	81.50	78.00	85.00
5	1	76.00	76.00	76.00
3	3	72.83	58.00	89.00
4	2	63.00	52.00	74.00

Used in a question-paper quality review: exam 4 has a low average and a wide spread.

Q7. Departments with an average CGPA above 3.4, most competitive first.

```sql
SELECT   dept_id, ROUND(AVG(cgpa),2) AS avg_cgpa
FROM     student GROUP BY dept_id HAVING AVG(cgpa) > 3.4 ORDER BY avg_cgpa DESC;
```

CSE (3.70) and BBA (3.45). *Used in an annual academic report.*

Q8. The three highest-paid instructors.

```sql
SELECT name, designation, salary FROM instructor ORDER BY salary DESC LIMIT 3;
```

Nasrin Akter (95000), Tanvir Ahmed (94000), Imran Hossain (78000). *Used in a payroll audit.*

Q9. Distinct designations in use.

```sql
SELECT DISTINCT designation FROM instructor ORDER BY designation;
```

Assistant Professor, Associate Professor, Lecturer, Professor. *Used to populate a dropdown in a recruitment form.*

Q10. Count of graded versus total enrollments.

```
SELECT COUNT(*) AS total, COUNT(grade) AS graded,
       COUNT(*) - COUNT(grade) AS pending FROM enrollment;
```

total	graded	pending
17	16	1

The difference between `COUNT(*)` and `COUNT(grade)` is exactly the number of NULLs — the standard idiom for counting missing values. *Used on a results-completion dashboard.*

Q11. Students whose CGPA is within 0.2 of the best CGPA in the table.

```
SELECT name, cgpa FROM student
WHERE  cgpa >= (SELECT MAX(cgpa) FROM student) - 0.2
ORDER  BY cgpa DESC;
```

Tania Sultana (3.95), Nila Rahman (3.90), Rahim Uddin (3.75). This is a first sight of a subquery, developed fully in Chapter 14. *Used to shortlist candidates for a scholarship.*

Q12. Percentage conversion with an alias, sorted.

```
SELECT name, cgpa, ROUND(cgpa * 25, 1) AS percentage
FROM   student WHERE cgpa IS NOT NULL ORDER BY percentage DESC LIMIT 5;
```

Used to produce a printed merit certificate, where percentage is required by a scholarship authority.

Chapter Summary

- The sample database `university_db` has eight tables and is used for every SQL example in this book.
- A query is written `SELECT`-first but evaluated `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`.
- `WHERE` filters rows before grouping; `HAVING` filters groups after it, and only `HAVING` may contain aggregates.
- `BETWEEN` is inclusive, `IN` is shorthand for chained `OR`, `LIKE` uses `%` and `_`, and NULL requires `IS NULL`.
- `COUNT(*)` counts rows; `COUNT(column)` and every other aggregate ignore NULL.
- `DISTINCT` applies to the entire select list, and without `ORDER BY` no row order is guaranteed.

Key Terms

`SELECT` · Projection · `WHERE` · Predicate · `DISTINCT` · `ORDER BY` · `ASC` / `DESC` · `LIMIT` / `OFFSET` · Alias · `GROUP BY` · Aggregate function · `HAVING` · `BETWEEN` · `IN` · `LIKE` · Wildcard · `IS NULL` · Logical processing order · `ONLY_FULL_GROUP_BY`

Common Mistakes

1. Using a `SELECT` alias inside `WHERE`.
2. Putting an aggregate in `WHERE` instead of `HAVING`.
3. Selecting a non-aggregated column that is not in `GROUP BY`.

4. Assuming `AVG` treats NULL as zero; it ignores NULL rows entirely.
5. Writing `= NULL` instead of `IS NULL`.
6. Forgetting parentheses when mixing `AND` with `OR`.
7. Expecting a stable row order without `ORDER BY`.
8. Using `SELECT *` in application code.
9. Assuming `BETWEEN` is exclusive; it includes both endpoints.
10. Using `LIMIT` without `ORDER BY` and calling the result "top *n*".

Solved Problems

Problem 11.1 — Write a query listing the name and CGPA of the top two CSE students, and explain why `ORDER BY` is essential.

Solution.

```sql
SELECT name, cgpa FROM student
WHERE  dept_id = 'CSE' ORDER BY cgpa DESC LIMIT 2;
```

`LIMIT` takes the first two rows *of whatever order the engine produces*. Without `ORDER BY`, that order is arbitrary and may change when an index is added, so the query would return two random CSE students rather than the top two.

Problem 11.2 — A department has 5 students, two of whom have NULL CGPA. Compare `COUNT(*)`, `COUNT(cgpa)` and `AVG(cgpa)`.

Solution. `COUNT(*)` = 5, since it counts rows regardless of content. `COUNT(cgpa)` = 3, since it counts non-NULL values only. `AVG(cgpa)` = (sum of the three known values) ÷ 3, not ÷ 5 — NULL rows are excluded from both numerator and denominator. If the intended semantics were "treat missing as zero", it must be written explicitly: `AVG(COALESCE(cgpa, 0))`.

Problem 11.3 — Explain the difference between these two queries and give the result of each on the sample data.

```sql
SELECT dept_id, COUNT(*) FROM student WHERE cgpa > 3.5 GROUP BY dept_id;
SELECT dept_id, COUNT(*) FROM student GROUP BY dept_id HAVING AVG(cgpa) > 3.5;
```

Solution. The first filters **rows** before grouping, so it counts only high-CGPA students per department: CSE 3, EEE 1, ME 1. The second groups all students, then keeps **departments** whose average exceeds 3.5, returning CSE with a count of 4 — a count that includes students below 3.5. The lesson: `WHERE` changes which rows form the groups; `HAVING` changes which groups survive.

Problem 11.4 — Find students whose name begins with a vowel.

Solution.

```sql
SELECT name FROM student
WHERE  name LIKE 'A%' OR name LIKE 'E%' OR name LIKE 'I%'
    OR name LIKE 'O%' OR name LIKE 'U%';

-- MySQL/PostgreSQL alternative using a regular expression
SELECT name FROM student WHERE name REGEXP '^[AEIOU]';     -- MySQL
SELECT name FROM student WHERE name ~ '^[AEIOU]';          -- PostgreSQL
```

Note that `LIKE` cannot express a character class; that is exactly when a regular expression is justified. Neither form can use an index efficiently, so on a large table a leading-character column or a full-text index would be preferable.

Problem 11.5 — Write a query producing, for each semester, the number of enrollments and the number of those still ungraded, and show the result.

Solution.

```
SELECT   semester,
         COUNT(*)                  AS enrollments,
         COUNT(*) - COUNT(grade)   AS ungraded
FROM     enrollment
GROUP BY semester
ORDER BY semester;
```

semester	enrollments	ungraded
1st	2	1
3rd	5	0
5th	9	0
7th	1	0

Problem 11.6 — Explain why the following returns an unexpected result and correct it: `SELECT name FROM student WHERE dept_id = 'CSE' OR dept_id = 'EEE' AND cgpa > 3.7;`

Solution. `AND` has higher precedence than `OR`, so the engine reads it as `dept_id = 'CSE' OR (dept_id = 'EEE' AND cgpa > 3.7)`: every CSE student is returned regardless of CGPA. The intended query needs parentheses:

```
SELECT name FROM student WHERE (dept_id = 'CSE' OR dept_id = 'EEE') AND cgpa > 3.7;
```

Problem 11.7 — Write a query returning the second-highest CGPA without using `LIMIT`.

Solution.

```
SELECT MAX(cgpa) AS second_highest
FROM   student
WHERE  cgpa < (SELECT MAX(cgpa) FROM student);
```

The inner query finds the maximum, and the outer query finds the maximum of everything strictly below it. This formulation is portable to systems without `LIMIT`, and it handles ties correctly in the sense that it returns the next *distinct* value.

Problem 11.8 — On a table of 5 million rows, `SELECT * FROM enrollment WHERE semester = '5th'` is slow. Give three improvements.

Solution. (i) Replace `*` with only the needed columns, so less data is read and transferred and an index-only scan may become possible. (ii) Create an index on `semester`, converting a full table scan into an index lookup — worthwhile only if `'5th'` selects a small fraction of the table (Chapter 23). (iii) Add `LIMIT` with pagination if the application only displays a page at a time. A fourth measure, if the table is huge and semester-based access dominates, is partitioning by semester (Chapter 39).

Review Questions

1. Draw the `university_db` schema and list every foreign key in it.
2. State the logical processing order of a `SELECT` and two consequences of it.
3. Distinguish `WHERE` from `HAVING` on at least five criteria.
4. Explain `BETWEEN`, `IN`, `LIKE` and `IS NULL` with examples.
5. Explain how NULL affects `COUNT`, `AVG` and comparison operators.

6. What exactly does `DISTINCT` remove?
7. Why is row order undefined without `ORDER BY`?
8. Explain table and column aliases and when an alias is mandatory.
9. Write the rules governing which columns may appear in a `GROUP BY` query's select list.
10. Why is `SELECT *` discouraged in application code?

Multiple Choice Questions

1. Which clause is evaluated first? (a) `SELECT` (b) `WHERE` (c) `FROM` (d) `ORDER BY`.
2. Aggregates may appear in (a) `WHERE` (b) `HAVING` (c) both (d) neither.
3. `COUNT(column)` ignores (a) duplicates (b) NULLs (c) zeros (d) nothing.
4. `BETWEEN 3 AND 5` includes (a) neither endpoint (b) both endpoints (c) only 3 (d) only 5.
5. `_` in `LIKE` matches (a) any string (b) exactly one character (c) a digit (d) whitespace.
6. `DISTINCT` applies to (a) the first column (b) the entire select list (c) the `WHERE` clause (d) grouped rows.
7. A `SELECT` alias may be used in (a) `WHERE` (b) `ORDER BY` (c) `FROM` (d) `GROUP BY` in all systems.
8. `AVG` of a column with NULLs divides by (a) all rows (b) non-NULL rows (c) NULL rows (d) one.
9. To get the top 5 rows reliably you need (a) `LIMIT` only (b) `ORDER BY` + `LIMIT` (c) `DISTINCT` (d) `GROUP BY`.
10. `WHERE dept='CSE' OR dept='EEE' AND cgpa>3` returns (a) high-CGPA students of both (b) all CSE plus high-CGPA EEE (c) an error (d) nothing.

Answers: 1-(c) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. Which clause filters groups?
2. What does `COUNT(*)` count?
3. Write a pattern matching any code beginning `CSE3` followed by three characters.
4. Why can't an alias be used in `WHERE`?
5. What does `OFFSET` do?
6. How do you count NULLs in a column?
7. Which two wildcards does `LIKE` support?
8. What is the default sort direction?
9. Name one vendor difference in NULL ordering.
10. When is a table alias mandatory?

Long / Descriptive Questions

1. Explain the logical processing order of a `SELECT` statement in detail, with examples of errors caused by misunderstanding it.
2. Explain `GROUP BY` and `HAVING` fully, including the select-list rule and NULL handling, with worked examples.
3. Discuss NULL semantics in SQL across comparisons, aggregates, `COUNT` and sorting.
4. Using `university_db`, write and explain ten queries of increasing difficulty covering all clauses in this chapter.
5. Discuss the performance implications of `SELECT *`, missing `ORDER BY`, and filtering in `HAVING` instead of `WHERE`.

Practical Exercises

1. Create and populate `university_db` exactly as given, and verify the row counts of all eight tables.
2. Write queries answering: students per department; average marks per exam; ungraded enrollments; the top three CGPAs; all courses of a given department.
3. Demonstrate the difference between `WHERE` and `HAVING` with two queries on the same data, and explain the different results.
4. Insert a student with a NULL CGPA and show how `COUNT`, `AVG` and `ORDER BY` treat that row.
5. Write a paginated query returning students five at a time, ordered by CGPA, and show pages 1 and 2.

CHAPTER 12

SQL Functions

SQL functions fall into two families that behave very differently. A **scalar** function takes one row's values and returns one value per row; an **aggregate** function takes many rows and returns one value per group. Confusing the two produces most of the errors in this area.

Family	Input	Output	Examples
Aggregate	A set of rows	One value per group	`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
Scalar — string	One row	One value per row	`CONCAT`, `UPPER`, `SUBSTRING`
Scalar — numeric	One row	One value per row	`ROUND`, `CEILING`, `FLOOR`, `MOD`
Scalar — date	One row	One value per row	`NOW`, `DATEDIFF`, `YEAR`
Scalar — conditional	One row	One value per row	`IFNULL`, `COALESCE`, `CASE`

12.1 Aggregate Functions

Function	Returns	NULL handling
`COUNT(*)`	Number of rows	Counts every row
`COUNT(col)`	Number of non-NULL values	Skips NULL
`COUNT(DISTINCT col)`	Number of distinct non-NULL values	Skips NULL
`SUM(col)`	Total	Skips NULL; returns NULL if no rows qualify
`AVG(col)`	Mean of non-NULL values	Divides by the count of non-NULL values
`MIN(col)`, `MAX(col)`	Smallest, largest	Skips NULL; work on text and dates too

Problem. Produce a summary of the `result` table.

```sql
SELECT COUNT(*)                 AS papers_marked,
       COUNT(DISTINCT student_id) AS students,
       ROUND(AVG(marks),2)      AS average,
       MIN(marks)               AS lowest,
       MAX(marks)               AS highest,
       SUM(marks)               AS total
FROM   result;
```

papers_marked	students	average	lowest	highest	total
12	7	76.00	52.00	91.00	912.00

Real-world interpretation. This is the header block of an examination-controller's report.

> Beware the empty set. `SUM` over zero qualifying rows returns **NULL**, not 0, and `AVG` returns NULL as well. Wrap with `COALESCE(SUM(marks), 0)` whenever a numeric result is required by the caller.

Aggregates with GROUP BY and DISTINCT

```sql
SELECT   dept_id,
```

```
        COUNT(*)                        AS students,
        COUNT(DISTINCT LEFT(student_id, 7)) AS admission_batches,
        ROUND(AVG(cgpa), 2)             AS avg_cgpa
FROM    student
GROUP BY dept_id;
```

12.2 String Functions

Function	MySQL	PostgreSQL	Result on `'Rahim Uddin'`
Join strings	`CONCAT(a, b)`	`a || b` or `CONCAT`	`CONCAT(name,' (CSE)')` → `Rahim Uddin (CSE)`
Upper case	`UPPER(s)`	`UPPER(s)`	`RAHIM UDDIN`
Lower case	`LOWER(s)`	`LOWER(s)`	`rahim uddin`
Length	`LENGTH(s)`, `CHAR_LENGTH(s)`	`LENGTH(s)`	11
Substring	`SUBSTRING(s, 1, 5)`	`SUBSTRING(s FROM 1 FOR 5)`	`Rahim`
Trim spaces	`TRIM(s)`, `LTRIM`, `RTRIM`	same	removes padding
Replace	`REPLACE(s, 'a', 'b')`	same	substitution
Position	`LOCATE(sub, s)`	`POSITION(sub IN s)`	index of substring
Pad	`LPAD(s, n, c)`, `RPAD`	same	fixed-width output
Left / right	`LEFT(s,n)`, `RIGHT(s,n)`	same	`LEFT(name,5)` → `Rahim`

```
SELECT student_id,
       UPPER(name)                              AS name_upper,
       CHAR_LENGTH(name)                        AS name_length,
       SUBSTRING(name, 1, LOCATE(' ', name)-1)  AS first_name,
       CONCAT(LEFT(name,1), '. ', SUBSTRING(name, LOCATE(' ', name)+1)) AS short_name,
       LOWER(email)                             AS email
FROM   student
WHERE  dept_id = 'CSE';
```

student_id	name_upper	name_length	first_name	short_name
ASH1901001	RAHIM UDDIN	11	Rahim	R. Uddin
ASH1901003	NILA RAHMAN	11	Nila	N. Rahman

Real-world interpretation. `short_name` is the form printed on an identity card; extracting `first_name` is how a mail-merge greeting is built.

> Note a design lesson hidden in this query: splitting a full name in SQL is fragile. Anyone with one name, three names or a title breaks it. The correct fix is at design time — store `first_name` and `last_name` separately, as Chapter 9 required when mapping a composite attribute.

12.3 Numeric Functions

Function	Meaning	Example	Result
`ROUND(x, d)`	Round to d decimals	`ROUND(76.456, 2)`	76.46
`CEILING(x)` / `CEIL(x)`	Smallest integer $\geq x$	`CEILING(76.1)`	77
`FLOOR(x)`	Largest integer $\leq x$	`FLOOR(76.9)`	76

Function	Meaning	Example	Result
`TRUNCATE(x, d)`	Cut off, no rounding	`TRUNCATE(76.789, 1)`	76.7
`ABS(x)`	Absolute value	`ABS(-12)`	12
`MOD(a, b)` or `a % b`	Remainder	`MOD(17, 5)`	2
`POWER(a, b)`	Exponentiation	`POWER(2, 10)`	1024
`SQRT(x)`	Square root	`SQRT(81)`	9

```sql
SELECT name, marks,
       ROUND(marks)            AS rounded,
       CEILING(marks)          AS ceiling,
       FLOOR(marks)            AS floor_val,
       ROUND(marks/100*4, 2)   AS grade_point
FROM   result r JOIN student s ON s.student_id = r.student_id
WHERE  r.exam_id = 1;
```

Real-world interpretation: converting raw marks into a grade point is exactly the arithmetic the examination software performs before a transcript is printed.

12.4 Date and Time Functions

Function	MySQL	PostgreSQL	Purpose
Current date	`CURDATE()`	`CURRENT_DATE`	Today
Current timestamp	`NOW()`	`NOW()`	Date and time
Extract part	`YEAR(d)`, `MONTH(d)`, `DAY(d)`	`EXTRACT(YEAR FROM d)`	Component
Difference in days	`DATEDIFF(d1, d2)`	`d1 - d2`	Day count
Age in years	`TIMESTAMPDIFF(YEAR, dob, CURDATE())`	`AGE(dob)`	Whole years
Add interval	`DATE_ADD(d, INTERVAL 7 DAY)`	`d + INTERVAL '7 days'`	Shift a date
Format	`DATE_FORMAT(d, '%d-%m-%Y')`	`TO_CHAR(d, 'DD-MM-YYYY')`	Display

```sql
SELECT name,
       dob,
       TIMESTAMPDIFF(YEAR, dob, CURDATE())      AS age_years,
       YEAR(admitted_on)                        AS admission_year,
       DATE_FORMAT(admitted_on, '%d %M %Y')     AS admitted_display,
       DATEDIFF(CURDATE(), admitted_on)         AS days_since_admission
FROM   student
ORDER  BY dob;
```

Real-world interpretation. `age_years` decides eligibility for an age-limited scholarship; `days_since_admission` feeds a maximum-duration-of-study rule.

> A performance warning that matters in practice: wrapping a column in a function disables any index on it. `WHERE YEAR(admitted_on) = 2020` cannot use an index on `admitted_on`, whereas the equivalent range predicate can:

>

```sql
WHERE admitted_on >= '2020-01-01' AND admitted_on < '2021-01-01'
```

12.5 Conditional and NULL Functions

Function	Meaning
`IFNULL(x, y)` (MySQL) / `COALESCE(x, y, …)`	First non-NULL argument
`NULLIF(a, b)`	NULL if a = b, else a — useful to avoid division by zero
`IF(cond, a, b)` (MySQL)	Two-way conditional
`CASE … WHEN … THEN … ELSE … END`	Multi-way conditional, standard SQL

```sql
SELECT name,
       COALESCE(cgpa, 0)                    AS cgpa_or_zero,
       CASE
           WHEN cgpa IS NULL      THEN 'Not yet graded'
           WHEN cgpa >= 3.75      THEN 'Outstanding'
           WHEN cgpa >= 3.50      THEN 'Excellent'
           WHEN cgpa >= 3.00      THEN 'Very Good'
           WHEN cgpa >= 2.50      THEN 'Good'
           ELSE                        'Needs improvement'
       END                              AS classification
FROM   student
ORDER  BY cgpa DESC;
```

name	cgpa_or_zero	classification
Tania Sultana	3.95	Outstanding
Nila Rahman	3.90	Outstanding
Rahim Uddin	3.75	Outstanding
Fahmida Nabi	3.70	Excellent
Karim Mia	3.60	Excellent
…	…	…
Mitu Barua	0.00	Not yet graded

`CASE` is evaluated top to bottom and stops at the first true branch, which is why the NULL test must come first and the thresholds must descend.

A useful idiom: conditional aggregation

```sql
SELECT dept_id,
       COUNT(*)                                          AS total,
       SUM(CASE WHEN cgpa >= 3.50 THEN 1 ELSE 0 END)     AS high_achievers,
       SUM(CASE WHEN cgpa IS NULL THEN 1 ELSE 0 END)     AS ungraded
FROM   student GROUP BY dept_id;
```

This produces several counts over different conditions in a single pass over the table — far cheaper than running three separate queries.

Chapter Summary

- Aggregate functions collapse many rows into one value per group and ignore NULLs; `COUNT(*)` is the only one that counts every row.
- `SUM` and `AVG` return NULL, not zero, over an empty set.
- String functions build display values, but splitting names in SQL signals a design flaw that should

have been fixed at mapping time.

- Numeric functions include `ROUND`, `CEILING`, `FLOOR`, `TRUNCATE` and `MOD`, which differ in how they discard fractions.
- Date functions extract, difference, shift and format dates; applying a function to a column in `WHERE` prevents index use.
- `COALESCE`, `NULLIF` and `CASE` handle missing data and multi-way conditions; `CASE` inside `SUM` gives conditional aggregation in one pass.

Key Terms

Aggregate function · Scalar function · `COUNT` · `SUM` · `AVG` · `MIN` · `MAX` · `COUNT(DISTINCT)` · `CONCAT` · `SUBSTRING` · `UPPER` / `LOWER` · `TRIM` · `ROUND` · `CEILING` · `FLOOR` · `TRUNCATE` · `MOD` · `CURDATE` · `DATEDIFF` · `TIMESTAMPDIFF` · `DATE_FORMAT` · `COALESCE` · `IFNULL` · `NULLIF` · `CASE` · Conditional aggregation · Sargable predicate

Common Mistakes

1. Expecting `SUM` to return 0 for an empty set.
2. Assuming `AVG` divides by the total row count when NULLs are present.
3. Using `ROUND` where `FLOOR` is meant, or the reverse, when computing pages or batches.
4. Writing `WHERE YEAR(date_col) = 2020`, which disables the index.
5. Ordering `CASE` branches so that a broad condition hides a narrow one.
6. Forgetting that `CASE` returns NULL when no branch matches and `ELSE` is absent.
7. Using `IFNULL` (MySQL-only) in code intended to be portable; prefer `COALESCE`.
8. Splitting names with string functions instead of storing them separately.

Solved Problems

Problem 12.1 — A column has values 10, 20, NULL, 40. Give `COUNT(*)`, `COUNT(col)`, `SUM(col)`, `AVG(col)` and `AVG(COALESCE(col,0))`.

Solution. `COUNT(*)` = 4. `COUNT(col)` = 3. `SUM(col)` = 70. `AVG(col)` = 70 ÷ 3 = 23.33. `AVG(COALESCE(col,0))` = 70 ÷ 4 = 17.50. The last two differ precisely because the first ignores the NULL row and the second converts it to a real zero — a semantic decision that should be made deliberately, not by accident.

Problem 12.2 — Write a query classifying every marks row as Pass (≥ 40) or Fail, and count each class per examination.

Solution.

```
SELECT exam_id,
       SUM(CASE WHEN marks >= 40 THEN 1 ELSE 0 END) AS passed,
       SUM(CASE WHEN marks <  40 THEN 1 ELSE 0 END) AS failed,
       ROUND(100.0 * SUM(CASE WHEN marks >= 40 THEN 1 ELSE 0 END) / COUNT(*), 1) AS
    ↪ pass_rate
FROM   result
GROUP BY exam_id
ORDER BY pass_rate;
```

Multiplying by `100.0` rather than `100` forces decimal arithmetic; with integer division the pass rate would be truncated to 0 or 1 in some systems.

Problem 12.3 — Compute each student's age and flag those over 22.

Solution.

```
SELECT name, dob,
       TIMESTAMPDIFF(YEAR, dob, CURDATE()) AS age,
       CASE WHEN TIMESTAMPDIFF(YEAR, dob, CURDATE()) > 22 THEN 'Yes' ELSE 'No' END AS
    ↪ over_22
FROM   student ORDER BY dob;
```

`TIMESTAMPDIFF(YEAR, …)` counts completed years, which is the legally correct definition of age; `DATEDIFF(...)/365` is wrong because of leap years.

Problem 12.4 — Explain why `WHERE YEAR(admitted_on) = 2020` is slower than a range predicate on a large table.

Solution. An index on `admitted_on` stores the dates themselves in sorted order. A range predicate lets the engine seek directly to the first value ≥ 2020-01-01 and stop at the first ≥ 2021-01-01, touching only matching entries. Once the column is wrapped in `YEAR(...)`, the index entries are no longer in the order of the expression's result, so the engine must evaluate the function for every row — a full scan. A predicate that can use an index in this way is called **sargable**; the rewrite is:

```
WHERE admitted_on >= '2020-01-01' AND admitted_on < '2021-01-01'
```

Problem 12.5 — Build a formatted transcript line: `"ASH1901001 | Rahim Uddin | CSE | 3.75 (93.8%)"`.

Solution.

```
SELECT CONCAT(student_id, ' | ', RPAD(name, 20, ' '), ' | ', dept_id, ' | ',
              COALESCE(CAST(cgpa AS CHAR), 'N/A'), ' (',
              COALESCE(CAST(ROUND(cgpa*25,1) AS CHAR), '--'), '%)') AS transcript_line
FROM   student ORDER BY student_id;
```

`COALESCE` is essential: concatenating anything with NULL yields NULL in standard SQL and in MySQL's `CONCAT`, so one ungraded student would blank out the entire line.

Problem 12.6 — Give a query that avoids division by zero when computing the pass percentage of a course with no results.

Solution.

```
SELECT course_id,
       COUNT(*) AS attempts,
       ROUND(100.0 * SUM(CASE WHEN grade <> 'F' THEN 1 ELSE 0 END)
             / NULLIF(COUNT(*), 0), 1) AS pass_pct
FROM   enrollment GROUP BY course_id;
```

`NULLIF(COUNT(*), 0)` turns a zero denominator into NULL, and division by NULL yields NULL instead of raising an error — the standard defensive idiom.

Review Questions

1. Distinguish aggregate from scalar functions with examples.
2. Explain how each aggregate function handles NULL.
3. Why does `SUM` return NULL over an empty set, and how is that handled?
4. Compare `ROUND`, `CEILING`, `FLOOR` and `TRUNCATE`.
5. Explain `COALESCE`, `IFNULL` and `NULLIF` with examples.

6. Write and explain a `CASE` expression classifying CGPA.
7. What is conditional aggregation and why is it efficient?
8. Explain why functions on columns in `WHERE` prevent index use.
9. Give three date functions and their MySQL and PostgreSQL forms.
10. Why is splitting a name with string functions a symptom of a design problem?

Multiple Choice Questions

1. `COUNT(*)` on a table with NULLs returns (a) non-NULL rows (b) all rows (c) distinct rows (d) zero.
2. `SUM` over an empty set returns (a) 0 (b) NULL (c) an error (d) 1.
3. `CEILING(76.1)` is (a) 76 (b) 77 (c) 76.1 (d) 76.0.
4. `MOD(17,5)` is (a) 3 (b) 2 (c) 3.4 (d) 0.
5. `COALESCE(NULL, NULL, 7, 9)` returns (a) NULL (b) 7 (c) 9 (d) 16.
6. `NULLIF(5,5)` returns (a) 5 (b) 0 (c) NULL (d) TRUE.
7. A `CASE` with no matching branch and no `ELSE` returns (a) 0 (b) NULL (c) an error (d) the first branch.
8. Which prevents index use? (a) `WHERE d >= '2020-01-01'` (b) `WHERE YEAR(d)=2020` (c) `WHERE d IS NULL` (d) `WHERE id=5`.
9. `COUNT(DISTINCT col)` counts (a) all rows (b) distinct values including NULL (c) distinct non-NULL values (d) NULLs only.
10. In MySQL, concatenating a string with NULL gives (a) the string (b) NULL (c) an empty string (d) an error.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(b) 6-(c) 7-(b) 8-(b) 9-(c) 10-(b)

Short Questions

1. Name the five aggregate functions.
2. What does `COUNT(DISTINCT x)` do with NULLs?
3. Give the MySQL and PostgreSQL forms of string concatenation.
4. What is a sargable predicate?
5. Which function returns the first non-NULL argument?
6. How do you avoid division by zero in SQL?
7. Which function gives completed years of age?
8. What is conditional aggregation?
9. What does `TRUNCATE(76.789, 1)` return?
10. Why prefer `COALESCE` to `IFNULL`?

Long / Descriptive Questions

1. Explain all aggregate functions with NULL semantics and worked examples on `university_db`.
2. Discuss string, numeric and date functions with MySQL and PostgreSQL equivalents and practical uses.
3. Explain `CASE`, `COALESCE` and `NULLIF` in depth, with conditional aggregation examples.
4. Explain why functions applied to columns hurt performance, and rewrite three non-sargable predicates.
5. Build a complete transcript-generation query using string, numeric, date and conditional functions, explaining each part.

Practical Exercises

1. Write a query producing, per department, total students, high achievers and ungraded students in one pass.
2. Produce a formatted merit list with padded names, rounded percentages and a classification column.
3. Compute each student's age and days since admission, and list those admitted more than four years ago.
4. Demonstrate the difference in execution plan between `WHERE YEAR(admitted_on)=2020` and the range form using `EXPLAIN`.
5. Write a query that safely computes a pass percentage for courses that may have no results.

CHAPTER 13

SQL Joins

A join combines rows from two or more relations by matching values. It is the operation that makes normalized design practical: data is stored once, in separate tables, and reassembled on demand.

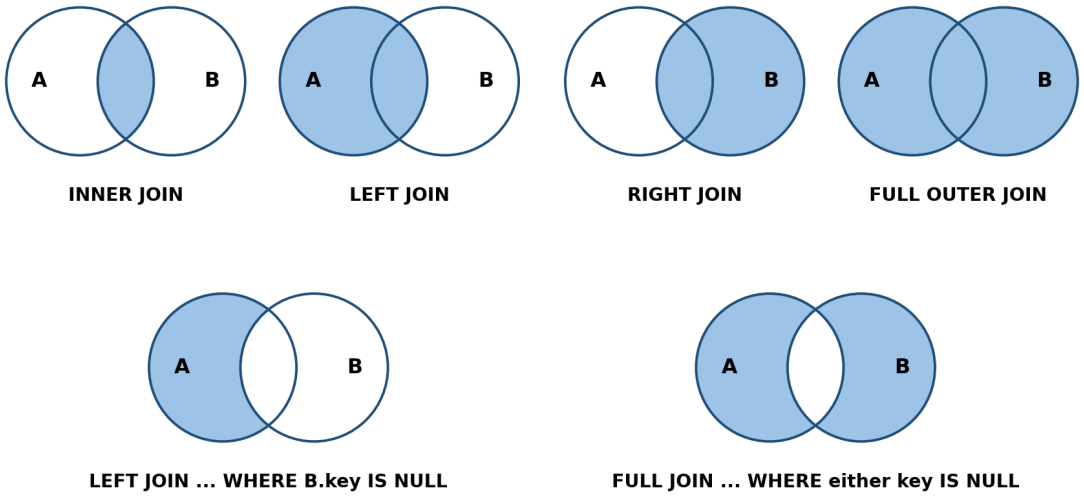


Figure 13.1: The join types as set diagrams (shaded region = rows returned)

13.1 Why Joins Exist

Because `student` stores `dept_id` but not the department name, a report listing students with their department names must fetch values from two tables:

```
SELECT s.name, d.dept_name
FROM   student s
JOIN   department d ON s.dept_id = d.dept_id;
```

The alternative — storing `dept_name` in `student` — would repeat the name in every student row, which is exactly the redundancy that normalization forbids (Chapter 19). The join is the price paid for a clean design, and it is a low price because the DBMS optimizes it aggressively (Chapter 26).

13.2 INNER JOIN

Returns only rows that have a match on both sides.

Problem. List each student with their department name.

```
SELECT s.student_id, s.name, d.dept_name
FROM   student s
INNER  JOIN department d ON s.dept_id = d.dept_id
ORDER  BY s.student_id;
```

student_id	name	dept_name
ASH1901001	Rahim Uddin	Computer Science and Engineering
ASH1901002	Karim Mia	Electrical and Electronic Engineering
ASH1901005	Jubayer Hossain	Mechanical Engineering

Explanation. For every `student` row, the engine finds `department` rows where `dept_id` matches, and emits the combined row. Unmatched rows on either side are discarded. `INNER` is optional — a bare `JOIN` means inner join.

Real-world interpretation. Every list screen in a university portal that shows a human-readable department, course or instructor name is executing an inner join.

13.3 LEFT OUTER JOIN

Returns every row of the left table; where no match exists on the right, the right-hand columns are NULL.

Problem. List every course with its number of enrollments, **including courses nobody has taken**.

```
SELECT   c.course_code, c.title, COUNT(e.enrollment_id) AS enrolled
FROM     course c
LEFT     JOIN enrollment e ON e.course_id = c.course_id
GROUP BY c.course_id, c.course_code, c.title
ORDER BY enrolled DESC, c.course_code;
```

course_code	title	enrolled
CSE3201	Database Management Systems	4
CSE2101	Data Structures	3
BBA1101	Principles of Management	2
CSE3203	Operating Systems	2
EEE3101	Digital Electronics	2
ME2101	Engineering Mechanics	2
CSE4101	Machine Learning	1
EEE3103	Signals and Systems	1

Explanation. With an inner join, a course with no enrollment would vanish from the report — and a course with no students is exactly what the report exists to find. Note `COUNT(e.enrollment_id)`, not `COUNT(*)`: for an unmatched course the right-hand columns are NULL, so `COUNT(e.enrollment_id)` correctly gives 0, whereas `COUNT(*)` would give 1 by counting the NULL-padded row.

Real-world interpretation. This report tells a curriculum committee which courses to withdraw.

Finding non-matching rows (anti-join)

Problem. Which students have never enrolled in anything?

```
SELECT s.student_id, s.name
FROM   student s
LEFT   JOIN enrollment e ON e.student_id = s.student_id
WHERE  e.enrollment_id IS NULL;
```

The `WHERE … IS NULL` test keeps only the NULL-padded rows — those with no match. This **left anti-join** idiom is one of the most useful patterns in SQL: find orphans, find unbilled orders, find inactive members.

13.4 RIGHT OUTER JOIN

The mirror image: every row of the right table is kept.

```
SELECT d.dept_name, s.name
FROM   student s
RIGHT  JOIN department d ON s.dept_id = d.dept_id;
```

Every department appears even if it has no students. Any right join can be rewritten as a left join by swapping the tables, and most practitioners do exactly that for readability — a query that reads top-to-bottom as "keep everything from the first table" is easier to follow.

13.5 FULL OUTER JOIN

Keeps unmatched rows from **both** sides.

```
-- PostgreSQL, Oracle, SQL Server
SELECT s.name, d.dept_name
FROM   student s
FULL   OUTER JOIN department d ON s.dept_id = d.dept_id;
```

MySQL has no `FULL OUTER JOIN`; it is emulated with a union of the two outer joins:

```
SELECT s.name, d.dept_name FROM student s LEFT JOIN department d ON s.dept_id = d.
    ↪ dept_id
UNION
SELECT s.name, d.dept_name FROM student s RIGHT JOIN department d ON s.dept_id = d.
    ↪ dept_id;
```

13.6 CROSS JOIN

Returns the Cartesian product: every row of A paired with every row of B. With 10 students and 4 departments the result has 40 rows.

```
SELECT s.name, d.dept_name FROM student s CROSS JOIN department d;
```

Legitimate uses are narrow but real: generating all (course, classroom) pairs to check scheduling feasibility, or building a calendar grid of all (date, room) slots.

```
-- every course paired with every room large enough to hold it
SELECT c.course_code, r.room_id, r.capacity
FROM   course c CROSS JOIN classroom r
WHERE  r.capacity >= 45
ORDER  BY c.course_code, r.capacity;
```

> The accidental cross join is the classic performance disaster: omitting the `ON` condition in an old-style comma join turns a 10,000 × 10,000 row query into 100 million rows. Always use explicit `JOIN … ON` syntax, where a missing condition is a visible mistake rather than an invisible one.

13.7 SELF JOIN

A table joined to itself under two aliases. It is how hierarchies and same-table comparisons are queried.

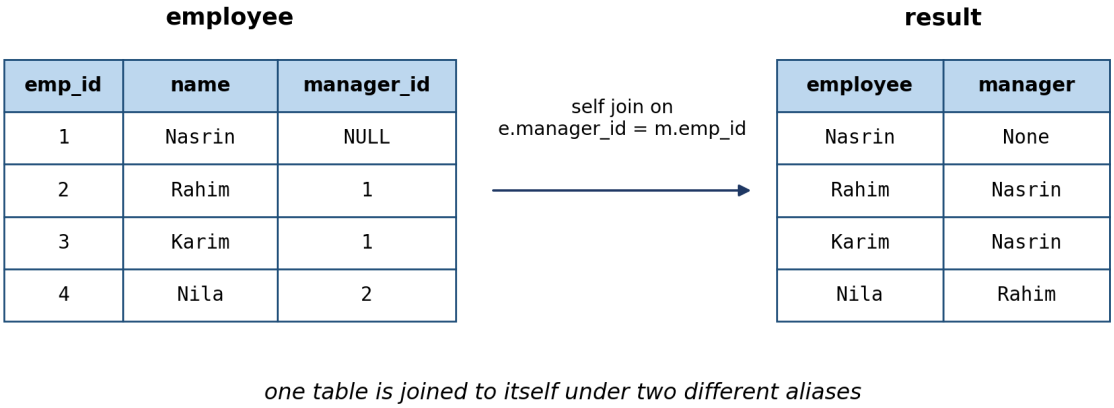


Figure 13.2: A self join resolving an employee–manager hierarchy

Problem. Pair each CSE student with every other CSE student who has a higher CGPA.

```sql
SELECT a.name AS student, b.name AS scores_higher
FROM   student a
JOIN   student b ON a.dept_id = b.dept_id AND b.cgpa > a.cgpa
WHERE  a.dept_id = 'CSE'
ORDER  BY a.name;
```

Explanation. Aliases `a` and `b` make the same physical table behave as two logical tables. The condition `b.cgpa > a.cgpa` both filters and prevents a row from pairing with itself.

Real-world interpretation. This is how a rank position is computed without window functions: the rank of a student is one more than the number of students ahead of them.

```sql
SELECT a.name, a.cgpa, COUNT(b.student_id) + 1 AS rank_position
FROM   student a
LEFT   JOIN student b ON b.cgpa > a.cgpa
WHERE  a.cgpa IS NOT NULL
GROUP  BY a.student_id, a.name, a.cgpa
ORDER  BY rank_position;
```

13.8 NATURAL JOIN and USING

```sql
SELECT * FROM student NATURAL JOIN department;      -- joins on ALL same-named columns
SELECT * FROM student JOIN department USING (dept_id);   -- joins on the named column
```

`NATURAL JOIN` is convenient and dangerous: it silently joins on every column that happens to share a name. Adding a `building` column to `student` would change the join condition and the result without a single line of the query being edited. Use `USING` when column names match, and `ON` otherwise.

13.9 Multi-Table Joins

Problem. Produce a full result sheet: student name, department, course title, semester, grade and marks.

```sql
SELECT s.name            AS student,
       d.dept_name       AS department,
       c.course_code,
       c.title,
       e.semester,
       e.grade,
```

```
        r.marks
FROM    enrollment e
JOIN    student    s ON s.student_id = e.student_id
JOIN    department d ON d.dept_id    = s.dept_id
JOIN    course     c ON c.course_id  = e.course_id
LEFT    JOIN examination x ON x.course_id = c.course_id
LEFT    JOIN result      r ON r.exam_id = x.exam_id AND r.student_id = s.student_id
ORDER   BY s.name, c.course_code;
```

Explanation. Five tables are joined. The first three are inner joins because every enrollment must have a student, a department and a course. The last two are **left** joins because an examination may not have been held yet, and a student may not have appeared for it — and those rows must still show on the sheet with blank marks.

This mixing of inner and outer joins is the single most important skill in practical SQL: an inner join asserts "this relationship must exist", and using one where an outer join belongs silently deletes rows from a report.

13.10 Join Comparison

Join	Keeps unmatched left rows?	Keeps unmatched right rows?	Typical use
`INNER JOIN`	No	No	Normal lookup of related data
`LEFT JOIN`	Yes	No	Reports that must list every left-hand row
`RIGHT JOIN`	No	Yes	Mirror of left join; usually rewritten
`FULL OUTER JOIN`	Yes	Yes	Reconciling two independent sources
`CROSS JOIN`	Not applicable	Not applicable	Generating all combinations
`SELF JOIN`	Depends on the join used	Depends	Hierarchies, ranking, comparisons

Chapter Summary

- A join reassembles normalized data by matching values; it is what allows a fact to be stored exactly once.
- Inner joins keep only matched rows; left and right joins preserve one side; full outer joins preserve both.
- `LEFT JOIN … WHERE right.key IS NULL` is the anti-join idiom for finding rows with no match.
- In an outer join, count a right-hand column rather than `*`, or unmatched rows are counted as one.
- Cross joins produce the Cartesian product and are almost always accidental; explicit `JOIN … ON` syntax makes the accident visible.
- Self joins use two aliases of one table and answer hierarchy, ranking and comparison questions.
- `NATURAL JOIN` should be avoided; use `USING` or `ON`.

Key Terms

Join · Join predicate · `INNER JOIN` · `LEFT OUTER JOIN` · `RIGHT OUTER JOIN` · `FULL OUTER JOIN` · `CROSS JOIN` · Cartesian product · `SELF JOIN` · Alias · Anti-join · `NATURAL JOIN` · `USING` · `ON` · NULL padding · Multi-table join

Common Mistakes

1. Using an inner join where an outer join is required, silently dropping rows from a report.
2. Using `COUNT(*)` in an outer-joined aggregate, so unmatched rows count as 1 instead of 0.
3. Placing a condition on the right table in `WHERE` instead of in `ON`, which converts a left join back into an inner join.
4. Forgetting the join condition and producing a Cartesian product.
5. Using `NATURAL JOIN` and having the result change when a column is added.
6. Omitting aliases in a self join.
7. Joining on columns of different types, defeating index use.
8. Assuming the join order written is the order executed — the optimizer decides that.

The third mistake deserves a demonstration, because it is subtle:

```sql
-- intended: every course, with enrollments in the 5th semester only
SELECT c.course_code, COUNT(e.enrollment_id)
FROM   course c LEFT JOIN enrollment e ON e.course_id = c.course_id
WHERE  e.semester = '5th'                -- WRONG: discards the NULL-padded rows
GROUP  BY c.course_code;

-- correct: the condition belongs in the ON clause
SELECT c.course_code, COUNT(e.enrollment_id)
FROM   course c LEFT JOIN enrollment e
       ON e.course_id = c.course_id AND e.semester = '5th'
GROUP  BY c.course_code;
```

Solved Problems

Problem 13.1 — Table A has 5 rows, table B has 3 rows, and 2 rows match. How many rows does each join return?

Solution. Inner join: 2. Left join: 5 (2 matched + 3 unmatched, NULL-padded). Right join: 3 (2 matched + 1 unmatched). Full outer join: 6 (2 matched + 3 left-only + 1 right-only). Cross join: $5 \times 3 = 15$. This counting question appears in almost every examination on joins.

Problem 13.2 — List all instructors together with the number of courses their department offers, including departments with no courses.

Solution.

```sql
SELECT   i.name, d.dept_name, COUNT(c.course_id) AS dept_courses
FROM     instructor i
JOIN     department d ON d.dept_id = i.dept_id
LEFT     JOIN course c ON c.dept_id = d.dept_id
GROUP BY i.instructor_id, i.name, d.dept_name
ORDER BY dept_courses DESC, i.name;
```

The instructor-to-department join is inner because every instructor has a department (the foreign key is `NOT NULL`); the department-to-course join is left because a department might offer nothing yet.

Problem 13.3 — Find courses that have never been enrolled in, in two different ways.

Solution.

```sql
-- (a) anti-join
SELECT c.course_code, c.title
FROM   course c LEFT JOIN enrollment e ON e.course_id = c.course_id
```

```sql
WHERE  e.enrollment_id IS NULL;

-- (b) NOT EXISTS (Chapter 14)
SELECT c.course_code, c.title
FROM   course c
WHERE  NOT EXISTS (SELECT 1 FROM enrollment e WHERE e.course_id = c.course_id);
```

Both are correct. `NOT EXISTS` is usually the clearer statement of intent and often the faster plan, because it can stop at the first matching row; the anti-join must build the full outer result before filtering.

Problem 13.4 — Explain why this query returns fewer rows than expected and correct it.

```sql
SELECT s.name, e.grade
FROM   student s LEFT JOIN enrollment e ON e.student_id = s.student_id
WHERE  e.grade = 'A';
```

Solution. The `WHERE` clause is applied after the join. For a student with no enrollment, `e.grade` is NULL, and `NULL = 'A'` is unknown, so the row is discarded — the left join has been silently converted to an inner join. If the intent is "every student, showing A grades where they exist", the condition must move into `ON`:

```sql
SELECT s.name, e.grade
FROM   student s LEFT JOIN enrollment e
       ON e.student_id = s.student_id AND e.grade = 'A';
```

If instead the intent really is "only students with an A", the original `WHERE` is right but the join should be written as an inner join to say so.

Problem 13.5 — Write a self join listing pairs of students in the same department with a CGPA difference of less than 0.1.

Solution.

```sql
SELECT a.name AS student_1, b.name AS student_2,
       ROUND(ABS(a.cgpa - b.cgpa), 2) AS gap
FROM   student a
JOIN   student b ON a.dept_id = b.dept_id
                AND a.student_id < b.student_id        -- each pair once, never self
WHERE  ABS(a.cgpa - b.cgpa) < 0.1
ORDER  BY gap;
```

The condition `a.student_id < b.student_id` is the standard trick: it excludes self-pairs and keeps each unordered pair exactly once instead of twice.

Problem 13.6 — Produce a matrix of every course against every classroom that can hold at least 60 students.

Solution.

```sql
SELECT c.course_code, r.room_id, r.capacity
FROM   course c CROSS JOIN classroom r
WHERE  r.capacity >= 60
ORDER  BY c.course_code, r.room_id;
```

This is a legitimate cross join: the task genuinely requires all combinations, as the scheduler must consider every course in every sufficiently large room.

Problem 13.7 — A five-table join runs slowly. Give four things to check.

Solution. (i) Are the join columns indexed? Foreign keys are not automatically indexed in PostgreSQL or SQL Server. (ii) Do the joined columns have identical data types and collations? A `CHAR(10)` joined to a `VARCHAR(10)` with a different collation may force a conversion that disables the index. (iii) Is the query selecting more columns or rows than needed — can filters be applied earlier? (iv) What does `EXPLAIN` report: nested loop, hash join or merge join, and are the row estimates close to reality? Bad estimates usually mean stale statistics (Chapter 26). A fifth check: is any outer join used where an inner join would do, since outer joins constrain the optimizer's freedom to reorder.

Review Questions

1. Why are joins necessary in a normalized database?
2. Explain inner, left, right, full outer, cross and self joins with set diagrams.
3. What is NULL padding in an outer join?
4. Explain the anti-join idiom and give a use case.
5. Why should `COUNT(column)` be used instead of `COUNT(*)` in outer-joined aggregates?
6. Explain the difference between putting a condition in `ON` and in `WHERE` for a left join.
7. Why is `NATURAL JOIN` discouraged?
8. Give two legitimate uses of a cross join.
9. How is a ranking computed using a self join?
10. Compare all join types in a table.

Multiple Choice Questions

1. A join that keeps unmatched rows from both tables is (a) inner (b) left (c) full outer (d) cross.
2. A join with no condition produces a (a) natural join (b) Cartesian product (c) self join (d) anti-join.
3. `LEFT JOIN … WHERE b.id IS NULL` finds (a) matches (b) non-matching left rows (c) duplicates (d) NULL columns.
4. A and B have 4 and 6 rows with 3 matching; a full outer join returns (a) 3 (b) 7 (c) 24 (d) 10.
5. Placing a right-table filter in `WHERE` of a left join (a) has no effect (b) makes it effectively inner (c) causes an error (d) makes it a cross join.
6. A self join requires (a) two tables (b) table aliases (c) a cross join (d) a subquery.
7. Which is unavailable in MySQL? (a) `LEFT JOIN` (b) `FULL OUTER JOIN` (c) `CROSS JOIN` (d) `INNER JOIN`.
8. `USING (dept_id)` is equivalent to (a) `ON a.dept_id = b.dept_id` (b) a cross join (c) a natural join on all columns (d) a self join.
9. In an outer join, unmatched columns are filled with (a) 0 (b) empty strings (c) NULL (d) defaults.
10. `COUNT(*)` in an outer-joined group counts unmatched rows as (a) 0 (b) 1 (c) NULL (d) an error.

Answers: 1-(c) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(a) 9-(c) 10-(b)

Short Questions

1. What does an inner join discard?
2. Define the Cartesian product.
3. Write the anti-join pattern.
4. Why alias tables in a self join?
5. How is a full outer join emulated in MySQL?
6. Name one risk of `NATURAL JOIN`.
7. What is NULL padding?

8. How many rows does a cross join of 7 and 8 rows produce?
9. Which clause should hold a right-table filter in a left join?
10. Give one legitimate use of a cross join.

Long / Descriptive Questions

1. Explain all join types with diagrams, sample tables, SQL, output and real-life examples.
2. Discuss the difference between `ON` and `WHERE` conditions in outer joins, with examples of the bug it causes.
3. Explain self joins in depth: hierarchies, ranking and pairwise comparison, with worked queries.
4. Write and explain a five-table report query mixing inner and outer joins, justifying each choice.
5. Discuss join performance: indexing, data types, join algorithms and the effect of outer joins on optimization.

Practical Exercises

1. Write queries using all six join types on `university_db` and record the row counts of each.
2. Find courses with no enrollments and students with no enrollments, using both the anti-join and `NOT EXISTS`.
3. Demonstrate the `ON` versus `WHERE` bug by writing both versions of the same left join and comparing results.
4. Compute a CGPA ranking using a self join, and verify it against `ORDER BY`.
5. Produce a complete result sheet joining five tables, ensuring that courses without examinations still appear.

CHAPTER 14

Subqueries and Set Operations

A **subquery** (inner query, nested query) is a `SELECT` written inside another SQL statement. It lets a query depend on a value, a list or an existence test that itself has to be computed from the data.

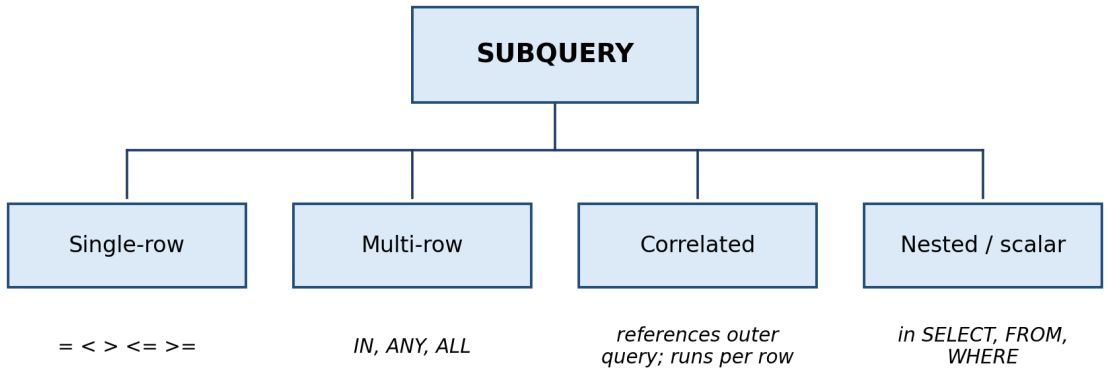


Figure 14.1: Classification of subqueries

14.1 Where a Subquery May Appear

Position	Must return	Example
`WHERE`	A scalar, a list, or an existence test	`WHERE cgpa > (SELECT AVG(cgpa) FROM student)`
`HAVING`	A scalar	`HAVING AVG(marks) > (SELECT AVG(marks) FROM result)`
`SELECT`	Exactly one value per row (scalar subquery)	`SELECT name, (SELECT COUNT(*) …) AS courses`
`FROM`	A table (a derived table; must be aliased)	`FROM (SELECT dept_id, AVG(cgpa) a FROM student GROUP BY dept_id) t`
`INSERT`, `UPDATE`, `DELETE`	Any of the above	`DELETE FROM enrollment WHERE course_id IN ( …)`

14.2 Single-Row Subqueries

A single-row subquery returns exactly one value and is compared with `=`, `<`, `>`, `<=`, `>=` or `<>`.

Problem. List students whose CGPA is above the university average.

```sql
SELECT name, cgpa
FROM   student
WHERE  cgpa > (SELECT AVG(cgpa) FROM student)
ORDER  BY cgpa DESC;
```

Explanation. The inner query computes 3.50 (the average of the nine non-NULL CGPAs) once; the outer query compares each row against it.

name	cgpa
Tania Sultana	3.95
Nila Rahman	3.90
Rahim Uddin	3.75
Fahmida Nabi	3.70
Karim Mia	3.60

Real-world interpretation. This is the query behind an "above average performers" report — and note that it *cannot* be written without a subquery, because `WHERE cgpa > AVG(cgpa)` is illegal: aggregates are not permitted in `WHERE`.

> If a single-row subquery unexpectedly returns several rows, the DBMS raises an error ("subquery returns more than 1 row"). The usual cause is a missing aggregate or a missing `WHERE` in the inner query.

14.3 Multi-Row Subqueries: IN, ANY, ALL

Operator	Meaning
`IN (subquery)`	True if the value equals **any** returned value
`NOT IN`	True if it equals none — **dangerous with NULLs**
`= ANY` / `> ANY`	True if the comparison holds for **at least one** value
`> ALL` / `< ALL`	True if the comparison holds for **every** value

```sql
-- students enrolled in any CSE course
SELECT DISTINCT s.name
FROM   student s
WHERE  s.student_id IN (
           SELECT e.student_id FROM enrollment e
           JOIN course c ON c.course_id = e.course_id
           WHERE c.dept_id = 'CSE');

-- instructors earning more than every lecturer
SELECT name, designation, salary
FROM   instructor
WHERE  salary > ALL (SELECT salary FROM instructor WHERE designation = 'Lecturer');

-- instructors earning more than at least one professor
SELECT name, salary
FROM   instructor
WHERE  salary > ANY (SELECT salary FROM instructor WHERE designation = 'Professor');
```

`> ALL` is equivalent to `> MAX(...)`, and `> ANY` to `> MIN(...)`. Stating that equivalence in an examination demonstrates real understanding.

The NOT IN / NULL trap

```sql
-- If ANY course_id in the inner list is NULL, this returns NOTHING.
SELECT name FROM course WHERE course_id NOT IN (SELECT course_id FROM enrollment);
```

The reason: `x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`, and the last comparison is *unknown*, so the whole conjunction can never be true. `NOT EXISTS` does not have this problem and should be preferred:

```sql
SELECT name FROM course c
WHERE  NOT EXISTS (SELECT 1 FROM enrollment e WHERE e.course_id = c.course_id);
```

14.4 Correlated Subqueries

A **correlated** subquery references a column from the outer query, so it must be evaluated once per outer row.

Problem. List students whose CGPA is above the average **of their own department**.

```sql
SELECT s.name, s.dept_id, s.cgpa
FROM   student s
WHERE  s.cgpa > (SELECT AVG(s2.cgpa)
                 FROM   student s2
                 WHERE  s2.dept_id = s.dept_id)     -- correlation: s.dept_id
ORDER  BY s.dept_id, s.cgpa DESC;
```

name	dept_id	cgpa
Tania Sultana	CSE	3.95
Nila Rahman	CSE	3.90
Rahim Uddin	CSE	3.75
Karim Mia	EEE	3.60
Fahmida Nabi	ME	3.70

Basis	Non-correlated	Correlated
References outer query	No	Yes
Executed	Once	Conceptually once per outer row
Can run standalone	Yes	No
Typical cost	Low	Higher, unless the optimizer rewrites it

Modern optimizers often rewrite a correlated subquery into a join or a grouped derived table, so the "once per row" cost is conceptual rather than guaranteed. The equivalent join form is usually faster and worth knowing:

```sql
SELECT s.name, s.dept_id, s.cgpa
FROM   student s
JOIN   (SELECT dept_id, AVG(cgpa) AS avg_cgpa FROM student GROUP BY dept_id) d
       ON d.dept_id = s.dept_id
WHERE  s.cgpa > d.avg_cgpa;
```

14.5 EXISTS and NOT EXISTS

`EXISTS` tests whether the subquery returns at least one row. It returns true or false and never examines the subquery's column values, which is why `SELECT 1` is the conventional body.

```sql
-- students who have at least one A grade
SELECT s.name FROM student s
WHERE  EXISTS (SELECT 1 FROM enrollment e
               WHERE e.student_id = s.student_id AND e.grade = 'A');

-- students who have never failed
SELECT s.name FROM student s
```

```sql
WHERE  NOT EXISTS (SELECT 1 FROM enrollment e
                   WHERE e.student_id = s.student_id AND e.grade = 'F');
```

Basis	IN	EXISTS
Compares	Values	Existence of rows
Subquery returns	A column of values	Any row at all
NULL behaviour	`NOT IN` breaks with NULLs	Safe
Correlation	Usually non-correlated	Usually correlated
Best when	The inner list is small	The inner table is large and indexed

14.6 Derived Tables and Common Table Expressions

A subquery in `FROM` is a **derived table** and must be aliased:

```sql
SELECT t.dept_id, t.avg_cgpa
FROM   (SELECT dept_id, ROUND(AVG(cgpa),2) AS avg_cgpa
        FROM   student GROUP BY dept_id) AS t
WHERE  t.avg_cgpa > 3.4;
```

A **common table expression** (CTE), available in MySQL 8 and PostgreSQL, expresses the same idea more readably and may be referenced several times:

```sql
WITH dept_avg AS (
    SELECT dept_id, ROUND(AVG(cgpa),2) AS avg_cgpa, COUNT(*) AS n
    FROM   student GROUP BY dept_id
)
SELECT d.dept_name, a.avg_cgpa, a.n
FROM   dept_avg a JOIN department d ON d.dept_id = a.dept_id
WHERE  a.avg_cgpa > (SELECT AVG(avg_cgpa) FROM dept_avg)
ORDER  BY a.avg_cgpa DESC;
```

14.7 Subqueries in DML

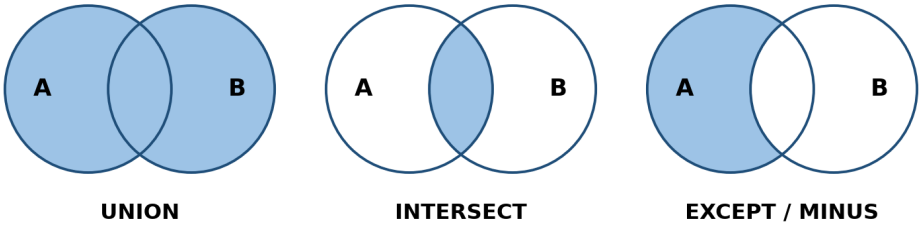
```sql
-- insert from a query
INSERT INTO honours_list (student_id, name, cgpa)
SELECT student_id, name, cgpa FROM student WHERE cgpa >= 3.75;

-- update using a correlated subquery
UPDATE student s
SET    cgpa = (SELECT ROUND(AVG(r.marks)/25, 2)
               FROM result r WHERE r.student_id = s.student_id)
WHERE  EXISTS (SELECT 1 FROM result r WHERE r.student_id = s.student_id);

-- delete based on a subquery
DELETE FROM enrollment
WHERE  course_id IN (SELECT course_id FROM course WHERE dept_id = 'ME');
```

The `WHERE EXISTS` in the `UPDATE` is not decoration: without it, students with no results would have their CGPA set to NULL, because a subquery returning no rows yields NULL.

14.8 Set Operations



both queries must have the same number of columns with compatible data types

Figure 14.2: The three set operations (shaded = rows returned)

Operation	Returns	Duplicates
`UNION`	Rows in either query	Removed
`UNION ALL`	Rows in either query	Kept
`INTERSECT`	Rows in both queries	Removed
`EXCEPT` (`MINUS` in Oracle)	Rows in the first but not the second	Removed

Two rules govern all of them: both queries must produce the **same number of columns**, and corresponding columns must have **compatible types**. Column names come from the first query, and `ORDER BY` may appear only once, at the end.

```
-- every person in the university, staff and students together
SELECT name, 'Student' AS role FROM student
UNION ALL
SELECT name, 'Instructor' FROM instructor
ORDER  BY role, name;
```

`UNION ALL` is used here deliberately: a student and an instructor may share a name, and both rows must appear. `UNION` would remove one of them and would also pay for a sort or hash to detect duplicates. **Use `UNION ALL` unless duplicate removal is actually required.**

```
-- PostgreSQL / Oracle / SQL Server
SELECT student_id FROM enrollment WHERE course_id = 1
INTERSECT
SELECT student_id FROM enrollment WHERE course_id = 2;

SELECT student_id FROM student
EXCEPT
SELECT student_id FROM enrollment;
```

MySQL 8.0.31 and later support `INTERSECT` and `EXCEPT`; on earlier versions they are emulated:

```
-- INTERSECT emulation
SELECT DISTINCT a.student_id FROM enrollment a
JOIN enrollment b ON a.student_id = b.student_id
WHERE a.course_id = 1 AND b.course_id = 2;

-- EXCEPT emulation
SELECT s.student_id FROM student s
LEFT JOIN enrollment e ON e.student_id = s.student_id
```

```
WHERE e.student_id IS NULL;
```

Chapter Summary

- A subquery may appear in `WHERE`, `HAVING`, `SELECT`, `FROM` and in DML statements.
- Single-row subqueries are compared with scalar operators; multi-row subqueries use `IN`, `ANY` or `ALL`; `> ALL` equals `> MAX` and `> ANY` equals `> MIN`.
- `NOT IN` returns nothing when the inner list contains NULL; use `NOT EXISTS`.
- A correlated subquery references the outer query and is conceptually evaluated per outer row; an equivalent join or derived table is often faster.
- Derived tables must be aliased; CTEs express the same idea more readably and may be reused.
- `UNION`, `INTERSECT` and `EXCEPT` require union-compatible queries and remove duplicates; `UNION ALL` does not and is cheaper.

Key Terms

Subquery · Inner query · Single-row subquery · Multi-row subquery · Correlated subquery · Scalar subquery · Derived table · Common table expression · `IN` · `NOT IN` · `ANY` · `ALL` · `EXISTS` · `NOT EXISTS` · `UNION` · `UNION ALL` · `INTERSECT` · `EXCEPT` · `MINUS` · Union compatibility

Common Mistakes

1. Using `NOT IN` with a subquery that can produce NULL.
2. Writing a multi-row subquery where a single value is expected.
3. Forgetting to alias a derived table.
4. Using `UNION` where `UNION ALL` is correct, silently deleting legitimate duplicate rows.
5. Placing `ORDER BY` inside one branch of a `UNION`.
6. Mismatched column counts or types in a set operation.
7. Writing a correlated subquery where a join would be clearer and faster.
8. Forgetting that an aggregate cannot appear in `WHERE` — that is exactly what a subquery is for.

Solved Problems

Problem 14.1 — Find the student with the highest CGPA without using `ORDER BY … LIMIT`.

Solution.

```
SELECT name, cgpa FROM student
WHERE  cgpa = (SELECT MAX(cgpa) FROM student);
```

This form also returns *all* students tied at the maximum, which `LIMIT 1` would not.

Problem 14.2 — Explain precisely why this returns no rows when `enrollment.course_id` can be NULL, and fix it.

```
SELECT title FROM course WHERE course_id NOT IN (SELECT course_id FROM enrollment);
```

Solution. `NOT IN` is evaluated as a chain of inequalities joined by `AND`. With a NULL in the list, one conjunct is `course_id <> NULL`, which is *unknown*; `TRUE AND UNKNOWN` is `UNKNOWN`, and `WHERE` keeps only `TRUE`. Every row therefore fails. Two fixes:

```
SELECT title FROM course c
```

```sql
WHERE  NOT EXISTS (SELECT 1 FROM enrollment e WHERE e.course_id = c.course_id);

-- or exclude NULLs explicitly
SELECT title FROM course
WHERE  course_id NOT IN (SELECT course_id FROM enrollment WHERE course_id IS NOT NULL);
```

Problem 14.3 — List departments whose average CGPA exceeds the overall university average.

Solution.

```sql
SELECT   d.dept_name, ROUND(AVG(s.cgpa),2) AS dept_avg
FROM     student s JOIN department d ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING   AVG(s.cgpa) > (SELECT AVG(cgpa) FROM student);
```

The subquery in `HAVING` computes a single overall value; the outer query compares each group's average against it.

Problem 14.4 — Write a query listing every student with the number of courses they have taken, using a scalar subquery, and then rewrite it as a join. Which is better?

Solution.

```sql
-- scalar subquery in SELECT
SELECT s.name,
       (SELECT COUNT(*) FROM enrollment e WHERE e.student_id = s.student_id) AS courses
FROM   student s ORDER BY courses DESC;

-- equivalent left join with grouping
SELECT   s.name, COUNT(e.enrollment_id) AS courses
FROM     student s LEFT JOIN enrollment e ON e.student_id = s.student_id
GROUP BY s.student_id, s.name ORDER BY courses DESC;
```

Both give the same answer, including 0 for students with no enrollment. The join form is generally preferable: it makes a single pass over `enrollment`, whereas the scalar subquery is conceptually one lookup per student. The subquery form wins in readability when several such counts over *different* tables are needed in one select list.

Problem 14.5 — Find students enrolled in both course 1 and course 2, using `INTERSECT` and without it.

Solution.

```sql
-- with INTERSECT
SELECT student_id FROM enrollment WHERE course_id = 1
INTERSECT
SELECT student_id FROM enrollment WHERE course_id = 2;

-- without: self join
SELECT DISTINCT a.student_id FROM enrollment a
JOIN enrollment b ON a.student_id = b.student_id
WHERE a.course_id = 1 AND b.course_id = 2;

-- without: grouped count, the most general form
SELECT   student_id FROM enrollment WHERE course_id IN (1,2)
GROUP BY student_id HAVING COUNT(DISTINCT course_id) = 2;
```

The third form generalizes to "enrolled in all of these n courses" simply by changing the list and the count — a relational division, treated formally in Chapter 20.

Problem 14.6 — Update every student's CGPA from their average marks, leaving students with no results untouched.

Solution.

```
UPDATE student s
SET    cgpa = (SELECT ROUND(AVG(r.marks) / 25, 2) FROM result r
               WHERE r.student_id = s.student_id)
WHERE  EXISTS (SELECT 1 FROM result r WHERE r.student_id = s.student_id);
```

Without the `WHERE EXISTS` guard, the correlated subquery returns NULL for students with no results and would erase their existing CGPA.

Problem 14.7 — Explain the difference between `> ALL` and `> ANY` with the sample instructor salaries.

Solution. The lecturer salary set is {45000}. `salary > ALL (…)` means greater than every member, equivalent to `> MAX` = `> 45000`, returning six instructors. The professor salary set is {95000, 94000}. `salary > ANY (…)` means greater than at least one member, equivalent to `> MIN` = `> 94000`, returning only Nasrin Akter at 95000. A subtle point: if the inner query returns no rows at all, `> ALL` is vacuously true for every outer row, while `> ANY` is false for all of them.

Review Questions

1. Where may a subquery appear in a SQL statement?
2. Distinguish single-row from multi-row subqueries with examples.
3. Explain `IN`, `ANY` and `ALL`, and their aggregate equivalents.
4. Explain the `NOT IN` NULL trap and two ways to avoid it.
5. Define a correlated subquery and explain how it is evaluated.
6. Compare `IN` and `EXISTS` on at least four criteria.
7. What is a derived table and why must it be aliased?
8. What is a CTE and what advantages does it have?
9. State the rules for union compatibility.
10. When should `UNION ALL` be used instead of `UNION`?

Multiple Choice Questions

1. A subquery that references the outer query is (a) nested (b) correlated (c) scalar (d) derived.
2. `> ALL (subquery)` is equivalent to (a) `> MIN` (b) `> MAX` (c) `= ANY` (d) `IN`.
3. `NOT IN` with a NULL in the list returns (a) all rows (b) no rows (c) an error (d) NULLs only.
4. `UNION` removes (a) NULLs (b) duplicates (c) nothing (d) unmatched rows.
5. A derived table must be (a) indexed (b) aliased (c) correlated (d) sorted.
6. `EXISTS` returns (a) a value (b) TRUE/FALSE (c) a list (d) a table.
7. Which cannot contain an aggregate? (a) `HAVING` (b) `SELECT` (c) `WHERE` (d) `ORDER BY`.
8. `INTERSECT` returns rows present in (a) either query (b) both queries (c) the first only (d) neither.
9. `UNION ALL` compared with `UNION` is (a) slower (b) faster (c) identical (d) unsupported.
10. A scalar subquery in `SELECT` must return (a) one row and one column (b) one row, many columns (c) many rows (d) a table.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(c) 8-(b) 9-(b) 10-(a)

Short Questions

1. What is a subquery?
2. Why can an aggregate not appear in `WHERE`?
3. Write the `NOT EXISTS` form of an anti-join.
4. What does `SELECT 1` inside `EXISTS` signify?
5. State one rule for union compatibility.
6. Which set operation corresponds to `EXCEPT` in Oracle?
7. Why must a derived table be aliased?
8. Give one advantage of a CTE over a derived table.
9. What happens if `> ALL` receives an empty subquery?
10. When is a correlated subquery unavoidable?

Long / Descriptive Questions

1. Explain all subquery types with position, examples, output and real-life interpretation.
2. Discuss the `NOT IN` NULL problem in depth with three-valued logic, and give safe alternatives.
3. Compare subqueries, joins and CTEs for the same problem, with performance discussion.
4. Explain all set operations with diagrams, rules and emulations for systems that lack them.
5. Write and explain a report query using a CTE, a correlated subquery and a set operation together.

Practical Exercises

1. Write queries using single-row, multi-row, correlated and scalar subqueries on `university_db`.
2. Demonstrate the `NOT IN` NULL trap by inserting a NULL and observing the result, then fix it with `NOT EXISTS`.
3. Rewrite a correlated subquery as a join and compare the execution plans with `EXPLAIN`.
4. Use `UNION ALL` to build a combined directory of students and instructors with a role column.
5. Find students enrolled in all CSE courses, using the grouped-count method.

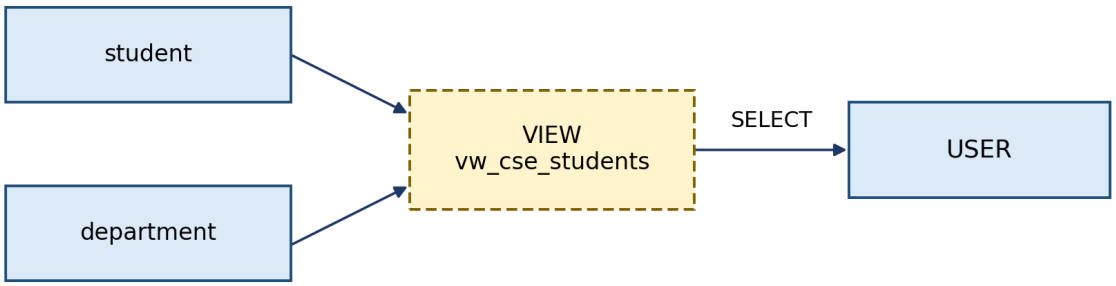
CHAPTER 15

Views, Triggers and Stored Procedures

These three objects move logic from application code into the database, where it applies to every user and every tool.

15.1 Views

A **view** is a named query. It stores no data of its own; each time it is used, the DBMS substitutes its defining query and produces rows from the base tables.



a view stores only the query text; rows are produced from the base tables on every access

Figure 15.1: A view is a virtual table computed from base tables on every access

```sql
CREATE VIEW vw_cse_students AS
SELECT s.student_id, s.name, s.cgpa, d.dept_name
FROM   student s JOIN department d ON d.dept_id = s.dept_id
WHERE  s.dept_id = 'CSE';

SELECT * FROM vw_cse_students WHERE cgpa > 3.5;      -- used exactly like a table
```

Why views exist

1. **Security.** Grant access to a view instead of a table, and a user sees only permitted rows and columns.

```sql
GRANT SELECT ON vw_cse_students TO 'cse_office'@'%';
-- no privilege on student itself: other departments remain invisible
```

2. **Simplicity.** A five-table join becomes one name that analysts can query without understanding the schema.
3. **Logical data independence.** When a base table is split, a view preserves the old shape so applications keep working (Section 3.5).
4. **Consistency.** One definition of “active student” or “failed course” is used everywhere instead of being re-derived, slightly differently, in each report.

```sql
CREATE VIEW vw_result_sheet AS
```

```sql
SELECT s.student_id, s.name, c.course_code, c.title,
       e.semester, e.grade, r.marks
FROM   enrollment e
JOIN   student s ON s.student_id = e.student_id
JOIN   course  c ON c.course_id  = e.course_id
LEFT   JOIN examination x ON x.course_id = c.course_id
LEFT   JOIN result r ON r.exam_id = x.exam_id AND r.student_id = s.student_id;
```

Updatable views

A view is updatable only if the DBMS can map each modified view row back to exactly one base-table row. A view is generally **not** updatable if it contains `DISTINCT`, `GROUP BY`, `HAVING`, an aggregate, a set operation, or (in most systems) a join.

```sql
CREATE VIEW vw_cse_simple AS
SELECT student_id, name, cgpa, dept_id FROM student WHERE dept_id = 'CSE'
WITH CHECK OPTION;

UPDATE vw_cse_simple SET cgpa = 3.80 WHERE student_id = 'ASH1901001';  -- allowed
UPDATE vw_cse_simple SET dept_id = 'EEE' WHERE student_id = 'ASH1901001';
-- ERROR: CHECK OPTION failed — the row would disappear from the view
```

`WITH CHECK OPTION` is what prevents a row from being updated or inserted *out of* the view — without it, a row can be moved somewhere the user cannot see, which is a genuine security hole.

Materialized views

A **materialized view** stores its result physically and must be refreshed. PostgreSQL and Oracle support them; MySQL does not, and the usual substitute is a summary table refreshed by an event or trigger.

```sql
-- PostgreSQL
CREATE MATERIALIZED VIEW mv_dept_summary AS
SELECT dept_id, COUNT(*) AS students, AVG(cgpa) AS avg_cgpa
FROM   student GROUP BY dept_id;

REFRESH MATERIALIZED VIEW mv_dept_summary;
```

Basis	View	Materialized view	Table
Stores data	No	Yes	Yes
Always current	Yes	Only after refresh	Yes
Query cost	Recomputed each time	Cheap (precomputed)	Cheap
Storage	Definition only	Full result	Full data
Best for	Security, simplification	Expensive aggregates	Base data

15.2 Triggers

A **trigger** is a block of code that the DBMS executes automatically in response to an `INSERT`, `UPDATE` or `DELETE` on a table.

	INSERT	UPDATE	DELETE
BEFORE	validate / default	validate / audit old	block or log
AFTER	log, update totals	write audit row	archive deleted row

NEW.column is available for INSERT and UPDATE; OLD.column for UPDATE and DELETE

Figure 15.2: Trigger timing and event combinations

Inside the body, `NEW.column` refers to the incoming value (available for `INSERT` and `UPDATE`) and `OLD.column` to the previous value (available for `UPDATE` and `DELETE`).

Audit-log example

```
CREATE TABLE grade_audit (
    audit_id    INT AUTO_INCREMENT PRIMARY KEY,
    student_id  CHAR(10)    NOT NULL,
    course_id   INT         NOT NULL,
    old_grade   CHAR(2),
    new_grade   CHAR(2),
    changed_by  VARCHAR(80) NOT NULL,
    changed_at  DATETIME    NOT NULL
);

DELIMITER $$
CREATE TRIGGER trg_grade_audit
AFTER UPDATE ON enrollment
FOR EACH ROW
BEGIN
    IF NOT (NEW.grade <=> OLD.grade) THEN          -- <=> is NULL-safe equality
        INSERT INTO grade_audit (student_id, course_id, old_grade, new_grade,
                                 changed_by, changed_at)
        VALUES (OLD.student_id, OLD.course_id, OLD.grade, NEW.grade,
                CURRENT_USER(), NOW());
    END IF;
END$$
DELIMITER ;
```

Now every grade change is recorded with who changed it and when, whether the change came from the portal, a bulk import or a clerk typing SQL directly. No application code can bypass it — which is exactly the property an auditor requires.

Validation example

```
DELIMITER $$
CREATE TRIGGER trg_check_marks
BEFORE INSERT ON result
FOR EACH ROW
BEGIN
```

```
    IF NEW.marks < 0 OR NEW.marks > 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Marks must be between 0 and 100';
    END IF;
END$$
DELIMITER ;
```

> Prefer a `CHECK` constraint to a trigger whenever the rule can be expressed as one. Constraints are declarative, visible in the schema, and cheaper. Triggers are for rules a constraint cannot express: writing to another table, comparing with `OLD` values, or cascading a derived total.

Maintaining a derived total

```
DELIMITER $$
CREATE TRIGGER trg_enrolment_count
AFTER INSERT ON enrollment
FOR EACH ROW
BEGIN
    UPDATE course SET enrolled_count = enrolled_count + 1
    WHERE course_id = NEW.course_id;
END$$
DELIMITER ;
```

Triggers have real costs: they run inside the user's transaction, so a slow trigger slows every write; they are invisible to someone reading the application code; and they can cascade, one trigger firing another. Keep them short, and document them.

15.3 Stored Procedures

A **stored procedure** is a named program stored in the database and invoked with `CALL`. Parameters may be `IN` (default), `OUT` or `INOUT`.

```
DELIMITER $$
CREATE PROCEDURE sp_enrol_student (
    IN  p_student_id CHAR(10),
    IN  p_course_id  INT,
    IN  p_semester   VARCHAR(10),
    OUT p_status     VARCHAR(100)
)
BEGIN
    DECLARE v_exists INT DEFAULT 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_status = 'Failed: database error, transaction rolled back';
    END;

    START TRANSACTION;

    SELECT COUNT(*) INTO v_exists FROM student WHERE student_id = p_student_id;
    IF v_exists = 0 THEN
        SET p_status = 'Failed: no such student';
        ROLLBACK;
    ELSE
        INSERT INTO enrollment (student_id, course_id, semester)
        VALUES (p_student_id, p_course_id, p_semester);
```

```
        COMMIT;
        SET p_status = CONCAT('Enrolled ', p_student_id, ' in course ', p_course_id);
    END IF;
END$$
DELIMITER ;

CALL sp_enrol_student('ASH1901001', 4, '7th', @msg);
SELECT @msg;
```

Output: `Enrolled ASH1901001 in course 4`.

Advantages	Disadvantages
Business logic is centralized and reusable	Written in a vendor-specific dialect; poor portability
Reduces network round trips for multi-step work	Harder to version-control, test and debug than application code
Can be granted separately from the underlying tables	Business logic split across two places confuses maintenance
Compiled and cached by the DBMS	Scaling means scaling the database, the most expensive tier

15.4 User-Defined Functions

A function returns a single value and may be used inside expressions — which a procedure cannot.

```
DELIMITER $$
CREATE FUNCTION fn_letter_grade (p_marks DECIMAL(5,2))
RETURNS CHAR(2)
DETERMINISTIC
BEGIN
    RETURN CASE
        WHEN p_marks IS NULL  THEN NULL
        WHEN p_marks >= 80    THEN 'A'
        WHEN p_marks >= 75    THEN 'A-'
        WHEN p_marks >= 70    THEN 'B+'
        WHEN p_marks >= 65    THEN 'B'
        WHEN p_marks >= 60    THEN 'B-'
        WHEN p_marks >= 55    THEN 'C+'
        WHEN p_marks >= 50    THEN 'C'
        WHEN p_marks >= 40    THEN 'D'
        ELSE                       'F'
    END;
END$$
DELIMITER ;

SELECT student_id, marks, fn_letter_grade(marks) AS grade FROM result WHERE exam_id = 1;
```

student_id	marks	grade
ASH1901001	82.00	A
ASH1901003	88.50	A
ASH1901004	67.00	B
ASH2001001	91.00	A

Basis	Stored procedure	Function
Invoked by	`CALL`	Inside an expression

Basis	Stored procedure	Function
Returns	Zero or more values via `OUT`	Exactly one value
Usable in `SELECT`	No	Yes
May modify data	Yes	Restricted or forbidden
Transactions	May contain `COMMIT`/`ROLLBACK`	Normally may not

Chapter Summary

- A view is a stored query providing security, simplification, consistency and logical data independence; it holds no data.
- A view is updatable only when each view row maps to one base row; `WITH CHECK OPTION` stops rows from being updated out of view.
- Materialized views store results and need refreshing; MySQL substitutes summary tables.
- Triggers fire automatically `BEFORE` or `AFTER` `INSERT`, `UPDATE` or `DELETE`, and use `NEW` and `OLD`; they are ideal for auditing and for rules a constraint cannot express.
- Prefer constraints to triggers whenever possible.
- Stored procedures centralize multi-step logic and are called with `CALL`; functions return one value and can be used in expressions.

Key Terms

View · Virtual table · Base table · Updatable view · `WITH CHECK OPTION` · Materialized view · Trigger · `BEFORE` / `AFTER` · `FOR EACH ROW` · `NEW` / `OLD` · `SIGNAL SQLSTATE` · Audit log · Stored procedure · `IN` / `OUT` / `INOUT` parameters · Exception handler · User-defined function · `DETERMINISTIC` · `DELIMITER`

Common Mistakes

1. Believing a view stores data or improves performance by caching. It does neither.
2. Creating a view over a view over a view; nesting makes optimization and debugging hard.
3. Omitting `WITH CHECK OPTION` on a security view.
4. Using a trigger where a `CHECK` constraint would do.
5. Writing long-running work inside a trigger, slowing every transaction.
6. Forgetting `DELIMITER` in MySQL, so the client cuts the body at the first semicolon.
7. Confusing procedures with functions, especially trying to call a procedure inside a `SELECT`.
8. Putting so much logic in stored procedures that the application becomes untestable.

Solved Problems

Problem 15.1 — Create a view exposing only student name and department, grant it to a clerk, and show that the clerk cannot see CGPA.

Solution.

```
CREATE VIEW vw_student_directory AS
SELECT s.student_id, s.name, d.dept_name
FROM   student s JOIN department d ON d.dept_id = s.dept_id;

GRANT SELECT ON university_db.vw_student_directory TO 'front_desk'@'%';
```

The clerk has no privilege on `student`, so `SELECT cgpa FROM student` is refused, and the view does not expose the column at all. This is column-level security implemented by projection.

Problem 15.2 — Explain why a view with `GROUP BY` is not updatable.

Solution. Each row of such a view summarizes many base rows. An `UPDATE` that sets a department's average CGPA to 3.6 has no unambiguous translation: the system cannot know which students' CGPAs to change or by how much. Updatability requires a one-to-one correspondence between view rows and base rows, which aggregation destroys.

Problem 15.3 — Write a trigger preventing a grade from being changed after results are published, and explain why a constraint cannot do this.

Solution.

```
DELIMITER $$
CREATE TRIGGER trg_lock_published_grade
BEFORE UPDATE ON enrollment
FOR EACH ROW
BEGIN
    IF OLD.grade IS NOT NULL AND NOT (NEW.grade <=> OLD.grade) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Published grades cannot be modified';
    END IF;
END$$
DELIMITER ;
```

A `CHECK` constraint can only see the row's new values; it has no access to the previous value. The rule here is about a *transition* — from non-NULL to a different value — and only a trigger can compare `OLD` with `NEW`.

Problem 15.4 — Write a procedure that transfers a student to another department and records the change, with rollback on failure.

Solution.

```
DELIMITER $$
CREATE PROCEDURE sp_transfer_student (IN p_student CHAR(10), IN p_new_dept CHAR(3))
BEGIN
    DECLARE v_old CHAR(3);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

    START TRANSACTION;
    SELECT dept_id INTO v_old FROM student WHERE student_id = p_student FOR UPDATE;
    UPDATE student SET dept_id = p_new_dept WHERE student_id = p_student;
    INSERT INTO transfer_log (student_id, from_dept, to_dept, moved_at)
    VALUES (p_student, v_old, p_new_dept, NOW());
    COMMIT;
END$$
DELIMITER ;
```

`FOR UPDATE` locks the student row so that two simultaneous transfers cannot interleave; the handler guarantees that a failure leaves neither the update nor the log entry behind.

Problem 15.5 — Compare enforcing "salary must be positive" with a `CHECK` constraint and with a trigger.

Solution. The `CHECK` constraint is declarative, appears in the schema where any developer can see it, is enforced by the storage engine at low cost, and cannot be bypassed. The trigger must be written procedurally, is invisible unless someone lists the triggers, runs interpreted code for every row, and can be accidentally dropped or disabled. The constraint is strictly better here. The trigger would only become necessary if the rule referred to another table — for instance, "salary must not exceed the maximum for

the designation stored in `pay_scale`" — since a `CHECK` constraint in most systems may not reference other tables.

Problem 15.6 — Write a function returning a student's total credits earned, and use it in a query.

Solution.

```
DELIMITER $$
CREATE FUNCTION fn_credits_earned (p_student CHAR(10))
RETURNS DECIMAL(5,1)
READS SQL DATA
BEGIN
    DECLARE v DECIMAL(5,1);
    SELECT COALESCE(SUM(c.credit), 0) INTO v
    FROM   enrollment e JOIN course c ON c.course_id = e.course_id
    WHERE  e.student_id = p_student AND e.grade IS NOT NULL AND e.grade <> 'F';
    RETURN v;
END$$
DELIMITER ;

SELECT student_id, name, fn_credits_earned(student_id) AS credits FROM student;
```

Note the cost: the function runs once per row of `student`, each time executing a join. For a large table, a single grouped join would be far cheaper — a good illustration of why row-by-row functions must be used carefully in set-oriented SQL.

Review Questions

1. What is a view, and what four purposes does it serve?
2. When is a view updatable, and what does `WITH CHECK OPTION` do?
3. Compare views, materialized views and tables.
4. Explain trigger timing and events, and the meaning of `NEW` and `OLD`.
5. Give three appropriate and two inappropriate uses of triggers.
6. Why is a `CHECK` constraint preferred to a trigger where both would work?
7. What is a stored procedure, and what parameter modes exist?
8. Compare stored procedures and functions on at least five criteria.
9. Discuss the advantages and disadvantages of putting logic in the database.
10. Explain how views implement column- and row-level security.

Multiple Choice Questions

1. A view stores (a) data (b) only its definition (c) indexes (d) triggers.
2. A view containing `GROUP BY` is (a) updatable (b) not updatable (c) materialized (d) indexed.
3. `WITH CHECK OPTION` prevents (a) deletes (b) rows being updated out of the view (c) NULLs (d) joins.
4. `OLD.column` is available in (a) `INSERT` triggers (b) `UPDATE` and `DELETE` triggers (c) all triggers (d) none.
5. A materialized view must be (a) dropped (b) refreshed (c) indexed (d) granted.
6. Which is invoked with `CALL`? (a) A function (b) A procedure (c) A trigger (d) A view.
7. Which can be used inside a `SELECT` list? (a) Procedure (b) Function (c) Trigger (d) Both a and b.
8. `SIGNAL SQLSTATE '45000'` is used to (a) commit (b) raise an error (c) create a view (d) grant access.

9. Which fires automatically? (a) View (b) Procedure (c) Trigger (d) Function.
10. MySQL lacks native support for (a) views (b) triggers (c) materialized views (d) procedures.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(c) 10-(c)

Short Questions

1. Define a virtual table.
2. Name two things that make a view non-updatable.
3. What does `FOR EACH ROW` mean?
4. When is `NEW` unavailable?
5. What is an audit trigger?
6. Give one rule a `CHECK` constraint cannot express.
7. What is an `OUT` parameter?
8. Why is `DELIMITER` needed in MySQL?
9. Name one disadvantage of stored procedures.
10. How does a view provide row-level security?

Long / Descriptive Questions

1. Explain views in depth: definition, purposes, updatability, `WITH CHECK OPTION`, materialized views and security uses.
2. Explain triggers fully with timing, events, `NEW`/`OLD`, audit and validation examples, and their costs.
3. Compare constraints and triggers, stating clear criteria for choosing between them.
4. Explain stored procedures and functions with complete examples, parameter modes and error handling.
5. Discuss the trade-offs of placing business logic in the database versus the application tier.

Practical Exercises

1. Create three views over `university_db`: a security view, a simplification view and a summary view. Grant each to a different user.
2. Demonstrate that an aggregate view is not updatable, and that a simple projection view is.
3. Build the grade-audit trigger, change a grade, and inspect the audit table.
4. Write a stored procedure enrolling a student with validation, transactions and an error handler; test both success and failure paths.
5. Write a function computing a letter grade, use it in a query over `result`, and measure the cost difference against an inline `CASE` expression.

CHAPTER 16

Integrity Constraints

An integrity constraint is a rule the DBMS enforces on every modification, from every application, through every tool. Constraints are the reason a database can be trusted: they make invalid data impossible rather than merely unlikely.

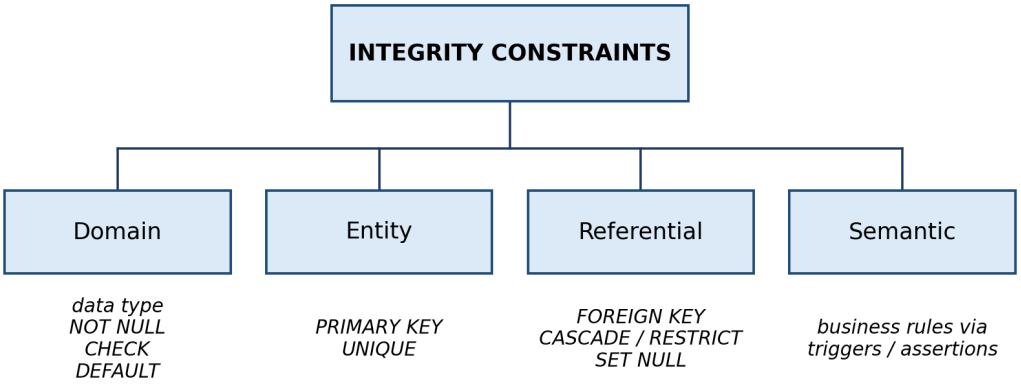


Figure 16.1: Categories of integrity constraint

16.1 Domain Constraints

Domain constraints restrict the values an attribute may take.

Data type

The first and strongest constraint. `cgpa DECIMAL(3,2)` makes `'excellent'` unstorable.

NOT NULL

```
ALTER TABLE student MODIFY name VARCHAR(100) NOT NULL;
INSERT INTO student (student_id, name) VALUES ('ASH2101001', NULL);
-- ERROR 1048: Column 'name' cannot be null
```

CHECK

```
ALTER TABLE student
  ADD CONSTRAINT ck_student_cgpa CHECK (cgpa BETWEEN 0.00 AND 4.00);

ALTER TABLE instructor
  ADD CONSTRAINT ck_inst_salary CHECK (salary > 0);

ALTER TABLE examination
  ADD CONSTRAINT ck_exam_date CHECK (exam_date >= DATE '2000-01-01');

-- a multi-column CHECK
ALTER TABLE enrollment
  ADD CONSTRAINT ck_grade_valid
  CHECK (grade IS NULL OR grade IN ('A','A-','B+','B','B-','C+','C','D','F'));
```

Vendor note: MySQL ignored `CHECK` constraints silently until version 8.0.16 — a notorious trap in older systems, where a rule appeared to exist but did nothing.

A `CHECK` on a single row cannot reference other tables or aggregate over the table; those rules need triggers or assertions.

DEFAULT

```
ALTER TABLE student ALTER COLUMN cgpa SET DEFAULT 0.00;      -- PostgreSQL
ALTER TABLE student ALTER cgpa SET DEFAULT 0.00;             -- MySQL
```

`DEFAULT` applies when the column is omitted from an `INSERT`; it does **not** apply when NULL is inserted explicitly — a distinction that surprises many students.

16.2 Entity Integrity

Entity integrity is enforced by the primary key: unique and never NULL.

```
ALTER TABLE student ADD CONSTRAINT pk_student PRIMARY KEY (student_id);
ALTER TABLE student ADD CONSTRAINT uq_student_email UNIQUE (email);
```

Basis	`PRIMARY KEY`	`UNIQUE`
Per table	One	Many
NULL	Not allowed	Allowed
Purpose	Principal identifier	Alternate key
Index	Created automatically, often clustered	Created automatically, non-clustered

A subtlety worth knowing: in MySQL and PostgreSQL a `UNIQUE` column may contain **several** NULLs, because two NULLs are not considered equal. In SQL Server only one NULL is permitted. If "at most one row may have no email" is the real rule, it must be stated explicitly, for example with a partial index in PostgreSQL.

16.3 Referential Integrity

Every non-NULL foreign key value must match an existing primary or unique key value in the parent relation.

```
ALTER TABLE enrollment
  ADD CONSTRAINT fk_enr_student FOREIGN KEY (student_id)
      REFERENCES student(student_id)
      ON DELETE CASCADE ON UPDATE CASCADE;

INSERT INTO enrollment (student_id, course_id, semester) VALUES ('ASH9999999', 1, '5th')
    ↪ ;
-- ERROR 1452: Cannot add or update a child row: a foreign key constraint fails
```

Referential action	On delete of parent	On update of parent key
`RESTRICT` / `NO ACTION`	Reject if children exist	Reject if children exist
`CASCADE`	Delete children too	Update children's keys
`SET NULL`	Set child key to NULL (column must be nullable)	Same
`SET DEFAULT`	Set child key to its default	Same

Choosing these is a business decision. In `university_db`:

- `enrollment` → `student` uses `ON DELETE CASCADE`: removing a student legitimately removes their enrollments.
- `enrollment` → `course` uses `ON DELETE RESTRICT`: a course with recorded results must not be deletable, because results are historical records.
- `student` → `department` uses `ON UPDATE CASCADE, ON DELETE RESTRICT`: a department code may be renamed, but a department with students may not be dissolved without first moving them.

> Never put `ON DELETE CASCADE` on financial, audit or examination data. A single mistaken `DELETE` on a parent row would silently erase the history that auditors depend on.

16.4 Semantic and Business Constraints

Rules that span rows or tables fall outside the declarative constraint types. Standard SQL defines `CREATE ASSERTION` for them, but almost no DBMS implements it, so in practice they are enforced by triggers (Chapter 15) or in application code.

Examples: “a student may not enrol in more than six courses in a semester”; “an examination room’s capacity must not be exceeded”; “an instructor’s salary must not exceed the maximum of their designation”.

```
DELIMITER $$
CREATE TRIGGER trg_max_six_courses
BEFORE INSERT ON enrollment
FOR EACH ROW
BEGIN
    DECLARE v_count INT;
    SELECT COUNT(*) INTO v_count FROM enrollment
    WHERE student_id = NEW.student_id AND semester = NEW.semester;
    IF v_count >= 6 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'A student may not exceed six courses in one semester';
    END IF;
END$$
DELIMITER ;
```

16.5 When Constraints Are Checked

Mode	Checked	Support
`IMMEDIATE` (default)	At the end of each statement	All systems
`DEFERRED`	At `COMMIT`	PostgreSQL, Oracle; not MySQL

Deferred checking solves the circular-insert problem: a department needs a head who is an instructor, and the instructor needs a department.

```
-- PostgreSQL
ALTER TABLE department
  ADD CONSTRAINT fk_dept_head FOREIGN KEY (head_id) REFERENCES instructor(instructor_id)
  DEFERRABLE INITIALLY DEFERRED;

BEGIN;
INSERT INTO department VALUES ('CSE','Computer Science','AB-3', 1);   -- head 1 not yet
    ↪ present
INSERT INTO instructor VALUES (1,'Nasrin Akter','Professor',95000,'CSE');
COMMIT;                     -- both constraints verified here, and both now hold
```

In MySQL the equivalent is to insert with a NULL head and update afterwards.

16.6 A Fully Constrained Table

```
CREATE TABLE enrollment (
    enrollment_id INT AUTO_INCREMENT,
    student_id    CHAR(10)    NOT NULL,
    course_id     INT         NOT NULL,
    semester      VARCHAR(10) NOT NULL,
    grade         CHAR(2)     NULL,
    enrolled_on   DATE        NOT NULL DEFAULT (CURRENT_DATE),

    CONSTRAINT pk_enrollment  PRIMARY KEY (enrollment_id),
    CONSTRAINT uq_enrollment  UNIQUE (student_id, course_id, semester),
    CONSTRAINT ck_semester    CHECK (semester IN ('1st','2nd','3rd','4th','5th','6th','7
    ↪ th','8th')),
    CONSTRAINT ck_grade       CHECK (grade IS NULL OR grade IN
                                  ('A','A-','B+','B','B-','C+','C','D','F')),
    CONSTRAINT fk_enr_student FOREIGN KEY (student_id) REFERENCES student(student_id)
                              ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_enr_course  FOREIGN KEY (course_id) REFERENCES course(course_id)
                              ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Every rule of the mini-world that *can* be declared is declared here: identity (`pk`), the natural key (`uq`), permitted values (`ck`), and the two relationships with their chosen behaviours. The only rule left to a trigger is the six-course limit, because it involves counting other rows.

16.7 Common Constraint Errors and Their Meaning

Error (MySQL)	Cause	Fix
1048 Column cannot be null	`NOT NULL` violated	Supply a value or allow NULL
1062 Duplicate entry	`PRIMARY KEY` or `UNIQUE` violated	Use a different value, or `UPSERT`
1452 Cannot add or update a child row	Parent row missing	Insert the parent first
1451 Cannot delete or update a parent row	Children exist, action is `RESTRICT`	Delete children or change the action
3819 Check constraint violated	`CHECK` failed	Correct the value
1364 Field doesn't have a default	`NOT NULL` with no default, omitted	Supply the column or add a `DEFAULT`

Reading these messages accurately is a practical skill: each one names the exact constraint, which is why Chapter 6 insisted on naming constraints meaningfully.

Chapter Summary

- Constraints make invalid data impossible for every user and tool, not merely unlikely.
- Domain constraints comprise data type, `NOT NULL`, `CHECK` and `DEFAULT`; `DEFAULT` does not apply to an explicit NULL.
- Entity integrity is the primary key: unique and non-NULL; `UNIQUE` provides alternate keys and permits NULL (multiple NULLs in MySQL and PostgreSQL).
- Referential integrity requires every non-NULL foreign key to match a parent key; four referential

actions decide what happens to children.

- Rules spanning rows or tables need triggers, since `CREATE ASSERTION` is essentially unimplemented.
- Constraints are checked immediately by default; PostgreSQL and Oracle allow deferral to `COMMIT`, which solves circular references.

Key Terms

Integrity constraint · Domain constraint · `NOT NULL` · `CHECK` · `DEFAULT` · Entity integrity · `PRIMARY KEY` · `UNIQUE` · Referential integrity · Foreign key · Parent / child · `CASCADE` · `RESTRICT` · `SET NULL` · `SET DEFAULT` · Semantic constraint · Assertion · `DEFERRABLE` · `INITIALLY DEFERRED` · Constraint violation

Common Mistakes

1. Enforcing rules only in application code, so another program or a manual edit bypasses them.
2. Assuming `CHECK` works in MySQL versions before 8.0.16.
3. Expecting `DEFAULT` to replace an explicitly inserted NULL.
4. Using `ON DELETE CASCADE` on financial or examination data.
5. Assuming `UNIQUE` permits only one NULL — true in SQL Server, false in MySQL and PostgreSQL.
6. Leaving constraints unnamed, so error messages are unreadable.
7. Attempting a cross-table rule in a `CHECK` constraint.
8. Forgetting to index a foreign key column, making cascading deletes slow.

Solved Problems

Problem 16.1 — Classify each: (a) `cgpa BETWEEN 0 AND 4`; (b) `student_id` cannot be NULL; (c) `dept_id` must exist in `department`; (d) a student may take at most six courses per semester.

Solution. (a) Domain constraint (`CHECK`). (b) Entity integrity (part of the primary key, or `NOT NULL`). (c) Referential integrity (`FOREIGN KEY`). (d) Semantic/business constraint — not expressible declaratively in any mainstream DBMS, so it requires a trigger.

Problem 16.2 — A delete of a department fails with error 1451. Explain the cause and give three possible resolutions.

Solution. Students, instructors or courses still reference that department, and the foreign key uses `RESTRICT`, so the DBMS refuses to create dangling references. Resolutions: (i) reassign the children to another department first, then delete — the correct business action in nearly all cases; (ii) delete the children first, if they are genuinely obsolete; (iii) change the constraint to `ON DELETE SET NULL` if an unassigned state is meaningful, or to `CASCADE` if the children are truly worthless without the parent. Option (iii) must be a considered policy decision, not a shortcut to make an error disappear.

Problem 16.3 — Write all constraints needed for a `loan` table: positive amount, interest rate between 0 and 25, a valid customer, and a status limited to three values.

Solution.

```sql
CREATE TABLE loan (
    loan_id      INT AUTO_INCREMENT PRIMARY KEY,
    customer_id  INT           NOT NULL,
    amount       DECIMAL(12,2) NOT NULL,
    rate         DECIMAL(5,2)  NOT NULL,
    status       VARCHAR(10)   NOT NULL DEFAULT 'ACTIVE',
    sanctioned   DATE          NOT NULL DEFAULT (CURRENT_DATE),
    CONSTRAINT ck_loan_amount CHECK (amount > 0),
```

```
    CONSTRAINT ck_loan_rate   CHECK (rate >= 0 AND rate <= 25),
    CONSTRAINT ck_loan_status CHECK (status IN ('ACTIVE','CLOSED','DEFAULTED')),
    CONSTRAINT fk_loan_cust   FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
                              ON DELETE RESTRICT ON UPDATE CASCADE
);
```

`RESTRICT` is mandatory here: a customer with an outstanding loan must never be deletable.

Problem 16.4 — Explain why the following succeeds although a default exists, and what value is stored.

```
ALTER TABLE student ALTER cgpa SET DEFAULT 0.00;
INSERT INTO student (student_id, name, dept_id, admitted_on, cgpa)
VALUES ('ASH2101002','Farha','CSE','2021-01-05', NULL);
```

Solution. The stored value is NULL, not 0.00. A `DEFAULT` supplies a value only when the column is **omitted** from the insert; here it was supplied explicitly as NULL, and since the column has no `NOT NULL` constraint, NULL is accepted. To force the default, either omit the column or write `DEFAULT` as the value. To forbid NULL entirely, the column must be declared `NOT NULL DEFAULT 0.00`.

Problem 16.5 — Two tables reference each other. Show how to load data in MySQL and in PostgreSQL.

Solution. **PostgreSQL** — declare the constraint deferrable and insert both rows in one transaction, as in Section 16.5; both checks run at `COMMIT`, by which time both rows exist. **MySQL** — no deferral exists, so the standard approach is:

```
INSERT INTO department (dept_id, dept_name, building, head_id)
VALUES ('CSE','Computer Science','AB-3', NULL);          -- head unknown for now
INSERT INTO instructor (instructor_id, name, designation, salary, dept_id)
VALUES (1,'Nasrin Akter','Professor',95000,'CSE');
UPDATE department SET head_id = 1 WHERE dept_id = 'CSE'; -- now valid
```

This works only if `head_id` is nullable, which is a good argument for modelling "head" as partial participation unless the business truly forbids a headless department.

Problem 16.6 — Argue for placing constraints in the database rather than in the application.

Solution. Four arguments. **Universality:** a database is typically accessed by several applications, batch jobs, import scripts and human administrators; only a database-level constraint applies to all of them. **Permanence:** applications are rewritten every few years while the data outlives them, so rules embedded in application code are lost at each rewrite. **Atomicity of enforcement:** the DBMS checks constraints inside the transaction, so a concurrent transaction cannot slip an invalid row between an application's check and its insert — a race condition application-level validation cannot close. **Documentation:** the schema becomes a precise statement of the business rules, readable by anyone. The counter-argument — that constraint errors give poor user messages — is real, and is addressed by validating in the application *as well*, for usability, while keeping the database as the authority.

Review Questions

1. Define integrity constraint and explain why constraints belong in the database.
2. Describe all domain constraints with examples.
3. Explain the difference between `PRIMARY KEY` and `UNIQUE`, including NULL behaviour across vendors.
4. State the referential integrity rule and describe all four referential actions.
5. Give two business rules that cannot be expressed as declarative constraints, and say how they are enforced.

6. What is an assertion, and why is it rarely available?
7. Explain immediate versus deferred constraint checking with a use case.
8. Why does `DEFAULT` not apply to an explicit NULL?
9. Interpret the five most common constraint error messages.
10. Why should constraints be named?

Multiple Choice Questions

1. "No primary key attribute may be NULL" is (a) domain (b) entity (c) referential (d) semantic integrity.
2. Which permits multiple NULLs in MySQL? (a) `PRIMARY KEY` (b) `UNIQUE` (c) `NOT NULL` (d) none.
3. `ON DELETE SET NULL` requires the child column to be (a) indexed (b) nullable (c) unique (d) a primary key.
4. `CHECK` constraints were enforced in MySQL from version (a) 5.5 (b) 5.7 (c) 8.0.16 (d) never.
5. Deferred constraint checking happens at (a) statement end (b) `COMMIT` (c) connection close (d) backup.
6. A rule spanning several tables needs (a) a `CHECK` (b) a trigger (c) a `UNIQUE` (d) a `DEFAULT`.
7. Error 1451 means (a) duplicate key (b) parent row has children (c) NULL violation (d) check failed.
8. `DEFAULT` applies when the column is (a) set to NULL (b) omitted (c) indexed (d) updated.
9. Which action rejects a deletion? (a) `CASCADE` (b) `SET NULL` (c) `RESTRICT` (d) `SET DEFAULT`.
10. Constraints are best placed in (a) the application (b) the database (c) documentation (d) triggers only.

Answers: 1-(b) 2-(b) 3-(b) 4-(c) 5-(b) 6-(b) 7-(b) 8-(b) 9-(c) 10-(b)

Short Questions

1. What is domain integrity?
2. Can a `CHECK` constraint reference another table?
3. How many NULLs does a `UNIQUE` column allow in PostgreSQL?
4. What does `ON UPDATE CASCADE` do?
5. Which systems support deferred constraints?
6. Why name a constraint?
7. What is a dangling reference?
8. Give one rule requiring a trigger.
9. What does MySQL error 1062 mean?
10. Why avoid `CASCADE` on audit tables?

Long / Descriptive Questions

1. Explain all categories of integrity constraint with SQL examples and the errors each produces.
2. Discuss referential integrity and all four referential actions, with realistic scenarios for choosing each.
3. Compare declarative constraints with trigger-based enforcement, giving criteria for choosing between them.
4. Explain immediate and deferred checking and how circular foreign key references are handled in MySQL and PostgreSQL.
5. Argue, with at least four supporting points, that integrity rules belong in the database rather than

the application.

Practical Exercises

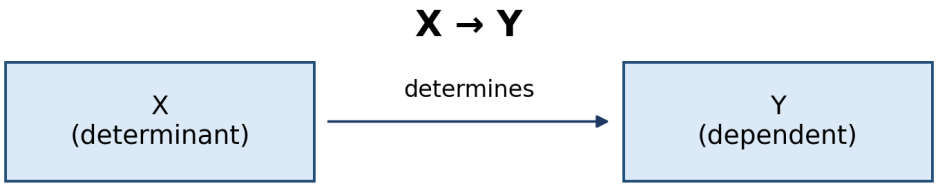
1. Add every constraint described in this chapter to `university_db` and verify each with a deliberately invalid statement.
2. Demonstrate `CASCADE`, `RESTRICT` and `SET NULL` by configuring three foreign keys differently and deleting parent rows.
3. Show that `DEFAULT` does not apply to an explicit NULL, then correct the column so that NULL is impossible.
4. Write a trigger enforcing a maximum of six enrollments per semester and test the seventh insert.
5. In PostgreSQL, create two mutually referencing tables with a deferred constraint and load them in a single transaction.

CHAPTER 17

Functional Dependencies

Normalization is the theory of good relational design, and functional dependencies are its vocabulary. A functional dependency records a fact about the *meaning* of the data — a rule of the mini-world — not a fact about the rows that happen to be present today.

17.1 Definition



if two tuples agree on X, they must agree on Y

```
student_id → name        (student_id, course_id) → grade
```

Figure 17.1: A functional dependency X → Y

Let R be a relation schema and let X and Y be subsets of its attributes. The functional dependency $\mathbf{X \rightarrow Y}$ holds on R if, for any two tuples t_1 and t_2 of any legal instance of R,

```
    t1[X] = t2[X]   implies   t1[Y] = t2[Y]
```

X is the **determinant**; Y is the **dependent**. Read `X → Y` as "X functionally determines Y", or "Y is functionally dependent on X".

```
student_id → name                 one student ID belongs to exactly one name
student_id → dept_id
(student_id, course_id) → grade   one grade per student per course
course_id  → title, credit
dept_id    → dept_name, building
```

Two clarifications prevent most confusion:

- An FD is a **constraint on every legal instance**, not an observation about the current rows. If the present data happens to show that no two students share a name, it does not follow that `name → student_id`; the rule holds only if the mini-world forbids duplicate names, which it does not.
- The direction matters. `student_id → name` does not imply `name → student_id`.

17.2 Trivial and Non-Trivial Dependencies

Type	Condition	Example
Trivial	$Y \subseteq X$	`(student_id, name) → name`
Non-trivial	$Y \not\subset X$, at least one attribute of Y outside X	`student_id → name`
Completely non-trivial	$X \cap Y = \varnothing$	`student_id → name`

Trivial dependencies always hold and carry no information; normalization is concerned only with non-trivial ones.

17.3 Full, Partial and Transitive Dependencies

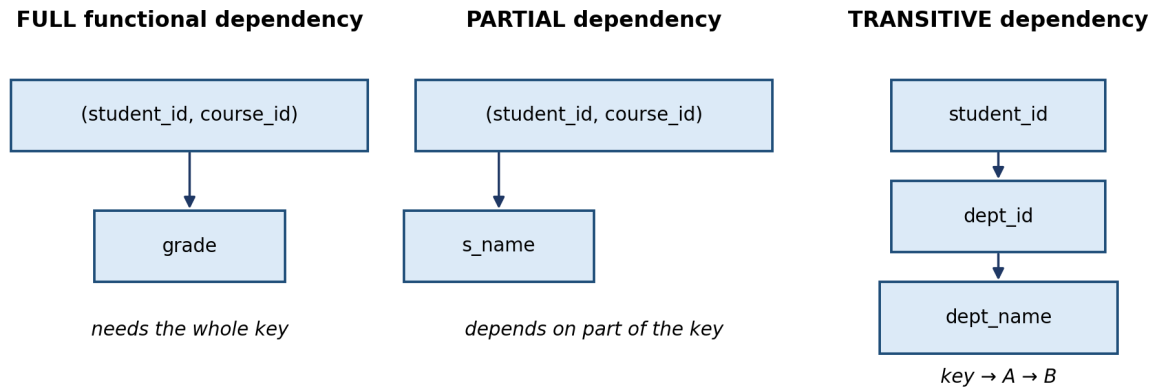


Figure 17.2: Full, partial and transitive dependencies

Consider the badly designed relation

```
ENROLMENT(student_id, course_id, s_name, c_title, grade)
```

with key `(student_id, course_id)`.

- **Full functional dependency.** `(student_id, course_id) → grade`. Removing either attribute from the left side destroys the dependency, because a grade is determined only by the *pair*. This is the dependency 2NF requires.
- **Partial dependency.** `student_id → s_name`. The dependent is determined by *part* of the composite key. This is what 2NF forbids.
- **Transitive dependency.** In `STUDENT(student_id, dept_id, dept_name)`, `student_id → dept_id` and `dept_id → dept_name`, so `student_id → dept_name` holds **through** a non-key attribute. This is what 3NF forbids.

Formally, $X \to Z$ is transitive if there exists Y with $X \to Y$, $Y \to Z$, $Y \not\subset X$, and Y is not a super key.

17.4 Armstrong's Axioms

Armstrong's axioms are a **sound and complete** set of inference rules: sound because every dependency they derive really holds, complete because every dependency that holds can be derived from them.

Axiom	Rule	Meaning
Reflexivity	If $Y \subseteq X$ then $X \to Y$	Trivial dependencies always hold
Augmentation	If $X \to Y$ then $XZ \to YZ$	Adding attributes to both sides preserves the FD
Transitivity	If $X \to Y$ and $Y \to Z$ then $X \to Z$	Dependencies chain

Three further rules follow from these and shorten proofs:

Derived rule	Statement
Union	If X → Y and X → Z then X → YZ
Decomposition	If X → YZ then X → Y and X → Z
Pseudo-transitivity	If X → Y and WY → Z then WX → Z

> A warning that is examined often: **decomposition applies only to the right-hand side.** From `AB → C` you may **not** conclude `A → C` or `B → C`.

Worked derivation. Given `A → B` and `B → C`, prove `A → AC`.

```
1. A → B                    given
2. B → C                    given
3. A → C                    transitivity on 1, 2
4. A → A                    reflexivity
5. A → AC                   union of 4 and 3
```

17.5 Attribute Closure

The **closure** X^+ of an attribute set X under a set F of dependencies is the set of all attributes functionally determined by X. It is the single most useful computational tool in this part of the subject.

```
Algorithm CLOSURE(X, F)
  1. result := X
  2. repeat
  3.     for each FD  A → B  in F
  4.         if A ⊆ result then result := result ∪ B
  5. until result stops changing
  6. return result
```

Example. R(A, B, C, D, E) with F = { A → BC, CD → E, B → D, E → A }. Compute A^+.

```
result = {A}
A → BC        A ⊆ {A}          result = {A,B,C}
B → D         B ⊆ {A,B,C}      result = {A,B,C,D}
CD → E        CD ⊆ {A,B,C,D}   result = {A,B,C,D,E}
E → A         already present   no change
A+ = {A,B,C,D,E}
```

Since A^+ contains every attribute, **A is a super key**, and because it is a single attribute it is also a candidate key.

Uses of closure

1. **Is X a super key?** Yes if X^+ = all attributes.
2. **Does X → Y hold?** Yes if $Y \subseteq X^+$. This test avoids proof by axioms entirely.
3. **Finding candidate keys.** Compute closures of attribute sets in increasing size and keep the minimal super keys.

Systematic search for candidate keys

Classify each attribute by where it appears in F:

Class	Appears	Conclusion
Only on the left	Left side only	Must be in **every** candidate key
Only on the right	Right side only	Can be in **no** candidate key
Both sides	Left and right	May or may not be in a key
Neither	Not in any FD	Must be in every candidate key

Start with the attributes that must be in every key, compute their closure, and if it is not everything, add "both sides" attributes one at a time.

Example. R(A, B, C, D) with F = { AB → C, C → D, D → A }.

Attribute *B* appears only on the left, so *B* is in every candidate key. B^+ = {B}, not everything, so *B* must be extended. AB^+ = {A,B,C,D} — candidate key. BC^+ = {B,C,D,A} — candidate key. BD^+ = {B,D,A,C} — candidate key. So the candidate keys are **AB, BC and BD**, and the prime attributes are A, B, C, D.

17.6 Canonical Cover

A **canonical cover** (minimal cover) F_c is a smallest set of dependencies equivalent to F: every FD has a single attribute on the right, no attribute on the left is redundant (no extraneous attributes), and no whole FD is redundant.

```
Algorithm MINIMAL COVER
  1. Split right sides:  X → YZ  becomes  X → Y  and  X → Z
  2. Remove extraneous LHS attributes: for  XA → B, if  B ∈ X+  then drop A
  3. Remove redundant FDs: drop  X → Y  if  Y ⊆ X+  computed without it
```

Example. F = { A → BC, B → C, A → B, AB → C }.

```
Step 1:  A → B, A → C, B → C, A → B, AB → C
Step 2:  in AB → C, A+ (without using AB → C) = {A,B,C} contains C, so B is
         extraneous  ->  A → C
Step 3:  A → C is derivable from A → B and B → C, so it is redundant
F_c = { A → B,  B → C }
```

Minimal covers matter because 3NF synthesis (Chapter 19) starts from one, and because a smaller dependency set means fewer constraints to check.

17.7 Equivalence of FD Sets

Two sets F and G are **equivalent** (F ≡ G) if $F^+ = G^+$, that is, each covers the other. To test whether F covers G, check for every X → Y in G that Y ⊆ X^+ computed **under F**. If every FD of G is derivable from F and every FD of F is derivable from G, the sets are equivalent.

Chapter Summary

- X → Y means any two tuples agreeing on X must agree on Y; it is a rule of the mini-world, not an accident of the current data.
- Dependencies are trivial (Y ⊆ X) or non-trivial; only non-trivial ones drive normalization.
- A dependency is full when no attribute may be removed from its left side, partial when part of a composite key determines a non-key attribute, and transitive when it passes through a non-key attribute.
- Armstrong's axioms — reflexivity, augmentation, transitivity — are sound and complete; union, decomposition and pseudo-transitivity are derived.

- Attribute closure X^+ answers "is X a super key?" and "does X → Y hold?", and drives the search for candidate keys.
- A canonical cover is a minimal equivalent set of dependencies and is the starting point for 3NF synthesis.

Key Terms

Functional dependency · Determinant · Dependent · Trivial FD · Non-trivial FD · Full functional dependency · Partial dependency · Transitive dependency · Armstrong's axioms · Reflexivity · Augmentation · Transitivity · Union rule · Decomposition rule · Pseudo-transitivity · Attribute closure · Super key · Candidate key · Prime attribute · Canonical cover · Minimal cover · Extraneous attribute · Equivalence of FD sets

Important Formulas / Notation

```
X → Y            functional dependency
X+              closure of X under F
X is a super key      iff  X+ = R
X → Y holds           iff  Y ⊆ X+
F ≡ G                 iff  F+ = G+
```

Common Mistakes

1. Inferring an FD from the current rows instead of from the meaning of the data.
2. Applying decomposition to the left side: from `AB → C`, concluding `A → C`.
3. Treating `X → Y` as symmetric.
4. Forgetting that attributes appearing only on the right can never be part of a candidate key.
5. Calling a dependency transitive when the middle attribute is itself a key — that is not a 3NF violation.
6. Computing a closure and stopping before it is stable.
7. Confusing a minimal cover with a minimal *key*.

Solved Problems

Problem 17.1 — R(A,B,C,D,E), F = { A → BC, CD → E, B → D, E → A }. Find A^+, B^+, and all candidate keys.

Solution. A^+ = {A,B,C,D,E} (worked in Section 17.5), so A is a candidate key. B^+: start {B}; `B → D` gives {B,D}; no further FD applies (CD needs C), so B^+ = {B,D} — not a super key. E^+: `E → A` gives {E,A}, then `A → BC` gives {E,A,B,C}, then `B → D` gives all — so E is a candidate key. CD^+: `CD → E` gives {C,D,E}, then `E → A` gives A, then `A → BC` closes everything — so CD is a candidate key. BC^+: `B → D` gives {B,C,D}, then `CD → E` gives E and `E → A` gives A — everything, and neither B^+ = {B,D} nor C^+ = {C} is a super key, so BC is a candidate key too. Candidate keys: **A, E, CD, BC**. Every attribute appears in some candidate key, so all five are prime. (Missing BC is the classic error here: always test the two-attribute combinations built from attributes that appear on both sides.)

Problem 17.2 — Prove `AB → D` from F = { A → C, BC → D }.

Solution.

```
1. A → C                  given
2. AB → CB                augmentation of 1 with B
3. BC → D                 given
```

```
4. AB → D                    transitivity on 2 and 3  (CB = BC)
```

Alternatively by closure: $AB^+ = \{A,B\}$ → add C by A → C → {A,B,C} → add D by BC → D → {A,B,C,D} . Since $D \in AB^+$, the dependency holds.

Problem 17.3 — Find a canonical cover of F = { A → BC, B → C, AB → D, ABC → D }.

Solution.

```
Step 1 (single attributes on the right):
       A → B,  A → C,  B → C,  AB → D,  ABC → D
Step 2 (extraneous left attributes):
       In AB → D: A+ = {A,B,C} contains B, so B is extraneous  ->  A → D
       ABC → D likewise reduces to A → D
Step 3 (redundant FDs):
       A → C is derivable from A → B and B → C  ->  remove
       Duplicate A → D  ->  keep one
F_c = { A → B,  B → C,  A → D }
```

Problem 17.4 — R(A,B,C,D), F = { AB → C, C → D, D → A }. Find all candidate keys.

Solution. B appears only on the left, so B is in every candidate key. $B^+ = \{B\}$. Try pairs: $AB^+ = \{A,B\}$ → C by AB → C → D by C → D = all ⇒ **AB** is a candidate key. $BC^+ = \{B,C\}$ → D → A = all ⇒ **BC**. $BD^+ = \{B,D\}$ → A → then AB gives C = all ⇒ **BD**. No single attribute works, so all three are minimal. Candidate keys: **AB, BC, BD**.

Problem 17.5 — Are F = { A → B, B → C, A → C } and G = { A → B, B → C } equivalent?

Solution. Every FD of G is in F, so F covers G. For the other direction, check each FD of F under G: A → B is in G; B → C is in G; A → C requires $C \in A^+$ under G, and $A^+ = \{A,B,C\}$, so it holds. Hence G covers F, and **F ≡ G**. The extra dependency in F is redundant, which is why the canonical cover of F is G.

Problem 17.6 — In `STUDENT(student_id, dept_id, dept_name, hod_name)` with `student_id` → `dept_id`, `dept_id` → `dept_name`, `dept_name` → `hod_name`, identify the transitive dependencies and the key.

Solution. `student_id` is the only candidate key: its closure reaches every attribute, and no other attribute determines it. Transitive dependencies: `student_id` → `dept_name` (through `dept_id`) and `student_id` → `hod_name` (through `dept_id` and `dept_name`). Both are 3NF violations, because a non-prime attribute is determined by another non-prime attribute. The fix is to split off `department(dept_id, dept_name)` and `head(dept_name, hod_name)`, or more practically `department(dept_id, dept_name, hod_name)` — resolved fully in Chapter 19.

Problem 17.7 — Explain why `name` → `student_id` does not hold, even though the sample data shows no repeated names.

Solution. A functional dependency is a statement about *every legal instance*, present and future. The mini-world permits two students called "Rahim Uddin"; the fact that today's ten rows happen not to contain such a pair is a property of this instance, not of the schema. Deriving FDs from sample data is the single most common error in normalization exercises: the correct source of FDs is the requirements document and the business rules, not a data listing.

Review Questions

1. Define a functional dependency formally and explain why it is a constraint on all legal instances.
2. Distinguish trivial, non-trivial and completely non-trivial dependencies.
3. Define full, partial and transitive dependencies with examples.

4. State Armstrong's axioms and explain what soundness and completeness mean.
5. State the three derived rules and prove one of them from the axioms.
6. Give the attribute closure algorithm and explain its three uses.
7. Describe the systematic method for finding all candidate keys.
8. Define a canonical cover and give the algorithm for computing one.
9. How is equivalence of two FD sets tested?
10. Why can attributes appearing only on the right never belong to a candidate key?

Multiple Choice Questions

1. If $X \to Y$ and $Y \subseteq X$, the dependency is (a) full (b) partial (c) trivial (d) transitive.
2. From $AB \to C$ one may conclude (a) $A \to C$ (b) $B \to C$ (c) $ABD \to CD$ (d) $C \to AB$.
3. X is a super key iff (a) $X^+ = X$ (b) $X^+ = R$ (c) $X \subseteq R$ (d) $X^+ = \varnothing$.
4. A dependency on part of a composite key is (a) full (b) partial (c) trivial (d) multivalued.
5. Armstrong's axioms are (a) sound only (b) complete only (c) sound and complete (d) neither.
6. An attribute appearing only on the right side of every FD is (a) prime (b) never in a candidate key (c) always a key (d) extraneous.
7. $X \to Y$ holds iff (a) $X \subseteq Y^+$ (b) $Y \subseteq X^+$ (c) $X^+ = Y^+$ (d) $X \cap Y = \varnothing$.
8. A canonical cover has right-hand sides of (a) one attribute (b) all attributes (c) two attributes (d) any size.
9. If $X \to Y$ and $Y \to Z$ then $X \to Z$ by (a) reflexivity (b) augmentation (c) transitivity (d) union.
10. FDs should be derived from (a) sample rows (b) business rules (c) indexes (d) query logs.

Answers: 1-(c) 2-(c) 3-(b) 4-(b) 5-(c) 6-(b) 7-(b) 8-(a) 9-(c) 10-(b)

Short Questions

1. What is a determinant?
2. Give an example of a trivial FD.
3. State the augmentation axiom.
4. What does X^+ mean?
5. When is an attribute prime?
6. What is an extraneous attribute?
7. Why is decomposition valid only on the right side?
8. Give one use of a minimal cover.
9. How do you test whether F covers G?
10. Can a single attribute be a candidate key if it appears on both sides?

Long / Descriptive Questions

1. Define functional dependency and explain all its types with examples from a university schema.
2. State and prove the derived rules from Armstrong's axioms.
3. Explain the closure algorithm and use it to find all candidate keys of a given relation, showing every step.
4. Explain canonical cover with the full algorithm and a worked example.
5. Discuss why FDs must come from business rules rather than data, with an example of the error that results.

Practical Exercises

1. For R(A,B,C,D,E,F) with F = { AB → C, C → D, D → EF, F → A }, compute AB^+, C^+ and D^+, and list all candidate keys.
2. Find a canonical cover of { A → BC, CD → E, B → D, E → A } showing every step.
3. Write down the functional dependencies of the `university_db` schema and identify the candidate keys of each table.
4. Prove or disprove `CD → AB` for the FD set of exercise 1.
5. Take a table from any application you use and list its FDs from the business rules, then verify with sample data whether any additional FD appears to hold accidentally.

CHAPTER 18

Database Anomalies

An anomaly is an undesirable side effect of a poor schema: a situation in which an ordinary insert, update or delete either cannot be done, or produces an inconsistent database. Anomalies are the practical motivation for normalization; a designer who can spot them can justify every decomposition.

18.1 The Badly Designed Relation

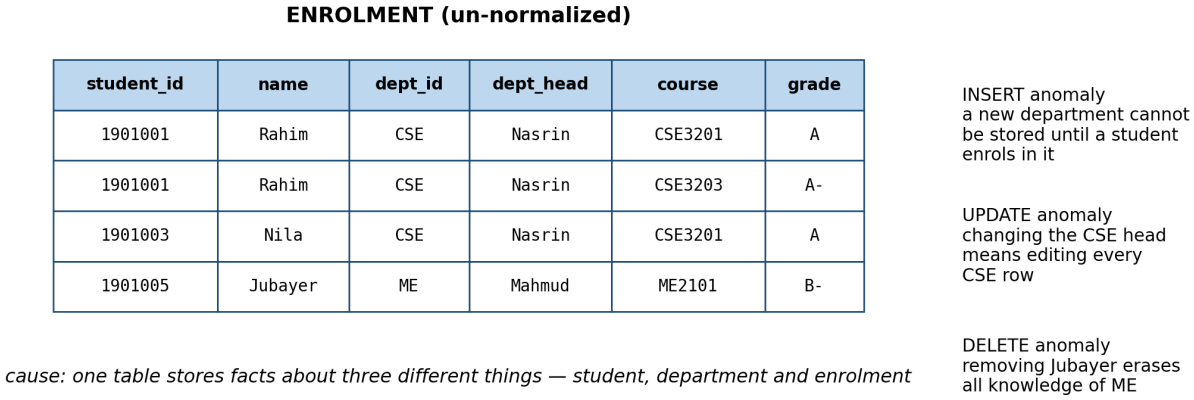


Figure 18.1: One un-normalized relation and the three anomalies it causes

```sql
CREATE TABLE enrolment_bad (
    student_id CHAR(10),
    s_name     VARCHAR(100),
    dept_id    CHAR(3),
    dept_head  VARCHAR(100),
    course_id  VARCHAR(10),
    c_title    VARCHAR(100),
    grade      CHAR(2),
    PRIMARY KEY (student_id, course_id)
);
```

student_id	s_name	dept_id	dept_head	course_id	c_title	grade
1901001	Rahim	CSE	Nasrin	CSE3201	Database Management Systems	A
1901001	Rahim	CSE	Nasrin	CSE3203	Operating Systems	A-
1901003	Nila	CSE	Nasrin	CSE3201	Database Management Systems	A
1901005	Jubayer	ME	Mahmud	ME2101	Engineering Mechanics	B-

The relation stores facts about **three different things** — students, departments and enrolments — in one place. Its functional dependencies are:

```
(student_id, course_id) → grade          the only full dependency on the key
student_id → s_name, dept_id             partial dependency
dept_id    → dept_head                   transitive through dept_id
course_id  → c_title                     partial dependency
```

Every anomaly below follows mechanically from those three offending dependencies.

18.2 Insertion Anomaly

An **insertion anomaly** occurs when a fact cannot be recorded because unrelated data is missing.

A new department, Civil Engineering, is created with Professor Selina as head. It has no students yet.

```
INSERT INTO enrolment_bad (student_id, s_name, dept_id, dept_head, course_id, c_title,
    ↪ grade)
VALUES (NULL, NULL, 'CE', 'Selina', NULL, NULL, NULL);
-- ERROR 1048: Column 'student_id' cannot be null
```

The department cannot be recorded, because `student_id` and `course_id` are the primary key and entity integrity forbids NULL there. The database is unable to store a true fact about the world purely because of how the schema was built. The same applies to a new course that nobody has yet registered for.

18.3 Update Anomaly

An **update anomaly** occurs when one fact is stored in many rows, so changing it requires changing all of them — and missing one leaves the database contradicting itself.

The CSE department gets a new head, Professor Imran.

```
UPDATE enrolment_bad SET dept_head = 'Imran' WHERE dept_id = 'CSE' AND student_id = '
    ↪ 1901001';
```

Only the rows for one student are corrected. The table now asserts that CSE has two different heads:

student_id	dept_id	dept_head
1901001	CSE	Imran
1901003	CSE	Nasrin

There is no way for the DBMS to detect the contradiction, because no constraint in the schema says the two must agree. Even a correctly written `UPDATE … WHERE dept_id = 'CSE'` does more work than necessary — one logical fact requires rewriting thousands of rows in a real database — and any process that updates a subset reintroduces the inconsistency.

18.4 Deletion Anomaly

A **deletion anomaly** occurs when removing one fact unintentionally destroys another.

Jubayer withdraws from the university:

```
DELETE FROM enrolment_bad WHERE student_id = '1901005';
```

That was the only row mentioning the Mechanical Engineering department and the only row mentioning course ME2101. Deleting a student has silently erased the existence of a department and a course. The information was not wrong and was not meant to be deleted; it disappeared because it had nowhere else to live.

18.5 Why All Three Have One Cause

Each anomaly traces back to storing **non-key facts alongside key facts**:

Anomaly	Offending dependency	Fact that has no home of its own
Insertion	`dept_id` → `dept_head`	A department
Update	`dept_id` → `dept_head`	A department's head
Deletion	`dept_id` → `dept_head`, `course_id` → `c_title`	A department, a course

The remedy is always the same: give every independent fact its own relation, so that it is stored exactly once and can exist independently.

```
department(dept_id, dept_name, dept_head)
student(student_id, s_name, dept_id)
course(course_id, c_title)
enrolment(student_id, course_id, grade)
```

With this design: a new department is a single `INSERT` into `department`; a change of head is a single `UPDATE` of one row; deleting a student removes only enrolment rows, and `department` and `course` are untouched. All three anomalies vanish simultaneously — which is exactly what Chapter 19 formalizes as 2NF and 3NF.

18.6 Redundancy and Its Costs

Cost	Explanation
Storage	"Database Management Systems" is stored once per enrolled student
Write amplification	One logical change means many physical writes
Inconsistency risk	Copies can disagree, and nothing detects it
Locking and concurrency	Mass updates hold many locks, blocking other transactions
Backup and transfer	Redundant bytes are backed up and shipped repeatedly

> The counter-case must also be stated honestly: redundancy is sometimes introduced **deliberately**, in data warehouses and reporting tables, to avoid expensive joins. That is **denormalization** (Section 19.9), and it is a conscious trade of update safety for read speed — never an excuse for an undesigned schema.

Chapter Summary

- An anomaly is an undesirable consequence of storing several independent facts in one relation.
- An insertion anomaly prevents a fact from being recorded without unrelated data.
- An update anomaly forces one logical change to be applied to many rows, risking contradiction.
- A deletion anomaly destroys information as a side effect of deleting something else.
- All three arise from partial and transitive dependencies — that is, from non-key attributes that depend on something other than the whole key.
- The cure is decomposition: one relation per independent fact, which is what normalization systematizes.

Key Terms

Anomaly · Insertion anomaly · Update anomaly · Deletion anomaly · Redundancy · Write amplification · Partial dependency · Transitive dependency · Decomposition · Denormalization

Common Mistakes

1. Describing an anomaly as "an error in the data" rather than a defect of the schema that makes errors possible.
2. Confusing the update anomaly with a simple mistake by a user; the point is that the schema *requires* multi-row updates.
3. Claiming a NULL primary key would solve the insertion anomaly — entity integrity forbids it.
4. Believing anomalies only affect large databases; they occur with four rows just as with four million.
5. Treating denormalization as a defence of bad design instead of a deliberate, documented trade-off.

Solved Problems

Problem 18.1 — For `EMPLOYEE(emp_id, name, dept_id, dept_name, dept_location)` identify all three anomalies with concrete examples.

Solution. **Insertion:** a newly created department with no employees cannot be recorded, because `emp_id` is the primary key and cannot be NULL. **Update:** relocating a department from Building A to Building B requires updating every employee row of that department; updating some of them leaves the department in two places at once. **Deletion:** deleting the last employee of a department erases the department's name and location entirely. All three come from the transitive dependency `emp_id` → `dept_id` → `dept_name, dept_location`, and all three disappear once `department(dept_id, dept_name, dept_location)` is a separate relation.

Problem 18.2 — A table `ORDER_LINE(order_id, line_no, product_id, product_name, unit_price, qty)` is proposed. Identify the anomalies and the dependencies that cause them.

Solution. Key = `(order_id, line_no)`. The offending dependency is `product_id` → `product_name, unit_price`, a partial dependency on a non-key determinant. **Insertion:** a new product cannot be catalogued until somebody orders it. **Update:** a price change must be applied to every line that mentions the product — and doing so would also, wrongly, rewrite history, since old orders should keep the price actually charged. **Deletion:** deleting the last order line for a product removes the product from the database. The correct design separates `product(product_id, product_name, current_price)` from `order_line(order_id, line_no, product_id, qty, price_charged)` — note that `price_charged` legitimately stays on the line, because it records a historical fact, not a repetition of the current price.

Problem 18.3 — Show, with SQL, that an update anomaly can leave a database self-contradictory, and state why no constraint prevents it.

Solution.

```sql
UPDATE enrolment_bad SET dept_head = 'Imran'
WHERE dept_id = 'CSE' AND course_id = 'CSE3201';       -- partial update

SELECT DISTINCT dept_id, dept_head FROM enrolment_bad WHERE dept_id = 'CSE';
```

dept_id	dept_head
CSE	Imran
CSE	Nasrin

No declarative constraint forbids this. A `UNIQUE` constraint on `(dept_id, dept_head)` would not help — the two rows differ. What is actually needed is the assertion "for all rows, dept_id determines dept_head", which is a functional dependency, and mainstream DBMSs cannot enforce FDs directly on a non-key attribute set. The only way to enforce it is structurally: put `dept_head` in a table where `dept_id` is the key, so uniqueness of the key enforces the dependency automatically.

Problem 18.4 — Are anomalies possible in a relation with no composite key and no non-key determinant? Justify.

Solution. No. If the only determinant is the primary key, then every non-key attribute is a fact about that key's entity and appears exactly once. There is no repeated value to fall out of step (no update anomaly), no independent fact trapped in the row (no deletion anomaly), and no fact requiring another entity's existence (no insertion anomaly). This is precisely the condition BCNF states formally: every determinant is a super key.

Problem 18.5 — A reporting table deliberately repeats `dept_name` in every student row. Is this an anomaly?

Solution. Not by itself — it is denormalization, and it is acceptable if three conditions hold: the table is **derived** and rebuilt from normalized sources rather than being the system of record; the rebuild process is the only writer, so partial updates cannot occur; and the design decision is documented with its reason (avoiding a join in a heavily used report). If any of those conditions fails — especially if users can update the table directly — it ceases to be denormalization and becomes precisely the un-normalized design of Section 18.1, with all three anomalies restored.

Review Questions

1. Define insertion, update and deletion anomalies with one example of each.
2. Show how all three anomalies arise from the same relation and explain the common cause.
3. Why can a partial dependency cause an insertion anomaly?
4. Explain why the DBMS cannot detect the inconsistency created by an update anomaly.
5. List the costs of redundancy beyond wasted storage.
6. How does decomposition eliminate all three anomalies at once?
7. Distinguish redundancy from denormalization.
8. Why does an order line legitimately store the price charged?
9. Can anomalies occur in a small table? Explain.
10. What structural property makes a relation anomaly-free?

Multiple Choice Questions

1. Being unable to add a department with no students is a(n) (a) update (b) insertion (c) deletion (d) referential anomaly.
2. Deleting a student and losing a course is a(n) (a) insertion (b) update (c) deletion (d) domain anomaly.
3. Anomalies are caused primarily by (a) NULL values (b) redundancy from partial and transitive dependencies (c) indexes (d) large tables.
4. Changing one fact in many rows is a(n) (a) insertion (b) update (c) deletion (d) key anomaly.
5. Which prevents an update anomaly structurally? (a) An index (b) Storing the fact in a table where its determinant is the key (c) A trigger only (d) A view.
6. Deliberate redundancy for read performance is called (a) normalization (b) denormalization (c) anomaly (d) decomposition.
7. Anomalies are a defect of the (a) data (b) schema (c) DBMS (d) hardware.
8. Which dependency causes a deletion anomaly in `ENROLMENT(student_id, course_id, c_title, grade)`? (a) `(student_id,course_id)` → `grade` (b) `course_id` → `c_title` (c) `grade` → `course_id` (d) none.
9. Entity integrity prevents solving an insertion anomaly with (a) a trigger (b) a NULL primary key (c) a view (d) an index.

10. A relation is anomaly-free when every determinant is a (a) foreign key (b) super key (c) non-key attribute (d) NULL.

Answers: 1-(b) 2-(c) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. Define an insertion anomaly.
2. Give one example of a deletion anomaly.
3. What causes an update anomaly?
4. Why is redundancy dangerous beyond storage cost?
5. What is write amplification?
6. Name the dependency type behind partial-key anomalies.
7. How does decomposition help?
8. When is redundancy acceptable?
9. Can a constraint enforce an arbitrary FD? Explain briefly.
10. Which normal form guarantees no anomalies from FDs?

Long / Descriptive Questions

1. Take an un-normalized relation and demonstrate all three anomalies step by step with SQL and results.
2. Explain the relationship between functional dependencies, redundancy and anomalies.
3. Show how decomposition into four relations removes every anomaly from the example of this chapter.
4. Discuss the costs of redundancy in a production system, with concrete examples.
5. Distinguish denormalization from poor design, stating the conditions under which deliberate redundancy is acceptable.

Practical Exercises

1. Create `enrolment_bad`, load the four sample rows, and reproduce each of the three anomalies, recording the exact error or inconsistent result.
2. Decompose it into four normalized tables, reload the same data, and repeat the three operations to show that the anomalies are gone.
3. Write a query that detects an existing update anomaly by finding departments with more than one recorded head.
4. Measure the storage used by the un-normalized and normalized versions with 100,000 generated rows.
5. Build a denormalized reporting table from the normalized schema and write the refresh statement that keeps it correct.

CHAPTER 19

Normalization

Normalization is the process of decomposing relations so that redundancy and the anomalies of Chapter 18 are eliminated, while preserving all the information and, as far as possible, all the dependencies. Each normal form is a testable condition, so the process is objective: a schema either satisfies 3NF or it does not.

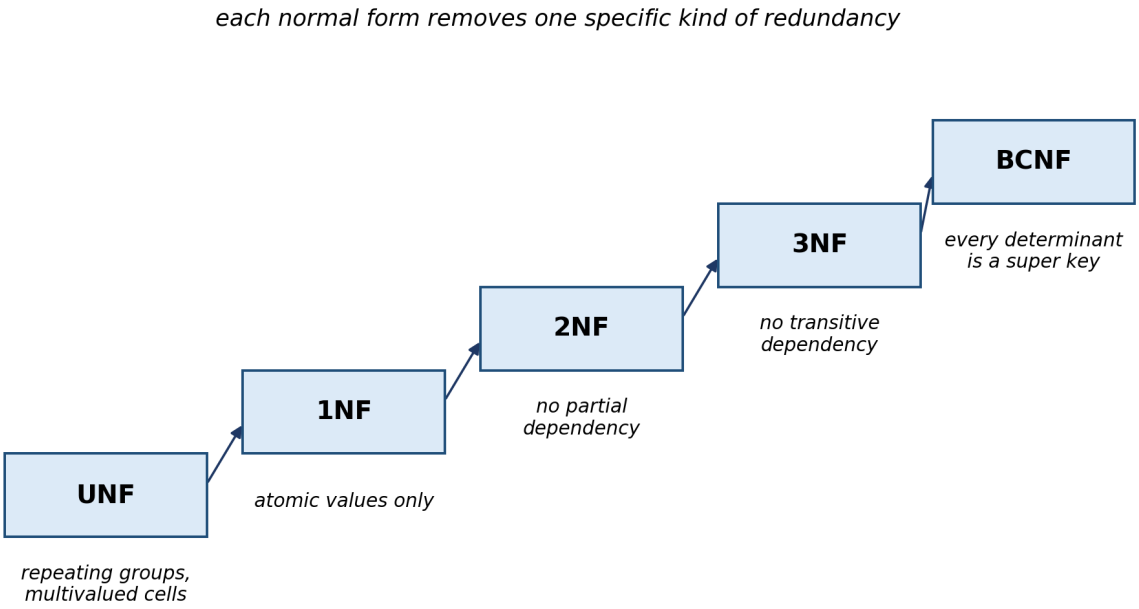


Figure 19.1: The normal forms, and what each one removes

19.1 First Normal Form (1NF)

Condition. A relation is in 1NF if every attribute value is **atomic**: no repeating groups, no multivalued attributes, no nested relations, and every row has the same number of columns.

Violation

student_id	name	phones	courses
1901001	Rahim	01712-345678, 01911-222333	CSE3201, CSE3203
1901003	Nila	01533-121212	CSE3201

Problems. Searching for a phone number needs string matching; counting courses needs parsing; a fourth phone number has nowhere to go; and `UPDATE` cannot change one value without rewriting the whole string.

Conversion to 1NF

The correct conversion is **not** to add `phone1`, `phone2`, `phone3` — that merely fixes an arbitrary limit and leaves most cells NULL. Each multivalued attribute becomes its own relation:

```
CREATE TABLE student (
    student_id CHAR(10) PRIMARY KEY,
    name       VARCHAR(100) NOT NULL
```

```
);
CREATE TABLE student_phone (
    student_id CHAR(10), phone VARCHAR(20),
    PRIMARY KEY (student_id, phone),
    FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE
);
CREATE TABLE enrolment (
    student_id CHAR(10), course_id VARCHAR(10),
    PRIMARY KEY (student_id, course_id)
);
```

19.2 Second Normal Form (2NF)

Condition. A relation is in 2NF if it is in 1NF **and** every non-prime attribute is **fully** functionally dependent on every candidate key — that is, there is no partial dependency.

2NF is only at risk when the key is **composite**. A relation in 1NF with a single-attribute key is automatically in 2NF.

Violation

```
ENROLMENT(student_id, course_id, s_name, c_title, credit, grade)
key = (student_id, course_id)

(student_id, course_id) → grade     full — correct
student_id → s_name                 PARTIAL — violates 2NF
course_id  → c_title, credit        PARTIAL — violates 2NF
```

Decomposition

```
STUDENT(student_id, s_name)
COURSE(course_id, c_title, credit)
ENROLMENT(student_id, course_id, grade)
```

```
CREATE TABLE student   (student_id CHAR(10) PRIMARY KEY, s_name VARCHAR(100) NOT NULL);
CREATE TABLE course    (course_id VARCHAR(10) PRIMARY KEY, c_title VARCHAR(120) NOT NULL
    ↪ ,
                        credit DECIMAL(2,1) NOT NULL);
CREATE TABLE enrolment (student_id CHAR(10), course_id VARCHAR(10), grade CHAR(2),
                        PRIMARY KEY (student_id, course_id),
                        FOREIGN KEY (student_id) REFERENCES student(student_id),
                        FOREIGN KEY (course_id)  REFERENCES course(course_id));
```

Result. A course title is now stored once instead of once per enrolled student, and a new course can be created before anyone registers for it.

19.3 Third Normal Form (3NF)

Condition. A relation is in 3NF if it is in 2NF and no non-prime attribute is **transitively** dependent on a candidate key. Equivalently, for every non-trivial dependency $X \to A$, either **X is a super key** or **A is prime**.

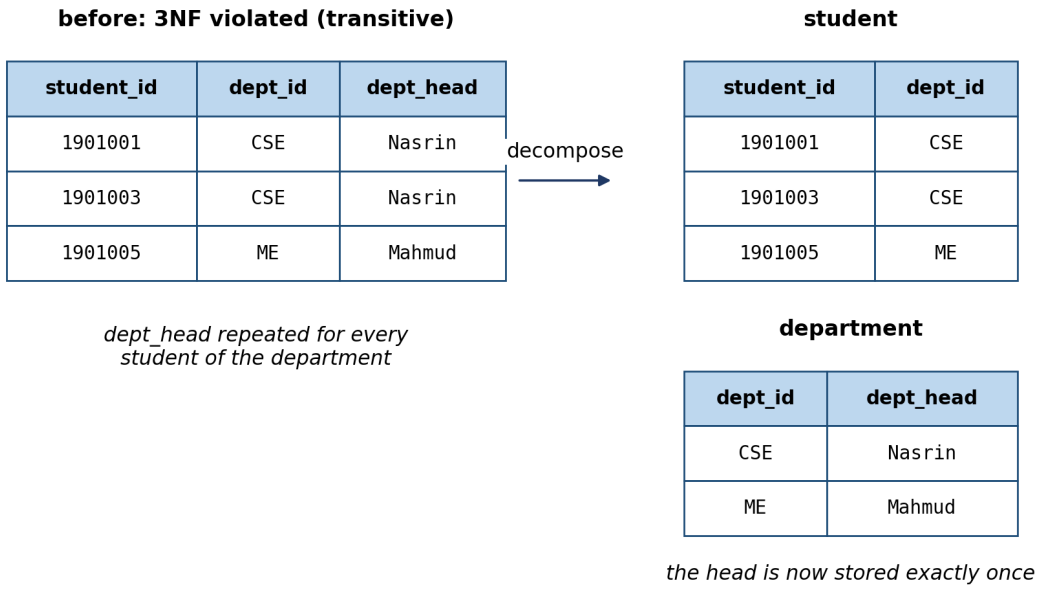


Figure 19.2: Removing a transitive dependency

Violation

```
STUDENT(student_id, s_name, dept_id, dept_name, dept_head)
key = student_id

student_id → dept_id            fine
dept_id    → dept_name, dept_head   dept_id is NOT a super key
therefore   student_id → dept_name  TRANSITIVELY   – violates 3NF
```

Decomposition

```
STUDENT(student_id, s_name, dept_id)
DEPARTMENT(dept_id, dept_name, dept_head)
```

Every department's head is now stored exactly once, so the update anomaly of Section 18.3 becomes structurally impossible: uniqueness of the primary key `dept_id` enforces the dependency automatically.

The second clause matters

The alternative form of the 3NF condition — "or A is prime" — is what makes 3NF weaker than BCNF, and it is examined often. Consider:

```
R(city, street, postcode)
   (city, street) → postcode
   postcode → city
```

`postcode → city` has a non-super-key determinant, which looks like a violation, but `city` **is** a prime attribute (it belongs to the candidate key `(city, street)`). So R is in 3NF — while failing BCNF, as Section 19.4 shows.

19.4 Boyce–Codd Normal Form (BCNF)

Condition. A relation is in BCNF if for **every** non-trivial dependency `X → Y`, **X is a super key**. There is no exemption for prime attributes.

BCNF is strictly stronger than 3NF: every BCNF relation is in 3NF, but not conversely.

The classic example

```
CLASS(student_id, course_id, instructor_id)

Rules:  a student takes a course from exactly one instructor
        each instructor teaches exactly one course

FDs:    (student_id, course_id) → instructor_id
        instructor_id → course_id

Candidate keys: (student_id, course_id) and (student_id, instructor_id)
Prime attributes: student_id, course_id, instructor_id
```

Is it in 3NF? Yes: `instructor_id → course_id` has a non-super-key determinant, but `course_id` is prime, so the second clause saves it. Is it in BCNF? **No**: `instructor_id` is not a super key.

The redundancy is real. The fact "instructor I teaches course C" is repeated once for every student taking that course, so it can fall out of step, and it cannot be recorded for an instructor with no students.

Decomposition

```
R1(instructor_id, course_id)        key: instructor_id
R2(student_id, instructor_id)       key: (student_id, instructor_id)
```

Both are in BCNF. But note the cost: the dependency `(student_id, course_id) → instructor_id` is **no longer enforceable within a single relation** — it spans both. This is the standard trade-off:

> **BCNF decomposition is always lossless, but it is not always dependency-preserving. 3NF decomposition is always both lossless and dependency-preserving.** This is why practical designs target 3NF and accept BCNF only when it comes free.

19.5 Lossless-Join Decomposition

A decomposition of R into R_1 and R_2 is **lossless** if joining them reconstructs R exactly — no tuples lost, and crucially no **spurious tuples** invented.

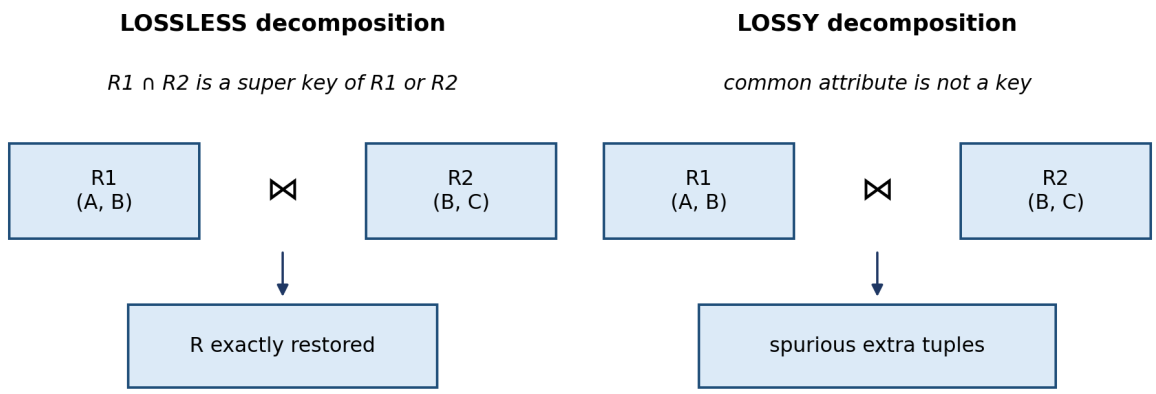


Figure 19.3: Lossless versus lossy decomposition

Test (binary decomposition). $R \to (R_1, R_2)$ is lossless if and only if the common attributes form a super key of at least one of them:

```
    (R1 ∩ R2) → R1     or     (R1 ∩ R2) → R2
```

Lossy example

```
R(student_id, course_id, instructor_id)
R1(student_id, course_id)      R2(course_id, instructor_id)
R1 ∩ R2 = {course_id}, which is a key of neither
```

R1			R2	
S1	C1		C1	I1
S2	C1		C1	I2

The natural join produces four rows — including "S1 taught by I2" and "S2 taught by I1", neither of which was in the original. Those are spurious tuples: the database now asserts facts that were never true. A lossy decomposition is not a performance problem, it is a **correctness** failure.

Lossless example

```
R1(instructor_id, course_id)        R2(student_id, instructor_id)
R1 ∩ R2 = {instructor_id}, which is the key of R1  ->  lossless
```

19.6 Dependency Preservation

A decomposition is **dependency-preserving** if the union of the dependencies enforceable *within* the individual relations is equivalent to the original set F. If a dependency spans two relations, checking it requires a join at every insert — which no DBMS does automatically, so in practice the rule goes unenforced.

In the `CLASS` example, the BCNF decomposition loses `(student_id, course_id) → instructor_id`: nothing now prevents a student from being recorded with two instructors for the same course, because that fact is visible only after joining R1 and R2.

Property	3NF	BCNF
Removes all partial and transitive dependencies	Yes	Yes
Removes every non-super-key determinant	No (prime exemption)	Yes
Lossless decomposition always achievable	Yes	Yes
Dependency preservation always achievable	Yes	**No**
Redundancy fully eliminated	Almost	Yes

19.7 3NF Synthesis Algorithm

Unlike BCNF, 3NF can be achieved constructively and with guarantees.

```
Algorithm 3NF SYNTHESIS
  1. Compute a canonical cover Fc of F.
  2. For each FD  X → A  in Fc, create a relation  R_i = X ∪ A
     (dependencies with the same X may be combined into one relation).
  3. If no R_i contains a candidate key of R, add one relation consisting
     of a candidate key.
  4. Remove any R_i that is a subset of another R_j.
```

The result is guaranteed to be in 3NF, lossless and dependency-preserving.

Worked example. R(A,B,C,D,E), F = { A → BC, CD → E, B → D, E → A }.

```
Step 1: canonical cover = { A → B, A → C, CD → E, B → D, E → A }
Step 2: R1(A,B,C)   from A → B and A → C
        R2(C,D,E)   from CD → E
```

```
        R3(B,D)     from B → D
        R4(E,A)     from E → A
Step 3: candidate keys are A, E, CD and BC (computed in Chapter 17);
        R1 contains A  ->  nothing to add
Step 4: no relation is contained in another
```

19.8 Higher Normal Forms

4NF — multivalued dependencies. A multivalued dependency `X →→ Y` means that for each X value, a set of Y values exists independently of the remaining attributes. A relation is in 4NF if every non-trivial MVD has a super key on its left.

```
INSTRUCTOR_INFO(instructor_id, course, phone)
   instructor_id →→ course
   instructor_id →→ phone
```

Courses and phone numbers are unrelated, yet storing them together forces every combination:

instructor_id	course	phone
I1	CSE3201	01712-111111
I1	CSE3201	01911-222222
I1	CSE3203	01712-111111
I1	CSE3203	01911-222222

Two courses and two phones produce four rows, and adding one phone adds two more. The fix is two relations: `teaches(instructor_id, course)` and `instructor_phone(instructor_id, phone)`.

5NF (project–join normal form). Concerns join dependencies that only a three-way (or higher) decomposition removes. Rare in practice.

6NF and DKNF. Mainly of theoretical interest; 6NF appears in temporal databases.

19.9 Denormalization

Denormalization deliberately reintroduces redundancy to avoid joins.

```sql
-- a reporting table refreshed nightly, never edited by users
CREATE TABLE rpt_student_summary AS
SELECT s.student_id, s.name, d.dept_name,
       COUNT(e.enrollment_id) AS courses_taken,
       ROUND(AVG(r.marks), 2) AS avg_marks
FROM   student s
JOIN   department d ON d.dept_id = s.dept_id
LEFT   JOIN enrollment e ON e.student_id = s.student_id
LEFT   JOIN result     r ON r.student_id = s.student_id
GROUP  BY s.student_id, s.name, d.dept_name;
```

When denormalization is justified	When it is not
Read-heavy reporting and dashboards	The system of record for transactional data
Data warehouses and star schemas (Chapter 36)	Tables users update directly
Precomputed aggregates refreshed on a schedule	To avoid learning how to write a join
A measured, documented performance problem	A guess about performance

The discipline is: normalize first, measure, and denormalize only where the measurement demands it — keeping the normalized tables as the authoritative source.

Chapter Summary

- 1NF requires atomic values; multivalued attributes become separate relations.
- 2NF removes partial dependencies and is only at risk with composite keys.
- 3NF removes transitive dependencies: every determinant is a super key **or** the dependent is prime.
- BCNF drops the prime exemption: every determinant must be a super key.
- A decomposition is lossless if the common attributes are a super key of one part; otherwise the join invents spurious tuples.
- 3NF decomposition is always lossless and dependency-preserving; BCNF is always lossless but may lose dependencies.
- 4NF removes multivalued dependencies; 5NF removes join dependencies.
- Denormalization is a deliberate, measured trade of write safety for read speed, appropriate for derived reporting data only.

Key Terms

Normalization · 1NF · Atomic value · Repeating group · 2NF · Partial dependency · Prime attribute · 3NF · Transitive dependency · BCNF · Determinant · Super key · Lossless join · Spurious tuple · Dependency preservation · Canonical cover · 3NF synthesis · Multivalued dependency · 4NF · Join dependency · 5NF · Denormalization

Important Formulas / Notation

```
1NF    all values atomic
2NF    1NF and every non-prime attribute fully dependent on every candidate key
3NF    2NF and for every X → A:  X is a super key  OR  A is prime
BCNF   for every non-trivial X → Y:  X is a super key
Lossless test:  (R1 ∩ R2) → R1   or   (R1 ∩ R2) → R2
```

Common Mistakes

1. Testing 2NF on a relation with a single-attribute key; it is automatically in 2NF.
2. Forgetting the "or A is prime" clause of 3NF, and calling a 3NF relation a violation.
3. Decomposing without checking losslessness, producing spurious tuples.
4. Believing BCNF is always achievable with dependency preservation.
5. Adding `phone1`, `phone2`, `phone3` as a "1NF fix".
6. Normalizing blindly to 5NF in an operational system, producing dozens of joins for every query.
7. Treating denormalization as an alternative to normalization rather than a later, measured exception.
8. Deriving the FDs from the sample rows and then normalizing the wrong dependencies.

Solved Problems

Problem 19.1 — Normalize to 3NF, showing each step:

`STUDENT(student_id, name, phones, dept_id, dept_name, dept_head, course_id, c_title,`

`grade)`.

Solution.

1NF — `phones` is multivalued; remove it to its own relation:

```
STUDENT_1NF(student_id, name, dept_id, dept_name, dept_head, course_id, c_title, grade)
STUDENT_PHONE(student_id, phone)
key of STUDENT_1NF = (student_id, course_id)
```

2NF — remove partial dependencies `student_id → name, dept_id, dept_name, dept_head` and `course_id → c_title`:

```
STUDENT_2NF(student_id, name, dept_id, dept_name, dept_head)
COURSE(course_id, c_title)
ENROLMENT(student_id, course_id, grade)
STUDENT_PHONE(student_id, phone)
```

3NF — in `STUDENT_2NF`, `dept_id → dept_name, dept_head` is transitive:

```
STUDENT(student_id, name, dept_id)
DEPARTMENT(dept_id, dept_name, dept_head)
COURSE(course_id, c_title)
ENROLMENT(student_id, course_id, grade)
STUDENT_PHONE(student_id, phone)
```

All five relations are in 3NF, and in fact in BCNF, since every determinant is now a key.

Problem 19.2 — R(A,B,C,D) with F = { AB → C, C → D, D → A }. Determine the highest normal form.

Solution. Candidate keys (Chapter 17, Problem 17.4): AB, BC, BD. Prime attributes: A, B, C, D — all of them. **1NF:** assume yes. **2NF:** the non-prime set is empty, so no partial dependency can exist ⇒ 2NF holds. **3NF:** every dependent is prime, so the second clause is satisfied for all FDs ⇒ 3NF holds. **BCNF:** `C → D` has determinant C, and $C^+ = \{C,D,A\} \neq R$, so C is not a super key ⇒ **BCNF fails**. Highest normal form: **3NF**.

Problem 19.3 — Is the decomposition of R(A,B,C) with F = { A → B, B → C } into R_1(A,B) and R_2(B,C) lossless and dependency-preserving?

Solution. $R_1 \cap R_2 = \{B\}$. Is B a super key of R_2? $B^+ = \{B,C\} = R_2$, yes ⇒ **lossless**. Dependencies: `A → B` is enforceable in R_1; `B → C` in R_2; their union is F ⇒ **dependency-preserving**. Both relations are also in BCNF, since A is the key of R_1 and B of R_2. This is the ideal outcome.

Problem 19.4 — Show that decomposing `CLASS(student_id, course_id, instructor_id)` into BCNF loses a dependency, and discuss the practical consequence.

Solution. With F = { (student_id, course_id) → instructor_id, instructor_id → course_id }, the BCNF decomposition is R_1(instructor_id, course_id) and R_2(student_id, instructor_id). $R_1 \cap R_2$ = {instructor_id}, a key of R_1 ⇒ lossless. But `(student_id, course_id) → instructor_id` involves attributes split across both relations, so no single-table constraint can enforce it. **Consequence:** a student may be recorded with two different instructors for the same course — R_2 holds (S1, I1) and (S1, I2), and R_1 says both teach C1 — which the original schema forbade. Enforcing the rule now requires a trigger performing a join, or accepting 3NF instead. Most practitioners accept 3NF here.

Problem 19.5 — Apply 3NF synthesis to R(A,B,C,D,E,F) with F = { A → B, B → CD, AD → E, F → A }.

Solution.

```
Canonical cover: A → B, B → C, B → D, AD → E, F → A
   check AD → E: A+ = {A,B,C,D}, which contains D, so D is extraneous  ->  A → E
Fc = { A → B, A → E, B → C, B → D, F → A }

Relations:  R1(A,B,E)   from A → B, A → E
            R2(B,C,D)   from B → C, B → D
            R3(F,A)     from F → A

Candidate key of R:  F appears only on the left  ->  F must be in every key.
F+ = {F,A,B,C,D,E} = R  ->  F is the only candidate key.
R3 contains F, so no extra relation is needed.
```

All three relations are in 3NF, the decomposition is lossless ($R3 \cap R1 = \{A\}$, a key of R1; $R1 \cap R2 = \{B\}$, a key of R2), and every original dependency is preserved.

Problem 19.6 — A relation is in 3NF but not BCNF. Give the general shape of such a relation and explain why it still has redundancy.

Solution. The shape is: a non-super-key determinant X whose dependent A is prime — `X → A` where $X^+ \neq R$ and A belongs to some candidate key. The `CLASS` relation and the `(city, street, postcode)` relation are both of this form. Redundancy remains because the fact `X → A` is re-stated in every tuple containing that X value: `postcode → city` means the city is repeated for every street sharing the postcode, and a postcode cannot be registered before a street is known. 3NF tolerates this because eliminating it may cost dependency preservation, which is judged the greater loss.

Problem 19.7 — When is denormalization appropriate, and what safeguards must accompany it?

Solution. Appropriate when a measured read-performance problem exists on data that is not the system of record — typically reporting tables, dashboards, star-schema fact and dimension tables, or a precomputed aggregate such as `course.enrolled_count`. Safeguards: (i) the normalized tables remain authoritative, and the denormalized copy is always derived; (ii) exactly one automated process writes the copy — a scheduled rebuild, a materialized-view refresh or a trigger — so no partial update is possible; (iii) the staleness window is documented and acceptable to users; (iv) the reason is recorded in the schema documentation so a later maintainer does not "fix" it; and (v) the improvement is verified by measurement, not assumed.

Review Questions

1. Define normalization and state its objectives.
2. State the condition for 1NF and convert a violating relation, explaining why extra columns are the wrong fix.
3. Define 2NF and explain why a single-attribute key guarantees it.
4. State both clauses of the 3NF condition and explain the prime-attribute exemption.
5. Define BCNF and prove that it is strictly stronger than 3NF with an example.
6. State the lossless-join test and demonstrate a lossy decomposition with spurious tuples.
7. Define dependency preservation and give an example where BCNF loses it.
8. Give the 3NF synthesis algorithm and its guarantees.
9. Explain 4NF and multivalued dependencies with an example.
10. Discuss denormalization: when it is right, and what safeguards it needs.

Multiple Choice Questions

1. A relation with a multivalued attribute violates (a) 1NF (b) 2NF (c) 3NF (d) BCNF.

2. Partial dependency is removed in (a) 1NF (b) 2NF (c) 3NF (d) 4NF.
3. Transitive dependency is removed in (a) 1NF (b) 2NF (c) 3NF (d) BCNF.
4. In BCNF every determinant must be a (a) prime attribute (b) super key (c) foreign key (d) single attribute.
5. A decomposition is lossless if the common attributes form a (a) foreign key (b) super key of one relation (c) candidate key of both (d) prime attribute.
6. Which is always achievable with dependency preservation? (a) BCNF (b) 3NF (c) 4NF (d) 5NF.
7. A relation with a single-attribute key and atomic values is at least in (a) 1NF (b) 2NF (c) 3NF (d) BCNF.
8. Spurious tuples result from a (a) lossless (b) lossy (c) dependency-preserving (d) 3NF decomposition.
9. 4NF concerns (a) functional (b) multivalued (c) join (d) inclusion dependencies.
10. Denormalization is appropriate for (a) OLTP tables (b) derived reporting tables (c) primary keys (d) constraints.

Answers: 1-(a) 2-(b) 3-(c) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What does atomic mean in 1NF?
2. When can 2NF be violated?
3. State the 3NF condition in one line.
4. Why is BCNF stronger than 3NF?
5. What is a spurious tuple?
6. State the binary lossless-join test.
7. What does dependency preservation mean?
8. Give one guarantee of 3NF synthesis.
9. Define a multivalued dependency.
10. Name two safeguards for denormalization.

Long / Descriptive Questions

1. Explain 1NF, 2NF, 3NF and BCNF with definitions, violating examples, decompositions and resulting SQL.
2. Explain lossless join and dependency preservation with tests and worked examples of failure.
3. Prove with an example that BCNF decomposition may sacrifice dependency preservation, and explain the practical decision this forces.
4. Present the 3NF synthesis algorithm and apply it fully to a given relation and FD set.
5. Discuss 4NF, 5NF and denormalization, with examples and practical guidance.

Practical Exercises

1. Normalize `enrolment_bad` from Chapter 18 step by step to BCNF, creating the tables at each stage and migrating the data.
2. For R(A,B,C,D,E) with F = { A → BC, CD → E, B → D, E → A }, determine the highest normal form and apply 3NF synthesis.
3. Construct a lossy decomposition in SQL, join the parts and count the spurious tuples produced.
4. Build the `CLASS` example, decompose it into BCNF, and write the trigger that would be needed to restore the lost dependency.

5. Create a denormalized reporting table from `university_db` with a refresh procedure, and measure the query time against the normalized join.

CHAPTER 20

Relational Algebra

Relational algebra is a **procedural** query language: an expression specifies a sequence of operations on relations, and each operation returns a relation. That closure property — relations in, relations out — is what allows operations to be composed into expressions of any depth, and it is the formal basis on which query optimization rests (Chapter 26).

20.1 The Operators

Group	Operator	Symbol	Purpose
Unary	Selection	σ	Choose rows
Unary	Projection	π	Choose columns
Unary	Rename	ρ	Rename a relation or its attributes
Set	Union	∪	Rows in either relation
Set	Intersection	∩	Rows in both
Set	Difference	−	Rows in the first but not the second
Set	Cartesian product	×	Every pairing
Join	Theta / equi / natural join	⋈	Combine on a condition
Join	Outer joins	⟕ ⟖ ⟗	Preserve unmatched rows
Special	Division	÷	“For all” queries
Extended	Aggregation	$\mathcal{G}$	Grouped summaries

Six of these — σ, π, ∪, −, × and ρ — are the **fundamental** (primitive) operators. Every other operator can be expressed in terms of them, which is why the set is called complete.

20.2 Selection (σ)

Selects the tuples satisfying a predicate. Degree is unchanged; cardinality shrinks.

```
σ_{condition}(R)

σ_{dept_id = 'CSE'}(STUDENT)
σ_{cgpa > 3.5 ∧ dept_id = 'CSE'}(STUDENT)
σ_{dept_id = 'CSE' ∨ dept_id = 'EEE'}(STUDENT)
```

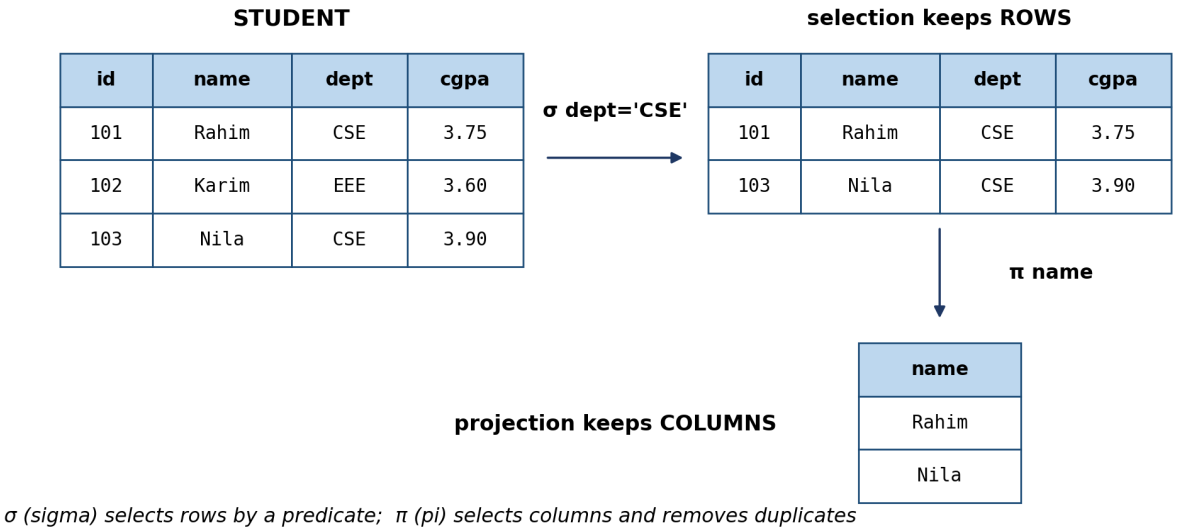


Figure 20.1: Selection keeps rows; projection keeps columns

SQL equivalent: `SELECT * FROM student WHERE dept_id = 'CSE';`

Selection is **commutative**: σ_p(σ_q(R)) = σ_q(σ_p(R)) = σ_{p ∧ q}(R).

20.3 Projection (π)

Selects attributes and — because the result is a set — **removes duplicates**.

```
π_{name, cgpa}(STUDENT)
π_{dept_id}(STUDENT)          duplicates eliminated automatically
```

SQL equivalent: `SELECT DISTINCT dept_id FROM student;`

> The duplicate removal is a genuine difference between algebra and SQL. Algebra works on sets, so π always eliminates duplicates; SQL works on multisets (bags), so `SELECT` keeps them unless `DISTINCT` is written. This is the most frequently examined discrepancy between the two.

Projection is **not** generally commutative with selection, but selection can be pushed *inside* a projection when the predicate uses only projected attributes — an optimization rule of Chapter 26:

```
π_{name}(σ_{dept_id='CSE'}(STUDENT))      cannot become σ_{dept_id='CSE'}(π_{name}(
    ↪ STUDENT))
```

because `dept_id` is gone after the inner projection.

20.4 Rename (ρ)

```
ρ_{S}(STUDENT)                        rename the relation to S
ρ_{S(id, nm, dp)}(STUDENT)            rename relation and attributes
```

Rename is what makes self joins expressible: two copies of one relation need distinct names.

```
σ_{A.cgpa > B.cgpa}(ρ_A(STUDENT) × ρ_B(STUDENT))
```

20.5 Set Operations

Union, intersection and difference require **union compatibility**: the same number of attributes, with pairwise compatible domains.

```
π_{student_id}(σ_{course_id=1}(ENROLMENT)) ∪ π_{student_id}(σ_{course_id=2}(ENROLMENT))
π_{student_id}(σ_{course_id=1}(ENROLMENT)) ∩ π_{student_id}(σ_{course_id=2}(ENROLMENT))
π_{student_id}(STUDENT) − π_{student_id}(ENROLMENT)      students who enrolled in
    ↪ nothing
```

Intersection is not primitive: `R ∩ S = R − (R − S)`.

20.6 Cartesian Product and Joins

```
STUDENT × DEPARTMENT        every student paired with every department
```

If STUDENT has m tuples of degree a and DEPARTMENT has n tuples of degree b, the product has $m \times n$ tuples of degree $a + b$.

Theta join and equi join

```
R ⋈_{θ} S  =  σ_{θ}(R × S)

STUDENT ⋈_{STUDENT.dept_id = DEPARTMENT.dept_id} DEPARTMENT     equi join
EXAM ⋈_{marks > pass_mark} GRADE_RULE                            theta join
```

A **theta join** uses any comparison; an **equi join** uses only equality. Both are defined in terms of the product and a selection, which is why they are not primitive.

Natural join

The natural join (⋈ with no condition) joins on all commonly named attributes and keeps only one copy of each:

```
STUDENT ⋈ DEPARTMENT        joins on dept_id automatically
```

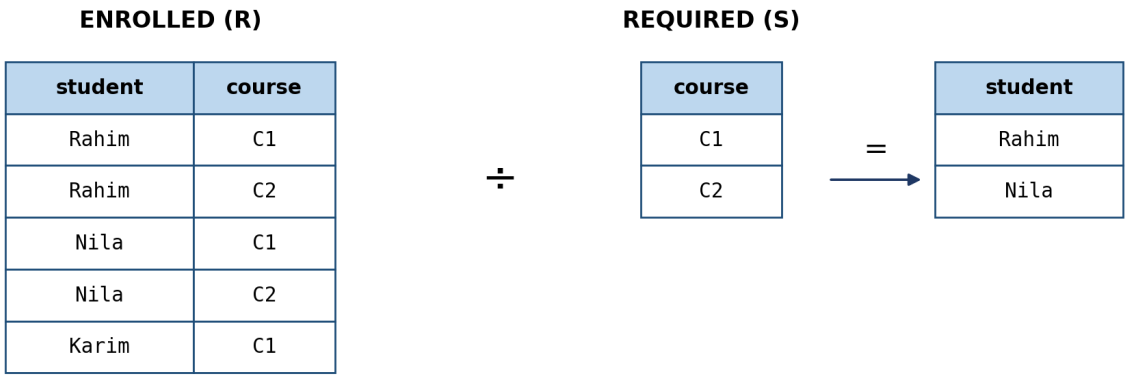
Formally: `R ⋈ S = π_{R ∪ S attributes}(σ_{R.A = S.A for all common A}(R × S))`.

Outer joins

```
STUDENT ⟕ ENROLMENT         left outer:  keep all students, pad with nulls
STUDENT ⟖ ENROLMENT         right outer
STUDENT ⟗ ENROLMENT         full outer
```

20.7 Division (÷)

Division answers **"for all"** questions, which no other single operator expresses.



division returns the students associated with EVERY course in S — Karim is excluded

Figure 20.2: Division — students enrolled in every required course

```
R ÷ S  =  { t | t ∈ π_{R−S}(R)  and  for every s ∈ S, (t, s) ∈ R }
```

Problem. Find students enrolled in **every** CSE course.

```
π_{student_id, course_id}(ENROLMENT) ÷ π_{course_id}(σ_{dept_id='CSE'}(COURSE))
```

Division is expressible in the primitives — the standard identity is

```
R ÷ S  =  π_{R−S}(R)  −  π_{R−S}( (π_{R−S}(R) × S)  −  R )
```

which reads: take all candidate values, pair each with every required value, subtract the pairs that actually exist, and whatever candidates remain are those *missing* something; subtract those from the candidates.

SQL has no division operator; the standard implementations are the grouped-count and the double `NOT EXISTS`:

```sql
-- grouped count: enrolled in as many CSE courses as exist
SELECT   e.student_id
FROM     enrollment e JOIN course c ON c.course_id = e.course_id
WHERE    c.dept_id = 'CSE'
GROUP BY e.student_id
HAVING   COUNT(DISTINCT e.course_id) = (SELECT COUNT(*) FROM course WHERE dept_id='CSE')
    ↪ ;

-- double NOT EXISTS: there is no CSE course that this student has not taken
SELECT s.student_id FROM student s
WHERE NOT EXISTS (
    SELECT 1 FROM course c WHERE c.dept_id = 'CSE'
    AND NOT EXISTS (SELECT 1 FROM enrollment e
                    WHERE e.course_id = c.course_id AND e.student_id = s.student_id));
```

20.8 Extended Operators

Aggregation

```
𝒢_{COUNT(student_id)}(STUDENT)
dept_id 𝒢_{AVG(cgpa) → avg_cgpa}(STUDENT)
```

The left subscript is the grouping attribute list; the right is the aggregate list. SQL equivalent: `SELECT`

```
dept_id, AVG(cgpa) FROM student GROUP BY dept_id;
```

Generalized projection

Allows expressions, not just attribute names: `π_{name, cgpa × 25}(STUDENT)`.

Assignment

```
CSE_STUDENTS ← σ_{dept_id='CSE'}(STUDENT)
RESULT       ← π_{name}(CSE_STUDENTS)
```

Assignment lets a long expression be built in readable steps — exactly what a CTE does in SQL.

20.9 Worked Expressions

E1. Names of CSE students with CGPA above 3.5.

```
π_{name}( σ_{dept_id='CSE' ∧ cgpa > 3.5}(STUDENT) )
```

E2. Names of students together with their department names.

```
π_{name, dept_name}( STUDENT ⋈ DEPARTMENT )
```

E3. Titles of courses taken by Rahim.

```
π_{title}( σ_{name='Rahim Uddin'}(STUDENT) ⋈ ENROLMENT ⋈ COURSE )
```

E4. Students who have taken no course.

```
π_{student_id}(STUDENT) − π_{student_id}(ENROLMENT)
```

E5. Students who have taken course 1 but not course 2.

```
π_{student_id}(σ_{course_id=1}(ENROLMENT)) − π_{student_id}(σ_{course_id=2}(ENROLMENT))
```

E6. The student with the highest CGPA.

```
π_{student_id}(STUDENT) − π_{A.student_id}( σ_{A.cgpa < B.cgpa}( ρ_A(STUDENT) × ρ_B(
    ↪ STUDENT) ) )
```

E6 is worth studying: "the maximum" is expressed as "everything that is not smaller than something else". Algebra has no aggregate `MAX` among its primitives, so extremal queries are written as differences.

Chapter Summary

- Relational algebra is procedural and closed: every operator takes relations and returns a relation.
- The six primitives are σ, π, ρ, ∪, − and ×; intersection, joins and division are derived.
- Selection filters rows, projection filters columns and removes duplicates.
- Set operations require union compatibility.
- Theta, equi and natural joins are products followed by selections; outer joins preserve unmatched rows with nulls.
- Division expresses "for all" queries and has no SQL counterpart; it is written with a grouped count or double `NOT EXISTS`.
- Extended operators add aggregation, generalized projection and assignment.

Key Terms

Relational algebra · Procedural language · Closure property · Selection σ · Projection π · Rename ρ · Union ∪ · Intersection ∩ · Set difference − · Cartesian product × · Theta join · Equi join · Natural join ⋈ · Outer join · Division ÷ · Aggregation $\mathcal{G}$ · Generalized projection · Assignment · Union compatibility · Fundamental operators

Important Formulas / Notation

```
R ∩ S      = R − (R − S)
R ⋈_θ S    = σ_θ(R × S)
R ÷ S      = π_{R−S}(R) − π_{R−S}((π_{R−S}(R) × S) − R)
|R × S|    = |R| × |S|,   degree(R × S) = degree(R) + degree(S)
```

Common Mistakes

1. Forgetting that projection removes duplicates, unlike SQL's `SELECT`.
2. Using intersection or join as if they were primitive operators.
3. Applying set operations to union-incompatible relations.
4. Omitting ρ in a self-referencing expression, so the two copies cannot be distinguished.
5. Pushing a selection inside a projection that has already discarded the predicate's attribute.
6. Writing division as a simple difference; it requires the full identity.
7. Expecting an aggregate function among the primitives.

Solved Problems

Problem 20.1 — Write in relational algebra: names of students who have scored an A in any course.

Solution.

```
π_{name}( STUDENT ⋈ σ_{grade='A'}(ENROLMENT) )
```

The selection is applied to ENROLMENT **before** the join, which is both correct and efficient — pushing selections down is the first rule of query optimization.

Problem 20.2 — Express `R ∩ S` using only primitive operators and verify on small relations.

Solution. `R ∩ S = R − (R − S)`. With R = {1,2,3} and S = {2,3,4}: `R − S` = {1}; `R − (R − S)` = {1,2,3} − {1} = {2,3} = `R ∩ S`. ✓

Problem 20.3 — Find students enrolled in all courses offered by their own department, and give both the algebra and the SQL.

Solution.

```
Algebra:
   ENR ← π_{student_id, course_id}(ENROLMENT)
   REQ ← π_{course_id}(σ_{dept_id='CSE'}(COURSE))
   ANSWER ← ENR ÷ REQ
```

```sql
SELECT   e.student_id
FROM     enrollment e
JOIN     course c ON c.course_id = e.course_id
JOIN     student s ON s.student_id = e.student_id
WHERE    c.dept_id = s.dept_id
```

```
GROUP BY e.student_id, s.dept_id
HAVING   COUNT(DISTINCT e.course_id) =
         (SELECT COUNT(*) FROM course WHERE dept_id = s.dept_id);
```

Problem 20.4 — Relations R(A,B) with 5 tuples and S(B,C) with 4 tuples, sharing 2 matching B values (each appearing once). Give the cardinality of R × S, R ⋈ S and R ⟕ S.

Solution. `R × S` = 5 × 4 = **20** tuples of degree 4. `R ⋈ S` (natural join on B) = **2** tuples of degree 3, one per matching pair. `R ⟕ S` = **5** tuples: the 2 matched plus the 3 unmatched R tuples padded with nulls.

Problem 20.5 — Write the algebra for "students who have not taken CSE3201", and explain why a selection alone is wrong.

Solution.

```
π_{student_id}(STUDENT) − π_{student_id}( σ_{course_code='CSE3201'}(ENROLMENT ⋈ COURSE)
    ↪ )
```

A selection such as `σ_{course_code ≠ 'CSE3201'}(ENROLMENT ⋈ COURSE)` is wrong because it returns students who took *some other* course — a student who took both CSE3201 and CSE3203 would appear, since the second row satisfies the predicate. "Not taken at all" is a statement about the whole set of a student's rows, so it requires a set difference, not a row filter. The same reasoning explains why the SQL form needs `NOT EXISTS` rather than `<>`.

Problem 20.6 — Prove that the natural join is not primitive by expressing it in primitives.

Solution. Let R(A,B) and S(B,C) with common attribute B.

```
R ⋈ S = π_{A, B, C}( σ_{R.B = S.B}( ρ_{R}(R) × ρ_{S}(S) ) )
```

The product pairs everything, the selection keeps only matching B values, and the projection drops the duplicated B column, keeping one copy. Since only ×, σ and π were used, the natural join adds no expressive power — it is notation for a common pattern.

Problem 20.7 — Give the algebra for the second-highest CGPA.

Solution.

```
MAXC   ← π_{cgpa}(STUDENT) − π_{A.cgpa}(σ_{A.cgpa < B.cgpa}(ρ_A(STUDENT) × ρ_B(STUDENT)
    ↪ ))
REST   ← π_{cgpa}(STUDENT) − MAXC
SECOND ← REST − π_{A.cgpa}(σ_{A.cgpa < B.cgpa}(ρ_A(REST) × ρ_B(REST)))
```

The maximum is computed, removed, and the maximum of what remains is computed the same way — precisely the logic of the SQL solution in Problem 11.7.

Review Questions

1. What does it mean to say relational algebra is procedural and closed?
2. List the six fundamental operators and explain why the rest are derived.
3. Explain selection and projection, and state how projection differs from SQL's `SELECT`.
4. Why is the rename operator necessary?
5. State the union-compatibility requirement.
6. Distinguish theta join, equi join and natural join.
7. Express intersection and natural join in primitive operators.
8. Explain division with an example and give its primitive identity.

9. Why does algebra need a difference to express “maximum”?
10. Describe the extended operators and their SQL counterparts.

Multiple Choice Questions

1. σ selects (a) columns (b) rows (c) relations (d) duplicates.
2. π automatically (a) sorts (b) removes duplicates (c) joins (d) groups.
3. Which is **not** a fundamental operator? (a) σ (b) π (c) ⋈ (d) ×.
4. `R ∩ S` equals (a) `R − S` (b) `R − (R − S)` (c) `S − R` (d) `R × S`.
5. Degree of `R × S` is (a) max (b) min (c) sum of degrees (d) product of degrees.
6. Division answers queries containing (a) “exists” (b) “for all” (c) “not” (d) “or”.
7. `R ⋈_θ S` is equal to (a) `σ_θ(R × S)` (b) `π_θ(R × S)` (c) `R − S` (d) `R ÷ S`.
8. Union compatibility requires (a) same names (b) same number of attributes and compatible domains (c) same keys (d) same cardinality.
9. ρ is used for (a) renaming (b) grouping (c) sorting (d) division.
10. In algebra, relations are (a) bags (b) sets (c) lists (d) arrays.

Answers: 1-(b) 2-(b) 3-(c) 4-(b) 5-(c) 6-(b) 7-(a) 8-(b) 9-(a) 10-(b)

Short Questions

1. Write σ for CGPA above 3.5.
2. Why is ρ needed in self joins?
3. Give the identity for intersection.
4. What does ÷ compute?
5. State one difference between π and `SELECT`.
6. What is a theta join?
7. How many tuples has `R × S` for |R|=4, |S|=5?
8. Write the algebra for students with no enrolment.
9. Which operator expresses “for all”?
10. What is the closure property?

Long / Descriptive Questions

1. Explain all relational algebra operators with symbols, definitions, examples and SQL equivalents.
2. Prove that intersection, natural join and division are derivable from the primitives.
3. Explain division fully, with the identity, an example and both SQL implementations.
4. Write ten algebra expressions on `university_db` and translate each to SQL.
5. Discuss the differences between relational algebra and SQL, with emphasis on sets versus bags.

Practical Exercises

1. Translate ten given SQL queries on `university_db` into relational algebra.
2. Translate ten given algebra expressions into SQL and verify that results match.
3. Implement a division query in SQL in both ways and compare execution plans.
4. Demonstrate the difference between π and `SELECT` by counting rows with and without `DISTINCT`.
5. Write the algebra and SQL for the highest and second-highest CGPA without using `LIMIT`.

CHAPTER 21

Relational Calculus

Relational calculus is a **non-procedural** query language: an expression describes *what* the answer contains, using predicate logic, without saying how to compute it. It comes in two equivalent forms — tuple relational calculus and domain relational calculus.

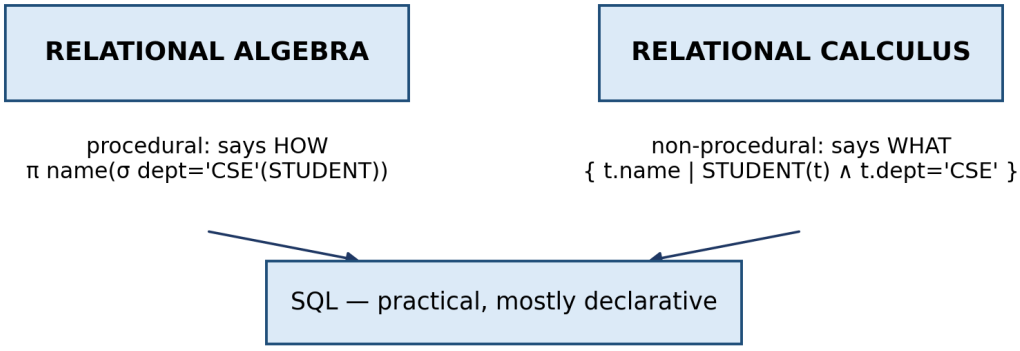


Figure 21.1: Procedural algebra, non-procedural calculus, and SQL between them

21.1 Tuple Relational Calculus (TRC)

A TRC query has the form

```
{ t | P(t) }
```

read as "the set of all tuples t for which the predicate $P(t)$ is true".

Components

Element	Meaning
`t`	A tuple variable
`R(t)`	t is a tuple of relation R
`t.A`	The value of attribute A in t
`∧ ∨ ¬`	And, or, not
`∃ t`	There exists a tuple t
`∀ t`	For all tuples t

Examples

All CSE students:

```
{ t | STUDENT(t) ∧ t.dept_id = 'CSE' }
```

Names of CSE students with CGPA above 3.5 (the result tuple has one attribute):

```
{ t.name | STUDENT(t) ∧ t.dept_id = 'CSE' ∧ t.cgpa > 3.5 }
```

Students and their department names (a join, expressed with an existential quantifier):

```
{ t.name, d.dept_name | STUDENT(t) ∧ DEPARTMENT(d) ∧ t.dept_id = d.dept_id }
```

Students who have taken at least one course:

```
{ t.name | STUDENT(t) ∧ ∃e ( ENROLMENT(e) ∧ e.student_id = t.student_id ) }
```

Students who have taken every CSE course — the universal quantifier does the work that division does in algebra:

```
{ t.name | STUDENT(t) ∧ ∀c ( ( COURSE(c) ∧ c.dept_id = 'CSE' ) →
              ∃e ( ENROLMENT(e) ∧ e.student_id = t.student_id ∧ e.course_id = c.
    ↪ course_id ) ) }
```

Note the implication: "for every course c, **if** it is a CSE course, **then** an enrolment exists". Writing `∀c ( COURSE(c) ∧ …)` without the implication would wrongly assert that every tuple in the database is a CSE course.

Students who have taken no course (negation with an existential):

```
{ t.name | STUDENT(t) ∧ ¬∃e ( ENROLMENT(e) ∧ e.student_id = t.student_id ) }
```

21.2 Domain Relational Calculus (DRC)

DRC uses **domain variables**, one per attribute, rather than tuple variables:

```
    { ⟨x1, x2, …, xn⟩ | P(x1, x2, …, xn) }
```

All CSE students:

```
{ ⟨i, n, e, b, g, d, a⟩ | STUDENT(i, n, e, b, g, d, a) ∧ d = 'CSE' }
```

Names of CSE students with CGPA above 3.5:

```
{ ⟨n⟩ | ∃i, g ( STUDENT(i, n, _, _, g, 'CSE', _) ∧ g > 3.5 ) }
```

Students with their department names:

```
{ ⟨n, dn⟩ | ∃d ( STUDENT(_, n, _, _, _, d, _) ∧ DEPARTMENT(d, dn, _) ) }
```

DRC is the basis of **Query-by-Example (QBE)**, the visual query interface used in Microsoft Access and similar tools, where the user fills example values into a grid of columns.

21.3 Safety of Expressions

A calculus expression is **unsafe** if it can produce an infinite result.

```
{ t | ¬ STUDENT(t) }
```

This asks for every tuple in the universe that is not a student — an infinite set, since the domains are unbounded. Such an expression has no computable answer.

An expression is **safe** if every value in its result comes from the **domain of the expression itself** — the set of values appearing in the relations mentioned or as constants in the formula. Practical query languages accept only safe expressions, which is why SQL has no unrestricted `NOT`: `NOT EXISTS` is always anchored

to a relation.

21.4 Equivalence of Algebra and Calculus

Codd's theorem states that relational algebra, safe tuple relational calculus and safe domain relational calculus are **equivalent in expressive power**: any query expressible in one is expressible in the others.

The consequence is practical, not merely theoretical. A language is called **relationally complete** if it can express everything relational algebra can. SQL is relationally complete — and goes further, with aggregation, ordering, grouping and recursion, none of which belong to the pure algebra.

Basis	Relational algebra	Tuple calculus	Domain calculus	SQL
Nature	Procedural	Non-procedural	Non-procedural	Mostly non-procedural
Variables	None	Tuple variables	Domain variables	Column references
Specifies	How	What	What	What
Quantifiers	None (implicit)	∃, ∀	∃, ∀	`EXISTS`, `NOT EXISTS`
"For all"	Division	∀	∀	Grouped count / double `NOT EXISTS`
Duplicates	Removed (sets)	Removed	Removed	Kept unless `DISTINCT`
Basis for	Query optimization	SQL's `WHERE`	QBE	Practice

21.5 Translating Between the Three

Query	Algebra	TRC	SQL
CSE students	σ_{dept='CSE'}(STUDENT)	`{t | STUDENT(t) ∧ t.dept='CSE'}`	`SELECT * FROM student WHERE dept_id='CSE'`
Names only	π_{name}(STUDENT)	`{t.name | STUDENT(t)}`	`SELECT DISTINCT name FROM student`
Join	STUDENT ⋈ DEPARTMENT	`{t.name, d.dept_name | …}`	`SELECT … FROM student JOIN department USING(dept_id)`
Not enrolled	π(STUDENT) − π(ENROLMENT)	`{t.name | STUDENT(t) ∧ ¬∃e(…)}`	`… WHERE NOT EXISTS (…)`
All courses	ENROLMENT ÷ COURSE	`{t | … ∀c(… → ∃e(…))}`	`GROUP BY … HAVING COUNT(DISTINCT …) = (…)`

Reading this table across each row is the fastest way to internalize the three languages: they say the same thing in three grammars.

Chapter Summary

- Relational calculus is non-procedural: it states the property the answer must satisfy.
- TRC uses tuple variables and the form `{ t | P(t) }`; DRC uses one domain variable per attribute.
- The existential quantifier expresses joins and "at least one"; the universal quantifier, with an implication, expresses "for all" — the counterpart of algebraic division.
- An expression is unsafe if it can yield an infinite result; practical languages admit only safe expressions.
- Codd's theorem establishes that algebra, safe TRC and safe DRC have identical expressive power; a language matching them is relationally complete.
- SQL is relationally complete and additionally supports aggregation, grouping, ordering and dupli-

cates.

Key Terms

Relational calculus · Non-procedural · Tuple relational calculus · Domain relational calculus · Tuple variable · Domain variable · Predicate · Existential quantifier ∃ · Universal quantifier ∀ · Implication · Free and bound variables · Safe expression · Domain of an expression · Codd's theorem · Relational completeness · Query-by-Example

Common Mistakes

1. Writing `∀c (COURSE(c) ∧ …)` instead of `∀c (COURSE(c) → …)`, which asserts that everything is a course.
2. Producing unsafe expressions with an unrestricted negation.
3. Confusing tuple variables with domain variables.
4. Believing calculus is more powerful than algebra, or the reverse; Codd's theorem says they are equal.
5. Forgetting that calculus, like algebra, returns sets, so duplicates never appear.
6. Using ∀ where ∃ is meant — "every" versus "at least one".
7. Leaving a variable free that should be bound by a quantifier.

Solved Problems

Problem 21.1 — Write in TRC and in SQL: names of students with CGPA above the value 3.5 who belong to CSE.

Solution.

```
TRC:  { t.name | STUDENT(t) ∧ t.dept_id = 'CSE' ∧ t.cgpa > 3.5 }
SQL:  SELECT DISTINCT name FROM student WHERE dept_id='CSE' AND cgpa > 3.5;
```

`DISTINCT` appears in the SQL because the calculus result is a set; without it SQL would return duplicates if two CSE students shared a name.

Problem 21.2 — Write in TRC: students who have taken at least one course taught in their own department.

Solution.

```
{ t.name | STUDENT(t) ∧ ∃e ∃c ( ENROLMENT(e) ∧ COURSE(c)
             ∧ e.student_id = t.student_id
             ∧ e.course_id  = c.course_id
             ∧ c.dept_id    = t.dept_id ) }
```

Two existential quantifiers chain the two joins; every variable is bound, so the expression is safe.

Problem 21.3 — Explain why `{ t | ¬ STUDENT(t) }` is unsafe and how SQL avoids the problem.

Solution. The expression asks for all tuples of the universe not appearing in STUDENT. Since attribute domains such as `VARCHAR(100)` contain astronomically many values, and the calculus places no restriction on where the values come from, the result is not finite and cannot be computed. SQL avoids this by never allowing an unanchored negation: every query must name the relations it draws from in `FROM`, and negation appears only as `NOT EXISTS`, `NOT IN` or `EXCEPT`, each of which is evaluated against a finite named relation. The formal condition is that a safe expression's values must lie within the *domain of the expression* — the constants and relation values it actually mentions.

Problem 21.4 — Write in DRC: the names and titles of all (student, course) enrolment pairs.

Solution.

```
{ ⟨n, ti⟩ | ∃s, c ( STUDENT(s, n, _, _, _, _, _) ∧ ENROLMENT(_, s, c, _, _)
                    ∧ COURSE(c, _, ti, _, _) ) }
```

The shared variables `s` and `c` perform the joins: the same domain variable appearing in two predicates forces the values to be equal, which is exactly how DRC expresses an equi join.

Problem 21.5 — Express "students who have taken every CSE course" in algebra, TRC and SQL, and comment on which is clearest.

Solution.

```
Algebra: π_{student_id, course_id}(ENROLMENT) ÷ π_{course_id}(σ_{dept_id='CSE'}(COURSE)
    ↪ )

TRC:     { t.name | STUDENT(t) ∧ ∀c ( (COURSE(c) ∧ c.dept_id='CSE') →
                    ∃e (ENROLMENT(e) ∧ e.student_id=t.student_id ∧ e.course_id=c.
    ↪ course_id) ) }
```

```
SELECT   e.student_id FROM enrollment e JOIN course c ON c.course_id=e.course_id
WHERE    c.dept_id='CSE'
GROUP BY e.student_id
HAVING   COUNT(DISTINCT e.course_id) = (SELECT COUNT(*) FROM course WHERE dept_id='CSE')
    ↪ ;
```

The TRC version states the requirement most directly — "for every CSE course there is an enrolment" — and is therefore the best specification. The algebraic version is the most compact but only if one already knows what division means. The SQL version is the only one that runs, and it works by counting rather than by quantifying, which is why it needs `DISTINCT`: without it, a student who repeated one course could reach the required count wrongly.

Problem 21.6 — State Codd's theorem and explain its practical significance.

Solution. Codd's theorem states that relational algebra, safe tuple relational calculus and safe domain relational calculus are equivalent in expressive power. Its significance is twofold. First, it justifies designing a *declarative* user language: since anything a user can specify declaratively can be rewritten as an algebraic expression, the DBMS can translate a query into algebra and then optimize it using algebraic equivalences without changing the answer. Second, it gives an objective yardstick — **relational completeness** — against which any query language can be measured. SQL meets it, which is the formal reason SQL is regarded as a full relational language rather than a convenience interface.

Review Questions

1. Define relational calculus and explain what "non-procedural" means.
2. Write the general form of a TRC expression and explain each part.
3. Write the general form of a DRC expression and contrast it with TRC.
4. Explain the roles of ∃ and ∀ with examples.
5. Why must ∀ be paired with an implication?
6. What makes an expression unsafe? Give an example.
7. State Codd's theorem and its practical significance.
8. Define relational completeness and explain whether SQL satisfies it.
9. Compare algebra, TRC, DRC and SQL in a table.

10. What is Query-by-Example, and which calculus is it based on?

Multiple Choice Questions

1. Relational calculus is (a) procedural (b) non-procedural (c) imperative (d) algebraic.
2. `{ t | P(t) }` is the form of (a) DRC (b) TRC (c) algebra (d) SQL.
3. DRC uses (a) tuple variables (b) domain variables (c) relation variables (d) constants only.
4. "For all" is expressed by (a) $\exists$ (b) $\forall$ (c) $\neg$ (d) $\vee$.
5. `{ t | ¬R(t) }` is (a) safe (b) unsafe (c) empty (d) a join.
6. Codd's theorem states algebra and safe calculus are (a) different (b) equivalent (c) unrelated (d) procedural.
7. QBE is based on (a) TRC (b) DRC (c) algebra (d) SQL.
8. A join in TRC is expressed with (a) $\forall$ (b) $\exists$ and equality (c) $\div$ (d) $\neg$.
9. Relational completeness means matching the power of (a) SQL (b) relational algebra (c) TRC only (d) DRC only.
10. Calculus results contain (a) duplicates (b) no duplicates (c) nulls only (d) ordered rows.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What is a tuple variable?
2. Write the TRC for all EEE students.
3. Why is `¬R(t)` unsafe?
4. What is a bound variable?
5. Which calculus underlies QBE?
6. What does relational completeness mean?
7. How is a join expressed in DRC?
8. Which quantifier corresponds to algebraic division?
9. State Codd's theorem in one sentence.
10. Does calculus return duplicates?

Long / Descriptive Questions

1. Explain TRC in detail with at least six examples of increasing complexity.
2. Explain DRC with examples and describe its relationship to QBE.
3. Discuss safety of expressions with examples of unsafe queries and how SQL prevents them.
4. State and explain Codd's theorem, relational completeness and their practical significance.
5. Translate ten queries between algebra, TRC and SQL, commenting on which formalism expresses each most clearly.

Practical Exercises

1. Write TRC expressions for ten queries on `university_db` and translate each into SQL; verify that the results agree.
2. Write DRC expressions for five of the same queries.
3. Construct an unsafe TRC expression and explain in writing why no SQL statement corresponds to it.
4. Express "students enrolled in every course of their department" in algebra, TRC and SQL, and run

the SQL.

5. Using a QBE-style tool (Microsoft Access or LibreOffice Base), build two queries visually and write the equivalent DRC.

CHAPTER 22

Storage and File Organization

Everything in the previous chapters has been logical. From here the subject becomes physical: where bytes actually live, how long they take to fetch, and how that shapes every design decision a DBMS makes.

22.1 The Storage Hierarchy

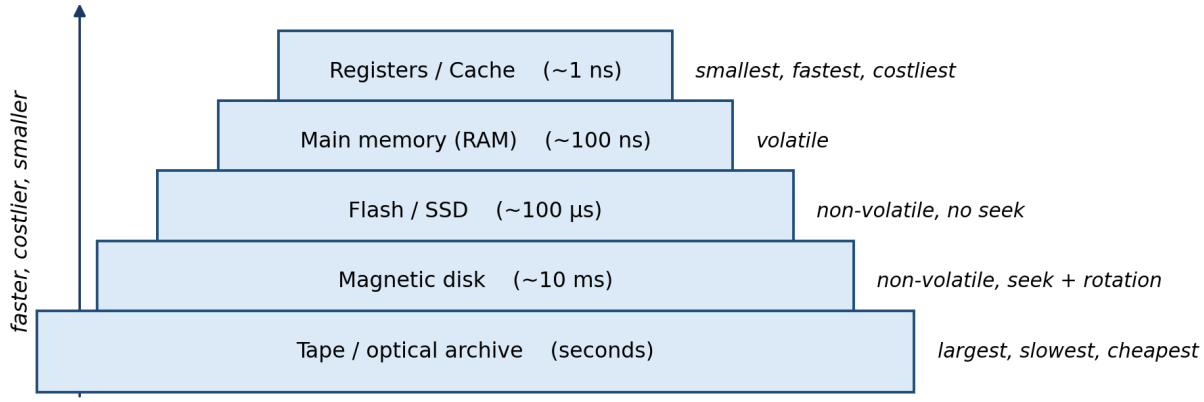


Figure 22.1: The storage hierarchy — faster and costlier at the top

Level	Typical access time	Volatile?	Capacity	Role in a DBMS
Registers, CPU cache	1–10 ns	Yes	KB–MB	Expression evaluation
Main memory (RAM)	~100 ns	Yes	GB–TB	Buffer pool, sort space, locks
Flash / SSD	50–150 µs	No	TB	Data and log files on modern servers
Magnetic disk	5–15 ms	No	TB–PB	Bulk data, cheap capacity
Optical / tape	seconds–minutes	No	PB	Archive and off-site backup

The ratios are what matter. Main memory is roughly **100,000 times** faster than a magnetic disk. An operation that takes 1 second in memory takes over a day if every step goes to disk. Consequently:

- The DBMS never thinks in rows when it comes to I/O; it thinks in **blocks** (pages), typically 4 KB, 8 KB or 16 KB.
- Performance analysis counts **block accesses**, not instructions.
- The **buffer pool** — a large region of RAM holding recently used pages — is the single most important tuning parameter in most systems.

Volatile storage loses its contents when power fails; **non-volatile** does not. Durability (the D of ACID) therefore requires that committed data reaches non-volatile storage, which is why the log is flushed before a commit is acknowledged (Chapter 32).

22.2 Magnetic Disk Structure

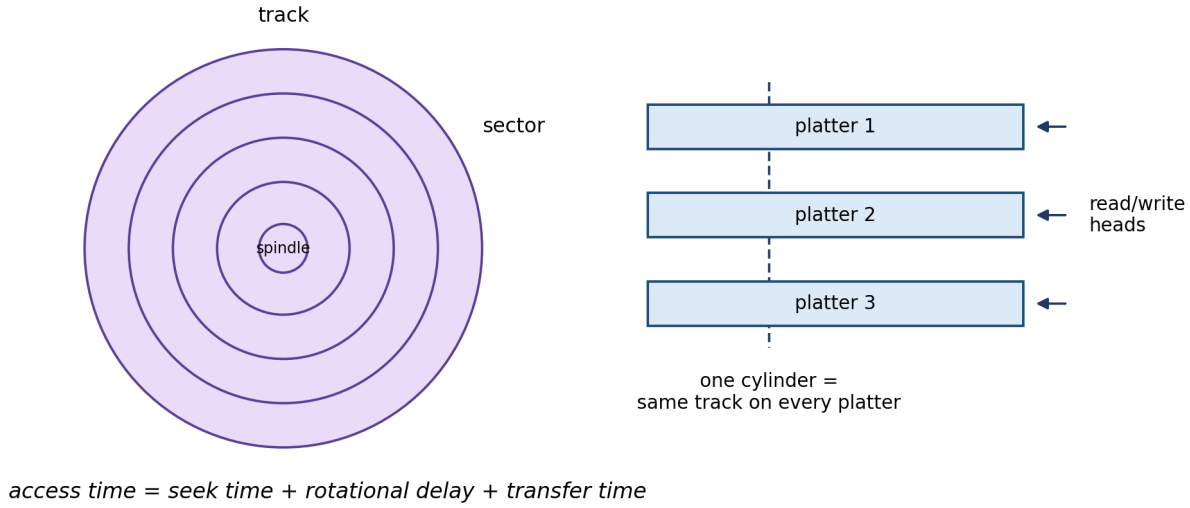


Figure 22.2: Disk geometry — tracks, sectors, cylinders and heads

Term	Meaning
Platter	A circular disk coated with magnetic material
Track	One concentric ring on a platter surface
Sector	A fixed-size arc of a track, the smallest unit the hardware reads (512 B or 4 KB)
Cylinder	The same track number on every platter, reachable without moving the arm
Head	One read/write element per surface, all moving together
Block / page	The DBMS's unit of I/O: a group of consecutive sectors

Disk access time

```
access time = seek time + rotational latency + transfer time
```

- **Seek time** — moving the arm to the right cylinder. The dominant cost: 3–10 ms.
- **Rotational latency** — waiting for the sector to arrive under the head. On average half a revolution; at 7,200 rpm one revolution is 8.33 ms, so the average is about 4.17 ms.
- **Transfer time** — reading the bytes once positioned: a fraction of a millisecond.

Worked calculation. A disk spins at 7,200 rpm with an average seek of 5 ms and a transfer rate of 100 MB/s. Reading one 8 KB block takes

```
seek                5.00 ms
rotational latency  4.17 ms      (½ × 60,000/7200)
transfer            0.08 ms      (8 KB ÷ 100 MB/s)
total             ≈ 9.25 ms
```

Reading 100 blocks scattered at random costs ≈ 925 ms. Reading the same 100 blocks **sequentially** costs one seek plus one rotation plus the transfer of 800 KB ≈ 17 ms — **fifty times faster**. This single comparison explains sequential scans, clustered indexes, read-ahead, log-structured writes and the layout of almost every physical structure in this book.

Solid-state drives eliminate seek and rotation, so random access costs roughly the same as sequential. They do not, however, eliminate the block as a unit, and they add the **erase-before-write** constraint, which is why write amplification and wear levelling matter for SSD-resident databases.

22.3 Records and Blocks

A **file** is a sequence of blocks; a block holds several records.

```
    blocking factor  bfr = ⌊ block size / record size ⌋
```

Example. Block = 4,096 B, record = 200 B ⇒ bfr = ⌊20.48⌋ = **20 records per block**. For 1,000,000 records: 1,000,000 ÷ 20 = **50,000 blocks**. The 0.48 of a record per block is wasted unless records are allowed to span blocks (**spanned** organization), which complicates access and is usually avoided.

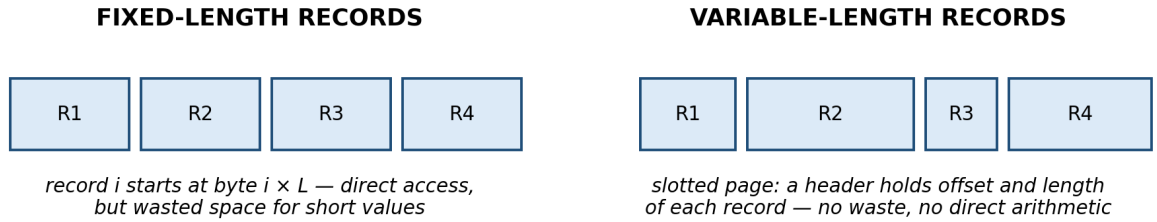


Figure 22.3: Fixed-length versus variable-length record layout

Fixed-length records permit direct arithmetic: record i begins at byte $i \times L$. Deletion leaves a hole, handled either by moving the last record into it or by chaining free slots in a **free list**.

Variable-length records — the normal case with `VARCHAR` — use a **slotted page**: a header at one end holds the offset and length of every record, records grow from the other end, and free space sits in the middle. Records can then be moved within the page to compact free space without changing their external identifiers, because those identifiers point at the slot entry, not at the byte offset.

22.4 File Organization

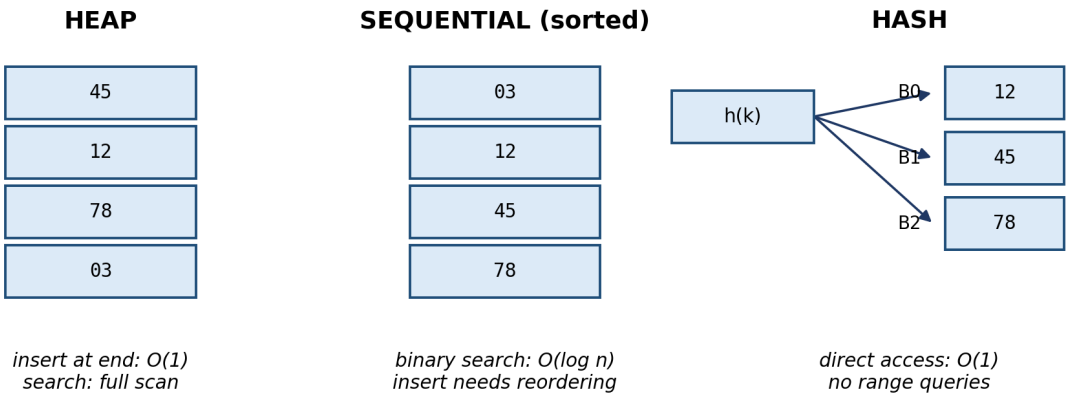


Figure 22.4: Heap, sequential and hash file organization

Heap (unordered) file

Records are appended wherever there is room.

Operation	Cost
Insert	O(1) — append to the last block
Search on any attribute	O(b) — scan all *b* blocks
Delete	O(b) to find, then mark free

Good for bulk loading and for small tables; bad for selective retrieval. Most SQL tables without a clustered index are heaps.

Sequential (ordered) file

Records are kept sorted on a **search key**.

Operation	Cost
Search on the ordering key	O($\log_2$ b) blocks by binary search
Range query	Excellent — matching records are contiguous
Search on any other attribute	O(b)
Insert	Expensive: must preserve order

Insertion is handled with an **overflow area**: new records go to an unordered overflow block chained to the main file, and the file is periodically reorganized. Performance degrades as the overflow chain grows, which is exactly the problem the B+ tree solves.

Hash file

A hash function maps the key to a bucket address.

Operation	Cost
Equality search on the hash key	O(1) — usually one block access
Range query	Impossible — hashing destroys order
Insert	O(1) unless the bucket overflows

Collisions (two keys mapping to the same bucket) are resolved with an **overflow chain**. Performance degrades as chains lengthen, which motivates dynamic hashing.

Clustered file

Records of two related tables are stored together in the same blocks, ordered on the joining attribute — for instance each department followed immediately by its students. A join then costs almost nothing, but queries touching only one of the tables become slower, because their records are interleaved with the other table's. Oracle supports this explicitly; in MySQL/InnoDB the primary key is always clustered, meaning the table itself is stored in primary key order.

Organization	Equality search	Range search	Insert	Best for
Heap	O(b)	O(b)	O(1)	Bulk load, small tables
Sequential	O(log b)	Excellent	Expensive	Read-mostly, range queries
Hash	O(1)	Impossible	O(1)	Exact-match lookups
Clustered	Good for the join key	Good	Complex	Frequently joined pairs

Chapter Summary

- Storage forms a hierarchy in which each level is faster, smaller and costlier than the one below; RAM is about 10^5 times faster than a magnetic disk.

- The DBMS's unit of I/O is the block, and performance is measured in block accesses.
- Disk access time = seek + rotational latency + transfer; seek dominates, so sequential access can be fifty times cheaper than random access for the same data.
- Blocking factor determines how many records fit in a block and therefore how many blocks a file occupies.
- Variable-length records are stored in slotted pages so that they can be moved without invalidating identifiers.
- Heap files are cheap to write and costly to search; sequential files serve range queries but suffer on insert; hash files give O(1) equality access but no ranges; clustered files accelerate a specific join.

Key Terms

Storage hierarchy · Volatile / non-volatile · Block / page · Buffer pool · Platter · Track · Sector · Cylinder · Seek time · Rotational latency · Transfer time · Sequential versus random access · Blocking factor · Spanned / unspanned records · Fixed-length record · Variable-length record · Slotted page · Free list · Heap file · Sequential file · Overflow area · Hash file · Collision · Overflow chain · Clustered file

Important Formulas / Notation

```
access time = seek + rotational latency + transfer
average rotational latency = ½ × (60,000 / rpm)  ms
bfr = ⌊ block size / record size ⌋
blocks needed b = ⌈ number of records / bfr ⌉
binary search on an ordered file = ⌈ log2 b ⌉ block accesses
```

Common Mistakes

1. Reasoning about I/O in rows rather than blocks.
2. Ignoring that seek time dominates, and treating random and sequential reads as equally expensive.
3. Forgetting to halve a full revolution when computing average rotational latency.
4. Using a hash organization for range queries.
5. Assuming SSDs remove the need for indexes; they reduce the random-access penalty but not the cost of reading unnecessary blocks.
6. Computing the blocking factor by rounding up instead of down.
7. Confusing a cylinder with a track.

Solved Problems

Problem 22.1 — A file has 500,000 records of 250 bytes; block size is 4,096 bytes. Compute the blocking factor, the number of blocks, and the cost of a full scan and of a binary search on an ordered file, given 10 ms per random block access.

Solution. bfr = $\lfloor 4096 \div 250 \rfloor$ = **16**. Blocks b = $\lceil 500{,}000 \div 16 \rceil$ = **31,250**. A full scan reads all 31,250 blocks; sequentially, at say 100 MB/s, the 128 MB take about 1.3 s. A binary search needs $\lceil \log_2 31{,}250 \rceil$ = **15** random block accesses ≈ 150 ms. The comparison shows why an ordering or an index is worth having, and also why a scan is not catastrophic if it is sequential.

Problem 22.2 — A disk rotates at 10,000 rpm, average seek is 4 ms, transfer rate 200 MB/s, block size 8 KB. Compute the time for one random block read and for 200 sequential blocks.

Solution. Rotational latency = ½ × (60,000 ÷ 10,000) = **3 ms**. Transfer of 8 KB at 200 MB/s = 0.04 ms. One random read = 4 + 3 + 0.04 ≈ **7.04 ms**. For 200 sequential blocks: one seek (4 ms) + one rotation (3

ms) + transfer of 1,600 KB at 200 MB/s (8 ms) ≈ **15 ms**, versus 200 × 7.04 = **1,408 ms** if read randomly — a factor of nearly 94.

Problem 22.3 — Explain why a DBMS reads a whole block to fetch one 200-byte row.

Solution. The block is the smallest unit the storage layer addresses, and almost the entire cost of the read is positioning the head, not transferring bytes. Reading 200 bytes and reading 8,192 bytes from the same position cost essentially the same. Moreover, fetching the whole block is usually profitable: neighbouring rows are often needed next (locality of reference), and the block sits in the buffer pool where subsequent requests are served without any I/O at all.

Problem 22.4 — Compare heap, sequential and hash organization for: (a) a log table written continuously and rarely read; (b) a lookup table queried by exact code; (c) a transaction table queried by date range.

Solution. (a) **Heap** — appends are O(1), and no search structure is needed. (b) **Hash** — exact-match lookup in one block access; ranges are never required. (c) **Sequential or B+-tree-indexed on date** — range queries retrieve contiguous blocks, so one seek serves many matching records; a hash organization would be useless here because hashing destroys ordering.

Problem 22.5 — Why are variable-length records stored in slotted pages rather than packed end to end?

Solution. Packed records cannot be located by arithmetic, so a directory is needed in any case. More importantly, when a record is updated to a longer value it no longer fits in place; with packing, every following record would have to move, and any external pointer to them would break. A slotted page separates the *identifier* (page number + slot number) from the *location* (offset stored in the slot). Records can then be shuffled within the page to reclaim fragmented free space while all external references — index entries in particular — remain valid.

Problem 22.6 — A clustered file stores each department followed by its students. Give one query it accelerates and one it penalizes.

Solution. Accelerated: `SELECT … FROM department d JOIN student s ON s.dept_id = d.dept_id` — each department and all of its students lie in the same blocks, so the join costs roughly one pass with no random access. Penalized: `SELECT COUNT(*) FROM department` — the department rows are scattered among student rows, so reading 4 department records may touch dozens of blocks that are mostly students. Clustering optimizes one access pattern at the direct expense of others, which is why it must be chosen against a known workload.

Review Questions

1. Describe the storage hierarchy and give typical access times at each level.
2. Why does a DBMS measure cost in block accesses rather than record accesses?
3. Define track, sector, cylinder and block.
4. Give the disk access time formula and explain which term dominates.
5. Calculate and explain the difference between random and sequential access for 100 blocks.
6. Define blocking factor and compute it for given parameters.
7. Explain slotted pages and why they are needed for variable-length records.
8. Compare heap, sequential, hash and clustered file organizations.
9. How are collisions handled in a hash file?
10. In what ways do SSDs change the analysis, and in what ways do they not?

Multiple Choice Questions

1. The dominant component of disk access time is (a) transfer (b) seek (c) rotation (d) latency of the bus.
2. Average rotational latency at 7,200 rpm is about (a) 8.3 ms (b) 4.2 ms (c) 2 ms (d) 16 ms.
3. The same track on all platters forms a (a) sector (b) cylinder (c) block (d) cluster.
4. Blocking factor for a 4,000 B block and 300 B records is (a) 13 (b) 13.3 (c) 14 (d) 12.
5. Which organization cannot support range queries? (a) Heap (b) Sequential (c) Hash (d) Clustered.
6. Slotted pages are used for (a) fixed-length (b) variable-length (c) hashed (d) indexed records.
7. Insertion is cheapest in a (a) heap (b) sequential (c) clustered (d) indexed file.
8. Binary search on 1,024 ordered blocks needs about (a) 10 (b) 32 (c) 512 (d) 1,024 accesses.
9. Volatile storage (a) survives power loss (b) does not survive power loss (c) is always cheapest (d) is sequential only.
10. Reading 200 bytes costs nearly the same as reading a block because (a) CPUs are slow (b) positioning dominates (c) buffers are small (d) blocks are tiny.

Answers: 1-(b) 2-(b) 3-(b) 4-(a) 5-(c) 6-(b) 7-(a) 8-(a) 9-(b) 10-(b)

Short Questions

1. What is a block?
2. Define seek time.
3. State the formula for average rotational latency.
4. What is the blocking factor?
5. What is an overflow chain?
6. Name one advantage of a heap file.
7. Why is a slotted page needed?
8. What is a clustered file?
9. Give one thing SSDs do not change.
10. What is the buffer pool?

Long / Descriptive Questions

1. Explain the storage hierarchy in detail, with access times, volatility and the role of each level in a DBMS.
2. Explain disk structure and access time with a full numerical comparison of random and sequential access.
3. Describe record and block organization, including fixed-length, variable-length and slotted pages.
4. Compare all file organizations with cost analysis and appropriate use cases.
5. Discuss how physical storage characteristics shape the design of indexes, logs and query plans.

Practical Exercises

1. Determine your DBMS's page size and buffer pool size, and record the settings.
2. Compute the blocking factor and block count for each table in `university_db` using `information_schema` row and size data.
3. Generate a million-row table, then time a full scan and an indexed lookup; explain the ratio.
4. Measure the difference between reading a file sequentially and reading the same blocks in random order with a small script.

5. Examine a table's storage statistics before and after deleting half its rows, and explain why the file size does not shrink.

CHAPTER 23

Indexing

An index is an auxiliary structure that makes retrieval on a search key fast, at the cost of extra space and slower writes. It is the single most effective tuning tool available, and the most commonly misused.

23.1 The Idea

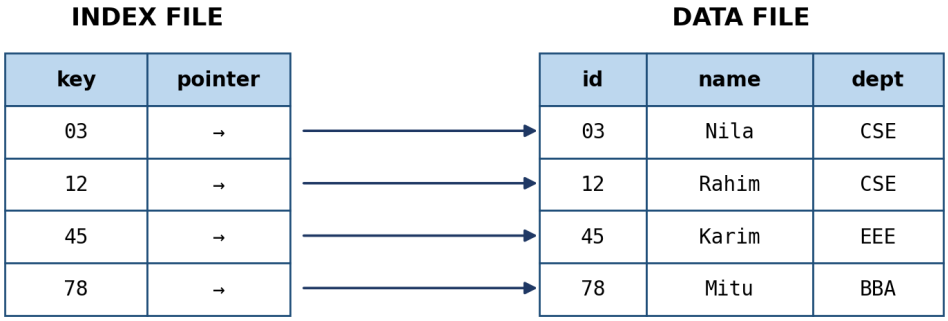


Figure 23.1: An index maps search-key values to record locations

An index is a file of `(search key, pointer)` entries, sorted by the key. Because it contains only two small fields per record, it occupies far fewer blocks than the data file, so searching it costs far fewer block accesses.

Term	Meaning
Search key	The attribute(s) the index is built on — not necessarily a key of the relation
Index entry	A `(key, pointer)` pair
Primary index	Built on the ordering key of a sequentially organized file
Clustering index	Built on an attribute the file is ordered by, which is not unique
Secondary index	Built on any other attribute; the file is not ordered by it
Dense index	One entry per record
Sparse index	One entry per block

23.2 Dense and Sparse Indexes

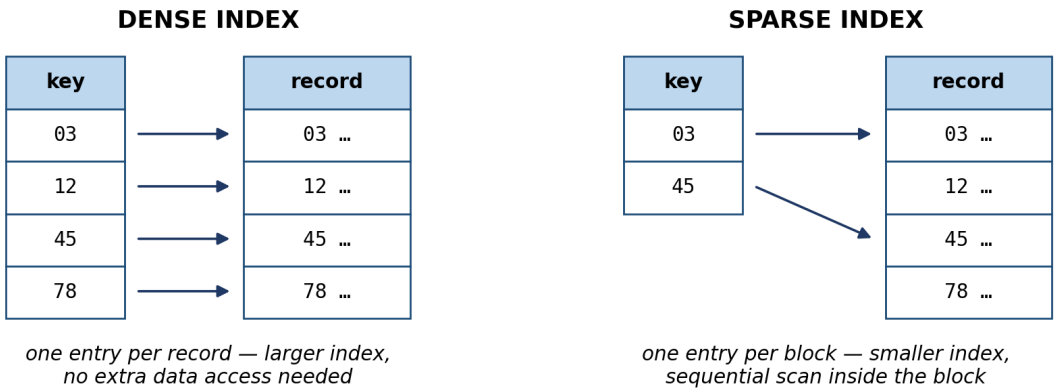


Figure 23.2: Dense versus sparse indexing

A **dense** index has an entry for every record, so a lookup never has to scan the data file beyond the target record. A **sparse** index has one entry per block: the search finds the largest key ≤ the target, reads that block and scans within it.

Basis	Dense	Sparse
Entries	One per record	One per block
Index size	Larger	Smaller
Lookup	Direct	Index lookup + in-block scan
Requires ordered data	No	Yes
Usable for secondary indexes	Yes	No

A sparse index is only possible when the data file is **ordered** on the search key, which is why secondary indexes must always be dense.

23.3 Multilevel Indexes

If the index itself is too large to search cheaply, an index is built over the index.

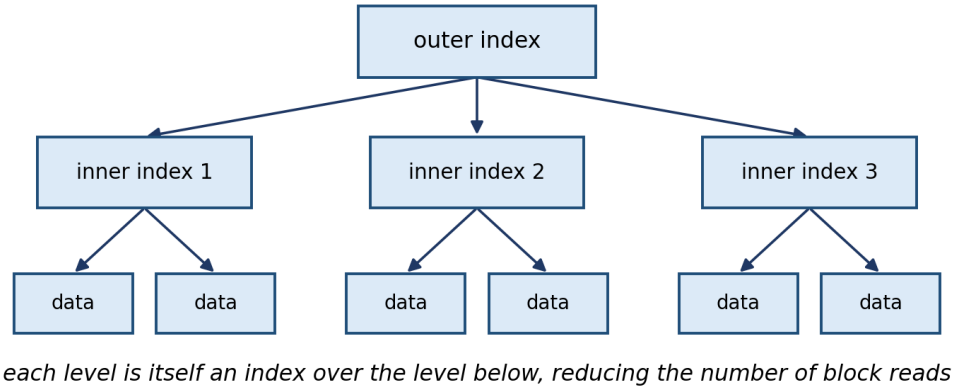


Figure 23.3: A multilevel index

Worked calculation. 1,000,000 records, 100 index entries per block.

```
Level 1 (dense): 1,000,000 entries ÷ 100 =  10,000 blocks
Level 2 (sparse over level 1):              100 blocks
Level 3 (sparse over level 2):                1 block
Search cost = 3 index reads + 1 data read = 4 block accesses
```

Compare with a binary search on the data file itself: $\lceil \log_2 50{,}000 \rceil \approx 16$ accesses. The multilevel index is four times cheaper — and this structure, maintained dynamically, is exactly a B^+ tree.

23.4 B-Trees and B+ Trees

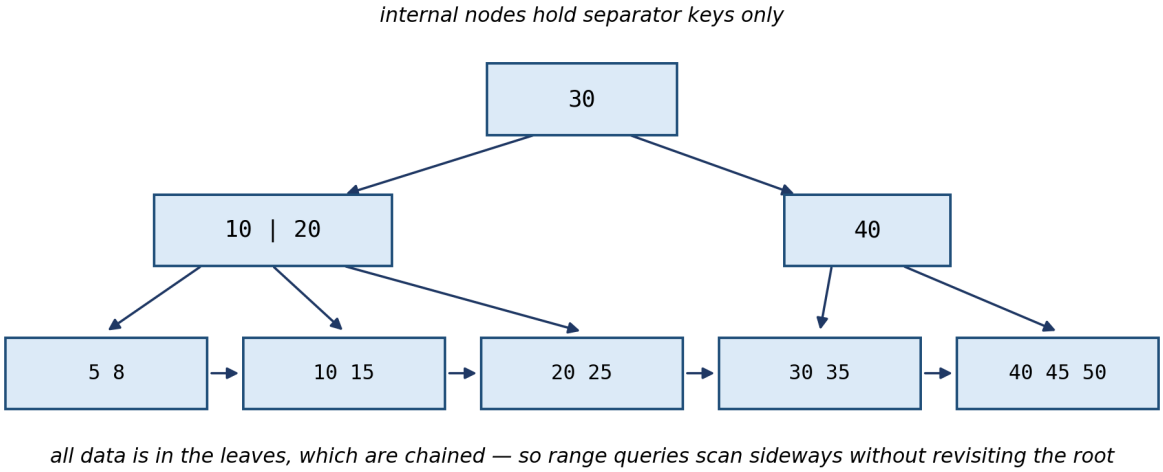


Figure 23.4: A B+ tree — separator keys above, chained data leaves below

A **B^+ tree** of order n satisfies:

1. Every node holds at most $n - 1$ keys and n pointers.
2. Every node except the root holds at least $\lceil n/2 \rceil$ pointers; the root holds at least 2 (unless it is a leaf).
3. **All data pointers are in the leaves**; internal nodes hold separator keys only.
4. All leaves are at the **same depth**, so the tree is perfectly balanced.
5. Leaves are **chained** in key order.

Property	B-tree	B^+ tree
Data pointers	In internal nodes and leaves	Leaves only
Key duplication	Keys appear once	Separator keys repeat in leaves
Leaf chaining	No	Yes
Range queries	Require tree traversal	Scan the leaf chain
Fan-out	Lower (nodes carry data)	Higher (nodes carry keys only)
Height	Slightly greater	Smaller
Used by	Rare in practice	Virtually every relational DBMS

The B^+ tree wins on both counts that matter: higher fan-out means a shallower tree (fewer block accesses), and leaf chaining makes `BETWEEN`, `>` and `ORDER BY` efficient.

Height and cost

```
height h ≈ ⌈ log_⌈n/2⌉ (N) ⌉      N = number of keys, n = order
```

With order 100 and 1,000,000 keys, the height is about 4 — meaning **four block accesses** to find any row among a million. With 100,000,000 keys it is about 5. This logarithmic growth is why B^+ trees scale.

Insertion with splitting

Inserting into a full leaf splits it: the keys are divided, and the **middle key is copied up** into the parent. If the parent is also full it splits too, propagating upward; if the root splits, a new root is created and the tree grows by one level — which is how a B^+ tree stays balanced without ever being rebuilt.

```
Insert 25 into a full leaf [20 | 22 | 24] of an order-4 tree:

   before:  parent [30]      leaf [20 22 24]
   split:   leaf1 [20 22]    leaf2 [24 25]
   copy up: parent [24 | 30]
```

Note "copy up" for a B^+ tree, not "move up": the separator must also remain in the leaf, because all data lives there. In a B-tree the middle key moves up and does not remain below.

Deletion with merging

If a node falls below the minimum occupancy, it **borrows** a key from a sibling, or **merges** with one and removes a separator from the parent, possibly propagating upward and reducing the height.

23.5 Hash Indexes

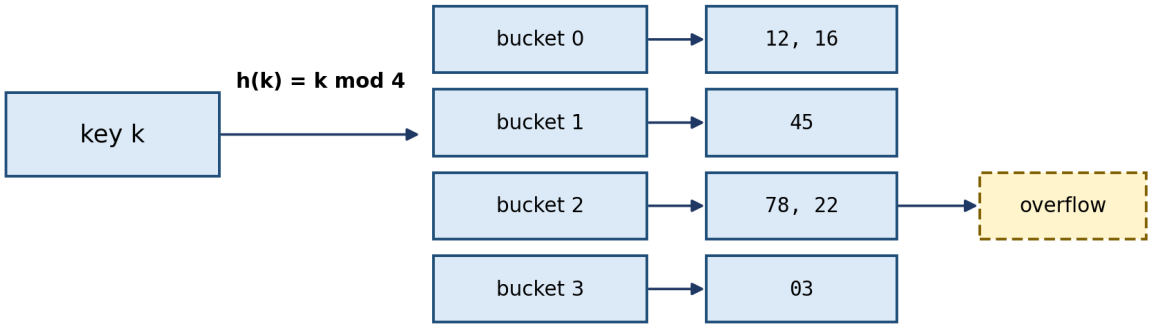


Figure 23.5: Static hashing with an overflow chain

Static hashing fixes the number of buckets in advance: `h(k) = k mod M`. One block access on average — but as the file grows, overflow chains lengthen and performance decays; as it shrinks, space is wasted.

Extendible hashing uses a directory of 2^d entries addressed by the first *d* bits of the hash value. When a bucket overflows, only that bucket splits and the local depth increases; the directory doubles only when the local depth would exceed the global depth. No full reorganization ever occurs.

Linear hashing grows one bucket at a time in a fixed order, avoiding a directory entirely.

Basis	B^+ tree index	Hash index
Equality search	O(log n), ~3–4 accesses	O(1), ~1 access
Range search	Excellent via leaf chain	Impossible
`ORDER BY` support	Yes	No
Prefix matching (`LIKE 'abc%'`)	Yes	No

Basis	B^+ tree index	Hash index
Behaviour as data grows	Graceful	Needs dynamic hashing
Default in MySQL/PostgreSQL	Yes	Optional (memory tables, explicit `USING HASH`)

Because most workloads mix equality and range predicates, the B^+ tree is the default everywhere; hash indexes are a specialist tool.

23.6 Indexes in Practice

```sql
CREATE INDEX idx_student_dept      ON student(dept_id);
CREATE INDEX idx_enrol_student     ON enrollment(student_id);
CREATE UNIQUE INDEX uq_student_mail ON student(email);

-- composite index: order of columns matters
CREATE INDEX idx_enrol_sem_course ON enrollment(semester, course_id);

DROP INDEX idx_student_dept ON student;          -- MySQL
EXPLAIN SELECT * FROM student WHERE dept_id = 'CSE';
```

The leftmost-prefix rule

A composite index on `(semester, course_id)` can serve:

```sql
WHERE semester = '5th'                          -- yes, leftmost prefix
WHERE semester = '5th' AND course_id = 1        -- yes, full key
```

but **not**

```sql
WHERE course_id = 1                             -- no: skips the leading column
```

Think of a telephone directory ordered by (surname, given name): it is useless for finding everyone named "Rahim" regardless of surname.

Covering index

If an index contains every column a query needs, the DBMS can answer from the index alone without touching the table — an **index-only scan**:

```sql
CREATE INDEX idx_cover ON student(dept_id, cgpa, name);
SELECT name, cgpa FROM student WHERE dept_id = 'CSE';    -- no table access needed
```

When an index is *not* used

- The predicate wraps the column in a function: `WHERE YEAR(admitted_on) = 2020`.
- There is an implicit type conversion between the column and the literal.
- The predicate begins with a wildcard: `LIKE '%Hasan'`.
- The condition matches a large fraction of the table — typically more than 20–30 % — where a sequential scan is genuinely cheaper.
- The column has very low **selectivity**, such as a two-valued flag.
- Statistics are stale, so the optimizer's row estimates are wrong.

The cost of indexes

Benefit	Cost
Faster selective reads	Every `INSERT`, `UPDATE` and `DELETE` must maintain each index
Faster joins on indexed keys	Extra disk space, often 10–30 % of the table per index
Enforce `UNIQUE`	Longer recovery and backup times
Support `ORDER BY` and `GROUP BY`	More work for the optimizer, and more plans to get wrong

> The practical rule: index foreign keys and the columns of frequent, selective `WHERE`, `JOIN` and `ORDER BY` clauses; do not index low-selectivity columns; and remove indexes that `EXPLAIN` shows are never used, because they are pure cost.

Chapter Summary

- An index maps search-key values to record locations and is small enough to search in a few block accesses.
- Dense indexes have one entry per record; sparse indexes have one per block and require ordered data.
- Multilevel indexes index the index; maintained dynamically, they are B^+ trees.
- A B^+ tree keeps all data in chained leaves, is perfectly balanced, and finds any row among a million in about four block accesses.
- B-trees store data in internal nodes, lowering fan-out and making range queries harder; B^+ trees are used almost universally.
- Hash indexes give one-access equality lookup but cannot do ranges; extendible and linear hashing avoid reorganization.
- Composite indexes follow the leftmost-prefix rule; covering indexes allow index-only scans.
- Indexes cost write performance and space, and are ignored by the optimizer when predicates are non-sargable or unselective.

Key Terms

Index · Search key · Index entry · Primary index · Clustering index · Secondary index · Dense index · Sparse index · Multilevel index · B-tree · B^+ tree · Order / fan-out · Node split · Copy-up · Merge · Balanced tree · Leaf chaining · Static hashing · Extendible hashing · Linear hashing · Global and local depth · Bucket · Overflow chain · Composite index · Leftmost-prefix rule · Covering index · Index-only scan · Selectivity · Sargable predicate

Important Formulas / Notation

```
B+ tree height  h ≈ ⌈ log_⌈n/2⌉ N ⌉
multilevel index levels = ⌈ log_fanout (number of entries) ⌉
search cost = height + 1 data block access
selectivity = distinct values / total rows   (higher is better for indexing)
```

Common Mistakes

1. Indexing every column, which cripples writes and wastes space.
2. Expecting a composite index to serve a predicate on a non-leading column.
3. Believing an index always helps; for an unselective predicate a scan is cheaper.
4. Applying a function to an indexed column in `WHERE`.
5. Using `LIKE '%text'` and expecting an index scan.

6. Confusing B-trees with B^+ trees in examinations — the data-in-leaves property is the distinguishing answer.
7. Using a hash index where range queries are needed.
8. Forgetting to index foreign key columns in PostgreSQL and SQL Server, where it is not automatic.

Solved Problems

Problem 23.1 — 1,000,000 records, 10 records per data block, 100 index entries per index block. Compute the block accesses for (a) a linear scan, (b) a binary search on an ordered file, (c) a three-level index.

Solution. Data blocks = 1,000,000 ÷ 10 = **100,000**. (a) Linear scan: up to **100,000** accesses. (b) Binary search: $\lceil \log_2 100{,}000 \rceil$ = **17** accesses. (c) Dense level-1 index: 1,000,000 ÷ 100 = 10,000 blocks; level 2: 100 blocks; level 3: 1 block; so 3 index reads + 1 data read = **4** accesses. The index is more than four times better than binary search and 25,000 times better than a scan.

Problem 23.2 — A B^+ tree of order 101 holds 10,000,000 keys. Estimate its height and the cost of a lookup.

Solution. Minimum occupancy is $\lceil 101/2 \rceil = 51$ pointers, so in the worst case $h \approx \lceil \log_{51} 10{,}000{,}000 \rceil = \lceil \log(10^7)/\log(51) \rceil = \lceil 7 \div 1.708 \rceil =$ **5**. If nodes are nearly full (fan-out ≈ 100), $h \approx \lceil 7/2 \rceil =$ **4**. A lookup therefore costs 4–5 block accesses, of which the upper levels are almost always already in the buffer pool — so the real cost is often one or two physical reads.

Problem 23.3 — Explain why a B^+ tree is preferred to a B-tree for database indexing.

Solution. Three reasons. **Higher fan-out:** internal nodes hold only keys and pointers, not data, so more separators fit in a block, the tree is shallower, and fewer block accesses are needed. **Range queries:** all records are in the leaves, which are chained, so `WHERE x BETWEEN a AND b` finds the first leaf and then scans sideways; a B-tree must walk back up and down the tree repeatedly. **Predictable cost:** every lookup in a B^+ tree traverses the full height, so every query costs the same, which makes the optimizer's estimates reliable — in a B-tree a key found high in the tree is fast and one found in a leaf is slow.

Problem 23.4 — Insert 25 into an order-4 B^+ tree whose relevant leaf `[20, 22, 24]` is full and whose parent is `[30]`. Show the result.

Solution. An order-4 node holds at most 3 keys, so the leaf is full. Split it into `[20, 22]` and `[24, 25]`. The first key of the right leaf, **24**, is **copied** (not moved) into the parent, giving parent `[24, 30]`. The two new leaves are linked into the leaf chain. The parent had one key and can hold three, so no further propagation occurs and the height is unchanged.

Problem 23.5 — For `SELECT * FROM enrollment WHERE course_id = 1`, an index on `(semester, course_id)` exists but is not used. Explain and fix.

Solution. The index is ordered first by `semester`; entries for `course_id = 1` are scattered across every semester's section of the index, so it cannot be searched efficiently — the leftmost-prefix rule. Fixes: create an index on `(course_id)` or `(course_id, semester)`; or, if both query shapes are common, keep both indexes; or, if `semester` is always supplied in practice, add it to the query. Note that in some systems the optimizer may still choose an **index skip scan** when the leading column has very few distinct values, but this should not be relied upon.

Problem 23.6 — When does an index make performance *worse*? Give three situations.

Solution. (i) On a write-heavy table: every insert must add an entry to every index, so ten indexes turn one write into eleven. (ii) When the predicate matches a large fraction of rows: following index pointers produces random reads, one per row, which is slower than a single sequential scan — this is why optimizers switch to a scan above roughly 20–30 % selectivity. (iii) On a low-selectivity column such as a gender or status flag, where the index adds maintenance cost and is never usefully selective. A fourth case: a

redundant index duplicating the leading columns of another wastes space and gives the optimizer more plans among which to choose badly.

Problem 23.7 — Design the indexes for `university_db` given that the common queries are: students by department, enrolments by student, results by exam, and courses by code.

Solution.

```sql
CREATE INDEX idx_student_dept    ON student(dept_id);          -- foreign key + filter
CREATE INDEX idx_enrol_student   ON enrollment(student_id);    -- foreign key + join
CREATE INDEX idx_enrol_course    ON enrollment(course_id);     -- foreign key + join
CREATE INDEX idx_result_exam     ON result(exam_id);           -- foreign key + group by
CREATE INDEX idx_result_student  ON result(student_id);
-- course_code already has a UNIQUE constraint, which creates its own index
```

Every foreign key is indexed, which serves both joins and cascading deletes; `course_code` needs nothing extra because `UNIQUE` already provides an index. No index is created on `grade` or `semester` alone, because both have very few distinct values and are unselective on their own.

Review Questions

1. What is an index, and why is it faster than scanning the data file?
2. Distinguish primary, clustering and secondary indexes.
3. Compare dense and sparse indexes, and explain why secondary indexes must be dense.
4. Explain multilevel indexes with a worked block-access calculation.
5. State the properties of a B^+ tree and compare it with a B-tree.
6. Explain insertion with splitting and the difference between copy-up and move-up.
7. Explain static, extendible and linear hashing.
8. Compare B^+ tree and hash indexes across at least five criteria.
9. State and illustrate the leftmost-prefix rule.
10. List six situations in which an index will not be used.

Multiple Choice Questions

1. In a B^+ tree, data pointers are stored in (a) the root (b) internal nodes (c) leaf nodes (d) all nodes.
2. A sparse index has one entry per (a) record (b) block (c) file (d) key value.
3. A secondary index must be (a) sparse (b) dense (c) clustered (d) hashed.
4. Which cannot support range queries? (a) B^+ tree (b) hash index (c) clustering index (d) multilevel index.
5. B^+ tree leaves are (a) unordered (b) chained in key order (c) stored in the root (d) sparse.
6. An index on `(a, b)` can serve a predicate on (a) b alone (b) a alone (c) neither (d) only both together.
7. When a B^+ tree leaf splits, the middle key is (a) moved up (b) copied up (c) deleted (d) ignored.
8. An index-only scan is possible with a (a) sparse (b) covering (c) hash (d) clustering index.
9. Indexes primarily slow down (a) reads (b) writes (c) joins (d) sorting.
10. Extendible hashing doubles the (a) buckets (b) directory (c) file (d) records.

Answers: 1-(c) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What is a search key?

2. Why is a B^+ tree balanced?
3. What is fan-out?
4. Define selectivity.
5. State the leftmost-prefix rule.
6. What is a covering index?
7. Why can a hash index not serve `ORDER BY`?
8. Give one cost of adding an index.
9. What happens when a B^+ tree root splits?
10. Name one predicate that defeats an index.

Long / Descriptive Questions

1. Explain indexing fully: types, dense versus sparse, multilevel, with block-access calculations.
2. Explain B^+ trees in detail — structure, height, search, insertion with splitting, deletion with merging — and compare with B-trees.
3. Explain static, extendible and linear hashing with diagrams, and compare hashing with tree indexing.
4. Discuss index design in practice: which columns to index, composite and covering indexes, and when indexes are ignored.
5. Analyse the costs and benefits of indexing for a write-heavy and for a read-heavy workload.

Practical Exercises

1. Create indexes on all foreign keys of `university_db` and compare `EXPLAIN` output for a five-table join before and after.
2. Generate a table of one million rows and measure lookup time with and without an index.
3. Build a composite index and demonstrate the leftmost-prefix rule with three queries and their execution plans.
4. Demonstrate a covering index producing an index-only scan, and verify it in `EXPLAIN`.
5. Measure insert throughput on a table with zero, three and eight indexes, and plot the result.

CHAPTER 24

RAID

A single disk is both a performance bottleneck and a single point of failure. **RAID** — Redundant Array of Independent (originally *Inexpensive*) Disks — combines several physical disks into one logical unit to improve throughput, reliability, or both. The DBMS sees one large, fast, fault-tolerant device; the RAID controller or software handles the distribution.

24.1 The Two Techniques

Every RAID level is built from two ideas, used alone or together.

Striping splits data across disks so that several disks work on one request in parallel. The **stripe unit** (chunk) is the amount written to one disk before moving to the next — typically 64 KB to 1 MB. Striping improves throughput but adds no redundancy.

Redundancy stores extra information from which lost data can be reconstructed. It takes two forms:

- **Mirroring** — a complete second copy.
- **Parity** — a computed checksum from which any one missing block can be rebuilt.

How parity works

Parity uses exclusive-or (XOR), which has the property that any one operand can be recovered from the others:

```
D0 = 1011
D1 = 0110
D2 = 1100
P  = D0 ⊕ D1 ⊕ D2 = 0001

If D1 is lost:   D1 = D0 ⊕ D2 ⊕ P = 1011 ⊕ 1100 ⊕ 0001 = 0110   ✓
```

This is why parity costs only one disk's worth of capacity regardless of the number of data disks — and why it can survive only **one** failure per parity block.

24.2 Mean Time to Failure

If one disk has a mean time to failure (MTTF) of 1,000,000 hours, an array of 100 disks without redundancy has an expected time to its *first* failure of roughly

```
1,000,000 ÷ 100 = 10,000 hours  ≈  14 months
```

More disks means more failures, not fewer. Redundancy is therefore not optional in a large array; it is the only thing that makes large arrays usable.

With mirroring, data is lost only if the second disk fails before the first is replaced and rebuilt. With a mean time to repair (MTTR) of 10 hours, the mean time to data loss for a mirrored pair is approximately

$$\text{MTTF}^2 \div (2 \times \text{MTTR}) = (10^6)^2 \div 20 = 5 \times 10^{10} \text{ hours}$$

— millions of years, assuming independent failures. In practice failures are correlated (same batch, same

power event, same heat), which is why real-world reliability is lower and why RAID is never a substitute for backup.

24.3 RAID 0 — Striping

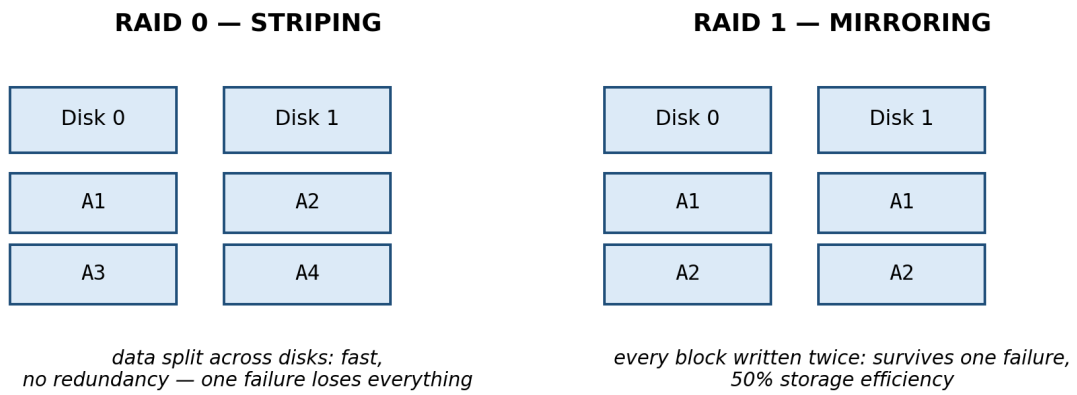


Figure 24.1: RAID 0 striping and RAID 1 mirroring

Data blocks are distributed round-robin across n disks with no redundancy.

Property	Value
Minimum disks	2
Usable capacity	100 % (n × disk size)
Fault tolerance	**None** — any single failure destroys the whole array
Read performance	Up to n×
Write performance	Up to n×

Use: scratch space, temporary tables, sort areas, caches — data that can be regenerated. **Never** for primary database files.

24.4 RAID 1 — Mirroring

Every block is written to two (or more) disks.

Property	Value
Minimum disks	2
Usable capacity	50 %
Fault tolerance	One disk per mirror pair
Read performance	Up to 2× (either copy can serve a read)
Write performance	1× (both copies must be written)
Rebuild	Simple copy from the survivor

Use: operating system volumes and — importantly — **database transaction logs**, which are written sequentially and must be both fast and durable.

24.5 RAID 2, 3 and 4 (Historical)

- **RAID 2** — bit-level striping with Hamming-code error correction. Obsolete; modern disks correct errors internally.

- **RAID 3** — byte-level striping with a dedicated parity disk. Every request involves every disk, so only one request can be served at a time. Obsolete.
- **RAID 4** — block-level striping with a dedicated parity disk. Reads are good, but **every write must update the single parity disk**, which becomes a bottleneck. RAID 5 exists to fix precisely this.

24.6 RAID 5 — Distributed Parity

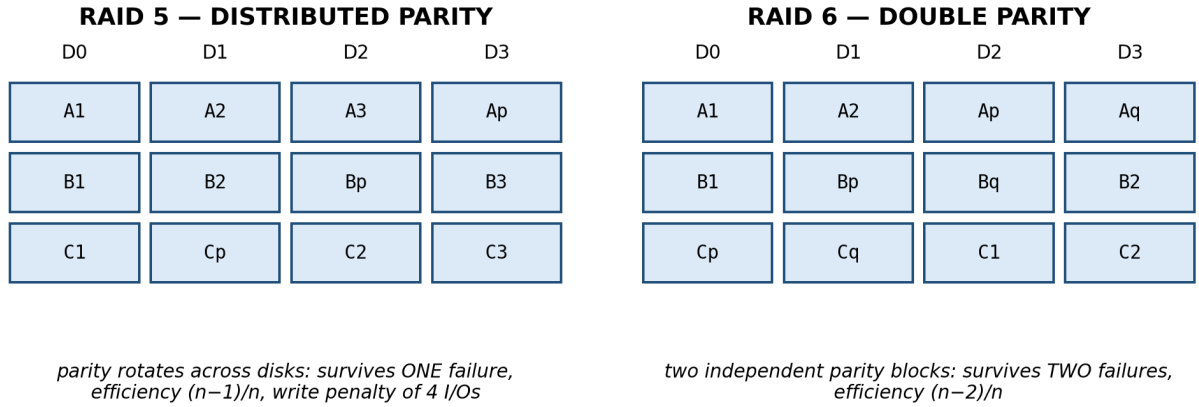


Figure 24.2: RAID 5 distributed parity and RAID 6 double parity

Block-level striping with parity **rotated** across all disks, so no single disk handles all parity writes.

Property	Value
Minimum disks	3
Usable capacity	(n − 1) / n — e.g. 75 % with 4 disks
Fault tolerance	One disk
Read performance	Excellent, (n − 1)×
Write performance	Poor for small random writes

The small-write penalty

Updating one data block requires recomputing its stripe's parity. The controller does it in **four I/Os**:

```
1. read old data block
2. read old parity block
3. write new data block
4. write new parity  =  old parity ⊕ old data ⊕ new data
```

So one logical write costs four physical I/Os. For an OLTP database dominated by small random updates, this is a serious penalty.

The rebuild risk

When a disk fails, every read of that disk's data must be reconstructed from all the others — the array runs **degraded** and slow. Rebuilding onto a replacement reads every block of every surviving disk. On modern multi-terabyte disks a rebuild can take a day or more, and during that window a second failure, or even one unreadable sector on a surviving disk, loses data. This is the main argument against RAID 5 for large disks today.

Use: read-mostly data, archives, reporting databases, file servers.

24.7 RAID 6 — Double Parity

Two independent parity blocks per stripe (commonly called P and Q, using XOR and a Reed–Solomon code respectively).

Property	Value
Minimum disks	4
Usable capacity	(n − 2) / n
Fault tolerance	**Two** disks
Write penalty	Six I/Os per small write

RAID 6 directly addresses the rebuild risk of RAID 5: a second failure during rebuild is survivable. It is the usual choice for large-capacity arrays where rebuilds are long.

24.8 RAID 10 (1 + 0) — Striped Mirrors

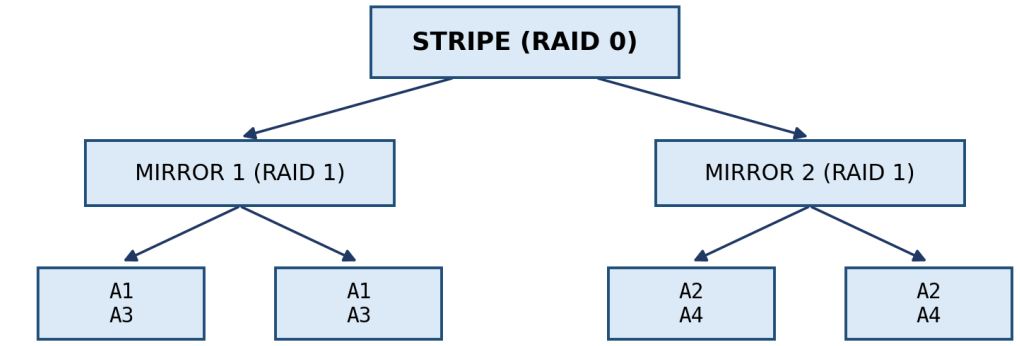


Figure 24.3: RAID 10 — mirrored pairs, striped together

Disks are first mirrored in pairs; the pairs are then striped.

Property	Value
Minimum disks	4
Usable capacity	50 %
Fault tolerance	One disk per mirror pair (possibly several in total)
Read performance	Excellent
Write performance	Excellent — two writes, no parity computation
Rebuild	Fast: copy one disk from its mirror partner

RAID 10 combines the speed of striping with the simplicity of mirroring, has no write penalty beyond the second copy, and rebuilds quickly because only one disk is copied. It is the standard recommendation for **OLTP database data files**.

> RAID 01 (0 + 1) — mirroring two striped sets — sounds equivalent but is not. One failure takes a whole stripe set offline, so a second failure on the *other* set destroys the array. RAID 10 tolerates any second failure except the partner of the first disk. Prefer RAID 10.

24.9 Comparison and Selection

Level	Technique	Min disks	Capacity	Tolerates	Read	Small write	Typical DB use
0	Striping	2	100 %	0	Excellent	Excellent	Temp / scratch only
1	Mirroring	2	50 %	1	Good	Good	Transaction logs, OS
5	Striping + distributed parity	3	(n−1)/n	1	Excellent	Poor (4 I/O)	Read-mostly, archives
6	Striping + double parity	4	(n−2)/n	2	Excellent	Very poor (6 I/O)	Large archives, backup targets
10	Mirrored pairs, striped	4	50 %	1 per pair	Excellent	Excellent	OLTP data files

A typical database server layout

Volume	RAID level	Reason
Operating system	RAID 1	Small, must survive a disk failure
Transaction log	RAID 1 or RAID 10	Sequential writes; durability is critical; no parity penalty
Data files (OLTP)	RAID 10	Random reads and writes; fast rebuild
Data warehouse / reporting	RAID 5 or RAID 6	Read-mostly; capacity efficiency matters
Temporary / sort space	RAID 0	Speed only; contents are disposable
Backups	RAID 6	Large, cheap, must survive two failures

24.10 Hardware versus Software RAID

Basis	Hardware RAID	Software RAID
Implemented by	Dedicated controller card	Operating system (Linux `mdadm`, ZFS, Windows Storage Spaces)
CPU cost	None on the host	Some (negligible on modern CPUs except for parity)
Write cache	Battery- or flash-backed cache	Relies on OS; no protected cache
Portability	Tied to the controller model	Array can move to any machine
Cost	Higher	Free

A battery-backed write cache is significant for databases: the controller can acknowledge a write as soon as it reaches cache, safely, because the cache survives a power failure. This removes most of the RAID 5 write penalty from the application's point of view.

24.11 What RAID Does Not Do

RAID protects against **disk failure**, and nothing else. It does **not** protect against:

- Accidental `DELETE` or `DROP` — the mistake is faithfully written to every mirror.
- Application bugs that corrupt data.
- Ransomware, which encrypts every copy simultaneously.
- Controller failure, which may corrupt the whole array.
- Fire, flood or theft of the server.

> **RAID is not a backup.** Backup, point-in-time recovery and off-site copies (Chapter 32) remain essential, and a DBA who relies on RAID alone will eventually lose data.

Chapter Summary

- RAID combines disks for performance (striping) and reliability (mirroring or parity).
- Parity uses XOR, allowing any one lost block to be reconstructed at the cost of one disk's capacity.
- More disks means more frequent failures; redundancy is what makes large arrays viable.
- RAID 0 is fast with no protection; RAID 1 mirrors; RAID 5 distributes parity with a four-I/O small-write penalty and a risky rebuild; RAID 6 adds a second parity and survives two failures; RAID 10 stripes mirrored pairs and is the standard for OLTP.
- Logs belong on RAID 1 or 10, OLTP data on RAID 10, read-mostly data on RAID 5 or 6, scratch space on RAID 0.
- RAID protects only against disk failure; it is never a substitute for backup.

Key Terms

RAID · Striping · Stripe unit · Mirroring · Parity · XOR · MTTF · MTTR · Mean time to data loss · RAID 0 · RAID 1 · RAID 4 · RAID 5 · RAID 6 · RAID 10 · RAID 01 · Distributed parity · Small-write penalty · Degraded mode · Rebuild · Hot spare · Hardware RAID · Software RAID · Battery-backed write cache

Important Formulas / Notation

```
array MTTF (no redundancy)    ≈ disk MTTF / number of disks
mirrored pair data-loss time  ≈ MTTF² / (2 × MTTR)
RAID 5 capacity  = (n − 1) × disk size        RAID 6 capacity = (n − 2) × disk size
RAID 1/10 capacity = n/2 × disk size
RAID 5 small write = 4 I/Os                   RAID 6 small write = 6 I/Os
parity P = D0 ⊕ D1 ⊕ … ⊕ Dk-1;   lost Di = P ⊕ (all other Dj)
```

Common Mistakes

1. Treating RAID as a backup.
2. Placing database data files on RAID 0.
3. Choosing RAID 5 for a write-heavy OLTP database without a write cache.
4. Confusing RAID 10 with RAID 01.
5. Forgetting that RAID 5 survives only one failure, and that rebuilds on large disks are long.
6. Believing more disks automatically means more reliability.
7. Stating RAID 1 capacity as 100 %; it is 50 %.

Solved Problems

Problem 24.1 — Six 4 TB disks are available. Compute the usable capacity and fault tolerance for RAID 0, 1 (as three pairs), 5, 6 and 10.

Solution.

Level	Usable capacity	Tolerates
RAID 0	6 × 4 = 24 TB	0 failures
RAID 1 (three pairs)	3 × 4 = 12 TB	1 per pair
RAID 5	(6 − 1) × 4 = 20 TB	1 failure
RAID 6	(6 − 2) × 4 = 16 TB	Any 2 failures
RAID 10	6/2 × 4 = 12 TB	1 per mirror pair

Problem 24.2 — Data blocks are D0 = 1100, D1 = 1010, D2 = 0111. Compute the parity and show the recovery of D2 after a failure.

Solution.

```
D0 ⊕ D1       = 1100 ⊕ 1010 = 0110
P = 0110 ⊕ D2 = 0110 ⊕ 0111 = 0001

Recover D2:  D0 ⊕ D1 ⊕ P = 1100 ⊕ 1010 ⊕ 0001
           = 0110 ⊕ 0001 = 0111   ✓
```

Problem 24.3 — An array has 50 disks, each with MTTF 1,200,000 hours and no redundancy. Estimate the expected time to the first failure.

Solution. 1,200,000 ÷ 50 = **24,000 hours ≈ 2.7 years**. With 500 disks the figure falls to 2,400 hours — about 100 days. At data-centre scale, disk failure is a routine weekly event, which is why redundancy and automated rebuilds are built in rather than treated as exceptional.

Problem 24.4 — Explain the RAID 5 small-write penalty with the I/O sequence, and state how RAID 10 avoids it.

Solution. To update one data block in RAID 5, the new parity must be computed as `P_new = P_old ⊕ D_old ⊕ D_new`. That requires reading the old data and old parity (2 I/Os) and writing the new data and new parity (2 I/Os): **four I/Os** for one logical write, two of them reads that must complete before the writes can begin. RAID 10 has no parity; a write simply goes to both disks of the mirror pair, in parallel — two I/Os, with no dependent reads. For small random updates this makes RAID 10 roughly twice as fast, which is why it is preferred for OLTP.

Problem 24.5 — Recommend RAID levels for a hospital database server with an OLTP patient system, a transaction log, a nightly reporting database and a backup store. Justify each.

Solution. **Transaction log — RAID 1** (or RAID 10 if throughput demands): writes are sequential and every commit waits for them, so durability and low latency matter more than capacity. **Patient OLTP data — RAID 10**: many small random reads and writes; no parity penalty; fast rebuild minimizes the degraded window. **Reporting database — RAID 5** (or RAID 6 for very large disks): read-mostly, rebuilt nightly from the OLTP source, so write penalty is irrelevant and capacity efficiency is valuable. **Backup store — RAID 6**: large and cheap, must survive two failures during long rebuilds, and should additionally be replicated off-site because RAID alone does not protect against site loss.

Problem 24.6 — Why is RAID 10 preferred to RAID 01, even though both use the same disks and give the same capacity?

Solution. In RAID 01, two striped sets are mirrored. A single disk failure takes its **entire** stripe set offline, leaving the array running on the other set alone; any second failure in that surviving set destroys the array. In RAID 10, disks are mirrored in pairs and the pairs are striped; after one failure, only that disk's partner is critical, and every other disk can fail without data loss. With four disks, RAID 10 survives two thirds of all possible second failures while RAID 01 survives only one third. Rebuild is also faster in RAID 10, because only one disk is copied rather than a whole stripe set.

Review Questions

1. Explain striping and redundancy and how each affects performance and reliability.
2. Explain how XOR parity allows reconstruction, with a worked example.
3. Why does adding disks decrease an array's time to first failure?
4. Describe RAID 0, 1, 5, 6 and 10 with capacity, fault tolerance and performance.
5. Explain the RAID 5 small-write penalty.

6. Why is rebuilding a large RAID 5 array risky?
7. Distinguish RAID 10 from RAID 01.
8. Recommend RAID levels for each volume of a database server and justify them.
9. Compare hardware and software RAID.
10. Explain why RAID is not a backup.

Multiple Choice Questions

1. RAID 0 provides (a) mirroring (b) striping only (c) parity (d) double parity.
2. RAID 1 usable capacity is (a) 100 % (b) 75 % (c) 50 % (d) 33 %.
3. RAID 5 with 5 disks of 2 TB gives usable capacity of (a) 10 TB (b) 8 TB (c) 6 TB (d) 5 TB.
4. RAID 6 tolerates (a) 0 (b) 1 (c) 2 (d) 3 disk failures.
5. A small write in RAID 5 costs (a) 1 (b) 2 (c) 4 (d) 6 I/Os.
6. The recommended level for OLTP data files is (a) RAID 0 (b) RAID 5 (c) RAID 6 (d) RAID 10.
7. Parity is computed with (a) AND (b) OR (c) XOR (d) NOT.
8. RAID 4 is limited by (a) no redundancy (b) the dedicated parity disk bottleneck (c) mirroring cost (d) double parity.
9. RAID protects against (a) accidental DELETE (b) ransomware (c) single disk failure (d) fire.
10. The minimum number of disks for RAID 10 is (a) 2 (b) 3 (c) 4 (d) 6.

Answers: 1-(b) 2-(c) 3-(b) 4-(c) 5-(c) 6-(d) 7-(c) 8-(b) 9-(c) 10-(c)

Short Questions

1. Expand RAID.
2. What is a stripe unit?
3. State the capacity formula for RAID 5.
4. What does degraded mode mean?
5. Why does RAID 5 distribute parity?
6. Which level suits transaction logs?
7. What is a hot spare?
8. What advantage does a battery-backed cache give?
9. How many failures does RAID 1 tolerate per pair?
10. Name two threats RAID does not address.

Long / Descriptive Questions

1. Explain every RAID level in detail with diagrams, capacity, fault tolerance and performance characteristics.
2. Explain parity and the RAID 5 write penalty, and discuss the rebuild risk on large disks.
3. Compare RAID 5, RAID 6 and RAID 10 for a database server, making a justified recommendation for each workload.
4. Discuss reliability of disk arrays using MTTF and MTTR, including the effect of correlated failures.
5. “RAID is not a backup.” Discuss with examples, and describe what additional protection a database requires.

Practical Exercises

1. On a Linux virtual machine, create a software RAID 1 array with `mdadm` using two virtual disks, fail one disk, and observe the rebuild.
2. Compute usable capacity and fault tolerance for every RAID level using eight 8 TB disks.
3. Work through XOR parity for four data blocks by hand and verify recovery of each block.
4. Design the storage layout for a university database server and justify every RAID choice.
5. Benchmark random write performance on RAID 0, RAID 1 and RAID 5 software arrays, and relate the results to the write-penalty analysis.

CHAPTER 25

Query Processing

A SQL statement says *what* is wanted. Query processing is everything the DBMS does to turn that statement into a sequence of block reads that produces the answer. This chapter follows a query from text to result and costs the main algorithms; Chapter 26 then shows how the cheapest plan is chosen.

25.1 The Stages

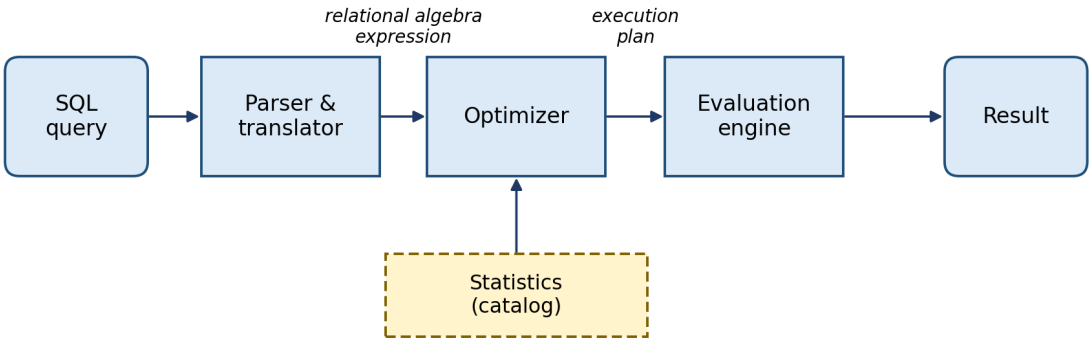


Figure 25.1: The stages of query processing

Stage	Input	Output	Work done
Parsing	SQL text	Parse tree	Syntax check against the grammar
Semantic analysis	Parse tree	Validated tree	Do the tables and columns exist? Are types compatible? Does the user have privileges?
Translation	Validated tree	Relational algebra expression	SQL constructs mapped to σ, π, $\bowtie$, $\mathcal{G}$
Optimization	Algebra expression	Execution plan	Choose operator order and algorithms (Chapter 26)
Evaluation	Execution plan	Result rows	Run the plan, reading pages through the buffer manager

Consider:

```
SELECT s.name, c.title
FROM   student s
JOIN   enrollment e ON e.student_id = s.student_id
JOIN   course     c ON c.course_id  = e.course_id
WHERE  s.dept_id = 'CSE';
```

The translator produces an algebra expression, which is best pictured as a **query tree** — leaves are base relations, internal nodes are operators, and data flows upward.

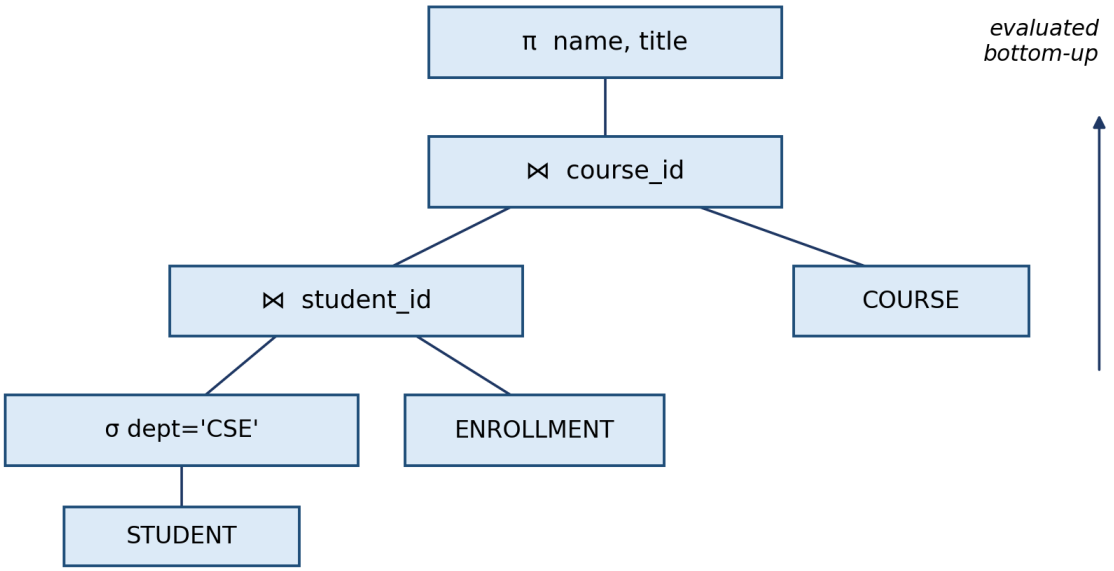


Figure 25.2: A query tree for the example query

An **execution plan** is a query tree in which every node is annotated with the *algorithm* to use: ”use the index on `student.dept_id` for σ“, ”hash join here“, ”nested loop there”. Two plans for the same tree can differ in cost by several orders of magnitude.

25.2 Measuring Cost

Because disk access dominates everything else (Chapter 22), cost is measured primarily in **block transfers** and **seeks**:

```
cost = (number of block transfers × tT) + (number of seeks × tS)
```

Notation used throughout this chapter:

Symbol	Meaning
n_R	Number of tuples in R
b_R	Number of blocks of R
M	Number of buffer pages available in memory
h_i	Height of index *i*

Figures are illustrative: `student` has n = 10,000 tuples in b = 500 blocks; `enrollment` has n = 100,000 tuples in b = 2,500 blocks.

25.3 Algorithms for Selection

Algorithm	Applicable when	Approximate cost
A1 Linear scan	Always	b_R (or b_R/2 on average for an equality on a key)
A2 Binary search	File ordered on the attribute, equality	$\lceil \log_2 b_R \rceil$ + matching blocks
A3 Primary B^+ tree, equality on key	Clustered index on a key	h_i + 1

Algorithm	Applicable when	Approximate cost
A4 Primary B^+ tree, equality on non-key	Clustered index, several matches	h_i + blocks holding matches
A5 Secondary B^+ tree, equality	Unclustered index	h_i + one block **per matching record**
A6 Range with primary index	`>`, `<`, `BETWEEN` on clustered index	h_i + blocks in range

The critical distinction is between A4 and A5. With a **clustered** index, matching records sit together in a few blocks. With a **secondary** index, each match may be in a different block, so 1,000 matches may cost 1,000 random reads — worse than a full scan of a 500-block table. This is exactly why optimizers abandon an index when a predicate is unselective.

Worked comparison. `SELECT * FROM student WHERE dept_id = 'CSE'` matches 2,500 of 10,000 students.

- Linear scan: 500 block transfers, one seek — sequential.
- Secondary index on `dept_id` (height 3): up to 3 + 2,500 random reads.

The scan wins by a factor of five. With 10 matches the index would cost 13 reads and win easily. The optimizer's decision rests entirely on the estimated number of matches.

Conjunctions and disjunctions

For `σ_{θ1 ∧ θ2}`, use the most selective condition that has an index and test the other on the fetched rows; or intersect the record-pointer sets from two indexes. For `σ_{θ1 ∨ θ2}`, an index can help only if *every* disjunct has one (union of pointer sets); otherwise a linear scan is required.

25.4 Sorting: External Merge Sort

`ORDER BY`, `GROUP BY`, `DISTINCT`, `UNION` and sort-merge join all require sorting, usually of data larger than memory.

```
Pass 0 (create runs):  read M blocks at a time, sort in memory, write out
                       → ⌈b_R / M⌉ sorted runs
Pass 1, 2, …:          merge (M − 1) runs at a time into longer runs
                       until one run remains
```

Number of merge passes = $\lceil \log_{M-1}(b_R / M) \rceil$. Each pass reads and writes every block.

```
    total block transfers ≈ 2 b_R × (1 + ⌈log_{M−1}(b_R / M)⌉)    (excluding final write)
```

Example. b_R = 2,500 blocks, M = 50 buffers. Pass 0 produces $\lceil 2500/50 \rceil = 50$ runs. One merge pass with 49-way merging cannot quite finish 50 runs, so two passes are needed:

$\lceil \log_{49} 50 \rceil = 2$. Transfers ≈ 2 × 2,500 × (1 + 2) = **15,000**. Doubling the memory to M = 100 gives 25 runs, merged in one pass: 2 × 2,500 × 2 = **10,000**. Memory given to sorting pays for itself directly — the reason `sort_buffer_size` and `work_mem` are tunable.

25.5 Join Algorithms

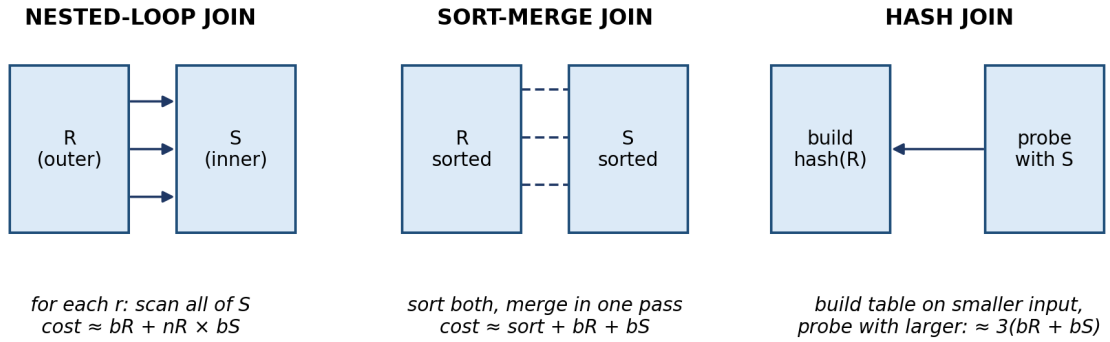


Figure 25.3: The three families of join algorithm

Nested-loop join

```
for each tuple r in R:
    for each tuple s in S:
        if r and s satisfy θ: output (r, s)
```

Cost with one buffer per relation: **n_R × b_S + b_R**. With R = `student` (10,000 tuples) outer and S = `enrollment` inner: 10,000 × 2,500 + 500 = **25,000,500** transfers — hopeless.

Block nested-loop join

Process the outer relation a block at a time rather than a tuple at a time:

```
    cost = b_R × b_S + b_R  =  500 × 2,500 + 500  =  1,250,500
```

With M buffers, reading M − 2 outer blocks at once:

```
    cost = ⌈b_R / (M − 2)⌉ × b_S + b_R
```

With M = 52: ⌈500/50⌉ × 2,500 + 500 = **25,500**. Choose the **smaller** relation as the outer, and give it as much memory as possible.

Indexed nested-loop join

If the inner relation has an index on the join attribute, each outer tuple probes the index instead of scanning:

```
    cost = b_R + n_R × c,     c = cost of one index lookup
```

With a B^+ tree on `enrollment.student_id` of height 3 plus one data block: 500 + 10,000 × 4 = **40,500**. Excellent when the outer relation is small after filtering — for example, only CSE students.

Sort-merge join

Sort both relations on the join attribute (if not already sorted), then merge them in a single synchronized pass:

```
    cost = sort(R) + sort(S) + b_R + b_S
```

If both are already sorted — for example, both have clustered indexes on the join key — the cost is just **b_R + b_S = 3,000**. Works only for equi joins and natural joins.

Hash join

```
1. Partition: hash R and S on the join attribute into buckets R_i and S_i
2. Build:     for each i, load R_i into an in-memory hash table
3. Probe:     scan S_i and look up each tuple in the hash table
```

Cost ≈ **3 (b_R + b_S) = 9,000** (read and write both during partitioning, read both again during build and probe). If the smaller relation fits entirely in memory, partitioning is unnecessary and the cost falls to **b_R + b_S**. Hash join is usually the fastest algorithm for large unsorted equi joins and is the workhorse of analytical queries.

Comparison

Algorithm	Cost (student ⋈ enrollment)	Best when	Join types
Tuple nested loop	25,000,500	Almost never	Any θ
Block nested loop	1,250,500 (25,500 with M = 52)	Small relations, no index	Any θ
Indexed nested loop	40,500	Small outer, index on inner	Equi
Sort-merge	3,000 if pre-sorted	Inputs already sorted	Equi
Hash join	≈ 9,000	Large, unsorted, equi	Equi

Note that only the nested-loop family handles non-equality conditions such as `r.marks > s.pass_mark`.

25.6 Other Operations

Operation	Typical implementation
Projection with duplicate removal	Sort then remove adjacent duplicates, or hash
`GROUP BY` with aggregates	Sort on grouping key, or hash aggregation
Set operations	Sort both inputs and merge, or hash
Outer joins	Modified merge or hash join that emits unmatched tuples padded with NULL

25.7 Evaluation Strategies

Materialization

Each operator runs to completion and writes its result to a temporary relation, which the next operator reads. Simple, but pays for writing and rereading every intermediate result.

Pipelining

Operators are chained so that each tuple is passed to the next operator as soon as it is produced, with no temporary relation. In the **iterator model**, every operator exposes `open()`, `next()` and `close()`; the root repeatedly calls `next()` on its child, which calls its own child, pulling tuples up the tree one at a time.

```
π.next() → ⋈.next() → σ.next() → scan.next()
```

Pipelining saves I/O and returns the first rows quickly. It is impossible across **blocking operators**, which must see their entire input before producing any output: sorting, hash-table building and most aggregations.

Chapter Summary

- A query passes through parsing, semantic analysis, translation to relational algebra, optimization and evaluation.
- An execution plan is a query tree annotated with an algorithm for each operator.
- Cost is measured in block transfers and seeks.
- Selection uses scans, binary search or indexes; a secondary index is only cheaper than a scan for selective predicates.
- External merge sort sorts data larger than memory in ⌈log_{M−1}(b/M)⌉ + 1 passes.
- Joins can be computed by nested loop, block nested loop, indexed nested loop, sort-merge or hash join, with costs differing by orders of magnitude.
- Pipelining avoids temporary results; blocking operators such as sort force materialization.

Key Terms

Query processing · Parser · Semantic analysis · Query tree · Execution plan · Cost model · Block transfer · Seek · Linear scan · Clustered versus secondary index access · Selectivity · External merge sort · Run · Merge pass · Nested-loop join · Block nested-loop join · Indexed nested-loop join · Sort-merge join · Hash join · Build and probe · Partitioning · Materialization · Pipelining · Iterator model · Blocking operator

Important Formulas / Notation

```
nested loop          n_R × b_S + b_R
block nested loop    ⌈b_R / (M − 2)⌉ × b_S + b_R
indexed nested loop  b_R + n_R × c
sort-merge           sort(R) + sort(S) + b_R + b_S
hash join            ≈ 3 (b_R + b_S)
external sort passes 1 + ⌈log_{M−1}(b_R / M)⌉
```

Common Mistakes

1. Assuming an index always beats a scan; for unselective predicates on a secondary index it does not.
2. Choosing the larger relation as the outer in a nested-loop join.
3. Using sort-merge or hash join for a non-equality condition — only nested loops handle θ-joins.
4. Forgetting that pipelining breaks at sorts and hash builds.
5. Costing in tuples rather than blocks.
6. Ignoring memory: sort and hash costs depend heavily on M.

Solved Problems

Problem 25.1 — R has 1,000 tuples in 100 blocks; S has 5,000 tuples in 400 blocks; M = 12. Compute the block nested-loop join cost with R as outer and with S as outer.

Solution. With R outer: ⌈100/10⌉ × 400 + 100 = 10 × 400 + 100 = **4,100**. With S outer: ⌈400/10⌉ × 100 + 400 = 40 × 100 + 400 = **4,400**. The smaller relation should be the outer, confirming the rule.

Problem 25.2 — Compute the hash join cost for the same R and S, and state when the cost would fall to b_R + b_S.

Solution. Partitioned hash join ≈ 3 (100 + 400) = **1,500** block transfers — far below the nested-loop cost. If R (100 blocks) fitted entirely in memory as a hash table — which requires M > 100 plus overhead — no partitioning would be needed; R is read once to build and S once to probe, costing **500**.

Problem 25.3 — A file of 10,000 blocks is sorted with M = 101 buffers. How many passes and block transfers are needed?

Solution. Pass 0 creates $\lceil 10{,}000 / 101 \rceil = 100$ runs. Merge fan-in is M − 1 = 100, so one merge pass combines all 100 runs. Total passes = 2. Transfers ≈ 2 × 10,000 × 2 = **40,000** (reading and writing each block in each pass). With M = 21 the same sort needs $1 + \lceil \log_{20} 477 \rceil = 1 + 3 = 4$ passes and 80,000 transfers.

Problem 25.4 — Explain why an optimizer may ignore an index on `enrollment.semester` for `WHERE semester = '5th'`.

Solution. `semester` has only about eight distinct values, so `'5th'` matches roughly one eighth of the table — 12,500 of 100,000 rows. Through a secondary index that means up to 12,500 random block reads, whereas a full scan reads 2,500 blocks sequentially. The scan is cheaper by a wide margin, so a correct optimizer rejects the index. This is an example where the index is not "broken"; it is simply not useful for that predicate.

Problem 25.5 — Why does pipelining not work across an `ORDER BY`?

Solution. Sorting is a blocking operation: the first output tuple is the smallest in the whole input, and that cannot be known until every input tuple has been seen. The sort operator must therefore consume its entire input — materializing it in memory or in sorted runs on disk — before emitting anything. Operators above the sort can resume pipelining once it begins producing output, but the pipeline cannot pass through it.

Problem 25.6 — For `student` ⋈ `enrollment` restricted to CSE students (2,500 students, 125 blocks after filtering), with a B^+ tree of height 3 on `enrollment.student_id`, which join algorithm is best?

Solution. Indexed nested loop: 125 (read the filtered outer) + 2,500 × (3 + 1) = **10,125** transfers. Hash join: ≈ 3 × (125 + 2,500) = **7,875** if partitioned, or 125 + 2,500 = **2,625** if the filtered student set fits in memory as a hash table — which at 125 blocks it very likely does. **Hash join** is best here. The indexed nested loop would win only if the filter were much more selective, say a handful of students.

Review Questions

1. List and explain the stages of query processing.
2. What is the difference between a query tree and an execution plan?
3. Why is cost measured in block transfers?
4. Compare the selection algorithms and explain why secondary indexes can lose to scans.
5. Explain external merge sort and derive its cost.
6. Describe and cost all join algorithms.
7. Which join algorithms work for non-equality conditions?
8. Explain hash join, including partitioning, build and probe.
9. Compare materialization and pipelining.
10. What is a blocking operator? Give three examples.

Multiple Choice Questions

1. Which stage checks that a table exists? (a) Parsing (b) Semantic analysis (c) Optimization (d) Evaluation.
2. The cost of a tuple nested-loop join is (a) b_R + b_S (b) n_R × b_S + b_R (c) 3(b_R + b_S) (d) b_R × b_S.
3. In a nested-loop join the outer relation should be the (a) larger (b) smaller (c) indexed (d) sorted one.

4. Which handles a θ-join with `>`? (a) Hash join (b) Sort-merge (c) Block nested loop (d) Indexed hash.
5. Hash join cost is approximately (a) b_R × b_S (b) 3(b_R + b_S) (c) n_R × n_S (d) log b_R.
6. Pipelining is blocked by (a) selection (b) projection (c) sorting (d) scanning.
7. A secondary index is best when the predicate is (a) unselective (b) highly selective (c) a range over most rows (d) on a flag.
8. External sort pass 0 creates (a) one run (b) ⌈b/M⌉ runs (c) M runs (d) b runs.
9. Sort-merge join is cheapest when inputs are (a) hashed (b) already sorted (c) small (d) unindexed.
10. In the iterator model each operator provides (a) open/next/close (b) read/write (c) lock/unlock (d) commit/rollback.

Answers: 1-(b) 2-(b) 3-(b) 4-(c) 5-(b) 6-(c) 7-(b) 8-(b) 9-(b) 10-(a)

Short Questions

1. What is an execution plan?
2. Give the block nested-loop join formula.
3. Why does the smaller relation go outer?
4. What are the two phases of a hash join?
5. What is a run in external sort?
6. Define pipelining.
7. Name one blocking operator.
8. When does indexed nested-loop join win?
9. What does M represent?
10. Why can hash join not do range joins?

Long / Descriptive Questions

1. Explain the stages of query processing with an example query, its query tree and an execution plan.
2. Explain and cost all selection algorithms, with attention to clustered versus secondary indexes.
3. Derive and explain the cost of external merge sort with a numerical example.
4. Describe and compare all join algorithms with full cost calculations on given relations.
5. Compare materialization and pipelining, explaining the iterator model and blocking operators.

Practical Exercises

1. Run `EXPLAIN` (and `EXPLAIN ANALYZE` in PostgreSQL or MySQL 8.0.18+) on five queries against `university_db` and identify the algorithm used at each node.
2. Force different join algorithms using optimizer hints and compare their measured times on a large generated table.
3. Measure sort performance while varying `sort_buffer_size` (MySQL) or `work_mem` (PostgreSQL).
4. Create a secondary index on a low-selectivity column and show the optimizer ignoring it.
5. Compute by hand the cost of three join algorithms for two tables of your choice, then compare with the plan the DBMS chooses.

CHAPTER 26

Query Optimization

The optimizer's job is to find, among the many plans that compute the same answer, one whose cost is low. It rarely finds the true optimum — the search space is too large — but it reliably avoids the disastrous plans, and that is what matters.

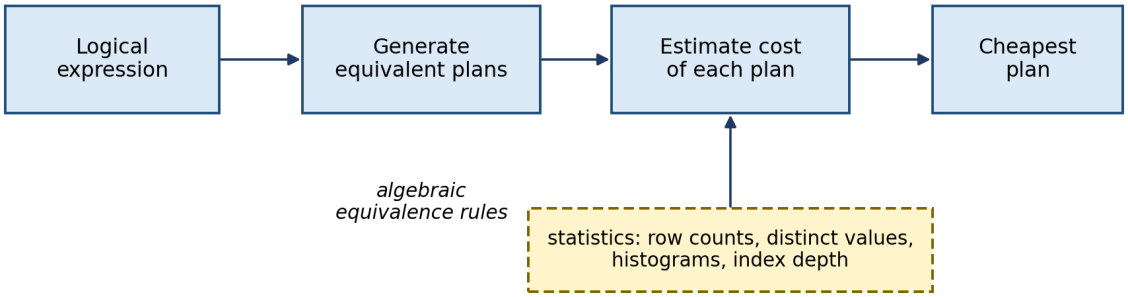


Figure 26.1: The structure of a cost-based optimizer

26.1 Why Optimization Matters

For a three-table join, the plans differ in join order (which pair first), join algorithm for each join, access method for each table, and where selections and projections are applied. Even with conservative choices the number of candidate plans runs to hundreds; for ten tables it runs into millions. Costs across those plans commonly vary by a factor of 10^3 to 10^6 — the difference between a query taking a millisecond and one taking hours.

Optimization therefore has two parts:

1. **Heuristic (rule-based) optimization** — apply transformations that are almost always beneficial.
2. **Cost-based optimization** — estimate the cost of alternative plans using statistics, and choose the cheapest.

26.2 Equivalence Rules

Transformations are legal only because relational algebra obeys algebraic laws. The key rules:

#	Rule	Statement
1	Cascade of σ	σ_{θ1 ∧ θ2}(E) = σ_{θ1}(σ_{θ2}(E))
2	Commutativity of σ	σ_{θ1}(σ_{θ2}(E)) = σ_{θ2}(σ_{θ1}(E))
3	Cascade of π	π_{L1}(π_{L2}(E)) = π_{L1}(E) when L1 ⊆ L2
4	σ into join	σ_{θ}(E1 × E2) = E1 ⋈_{θ} E2
5	Commutativity of ⋈	E1 ⋈ E2 = E2 ⋈ E1
6	Associativity of ⋈	(E1 ⋈ E2) ⋈ E3 = E1 ⋈ (E2 ⋈ E3)
7	σ distributes over ⋈	σ_{θ}(E1 ⋈ E2) = σ_{θ}(E1) ⋈ E2, if θ uses only E1's attributes

#	Rule	Statement
8	π distributes over ⋈	π_{L}(E1 ⋈ E2) = π_{L1}(E1) ⋈ π_{L2}(E2), keeping join attributes
9	σ over set operations	σ_{θ}(E1 ∪ E2) = σ_{θ}(E1) ∪ σ_{θ}(E2), likewise for ∩ and −

Rules 5 and 6 generate the join-order search space. Rules 7 and 8 are the basis of the most important heuristics.

26.3 Heuristic Optimization

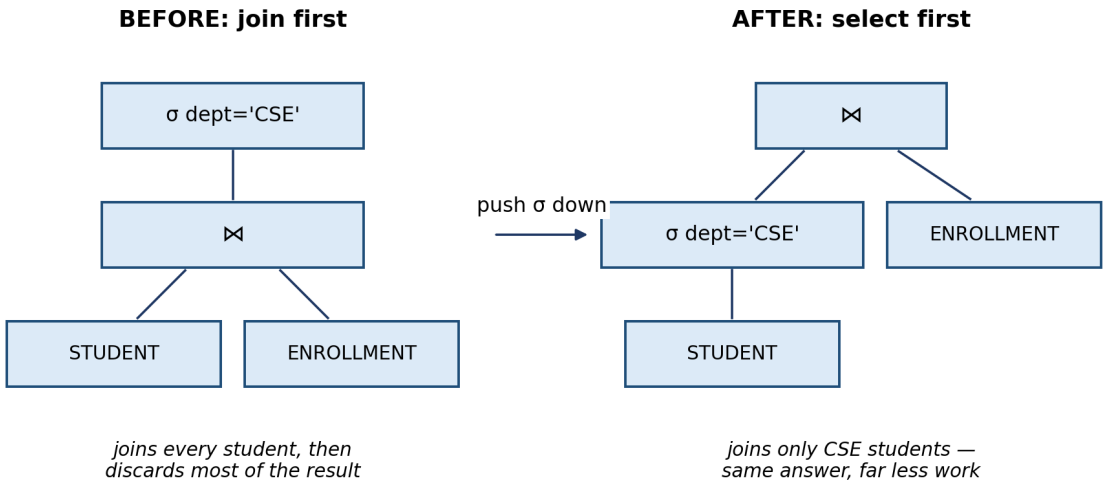


Figure 26.2: Pushing a selection below a join

The classic heuristic sequence:

1. **Break up conjunctive selections** into a cascade (rule 1), so each condition can move independently.
2. **Push selections down** as far as possible (rules 2, 7, 9). Filtering early shrinks every relation that flows upward.
3. **Perform the most restrictive selections and joins first**, those producing the fewest tuples.
4. **Replace a Cartesian product followed by a selection with a join** (rule 4). Never compute a product if a join condition exists.
5. **Push projections down** (rules 3, 8), keeping only the attributes needed above, so tuples are narrower and more fit per block.
6. **Identify subtrees that can be pipelined.**

Worked example. *Names of CSE students who scored above 80 in any examination.*

```
Initial:
   π_{name}( σ_{dept_id='CSE' ∧ marks > 80}( STUDENT × RESULT ) ∧ join condition )

Step 1 – split the conjunction:
   π_{name}( σ_{dept_id='CSE'}( σ_{marks>80}( σ_{S.id=R.id}( STUDENT × RESULT ) ) ) )

Step 2 – turn σ over × into a join:
   π_{name}( σ_{dept_id='CSE'}( σ_{marks>80}( STUDENT ⋈ RESULT ) ) )

Step 3 – push each selection to its own relation:
   π_{name}( σ_{dept_id='CSE'}(STUDENT) ⋈ σ_{marks>80}(RESULT) )
```

```
Step 4 — push projections down, keeping join attributes:
   π_{name}( π_{student_id,name}(σ_{dept_id='CSE'}(STUDENT))
             ⋈ π_{student_id}(σ_{marks>80}(RESULT)) )
```

If 25 % of students are in CSE and 20 % of results exceed 80, the join now combines roughly 2,500 × 20,000 candidate tuples instead of 10,000 × 100,000 — about a 20-fold reduction before any algorithm is chosen.

26.4 Cost-Based Optimization

Heuristics decide the shape of the tree; cost-based optimization chooses join order and algorithms by estimating costs.

Statistics in the catalog

Statistic	Used for
n_R, b_R	Scan and join costs
V(A, R) — distinct values of A in R	Selectivity of equality predicates
min(A), max(A)	Selectivity of range predicates
Histograms	Selectivity under skewed distributions
Index height, leaf blocks, clustering	Index access cost

Selectivity estimation

Predicate	Estimated matching tuples
A = v	n_R / V(A, R)
A > v (uniform)	n_R × (max − v) / (max − min)
θ1 ∧ θ2	n_R × s1 × s2 (assuming independence)
θ1 ∨ θ2	n_R × (s1 + s2 − s1 × s2)
¬θ	n_R × (1 − s)

Join size estimation

For `R ⋈ S` on attribute A:

```
    |R ⋈ S| ≈ (n_R × n_S) / max( V(A, R), V(A, S) )
```

If A is a key of S and a foreign key in R, the result has exactly n_R tuples — each R tuple matches one S tuple. For `enrollment ⋈ student` on `student_id`: 100,000 × 10,000 / max(10,000, 10,000) = **100,000**, as expected.

Join ordering

For *n* relations there are (2(n−1))! / (n−1)! possible join trees. Optimizers restrict the search:

- **Left-deep trees only** — the right input of every join is a base relation. This matches the pipelined nested-loop pattern and reduces the space to *n!* orders.
- **Dynamic programming** (System R) — compute the best plan for every subset of relations, building larger subsets from the best smaller ones. Optimal within the space considered, with complexity roughly $O(3^n)$; practical up to about 10–15 tables.
- **Greedy or genetic search** for very large joins (PostgreSQL switches to its genetic optimizer, GEQO, above 12 tables by default).

Interesting orders. A plan that is slightly more expensive but produces output sorted on a column needed later — for a subsequent merge join or an `ORDER BY` — may be cheaper overall. System R-style optimizers therefore keep the best plan *for each useful sort order*, not only the single cheapest.

26.5 Why Estimates Go Wrong

Cause	Effect	Remedy
Stale statistics after bulk load	Wrong cardinalities, wrong join order	`ANALYZE TABLE` (MySQL), `ANALYZE` (PostgreSQL)
Correlated columns (`city` and `postcode`)	Independence assumption underestimates rows	Extended / multi-column statistics
Skewed data (one department has 90 % of students)	Uniform assumption misestimates	Histograms
Functions on columns, parameters unknown at compile time	Default guesses used	Rewrite as sargable; use literal values or re-plan
Errors compound up the tree	A 10× error per join becomes 1,000× after three joins	Better statistics at the leaves

That last row explains a familiar experience: a query that runs well for months suddenly takes an hour after data volume changes, because a small estimation error has flipped the plan from a hash join to a nested loop.

26.6 Reading an Execution Plan

```
EXPLAIN
SELECT s.name, c.title
FROM   student s
JOIN   enrollment e ON e.student_id = s.student_id
JOIN   course c     ON c.course_id  = e.course_id
WHERE  s.dept_id = 'CSE';
```

Typical MySQL output (simplified):

id	table	type	key	rows	Extra
1	s	ref	idx_student_dept	2500	Using where
1	e	ref	idx_enrol_student	10	
1	c	eq_ref	PRIMARY	1	

Reading it: the optimizer starts with `student` using the department index (`ref` access, about 2,500 rows), then for each student looks up enrollments by the student index (about 10 each), then fetches each course by primary key (`eq_ref`, exactly one). This is a left-deep, indexed nested-loop plan. The `type` column ranks access methods roughly from best to worst: `const`, `eq_ref`, `ref`, `range`, `index`, `ALL` — where `ALL` means a full table scan.

PostgreSQL's `EXPLAIN ANALYZE` additionally prints the **actual** row counts next to the estimates. A large gap between estimated and actual rows is the first thing to look for when a query is slow.

26.7 Helping the Optimizer

1. Keep statistics current, especially after bulk loads.
2. Write sargable predicates — no functions or conversions on indexed columns.
3. Index foreign keys and selective filter columns.

4. Avoid `SELECT *`, so covering indexes can be used.
5. Prefer `EXISTS` / `NOT EXISTS` over `NOT IN` with nullable subqueries.
6. Use hints (`USE INDEX`, `/*+ HASH_JOIN */`, `SET enable_nestloop = off`) only as a diagnosed, documented last resort; they freeze a plan that may become wrong as data changes.

Chapter Summary

- Many plans compute the same answer, and their costs can differ by factors of a million.
- Algebraic equivalence rules make plan transformations legal.
- Heuristic optimization pushes selections and projections down, replaces products with joins, and performs restrictive operations first.
- Cost-based optimization estimates plan costs from catalog statistics: row counts, distinct values, ranges and histograms.
- Selectivity of `A = v` is estimated as 1/V(A, R); join size as n_R n_S / max(V(A,R), V(A,S)).
- Join order is searched with dynamic programming over left-deep trees, keeping plans with interesting orders.
- Estimates fail through stale statistics, correlation, skew and compounding errors; `EXPLAIN ANALYZE` exposes the gap.

Key Terms

Query optimization · Heuristic optimization · Cost-based optimization · Equivalence rule · Selection pushdown · Projection pushdown · Catalog statistics · Cardinality · Selectivity · V(A, R) · Histogram · Independence assumption · Join size estimation · Join ordering · Left-deep tree · Bushy tree · Dynamic programming · System R · Interesting order · Genetic optimizer · Execution plan · `EXPLAIN` · `EXPLAIN ANALYZE` · Optimizer hint

Important Formulas / Notation

```
selectivity(A = v)   = 1 / V(A, R)
selectivity(A > v)   = (max − v) / (max − min)
selectivity(θ1 ∧ θ2) = s1 × s2
selectivity(θ1 ∨ θ2) = s1 + s2 − s1 s2
|R ⋈ S|              ≈ n_R n_S / max(V(A,R), V(A,S))
join trees for n relations = (2(n−1))! / (n−1)!;  left-deep orders = n!
```

Common Mistakes

1. Believing the optimizer finds the true optimum; it finds a good plan within a restricted space.
2. Pushing a selection above a join to "simplify" — the opposite of the heuristic.
3. Assuming independence between correlated columns without realizing it.
4. Forgetting to re-analyse statistics after large data changes.
5. Using hints as a first resort instead of fixing statistics or predicates.
6. Reading `EXPLAIN` row estimates as actual counts.
7. Pushing a projection down so far that it discards a join attribute.

Solved Problems

Problem 26.1 — `student` has 10,000 rows; `dept_id` has 4 distinct values; `cgpa` ranges from 2.0 to 4.0. Estimate the rows for (a) `dept_id = 'CSE'`, (b) `cgpa > 3.5`, (c) both, (d) either.

Solution. (a) 10,000 / 4 = **2,500**, selectivity 0.25. (b) (4.0 − 3.5)/(4.0 − 2.0) = 0.25, so **2,500**. (c) 10,000 × 0.25 × 0.25 = **625**, assuming independence. (d) 10,000 × (0.25 + 0.25 − 0.0625) = **4,375**. If high CGPAs are actually concentrated in CSE, estimate (c) is too low — the correlation problem of Section 26.5.

Problem 26.2 — Apply heuristic optimization to: `π_{title}( σ_{S.dept_id='CSE' ∧ E.grade='A'}( STUDENT × ENROLLMENT × COURSE ) )`, with the join conditions `S.student_id = E.student_id` and `E.course_id = C.course_id`.

Solution.

```
1. Convert products + join conditions into joins:
   π_{title}( σ_{dept='CSE' ∧ grade='A'}( (STUDENT ⋈ ENROLLMENT) ⋈ COURSE ) )
2. Split and push selections to their relations:
   π_{title}( (σ_{dept='CSE'}(STUDENT) ⋈ σ_{grade='A'}(ENROLLMENT)) ⋈ COURSE )
3. Push projections, keeping join attributes:
   π_{title}( ( π_{student_id}(σ_{dept='CSE'}(STUDENT))
                ⋈ π_{student_id,course_id}(σ_{grade='A'}(ENROLLMENT)) )
              ⋈ π_{course_id,title}(COURSE) )
```

The first join now combines only CSE students with only A-grade enrollments, both narrowed to the columns actually needed.

Problem 26.3 — Estimate the size of `enrollment ⋈ course` on `course_id`, with 100,000 enrollments, 500 courses, and every course appearing in enrollment.

Solution. $|E \bowtie C| \approx (100{,}000 \times 500) / \max(500, 500) =$ **100,000**. Because `course_id` is the key of `course` and a foreign key in `enrollment`, every enrollment matches exactly one course, so the result has as many rows as `enrollment` — which the formula confirms.

Problem 26.4 — How many join orders exist for 5 relations using left-deep trees only, and how many join trees in total?

Solution. Left-deep orders = 5! = **120**. All join trees = (2 × 4)! / 4! = 40,320 / 24 = **1,680**. Restricting to left-deep trees cuts the space by a factor of 14 while still including the plans best suited to pipelining.

Problem 26.5 — A query that took 50 ms now takes 40 minutes after a bulk load of 5 million rows. Diagnose.

Solution. The most likely cause is **stale statistics**. The optimizer still believes the table is small, so it chooses a nested-loop plan that was right for 10,000 rows and is catastrophic for 5 million. `EXPLAIN ANALYZE` would show estimated rows in the thousands and actual rows in the millions. Fix: run `ANALYZE` on the loaded tables so the optimizer re-plans, typically switching to a hash join. As a process fix, add a statistics refresh as the final step of every bulk-load job.

Problem 26.6 — Why do optimizers keep plans with "interesting orders" rather than only the cheapest plan for each subset?

Solution. A plan for a subset of relations may produce output already sorted on a column needed later — for a merge join with the next relation, or for `GROUP BY` or `ORDER BY`. That plan might cost slightly more than the cheapest unsorted plan for the same subset, but it avoids a later sort that would cost more than the difference. If the optimizer discarded everything but the cheapest plan per subset, it would lose these globally better options. Keeping one best plan per interesting order preserves them at modest extra search cost.

Review Questions

1. Why is query optimization necessary? Give an order-of-magnitude illustration.
2. State six equivalence rules and explain which heuristics rely on them.

3. List the heuristic optimization steps and apply them to an example.
4. What statistics does the catalog keep, and what is each used for?
5. Give the selectivity formulas for equality, range, conjunction and disjunction.
6. Derive the join-size estimate and apply it to a key–foreign key join.
7. Explain dynamic programming for join ordering and why left-deep trees are preferred.
8. What are interesting orders?
9. Give four reasons cardinality estimates go wrong and their remedies.
10. How is an `EXPLAIN` plan read, and what does a large estimated-versus-actual gap indicate?

Multiple Choice Questions

1. The most important heuristic is to (a) push selections down (b) push joins down (c) compute products first (d) sort early.
2. Selectivity of `A = v` is estimated as (a) V(A,R) (b) 1/V(A,R) (c) n_R (d) 1/n_R.
3. Left-deep join orders for 4 relations number (a) 12 (b) 24 (c) 120 (d) 4.
4. Stale statistics are refreshed with (a) `OPTIMIZE` (b) `ANALYZE` (c) `VACUUM FULL` (d) `REINDEX`.
5. The independence assumption fails for (a) keys (b) correlated columns (c) indexed columns (d) sorted tables.
6. System R used (a) genetic search (b) dynamic programming (c) random plans (d) no optimization.
7. In MySQL `EXPLAIN`, type `ALL` means (a) index lookup (b) full table scan (c) constant (d) range.
8. $|R \bowtie S|$ for a key–foreign key join equals approximately (a) $n_R \times n_S$ (b) n of the foreign-key side (c) 1 (d) V(A,R).
9. Pushing a projection down must retain (a) all columns (b) join attributes (c) indexes (d) aggregates.
10. Optimizer hints should be used (a) always (b) first (c) as a last resort (d) never.

Answers: 1-(a) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(c)

Short Questions

1. What is heuristic optimization?
2. State the rule for pushing σ into a join.
3. What does V(A, R) denote?
4. Why use histograms?
5. What is a left-deep tree?
6. Give the join-size estimation formula.
7. What is an interesting order?
8. What does `EXPLAIN ANALYZE` add to `EXPLAIN`?
9. Why do estimation errors compound?
10. Name one danger of optimizer hints.

Long / Descriptive Questions

1. Explain heuristic optimization with the equivalence rules and a complete worked transformation.
2. Explain cost-based optimization: statistics, selectivity estimation, join size estimation and plan enumeration.
3. Discuss join ordering strategies, including dynamic programming, left-deep trees and interesting orders.
4. Analyse the causes of poor cardinality estimates and describe how to diagnose and fix them.

5. Show how to read and interpret execution plans in MySQL and PostgreSQL using a multi-table query.

Practical Exercises

1. Write the same query three different ways (join, subquery, `EXISTS`) and compare their execution plans.
2. Bulk-load 100,000 rows into a table, run a join before and after `ANALYZE`, and compare plans and timings.
3. Use `EXPLAIN ANALYZE` in PostgreSQL to find a query where estimated and actual rows differ by more than 10×, and explain why.
4. Demonstrate the effect of a non-sargable predicate on the chosen plan, then rewrite it.
5. Create correlated columns, show the resulting misestimate, and correct it with extended statistics in PostgreSQL (`CREATE STATISTICS`).

CHAPTER 27

Transactions

A **transaction** is a sequence of database operations that forms a single logical unit of work: it either happens completely or not at all, and it leaves the database in a consistent state. Transactions are the mechanism that lets thousands of users update shared data at once, and lets the database survive crashes, without ever exposing a half-finished change.

27.1 The Canonical Example

Transfer 1,000 taka from account A to account B.

```
T:  read(A)
    A := A − 1000
    write(A)
    read(B)
    B := B + 1000
    write(B)
```

```sql
START TRANSACTION;
UPDATE account SET balance = balance - 1000 WHERE acc_no = 'A';
UPDATE account SET balance = balance + 1000 WHERE acc_no = 'B';
COMMIT;
```

Two things could go wrong. The system could crash between `write(A)` and `write(B)`, leaving 1,000 taka destroyed. Or another transaction could read A and B in the middle, seeing a total 1,000 lower than it really is. The ACID properties exist to rule out both.

27.2 The ACID Properties

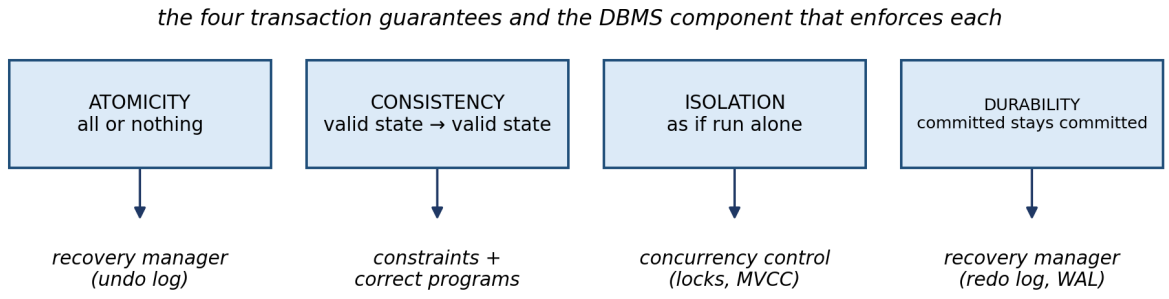


Figure 27.1: The ACID properties and the components that enforce them

Atomicity

All operations of a transaction are reflected in the database, or none are. If the transfer fails after debiting A, the debit must be undone.

Enforced by the recovery manager, using the **undo** information in the log (Chapter 32).

Consistency

A transaction executed alone takes the database from one consistent state to another. In the transfer, `A + B` is the same before and after.

Enforced by the declared integrity constraints *and* by correct application logic. The DBMS cannot know that a transfer should preserve the sum; the programmer must write a transaction that does. Consistency is therefore partly the programmer's responsibility — the only one of the four that is.

Isolation

Concurrent transactions do not see each other's intermediate states; the result is as if they had run one at a time in some order.

Enforced by the concurrency control subsystem: locking, timestamps or multiversion schemes (Chapters 28–30).

Durability

Once a transaction commits, its changes survive any subsequent failure, including a power cut one microsecond later.

Enforced by the recovery manager, which forces the log to stable storage before acknowledging the commit and uses **redo** information after a crash.

Property	Question it answers	Violated when	Enforced by
Atomicity	Did it all happen?	Half a transfer is applied	Undo logging
Consistency	Is the result valid?	A constraint or invariant breaks	Constraints + programmer
Isolation	Did others interfere?	A transaction sees another's partial work	Locks / MVCC
Durability	Will it last?	A committed change is lost in a crash	Redo logging, WAL

27.3 Transaction States

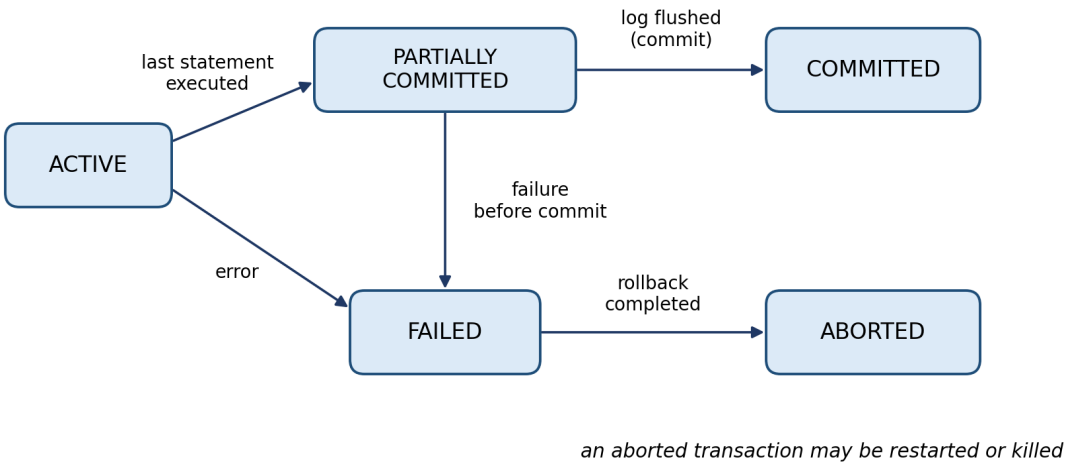


Figure 27.2: The transaction state diagram

State	Meaning
Active	Executing its operations
Partially committed	The last operation has run, but changes may still be only in memory
Committed	The commit log record is on stable storage; the changes are durable

State	Meaning
Failed	An error means normal execution cannot continue
Aborted	All changes have been rolled back; the database is as it was before the transaction began

The distinction between **partially committed** and **committed** is exactly the durability boundary. A crash while partially committed means the transaction must be rolled back, because its commit record may not have reached the log. After an abort, the system can **restart** the transaction if the failure was transient (a deadlock, a timeout) or **kill** it if the failure was a logical error in the program.

27.4 Schedules

A **schedule** is the chronological order in which the operations of several concurrent transactions are executed. It preserves the order of operations within each transaction.

Serial schedule — transactions run one after another with no interleaving. Always correct, because each transaction starts from a consistent state.

```
    Serial S1:  T1: r(A) w(A) r(B) w(B)  then  T2: r(A) w(A) r(B) w(B)
```

Concurrent (interleaved) schedule — operations of different transactions alternate.

```
    Concurrent S2:  T1: r(A) w(A)   T2: r(A) w(A)   T1: r(B) w(B)   T2: r(B) w(B)
```

S2 may or may not be correct. Chapter 29 gives the precise test: a concurrent schedule is acceptable if it is **serializable** — equivalent in effect to some serial schedule.

27.5 Why Allow Concurrency at All?

If serial schedules are always correct, why not run everything serially?

1. **Throughput.** While one transaction waits for a disk read, the CPU and other disks sit idle. Interleaving keeps every resource busy; a system that processes one transaction at a time may be ten or a hundred times slower.
2. **Response time.** A short query should not wait behind a long report. With interleaving, short transactions finish quickly even while long ones run.
3. **Utilization.** Modern servers have dozens of cores. Serial execution would use one of them.

The whole of concurrency control is the engineering required to get these benefits while keeping the result identical to some serial order.

27.6 Transactions in SQL

```sql
-- MySQL: autocommit is ON by default; each statement is its own transaction
SET autocommit = 0;                     -- or use START TRANSACTION explicitly

START TRANSACTION;
UPDATE enrollment SET grade = 'A' WHERE enrollment_id = 7;
UPDATE student SET cgpa = 3.80 WHERE student_id = 'ASH1901004';
COMMIT;

-- choosing an isolation level for the next transaction
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

A transaction ends in one of four ways: an explicit `COMMIT`, an explicit `ROLLBACK`, an implicit commit caused by a DDL statement, or an abort by the system (deadlock victim, crash, constraint violation with error handling).

Chapter Summary

- A transaction is a logical unit of work that either completes entirely or has no effect.
- Atomicity, consistency, isolation and durability are enforced by undo logging, constraints plus correct programs, concurrency control, and redo logging with write-ahead logging, respectively.
- Consistency is the only ACID property that depends partly on the programmer.
- Transactions move through active, partially committed, committed, failed and aborted states; commit occurs when the commit record reaches stable storage.
- A schedule is an interleaving of transactions' operations; serial schedules are always correct, and concurrent schedules must be serializable.
- Concurrency is permitted because it multiplies throughput and improves response time.

Key Terms

Transaction · ACID · Atomicity · Consistency · Isolation · Durability · Commit · Rollback · Abort · Transaction states · Active · Partially committed · Committed · Failed · Aborted · Restart · Kill · Schedule · Serial schedule · Concurrent schedule · Interleaving · Autocommit

Common Mistakes

1. Claiming the DBMS alone guarantees consistency; application logic is also responsible.
2. Treating "partially committed" as durable.
3. Confusing atomicity (all or nothing) with isolation (no interference).
4. Assuming a serial schedule is required for correctness; serializability is enough.
5. Forgetting that MySQL runs in autocommit mode by default, so multi-statement work needs an explicit transaction.
6. Forgetting that DDL causes an implicit commit.

Solved Problems

Problem 27.1 — For the transfer transaction, state which ACID property is violated in each case: (a) a crash after `write(A)` leaves A debited and B unchanged; (b) the program credits B with 100 instead of 1,000; (c) another transaction reads A after the debit and B before the credit; (d) the server loses power a moment after reporting success, and the change is gone after restart.

Solution. (a) **Atomicity** — only part of the transaction took effect. (b) **Consistency** — the invariant A + B = constant is broken, by a programming error the DBMS cannot detect. (c) **Isolation** — an intermediate state was visible to another transaction. (d) **Durability** — a committed change was lost.

Problem 27.2 — Two transactions each add 10 % interest to account A (initially 1,000). Show a serial schedule and give its result.

Solution. T1: `read(A)=1000; A:=1100; write(A)`. Then T2: `read(A)=1100; A:=1210; write(A)`. Final value **1,210**. The alternative serial order gives the same result here, since both apply the same operation; in general different serial orders may give different results, and serializability accepts any of them.

Problem 27.3 — Explain why a transaction in the partially committed state must still be rolled back after a crash.

Solution. “Partially committed” means the transaction has executed its last statement, but its commit record may not yet be on stable storage and its data changes may be only in buffer pages in memory. If the system crashes at that moment, there is no durable evidence that the transaction committed. The recovery manager must therefore treat it as incomplete and undo it — otherwise atomicity would be violated for a transaction that never officially committed. Only when the commit record is on disk does the transaction become committed and durable.

Problem 27.4 — Why might a DBMS abort a transaction that contains no programming error? Give three reasons, and state whether each should be restarted.

Solution. (i) It is chosen as the **victim of a deadlock** — restart, since retrying will usually succeed. (ii) It is aborted by a **timestamp or validation protocol** because it conflicts with a younger or older transaction — restart. (iii) A **system crash** occurs while it is active — recovery undoes it; the application should resubmit it. By contrast, a transaction aborted for violating an integrity constraint because of bad input should be **killed**, not restarted, since it would fail again.

Problem 27.5 — Why is concurrency worth its complexity? Answer with a numerical illustration.

Solution. Suppose each transaction spends 1 ms on CPU work and 9 ms waiting for disk I/O. Run serially, the CPU is busy 10 % of the time and the system completes 100 transactions per second. With ten transactions interleaved, while one waits for its disk read the CPU serves others, so in principle up to ten times as many complete per second, limited only by disk and CPU capacity. A short query arriving behind a 30-second report also no longer waits 30 seconds. Concurrency control is the price of that throughput and responsiveness.

Review Questions

1. Define a transaction and give a real-world example.
2. Explain each ACID property with an example of its violation.
3. Which DBMS component enforces each ACID property?
4. Why is consistency partly the programmer’s responsibility?
5. Draw and explain the transaction state diagram.
6. Distinguish partially committed from committed.
7. Define schedule, serial schedule and concurrent schedule.
8. Why is concurrent execution desirable?
9. In what four ways can a transaction end?
10. What is autocommit, and why does it matter?

Multiple Choice Questions

1. “All or nothing” is (a) consistency (b) atomicity (c) isolation (d) durability.
2. Which ACID property depends partly on application logic? (a) Atomicity (b) Consistency (c) Isolation (d) Durability.
3. Durability is achieved through (a) locks (b) the log and stable storage (c) views (d) indexes.
4. A transaction whose last statement has run but whose commit record is not yet on disk is (a) active (b) partially committed (c) committed (d) aborted.
5. Isolation is enforced by (a) the optimizer (b) concurrency control (c) the parser (d) RAID.
6. A serial schedule is (a) always correct (b) never correct (c) always fastest (d) always deadlocked.
7. After an abort caused by deadlock, the transaction should usually be (a) killed (b) restarted (c) committed (d) ignored.
8. Concurrency primarily improves (a) consistency (b) throughput and response time (c) durability

(d) normalization.

9. In MySQL, autocommit is by default (a) off (b) on (c) unavailable (d) per table.
10. A `CREATE TABLE` inside a MySQL transaction causes (a) rollback (b) implicit commit (c) deadlock (d) nothing.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(b) 6-(a) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. Expand ACID.
2. What does atomicity guarantee?
3. What does durability guarantee?
4. What is a schedule?
5. Name the five transaction states.
6. When exactly is a transaction committed?
7. Why can a serial schedule be inefficient?
8. Give one reason to restart an aborted transaction.
9. What is interleaving?
10. How do you disable autocommit in MySQL?

Long / Descriptive Questions

1. Explain the ACID properties in detail with the funds-transfer example, and identify the component enforcing each.
2. Draw and explain the transaction state diagram, including all transitions and their causes.
3. Explain schedules, distinguishing serial and concurrent execution, and justify why concurrency is allowed.
4. Discuss the responsibilities of the DBMS and of the programmer in maintaining consistency.
5. Explain how transactions are controlled in SQL, including autocommit, isolation levels and implicit commits.

Practical Exercises

1. Write the funds-transfer transaction in SQL, simulate a failure between the two updates with `ROLLBACK`, and verify that balances are unchanged.
2. Demonstrate autocommit behaviour in MySQL by running two updates with autocommit on and then off.
3. Show that a `CREATE TABLE` statement implicitly commits a pending transaction in MySQL but not in PostgreSQL.
4. Open two sessions and observe that uncommitted changes in one are invisible to the other under the default isolation level.
5. Write a stored procedure that performs a transfer with an error handler that rolls back on any failure.

CHAPTER 28

Concurrency Control

Concurrency control is the part of the DBMS that lets transactions run simultaneously while making the result indistinguishable from running them one at a time. This chapter catalogues what goes wrong without it, and how SQL isolation levels expose the trade-off between correctness and speed.

28.1 The Lost Update Problem

Two transactions read the same item, each modifies its own copy, and the second write overwrites the first.

time	T1 (deposit 1000)	T2 (withdraw 500)	balance on disk
t1	`read(A) = 5000`		5000
t2		`read(A) = 5000`	5000
t3	`A := A + 1000`		5000
t4		`A := A − 500`	5000
t5	`write(A) = 6000`		6000
t6		`write(A) = 4500`	4500 ← T1 lost

correct final value is 5500; T2 overwrote T1's update because it read a stale value

Figure 28.1: A lost update

A deposit of 1,000 and a withdrawal of 500 against a balance of 5,000 should leave 5,500. The interleaving above leaves 4,500: the deposit has vanished. Neither transaction did anything wrong in isolation; the fault is entirely in the interleaving.

Real-life form: two clerks booking the last seat on a flight both see it as free, and both sell it.

28.2 The Dirty Read Problem (Uncommitted Dependency)

A transaction reads a value written by another transaction that has not yet committed — and later rolls back.

time	T1	T2	note
t1	write(A) = 9000		uncommitted
t2		read(A) = 9000	dirty read
t3		commit	T2 acted on 9000
t4	ROLLBACK		A reverts to 5000

T2 has committed a decision based on a value that never officially existed

Figure 28.2: A dirty read

T2 has made a decision — perhaps approved a loan against a balance of 9,000 — using a value that was rolled back and, officially, never existed.

Real-life form: a stock-level report counts goods from an order that is then cancelled.

28.3 The Non-Repeatable Read Problem

A transaction reads the same item twice and gets two different values, because another transaction updated and committed it in between.

time	T1	T2
t1	read(A) = 5000	
t2		write(A) = 7000; commit
t3	read(A) = 7000	

T1 has seen two different "truths" within one transaction. If T1 was computing a report that uses A twice, the report is internally inconsistent.

28.4 The Phantom Read Problem

A transaction re-runs a query with a condition, and the *set* of rows changes because another transaction inserted or deleted a matching row.

```
-- T1
SELECT COUNT(*) FROM enrollment WHERE course_id = 1;    -- returns 5
-- T2 inserts a new enrollment for course 1 and commits
SELECT COUNT(*) FROM enrollment WHERE course_id = 1;    -- returns 6
```

The difference from a non-repeatable read is subtle but examined often: a non-repeatable read concerns an **existing row whose value changed**; a phantom concerns a **row that appeared or disappeared**. Locking the rows T1 read does not prevent phantoms, because the new row did not exist to be locked — preventing them requires locking the *range* or predicate (Chapter 30).

28.5 The Incorrect Summary Problem

One transaction computes an aggregate while another updates some of the items being summed.

time	T1 (sum balances)	T2 (transfer 1000 A → B)
t1	read(A) = 5000; sum = 5000	
t2		A := 4000; write(A)
t3		B := 4000; write(B)
t4	read(B) = 4000; sum = 9000	

The true total was always 8,000. T1 counted the 1,000 twice — once in A before the transfer, once in B after it.

Problem	What goes wrong	Caused by
Lost update	One write overwrites another	Write–write conflict on stale data
Dirty read	Reading uncommitted data later rolled back	Write–read conflict with uncommitted writer
Non-repeatable read	Same row read twice gives different values	Read–write conflict, writer commits in between
Phantom read	Same query returns a different set of rows	Insert or delete matching a predicate
Incorrect summary	Aggregate mixes before and after states	Reads interleaved with a multi-item update

28.6 SQL Isolation Levels

SQL-92 defines four isolation levels by which of these problems they permit.

Isolation level	Dirty read	Non-repeatable read	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible (in the standard)
SERIALIZABLE	Prevented	Prevented	Prevented

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;          -- next transaction only
SELECT @@transaction_isolation;                        -- MySQL 8
SHOW transaction_isolation;                            -- PostgreSQL
```

Vendor defaults and behaviour

DBMS	Default level	Notes
MySQL / InnoDB	REPEATABLE READ	Uses MVCC snapshots plus next-key locks, so phantoms are prevented for locking reads in most cases
PostgreSQL	READ COMMITTED	REPEATABLE READ is true snapshot isolation; SERIALIZABLE uses Serializable Snapshot Isolation
Oracle	READ COMMITTED	Offers READ COMMITTED and SERIALIZABLE only
SQL Server	READ COMMITTED	Optional snapshot isolation via row versioning

The trade-off

Higher isolation means fewer anomalies but more blocking, more aborts and lower throughput. Lower isolation means more concurrency but places the burden of correctness on the application.

Choose	When
READ COMMITTED	Most OLTP work where each statement needs committed data but small drifts between statements are acceptable
REPEATABLE READ	Reports and multi-step reads that must see a stable snapshot
SERIALIZABLE	Financial logic, inventory, seat booking — anything where a write depends on a condition read earlier
READ UNCOMMITTED	Rough monitoring queries only; almost never appropriate

28.7 Preventing the Lost Update in Practice

Even under READ COMMITTED, the classic read-modify-write pattern loses updates:

```
-- UNSAFE: two sessions can both read 5000
SELECT balance FROM account WHERE acc_no = 'A';        -- application computes new value
UPDATE account SET balance = 6000 WHERE acc_no = 'A';
```

Three correct alternatives:

```
-- 1. Let the database do the arithmetic atomically
UPDATE account SET balance = balance + 1000 WHERE acc_no = 'A';

-- 2. Pessimistic locking: lock the row while reading it
START TRANSACTION;
SELECT balance FROM account WHERE acc_no = 'A' FOR UPDATE;
UPDATE account SET balance = balance + 1000 WHERE acc_no = 'A';
COMMIT;

-- 3. Optimistic locking: detect a concurrent change with a version column
UPDATE account
SET    balance = 6000, version = version + 1
WHERE  acc_no = 'A' AND version = 17;       -- 0 rows affected => someone else changed
    ↪ it; retry
```

Approach	Blocking	Best when
Atomic `UPDATE … SET x = x + …`	Minimal	The new value is a simple function of the old
Pessimistic `SELECT … FOR UPDATE`	Holds a lock	Conflicts are frequent
Optimistic version check	None; retry on conflict	Conflicts are rare, e.g. web forms edited over minutes

28.8 Families of Concurrency Control

Family	Idea	Chapter
Lock-based	Transactions acquire locks before access; conflicting requests wait	30
Timestamp-based	Transactions are ordered by start time; out-of-order operations are rejected	30
Optimistic (validation)	Run freely, validate at commit, abort on conflict	30
Multiversion (MVCC)	Writers create new versions; readers see a consistent snapshot	30

Chapter Summary

- Uncontrolled interleaving causes lost updates, dirty reads, non-repeatable reads, phantoms and incorrect summaries.
- A non-repeatable read concerns a changed row; a phantom concerns an inserted or deleted row, and cannot be prevented by row locks alone.
- SQL defines four isolation levels, each preventing more anomalies at the cost of concurrency.
- MySQL defaults to REPEATABLE READ; PostgreSQL, Oracle and SQL Server default to READ COMMITTED.
- Lost updates in application code are prevented with atomic updates, `SELECT … FOR UPDATE`, or optimistic version checks.
- Concurrency control is implemented by locking, timestamps, validation or multiversioning.

Key Terms

Concurrency control · Lost update · Dirty read · Uncommitted dependency · Non-repeatable read · Phantom read · Incorrect summary · Isolation level · READ UNCOMMITTED · READ COMMITTED · REPEATABLE READ · SERIALIZABLE · Snapshot isolation · Pessimistic locking · Optimistic locking · `SELECT … FOR UPDATE` · Version column · Read-modify-write

Common Mistakes

1. Confusing non-repeatable reads with phantoms.
2. Assuming every DBMS has the same default isolation level.
3. Believing READ COMMITTED prevents lost updates in a read-modify-write sequence.
4. Using SERIALIZABLE everywhere without considering its throughput cost.
5. Using READ UNCOMMITTED for any report whose numbers matter.
6. Implementing optimistic locking without checking the affected row count.

Solved Problems

Problem 28.1 — Identify the problem in each: (a) T1 reads a product's stock twice and sees 10 then 8; (b) T1 counts students in CSE twice and gets 40 then 41; (c) T1 reads a price written by T2, and T2 then rolls back; (d) two clerks each add a payment to the same balance and one payment disappears.

Solution. (a) **Non-repeatable read** — an existing row's value changed. (b) **Phantom** — a new matching row appeared. (c) **Dirty read**. (d) **Lost update**.

Problem 28.2 — Which isolation level is the minimum that prevents each of the anomalies in Problem 28.1?

Solution. (a) REPEATABLE READ. (b) SERIALIZABLE in the SQL standard (InnoDB's REPEATABLE READ also prevents it for locking reads). (c) READ COMMITTED. (d) The standard isolation levels do not by themselves prevent the application-level read-modify-write lost update under READ COMMITTED; use an atomic `UPDATE`, `SELECT … FOR UPDATE`, or SERIALIZABLE.

Problem 28.3 — Reproduce a lost update with two sessions and then fix it using `SELECT … FOR UPDATE`.

Solution.

```
-- Session 1                                -- Session 2
START TRANSACTION;                          START TRANSACTION;
SELECT balance FROM account                 SELECT balance FROM account
  WHERE acc_no='A';     -- 5000               WHERE acc_no='A';     -- 5000
```

```
UPDATE account SET balance=6000
  WHERE acc_no='A';
COMMIT;                                          UPDATE account SET balance=4500
                                                   WHERE acc_no='A';
                                                 COMMIT;                -- final 4500: lost
    ↪ update
```

Fix: both sessions use `SELECT balance … FOR UPDATE`. Session 2's `SELECT` now **blocks** until session 1 commits, then reads 6,000, computes 5,500 and writes it. The final balance is correct.

Problem 28.4 — Explain why locking the rows a query has read does not prevent a phantom.

Solution. A phantom is a row that did not exist when the first query ran. Row locks can only be placed on rows that exist, so the inserting transaction finds nothing locked and proceeds. To prevent phantoms the system must lock the *condition* — the range of index keys covering `course_id = 1`, or the gap where a new key could be inserted. InnoDB does exactly this with **next-key locks** (a row lock plus a lock on the gap before it); predicate locking and index-range locking are the general techniques (Chapter 30).

Problem 28.5 — A web form lets a user edit a student record; the user may take ten minutes before saving. Which concurrency technique is appropriate, and why not `SELECT … FOR UPDATE`?

Solution. **Optimistic locking** with a version column. Holding a database lock for ten minutes while a person thinks would block every other writer of that row — and if the user abandons the form, the lock may be held until a timeout. Instead, the form loads `version = 17` with the data; on save, `UPDATE … WHERE student_id = ? AND version = 17` succeeds only if nobody else has saved in the meantime. If zero rows are affected, the application tells the user the record changed and offers to reload. No lock is held while the user is idle.

Problem 28.6 — Show how the incorrect summary problem produces a wrong total, and name the isolation level that prevents it.

Solution. Using the schedule in Section 28.5, T1 reads A before the transfer (5,000) and B after it (4,000), reporting 9,000 when the true total is 8,000 at every instant. The fix is to give T1 a consistent snapshot — **REPEATABLE READ** under snapshot isolation (MySQL, PostgreSQL) ensures T1 sees A and B as of the same moment, so it reports 8,000. Under pure locking, SERIALIZABLE achieves the same by preventing T2 from writing items T1 has read until T1 finishes.

Review Questions

1. Describe the lost update problem with a schedule.
2. Explain the dirty read problem and its real-world consequences.
3. Distinguish non-repeatable reads from phantom reads.
4. Explain the incorrect summary problem.
5. List the four SQL isolation levels and the anomalies each permits.
6. State the default isolation levels of four major DBMSs.
7. Discuss the trade-off between isolation and throughput.
8. Give three ways to prevent a lost update in application code.
9. Compare pessimistic and optimistic locking.
10. Name the four families of concurrency control mechanisms.

Multiple Choice Questions

1. Reading data written by an uncommitted transaction is a (a) lost update (b) dirty read (c) phantom (d) deadlock.

2. A new row appearing in a repeated query is a (a) dirty read (b) phantom (c) non-repeatable read (d) lost update.
3. The lowest level preventing dirty reads is (a) READ UNCOMMITTED (b) READ COMMITTED (c) REPEATABLE READ (d) SERIALIZABLE.
4. MySQL InnoDB's default isolation level is (a) READ COMMITTED (b) REPEATABLE READ (c) SERIALIZABLE (d) READ UNCOMMITTED.
5. PostgreSQL's default isolation level is (a) READ COMMITTED (b) REPEATABLE READ (c) SERIALIZABLE (d) READ UNCOMMITTED.
6. `SELECT … FOR UPDATE` is an example of (a) optimistic (b) pessimistic locking (c) MVCC (d) timestamping.
7. Optimistic locking detects conflicts using (a) a lock table (b) a version column (c) a trigger (d) an index.
8. Which isolation level prevents all three standard anomalies? (a) READ COMMITTED (b) REPEATABLE READ (c) SERIALIZABLE (d) READ UNCOMMITTED.
9. Phantoms are prevented by locking (a) rows only (b) ranges or predicates (c) tables' metadata (d) indexes' roots only.
10. The atomic statement `UPDATE t SET x = x + 1` prevents (a) phantoms (b) lost updates for that change (c) dirty reads (d) deadlocks.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(a) 6-(b) 7-(b) 8-(c) 9-(b) 10-(b)

Short Questions

1. Define a lost update.
2. Define a dirty read.
3. What is a phantom?
4. Name the four isolation levels.
5. Which anomaly does READ COMMITTED still allow?
6. What is a version column used for?
7. Why not hold locks during user think-time?
8. What is a next-key lock?
9. Give Oracle's default isolation level.
10. What is the incorrect summary problem?

Long / Descriptive Questions

1. Explain all five concurrency problems with schedules and real-world examples.
2. Explain the SQL isolation levels, the anomalies each permits, and vendor-specific behaviour.
3. Discuss how to choose an isolation level for different kinds of workload.
4. Compare atomic updates, pessimistic locking and optimistic locking for preventing lost updates, with SQL.
5. Explain why phantoms need range or predicate locking, with an example.

Practical Exercises

1. Using two sessions, reproduce a dirty read under READ UNCOMMITTED and show that READ COMMITTED prevents it.
2. Reproduce a non-repeatable read under READ COMMITTED and show that REPEATABLE READ prevents it.

3. Reproduce a phantom and determine which isolation level prevents it in your DBMS.
4. Reproduce a lost update and fix it three different ways.
5. Implement optimistic locking for a student-edit form, including retry handling when the row count is zero.

CHAPTER 29

Serializability

Chapter 27 stated that a concurrent schedule is acceptable if it is equivalent to some serial schedule. This chapter makes "equivalent" precise, gives an algorithm for testing it, and connects it to recoverability.

29.1 Notation

Schedules are written with `r_i(X)` for "transaction T_i reads X", `w_i(X)` for "T_i writes X", and `c_i`, `a_i` for commit and abort.

```
    S:  r1(A) w1(A) r2(A) w2(A) r1(B) w1(B) c1 r2(B) w2(B) c2
```

29.2 Conflicting Operations

Two operations **conflict** if all three conditions hold:

1. They belong to **different** transactions.
2. They access the **same** data item.
3. **At least one** of them is a write.

Pair	Conflict?	Reason
r1(A), r2(A)	No	Both reads; order is irrelevant
r1(A), w2(A)	Yes	Read–write
w1(A), r2(A)	Yes	Write–read
w1(A), w2(A)	Yes	Write–write
w1(A), w2(B)	No	Different items
r1(A), w1(A)	No	Same transaction

The significance: swapping two adjacent **non-conflicting** operations cannot change the result, whereas swapping a conflicting pair can.

29.3 Conflict Serializability

Two schedules are **conflict equivalent** if one can be transformed into the other by a sequence of swaps of adjacent non-conflicting operations — equivalently, if every pair of conflicting operations appears in the same order in both.

A schedule is **conflict serializable** if it is conflict equivalent to some serial schedule.

Example by swapping.

```
S:   r1(A) w1(A) r2(A) w2(A) r1(B) w1(B) r2(B) w2(B)

swap r2(A)w2(A) with r1(B)w1(B)  — different items, no conflict:
S':  r1(A) w1(A) r1(B) w1(B) r2(A) w2(A) r2(B) w2(B)   = serial  T1, T2
```

S is conflict serializable.

29.4 Testing with a Precedence Graph

Swapping by hand is impractical for long schedules. The standard test constructs a **precedence graph** (serialization graph):

```
Algorithm PRECEDENCE-GRAPH TEST
  1. Create one node for each transaction.
  2. For each pair of conflicting operations  op_i(X) … op_j(X)  with op_i earlier,
     add an edge  T_i → T_j.
  3. If the graph has a cycle, S is NOT conflict serializable.
     If it is acyclic, S IS conflict serializable, and any topological order
     of the graph is an equivalent serial schedule.
```

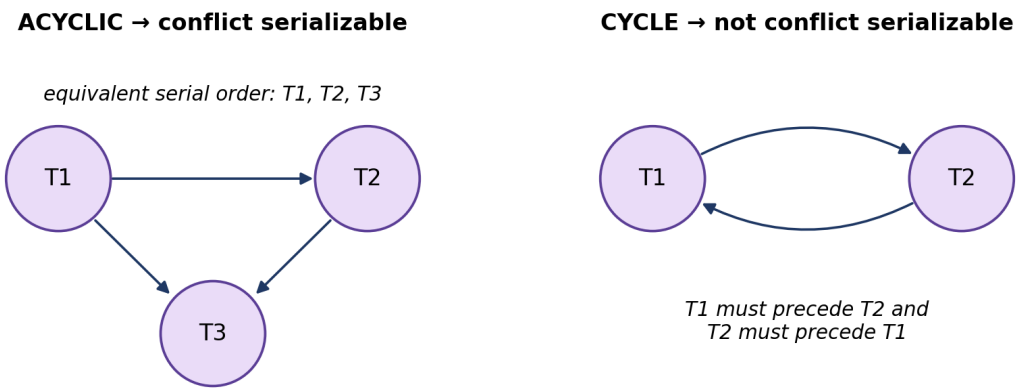


Figure 29.1: An acyclic and a cyclic precedence graph

Worked example 1.

```
S1:  r1(A) r2(A) w1(A) w2(A)
```

Conflict	Order	Edge
r1(A) … w2(A)	T1 first	T1 → T2
r2(A) … w1(A)	T2 first	T2 → T1
w1(A) … w2(A)	T1 first	T1 → T2

Edges T1 → T2 and T2 → T1 form a cycle ⇒ **not conflict serializable**. This is exactly the lost-update schedule.

Worked example 2.

```
S2:  r1(A) w1(A) r2(A) r3(B) w2(A) w3(B) r1(B) w1(B)
```

Conflict	Edge
w1(A) … r2(A)	T1 → T2
w1(A) … w2(A)	T1 → T2
r3(B) … w1(B)	T3 → T1
w3(B) … r1(B)	T3 → T1
w3(B) … w1(B)	T3 → T1

Edges: T3 → T1 → T2. No cycle ⇒ **conflict serializable**, equivalent to the serial order **T3, T1, T2**.

29.5 View Serializability

Two schedules are **view equivalent** if, for every data item:

1. **Initial reads match** — if T_i reads the initial value of X in one schedule, it does so in the other.
2. **Read-from relationships match** — if T_i reads a value of X written by T_j in one, it does so in the other.
3. **Final writes match** — the transaction performing the last write of X is the same in both.

A schedule is **view serializable** if it is view equivalent to some serial schedule.

Every conflict-serializable schedule is view serializable, but not conversely. The difference arises only with **blind writes** — writes of an item not previously read by that transaction.

```
S3:  r1(A) w2(A) w1(A) w3(A)
```

Precedence graph: r1(A) → w2(A) gives T1 → T2; w2(A) → w1(A) gives T2 → T1. A cycle, so S3 is **not** conflict serializable. But compare with the serial schedule T1, T2, T3:

- Initial read of A: T1 in both. ✓
- No transaction reads a value written by another in either. ✓
- Final write of A: T3 in both. ✓

So S3 **is** view serializable. The writes by T1 and T2 are overwritten by T3's blind write, so their order does not matter to anyone.

Basis	Conflict serializability	View serializability
Definition	Conflicting pairs in same order as a serial schedule	Same reads-from and final writes as a serial schedule
Test	Precedence graph — polynomial time	NP-complete
Includes	Subset	Superset
Practical use	Basis of every real protocol	Theoretical

Because testing view serializability is NP-complete, real systems enforce conflict serializability — usually through locking protocols that guarantee it without ever building the graph.

29.6 Recoverability

Serializability is not sufficient; schedules must also behave correctly when transactions **abort**.

Recoverable schedules

A schedule is **recoverable** if, whenever T_j reads a value written by T_i, **T_i commits before T_j commits**.

```
Non-recoverable:  w1(A) r2(A) c2 a1
```

T2 read A from T1 and committed. T1 then aborted, so the value T2 used never existed — but T2 is committed and, by durability, cannot be undone. The database is irreparably inconsistent. Such schedules must never be allowed.

Cascading rollback and cascadeless schedules

```
w1(A) r2(A) r3(A) … a1
```

If T1 aborts, T2 and T3 — which read its uncommitted value — must also abort, and anything that read from them, and so on. This **cascading rollback** can undo a large amount of work.

A schedule is **cascadeless** (avoids cascading aborts) if every transaction reads only values written by **committed** transactions. This is exactly the guarantee that eliminates dirty reads.

Strict schedules

A schedule is **strict** if no transaction reads *or writes* an item until the last transaction that wrote it has committed or aborted. Strictness makes recovery simple: undoing a write means restoring the before-image, with no risk of clobbering another transaction's later write.

```
strict ⊂ cascadeless ⊂ recoverable
```

Property	Condition	Prevents
Recoverable	T_i commits before any T_j that read from it commits	Committing on data that is later rolled back
Cascadeless	Read only committed data	Cascading aborts
Strict	Neither read nor overwrite uncommitted data	Complex undo

Strict two-phase locking (Chapter 30) produces schedules that are both conflict serializable and strict — which is why it is the protocol most systems implement.

Chapter Summary

- Two operations conflict if they are in different transactions, access the same item, and at least one writes.
- A schedule is conflict serializable if its conflicting operations can be ordered as in some serial schedule.
- The precedence graph has an edge $T_i \to T_j$ for each conflict in which T_i acts first; the schedule is conflict serializable if and only if the graph is acyclic, and a topological order gives the equivalent serial schedule.
- View serializability is more permissive, admitting some schedules with blind writes, but is NP-complete to test.
- A schedule must also be recoverable; cascadeless schedules avoid chains of aborts; strict schedules make undo simple.
- Strict $\subset$ cascadeless $\subset$ recoverable.

Key Terms

Schedule · Conflicting operations · Read–write conflict · Write–write conflict · Conflict equivalence · Conflict serializability · Precedence graph · Serialization graph · Topological sort · View equivalence · View serializability · Blind write · Recoverable schedule · Cascading rollback · Cascadeless schedule · Strict schedule

Important Formulas / Notation

```
r_i(X), w_i(X), c_i, a_i      read, write, commit, abort by T_i
edge T_i → T_j                a conflicting op of T_i precedes one of T_j on the
 ↪ same item
conflict serializable  ⇔  precedence graph acyclic
conflict serializable  ⊂  view serializable
strict ⊂ cascadeless ⊂ recoverable
```

Common Mistakes

1. Adding edges for read–read pairs, which do not conflict.
2. Adding edges between operations of the same transaction.
3. Drawing the edge in the wrong direction — it points from the earlier operation's transaction.
4. Concluding a schedule is not view serializable because it is not conflict serializable.
5. Treating serializability as sufficient without checking recoverability.
6. Confusing cascadeless with strict: strict also forbids overwriting uncommitted data.

Solved Problems

Problem 29.1 — Test `S: r1(X) r2(X) w1(X) r3(X) w2(X)` for conflict serializability.

Solution. Conflicts on X in time order:

Pair	Edge
r1(X) … w2(X)	T1 → T2
r2(X) … w1(X)	T2 → T1
w1(X) … r3(X)	T1 → T3
w1(X) … w2(X)	T1 → T2
r3(X) … w2(X)	T3 → T2

T1 → T2 and T2 → T1 form a cycle ⇒ **not conflict serializable**.

Problem 29.2 — Test `S: r1(A) r2(B) w1(A) r3(A) w2(B) w3(A) r2(A)` and, if serializable, give the equivalent serial order.

Solution. Conflicts on A: w1(A) → r3(A) gives T1 → T3; w1(A) → w3(A) gives T1 → T3; r1(A) → w3(A) gives T1 → T3; w3(A) → r2(A) gives T3 → T2; w1(A) → r2(A) gives T1 → T2. On B: r2(B) and w2(B) belong to the same transaction, so no edge. Graph: T1 → T3 → T2, and T1 → T2. **Acyclic** ⇒ conflict serializable; topological order **T1, T3, T2**.

Problem 29.3 — Show that `S: r1(A) w2(A) w1(A) w3(A)` is view serializable but not conflict serializable.

Solution. Worked in Section 29.5: the precedence graph contains T1 → T2 and T2 → T1, so it is not conflict serializable. It is view equivalent to the serial schedule T1, T2, T3: T1 performs the initial read of A in both; there are no reads-from relationships between transactions in either; and T3 performs the final write in both. The key is that T1's and T2's writes are **blind** and immediately overwritten by T3's blind write, so their relative order is invisible.

Problem 29.4 — Classify each schedule as recoverable, cascadeless or strict: (a) `w1(A) r2(A) c1 c2`; (b) `w1(A) r2(A) c2 c1`; (c) `w1(A) c1 r2(A) c2`; (d) `w1(A) w2(A) c1 c2`.

Solution. (a) T2 read uncommitted data from T1, but T1 committed first ⇒ **recoverable**, not cascadeless. (b) T2 committed before T1, from which it read ⇒ **not recoverable**. (c) T2 read only after T1 committed ⇒ **strict** (and therefore cascadeless and recoverable). (d) No reads-from relationship, so recoverable and cascadeless; but T2 overwrote an item written by uncommitted T1 ⇒ **not strict**.

Problem 29.5 — Explain why real systems do not test schedules for view serializability.

Solution. Testing view serializability is NP-complete, so no efficient general algorithm is known. The extra schedules it admits over conflict serializability involve blind writes, which are uncommon in practice, so the gain in concurrency would be small. Moreover, real systems do not test schedules after the fact at all: they use a protocol — typically strict two-phase locking or serializable snapshot isolation — that makes

non-serializable schedules impossible in the first place. Conflict serializability has the right properties for such protocols.

Problem 29.6 — Construct a schedule for the funds transfer and a concurrent audit that is serializable, and one that is not.

Solution. Let T1 transfer (r1(A) w1(A) r1(B) w1(B)) and T2 sum (r2(A) r2(B)).

Serializable: `r2(A) r2(B) r1(A) w1(A) r1(B) w1(B)` — T2 runs entirely first. Or `r1(A) w1(A) r2(A) r1(B) w1(B) r2(B)`: edges w1(A) → r2(A) gives T1 → T2 and w1(B) → r2(B) gives T1 → T2; acyclic ⇒ equivalent to T1, T2.

Not serializable: `r1(A) w1(A) r2(A) r2(B) r1(B) w1(B)` — edges T1 → T2 (on A) and r2(B) → w1(B) gives T2 → T1 (on B). Cycle. This is precisely the incorrect summary problem of Section 28.5.

Review Questions

1. Define conflicting operations and list the three conditions.
2. Define conflict equivalence and conflict serializability.
3. Explain how to test conflict serializability by swapping.
4. Give the precedence graph algorithm and explain why a cycle means non-serializability.
5. Define view equivalence using its three conditions.
6. Show with an example that view serializability is broader than conflict serializability.
7. Why is conflict serializability used in practice?
8. Define recoverable, cascadeless and strict schedules.
9. What is a cascading rollback and why is it undesirable?
10. State the containment relationships among the schedule classes.

Multiple Choice Questions

1. `r1(A)` and `r2(A)` (a) conflict (b) do not conflict (c) always deadlock (d) are blind.
2. A schedule is conflict serializable iff its precedence graph is (a) connected (b) acyclic (c) complete (d) cyclic.
3. An edge T1 → T2 means (a) T2 acted first (b) T1's conflicting operation came first (c) T1 waits for T2 (d) both read.
4. Testing view serializability is (a) linear (b) polynomial (c) NP-complete (d) trivial.
5. A write of an item not previously read is a (a) dirty write (b) blind write (c) phantom (d) cascade.
6. Every conflict-serializable schedule is (a) view serializable (b) strict (c) serial (d) deadlock-free.
7. A schedule where T2 commits before T1, from which it read, is (a) strict (b) cascadeless (c) non-recoverable (d) serial.
8. Reading only committed data makes a schedule (a) cascadeless (b) non-recoverable (c) serial (d) blind.
9. Which is the smallest class? (a) Recoverable (b) Cascadeless (c) Strict (d) All equal.
10. An equivalent serial order is found by (a) topological sort (b) hashing (c) sorting by timestamp only (d) random choice.

Answers: 1-(b) 2-(b) 3-(b) 4-(c) 5-(b) 6-(a) 7-(c) 8-(a) 9-(c) 10-(a)

Short Questions

1. When do two operations conflict?

2. What is a precedence graph?
3. How is an equivalent serial schedule obtained?
4. What is a blind write?
5. State the three view-equivalence conditions.
6. Define recoverable schedule.
7. What is a cascading rollback?
8. Define strict schedule.
9. Is every view-serializable schedule conflict serializable?
10. Which protocol yields strict, conflict-serializable schedules?

Long / Descriptive Questions

1. Explain conflict serializability fully, with the swapping method and the precedence-graph algorithm applied to two schedules.
2. Explain view serializability, construct a schedule that is view but not conflict serializable, and discuss why view serializability is not used in practice.
3. Explain recoverability, cascadelessness and strictness with example schedules of each class.
4. Given five schedules, classify each by serializability and recoverability, justifying every decision.
5. Relate the concurrency problems of Chapter 28 to precedence-graph cycles.

Practical Exercises

1. For ten given schedules, draw precedence graphs and determine conflict serializability and equivalent serial orders.
2. Write a short program that takes a schedule as input, builds the precedence graph, detects cycles, and prints a serial order if one exists.
3. Construct three schedules that are view but not conflict serializable.
4. Construct one schedule each that is non-recoverable, recoverable but not cascadeless, cascadeless but not strict, and strict.
5. Using two database sessions, produce a schedule whose precedence graph has a cycle and observe how your DBMS prevents it under SERIALIZABLE.

CHAPTER 30

Locking Protocols

A locking protocol is a set of rules about when a transaction may acquire and release locks. The rules are chosen so that every schedule the protocol permits is serializable — and, ideally, recoverable and strict — without the system ever having to test a schedule.

30.1 Lock Modes

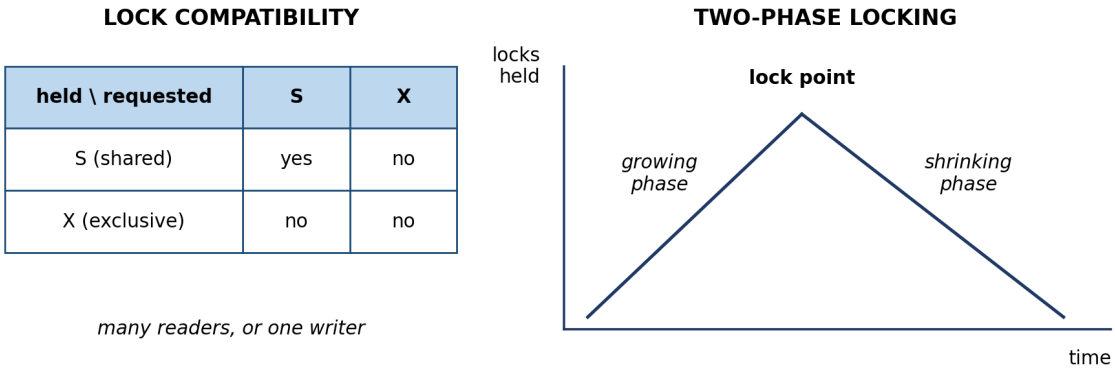


Figure 30.1: Lock compatibility, and the two phases of 2PL

Mode	Symbol	Permits	Also called
Shared	S	Read only	Read lock
Exclusive	X	Read and write	Write lock

The **compatibility matrix** records which requests can be granted while another lock is held:

Held \ Requested	S	X
S	Granted	Wait
X	Wait	Wait

Any number of transactions may share a read lock; a write lock excludes everyone. A request that is incompatible with a held lock waits until that lock is released.

Lock upgrade converts S to X (a transaction that read an item decides to write it); **lock downgrade** converts X to S.

30.2 Why Locks Alone Are Not Enough

Locking each item just long enough to access it does **not** guarantee serializability:

```
T1: lock-X(A) r(A) w(A) unlock(A)                    lock-X(B) r(B) w(B) unlock(B)
T2:                         lock-S(A) r(A) unlock(A) lock-S(B) r(B) unlock(B)
```

Every access was properly locked, yet T2 read A after the transfer and B before it — the incorrect summary

problem. The fault is that T1 **released a lock and then acquired another**. The two-phase locking protocol forbids exactly that.

30.3 Two-Phase Locking (2PL)

Rule. Each transaction acquires and releases locks in two phases:

1. **Growing phase** — the transaction may acquire locks but may not release any.
2. **Shrinking phase** — the transaction may release locks but may not acquire any.

The moment of the last acquisition is the transaction's **lock point**.

Theorem. Every schedule produced by 2PL is conflict serializable; an equivalent serial order is the order of the transactions' lock points.

Intuition. If T1 and T2 conflict on an item, the one that locks it first holds the lock until after its own lock point; the other must wait, and therefore reaches its lock point later. Every precedence edge points from an earlier lock point to a later one, so no cycle can form.

Limitations of basic 2PL

1. **Deadlock is possible.** T1 locks A and wants B; T2 locks B and wants A. Neither can proceed (Chapter 31).
2. **Cascading rollback is possible.** A transaction in its shrinking phase may release an X lock before committing; another reads the uncommitted value; if the first aborts, the second must too.

30.4 Strict and Rigorous 2PL

Strict 2PL. A transaction holds all its **exclusive** locks until it commits or aborts.

- Guarantees conflict serializability (it is still 2PL).
- Guarantees **strict** schedules: nobody can read or overwrite uncommitted data, so there are no cascading rollbacks and undo is simple.

Rigorous 2PL. A transaction holds **all** locks — shared and exclusive — until commit or abort.

- Serial order equals commit order, which is convenient for replication and distributed commit.

Protocol	Serializable	Cascadeless	Strict	Deadlock-free	Used by
Basic 2PL	Yes	No	No	No	Theory
Strict 2PL	Yes	Yes	Yes	No	Most lock-based DBMSs
Rigorous 2PL	Yes	Yes	Yes	No	Many commercial systems
Conservative 2PL	Yes	Depends	Depends	**Yes**	Rare in practice

Conservative (static) 2PL acquires every lock the transaction will need *before* it begins. It never deadlocks, but it requires knowing the full read and write set in advance, which is rarely possible.

30.5 Lock Granularity

Locks can be taken at different sizes:

```
database ⊃ table ⊃ page ⊃ row ⊃ field
```

Granularity	Concurrency	Lock overhead
Database / table	Very low	Very low
Page	Medium	Medium
Row	High	High (many locks)

Row locking maximizes concurrency for OLTP; table locking is efficient for bulk operations. Systems support both through **multiple-granularity locking** with **intention locks**:

Mode	Meaning
IS (intention shared)	Some descendant will be locked S
IX (intention exclusive)	Some descendant will be locked X
SIX	The whole node S, plus some descendant X

To lock a row in X mode, a transaction first takes IX on the table. Another transaction requesting X on the whole table sees the IX and waits — without having to inspect millions of row locks. This is how `LOCK TABLES` and row locking coexist in InnoDB.

Lock escalation automatically converts many row locks into one table lock when their number becomes excessive (SQL Server does this; InnoDB does not).

30.6 Timestamp-Ordering Protocol

Instead of locks, each transaction receives a unique **timestamp** TS(T) when it starts. The protocol forces conflicting operations to execute in timestamp order.

Each item X keeps:

- `W-TS(X)` — the largest timestamp of any transaction that wrote X.
- `R-TS(X)` — the largest timestamp of any transaction that read X.

```
When T issues read(X):
   if TS(T) < W-TS(X):  X was overwritten by a younger txn  →  ABORT T, restart
   else:                perform read;  R-TS(X) := max(R-TS(X), TS(T))

When T issues write(X):
   if TS(T) < R-TS(X):  a younger txn already read the old value  →  ABORT T
   if TS(T) < W-TS(X):  a younger txn already wrote X              →  ABORT T
   else:                perform write;  W-TS(X) := TS(T)
```

Thomas' write rule refines the last case: if TS(T) < W-TS(X), T's write is obsolete and can simply be **ignored** rather than causing an abort, because a younger transaction's value will overwrite it anyway. This admits some view-serializable schedules that are not conflict serializable.

Basis	2PL	Timestamp ordering
Conflicts resolved by	Waiting	Aborting and restarting
Deadlock	Possible	Impossible — nobody waits
Serial order	Lock-point order	Timestamp order
Starvation	Possible	Possible (long transactions repeatedly aborted)
Overhead	Lock table	Timestamps on every item

30.7 Optimistic (Validation-Based) Concurrency Control

Assumes conflicts are rare. Each transaction runs in three phases:

1. **Read phase** — execute, writing only to a private workspace.
2. **Validation phase** — check whether the transaction conflicts with any that committed during its read phase.
3. **Write phase** — if validation succeeds, copy private writes to the database; otherwise abort and restart.

Excellent for read-mostly workloads with little contention; wasteful when conflicts are frequent, because work is thrown away at validation. The application-level version is the version-column technique of Section 28.7.

30.8 Multiversion Concurrency Control (MVCC)

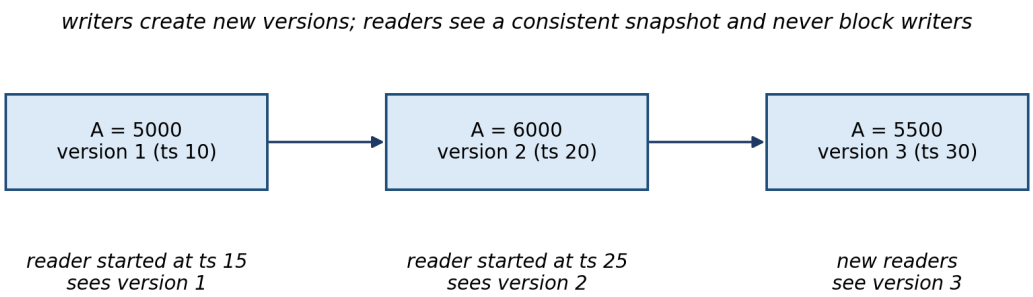


Figure 30.2: Multiversion concurrency control

A write creates a **new version** of the item instead of overwriting it. Each version is stamped with the transaction that created it. A reader sees the latest version that was committed **as of the reader's snapshot** — so readers never block writers, and writers never block readers.

```
    T_read starts at time 15.
    A has versions: v1 (committed at 10, value 5000), v2 (committed at 20, value 6000)
    T_read sees 5000 for its entire duration, even after v2 is committed.
```

Writers still conflict with writers; two transactions updating the same row are serialized by row locks (InnoDB, PostgreSQL) or by first-committer-wins rules.

Aspect	Consequence
Readers never wait	Reports and queries run without blocking OLTP updates
Consistent snapshot	Each transaction sees one coherent state — no non-repeatable reads
Old versions accumulate	Must be garbage-collected: PostgreSQL `VACUUM`, InnoDB purge of undo logs
Snapshot isolation is not fully serializable	Permits **write skew** unless extra checks are added

Write skew. Two doctors are on call; the rule is that at least one must remain. Each transaction checks "is the other still on call?", sees yes in its own snapshot, and removes itself. Both commit, and nobody is on call. Neither wrote what the other wrote, so no write–write conflict was detected. PostgreSQL's

SERIALIZABLE level adds **Serializable Snapshot Isolation (SSI)** to detect such dangerous read–write patterns and abort one transaction.

MVCC is the concurrency mechanism of PostgreSQL, MySQL InnoDB, Oracle and SQL Server's snapshot modes — the dominant approach in modern systems.

Chapter Summary

- Shared locks permit concurrent reads; exclusive locks permit one writer; incompatible requests wait.
- Locking items only while accessing them does not guarantee serializability.
- Two-phase locking — a growing phase then a shrinking phase — guarantees conflict serializability in lock-point order, but allows deadlock and cascading rollback.
- Strict 2PL holds exclusive locks until commit, producing strict, cascadeless schedules; it is the standard lock-based protocol.
- Multiple-granularity locking with intention locks lets row and table locks coexist efficiently.
- Timestamp ordering aborts out-of-order operations instead of waiting, so it cannot deadlock; Thomas' write rule ignores obsolete writes.
- Optimistic concurrency validates at commit time and suits low-contention workloads.
- MVCC keeps multiple versions so readers never block writers; it is used by nearly every modern DBMS, and snapshot isolation must be extended to prevent write skew.

Key Terms

Lock · Shared lock · Exclusive lock · Compatibility matrix · Lock upgrade · Two-phase locking · Growing phase · Shrinking phase · Lock point · Strict 2PL · Rigorous 2PL · Conservative 2PL · Lock granularity · Intention lock · IS · IX · SIX · Lock escalation · Timestamp ordering · R-TS · W-TS · Thomas' write rule · Optimistic concurrency control · Validation · MVCC · Snapshot · Snapshot isolation · Write skew · Serializable snapshot isolation · Vacuum / purge

Common Mistakes

1. Believing that locking every access guarantees serializability; the two-phase rule is essential.
2. Claiming 2PL prevents deadlock; only conservative 2PL does.
3. Confusing strict 2PL (hold X locks) with rigorous 2PL (hold all locks).
4. Forgetting that timestamp ordering resolves conflicts by abort, not by waiting.
5. Assuming snapshot isolation is fully serializable; it permits write skew.
6. Ignoring the need to vacuum or purge old versions under MVCC.

Solved Problems

Problem 30.1 — Is the following transaction two-phase? `lock-S(A) r(A) lock-X(B) r(B) w(B) unlock(A) unlock(B)`.

Solution. Yes. All acquisitions (S on A, X on B) occur before any release, so the growing phase ends at `lock-X(B)` — the lock point — and the shrinking phase contains only releases. It is also **rigorous** if both unlocks happen at commit; it is strict provided the X lock on B is released no earlier than commit.

Problem 30.2 — Show a deadlock under 2PL.

Solution.

time	T1	T2
t1	lock-X(A)	
t2		lock-X(B)
t3	request lock-X(B) — waits for T2	
t4		request lock-X(A) — waits for T1

Both transactions are in their growing phase and obey 2PL, yet each waits for the other forever. 2PL guarantees serializability, not freedom from deadlock.

Problem 30.3 — Apply timestamp ordering with TS(T1) = 5, TS(T2) = 10 and initially R-TS(A) = W-TS(A) = 0 to: `r2(A) w1(A)`.

Solution. `r2(A)`: TS(T2) = 10 ≥ W-TS(A) = 0, so the read proceeds and R-TS(A) becomes 10. `w1(A)`: TS(T1) = 5 < R-TS(A) = 10 — a younger transaction has already read the old value, so T1's write would invalidate that read. **T1 is aborted** and restarted with a new, larger timestamp.

Problem 30.4 — With TS(T1) = 5, TS(T2) = 10: `w2(A) w1(A)`. What happens under basic timestamp ordering and under Thomas' write rule?

Solution. After `w2(A)`, W-TS(A) = 10. `w1(A)` has TS 5 < W-TS 10. **Basic TO:** abort T1. **Thomas' write rule:** T1's write is obsolete — in the equivalent serial order (T1 then T2) it would be overwritten by T2 anyway — so it is simply **ignored**, and T1 continues. The resulting schedule is view serializable but not conflict serializable.

Problem 30.5 — Explain write skew with the on-call example and state how PostgreSQL prevents it.

Solution. Initially both Dr. A and Dr. B are on call. T1 (Dr. A going off call) reads `SELECT COUNT(*) FROM oncall WHERE on_call` = 2, so it is safe to leave; it updates Dr. A's row. T2 (Dr. B) takes its snapshot at the same time, also sees 2, and updates Dr. B's row. The two transactions wrote **different** rows, so no write–write conflict occurs and both commit under snapshot isolation. Result: nobody on call, violating the rule each transaction checked. PostgreSQL's **SERIALIZABLE** level uses Serializable Snapshot Isolation, which tracks read–write dependencies between concurrent transactions; on detecting the dangerous structure (each read something the other wrote) it aborts one with a serialization failure, which the application retries. Alternatively, the application can use `SELECT … FOR UPDATE` on the rows it checks.

Problem 30.6 — Why does MVCC let a long report run without blocking updates, and what does it cost?

Solution. Under MVCC, an update creates a new version rather than overwriting the old one. The report reads the versions committed as of its start, which remain available however many updates occur afterward. It therefore needs no read locks, and updaters never wait for it. The costs: (i) storage for old versions, which must be retained as long as any transaction might need them — a report running for hours keeps hours of versions alive; (ii) garbage collection work (`VACUUM` in PostgreSQL, purge in InnoDB) to reclaim them; (iii) extra work to find the correct visible version when many versions exist. A long-running transaction is therefore a common cause of table bloat in PostgreSQL.

Review Questions

1. Explain shared and exclusive locks with the compatibility matrix.
2. Show why locking each access individually does not guarantee serializability.
3. State the two-phase locking rule and prove that it guarantees conflict serializability.
4. What are the limitations of basic 2PL?
5. Compare basic, strict, rigorous and conservative 2PL.
6. Explain multiple-granularity locking and intention locks.

7. Describe the timestamp-ordering protocol and Thomas' write rule.
8. Explain optimistic concurrency control and when it is appropriate.
9. Explain MVCC, its advantages and its costs.
10. What is write skew, and how can it be prevented?

Multiple Choice Questions

1. S locks are compatible with (a) X locks (b) S locks (c) nothing (d) IX only.
2. In 2PL, a transaction may not acquire locks after (a) its first read (b) its first release (c) commit only (d) any write.
3. Strict 2PL holds (a) S locks until commit (b) X locks until commit (c) no locks (d) only table locks.
4. Which variant is deadlock-free? (a) Basic 2PL (b) Strict 2PL (c) Rigorous 2PL (d) Conservative 2PL.
5. Timestamp ordering resolves conflicts by (a) waiting (b) aborting (c) escalation (d) vacuuming.
6. Thomas' write rule (a) aborts obsolete writes (b) ignores obsolete writes (c) delays writes (d) upgrades locks.
7. An IX lock on a table indicates (a) the table is exclusively locked (b) some row will be X-locked (c) the table is read-only (d) nothing.
8. Under MVCC, readers (a) block writers (b) never block writers (c) always abort (d) take X locks.
9. Snapshot isolation can permit (a) dirty reads (b) write skew (c) lost updates on the same row (d) phantoms of committed data only.
10. The serial order equivalent to a 2PL schedule is given by (a) start order (b) lock-point order (c) alphabetical order (d) commit order always.

Answers: 1-(b) 2-(b) 3-(b) 4-(d) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What is a lock point?
2. Why can basic 2PL cause cascading rollbacks?
3. What is lock escalation?
4. Define W-TS(X).
5. Why can timestamp ordering not deadlock?
6. Name the three phases of optimistic concurrency control.
7. What does `VACUUM` do in PostgreSQL?
8. What is write skew?
9. Why are intention locks needed?
10. What is rigorous 2PL?

Long / Descriptive Questions

1. Explain two-phase locking and its variants in detail, with schedules showing serializability, deadlock and cascading rollback.
2. Explain multiple-granularity locking with intention locks and a compatibility matrix.
3. Explain the timestamp-ordering protocol with worked examples, including Thomas' write rule.
4. Explain MVCC and snapshot isolation, including write skew and serializable snapshot isolation.
5. Compare locking, timestamp ordering, optimistic control and MVCC for different workloads.

Practical Exercises

1. In two sessions, use `SELECT … FOR SHARE` and `SELECT … FOR UPDATE` to observe lock compatibility.
2. Examine current locks using `performance_schema.data_locks` (MySQL 8) or `pg_locks` (PostgreSQL) while a transaction holds row locks.
3. Demonstrate that a long-running read under MVCC does not block concurrent updates.
4. Reproduce write skew under REPEATABLE READ in PostgreSQL, then show that SERIALIZABLE prevents it.
5. Simulate the timestamp-ordering protocol by hand for a schedule of four transactions and record every abort.

CHAPTER 31

Deadlock

A **deadlock** is a state in which each transaction of a set is waiting for a resource held by another transaction in the same set, so none can ever proceed. Deadlock is the price of lock-based concurrency control; every lock-based DBMS must either prevent it or detect and break it.

31.1 Example

time	T1 (transfer A → B)	T2 (transfer B → A)
t1	lock-X(A)	
t2		lock-X(B)
t3	request lock-X(B) → waits for T2	
t4		request lock-X(A) → waits for T1

Neither can continue, and neither will ever release its lock voluntarily. Without intervention both wait forever.

```
-- Session 1                                  -- Session 2
START TRANSACTION;                            START TRANSACTION;
UPDATE account SET balance=balance-100        UPDATE account SET balance=balance-100
  WHERE acc_no='A';                             WHERE acc_no='B';
UPDATE account SET balance=balance+100        UPDATE account SET balance=balance+100
  WHERE acc_no='B';     -- waits                  WHERE acc_no='A';
                                              -- ERROR 1213 (40001): Deadlock found
    ↪ when
                                              -- trying to get lock; try restarting
    ↪ transaction
```

InnoDB detects the cycle within milliseconds and aborts one session, which must retry.

31.2 The Four Necessary Conditions (Coffman Conditions)

A deadlock can occur only if **all four** hold simultaneously:

Condition	Meaning	In a DBMS
Mutual exclusion	A resource can be held by only one transaction at a time	Exclusive locks
Hold and wait	A transaction holds resources while waiting for others	Growing phase of 2PL
No preemption	A resource cannot be forcibly taken from its holder	Locks are released only voluntarily
Circular wait	A cycle of transactions each waiting for the next	T1 → T2 → … → T1

Breaking any one condition prevents deadlock. Prevention schemes attack hold-and-wait, no-preemption or circular wait; mutual exclusion is essential to correctness and cannot be dropped.

31.3 Deadlock Prevention

Conservative 2PL (breaks hold-and-wait)

Acquire all required locks before starting; if any is unavailable, acquire none and wait. No transaction ever holds some locks while waiting for others. Impractical when the needed data is not known in advance.

Resource ordering (breaks circular wait)

Impose a total order on all data items and require every transaction to lock in that order. A cycle becomes impossible, because a cycle would require some transaction to request a lower-ordered item while holding a higher one.

In practice this is the most useful prevention technique for application developers: **always update tables and rows in a consistent order.** The transfer deadlock above disappears if both transactions lock the lower account number first.

```
-- both transfers lock the smaller acc_no first
SELECT * FROM account WHERE acc_no IN ('A','B') ORDER BY acc_no FOR UPDATE;
```

Timestamp-based schemes (break no-preemption selectively)

	older requests younger's lock	younger requests older's lock
WAIT-DIE (non-preemptive)	older WAITS	younger DIES (rolls back)
WOUND-WAIT (preemptive)	older WOUNDS younger (rolls it back)	younger WAITS

in both schemes the rolled-back transaction keeps its original timestamp, so it eventually becomes oldest

Figure 31.1: Wait-die and wound-wait

Each transaction receives a timestamp at start; a smaller timestamp means **older**.

Wait-die (non-preemptive). When T_i requests an item held by T_j:

- if T_i is **older** than T_j, T_i **waits**;
- if T_i is **younger**, T_i **dies** — it is rolled back and restarted later.

Wound-wait (preemptive). When T_i requests an item held by T_j:

- if T_i is **older**, it **wounds** T_j — T_j is rolled back and T_i takes the item;
- if T_i is **younger**, T_i **waits**.

In both schemes, the older transaction is never rolled back by a younger one, and waiting only ever happens in one direction of age, so no cycle can form. A restarted transaction keeps its **original timestamp**, so it grows relatively older and eventually cannot be rolled back — which prevents starvation.

Basis	Wait-die	Wound-wait
Older requests younger's lock	Older waits	Older preempts (younger rolled back)
Younger requests older's lock	Younger dies	Younger waits

Basis	Wait-die	Wound-wait
Preemptive	No	Yes
Who is rolled back	The requester (always younger)	The holder (always younger)
Typical number of rollbacks	Higher — a young transaction may die repeatedly	Lower

Timeout

A transaction that waits longer than a threshold is assumed to be deadlocked and is rolled back. Simple and cheap, but it may abort transactions that were merely slow, and it may wait uselessly long for genuine deadlocks. `innodb_lock_wait_timeout` (default 50 s) is this mechanism, used alongside detection.

31.4 Deadlock Detection

Instead of preventing deadlock, the system allows it and periodically checks for it — efficient when deadlocks are rare, which they usually are.

The wait-for graph

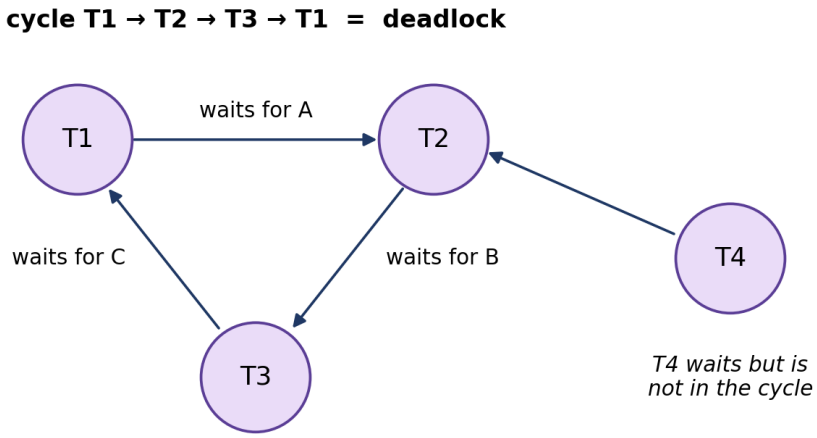


Figure 31.2: A wait-for graph containing a deadlock cycle

- One node per active transaction.
- An edge T_i → T_j whenever T_i is waiting for a lock held by T_j.

A deadlock exists if and only if the wait-for graph contains a cycle. Every transaction on the cycle is deadlocked; a transaction merely waiting *for* a cycle member (like T4 in Figure 31.2) is blocked but not part of the deadlock, and will proceed once the cycle is broken.

Detection runs either every time an edge is added (InnoDB, immediate detection) or periodically.

31.5 Recovery from Deadlock

1. **Choose a victim.** Criteria include: the transaction that has done the least work; the one holding the fewest locks; the one closest to its start; the one whose rollback breaks the most cycles; or the youngest. InnoDB rolls back the transaction that has modified the fewest rows.
2. **Roll back.** Either a **total rollback** (abort and restart) or a **partial rollback** to a savepoint just before the conflicting lock was requested.
3. **Avoid starvation.** If the same transaction is chosen repeatedly it may never finish; include the number of previous rollbacks in the victim-selection cost.

31.6 Starvation versus Deadlock

Basis	Deadlock	Starvation (livelock)
Definition	A set of transactions wait for each other forever	One transaction is repeatedly delayed or rolled back while others proceed
Progress	No transaction in the set progresses	The system progresses; one transaction does not
Cause	Circular wait	Unfair scheduling or victim selection
Remedy	Prevention or detection and rollback	Fair queuing, ageing, keeping original timestamps

31.7 Practical Guidance for Developers

1. Access tables and rows in a **consistent order** across the application.
2. Keep transactions **short**: no user interaction inside a transaction.
3. Use the **lowest isolation level** that is correct.
4. **Index** columns used in `WHERE` clauses of updates, so locking scans touch fewer rows.
5. Write application code that **retries** a transaction on a deadlock error (MySQL 1213, SQLSTATE 40001, PostgreSQL 40P01).
6. Inspect `SHOW ENGINE INNODB STATUS` (the LATEST DETECTED DEADLOCK section) or the PostgreSQL log to see which statements deadlocked.

Chapter Summary

- A deadlock is a cycle of transactions each waiting for another's lock.
- Deadlock requires mutual exclusion, hold-and-wait, no preemption and circular wait; breaking any one prevents it.
- Prevention techniques include conservative 2PL, resource ordering, wait-die, wound-wait and timeouts.
- Wait-die is non-preemptive (younger requesters die); wound-wait is preemptive (older requesters wound younger holders); both keep original timestamps to avoid starvation.
- Detection uses a wait-for graph; a cycle means deadlock, broken by rolling back a chosen victim.
- Starvation is distinct from deadlock: the system progresses but one transaction never completes.
- Applications should lock in a consistent order, keep transactions short, and retry on deadlock errors.

Key Terms

Deadlock · Coffman conditions · Mutual exclusion · Hold and wait · No preemption · Circular wait · Deadlock prevention · Conservative 2PL · Resource ordering · Wait-die · Wound-wait · Timestamp · Timeout · Deadlock detection · Wait-for graph · Victim selection · Total rollback · Partial rollback · Starvation · Livelock

Common Mistakes

1. Confusing deadlock with starvation.
2. Reversing wait-die and wound-wait; remember "wound-wait: the **old** one wounds".
3. Thinking a transaction waiting on a cycle is itself deadlocked.
4. Assigning a new timestamp to a restarted transaction, which permits starvation.
5. Believing 2PL prevents deadlock.
6. Not writing retry logic for deadlock errors in application code.

Solved Problems

Problem 31.1 — Transactions T1 (TS 10), T2 (TS 20), T3 (TS 30). T2 holds A; T1 requests A; T3 requests A. What happens under wait-die and under wound-wait?

Solution. Smaller timestamp = older, so T1 is oldest and T3 youngest. **Wait-die:** T1 is older than holder T2 ⇒ T1 **waits**. T3 is younger than T2 ⇒ T3 **dies**. **Wound-wait:** T1 is older than T2 ⇒ T1 **wounds** T2; T2 is rolled back and T1 gets A. T3 is younger than the holder ⇒ T3 **waits**.

Problem 31.2 — Draw the wait-for graph and identify deadlocked transactions: T1 waits for T2; T2 waits for T3; T3 waits for T1; T4 waits for T2; T5 waits for T4.

Solution. Edges: T1 → T2, T2 → T3, T3 → T1, T4 → T2, T5 → T4. The cycle is **T1 → T2 → T3 → T1**, so T1, T2 and T3 are deadlocked. T4 and T5 are blocked, waiting in a chain that leads into the cycle, but are not part of it; once a victim among T1–T3 is rolled back and the cycle breaks, T4 and then T5 will proceed.

Problem 31.3 — Show how consistent lock ordering removes the transfer deadlock.

Solution. The deadlock arose because T1 locked A then B, while T2 locked B then A. If every transfer locks the lower account number first:

time	T1 (A → B)	T2 (B → A)
t1	lock-X(A)	
t2		request lock-X(A) → waits
t3	lock-X(B); transfer; commit; release	
t4		lock-X(A); lock-X(B); transfer; commit

T2 waits but never holds a lock that T1 needs, so no cycle is possible. This is the resource-ordering technique, and it costs nothing but discipline.

Problem 31.4 — Explain why wait-die and wound-wait cannot produce a deadlock.

Solution. In **wait-die**, a transaction waits only for a *younger* one; a younger requester never waits for an older holder — it dies instead. So every wait-for edge points from older to younger, and following edges strictly decreases age, which cannot return to its start. In **wound-wait**, a transaction waits only for an *older* one; every edge points from younger to older, and again a cycle is impossible. In each scheme all edges point in one direction of a total order, so circular wait cannot occur.

Problem 31.5 — A transaction is rolled back as a deadlock victim five times in a row. What is happening and how is it fixed?

Solution. It is **starving**: the victim-selection policy keeps choosing it, perhaps because it is always the youngest or always the one with the least work done at the moment of detection. Fixes: include the number of previous rollbacks in the victim cost so a repeatedly aborted transaction becomes expensive to choose; keep its original timestamp so it ages into the "never roll back" position in timestamp schemes; or, at the application level, add randomized back-off before retrying so the same collision is not reproduced.

Review Questions

1. Define deadlock and give a two-transaction example.
2. State and explain the four Coffman conditions.
3. How does resource ordering prevent deadlock?
4. Explain wait-die and wound-wait and prove that each is deadlock-free.
5. Why must a restarted transaction keep its original timestamp?

6. Explain deadlock detection with a wait-for graph.
7. What criteria are used to select a deadlock victim?
8. Distinguish deadlock from starvation.
9. What is the role of timeouts?
10. Give five practical measures developers can take to reduce deadlocks.

Multiple Choice Questions

1. A deadlock requires (a) any one Coffman condition (b) all four Coffman conditions (c) only circular wait (d) only mutual exclusion.
2. In wait-die, a younger transaction requesting an older one's lock (a) waits (b) dies (c) wounds (d) upgrades.
3. In wound-wait, an older transaction requesting a younger one's lock (a) waits (b) dies (c) wounds the younger (d) times out.
4. A deadlock is detected when the wait-for graph (a) is empty (b) has a cycle (c) is a tree (d) has one node.
5. Consistent lock ordering breaks (a) mutual exclusion (b) hold and wait (c) circular wait (d) no preemption.
6. Conservative 2PL breaks (a) mutual exclusion (b) hold and wait (c) circular wait (d) no preemption.
7. Starvation means (a) all transactions wait (b) one transaction never completes while others do (c) no locks exist (d) the log is full.
8. InnoDB's deadlock error code is (a) 1062 (b) 1213 (c) 1452 (d) 1048.
9. Restarted transactions keep their original timestamp to prevent (a) deadlock (b) starvation (c) phantoms (d) dirty reads.
10. Which scheme is preemptive? (a) Wait-die (b) Wound-wait (c) Timeout (d) Resource ordering.

Answers: 1-(b) 2-(b) 3-(c) 4-(b) 5-(c) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What is circular wait?
2. Define a wait-for graph.
3. What happens to a deadlock victim?
4. Give one victim-selection criterion.
5. What is a partial rollback?
6. Which scheme lets the older transaction preempt?
7. Why should transactions be short?
8. What is `innodb_lock_wait_timeout`?
9. How can an application respond to a deadlock error?
10. Is a transaction waiting on a cycle deadlocked?

Long / Descriptive Questions

1. Explain deadlock, the Coffman conditions, and how each prevention technique attacks a condition.
2. Explain wait-die and wound-wait in detail with examples, and prove freedom from deadlock and starvation.
3. Explain deadlock detection and recovery, including the wait-for graph and victim selection.
4. Compare prevention, detection and timeout approaches for an OLTP system.

5. Describe practical techniques for reducing deadlocks in an application, with SQL examples.

Practical Exercises

1. Create a deadlock between two MySQL sessions and capture the output of `SHOW ENGINE INNODB STATUS`.
2. Reproduce the same deadlock in PostgreSQL and read the deadlock report in the server log.
3. Modify the transactions to lock rows in a consistent order and demonstrate that the deadlock no longer occurs.
4. Write application code (in any language) that retries a transaction up to three times on a deadlock error, with randomized back-off.
5. Simulate wait-die and wound-wait by hand for a scenario of four transactions with given timestamps and lock requests.

CHAPTER 32

Database Recovery

Recovery is the part of the DBMS that makes atomicity and durability true in the presence of failures. After any crash, the recovery manager must restore the database to a state that contains the effects of every committed transaction and none of the effects of any uncommitted one.

32.1 Types of Failure

Failure	Example	Memory lost?	Disk lost?	Handled by
Transaction failure	Constraint violation, deadlock victim, explicit rollback	No	No	Undo that transaction using the log
System crash	Power loss, OS crash, DBMS bug	**Yes**	No	Restart recovery: redo committed, undo uncommitted
Media failure	Disk head crash, corrupted file	Maybe	**Yes**	Restore backup, then roll forward using archived logs
Catastrophe	Fire, flood, ransomware	Yes	Yes	Off-site backup or replica

The central difficulty is the system crash: the buffer pool in memory holds pages modified by committed transactions that have not yet reached disk, and the disk may already hold pages modified by transactions that never committed.

Storage types

- **Volatile** — main memory; lost on crash.
- **Non-volatile** — disk, SSD; survives a crash but not media failure.
- **Stable storage** — an abstraction of storage that never loses data, approximated by writing to several independent non-volatile devices (mirrored disks, remote replicas). The log must be on stable storage.

32.2 Buffer Management Policies

Two policy decisions determine what recovery must do.

Policy	Question	Choices
Steal	May a page modified by an **uncommitted** transaction be written to disk (to free a buffer)?	Steal / No-steal
Force	Must every page modified by a transaction be written to disk **before** it commits?	Force / No-force

Combination	Undo needed?	Redo needed?	Performance
No-steal, Force	No	No	Poor — commits wait for many random writes; buffer pool fills up
Steal, Force	Yes	No	Poor commit latency
No-steal, No-force	No	Yes	Buffer pool constrained

Combination	Undo needed?	Redo needed?	Performance
Steal, No-force	**Yes**	**Yes**	**Best** — used by virtually every real DBMS

Steal/no-force gives the buffer manager complete freedom and makes commits fast — but it means the disk can contain uncommitted changes (requiring **undo**) and can lack committed ones (requiring **redo**). The log exists to make both possible.

32.3 Log-Based Recovery

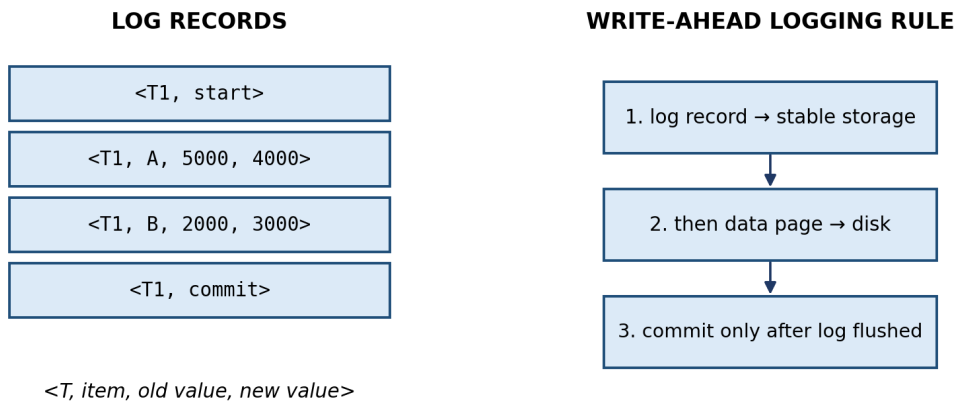


Figure 32.1: Log records and the write-ahead logging rule

The **log** is a sequential file of records describing every change:

```
<T1, start>
<T1, A, 5000, 4000>          T1 changed A from 5000 (before-image) to 4000 (after-image)
<T1, B, 2000, 3000>
<T1, commit>
<T2, start>
<T2, C, 700, 600>
<T2, abort>
```

Each update record carries the **old value** (for undo) and the **new value** (for redo). Every record has a monotonically increasing **log sequence number (LSN)**.

The Write-Ahead Logging (WAL) rule

1. **Before a data page is written to disk, all log records describing changes to that page must already be on stable storage.** This guarantees that any uncommitted change on disk can be undone.
2. **A transaction is committed only when its commit record — and every log record before it — is on stable storage.** This guarantees that any committed change not yet on disk can be redone.

Rule 2 is why a commit is fast under no-force: the DBMS forces only a **sequential append** to the log, not the scattered data pages. Rule 1 is why steal is safe. Together they make steal/no-force correct.

Undo and redo operations

- **undo(T_i)** — scan the log backward, restoring every item T_i wrote to its old value. Each undo action is itself logged (as a compensation record) so that a crash during recovery does not cause confusion.

- **redo(T_i)** — scan the log forward, setting every item T_i wrote to its new value.

Both are **idempotent**: applying them twice has the same effect as once. This matters because recovery itself can be interrupted by another crash.

Restart recovery (basic form)

```
1. Scan the log. Classify each transaction:
      has <T, commit> or <T, abort>         → completed
      has <T, start> but neither of the above → incomplete
2. REDO all changes of committed transactions, scanning forward.
3. UNDO all changes of incomplete transactions, scanning backward;
   write <T, abort> for each.
```

Example.

```
<T1,start> <T1,A,100,50> <T2,start> <T1,commit> <T2,B,200,250>
<T3,start> <T3,C,300,350> <T3,commit> <T2,C,350,400>    *** CRASH ***
```

T1 and T3 committed ⇒ **redo** both: A = 50, C = 350. T2 has no commit ⇒ **undo** backward: C restored to 350, B restored to 200. Final state: A = 50, B = 200, C = 350.

Note the order of operations: T2's change to C came after T3's commit, so C must first be redone to T3's value and then undone to the before-image recorded by T2 — which is exactly 350. Doing redo before undo produces the right answer.

32.4 Deferred and Immediate Update

Technique	Writes to the database happen	On crash	Log contents needed
Deferred update	Only after commit	Redo committed; nothing to undo	New values only
Immediate update	Any time during the transaction	Redo committed and undo uncommitted	Old and new values

Deferred update corresponds to no-steal: uncommitted changes never reach the database, so undo is never needed. It restricts the buffer manager and is impractical for large transactions. **Immediate update** corresponds to steal and is what real systems use.

32.5 Checkpoints

Without checkpoints, restart recovery would have to scan the entire log from the beginning of time — possibly months of records — and redo work that reached disk long ago.

A **checkpoint** records a point from which recovery can start. A simple (quiescent) checkpoint:

```
1. Stop accepting new transactions and wait for active ones to finish.
2. Flush all log records in memory to stable storage.
3. Flush all modified buffer pages to disk.
4. Write <checkpoint> to the log and flush it.
5. Resume.
```

Stopping all activity is unacceptable in practice, so real systems use **fuzzy checkpoints**: the checkpoint record lists the transactions active at that moment, and dirty pages are flushed in the background without halting new work.

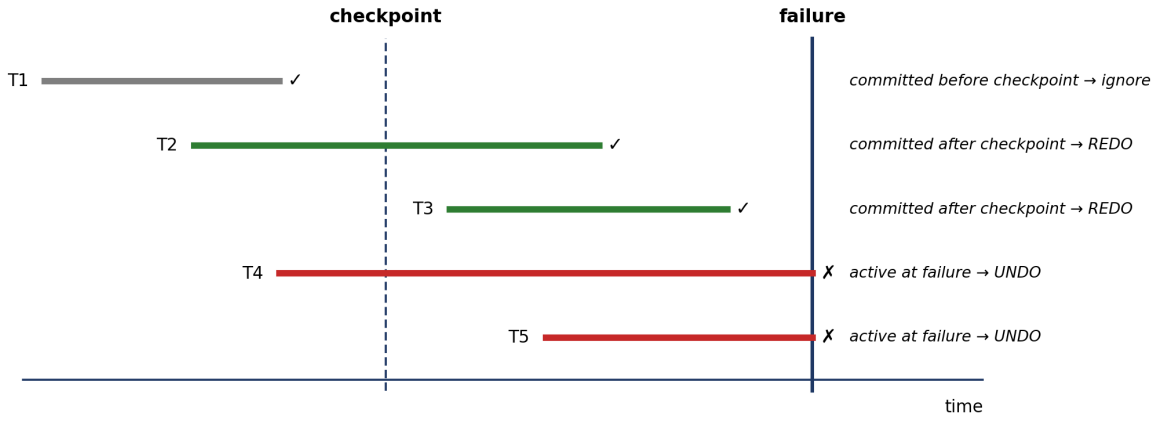


Figure 32.2: Classifying transactions relative to a checkpoint and a failure

Relative to the last checkpoint before a crash:

Transaction	Situation	Action
T1	Committed before the checkpoint	**None** — its changes were flushed at the checkpoint
T2	Started before, committed after the checkpoint	**Redo**
T3	Started and committed after the checkpoint	**Redo**
T4	Started before, still active at the crash	**Undo**
T5	Started after, still active at the crash	**Undo**

The benefit is that recovery reads only the log from (roughly) the last checkpoint onward.

32.6 ARIES

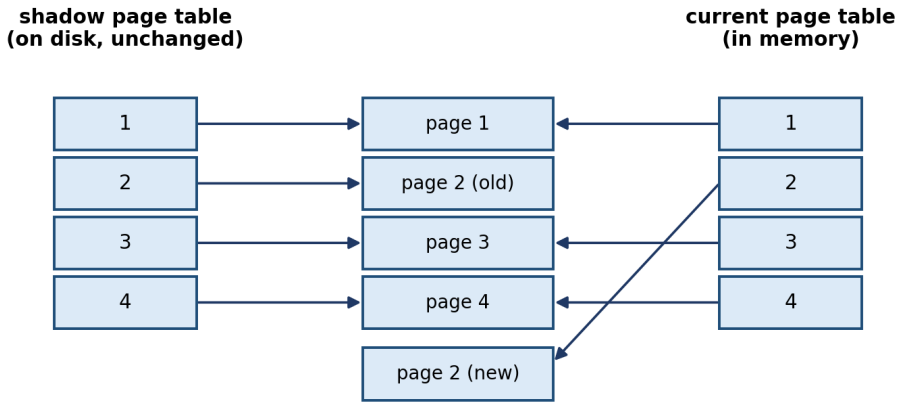
ARIES (Algorithm for Recovery and Isolation Exploiting Semantics), developed at IBM, is the recovery algorithm on which most modern systems are modelled. It uses steal/no-force, WAL and fuzzy checkpoints, and adds:

- **LSNs on pages.** Each data page stores the LSN of the last log record applied to it, so recovery can tell whether a change already reached disk and skip redoing it.
- **Compensation log records (CLRs).** Each undo action is logged, so undo is never repeated after a second crash.
- **Repeating history.** Redo replays **all** logged changes, including those of transactions that will later be undone, restoring the exact pre-crash state before undoing losers.

ARIES recovery runs in three passes:

Pass	Direction	Purpose
Analysis	Forward from the last checkpoint	Determine which transactions were active and which pages were dirty at the crash
Redo	Forward from the earliest relevant LSN	Repeat history: reapply every change not already on the page
Undo	Backward	Roll back every transaction that did not commit, writing CLRs

32.7 Shadow Paging



commit = atomically replace the shadow table with the current one; abort = discard the current table

Figure 32.3: Shadow paging

Shadow paging is an alternative that needs no log for single-user recovery.

- The database is a set of pages accessed through a **page table**.
- When a transaction starts, the current page table is copied to a **shadow page table**, which is kept on disk and never modified.
- Every write goes to a **new** copy of the page; the **current** page table is updated to point to it, while the shadow table still points to the old version.
- **Commit** = atomically write the current page table to disk and make it the new shadow (one pointer swap).
- **Abort or crash** = discard the current table; the shadow table still describes the database exactly as it was.

Advantages	Disadvantages
No undo or redo needed	Committing writes the page table, costly for large databases
Recovery is instantaneous	Data pages become scattered, destroying locality
Simple to implement	Old pages must be garbage-collected
	Difficult to extend to concurrent transactions

Shadow paging survives today in modified forms — copy-on-write file systems such as ZFS and Btrfs, and databases such as LMDB and SQLite's rollback journal alternative — but log-based recovery dominates general-purpose DBMSs.

32.8 Backup and Media Recovery

Logs and checkpoints recover from system crashes. Media failure destroys the data files themselves and requires backups.

Backup type	Contents	Restore speed	Space
Full	Entire database	Fastest	Largest
Incremental	Changes since the last backup of any kind	Slowest (chain of restores)	Smallest
Differential	Changes since the last full backup	Medium	Medium

Backup type	Contents	Restore speed	Space
Logical (`mysqldump`, `pg_dump`)	SQL statements recreating the data	Slow for large databases	Portable
Physical (file copy, `pg_basebackup`, Percona XtraBackup)	Data files and logs	Fast	Exact copy

Point-in-time recovery (PITR). Restore the latest full backup, then replay archived log files (MySQL binary logs, PostgreSQL WAL archive) up to a chosen moment — for example one second before an accidental `DROP TABLE`. This is the only way to recover from a logical error that RAID and replication faithfully copied.

```
# MySQL: restore the backup, then replay binary logs up to a moment
mysql university_db < full_backup.sql
mysqlbinlog --stop-datetime="2026-09-21 10:14:59" binlog.000042 | mysql university_db
```

> The **3-2-1 rule**: keep at least three copies of the data, on two different kinds of media, with one copy off-site. And test restores regularly — a backup that has never been restored is a hope, not a backup.

Chapter Summary

- Failures are transaction failures, system crashes, media failures and catastrophes; each needs a different response.
- Steal/no-force buffer management gives the best performance and requires both undo and redo.
- The log records before- and after-images of every change; write-ahead logging requires log records to reach stable storage before data pages, and the commit record before the commit is acknowledged.
- Restart recovery redoes committed transactions and undoes incomplete ones; undo and redo are idempotent.
- Checkpoints bound the amount of log that recovery must process; fuzzy checkpoints avoid halting the system.
- ARIES recovers in analysis, redo and undo passes, using page LSNs, compensation log records and repeating history.
- Shadow paging achieves atomicity by pointer swap without a log, at the cost of locality and scalability.
- Media failure requires backups; point-in-time recovery replays archived logs to any chosen moment.

Key Terms

Transaction failure · System crash · Media failure · Volatile / non-volatile / stable storage · Buffer pool · Steal / no-steal · Force / no-force · Log · Log sequence number · Before-image · After-image · Write-ahead logging · Undo · Redo · Idempotent · Deferred update · Immediate update · Checkpoint · Fuzzy checkpoint · ARIES · Analysis pass · Repeating history · Compensation log record · Shadow paging · Page table · Full / incremental / differential backup · Point-in-time recovery · Binary log · WAL archive · 3-2-1 rule

Important Formulas / Notation

```
    <T, start>   <T, X, old, new>   <T, commit>   <T, abort>   <checkpoint L>
    WAL:  log record on stable storage  before  data page on disk
    commit: commit record (and all prior log) on stable storage  before  acknowledging
    restart: REDO committed (forward),  then UNDO incomplete (backward)
```

Common Mistakes

1. Undoing before redoing in restart recovery, which can produce wrong values when both touch the same item.
2. Undoing committed transactions or redoing aborted ones.
3. Forgetting to redo transactions that committed after the checkpoint.
4. Believing the WAL rule means writing data before the log — it is the reverse.
5. Treating RAID or replication as a substitute for backups and point-in-time recovery.
6. Assuming shadow paging needs a log.
7. Never testing a restore.

Solved Problems

Problem 32.1 — Given the log below and a crash at its end, state the actions and the final values. Initially A = 1000, B = 2000, C = 3000.

```
<T1,start> <T1,A,1000,900> <T2,start> <T2,B,2000,2500> <T1,commit>
<T3,start> <T3,C,3000,3200> <T2,A,900,800>   *** CRASH ***
```

Solution. T1 committed ⇒ **redo**: A = 900. T2 and T3 did not commit ⇒ **undo**, backward: `<T2,A,900,800>` restores A = 900; `<T3,C,3000,3200>` restores C = 3000; `<T2,B,2000,2500>` restores B = 2000. Final: **A = 900, B = 2000, C = 3000**. Write `<T2,abort>` and `<T3,abort>` to the log.

Problem 32.2 — A checkpoint is taken at time 40 and a crash occurs at time 80. T1 runs 10–30, T2 runs 20–60, T3 runs 45–70, T4 runs 35–(active), T5 runs 55–(active). Classify each.

Solution. T1 committed before the checkpoint ⇒ **nothing to do**. T2 and T3 committed after the checkpoint ⇒ **redo**. T4 and T5 were active at the crash ⇒ **undo**. Exactly the classification of Figure 32.2.

Problem 32.3 — Explain why steal/no-force is used in practice despite needing both undo and redo.

Solution. **No-force** means a commit writes only the log — one sequential append of a few records — instead of forcing every modified data page to scattered locations on disk. Since sequential writes are orders of magnitude faster than random ones (Chapter 22), commit latency and throughput improve enormously. **Steal** means the buffer manager can evict any page when memory is needed, even if an uncommitted transaction modified it; without it, a single large transaction could exhaust the buffer pool. The cost — maintaining undo and redo information — is paid in cheap sequential log writes during normal operation and only occasionally during recovery, which is a very favourable trade.

Problem 32.4 — Why must the log record reach stable storage before the corresponding data page? Give the failure scenario that WAL prevents.

Solution. Suppose uncommitted transaction T changes A from 100 to 200, and the buffer manager writes the page containing A = 200 to disk **before** the log record `<T, A, 100, 200>` is written. The system then crashes. On restart, the disk holds A = 200, T never committed, and the log contains no record of T's change — so recovery has no before-image and cannot restore A = 100. Atomicity is violated permanently. WAL forbids this ordering: because the log record is on stable storage first, recovery always knows the before-image of any uncommitted change that reached disk.

Problem 32.5 — Compare log-based recovery with shadow paging.

Solution. Log-based recovery keeps pages in place, records every change in a sequential log, and after a crash redoes committed and undoes uncommitted work; it supports concurrency, preserves physical locality and has low commit cost under no-force, at the price of a more complex recovery procedure. Shadow paging never overwrites a page in place: it writes changed pages to new locations and commits by swapping a page-table pointer; recovery is instantaneous and needs no log, but commits must write

the page table, data becomes fragmented across the disk, garbage collection is needed, and extending it to many concurrent transactions is difficult. General-purpose DBMSs therefore use log-based recovery.

Problem 32.6 — A developer accidentally runs `DELETE FROM enrollment;` at 10:15 and commits. The database is on RAID 10 with a replica. How is the data recovered?

Solution. Neither RAID nor the replica helps: the delete was a committed transaction, faithfully written to every mirror and replicated within seconds. The procedure is **point-in-time recovery**: (i) restore the most recent full backup — say from 01:00 — to a separate server; (ii) replay the archived binary logs (MySQL) or WAL (PostgreSQL) from 01:00 up to 10:14:59, one second before the delete; (iii) extract the `enrollment` table from the restored copy and reload it into production, or promote the restored copy after reconciling changes made since 10:15. This scenario is the single strongest argument for keeping archived logs and regularly tested backups.

Review Questions

1. Classify the types of failure and state how each is handled.
2. Define volatile, non-volatile and stable storage.
3. Explain steal/no-steal and force/no-force and their implications for recovery.
4. Describe the structure of log records and the purpose of LSNs.
5. State the write-ahead logging rule and explain what it guarantees.
6. Describe restart recovery, and explain why redo precedes undo.
7. Compare deferred and immediate update.
8. Explain checkpoints and fuzzy checkpoints, and classify transactions relative to a checkpoint.
9. Describe the three passes of ARIES.
10. Explain shadow paging and compare it with log-based recovery.

Multiple Choice Questions

1. A power failure is a (a) transaction failure (b) system crash (c) media failure (d) catastrophe.
2. WAL requires the log record to be written (a) after the data page (b) before the data page (c) at checkpoint only (d) never.
3. A transaction active at the crash must be (a) redone (b) undone (c) ignored (d) committed.
4. Steal/no-force requires (a) undo only (b) redo only (c) both undo and redo (d) neither.
5. Deferred update never needs (a) redo (b) undo (c) a log (d) commit.
6. Checkpoints reduce (a) lock contention (b) the log scanned at recovery (c) disk usage (d) deadlocks.
7. ARIES's three passes are (a) redo, undo, commit (b) analysis, redo, undo (c) scan, sort, merge (d) lock, log, flush.
8. Shadow paging commits by (a) writing the log (b) swapping the page-table pointer (c) flushing the buffer (d) taking a checkpoint.
9. Recovery from an accidental committed `DELETE` requires (a) RAID (b) replication (c) point-in-time recovery from backup and logs (d) a checkpoint.
10. Undo and redo operations must be (a) commutative (b) idempotent (c) associative (d) reversible only once.

Answers: 1-(b) 2-(b) 3-(b) 4-(c) 5-(b) 6-(b) 7-(b) 8-(b) 9-(c) 10-(b)

Short Questions

1. What is stable storage?

2. What does a log update record contain?
3. State the WAL rule.
4. What is an LSN?
5. Why must undo and redo be idempotent?
6. What is a fuzzy checkpoint?
7. What is a compensation log record?
8. Give one disadvantage of shadow paging.
9. Define point-in-time recovery.
10. State the 3-2-1 backup rule.

Long / Descriptive Questions

1. Explain log-based recovery fully: log records, WAL, undo, redo and restart recovery, with a worked example.
2. Explain buffer-management policies and their effect on recovery, justifying the choice of steal/no-force.
3. Explain checkpoints, including fuzzy checkpoints, and classify transactions relative to a checkpoint and a crash.
4. Describe ARIES in detail, including LSNs, CLRs, repeating history and its three passes.
5. Compare log-based recovery and shadow paging, and describe a complete backup and point-in-time recovery strategy for a university database.

Practical Exercises

1. Enable binary logging in MySQL (or WAL archiving in PostgreSQL) and verify that log files are produced as data changes.
2. Take a full backup with `mysqldump` or `pg_dump`, make changes, and restore the backup to a separate database.
3. Perform a point-in-time recovery to one minute before a deliberate `DELETE`.
4. For three given logs with crash points, determine by hand which transactions to redo and undo, and the final values.
5. Design a backup schedule (full, differential, log archiving, off-site copy) for `university_db` and state its recovery-point and recovery-time objectives.

CHAPTER 33

Database Security

A database concentrates an organization's most valuable information in one place, which makes it the most attractive target in the system. Database security is the set of mechanisms that ensure only the right people can do the right things with the right data — and that there is a trustworthy record of what they did.

33.1 Goals and Threats

Goal	Meaning	Threat it counters
Confidentiality	Data is disclosed only to authorized users	Theft of student records or bank balances
Integrity	Data is modified only by authorized users, in authorized ways	A clerk changing their own grade
Availability	Authorized users can access data when needed	Denial of service, ransomware
Accountability	Every action can be traced to an identity	Untraceable insider abuse

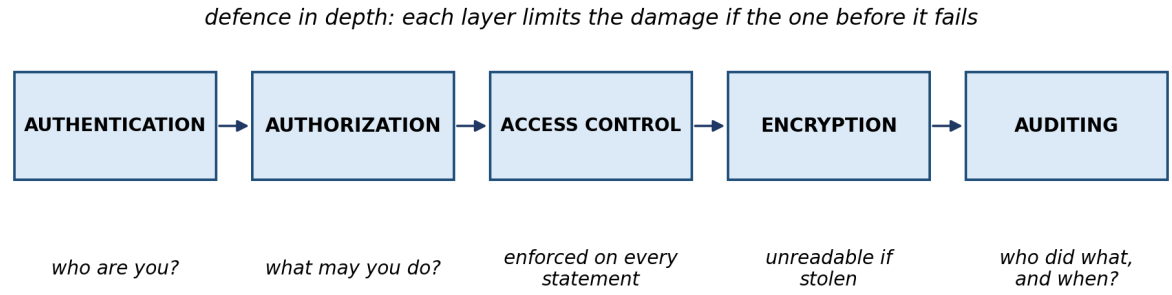


Figure 33.1: Layers of database security

The layers in Figure 33.1 are complementary. Authentication without authorization lets any logged-in user do anything; authorization without encryption leaves data exposed to anyone who steals a disk or backup; neither helps after the fact without auditing.

33.2 Authentication

Authentication verifies *who* a user is.

Method	Description	Strength
Password	Secret known to the user	Weak alone; must be hashed and strong
Certificate / key	Client presents a cryptographic certificate	Strong; standard for service accounts
OS / directory integration	LDAP, Active Directory, Kerberos	Central management of identities
Multi-factor	Something known + something possessed	Strong; required for administrators

```
-- MySQL 8: create an account with a strong authentication plugin
CREATE USER 'exam_clerk'@'10.0.%.%'
       IDENTIFIED WITH caching_sha2_password BY 'Str0ng!Passphrase#2026'
       PASSWORD EXPIRE INTERVAL 90 DAY
       FAILED_LOGIN_ATTEMPTS 5 PASSWORD_LOCK_TIME 1;
```

The host part `'10.0.%.%'` restricts the account to the internal network: the same password is useless from the internet. Passwords are stored by the DBMS as **salted hashes**, never in plain text.

33.3 Authorization, Privileges and Access Control

Authorization determines *what* an authenticated user may do. It is expressed as **privileges** — permissions to perform an operation on an object.

Privilege	Object	Allows
`SELECT`	Table, view, column	Read
`INSERT`	Table, column	Add rows
`UPDATE`	Table, column	Modify rows
`DELETE`	Table	Remove rows
`EXECUTE`	Procedure, function	Run stored code
`CREATE`, `ALTER`, `DROP`	Database, table	Change structure
`GRANT OPTION`	Any	Pass one's privileges on to others

GRANT and REVOKE

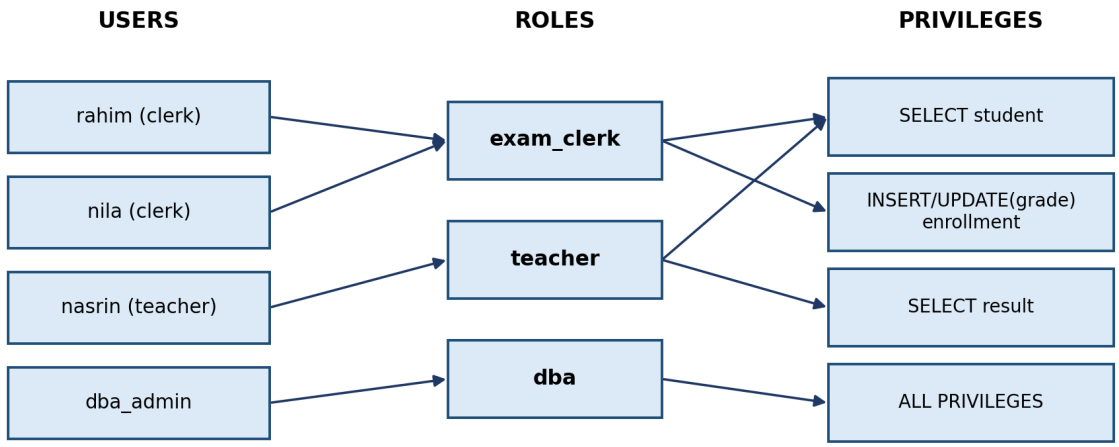
```
GRANT SELECT ON university_db.student TO 'exam_clerk'@'10.0.%.%';
GRANT SELECT, INSERT ON university_db.enrollment TO 'exam_clerk'@'10.0.%.%';
GRANT UPDATE (grade) ON university_db.enrollment TO 'exam_clerk'@'10.0.%.%';   -- column
    ↪ -level
GRANT EXECUTE ON PROCEDURE university_db.sp_enrol_student TO 'portal_app'@'%';

REVOKE INSERT ON university_db.enrollment FROM 'exam_clerk'@'10.0.%.%';
SHOW GRANTS FOR 'exam_clerk'@'10.0.%.%';
```

Access-control models

Model	Idea	Example
Discretionary (DAC)	The object's owner decides who gets access, and may delegate with `GRANT OPTION`	SQL `GRANT` / `REVOKE`
Mandatory (MAC)	The system assigns security labels to data and clearances to users; access follows fixed rules	Military "secret" and "top secret" classifications
Role-based (RBAC)	Privileges are granted to roles; users are assigned roles	Clerk, teacher, DBA

Roles



privileges are granted to roles, roles to users — adding a clerk means one GRANT, not twenty

Figure 33.2: Role-based access control

```
CREATE ROLE exam_clerk_role, teacher_role, report_role;

GRANT SELECT ON university_db.student TO exam_clerk_role;
GRANT SELECT, INSERT, UPDATE (grade) ON university_db.enrollment TO exam_clerk_role;
GRANT SELECT ON university_db.vw_result_sheet TO teacher_role;
GRANT SELECT ON university_db.* TO report_role;

GRANT exam_clerk_role TO 'rahim'@'10.0.%.%', 'nila'@'10.0.%.%';
SET DEFAULT ROLE ALL TO 'rahim'@'10.0.%.%', 'nila'@'10.0.%.%';
```

When a new clerk joins, one statement grants the role; when a clerk leaves, one statement revokes it. Changing what clerks may do changes the role once, for everybody.

The principle of least privilege

Every account receives exactly the privileges its job requires, and no more:

- The web application's account can `EXECUTE` procedures and `SELECT` from views; it cannot `DROP` tables.
- Reporting accounts are read-only.
- The `root` or `postgres` superuser is never used by an application.
- Administrators use personal accounts, so actions are attributable.

Views and row-level security

Views restrict columns and rows (Chapter 15). PostgreSQL additionally supports declarative **row-level security**:

```
-- PostgreSQL: each teacher sees only enrollments in courses they teach
ALTER TABLE enrollment ENABLE ROW LEVEL SECURITY;
CREATE POLICY teacher_sees_own ON enrollment
    FOR SELECT
    USING (course_id IN (SELECT course_id FROM teaches
                         WHERE instructor_login = current_user));
```

33.4 Encryption

Where	Protects against	Mechanism
In transit	Eavesdropping on the network	TLS between client and server (`REQUIRE SSL` in MySQL, `hostssl` in PostgreSQL)
At rest	Stolen disks, backups, snapshots	Transparent Data Encryption (TDE) of data files and logs
Column level	Insiders with table access, leaked query results	Application-side or function-based encryption of specific columns
Backups	Lost or stolen backup media	Encrypt every backup file

```
ALTER USER 'portal_app'@'%' REQUIRE SSL;                    -- MySQL: force TLS
ALTER TABLE student ENCRYPTION = 'Y';                       -- MySQL InnoDB tablespace
    ↪ encryption
```

Passwords of *application users* must never be stored reversibly encrypted; they are stored as salted, slow hashes (bcrypt, scrypt, Argon2) so that even a complete database leak does not reveal them.

> Encryption is only as strong as key management. Keys stored in the same place as the encrypted data provide little protection; production systems keep keys in a separate key-management service or hardware security module.

33.5 Auditing

Auditing records who performed which operation, on which object, when, from where, and whether it succeeded.

Mechanism	Example
Built-in audit plugins	MySQL Enterprise Audit, Percona Audit Log, PostgreSQL `pgaudit`
Triggers writing audit tables	The grade-audit trigger of Section 15.2
Connection and failed-login logs	Detecting password-guessing attempts
Binary / WAL logs	A complete record of committed changes

An audit trail is useful only if it is **tamper-resistant** — written to a location the audited users cannot modify — and **reviewed**; an unread audit log detects nothing.

33.6 SQL Injection

SQL injection is a class of vulnerability in which untrusted input is inserted into the *text* of an SQL statement, so that the input can change the statement's structure rather than being treated purely as a data value. It is consistently among the most common and damaging web application vulnerabilities.

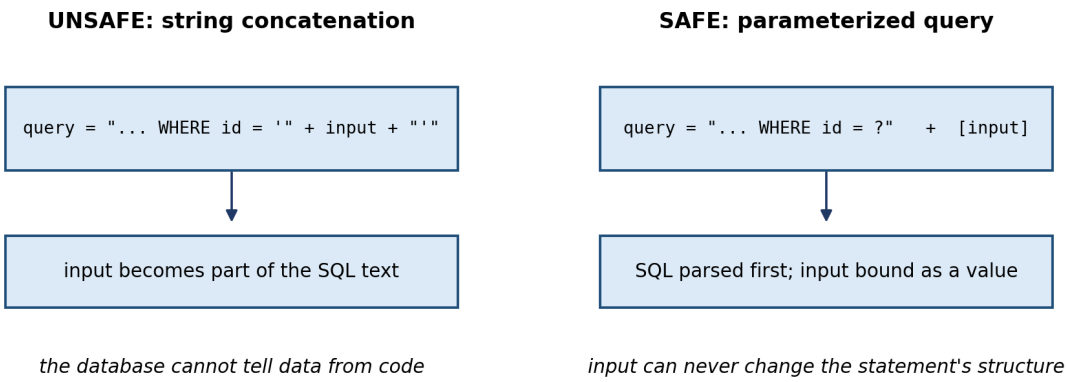


Figure 33.3: String concatenation versus a parameterized query

How the vulnerability arises (conceptually)

Consider application code that builds a query by joining strings:

```
# VULNERABLE — do not write code like this
query = "SELECT name, cgpa FROM student WHERE student_id = '" + user_input + "'"
cursor.execute(query)
```

The programmer intends `user_input` to be a value such as `ASH1901001`. But because it is pasted into the SQL text, the database cannot distinguish the programmer's code from the user's data. Input containing quote characters or SQL keywords would be parsed as part of the statement itself, potentially altering which rows are returned or what operation is performed. The root cause is always the same: **mixing code and data in one string.**

The defence: parameterized queries

A **parameterized query** (prepared statement) sends the SQL structure and the data values separately. The DBMS parses and plans the statement *before* seeing any value, so the value can only ever occupy a data position.

```
# SAFE — Python (PEP 249 drivers such as mysql-connector, psycopg)
cursor.execute(
    "SELECT name, cgpa FROM student WHERE student_id = %s",
    (user_input,)
)
```

```
// SAFE — Java JDBC
PreparedStatement ps = conn.prepareStatement(
    "SELECT name, cgpa FROM student WHERE student_id = ?");
ps.setString(1, userInput);
ResultSet rs = ps.executeQuery();
```

```
// SAFE — PHP PDO
$stmt = $pdo->prepare("SELECT name, cgpa FROM student WHERE student_id = :id");
$stmt->execute([':id' => $userInput]);
```

```
-- SAFE — server-side prepared statement in MySQL
PREPARE get_student FROM 'SELECT name, cgpa FROM student WHERE student_id = ?';
SET @id = 'ASH1901001';
EXECUTE get_student USING @id;
DEALLOCATE PREPARE get_student;
```

Whatever characters the input contains, it is compared against `student_id` as a literal value and nothing else.

Defence in depth

Measure	Why it helps
Parameterized queries everywhere	Eliminates the vulnerability at its source
Stored procedures with parameters	Same separation, plus a narrow interface
Least-privilege application accounts	Limits the damage of any flaw that remains
Input validation (type, length, format)	Rejects obviously invalid input early; a supplement, never a substitute
Allow-lists for identifiers	Table or column names cannot be parameterized, so choose them from a fixed list in code
Generic error messages	Detailed database errors should be logged, not shown to users
ORM frameworks used correctly	Parameterize automatically, unless raw-SQL escape hatches are misused

> Escaping quote characters by hand is **not** an adequate defence: it is error-prone and depends on character sets and SQL modes. Parameterization is the correct solution.

Chapter Summary

- Security aims at confidentiality, integrity, availability and accountability, delivered in layers.
- Authentication verifies identity; authorization, expressed as privileges, controls actions.
- `GRANT` and `REVOKE` implement discretionary access control; roles implement RBAC; mandatory access control uses labels and clearances.
- Least privilege gives every account exactly the access its function needs.
- Encryption protects data in transit (TLS), at rest (TDE) and in backups; key management determines its real strength.
- Auditing provides accountability and must be tamper-resistant and reviewed.
- SQL injection arises from concatenating untrusted input into SQL text; parameterized queries eliminate it by keeping code and data separate.

Key Terms

Confidentiality · Integrity · Availability · Accountability · Authentication · Multi-factor authentication · Authorization · Privilege · `GRANT` · `REVOKE` · `GRANT OPTION` · Discretionary access control · Mandatory access control · Role-based access control · Role · Least privilege · Row-level security · Encryption in transit · Encryption at rest · Transparent Data Encryption · Key management · Salted hash · Auditing · Audit trail · SQL injection · Parameterized query · Prepared statement · Input validation · Allow-list

Common Mistakes

1. Letting the application connect as a superuser.
2. Granting privileges to individual users instead of roles.
3. Relying on input escaping instead of parameterized queries.
4. Storing user passwords with reversible encryption or plain hashing without salt.
5. Encrypting data but storing the key beside it.
6. Collecting audit logs that no one reviews.

7. Displaying raw database error messages to end users.
8. Forgetting that backups contain the same sensitive data as the live database.

Solved Problems

Problem 33.1 — Design roles and grants for `university_db` with three kinds of users: examination clerks (record grades), teachers (view results of all students) and the reporting team (read everything, change nothing).

Solution.

```
CREATE ROLE exam_clerk_role, teacher_role, report_role;
GRANT SELECT ON university_db.student TO exam_clerk_role;
GRANT SELECT, INSERT ON university_db.enrollment TO exam_clerk_role;
GRANT UPDATE (grade) ON university_db.enrollment TO exam_clerk_role;
GRANT SELECT ON university_db.vw_result_sheet TO teacher_role;
GRANT SELECT ON university_db.* TO report_role;
```

No role has `DELETE`, `DROP` or `GRANT OPTION`. Clerks can change only the `grade` column; teachers see results only through a view; reporting is read-only.

Problem 33.2 — Explain, without giving an attack string, why concatenating user input into SQL is dangerous, and show the corrected code.

Solution. When input is concatenated into SQL text, the DBMS parses the combined string as one statement, so characters in the input that have meaning in SQL — quotes, operators, keywords — become part of the statement's structure. The input can therefore alter the logic of the query instead of acting only as a value, and the database has no way to tell which parts the programmer wrote. The correction separates structure from data:

```
cursor.execute("SELECT * FROM student WHERE email = %s AND dept_id = %s",
               (email_input, dept_input))
```

The statement is parsed with placeholders; the values are bound afterward and can never be interpreted as SQL.

Problem 33.3 — Why can table names not be parameterized, and how should a report page that lets users choose a sort column be written safely?

Solution. Placeholders stand only for *values*; identifiers such as table and column names are part of the statement's structure, which must be known when the statement is parsed. The safe approach is an **allow-list** in application code:

```
ALLOWED = {"name": "name", "cgpa": "cgpa", "admitted": "admitted_on"}
col = ALLOWED.get(user_choice, "name")          # anything else falls back to a safe
    ↪ default
cursor.execute(f"SELECT name, cgpa FROM student ORDER BY {col}")
```

The user's text never reaches the SQL; only one of three programmer-written column names can.

Problem 33.4 — Compare DAC, MAC and RBAC.

Solution. **DAC**: the owner of an object grants and revokes access at their discretion and may delegate; flexible but hard to control at scale, since privileges can spread through `GRANT OPTION`. **MAC**: access is determined by system-enforced labels and clearances that users cannot change; rigid but strong, used in military and intelligence settings. **RBAC**: privileges attach to organizational roles and users acquire them through role membership; it scales well, matches how organizations actually think about access, and is

what most enterprise databases use, typically implemented on top of DAC mechanisms.

Problem 33.5 — A laptop containing an unencrypted nightly backup of the student database is stolen. Which controls failed and what should be in place?

Solution. Three controls failed. **Encryption at rest** for backups was absent, so the file is readable by anyone. **Data-handling procedure** failed, since production data was copied to a portable device at all. **Least privilege** may have failed, if the person carrying the laptop did not need a full copy. Remedies: encrypt every backup with keys held in a separate key-management system; store backups only in controlled locations; use masked or synthetic data for development and testing; and audit who exports data and when.

Review Questions

1. State the four goals of database security.
2. Distinguish authentication from authorization.
3. List the main SQL privileges and the objects they apply to.
4. Explain `GRANT`, `REVOKE` and `GRANT OPTION` with examples.
5. Compare DAC, MAC and RBAC.
6. Explain roles and the principle of least privilege.
7. Describe encryption in transit, at rest and at column level.
8. What makes an audit trail useful?
9. Explain conceptually how SQL injection arises.
10. Explain how parameterized queries prevent SQL injection, and list supporting measures.

Multiple Choice Questions

1. Verifying identity is (a) authorization (b) authentication (c) auditing (d) encryption.
2. `GRANT UPDATE (grade) ON enrollment` is a (a) table-level (b) column-level (c) row-level (d) database-level privilege.
3. Assigning privileges to job functions is (a) DAC (b) MAC (c) RBAC (d) TDE.
4. TLS protects data (a) at rest (b) in transit (c) in backups only (d) in memory.
5. The primary defence against SQL injection is (a) escaping quotes (b) parameterized queries (c) longer passwords (d) views.
6. User passwords should be stored as (a) plain text (b) reversible ciphertext (c) salted slow hashes (d) base64.
7. `WITH GRANT OPTION` lets a user (a) drop tables (b) pass privileges to others (c) bypass audit (d) encrypt data.
8. Least privilege means (a) no privileges (b) all privileges (c) only necessary privileges (d) read-only for all.
9. Identifiers such as table names should be handled with (a) placeholders (b) an allow-list (c) escaping (d) encryption.
10. Mandatory access control relies on (a) owner discretion (b) labels and clearances (c) roles only (d) passwords only.

Answers: 1-(b) 2-(b) 3-(c) 4-(b) 5-(b) 6-(c) 7-(b) 8-(c) 9-(b) 10-(b)

Short Questions

1. Define authorization.

2. What does `REVOKE` do?
3. What is a role?
4. Name two authentication methods.
5. What is Transparent Data Encryption?
6. Why must audit logs be tamper-resistant?
7. Define SQL injection in one sentence.
8. What is a prepared statement?
9. Why is input validation not sufficient alone?
10. What is row-level security?

Long / Descriptive Questions

1. Explain the layers of database security with examples of the controls at each layer.
2. Explain privileges, `GRANT` and `REVOKE`, and roles, and design a complete role scheme for a university.
3. Compare DAC, MAC and RBAC with examples of where each is appropriate.
4. Discuss encryption and key management for a database system, including backups.
5. Explain SQL injection conceptually and describe a complete defence-in-depth strategy centred on parameterized queries.

Practical Exercises

1. Create three roles for `university_db`, grant appropriate privileges, assign users, and verify with `SHOW GRANTS` that each user can and cannot perform the intended operations.
2. Configure your DBMS to require TLS for an application account and verify that an unencrypted connection is refused.
3. Write the same database lookup in Python (or Java) twice — once with string concatenation and once parameterized — and explain in comments why only the second is safe.
4. Enable an audit mechanism (a trigger-based audit table or `pgaudit`) and record a week of simulated grade changes.
5. Implement a sortable report page using an allow-list for the sort column.

CHAPTER 34

Distributed Databases

A **distributed database** is a single logical database whose data is physically stored at several sites connected by a network. A **distributed DBMS (DDBMS)** manages it so that users can query and update it as though it were one centralized database.

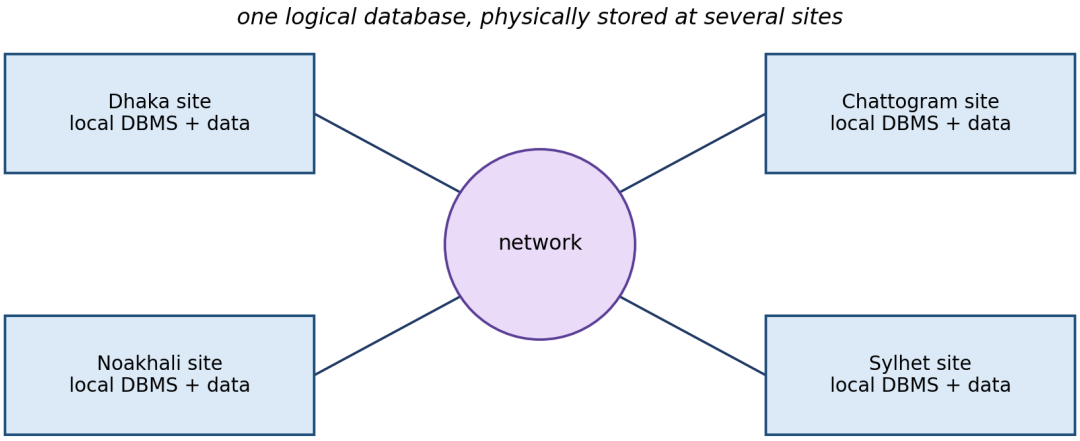


Figure 34.1: A distributed database spread across four sites

34.1 Why Distribute?

Motivation	Explanation
Organizational structure	A bank with branches in Dhaka, Chattogram and Sylhet naturally keeps each branch's accounts locally
Local autonomy	Each site controls its own data and continues working if others are unreachable
Performance	Queries run against nearby data, avoiding long-distance network trips
Availability	Replicated data survives the failure of one site
Scalability	Capacity grows by adding sites rather than buying one ever-larger machine

Basis	Centralized database	Distributed database
Data location	One site	Several sites
Control	Single point	Shared or coordinated
Failure impact	Whole system down	Other sites may continue
Query processing	Local	May require network transfer
Transaction management	Local commit	Distributed commit protocol (2PC)
Complexity	Lower	Much higher
Cost	One large server	Several servers plus network
Scalability	Vertical (bigger machine)	Horizontal (more machines)

34.2 Types of Distributed System

Type	Description	Example
Homogeneous	Every site runs the same DBMS software with the same schema conventions	A bank running PostgreSQL at every branch
Heterogeneous	Sites run different DBMSs, possibly with different data models	A university joining an Oracle HR system with a MySQL student system
Federated (multidatabase)	Autonomous existing databases present a combined view through a middleware layer	A government portal querying separate land, tax and NID databases

Homogeneous systems are far easier to build. Heterogeneous and federated systems must translate queries, data types and transaction semantics between systems, and often sacrifice full transactional guarantees across the boundary.

34.3 Distribution Transparency

The goal of a DDBMS is to hide distribution from users as far as possible.

Transparency	The user need not know
Location	Where the data is stored
Fragmentation	That a table is split into pieces
Replication	That several copies exist
Naming	Site-specific names of objects
Failure	That a site has failed (if a replica can serve instead)

34.4 Data Fragmentation

Fragmentation divides a relation into smaller pieces stored at different sites.

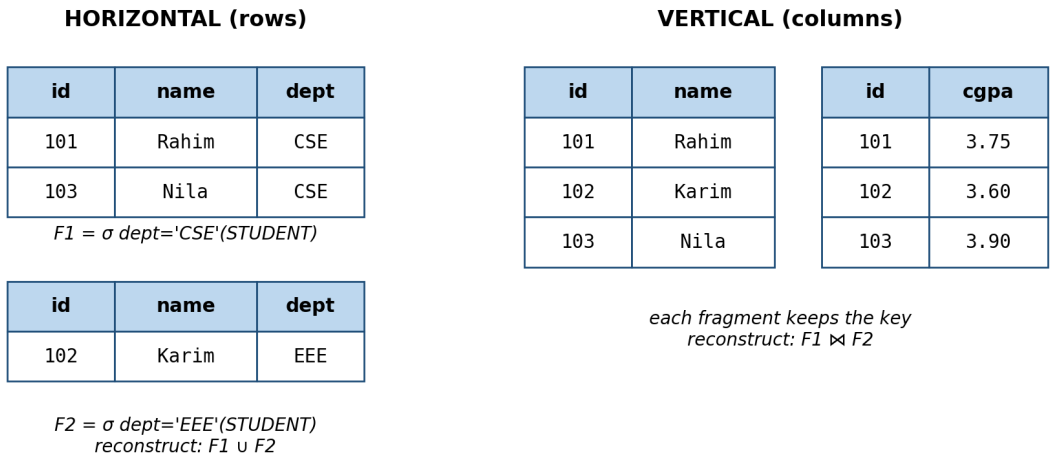


Figure 34.2: Horizontal and vertical fragmentation

Horizontal fragmentation

Rows are divided by a selection predicate. Each fragment has the full set of columns.

```
STUDENT_CSE = σ_{dept_id='CSE'}(STUDENT)
STUDENT_EEE = σ_{dept_id='EEE'}(STUDENT)
STUDENT     = STUDENT_CSE ∪ STUDENT_EEE
```

Use: place each department's or each branch's rows where they are used most.

Vertical fragmentation

Columns are divided by projection. **Every fragment must include the primary key** so that the original can be reconstructed by a join.

```
STUDENT_PUBLIC  = π_{student_id, name, dept_id}(STUDENT)
STUDENT_PRIVATE = π_{student_id, dob, cgpa, email}(STUDENT)
STUDENT         = STUDENT_PUBLIC ⋈ STUDENT_PRIVATE
```

Use: separate frequently accessed columns from rarely used or sensitive ones.

Mixed (hybrid) fragmentation

Horizontal fragmentation of a vertical fragment, or vice versa.

Correctness rules

Rule	Requirement
Completeness	Every tuple (or attribute) of the original appears in at least one fragment
Reconstruction	The original can be rebuilt — by union for horizontal, by join on the key for vertical
Disjointness	For horizontal fragmentation, a tuple belongs to exactly one fragment; for vertical, only the key is repeated

34.5 Replication

Replication keeps copies of data at more than one site.

Strategy	Description	Trade-off
Full replication	Every site has the whole database	Fastest reads and highest availability; costly updates
Partial replication	Selected fragments are copied to selected sites	Balanced
No replication	Each fragment at exactly one site	Cheapest updates; no redundancy

Update propagation	How it works	Consistency
Synchronous (eager)	A transaction commits only when all replicas are updated	Strong, but slower and blocked by unreachable replicas
Asynchronous (lazy)	The primary commits; replicas are updated shortly after	Fast, but replicas may briefly lag — **replication lag**

Topology	Description
Primary–replica (master–slave)	All writes go to one primary; replicas serve reads
Multi-primary (multi-master)	Several sites accept writes; conflicts must be detected and resolved

34.6 Distributed Query Processing

In a centralized system cost is dominated by disk I/O; in a distributed system **network transfer** often dominates. The optimizer therefore tries to minimize the volume of data shipped between sites.

Example. `STUDENT` (10,000 rows × 200 bytes = 2 MB) is at site 1; `ENROLLMENT` (100,000 rows × 40 bytes = 4 MB) is at site 2; the result is needed at site 3.

Strategy	Data transferred
Ship both tables to site 3 and join there	2 MB + 4 MB = 6 MB
Ship STUDENT to site 2, join, ship result to site 3	2 MB + result
Ship ENROLLMENT to site 1, join, ship result to site 3	4 MB + result

Semi-join

A **semi-join** reduces transfer by shipping only join-column values first:

```
1. Site 2 projects π_{student_id}(ENROLLMENT) — distinct IDs, perhaps 8,000 × 10 B = 80
    ↪ KB — and ships to site 1.
2. Site 1 keeps only STUDENT rows whose IDs appear:  STUDENT ⋉ ENROLLMENT.
3. Site 1 ships the reduced STUDENT (8,000 rows, 1.6 MB, or fewer columns) to site 2 for
    ↪ the join.
```

Semi-joins pay off when the join is selective — when many rows of one relation have no match in the other.

34.7 Distributed Transactions and Two-Phase Commit

A **distributed transaction** updates data at more than one site. Atomicity now requires that **all sites commit or all abort** — a site cannot commit independently if another has failed.

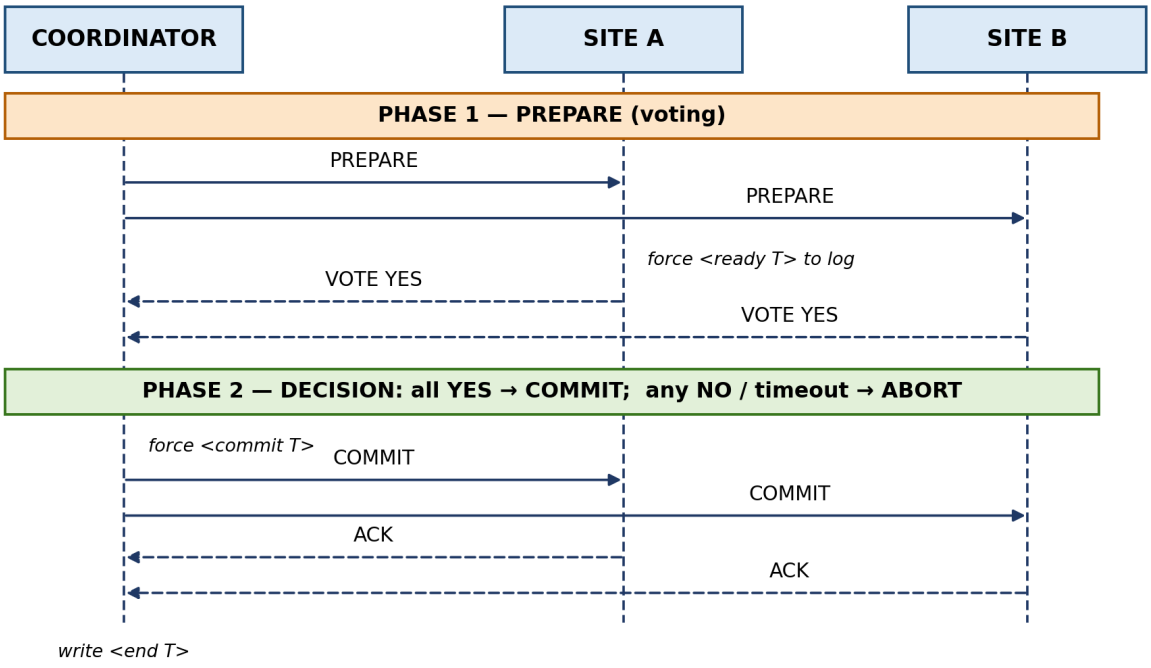


Figure 34.3: The two-phase commit protocol

The **two-phase commit (2PC)** protocol uses a **coordinator** (usually the site where the transaction began) and **participants**.

```
PHASE 1 — PREPARE (voting)
  1. Coordinator writes <prepare T> to its log and sends PREPARE to all participants.
  2. Each participant decides whether it can commit:
       YES: force <ready T> to its log (it can now commit even after a crash) and reply
    ↪ YES.
```

```
        NO:  write <abort T> and reply NO.

PHASE 2 — COMMIT / ABORT (decision)
  3. If every participant voted YES: coordinator forces <commit T> and sends COMMIT.
     If any voted NO or failed to respond: coordinator forces <abort T> and sends ABORT.
  4. Participants carry out the decision, log it, and acknowledge.
  5. Coordinator writes <end T> when all acknowledgements arrive.
```

The blocking problem. A participant that has voted YES has promised to commit if told to and cannot unilaterally abort. If the coordinator then crashes before sending the decision, the participant must **wait** — holding its locks — until the coordinator recovers. This makes 2PC a *blocking* protocol. **Three-phase commit** and consensus protocols such as **Paxos** and **Raft** avoid indefinite blocking by electing a new coordinator; modern distributed SQL systems (Chapter 38) are built on consensus.

34.8 Advantages and Limitations

Advantages	Limitations
Reflects organizational structure	Much greater design and administration complexity
Local autonomy	Distributed transactions are slower (extra round trips)
Improved availability through replication	Network partitions create hard consistency trade-offs (CAP, Chapter 35)
Horizontal scalability	Security must be enforced at many sites
Faster local access	Global query optimization is harder

Chapter Summary

- A distributed database is one logical database stored at several networked sites and managed by a DDBMS.
- Systems may be homogeneous, heterogeneous or federated.
- Distribution transparency hides location, fragmentation, replication and failures from users.
- Horizontal fragmentation splits rows and is reconstructed by union; vertical fragmentation splits columns, keeps the key in every fragment, and is reconstructed by join.
- Replication improves availability and read performance at the cost of update propagation; it may be synchronous or asynchronous.
- Distributed query processing minimizes network transfer, using techniques such as semi-joins.
- Two-phase commit guarantees atomicity across sites but blocks if the coordinator fails after participants vote yes.

Key Terms

Distributed database · DDBMS · Site · Homogeneous · Heterogeneous · Federated database · Distribution transparency · Location transparency · Fragmentation · Horizontal fragmentation · Vertical fragmentation · Mixed fragmentation · Completeness · Reconstruction · Disjointness · Replication · Full / partial replication · Synchronous / asynchronous replication · Replication lag · Primary–replica · Multi-primary · Semi-join · Distributed transaction · Two-phase commit · Coordinator · Participant · Blocking · Three-phase commit · Consensus

Important Formulas / Notation

```
    horizontal:  R = F1 ∪ F2 ∪ … ∪ Fn,   Fi = σ_{pi}(R)
    vertical:    R = F1 ⋈ F2 ⋈ … ⋈ Fn,   Fi = π_{key, Ai}(R)
```

```
semi-join:   R ⋉ S = π_{attributes of R}(R ⋈ S)
```

Common Mistakes

1. Omitting the primary key from a vertical fragment, making reconstruction impossible.
2. Confusing fragmentation (splitting) with replication (copying).
3. Believing 2PC is non-blocking.
4. Ignoring network cost when reasoning about distributed query plans.
5. Assuming asynchronous replicas are always current.
6. Designing a distributed database when a single well-indexed server would suffice.

Solved Problems

Problem 34.1 — Fragment `ACCOUNT(acc_no, branch, customer_id, balance, open_date)` horizontally for three branches and vertically into operational and historical parts; state how each is reconstructed.

Solution. Horizontal: `ACC_DHK = σ_{branch='Dhaka'}(ACCOUNT)`, `ACC_CTG = σ_{branch='Chattogram'}(ACCOUNT)`, `ACC_SYL = σ_{branch='Sylhet'}(ACCOUNT)`; reconstruct with `ACC_DHK ∪ ACC_CTG ∪ ACC_SYL`. The fragments are complete and disjoint if every account belongs to exactly one of the three branches. Vertical: `ACC_OPS = π_{acc_no, branch, balance}(ACCOUNT)` and `ACC_HIST = π_{acc_no, customer_id, open_date}(ACCOUNT)`; reconstruct with `ACC_OPS ⋈ ACC_HIST` on `acc_no`, which appears in both.

Problem 34.2 — Walk through 2PC for a transfer between a Dhaka account and a Sylhet account when the Sylhet site cannot commit.

Solution. The coordinator sends PREPARE to both sites. Dhaka can commit: it forces `<ready T>` and replies YES. Sylhet cannot — perhaps the account is frozen — so it writes `<abort T>` and replies NO. Since one vote is NO, the coordinator forces `<abort T>` and sends ABORT to both. Dhaka undoes its debit using its log; Sylhet has already aborted. Neither site committed, so atomicity is preserved and no money is created or destroyed.

Problem 34.3 — Explain the blocking problem of 2PC.

Solution. After voting YES, a participant has written `<ready T>` and surrendered its right to abort unilaterally, because the coordinator might have decided to commit. If the coordinator crashes before the participant learns the decision, the participant cannot safely commit (others may have voted NO) nor safely abort (the coordinator may have logged commit and told other participants). It must keep T's locks and wait for the coordinator to recover, blocking any other transaction that needs those items. Three-phase commit adds a pre-commit round, and consensus protocols elect a replacement coordinator, to avoid this indefinite wait.

Problem 34.4 — STUDENT (2 MB) is at site 1, ENROLLMENT (4 MB) at site 2, and the query result (0.5 MB) is needed at site 2. Which strategy minimizes transfer?

Solution. Ship STUDENT to site 2 and join there: **2 MB**, and the result is already at site 2. Shipping ENROLLMENT to site 1 costs 4 MB plus 0.5 MB to return the result — 4.5 MB. With a semi-join, site 2 sends distinct student IDs (say 80 KB), site 1 returns only matching students with only the needed columns — perhaps 0.3 MB — for a total below 0.4 MB, the best of all when many students have no enrollments or many columns are unnecessary.

Problem 34.5 — Compare synchronous and asynchronous replication for a bank's account balances and for a university's public course catalogue.

Solution. **Account balances** need synchronous replication (or a consensus-based system): a withdrawal acknowledged by one site must not be invisible at another, or money could be withdrawn twice. The price is higher commit latency and the inability to commit if a replica is unreachable. **Course catalogue** suits asynchronous replication: it changes rarely, a delay of a few seconds before a new course appears at a remote replica harms nobody, and read performance and availability matter more than instant consistency.

Review Questions

1. Define a distributed database and a DDBMS.
2. Why do organizations distribute data?
3. Compare centralized and distributed databases.
4. Distinguish homogeneous, heterogeneous and federated systems.
5. Explain the forms of distribution transparency.
6. Explain horizontal, vertical and mixed fragmentation with reconstruction.
7. State the three correctness rules of fragmentation.
8. Compare replication strategies and update-propagation methods.
9. Explain semi-join and when it helps.
10. Explain two-phase commit and its blocking problem.

Multiple Choice Questions

1. Splitting a table by rows is (a) vertical (b) horizontal (c) replication (d) normalization.
2. Vertical fragments must all contain (a) every column (b) the primary key (c) foreign keys only (d) indexes.
3. Horizontal fragments are reconstructed by (a) join (b) union (c) division (d) difference.
4. A system with different DBMSs at different sites is (a) homogeneous (b) heterogeneous (c) centralized (d) replicated.
5. In 2PC, a participant that votes YES (a) may abort at will (b) must await the decision (c) commits immediately (d) becomes coordinator.
6. Asynchronous replication can cause (a) deadlock (b) replication lag (c) fragmentation (d) phantoms only.
7. The dominant cost in distributed query processing is often (a) CPU (b) network transfer (c) parsing (d) sorting.
8. A semi-join first ships (a) the whole relation (b) join-attribute values (c) the result (d) the log.
9. 2PC is (a) non-blocking (b) blocking (c) consensus-based (d) single-phase.
10. Hiding where data is stored is (a) replication transparency (b) location transparency (c) failure transparency (d) naming.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What is a site?
2. Define fragmentation transparency.
3. What is mixed fragmentation?
4. What is replication lag?
5. What does the coordinator do in 2PC?
6. Why is 2PC called blocking?
7. Give one advantage of full replication.

8. What is a federated database?
9. What is a semi-join?
10. Name one protocol that avoids 2PC blocking.

Long / Descriptive Questions

1. Explain distributed database architecture, types of system and forms of transparency.
2. Explain horizontal, vertical and mixed fragmentation with correctness rules and examples.
3. Discuss replication strategies and synchronous versus asynchronous propagation with use cases.
4. Explain distributed query processing, including transfer-cost analysis and semi-joins.
5. Explain two-phase commit step by step, including failure handling and the blocking problem.

Practical Exercises

1. Set up MySQL or PostgreSQL primary–replica replication between two instances and measure replication lag under load.
2. Horizontally fragment `student` by department into separate tables and write a view that reconstructs the whole relation with `UNION ALL`.
3. Vertically fragment `student` into public and private parts and reconstruct it with a join.
4. Use MySQL XA transactions (`XA START`, `XA PREPARE`, `XA COMMIT`) to perform a distributed transaction across two databases.
5. For two given tables at two sites, compute the data transferred under three strategies and identify the cheapest.

CHAPTER 35

NoSQL Databases

NoSQL ("not only SQL") refers to a family of database systems that do not use the relational table model as their primary abstraction. They emerged to meet requirements that traditional relational systems handled poorly — and they complement relational databases rather than replace them.

35.1 Why NoSQL Emerged

By the late 2000s, companies such as Google, Amazon and Facebook faced workloads that strained single-server relational databases:

Pressure	Relational difficulty	NoSQL response
Massive scale — billions of rows, millions of requests per second	Scaling a single server vertically hits hardware limits; joins and distributed transactions across many servers are expensive	Scale **horizontally** across hundreds of commodity machines
Always-on availability across data centres	Strong consistency across regions increases latency and blocks during partitions	Relax consistency to stay available
Rapidly changing, semi-structured data	Every schema change is a migration	**Flexible schemas**
Specific access patterns — lookups by key, graph traversal	General-purpose engines carry overhead for patterns they are not optimized for	Specialized data models

Two papers shaped the field: Google's **Bigtable** (2006), the ancestor of column-family stores, and Amazon's **Dynamo** (2007), the ancestor of key-value stores with eventual consistency.

35.2 The Four NoSQL Data Models

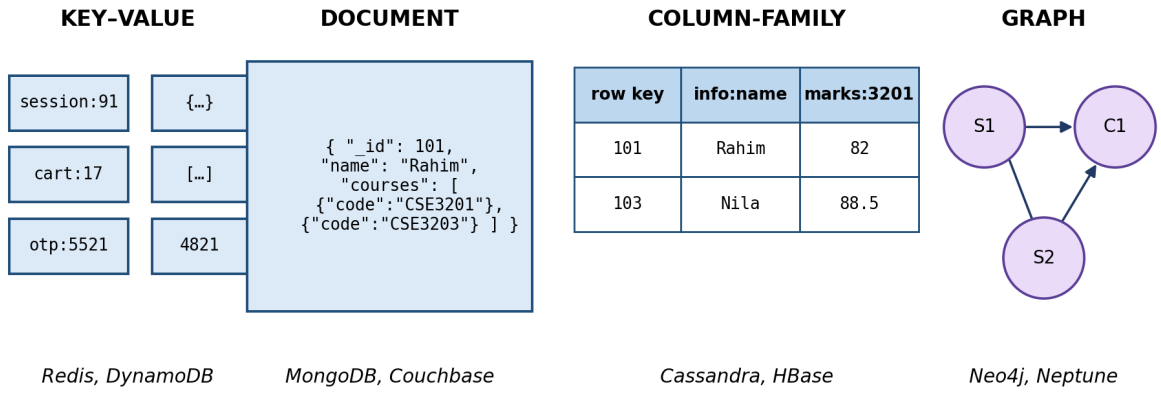


Figure 35.1: The four families of NoSQL data model

Key–value stores

Data is a collection of `(key, value)` pairs; the value is opaque to the database, which supports only `get(key)`, `put(key, value)` and `delete(key)`.

```
SET  session:91a7  '{"user":"ASH1901001","expires":1726912000}'
GET  session:91a7
SET  otp:01712345678  4821  EX 300          -- Redis: expires after 300 seconds
```

Strengths: extreme speed (often in memory), trivial horizontal partitioning by key. *Weaknesses:* no queries on the value's contents, no joins. *Uses:* caching, sessions, shopping carts, rate limiters, leaderboards. *Systems:* Redis, Amazon DynamoDB, Riak.

Document stores

Data is stored as self-describing **documents** — typically JSON or its binary form BSON — grouped into **collections**. Unlike key-value stores, the database understands document structure and can query and index fields inside it.

```
// MongoDB-style document in the "students" collection
{
  "_id": "ASH1901001",
  "name": { "first": "Rahim", "last": "Uddin" },
  "department": "CSE",
  "cgpa": 3.75,
  "phones": ["01712-345678", "01911-222333"],
  "enrollments": [
    { "course": "CSE3201", "semester": "5th", "grade": "A"  },
    { "course": "CSE3203", "semester": "5th", "grade": "A-" }
  ]
}
```

Everything the relational schema spread across `student`, `student_phone` and `enrollment` sits in one document. Reading a student's full record is a single lookup with no join.

```
// MongoDB query API (conceptual)
db.students.find({ department: "CSE", cgpa: { $gt: 3.5 } }, { name: 1, cgpa: 1 })
db.students.find({ "enrollments.course": "CSE3201" })
db.students.createIndex({ department: 1, cgpa: -1 })
db.students.updateOne({ _id: "ASH1901001" }, { $push: { phones: "01533-000000" } })
db.students.aggregate([
  { $group: { _id: "$department", avgCgpa: { $avg: "$cgpa" }, n: { $sum: 1 } } },
  { $sort: { avgCgpa: -1 } }
])
```

Embedding versus referencing is the central design decision:

Embed (nest inside the parent)	Reference (store an ID and look up separately)
Data is read together	Data is read independently
One-to-few relationship	One-to-many or many-to-many with large sets
Child has no meaning outside parent	Child is shared by many parents
Example: a student's phone numbers	Example: a course referenced by thousands of students

Embedding course details (title, credit) inside every student's enrollments would reintroduce exactly the redundancy and update anomalies of Chapter 18. The normalization lessons do not disappear in a document database; they become design trade-offs made per access pattern.

Systems: MongoDB, Couchbase, Amazon DocumentDB, Firestore.

Column-family (wide-column) stores

Rows are identified by a **row key**; columns are grouped into **column families**; each row may have a different set of columns, and there can be millions of columns per row. Data is stored and distributed by row key and physically grouped by column family.

```
row key      | info:name   info:dept | marks:CSE3201  marks:CSE3203
ASH1901001   | Rahim       CSE       | 82             78
ASH1901003   | Nila        CSE       | 88.5           85
```

Strengths: very high write throughput, linear horizontal scaling, efficient scans of a key range. *Weaknesses:* queries must be designed around the row key; ad-hoc queries are limited. *Uses:* time-series and sensor data, event logs, messaging history. *Systems:* Apache Cassandra, HBase, Google Bigtable, ScyllaDB.

Graph databases

Data is stored as **nodes** (entities) and **edges** (relationships), both of which carry properties. Relationships are first-class and traversed directly, without joins.

```
// Neo4j Cypher (conceptual)
CREATE (s:Student {id:'ASH1901001', name:'Rahim'})
CREATE (c:Course  {code:'CSE3201'})
CREATE (s)-[:ENROLLED_IN {grade:'A'}]->(c)

// students who took a course with Rahim — two hops
MATCH (:Student {name:'Rahim'})-[:ENROLLED_IN]->(c)<-[:ENROLLED_IN]-(peer)
RETURN DISTINCT peer.name
```

Strengths: multi-hop traversals (friends-of-friends, shortest path, recommendations) are fast regardless of total data size. *Weaknesses:* bulk aggregate analytics and horizontal partitioning are harder. *Uses:* social networks, fraud rings, recommendation engines, knowledge graphs, network topology. *Systems:* Neo4j, Amazon Neptune, TigerGraph.

35.3 Relational versus NoSQL

Basis	Relational database	NoSQL database
Data model	Tables of rows and columns	Collections/documents, key-value pairs, column families, graphs
Schema	Fixed, declared in advance	Flexible; enforced by the application or optionally by validation rules
Query language	SQL, standardized	Database-specific APIs and query languages
Theoretical basis	Strong relational model, normalization	Multiple data models, designed around access patterns
Relationships	Foreign keys and joins	Embedding, references, or native edges (graph)
Transactions	Full ACID, multi-row and multi-table	Often limited to a single document or key; some now offer multi-document ACID
Consistency	Strong by default	Often tunable; frequently eventual
Scaling	Primarily vertical; horizontal is harder	Designed for horizontal scale-out
Best for	Complex queries, integrity, transactions	Massive scale, flexible data, specific access patterns

> Modern practice is **polyglot persistence**: using several database types in one system, each for what it does best. A typical e-commerce platform keeps orders and payments in a relational database, sessions and carts in Redis, the product catalogue in a document store, and recommendations in a graph database.

35.4 The CAP Theorem

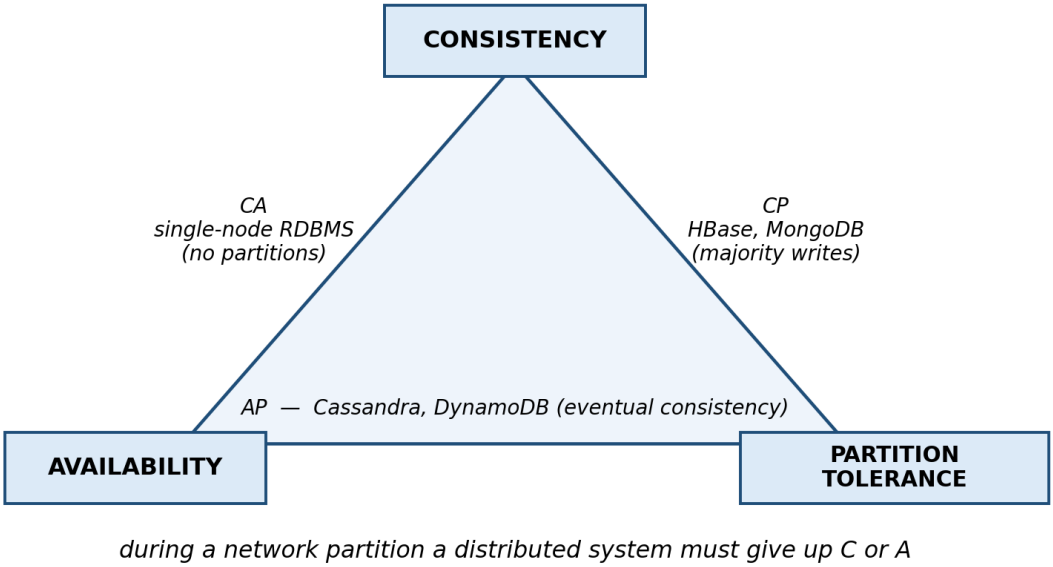


Figure 35.2: The CAP theorem

For a distributed data store, Brewer's **CAP theorem** considers three properties:

- **Consistency (C)** — every read receives the most recent write, or an error. (This is *linearizability*, not the C of ACID.)
- **Availability (A)** — every request to a non-failed node receives a non-error response.
- **Partition tolerance (P)** — the system keeps operating even when network messages between nodes are lost.

The theorem states that when a **network partition** occurs, a system must choose between consistency and availability. Since partitions cannot be ruled out in any real network, the practical choice is between:

- **CP systems** — during a partition, refuse some requests (sacrifice availability) rather than return possibly stale data. Examples: HBase, MongoDB with majority reads and writes, etcd, ZooKeeper.
- **AP systems** — during a partition, keep answering (sacrifice consistency), possibly returning stale data, and reconcile later. Examples: Cassandra, DynamoDB, CouchDB in their default configurations.

"CA" describes a single-node system that simply never experiences partitions — such as one relational database server.

> CAP is often misread. It does not say a system permanently gives up one property; it describes the choice made *during a partition*. When the network is healthy, a well-designed system provides both C and A. The later **PACELC** formulation adds: *else* (no partition), there is a trade-off between

latency and consistency.

35.5 BASE and Eventual Consistency

Many NoSQL systems adopt **BASE** instead of ACID:

Letter	Meaning
Basically **A**vailable	The system responds to every request, possibly with stale or partial data
Soft state	State may change over time even without new input, as replicas converge
Eventually consistent	If no new updates are made, all replicas eventually return the same value

Basis	ACID	BASE
Priority	Correctness	Availability and scale
Consistency	Immediate, strong	Eventual
Suited to	Money, inventory, grades	Likes, view counts, feeds, product browsing

Eventual consistency in practice. A student updates their profile photo; for a second or two, friends connected to a distant replica still see the old photo. Nobody is harmed. The same delay applied to a bank balance could permit a double withdrawal — which is why banking cores remain ACID.

Tunable consistency. Cassandra and DynamoDB let each operation choose. With N replicas, a write acknowledged by W replicas and a read consulting R replicas return the latest value whenever $\mathbf{R + W > N}$. With N = 3, choosing W = 2 and R = 2 gives strong reads at the cost of latency; W = 1 and R = 1 gives the fastest, eventually consistent operation.

Chapter Summary

- NoSQL emerged to meet requirements of massive scale, continuous availability, flexible schemas and specialized access patterns.
- Key-value stores offer fast lookup by key; document stores hold queryable JSON-like documents; column-family stores scale writes over wide rows; graph databases traverse relationships natively.
- Document design centres on embedding versus referencing; normalization trade-offs remain.
- Relational databases provide fixed schemas, SQL, joins and full ACID; NoSQL systems provide flexible schemas, database-specific APIs, multiple data models and horizontal scale.
- The CAP theorem says a partitioned distributed system must sacrifice consistency or availability; systems are classified CP or AP.
- BASE favours availability with eventual consistency; tunable systems achieve strong reads when R + W > N.
- Polyglot persistence uses several database types in one application.

Key Terms

NoSQL · Horizontal scaling · Key-value store · Document store · Collection · Document · BSON · Embedding · Referencing · Column-family store · Row key · Wide row · Graph database · Node · Edge · Traversal · Polyglot persistence · CAP theorem · Consistency · Availability · Partition tolerance · CP · AP · PACELC · BASE · Soft state · Eventual consistency · Tunable consistency · Quorum

Important Formulas / Notation

```
strong read under quorum replication:   R + W > N
```

```
    N = replicas, W = replicas acknowledging a write, R = replicas consulted on a read
```

Common Mistakes

1. Treating "NoSQL" as one technology rather than four very different models.
2. Assuming NoSQL means "no schema"; the schema moves into the application.
3. Embedding everything in documents and recreating update anomalies.
4. Reading CAP as a permanent two-out-of-three choice rather than a partition-time trade-off.
5. Confusing the C of CAP (linearizability) with the C of ACID (integrity).
6. Choosing an eventually consistent store for financial balances.
7. Believing NoSQL has replaced relational databases.

Solved Problems

Problem 35.1 — Choose a database type for each and justify: (a) login sessions; (b) product catalogue with category-specific attributes; (c) IoT sensor readings every second from 50,000 devices; (d) detecting rings of fraudulent accounts sharing phones and addresses; (e) university grades.

Solution. (a) **Key-value** (Redis) — lookup by session ID only, with expiry. (b) **Document** — each product carries its own attributes, and the catalogue is read as whole documents. (c) **Column-family** (Cassandra) — enormous write rate, partitioned by device, read by time range. (d) **Graph** — the question is about paths between accounts through shared attributes. (e) **Relational** — integrity, transactions and ad-hoc reporting matter more than scale.

Problem 35.2 — Model a student with phone numbers and enrollments as a MongoDB-style document, and state what should be embedded and what referenced.

Solution. Embed phone numbers (few per student, meaningless without the student) and enrollment entries (read together with the student). Reference courses by code rather than embedding course title and credit, since a course is shared by thousands of students and its details must be changed in one place. The document of Section 35.2 follows this design; course details live in a separate `courses` collection.

Problem 35.3 — A cluster has N = 5 replicas. Which (R, W) settings guarantee reading the latest write? Which is fastest for writes?

Solution. The condition is R + W > 5. Valid pairs include (R=3, W=3), (R=1, W=5), (R=5, W=1), (R=2, W=4), (R=4, W=2). The fastest for writes among strongly consistent options is **W = 1, R = 5** — every write needs only one acknowledgement, but every read must consult all five replicas. A balanced choice is W = 3, R = 3, which also tolerates two unavailable replicas for both reads and writes.

Problem 35.4 — Explain why a banking system is usually CP while a social media feed is usually AP.

Solution. During a partition, an AP bank would keep accepting withdrawals at both sides of the split; each side could approve a withdrawal against the same balance, and the account would be overdrawn when the partition heals — an unacceptable loss. The bank therefore prefers to reject or delay transactions (lose availability) to keep balances consistent. A social feed that shows a post a few seconds late, or a like-count briefly out of date, causes no harm, while refusing to load the feed would frustrate millions of users; the platform therefore prefers availability with eventual consistency.

Problem 35.5 — Translate the SQL query "average CGPA per department, highest first" into a MongoDB-style aggregation.

Solution.

```
db.students.aggregate([
```

```
    { $match: { cgpa: { $ne: null } } },
    { $group: { _id: "$department", avgCgpa: { $avg: "$cgpa" }, students: { $sum: 1 } } },
    { $sort:  { avgCgpa: -1 } }
  ])
```

`$match` corresponds to `WHERE`, `$group` to `GROUP BY` with aggregates, and `$sort` to `ORDER BY`. The pipeline stages map closely onto the logical processing order of SQL in Chapter 11.

Review Questions

1. Why did NoSQL databases emerge?
2. Describe the four NoSQL data models with examples and use cases.
3. Explain embedding versus referencing in document design.
4. Compare relational and NoSQL databases on at least eight criteria.
5. State the CAP theorem and explain what it does and does not claim.
6. Classify systems as CP or AP and justify.
7. Explain BASE and compare it with ACID.
8. Explain eventual consistency with an example.
9. Explain tunable consistency and the R + W > N rule.
10. What is polyglot persistence?

Multiple Choice Questions

1. Redis is primarily a (a) document (b) key-value (c) graph (d) column-family store.
2. MongoDB stores data as (a) tables (b) documents (c) edges (d) wide rows.
3. Cassandra is a (a) graph (b) column-family (c) document (d) relational database.
4. Friend-of-friend queries suit a (a) key-value (b) column-family (c) graph (d) spreadsheet.
5. In CAP, P stands for (a) performance (b) persistence (c) partition tolerance (d) primary key.
6. During a partition, an AP system sacrifices (a) availability (b) consistency (c) partition tolerance (d) durability.
7. The E in BASE means (a) exact (b) eventually consistent (c) encrypted (d) embedded.
8. Strong reads with quorum replication require (a) R + W < N (b) R + W > N (c) R = W (d) N = 1.
9. Embedding course details in every student document risks (a) phantoms (b) update anomalies (c) deadlock (d) fragmentation.
10. Using several database types in one system is (a) sharding (b) polyglot persistence (c) federation (d) replication.

Answers: 1-(b) 2-(b) 3-(b) 4-(c) 5-(c) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What does NoSQL stand for?
2. Name one key-value store.
3. What is a collection?
4. When should a child be referenced rather than embedded?
5. What is a column family?
6. What is a graph edge?
7. State the CAP theorem briefly.
8. What is soft state?

9. What is a quorum?
10. Give one use case for eventual consistency.

Long / Descriptive Questions

1. Explain the motivations for NoSQL and describe all four data models with examples, strengths and weaknesses.
2. Discuss document database design, including embedding, referencing and the persistence of normalization concerns.
3. Explain the CAP theorem in depth, including common misconceptions and PACELC.
4. Compare ACID and BASE, and explain eventual and tunable consistency with numerical examples.
5. Design a polyglot persistence architecture for a food-delivery platform, justifying each database choice.

Practical Exercises

1. Install MongoDB (or use a free cloud tier), load ten student documents, and write queries for filtering, projection, array matching and aggregation.
2. Use Redis to store sessions with expiry, and measure read latency.
3. Model the same student–course data in relational, document and graph form, and compare the query for "students who share a course with Rahim".
4. Configure a three-node Cassandra or MongoDB replica set and observe behaviour when one node is stopped.
5. Write a report comparing when your university's systems should use relational versus NoSQL databases.

CHAPTER 36

Data Warehousing

Operational databases run the organization day to day; a **data warehouse** helps it understand itself. This chapter explains why the two are separated, how data moves from one to the other, and how warehouse schemas are designed.

36.1 Definitions

A **data warehouse** is a *subject-oriented, integrated, time-variant, non-volatile* collection of data that supports management decision-making (Inmon's definition).

Property	Meaning	Example
Subject-oriented	Organized around business subjects, not applications	"Student performance", not "the admission program"
Integrated	Data from many sources is cleaned into consistent formats	Department codes unified across admission, exam and accounts
Time-variant	Historical data is kept, with time as an explicit dimension	Ten years of results for trend analysis
Non-volatile	Data is loaded and read, not updated in place	Last year's results are never edited

An **operational database** (OLTP database) supports daily transactions: enrolling a student, recording a payment, publishing a grade.

36.2 OLTP versus OLAP

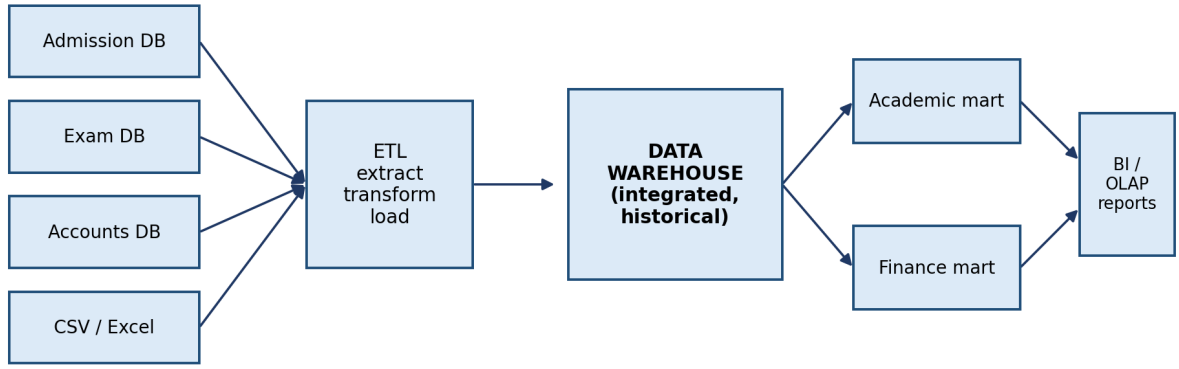
OLTP (Online Transaction Processing) systems handle many short, concurrent transactions. **OLAP** (Online Analytical Processing) systems handle fewer, much larger analytical queries.

Basis	OLTP (operational database)	OLAP (data warehouse)
Purpose	Run the business	Analyse the business
Users	Clerks, customers, applications	Managers, analysts, data scientists
Typical operation	Insert, update, delete a few rows	Read and aggregate millions of rows
Query example	"Record grade A for student 1901001"	"Average CGPA by department and year, 2015–2025"
Data	Current, detailed	Historical, summarized and detailed
Schema design	Normalized (3NF)	Denormalized (star / snowflake)
Transaction size	Small	Large, read-only
Response time	Milliseconds	Seconds to minutes
Concurrency	Thousands of users	Tens of users
Size	Gigabytes	Terabytes to petabytes
Updates	Continuous	Periodic batch loads

Why not run analytics on the operational database directly? Three reasons: analytical queries scanning millions of rows would slow down the transactions that run the organization; the operational database usually holds only current data, not history; and the data needed is often spread across several incompatible

systems.

36.3 Warehouse Architecture



operational sources → cleaned, integrated, time-stamped copy → subject-specific marts → analysis

Figure 36.1: Data warehouse architecture

Layer	Role
Source systems	Operational databases, spreadsheets, external data
Staging area	Temporary storage where extracted data is cleaned
ETL	Extract, transform, load
Warehouse	Integrated, historical, subject-oriented store
Data marts	Subject- or department-specific subsets
Front-end tools	Reports, dashboards, OLAP tools, data mining

36.4 ETL — Extract, Transform, Load

Stage	Work	University example
Extract	Read data from source systems, fully or incrementally	Pull new results from the exam system every night
Transform	Clean, standardize, deduplicate, integrate, derive, aggregate	Map `CSE`, `Cse`, `CS&E` to `CSE`; convert marks to grade points; remove duplicate students; add surrogate keys
Load	Insert into the warehouse, maintaining history	Append new fact rows; update dimensions

```sql
-- a simple transform-and-load step (MySQL)
INSERT INTO dw.fact_result (date_key, student_key, course_key, dept_key, marks,
    ↪ grade_point)
SELECT d.date_key, s.student_key, c.course_key, dp.dept_key,
       r.marks,
       CASE WHEN r.marks >= 80 THEN 4.00 WHEN r.marks >= 75 THEN 3.75
            WHEN r.marks >= 70 THEN 3.50 WHEN r.marks >= 65 THEN 3.25
            WHEN r.marks >= 60 THEN 3.00 WHEN r.marks >= 55 THEN 2.75
            WHEN r.marks >= 50 THEN 2.50 WHEN r.marks >= 40 THEN 2.00 ELSE 0.00 END
FROM   university_db.result r
JOIN   university_db.examination x ON x.exam_id = r.exam_id
JOIN   dw.dim_date     d  ON d.full_date   = x.exam_date
JOIN   dw.dim_student  s  ON s.student_id  = r.student_id AND s.is_current = 1
```

```
  JOIN   dw.dim_course   c  ON c.course_id   = x.course_id
  JOIN   dw.dim_department dp ON dp.dept_id  = s.dept_id
  WHERE  x.exam_date > (SELECT MAX(full_date) FROM dw.load_log);    -- incremental extract
```

ELT reverses the last two steps: raw data is loaded into a powerful cloud warehouse first and transformed there with SQL — the common pattern with systems such as BigQuery, Snowflake and Redshift.

36.5 Data Marts

A **data mart** is a smaller warehouse focused on one subject or department.

Type	Built from	Advantage
Dependent	The central warehouse	Consistent with the enterprise view
Independent	Source systems directly	Quick to build, but risks inconsistent numbers across departments

A university might have an *academic mart* (enrolment, results, graduation rates) for the registrar and a *finance mart* (fees, scholarships, payroll) for the accounts office.

36.6 Fact and Dimension Tables

Table	Contains	Characteristics	Example
Fact table	Numeric **measures** of business events, plus foreign keys to dimensions	Very many rows; narrow; grows continuously	`fact_result(date_key, student_key, course_key, marks, grade_point)`
Dimension table	Descriptive **context** for facts	Fewer rows; wide; textual attributes	`dim_course(course_key, code, title, credit, level, department)`

The **grain** of a fact table is the level of detail one row represents — "one row per student per course per exam". Choosing the grain is the first and most important decision in warehouse design.

Measure type	Can be summed across	Example
Additive	All dimensions	Sales amount, quantity
Semi-additive	Some dimensions, not time	Account balance (sum across branches, not across months)
Non-additive	None — must be recomputed	Ratios such as percentage, CGPA

Slowly changing dimensions. When a dimension attribute changes — a student transfers department — the warehouse must decide whether to overwrite it (**Type 1**, history lost), add a new dimension row with validity dates (**Type 2**, history kept — the `is_current` flag in the ETL above), or keep old and new values in separate columns (**Type 3**).

36.7 Star Schema

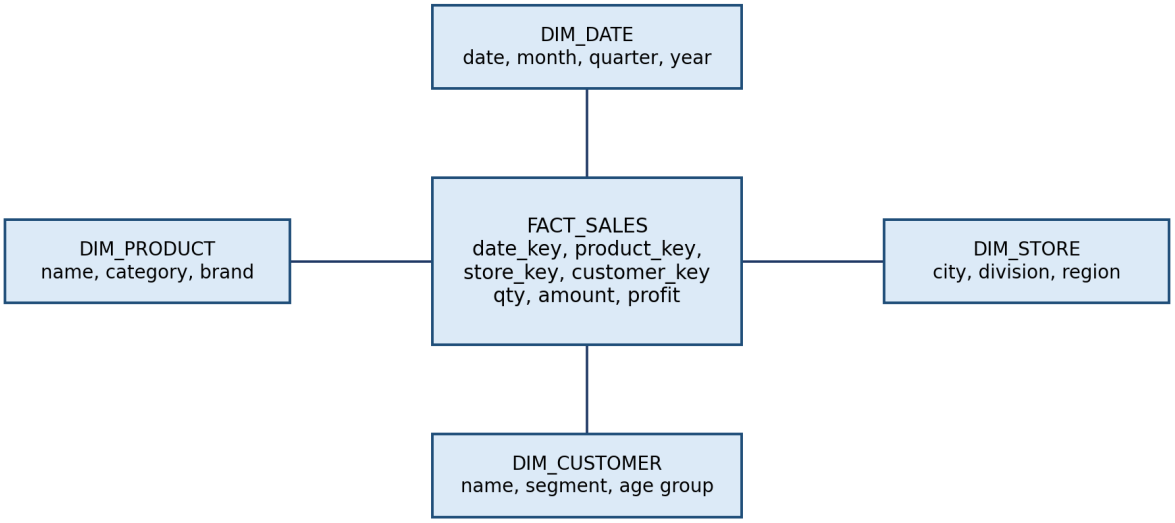


Figure 36.2: A star schema for sales analysis

One central fact table joins directly to a set of **denormalized** dimension tables.

```
CREATE TABLE dim_date (
    date_key INT PRIMARY KEY, full_date DATE, day TINYINT, month TINYINT,
    month_name VARCHAR(10), quarter TINYINT, year SMALLINT
);
CREATE TABLE dim_product (
    product_key INT PRIMARY KEY, product_name VARCHAR(100),
    category VARCHAR(50), brand VARCHAR(50)
);
CREATE TABLE dim_store (
    store_key INT PRIMARY KEY, store_name VARCHAR(80),
    city VARCHAR(40), division VARCHAR(40)
);
CREATE TABLE dim_customer (
    customer_key INT PRIMARY KEY, customer_name VARCHAR(100),
    segment VARCHAR(30), age_group VARCHAR(20)
);
CREATE TABLE fact_sales (
    date_key INT, product_key INT, store_key INT, customer_key INT,
    quantity INT, amount DECIMAL(12,2), profit DECIMAL(12,2),
    PRIMARY KEY (date_key, product_key, store_key, customer_key),
    FOREIGN KEY (date_key)     REFERENCES dim_date(date_key),
    FOREIGN KEY (product_key)  REFERENCES dim_product(product_key),
    FOREIGN KEY (store_key)    REFERENCES dim_store(store_key),
    FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key)
);
```

A typical analytical query joins the fact table to a few dimensions and aggregates:

```
SELECT d.year, p.category, s.division, SUM(f.amount) AS revenue
FROM   fact_sales f
JOIN   dim_date d    ON d.date_key    = f.date_key
JOIN   dim_product p ON p.product_key = f.product_key
JOIN   dim_store s   ON s.store_key   = f.store_key
WHERE  d.year BETWEEN 2023 AND 2025
```

```
GROUP BY d.year, p.category, s.division
ORDER BY d.year, revenue DESC;
```

36.8 Snowflake Schema

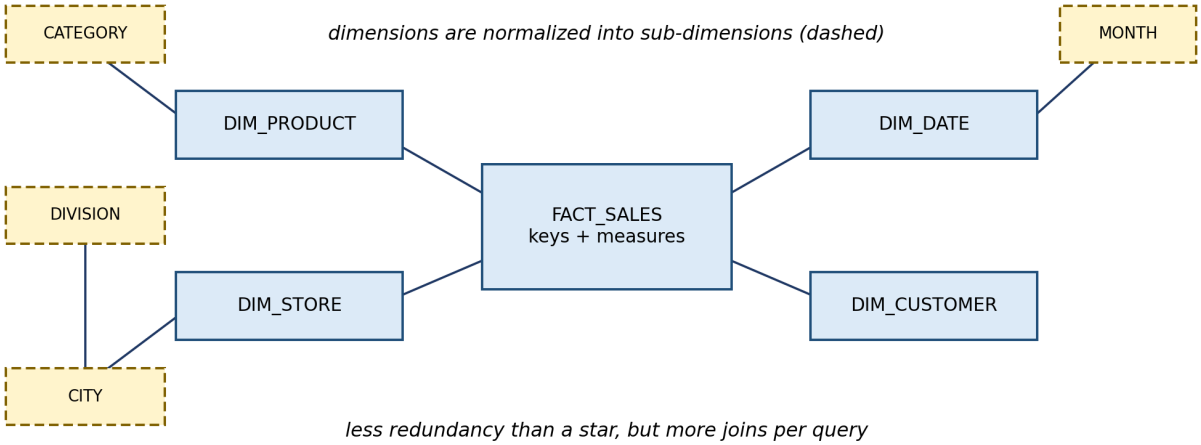


Figure 36.3: A snowflake schema — dimensions normalized into sub-dimensions

The dimensions are **normalized**: `dim_product` references `dim_category`; `dim_store` references `dim_city`, which references `dim_division`.

Basis	Star schema	Snowflake schema
Dimension tables	Denormalized	Normalized
Number of tables	Fewer	More
Joins per query	Fewer	More
Query performance	Usually faster	Usually slower
Storage	More redundancy	Less redundancy
Maintenance of dimensions	Harder (repeated values)	Easier
Understandability	Simple	More complex
Typical use	Most warehouses and BI tools	Very large or frequently changing dimensions

A **galaxy (fact constellation)** schema has several fact tables — for example `fact_sales` and `fact_inventory` — sharing conformed dimensions such as `dim_date` and `dim_product`.

Chapter Summary

- A data warehouse is subject-oriented, integrated, time-variant and non-volatile, and supports decision-making.
- OLTP runs the business with many small, current, normalized transactions; OLAP analyses it with large read-only queries over historical, denormalized data.
- ETL extracts, cleans and transforms, and loads data; ELT transforms after loading in the warehouse.
- Data marts are subject-specific subsets, dependent or independent.
- Fact tables hold additive, semi-additive or non-additive measures at a chosen grain; dimension tables hold descriptive context; slowly changing dimensions preserve history.
- A star schema joins a fact table to denormalized dimensions; a snowflake schema normalizes dimensions at the cost of extra joins.

Key Terms

Data warehouse · Subject-oriented · Integrated · Time-variant · Non-volatile · Operational database · OLTP · OLAP · Staging area · ETL · ELT · Data mart · Dependent / independent mart · Fact table · Measure · Additive / semi-additive / non-additive · Grain · Dimension table · Surrogate key · Slowly changing dimension · Star schema · Snowflake schema · Galaxy schema · Conformed dimension

Common Mistakes

1. Running heavy analytics directly on the OLTP database.
2. Normalizing a warehouse to 3NF by habit, forcing many joins per query.
3. Summing a semi-additive measure such as a balance across time.
4. Averaging averages instead of recomputing from sums and counts.
5. Leaving the grain of a fact table undefined.
6. Overwriting dimension history when trends over time matter.
7. Building independent marts that report different numbers for the same concept.

Solved Problems

Problem 36.1 — Classify each as OLTP or OLAP: (a) booking an airline seat; (b) revenue per route per quarter for five years; (c) updating a patient's address; (d) identifying the ten slowest-selling products by region.

Solution. (a) OLTP. (b) OLAP. (c) OLTP. (d) OLAP.

Problem 36.2 — Design a star schema for university result analysis.

Solution. Grain: one row per student per course per examination. Fact: `fact_result(date_key, student_key, course_key, dept_key, instructor_key, marks, grade_point, credit_earned)`. Dimensions: `dim_date(date_key, full_date, semester, academic_year)`, `dim_student(student_key, student_id, name, gender, admission_year, dept_id, is_current)` as a Type 2 dimension, `dim_course(course_key, code, title, credit, level)`, `dim_department(dept_key, dept_id, dept_name, faculty)`, `dim_instructor(instructor_key, name, designation)`. Measures: `marks` and `credit_earned` are additive; `grade_point` must be combined as a credit-weighted average — non-additive.

Problem 36.3 — Why is account balance semi-additive, and how should the total balance for March be computed?

Solution. Summing balances across branches at one moment gives the bank's total holdings — meaningful. Summing the same account's balance across the 31 days of March gives 31 times the balance — meaningless. Balance can therefore be summed across every dimension except time. The March total is computed by taking each account's balance at a single point — typically the last day of the month — and summing across accounts: `SUM(balance) WHERE date = '2026-03-31'`.

Problem 36.4 — A student transfers from EEE to CSE in 2024. Show how a Type 2 dimension records this and why it matters.

Solution.

student_key	student_id	dept_id	valid_from	valid_to	is_current
501	ASH2001002	EEE	2020-01-12	2024-01-31	0
877	ASH2001002	CSE	2024-02-01	9999-12-31	1

Results from 2020–2023 reference key 501 and are counted under EEE; results from 2024 onward ref-

erence key 877 and are counted under CSE. A Type 1 overwrite would have moved four years of EEE results into CSE, falsifying every historical departmental statistic.

Problem 36.5 — Compare star and snowflake schemas for a product dimension with 2 million products in 40 categories.

Solution. In a star schema the category name and description are repeated in 2 million rows; renaming a category means updating tens of thousands of rows, but queries grouping by category need one join. In a snowflake schema categories live in a 40-row table; renaming touches one row and storage shrinks, but every category-level query needs an extra join. Since dimension tables are usually tiny compared with fact tables, and analytical query speed dominates, most designers keep the **star** — accepting the redundancy — unless the dimension changes very frequently.

Review Questions

1. Define a data warehouse and explain its four properties.
2. Compare OLTP and OLAP on at least eight criteria.
3. Why should analytics not run on the operational database?
4. Describe data warehouse architecture layer by layer.
5. Explain ETL with examples of transformations, and contrast it with ELT.
6. Define data marts and distinguish dependent from independent marts.
7. Explain fact and dimension tables, grain, and types of measure.
8. What is a slowly changing dimension? Explain Types 1, 2 and 3.
9. Draw and explain a star schema with SQL.
10. Compare star and snowflake schemas.

Multiple Choice Questions

1. “Non-volatile” in a warehouse means data is (a) encrypted (b) not updated in place (c) in memory (d) deleted daily.
2. OLTP systems are optimized for (a) large scans (b) many short transactions (c) historical trends (d) data mining.
3. The T in ETL stands for (a) transfer (b) transform (c) transaction (d) table.
4. A fact table mainly contains (a) descriptions (b) measures and foreign keys (c) only text (d) indexes.
5. Account balance is a(n) (a) additive (b) semi-additive (c) non-additive (d) dimension attribute.
6. A star schema’s dimensions are (a) normalized (b) denormalized (c) absent (d) fact tables.
7. A snowflake schema requires (a) fewer joins (b) more joins (c) no fact table (d) no keys.
8. Keeping history of dimension changes with new rows is SCD (a) Type 1 (b) Type 2 (c) Type 3 (d) Type 0.
9. A department-specific subset of a warehouse is a (a) data lake (b) data mart (c) cube (d) view.
10. The level of detail of a fact row is its (a) cardinality (b) grain (c) degree (d) key.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What does time-variant mean?
2. Give one OLAP query.
3. What is a staging area?
4. Define grain.

5. What is a conformed dimension?
6. Why are CGPA values non-additive?
7. What is ELT?
8. Name the three SCD types.
9. What is a galaxy schema?
10. Why use surrogate keys in dimensions?

Long / Descriptive Questions

1. Explain the data warehouse concept, its properties and its architecture with a diagram.
2. Compare OLTP and OLAP in depth and justify separating them.
3. Explain the ETL process with detailed transformation examples and SQL.
4. Explain fact and dimension tables, grain, measure types and slowly changing dimensions.
5. Design star and snowflake schemas for a retail chain, write the SQL, and compare them.

Practical Exercises

1. Build a star schema for university results and load it from `university_db` with an ETL script.
2. Write five analytical queries against the star schema (by year, department, course level, instructor, and combinations).
3. Implement a Type 2 slowly changing dimension for student department transfers.
4. Convert the star schema into a snowflake schema and compare the query for revenue by division.
5. Measure the difference in execution time between an analytical query on the normalized OLTP schema and on the star schema.

CHAPTER 37

OLAP

OLAP (Online Analytical Processing) lets analysts explore multidimensional data interactively — summarizing, drilling into detail, and viewing it from different angles. Its central abstraction is the **data cube**.

37.1 The Data Cube

A **data cube** organizes a measure (such as sales amount) along several **dimensions** (such as time, product and store). Each cell holds the aggregated measure for one combination of dimension values.

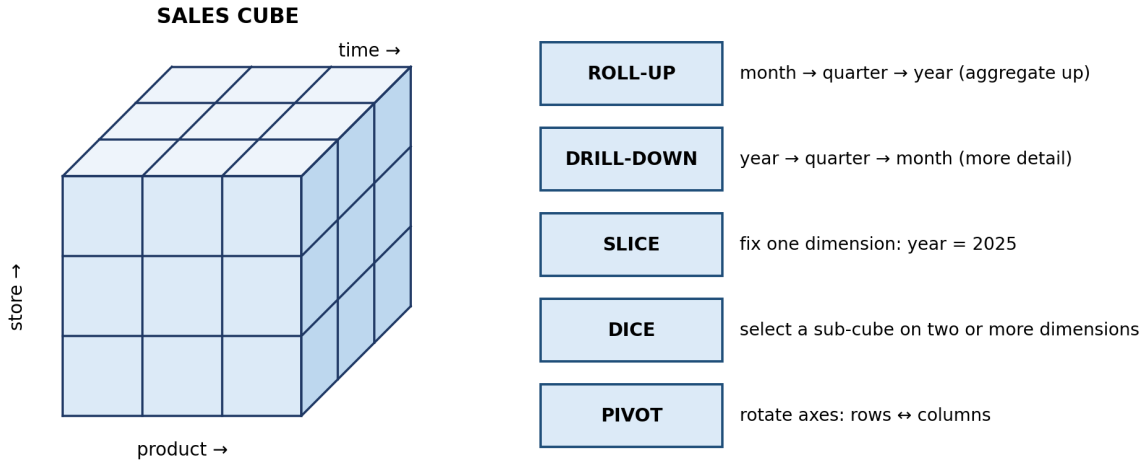


Figure 37.1: A sales cube and the five OLAP operations

Dimensions are organized into **hierarchies**:

```
Time:     day → month → quarter → year
Product:  item → brand → category
Store:    store → city → division → country
```

Running example: a sales database

year	quarter	division	category	amount (thousand taka)
2025	Q1	Dhaka	Electronics	520
2025	Q1	Dhaka	Clothing	310
2025	Q1	Chattogram	Electronics	280
2025	Q1	Chattogram	Clothing	190
2025	Q2	Dhaka	Electronics	610
2025	Q2	Dhaka	Clothing	290
2025	Q2	Chattogram	Electronics	300
2025	Q2	Chattogram	Clothing	210

37.2 Roll-Up

Roll-up (drill-up, consolidation) aggregates to a *coarser* level — either by climbing a hierarchy or by removing a dimension.

Example: from quarter to year, across all divisions and categories.

```
SELECT d.year, SUM(f.amount) AS revenue
FROM   fact_sales f JOIN dim_date d ON d.date_key = f.date_key
GROUP  BY d.year;
```

year	revenue
2025	2,710

Interpretation: the board sees one annual figure instead of eight detailed rows.

37.3 Drill-Down

Drill-down is the reverse: it moves to *finer* detail — down a hierarchy or by adding a dimension.

Example: from year to quarter, then to division.

```
SELECT d.year, d.quarter, s.division, SUM(f.amount) AS revenue
FROM   fact_sales f
JOIN   dim_date d  ON d.date_key  = f.date_key
JOIN   dim_store s ON s.store_key = f.store_key
GROUP  BY d.year, d.quarter, s.division
ORDER  BY d.year, d.quarter, s.division;
```

year	quarter	division	revenue
2025	Q1	Chattogram	470
2025	Q1	Dhaka	830
2025	Q2	Chattogram	510
2025	Q2	Dhaka	900

Interpretation: a manager who saw that annual revenue fell short drills down to discover which quarter and which division caused it.

37.4 Slice

A **slice** fixes **one** dimension at a single value, producing a sub-cube with one fewer dimension.

Example: slice on `category = 'Electronics'`.

```
SELECT d.quarter, s.division, SUM(f.amount) AS revenue
FROM   fact_sales f
JOIN   dim_date d    ON d.date_key    = f.date_key
JOIN   dim_store s   ON s.store_key   = f.store_key
JOIN   dim_product p ON p.product_key = f.product_key
WHERE  p.category = 'Electronics'
GROUP  BY d.quarter, s.division;
```

quarter	division	revenue
Q1	Dhaka	520

quarter	division	revenue
Q1	Chattogram	280
Q2	Dhaka	610
Q2	Chattogram	300

37.5 Dice

A **dice** selects a sub-cube by specifying ranges or sets of values on **two or more** dimensions.

Example: Q2 only, Dhaka only, both categories.

```
SELECT p.category, SUM(f.amount) AS revenue
FROM   fact_sales f
JOIN   dim_date d    ON d.date_key    = f.date_key
JOIN   dim_store s   ON s.store_key   = f.store_key
JOIN   dim_product p ON p.product_key = f.product_key
WHERE  d.quarter = 2 AND s.division = 'Dhaka'
GROUP  BY p.category;
```

category	revenue
Electronics	610
Clothing	290

Basis	Slice	Dice
Dimensions restricted	One, to a single value	Two or more, to values or ranges
Result	Cube with one fewer dimension	Smaller cube with the same number of dimensions
SQL analogue	`WHERE` on one dimension	`WHERE` on several dimensions

37.6 Pivot

Pivot (rotate) re-orients the view — swapping rows and columns — without changing the data.

Example: divisions as columns, categories as rows.

```
SELECT p.category,
       SUM(CASE WHEN s.division = 'Dhaka'      THEN f.amount ELSE 0 END) AS dhaka,
       SUM(CASE WHEN s.division = 'Chattogram' THEN f.amount ELSE 0 END) AS chattogram
FROM   fact_sales f
JOIN   dim_store s   ON s.store_key   = f.store_key
JOIN   dim_product p ON p.product_key = f.product_key
GROUP  BY p.category;
```

category	dhaka	chattogram
Electronics	1,130	580
Clothing	600	400

This is the conditional-aggregation idiom of Section 12.5, used to turn row values into columns. SQL Server and Oracle provide a dedicated `PIVOT` operator.

37.7 SQL Extensions for OLAP

ROLLUP

Produces subtotals at each level of a hierarchy, plus a grand total, in one query:

```sql
SELECT s.division, p.category, SUM(f.amount) AS revenue
FROM   fact_sales f
JOIN   dim_store s   ON s.store_key   = f.store_key
JOIN   dim_product p ON p.product_key = f.product_key
GROUP  BY s.division, p.category WITH ROLLUP;          -- MySQL
-- GROUP BY ROLLUP (s.division, p.category)            -- PostgreSQL, SQL Server, Oracle
```

division	category	revenue
Chattogram	Clothing	400
Chattogram	Electronics	580
Chattogram	NULL	980
Dhaka	Clothing	600
Dhaka	Electronics	1,130
Dhaka	NULL	1,730
NULL	NULL	2,710

The NULL rows are subtotals: division totals and the grand total. `GROUPING(column)` returns 1 on those rows, distinguishing a subtotal NULL from a genuine NULL in the data.

CUBE

Produces subtotals for **every** combination of the grouping columns — 2^n groupings for n columns (PostgreSQL, SQL Server, Oracle):

```sql
SELECT division, category, SUM(amount)
FROM   sales_view
GROUP  BY CUBE (division, category);
-- (division, category), (division), (category), ()
```

Window functions

Rank and running totals without collapsing rows:

```sql
SELECT division, quarter, revenue,
       SUM(revenue) OVER (PARTITION BY division ORDER BY quarter) AS running_total,
       RANK()       OVER (PARTITION BY quarter ORDER BY revenue DESC) AS rank_in_quarter
FROM   quarterly_division_sales;
```

37.8 OLAP Server Architectures

Type	Storage	Strength	Weakness
ROLAP (relational)	Star schema in a relational DBMS	Scales to very large data; uses SQL	Slower for complex aggregations
MOLAP (multidimensional)	Pre-computed arrays (cubes)	Very fast queries	Limited scale; long cube-build times; sparse data wastes space
HOLAP (hybrid)	Detail in relational, aggregates in cubes	Balance of both	More complex

Modern cloud warehouses with **columnar storage** — which stores each column contiguously, so an aggregate reads only the columns it needs — have made ROLAP fast enough for most interactive use.

Chapter Summary

- OLAP explores measures along dimensions organized into hierarchies, conceptually a data cube.
- Roll-up aggregates to coarser levels; drill-down moves to finer detail.
- Slice fixes one dimension; dice restricts two or more.
- Pivot rotates the view, swapping rows and columns.
- SQL supports OLAP through `GROUP BY ROLLUP`, `CUBE`, conditional aggregation and window functions.
- ROLAP, MOLAP and HOLAP trade query speed against scale; columnar storage makes ROLAP fast.

Key Terms

OLAP · Data cube · Dimension · Measure · Hierarchy · Roll-up · Drill-down · Slice · Dice · Pivot · `ROLLUP` · `CUBE` · `GROUPING` · Window function · `PARTITION BY` · ROLAP · MOLAP · HOLAP · Columnar storage

Common Mistakes

1. Confusing slice (one dimension) with dice (several dimensions).
2. Believing pivot changes the data; it changes only the presentation.
3. Misreading `ROLLUP` subtotal NULLs as missing data; use `GROUPING()`.
4. Rolling up non-additive measures by summing them.
5. Expecting `CUBE` in MySQL, which supports only `WITH ROLLUP`.

Solved Problems

Problem 37.1 — Identify the operation: (a) viewing yearly totals after quarterly; (b) viewing only 2025 data; (c) viewing Dhaka electronics in Q1–Q2; (d) switching divisions from rows to columns; (e) moving from category to individual product.

Solution. (a) Roll-up. (b) Slice. (c) Dice. (d) Pivot. (e) Drill-down.

Problem 37.2 — Using the running example, compute the roll-up to division level.

Solution. Dhaka = 520 + 310 + 610 + 290 = **1,730**. Chattogram = 280 + 190 + 300 + 210 = **980**. Grand total = **2,710**.

Problem 37.3 — How many groupings does `GROUP BY CUBE(year, division, category)` produce, and list them.

Solution. $2^3 = \mathbf{8}$: (year, division, category), (year, division), (year, category), (division, category), (year), (division), (category), and () — the grand total.

Problem 37.4 — Write a query ranking divisions by revenue within each quarter.

Solution.

```
WITH q AS (
  SELECT d.quarter, s.division, SUM(f.amount) AS revenue
  FROM   fact_sales f
  JOIN   dim_date d  ON d.date_key  = f.date_key
  JOIN   dim_store s ON s.store_key = f.store_key
```

```
  GROUP  BY d.quarter, s.division
)
SELECT quarter, division, revenue,
       RANK() OVER (PARTITION BY quarter ORDER BY revenue DESC) AS rnk
FROM   q ORDER BY quarter, rnk;
```

Dhaka ranks first in both quarters (830, 900) and Chattogram second (470, 510).

Problem 37.5 — Explain why average CGPA cannot be rolled up by averaging departmental averages.

Solution. Suppose CSE has 100 students averaging 3.6 and ME has 10 averaging 3.0. The average of the averages is 3.3, but the true university average is (100 × 3.6 + 10 × 3.0) / 110 = 3.545. Averages are non-additive: the correct roll-up stores the additive components — sum of CGPAs and count of students — and divides only at the final level.

Review Questions

1. Define a data cube, dimensions, measures and hierarchies.
2. Explain roll-up and drill-down with examples and SQL.
3. Explain slice and dice and distinguish them.
4. Explain pivot and write SQL to perform it.
5. Explain `ROLLUP` and interpret its NULL rows.
6. Explain `CUBE` and the number of groupings it produces.
7. Show how window functions support ranking and running totals.
8. Compare ROLAP, MOLAP and HOLAP.
9. Why is columnar storage beneficial for OLAP?
10. Why must non-additive measures be handled carefully in roll-ups?

Multiple Choice Questions

1. Moving from month to year is (a) drill-down (b) roll-up (c) slice (d) pivot.
2. Fixing a single dimension value is (a) dice (b) slice (c) pivot (d) roll-up.
3. Restricting two or more dimensions is (a) slice (b) dice (c) drill-down (d) pivot.
4. Rotating rows and columns is (a) pivot (b) roll-up (c) dice (d) slice.
5. `GROUP BY CUBE(a, b)` produces how many groupings? (a) 2 (b) 3 (c) 4 (d) 8.
6. In `ROLLUP` output, a subtotal row shows (a) 0 (b) NULL in rolled-up columns (c) an error (d) duplicates.
7. MOLAP stores data in (a) relational tables (b) multidimensional arrays (c) documents (d) graphs.
8. MySQL supports (a) `CUBE` (b) `WITH ROLLUP` (c) `PIVOT` (d) all of these.
9. Drill-down moves to (a) coarser (b) finer (c) the same (d) no level of detail.
10. Columnar storage speeds aggregates because it (a) compresses indexes (b) reads only needed columns (c) avoids joins entirely (d) caches rows.

Answers: 1-(b) 2-(b) 3-(b) 4-(a) 5-(c) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What is a dimension hierarchy?
2. Define roll-up.
3. Define drill-down.
4. Distinguish slice from dice.

5. What does `GROUPING()` return?
6. How many groupings does `CUBE` on three columns give?
7. What is ROLAP?
8. What is a running total?
9. Give the SQL idiom for pivoting in MySQL.
10. Why are averages non-additive?

Long / Descriptive Questions

1. Explain the data cube and all five OLAP operations with a sales example, SQL and results.
2. Explain SQL OLAP extensions — `ROLLUP`, `CUBE`, `GROUPING` and window functions — with examples.
3. Compare ROLAP, MOLAP and HOLAP architectures.
4. Discuss the handling of additive, semi-additive and non-additive measures in OLAP.
5. Design an OLAP analysis for university results, describing the cube, hierarchies and typical operations.

Practical Exercises

1. Build the sales star schema, load the running-example data, and perform each OLAP operation in SQL.
2. Use `WITH ROLLUP` to produce division and category subtotals, and use `GROUPING()` to label them.
3. Pivot sales by quarter into columns using conditional aggregation.
4. Use window functions to compute running totals and ranks by division.
5. Build a pivot table over the same data in a spreadsheet and compare it with the SQL result.

CHAPTER 38

Modern Database Systems

The principles of the previous 37 chapters — the relational model, normalization, transactions, recovery — remain the foundation. What has changed in the last fifteen years is *where* databases run, *how large* they grow, and *what kinds of data* they hold. This chapter introduces the main developments at an undergraduate level, distinguishing clearly between fundamental ideas and particular products.

38.1 Scalability: Vertical and Horizontal

Scalability is the ability of a system to handle growing workload by adding resources.

Basis	Vertical scaling (scale up)	Horizontal scaling (scale out)
Method	A bigger machine: more CPU, RAM, faster disks	More machines sharing the load
Limit	The largest available server	Practically unlimited
Application changes	None	Often significant
Cost curve	Rises steeply at the high end	Roughly linear with commodity hardware
Failure impact	One machine is a single point of failure	Nodes can fail individually
Typical for	Traditional relational databases	NoSQL, distributed SQL, cloud databases

Read-heavy workloads scale out easily with **replicas**; write-heavy workloads require **sharding**.

38.2 Database Replication

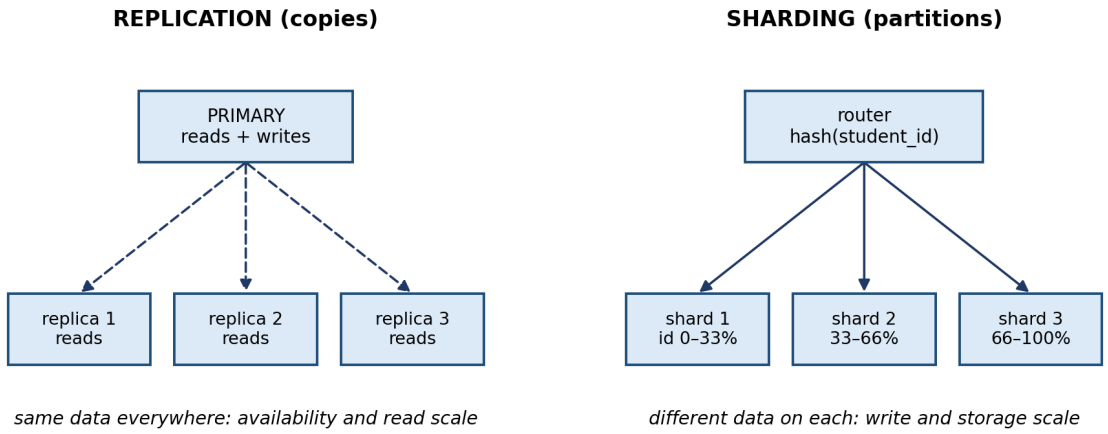


Figure 38.1: Replication copies data; sharding partitions it

Replication maintains copies of the same data on several servers (Section 34.5). In the common **primary–replica** arrangement, writes go to the primary and are streamed to replicas, which serve reads.

```
-- MySQL 8: on a replica, connect to the primary and start replicating
CHANGE REPLICATION SOURCE TO
```

```
    SOURCE_HOST = 'db-primary.university.local',
    SOURCE_USER = 'repl_user',
    SOURCE_PASSWORD = '********',
    SOURCE_AUTO_POSITION = 1;          -- GTID-based positioning
START REPLICA;
SHOW REPLICA STATUS\G                  -- check Seconds_Behind_Source for lag
```

Benefit	Cost or risk
Read scalability	Replication lag: replicas may be slightly behind
High availability through failover	Writes still limited by one primary
Offloading backups and reports to a replica	More servers to manage and monitor
Geographic distribution	Asynchronous replicas may lose recent writes on failover

An application reading its own write from a lagging replica may not see it — the **read-your-writes** problem — so reads that must be fresh are sent to the primary.

38.3 Sharding

Sharding is horizontal partitioning across servers: each **shard** holds a disjoint subset of rows, chosen by a **shard key**.

Strategy	Rule	Advantage	Disadvantage
Range	`student_id` 0–99,999 → shard 1, …	Range queries stay on one shard	Hot spots if new rows cluster at one end
Hash	`hash(student_id) mod N`	Even distribution	Range queries hit every shard; re-sharding is expensive
Directory	A lookup table maps keys to shards	Flexible placement	Lookup service is a dependency
Geographic	By region or country	Data near its users; data-residency compliance	Uneven shard sizes

Choosing the shard key is the most consequential decision: queries that include the key go to one shard; queries without it must be **scattered** to every shard and **gathered**. Cross-shard joins and transactions are expensive, which is why sharding is adopted only when a single primary can no longer keep up.

Basis	Replication	Sharding	Partitioning (single server)
Data on each node	All of it	A disjoint subset	A disjoint subset, same server
Scales	Reads	Reads and writes, storage	Manageability, query pruning
Main purpose	Availability, read load	Capacity	Maintenance and performance

38.4 High Availability

High availability (HA) is the ability of a system to remain operational despite component failures. It is expressed as a percentage of uptime:

Availability	Downtime per year
99 % (“two nines”)	3.65 days
99.9 %	8.76 hours
99.99 %	52.6 minutes
99.999 % (“five nines”)	5.26 minutes

HA techniques:

- **Redundancy** at every layer — RAID, replicas, multiple application servers, redundant network paths.
- **Automatic failover** — when the primary fails, a replica is promoted and clients are redirected (MySQL InnoDB Cluster, PostgreSQL with Patroni, cloud-managed failover).
- **Health checks and fencing** — ensuring a failed primary cannot keep accepting writes after a replica has been promoted, which would cause **split-brain**.
- **Multi-zone and multi-region deployment** — surviving the loss of a whole data centre.

Two targets frame HA planning: the **recovery point objective (RPO)** — how much data may be lost — and the **recovery time objective (RTO)** — how long recovery may take. Synchronous replication gives RPO = 0; asynchronous replication gives RPO of seconds.

38.5 Cloud Databases

A **cloud database** runs on infrastructure operated by a cloud provider.

Model	Provider manages	Examples
Self-managed on cloud VMs (IaaS)	Hardware only	MySQL installed on a virtual machine
Database-as-a-Service (DBaaS)	Installation, patching, backups, replication, failover	Amazon RDS, Azure Database for MySQL/PostgreSQL, Google Cloud SQL
Serverless	Also capacity; scales to zero when idle	Aurora Serverless, Azure SQL serverless, Neon
Cloud data warehouse	Storage and compute separated; elastic analytics	Snowflake, BigQuery, Redshift

Advantages	Considerations
No hardware to buy; pay per use	Ongoing cost can exceed on-premises at steady large scale
Automated backups, patching, HA	Less control over configuration
Elastic scaling	Data residency and regulatory requirements
Global deployment in minutes	Network latency from on-premises applications; vendor lock-in

A key architectural idea in modern cloud databases is **separation of storage and compute**: data lives in durable shared storage, and any number of compute nodes can be started to query it, so analytics can scale independently of storage.

38.6 NewSQL and Distributed SQL

For years the choice seemed binary: relational databases with ACID but limited scale-out, or NoSQL with scale-out but weak consistency. **NewSQL** describes systems that provide **SQL and full ACID transactions** while **scaling horizontally** like NoSQL systems. **Distributed SQL** is the current name for the most prominent group.

How they do it:

- Data is automatically split into ranges and distributed across nodes.
- Each range is replicated, typically three times, using a **consensus protocol** (Raft or Paxos), so a write commits once a majority of replicas agree — no blocking two-phase commit coordinator.

- Distributed transactions use consensus plus carefully synchronized timestamps to provide serializable isolation.

System	Notes
Google Spanner	Globally distributed; uses atomic clocks and GPS (TrueTime) to bound clock uncertainty
CockroachDB	PostgreSQL-compatible wire protocol
YugabyteDB	PostgreSQL-compatible query layer
TiDB	MySQL-compatible

The trade-off is latency: a commit requires a round trip to a majority of replicas, which may be in other zones or regions.

38.7 In-Memory Databases

An **in-memory database** keeps its primary copy of data in RAM rather than on disk, eliminating disk I/O from the query path and achieving microsecond latencies.

Aspect	Explanation
Performance	No buffer-pool misses; data structures optimized for memory, not disk blocks
Durability	Achieved through a transaction log on disk, periodic snapshots, and/or replication
Capacity	Limited and expensive per gigabyte compared with disk
Examples	Redis, SAP HANA, VoltDB, MySQL `MEMORY` engine (non-durable), SQL Server In-Memory OLTP
Uses	Caching, session stores, real-time analytics, trading systems, leaderboards

38.8 Graph Databases

Introduced in Section 35.2, graph databases store nodes and edges natively and follow relationships by pointer rather than by join. The cost of a traversal depends on the number of relationships actually followed, not on the total size of the database — the property called **index-free adjacency**.

A relational query for "friends of friends of friends" requires three self-joins on a friendship table, each growing in cost with table size; a graph query follows edges from one starting node.

Uses: social networks, recommendation ("customers who bought this also bought"), fraud detection, supply-chain and network analysis, knowledge graphs. SQL:2023 added **property graph queries (SQL/PGQ)**, bringing graph pattern matching into the SQL standard.

38.9 Spatial Databases

A **spatial database** stores and queries geometric data — points, lines and polygons — with spatial indexes and functions.

```
-- PostgreSQL + PostGIS: nearest five hospitals to a location in Dhaka
SELECT name,
       ST_Distance(location::geography, ST_MakePoint(90.4125, 23.8103)::geography) AS
    ↪ metres
FROM   hospital
ORDER  BY location <-> ST_SetSRID(ST_MakePoint(90.4125, 23.8103), 4326)
LIMIT  5;
```

Feature	Explanation
Geometry types	`POINT`, `LINESTRING`, `POLYGON`, and collections
Spatial predicates	`ST_Contains`, `ST_Intersects`, `ST_Within`, `ST_DWithin`
Spatial indexes	**R-trees** (or GiST in PostgreSQL) index bounding rectangles
Uses	Ride-hailing, food delivery radius, land records, flood mapping, logistics

MySQL 8 includes spatial types and R-tree indexes; PostGIS is the most complete open-source implementation.

38.10 Temporal Databases

A **temporal database** records how data changes over time rather than only its current state. Two time dimensions are distinguished:

Time	Meaning	Example
Valid time (application time)	When a fact is true in the real world	A student belonged to EEE from 2020 to January 2024
Transaction time (system time)	When the fact was recorded in the database	The transfer was entered on 3 February 2024

A **bitemporal** table tracks both. SQL:2011 standardized **system-versioned tables**:

```sql
-- SQL Server / MariaDB syntax for a system-versioned table
CREATE TABLE salary (
    instructor_id INT PRIMARY KEY,
    amount        DECIMAL(10,2),
    valid_from    DATETIME2 GENERATED ALWAYS AS ROW START,
    valid_to      DATETIME2 GENERATED ALWAYS AS ROW END,
    PERIOD FOR SYSTEM_TIME (valid_from, valid_to)
) WITH (SYSTEM_VERSIONING = ON);

SELECT * FROM salary FOR SYSTEM_TIME AS OF '2025-01-01';   -- salaries as they stood
    ↪ then
```

Uses: auditing, regulatory compliance, "what did we know and when?", correcting historical errors without losing the record of what was believed.

38.11 Big-Data Databases

Big data is commonly characterized by the **three Vs** — **volume** (terabytes to petabytes), **velocity** (high-speed streams) and **variety** (structured, semi-structured and unstructured) — often extended with **veracity** (uncertain quality) and **value**.

Technology	Role
Hadoop HDFS	Distributed file system storing huge files across many machines
MapReduce / Apache Spark	Distributed batch (and, for Spark, streaming) computation
Apache Hive, Spark SQL, Presto/Trino	SQL query engines over files in distributed storage
Apache Kafka	Distributed log for high-velocity event streams
Columnar file formats (Parquet, ORC)	Compressed, column-oriented storage for analytics
Data lake	Raw data of all kinds stored cheaply in its native format
Lakehouse (Delta Lake, Apache Iceberg)	Adds ACID transactions and schemas to a data lake

The contrast with Chapter 36: a **data warehouse** stores cleaned, structured data with a schema designed in advance (**schema-on-write**); a **data lake** stores raw data and applies structure when it is read (**schema-on-read**). Lakehouses aim to combine the flexibility of the first with the reliability of the second.

38.12 Summary Map

Need	Modern answer
More read capacity	Replicas
More write capacity or storage	Sharding, distributed SQL
Survive failures	Replication with automatic failover, multi-zone deployment
No infrastructure management	DBaaS, serverless
SQL + ACID at global scale	NewSQL / distributed SQL
Microsecond latency	In-memory databases
Relationship-centric queries	Graph databases
Location queries	Spatial databases
History and audit	Temporal tables
Petabyte analytics	Big-data engines, cloud warehouses, lakehouses

Chapter Summary

- Vertical scaling adds power to one machine; horizontal scaling adds machines.
- Replication copies data to scale reads and provide availability; sharding partitions data across servers to scale writes and storage, with the shard key as the critical choice.
- High availability relies on redundancy, automatic failover and fencing, and is measured by uptime percentages, RPO and RTO.
- Cloud databases range from self-managed VMs to fully managed and serverless services, with storage and compute increasingly separated.
- NewSQL and distributed SQL systems provide SQL and ACID with horizontal scale using consensus replication.
- In-memory, graph, spatial and temporal databases specialize in speed, relationships, geometry and history respectively.
- Big-data systems handle volume, velocity and variety through distributed storage and computation, data lakes and lakehouses.

Key Terms

Scalability · Vertical scaling · Horizontal scaling · Replication · Primary–replica · Replication lag · Read-your-writes · Sharding · Shard key · Range / hash / directory sharding · Scatter-gather · High availability · Failover · Split-brain · Fencing · RPO · RTO · Cloud database · DBaaS · Serverless database · Separation of storage and compute · NewSQL · Distributed SQL · Consensus · Raft · Paxos · In-memory database · Graph database · Index-free adjacency · Spatial database · R-tree · Temporal database · Valid time · Transaction time · Bitemporal · Big data · Three Vs · Data lake · Lakehouse · Schema-on-read · Schema-on-write

Important Formulas / Notation

```
downtime per year = (1 − availability) × 8,760 hours
hash sharding: shard = hash(key) mod N
```

```
    consensus commit: a write is durable once acknowledged by a majority ⌊N/2⌋ + 1
```

Common Mistakes

1. Confusing replication (copies) with sharding (partitions).
2. Sharding prematurely, before indexing, caching and replicas have been exhausted.
3. Choosing a shard key that most queries do not include.
4. Reading from asynchronous replicas where fresh data is required.
5. Treating "cloud" as automatically cheaper or automatically highly available.
6. Assuming NewSQL removes the latency cost of consistency across regions.
7. Using an in-memory database without a durability strategy for data that matters.

Solved Problems

Problem 38.1 — A system promises 99.95 % availability. How much downtime per year and per month does this allow?

Solution. Per year: 0.0005 × 8,760 h = **4.38 hours**. Per 30-day month: 0.0005 × 720 h = **0.36 hours ≈ 21.6 minutes**.

Problem 38.2 — A university portal's database is overloaded during result publication, with 95 % of queries being reads. Recommend a scaling approach.

Solution. Since the load is overwhelmingly reads, add **read replicas** behind the application tier and route read-only queries (result lookups, transcripts) to them, while writes (grade entry, which is complete before publication anyway) go to the primary. Combine with a **cache** (Redis) for identical repeated queries and ensure the result queries are **indexed**. Sharding is unnecessary: it addresses write and storage limits, which are not the problem here.

Problem 38.3 — Evaluate `student_id`, `dept_id` and `admission_year` as shard keys for a national student database of 40 million students.

Solution. **`student_id` (hashed)** gives even distribution, and single-student operations — the most frequent — hit one shard; departmental reports must scatter-gather, which is acceptable for batch reporting. **`dept_id`** has few distinct values, so shards would be few and badly unbalanced (large departments become hot shards). **`admission_year`** creates a hot shard: every new admission and most current activity fall on the newest year's shard. The best choice is **hash of `student_id`**.

Problem 38.4 — Explain how distributed SQL avoids the blocking problem of two-phase commit.

Solution. In classic 2PC, participants that have voted yes must wait for a single coordinator; if it fails, they block. Distributed SQL systems replicate every piece of data — including transaction state — through a consensus protocol such as Raft. A decision is durable once a majority of replicas have recorded it, and if the current leader fails, the remaining replicas elect a new leader that already holds the committed state. No single node's failure can leave a transaction undecided, so there is no indefinite blocking, at the cost of a majority round trip on every commit.

Problem 38.5 — A food-delivery app must find restaurants within 3 km of a customer. Why is a spatial index better than a B+ tree on latitude?

Solution. A B+ tree on latitude can narrow the search to a horizontal band of the map, but every restaurant in that band — across the whole country's longitude — must still be examined. A spatial index (R-tree) indexes two-dimensional bounding rectangles, so the query descends only into regions that intersect the 3 km circle, examining a handful of candidates regardless of how many restaurants exist nationwide. Spatial functions then compute exact distances on the earth's surface rather than on a flat grid.

Review Questions

1. Compare vertical and horizontal scaling.
2. Explain primary–replica replication and its benefits and risks.
3. Explain sharding strategies and the importance of the shard key.
4. Distinguish replication, sharding and partitioning.
5. Define high availability, RPO and RTO, and describe HA techniques.
6. Compare the cloud database service models.
7. What problem do NewSQL and distributed SQL systems solve, and how?
8. Describe in-memory, spatial and temporal databases with use cases.
9. Explain the three Vs of big data and the main technologies.
10. Distinguish a data warehouse, a data lake and a lakehouse.

Multiple Choice Questions

1. Adding more servers is (a) vertical (b) horizontal scaling (c) replication lag (d) failover.
2. Sharding primarily scales (a) reads only (b) writes and storage (c) security (d) backups.
3. Replicas mainly scale (a) writes (b) reads (c) schema changes (d) DDL.
4. Five nines availability allows yearly downtime of about (a) 5 minutes (b) 52 minutes (c) 8.8 hours (d) 3.6 days.
5. Split-brain occurs when (a) two nodes both act as primary (b) a disk fails (c) a query deadlocks (d) a replica lags.
6. Distributed SQL systems typically use (a) 2PC only (b) consensus protocols (c) no replication (d) eventual consistency only.
7. Spatial data is commonly indexed with (a) hash indexes (b) R-trees (c) bitmap only (d) no index.
8. Transaction time records (a) when a fact was true (b) when it was stored (c) query duration (d) lock duration.
9. Schema-on-read is characteristic of a (a) data warehouse (b) data lake (c) OLTP database (d) star schema.
10. RPO measures (a) acceptable downtime (b) acceptable data loss (c) query latency (d) replication count.

Answers: 1-(b) 2-(b) 3-(b) 4-(a) 5-(a) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What is a shard key?
2. What is replication lag?
3. Define failover.
4. What is DBaaS?
5. What does separation of storage and compute mean?
6. Name two distributed SQL systems.
7. How does an in-memory database remain durable?
8. What is valid time?
9. Name the three Vs.
10. What is a lakehouse?

Long / Descriptive Questions

1. Explain replication and sharding in detail, including strategies, benefits, risks and when to use each.
2. Discuss high availability: metrics, techniques, failover and split-brain.
3. Explain cloud database models and the idea of separating storage from compute.
4. Explain NewSQL/distributed SQL, including how consensus replaces two-phase commit.
5. Survey in-memory, graph, spatial, temporal and big-data databases with examples and use cases.

Practical Exercises

1. Set up a primary and one replica of `university_db`, generate write load, and monitor replication lag.
2. Implement application-level hash sharding of a `student` table across three database instances and write the routing function.
3. Create a free-tier managed database in a cloud provider and compare its backup and failover features with a self-managed installation.
4. Install PostGIS, load sample hospital locations, and write nearest-neighbour and within-radius queries.
5. Create a system-versioned temporal table (MariaDB or SQL Server) and query its history as of past dates.

CHAPTER 39

Database Performance and Tuning

Performance tuning is the disciplined process of making a database do its work faster, or with fewer resources, without changing what it computes. It draws on every earlier chapter — indexing, query processing, optimization, concurrency, storage — and adds method.

39.1 The Tuning Method

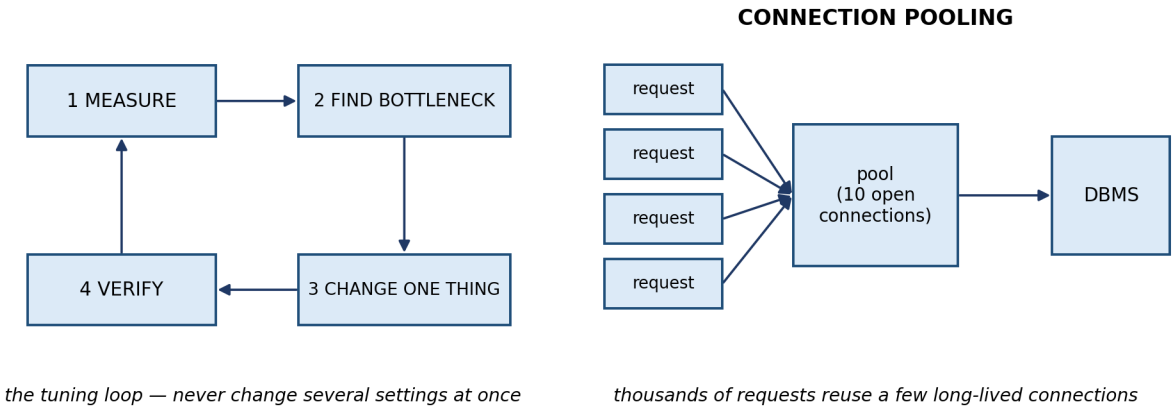


Figure 39.1: The tuning cycle and connection pooling

1. **Measure** — establish a baseline with real metrics, not impressions.
2. **Find the bottleneck** — the single resource or query limiting throughput.
3. **Change one thing** — so that its effect can be attributed.
4. **Verify** — measure again; keep the change only if it helped.

Tuning proceeds from the cheapest, highest-impact changes to the most expensive:

Level	Examples	Typical impact
Query	Rewrite, remove `SELECT *`, make predicates sargable	Very high
Indexing	Add, drop or reshape indexes	Very high
Schema	Normalize or selectively denormalize, choose data types	High
Configuration	Buffer pool, memory, log settings	Medium
Architecture	Caching, replicas, partitioning, sharding	High, but costly
Hardware	More RAM, faster SSDs	Medium, costly

39.2 Query Performance

The slow-query log is the starting point:

```
-- MySQL: log statements slower than 0.5 s
SET GLOBAL slow_query_log = ON;
SET GLOBAL long_query_time = 0.5;
SET GLOBAL log_queries_not_using_indexes = ON;
```

```sql
-- MySQL performance_schema: the ten most expensive statement patterns
SELECT digest_text, count_star AS executions,
       ROUND(sum_timer_wait / 1e12, 2) AS total_seconds,
       ROUND(avg_timer_wait / 1e9, 2)  AS avg_ms
FROM   performance_schema.events_statements_summary_by_digest
ORDER  BY sum_timer_wait DESC
LIMIT  10;
```

The right target is the query with the largest **total** time (frequency × duration), not necessarily the slowest single query: a 5 ms query run a million times a day costs more than a 20-second report run once.

Query rewriting checklist

Problem	Rewrite
`SELECT *`	Name only the needed columns; enables covering indexes
`WHERE YEAR(d) = 2025`	`WHERE d >= '2025-01-01' AND d < '2026-01-01'`
`WHERE name LIKE '%rahim%'`	Full-text index, or a prefix search `LIKE 'rahim%'`
`NOT IN (subquery)` with NULLs	`NOT EXISTS`
Correlated subquery per row	Join or grouped derived table
`OR` across different columns	`UNION ALL` of two indexed queries
`OFFSET 100000` pagination	Keyset pagination: `WHERE id > :last_id ORDER BY id LIMIT 20`
N+1 queries from application loops	One query with a join or `IN (…)` list

The N+1 problem deserves emphasis, because it is the most common performance defect in real applications: code fetches 100 students, then loops and runs one query per student to fetch their enrollments — 101 round trips where one join would do.

39.3 Index Tuning

The principles of Chapter 23, applied as a process:

1. Index every foreign key used in joins.
2. For frequent queries, design composite indexes with **equality columns first, then range or sort columns**.
3. Use covering indexes for the hottest read queries.
4. Remove unused and duplicate indexes — each costs every write.

```sql
-- MySQL sys schema: indexes never used since the server started
SELECT * FROM sys.schema_unused_indexes WHERE object_schema = 'university_db';

-- redundant indexes (one is a prefix of another)
SELECT * FROM sys.schema_redundant_indexes WHERE table_schema = 'university_db';
```

Worked design. The results portal runs, thousands of times a minute:

```sql
SELECT course_id, semester, grade FROM enrollment
WHERE  student_id = ? ORDER BY semester;
```

The ideal index is `(student_id, semester, course_id, grade)`: the equality column first, then the sort column, then the remaining selected columns so that the query is answered from the index alone — no table access and no sort.

39.4 Query Plan Analysis

```
EXPLAIN ANALYZE
SELECT s.name, COUNT(*) AS courses
FROM   student s JOIN enrollment e ON e.student_id = s.student_id
WHERE  s.dept_id = 'CSE'
GROUP  BY s.student_id, s.name;
```

What to look for:

Symptom in the plan	Likely cause	Remedy
Full scan (`type = ALL`, `Seq Scan`) of a large table	Missing or unusable index	Add an index; make the predicate sargable
Estimated rows far from actual rows	Stale or insufficient statistics	`ANALYZE TABLE`; histograms
`Using filesort`, explicit Sort node	No index delivers the required order	Index on the `ORDER BY` columns
`Using temporary`	Grouping or distinct needs a temp table	Index on the grouping columns
Nested loop over large inputs	Bad join order from bad estimates	Fix statistics
High “rows examined” relative to rows returned	Poor selectivity of chosen access path	Better composite index

39.5 Buffer Management

The buffer pool (Chapter 22) holds recently used pages in memory. The goal is for the **working set** — the pages accessed frequently — to fit in it.

```
    buffer pool hit ratio = 1 − (physical reads / logical reads)
```

A hit ratio above 99 % is typical for a well-sized OLTP system.

```
-- MySQL: size and hit ratio
SELECT @@innodb_buffer_pool_size / 1024 / 1024 / 1024 AS buffer_pool_gb;
SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool_read%';
-- hit ratio = 1 - Innodb_buffer_pool_reads / Innodb_buffer_pool_read_requests
```

Setting (MySQL / PostgreSQL)	Guidance
`innodb_buffer_pool_size` / `shared_buffers`	MySQL: often 50–75 % of RAM on a dedicated server; PostgreSQL: about 25 %, relying on the OS cache for the rest
`innodb_log_file_size` / `max_wal_size`	Large enough that checkpoints are not constant
`sort_buffer_size` / `work_mem`	Per operation: too high multiplied by many connections exhausts RAM

Replacement policy matters: InnoDB uses a modified LRU with a “young” and “old” sublist so that a single large scan does not evict the entire hot working set.

39.6 Caching

Caching keeps the results of expensive work so that repeated requests avoid the database altogether.

Layer	Example
Application / object cache	Redis or Memcached holding query results or rendered objects

Layer	Example
Database buffer pool	Pages in memory (automatic)
HTTP / CDN cache	Whole pages or API responses at the edge

The **cache-aside** pattern:

```
read(key):
    value = cache.get(key)
    if value is missing:
        value = database.query(...)
        cache.set(key, value, ttl = 300 seconds)
    return value

write(key, new_value):
    database.update(...)
    cache.delete(key)          -- invalidate, so the next read refetches
```

The hard part is **invalidation**: a cache that is not invalidated on write serves stale data. Time-to-live (TTL) bounds the staleness. Note that MySQL 8 removed its built-in query cache because invalidation contention made it a bottleneck on multi-core servers — caching belongs in the application tier.

39.7 Connection Pooling

Opening a database connection is expensive: TCP handshake, TLS negotiation, authentication and per-session memory allocation can take tens of milliseconds. A **connection pool** keeps a set of open connections and lends them to requests.

Without a pool	With a pool
One connection opened per request	Connections reused
Connection overhead on every request	Overhead paid once
Thousands of idle sessions under load	A bounded number of sessions
Database memory exhaustion at peak	Predictable resource use

Pools are provided by application frameworks (HikariCP for Java, SQLAlchemy's pool for Python) or as middleware (PgBouncer, ProxySQL). A pool of **tens** of connections usually outperforms hundreds, because beyond the number of CPU cores and disk channels, extra concurrent queries only increase contention.

39.8 Partitioning

Partitioning divides one large table into smaller physical pieces on the **same server**, while it remains one logical table.

```
-- MySQL: range partitioning of results by year
CREATE TABLE result_history (
    result_id  BIGINT NOT NULL,
    student_id CHAR(10) NOT NULL,
    exam_date  DATE NOT NULL,
    marks      DECIMAL(5,2),
    PRIMARY KEY (result_id, exam_date)
)
PARTITION BY RANGE (YEAR(exam_date)) (
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION p2023 VALUES LESS THAN (2024),
```

```
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION pmax  VALUES LESS THAN MAXVALUE
);
```

Benefit	Explanation
Partition pruning	`WHERE exam_date >= '2025-01-01'` reads only the 2025 partition
Fast archiving	`ALTER TABLE … DROP PARTITION p2022` removes a year instantly, instead of a massive `DELETE`
Maintenance	Rebuild or back up one partition at a time

Partitioning type	Rule
Range	By value ranges — dates are typical
List	By explicit value lists — divisions, departments
Hash	By a hash of the key — even spread

Note that in MySQL the partitioning column must be part of every unique key, which is why `exam_date` appears in the primary key above.

39.9 Replication and Sharding for Performance

Covered in Chapter 38; from a tuning perspective:

- **Replicas** offload reads, reports and backups from the primary.
- **Sharding** is the last resort for write throughput and storage, after query tuning, indexing, caching, partitioning and hardware have been exhausted, because it permanently complicates the application.

39.10 Performance Monitoring

Metric	Why it matters
Queries per second, transactions per second	Throughput
Query latency percentiles (p50, p95, p99)	Tail latency is what users notice
Slow queries per minute	Regressions
Buffer pool hit ratio	Memory sufficiency
Lock waits and deadlocks per minute	Contention
Active and total connections	Pool sizing and leaks
Replication lag	Freshness of replicas
Disk I/O utilization and latency	Storage bottlenecks
CPU utilization	Compute bottleneck

Tools include MySQL `performance_schema` and `sys` schema, PostgreSQL `pg_stat_statements` and `pg_stat_activity`, and dashboards such as Percona Monitoring and Management, Grafana with Prometheus exporters, and cloud-provider monitoring.

Averages hide problems. If the average latency is 20 ms but the 99th percentile is 3 seconds, one request in a hundred is intolerably slow — and heavy users hit it constantly. Always monitor percentiles.

Chapter Summary

- Tuning is a cycle: measure, find the bottleneck, change one thing, verify.
- The largest gains usually come from query rewriting and indexing, before configuration and hardware.
- Target queries by total time (frequency × duration); eliminate N+1 query patterns.
- Composite indexes place equality columns first, then range and sort columns; unused indexes should be removed.
- Execution plans reveal full scans, estimation errors, sorts and temporary tables.
- The buffer pool should hold the working set; hit ratios above 99 % are normal.
- Application caching with invalidation and connection pooling reduce database load dramatically.
- Partitioning enables pruning and fast archiving on one server; replication and sharding scale beyond it.
- Monitoring must include latency percentiles, not only averages.

Key Terms

Performance tuning · Baseline · Bottleneck · Slow query log · Total time · N+1 problem · Keyset pagination · Index tuning · Covering index · Unused index · Query plan analysis · Buffer pool hit ratio · Working set · Cache-aside · TTL · Cache invalidation · Connection pool · Partitioning · Partition pruning · Range / list / hash partitioning · Latency percentile · p95 / p99 · Throughput

Important Formulas / Notation

```
buffer pool hit ratio = 1 − physical reads / logical reads
query priority        = executions × average duration   (total time)
pool size heuristic   ≈ (CPU cores × 2) + effective disk spindles
```

Common Mistakes

1. Changing several settings at once, so nobody knows what helped.
2. Tuning the slowest query instead of the most expensive in total.
3. Adding indexes indiscriminately.
4. Setting per-operation memory (`work_mem`, `sort_buffer_size`) so high that many concurrent queries exhaust RAM.
5. Caching without invalidation.
6. Oversized connection pools that increase contention.
7. Using `OFFSET` pagination deep into large tables.
8. Monitoring only averages.

Solved Problems

Problem 39.1 — Query A takes 4 ms and runs 2,000,000 times a day. Query B takes 25 s and runs twice a day. Which should be tuned first?

Solution. Total time: A = 2,000,000 × 0.004 s = **8,000 s** (2.2 hours) per day; B = 2 × 25 s = **50 s** per day. **Query A** consumes 160 times more database time; halving it saves an hour a day, while eliminating B entirely saves under a minute. Query B may still deserve attention if a user is waiting on it, but for overall load A comes first.

Problem 39.2 — Rewrite an N+1 pattern that lists each CSE student with their course count.

Solution. The N+1 version runs `SELECT student_id, name FROM student WHERE dept_id='CSE'` and then, for each of the rows, `SELECT COUNT(*) FROM enrollment WHERE student_id = ?` — one plus N queries and N network round trips. The single-query version:

```sql
SELECT s.student_id, s.name, COUNT(e.enrollment_id) AS courses
FROM   student s LEFT JOIN enrollment e ON e.student_id = s.student_id
WHERE  s.dept_id = 'CSE'
GROUP  BY s.student_id, s.name;
```

One round trip and one plan, typically tens or hundreds of times faster.

Problem 39.3 — A buffer pool shows 5,000,000 read requests and 250,000 disk reads in an hour. Compute the hit ratio and comment.

Solution. Hit ratio = 1 − 250,000 / 5,000,000 = 1 − 0.05 = **95 %**. For OLTP this is low: one logical read in twenty goes to disk. The working set does not fit in memory. Options: increase the buffer pool if RAM allows; reduce the working set by removing full scans (better indexes) or archiving old data (partitioning); or add RAM.

Problem 39.4 — Explain keyset pagination and why it outperforms `OFFSET`.

Solution. `SELECT … ORDER BY id LIMIT 20 OFFSET 100000` forces the database to read and discard 100,000 rows before returning 20, so page 5,000 is thousands of times slower than page 1. Keyset pagination remembers the last key shown and asks for rows after it:

```sql
SELECT student_id, name FROM student
WHERE  student_id > 'ASH2001005'          -- last key of the previous page
ORDER  BY student_id LIMIT 20;
```

With an index on `student_id`, the engine seeks directly to the starting point and reads exactly 20 rows, so every page costs the same.

Problem 39.5 — Design a performance plan for the university results portal on publication day.

Solution. (1) **Queries:** ensure the result lookup is a single join with named columns, no N+1 pattern. (2) **Indexes:** covering index `enrollment(student_id, semester, course_id, grade)` and index on `result(student_id)`. (3) **Caching:** results do not change after publication, so cache each student's result page in Redis with a long TTL, invalidated only if a correction is made. (4) **Connection pooling:** a bounded pool of about 50 connections per application server. (5) **Replicas:** route read traffic to two replicas. (6) **Monitoring:** watch p99 latency, connections and replication lag, with an alert threshold. (7) **Load test** the day before with simulated traffic at twice the expected peak.

Review Questions

1. Describe the tuning cycle and the order in which tuning levels are considered.
2. How are the queries most worth tuning identified?
3. List six query rewrites that improve performance.
4. Explain the N+1 problem and its solution.
5. Explain index tuning principles, including composite column order and removal of unused indexes.
6. What symptoms in an execution plan indicate problems, and what are the remedies?
7. Explain buffer pool sizing and the hit ratio.
8. Describe the cache-aside pattern and the invalidation problem.
9. Explain connection pooling and why larger pools are not always better.
10. Explain partitioning, pruning and its benefits.

Multiple Choice Questions

1. The first step of tuning is to (a) add RAM (b) measure (c) shard (d) rewrite everything.
2. The query to tune first is the one with the highest (a) single duration (b) total time (c) text length (d) number of joins.
3. Running one query per row fetched is the (a) phantom (b) N+1 (c) deadlock (d) cascade problem.
4. Deep `OFFSET` pagination should be replaced by (a) `LIMIT` only (b) keyset pagination (c) `DISTINCT` (d) a view.
5. A hit ratio of 1 − 100/10,000 is (a) 90 % (b) 99 % (c) 1 % (d) 10 %.
6. Removing an entire year of data instantly is possible with (a) `DELETE` (b) `DROP PARTITION` (c) `TRUNCATE` of the whole table (d) `VACUUM`.
7. The cache-aside pattern invalidates the cache on (a) read (b) write (c) startup (d) never.
8. Connection pooling mainly avoids (a) deadlocks (b) the cost of opening connections (c) phantoms (d) index scans.
9. Partition pruning means (a) deleting partitions (b) scanning only relevant partitions (c) merging partitions (d) indexing partitions.
10. Latency should be monitored using (a) averages only (b) percentiles (c) maximums only (d) row counts.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What is a baseline?
2. What is the slow-query log?
3. What is a covering index?
4. Define working set.
5. What is TTL in caching?
6. Why did MySQL remove its query cache?
7. What is partition pruning?
8. What is p99 latency?
9. Give one sign of stale statistics in a plan.
10. Why should only one setting be changed at a time?

Long / Descriptive Questions

1. Explain the performance tuning method and the levels at which tuning is applied, with examples.
2. Discuss query performance analysis and rewriting, including the N+1 problem and keyset pagination.
3. Explain index tuning and query plan analysis with a worked example.
4. Explain buffer management, caching and connection pooling as performance tools.
5. Design a complete performance plan for a high-traffic database application, covering queries, indexes, caching, pooling, partitioning, replication and monitoring.

Practical Exercises

1. Enable the slow-query log, run a workload against `university_db`, and identify the three most expensive statements by total time.
2. Find unused and redundant indexes using the MySQL `sys` schema or PostgreSQL statistics views,

and justify removing them.

3. Implement an N+1 pattern in application code, measure it, then replace it with a single join and measure again.
4. Partition a large generated results table by year and show partition pruning in `EXPLAIN`.
5. Implement cache-aside for the student result lookup using Redis and measure latency with and without the cache.

CHAPTER 40

Complete Database Project: University Management System

This chapter brings the whole book together. It follows one project — a University Management System (UMS) — through every stage of the database life cycle: requirements, conceptual design, logical design, normalization, physical implementation, programming, performance and security. The schema extends the `university_db` of Chapter 11, so the queries of earlier chapters carry over with little or no change.

40.1 Requirements Analysis

Problem statement

A university needs a central system to manage its departments, teaching staff, students, courses, course offerings, enrolment, examinations, results and tuition fees, replacing separate spreadsheets kept by the admission office, the examination controller and the accounts office.

Functional requirements

1. Record departments, each with a unique code, a name, a building and one head who is an instructor of that department.
2. Record instructors with their designation, salary and department.
3. Record students with their department, contact details, admission date and an academic advisor who is an instructor.
4. Record courses offered by each department, with a unique code, title and credit value; a course may have prerequisite courses.
5. Record which instructor teaches which course in which semester.
6. Enrol students in courses per semester and record the letter grade.
7. Schedule examinations for courses in classrooms and record each student's marks.
8. Record fee payments per student per semester.
9. Produce transcripts, CGPA, course rosters, departmental statistics and fee-due reports.
10. Keep an audit trail of every grade change.

Business rules

#	Rule	Enforced by
BR1	A department code and name are unique	`PRIMARY KEY`, `UNIQUE`
BR2	Every student and instructor belongs to exactly one department	`NOT NULL` foreign key
BR3	A department cannot be deleted while it has students, staff or courses	`ON DELETE RESTRICT`
BR4	CGPA lies between 0.00 and 4.00; marks between 0 and 100	`CHECK`
BR5	Course credit lies between 0.5 and 4.0	`CHECK`
BR6	A student may enrol in a course at most once per semester	`UNIQUE (student_id, course_id, semester)`
BR7	A student may take at most six courses per semester	Trigger
BR8	A student may not enrol in a course without passing its prerequisites	Stored procedure

#	Rule	Enforced by
BR9	Published grades may be changed only by the controller, and every change is audited	Privileges + audit trigger
BR10	Fee amounts are positive	`CHECK`
BR11	Examination capacity must not exceed room capacity	Procedure check

Non-functional requirements

- Results publication must support thousands of concurrent readers.
- Grade and fee data must be durable and recoverable to any point in time.
- Each user group sees and changes only what its role requires.

40.2 Entities, Attributes and Relationships

Entity	Key	Principal attributes
DEPARTMENT	`dept_id`	`dept_name`, `building`, `head_id`
INSTRUCTOR	`instructor_id`	`name`, `email`, `designation`, `salary`, `dept_id`
STUDENT	`student_id`	`name`, `email`, `phone`, `dob`, `cgpa`, `dept_id`, `advisor_id`, `admitted_on`
COURSE	`course_id`	`course_code`, `title`, `credit`, `dept_id`
CLASSROOM	`room_id`	`building`, `capacity`
EXAMINATION	`exam_id`	`course_id`, `exam_date`, `room_id`, `exam_type`
FEE_PAYMENT	`payment_id`	`student_id`, `semester`, `amount`, `paid_on`, `method`

Relationship	Between	Cardinality	Participation	Attributes
BELONGS TO	STUDENT – DEPARTMENT	N : 1	Total on STUDENT	—
EMPLOYS	INSTRUCTOR – DEPARTMENT	N : 1	Total on INSTRUCTOR	—
HEADS	INSTRUCTOR – DEPARTMENT	1 : 1	Partial on both (initially)	—
OFFERS	DEPARTMENT – COURSE	1 : N	Total on COURSE	—
ADVISES	INSTRUCTOR – STUDENT	1 : N	Partial	—
PREREQUISITE OF	COURSE – COURSE (recursive)	M : N	Partial	—
TEACHES	INSTRUCTOR – COURSE	M : N	Partial	`semester`
ENROLS	STUDENT – COURSE	M : N	Partial	`semester`, `grade`
HELD FOR	COURSE – EXAMINATION	1 : N	Total on EXAMINATION	—
HELD IN	EXAMINATION – CLASSROOM	N : 1	Total on EXAMINATION	—
SITS	STUDENT – EXAMINATION	M : N	Partial	`marks`
PAYS	STUDENT – FEE_PAYMENT	1 : N	Total on FEE_PAYMENT	—

40.3 ER Diagram

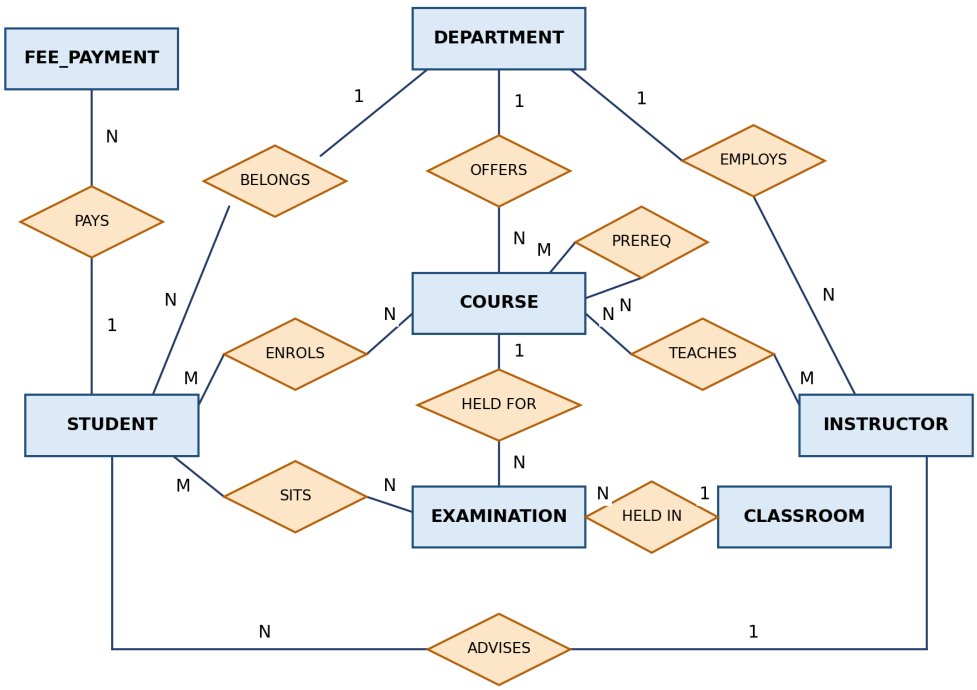


Figure 40.1: ER diagram of the University Management System

The diagram is deliberately laid out without crossing lines. Every M : N relationship — ENROLS, TEACHES, SITS, PREREQUISITE OF — will become a junction table; every 1 : N relationship becomes a foreign key on the N side; HEADS becomes a unique foreign key in DEPARTMENT; and ADVISES becomes `student.advisor_id`.

40.4 Relational Schema

Applying the mapping algorithm of Chapter 9:

```
department   (dept_id, dept_name, building, head_id → instructor)
instructor   (instructor_id, name, email, designation, salary, dept_id → department)
student      (student_id, name, email, phone, dob, cgpa, dept_id → department,
              advisor_id → instructor, admitted_on)
course       (course_id, course_code, title, credit, dept_id → department)
prerequisite (course_id → course, prereq_id → course)
teaches      (instructor_id → instructor, course_id → course, semester)
classroom    (room_id, building, capacity)
enrollment   (enrollment_id, student_id → student, course_id → course, semester, grade)
    ↪
examination  (exam_id, course_id → course, room_id → classroom, exam_date, exam_type)
result       (result_id, exam_id → examination, student_id → student, marks)
fee_payment  (payment_id, student_id → student, semester, amount, paid_on, method)
grade_audit  (audit_id, enrollment_id, old_grade, new_grade, changed_by, changed_at)
```

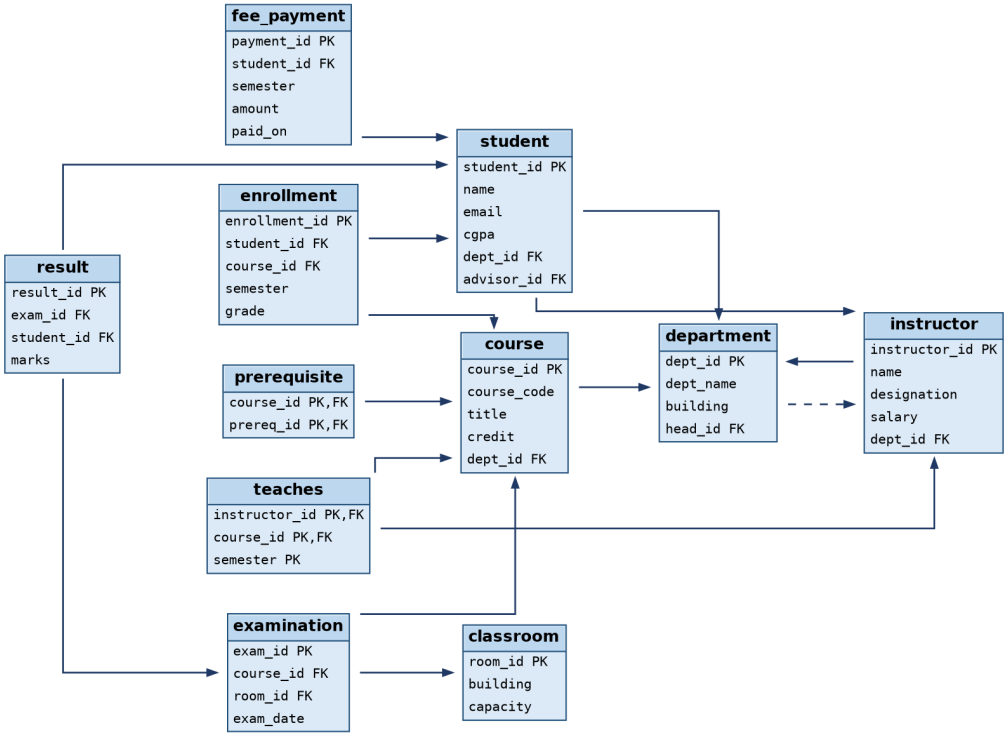


Figure 40.2: Relational schema with foreign keys

Keys

Table	Primary key	Alternate keys	Foreign keys
department	`dept_id`	`dept_name`	`head_id`
instructor	`instructor_id`	`email`	`dept_id`
student	`student_id`	`email`	`dept_id`, `advisor_id`
course	`course_id`	`course_code`	`dept_id`
prerequisite	(`course_id`, `prereq_id`)	—	both columns
teaches	(`instructor_id`, `course_id`, `semester`)	—	`instructor_id`, `course_id`
enrollment	`enrollment_id` (surrogate)	(`student_id`, `course_id`, `semester`)	`student_id`, `course_id`
examination	`exam_id`	—	`course_id`, `room_id`
result	`result_id` (surrogate)	(`exam_id`, `student_id`)	`exam_id`, `student_id`
fee_payment	`payment_id`	—	`student_id`

40.5 Normalization Check

Functional dependencies (from the business rules, not from data):

```
dept_id       → dept_name, building, head_id
instructor_id → name, email, designation, salary, dept_id
student_id    → name, email, phone, dob, cgpa, dept_id, advisor_id, admitted_on
course_id     → course_code, title, credit, dept_id
course_code   → course_id
room_id       → building, capacity
exam_id       → course_id, room_id, exam_date, exam_type
(student_id, course_id, semester) → grade
```

```
(exam_id, student_id) → marks
payment_id    → student_id, semester, amount, paid_on, method
```

Normal form	Check	Result
1NF	Every attribute atomic; the multivalued phone was restricted to one contact number, and prerequisites are a separate table	Satisfied
2NF	Composite keys (`prerequisite`, `teaches`) have no non-key attributes; alternate composite keys determine only `grade` and `marks`, fully	Satisfied
3NF	No non-key attribute determines another: department names live only in `department`, course titles only in `course`	Satisfied
BCNF	Every determinant listed above is a candidate key of its table (`course_code` is an alternate key of `course`)	Satisfied

One deliberate exception is documented: `student.cgpa` is **derivable** from enrollments and course credits. It is stored for performance — the portal reads it constantly — and is maintained only by the stored procedure `sp_recompute_cgpa`, never edited directly. This is a controlled denormalization in the sense of Section 19.9.

40.6 SQL Table Creation (MySQL 8)

```sql
DROP DATABASE IF EXISTS ums_db;
CREATE DATABASE ums_db CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE ums_db;

-- 1. Departments (head_id added after instructor exists)
CREATE TABLE department (
    dept_id    CHAR(3)      NOT NULL,
    dept_name  VARCHAR(80)  NOT NULL,
    building   VARCHAR(40),
    head_id    INT          NULL,
    CONSTRAINT pk_department   PRIMARY KEY (dept_id),
    CONSTRAINT uq_dept_name    UNIQUE (dept_name)
);

-- 2. Instructors
CREATE TABLE instructor (
    instructor_id INT           NOT NULL AUTO_INCREMENT,
    name          VARCHAR(100)  NOT NULL,
    email         VARCHAR(100)  NOT NULL,
    designation   VARCHAR(40)   NOT NULL,
    salary        DECIMAL(10,2) NOT NULL,
    dept_id       CHAR(3)       NOT NULL,
    CONSTRAINT pk_instructor   PRIMARY KEY (instructor_id),
    CONSTRAINT uq_inst_email   UNIQUE (email),
    CONSTRAINT ck_inst_salary  CHECK (salary > 0),
    CONSTRAINT ck_inst_desig   CHECK (designation IN
               ('Lecturer','Assistant Professor','Associate Professor','Professor')),
    CONSTRAINT fk_inst_dept    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
               ON UPDATE CASCADE ON DELETE RESTRICT
);

-- close the HEADS relationship (1:1, so head_id is UNIQUE)
ALTER TABLE department
    ADD CONSTRAINT uq_dept_head UNIQUE (head_id),
```

```
    ADD CONSTRAINT fk_dept_head FOREIGN KEY (head_id) REFERENCES instructor(
    ↪ instructor_id)
        ON UPDATE CASCADE ON DELETE SET NULL;

-- 3. Students
CREATE TABLE student (
    student_id  CHAR(10)     NOT NULL,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(100) NOT NULL,
    phone       VARCHAR(20),
    dob         DATE,
    cgpa        DECIMAL(3,2) DEFAULT NULL,
    dept_id     CHAR(3)      NOT NULL,
    advisor_id  INT          NULL,
    admitted_on DATE         NOT NULL,
    CONSTRAINT pk_student      PRIMARY KEY (student_id),
    CONSTRAINT uq_student_mail UNIQUE (email),
    CONSTRAINT ck_student_cgpa CHECK (cgpa IS NULL OR cgpa BETWEEN 0.00 AND 4.00),
    CONSTRAINT fk_stu_dept     FOREIGN KEY (dept_id) REFERENCES department(dept_id)
               ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_stu_advisor  FOREIGN KEY (advisor_id) REFERENCES instructor(
    ↪ instructor_id)
               ON UPDATE CASCADE ON DELETE SET NULL
);

-- 4. Courses
CREATE TABLE course (
    course_id   INT          NOT NULL AUTO_INCREMENT,
    course_code VARCHAR(10)  NOT NULL,
    title       VARCHAR(120) NOT NULL,
    credit      DECIMAL(2,1) NOT NULL DEFAULT 3.0,
    dept_id     CHAR(3)      NOT NULL,
    CONSTRAINT pk_course        PRIMARY KEY (course_id),
    CONSTRAINT uq_course_code   UNIQUE (course_code),
    CONSTRAINT ck_course_credit CHECK (credit BETWEEN 0.5 AND 4.0),
    CONSTRAINT fk_course_dept   FOREIGN KEY (dept_id) REFERENCES department(dept_id)
               ON UPDATE CASCADE ON DELETE RESTRICT
);

-- 5. Prerequisites (recursive M:N)
CREATE TABLE prerequisite (
    course_id INT NOT NULL,
    prereq_id INT NOT NULL,
    CONSTRAINT pk_prerequisite PRIMARY KEY (course_id, prereq_id),
    CONSTRAINT ck_prereq_self  CHECK (course_id <> prereq_id),
    CONSTRAINT fk_pre_course   FOREIGN KEY (course_id) REFERENCES course(course_id) ON
    ↪ DELETE CASCADE,
    CONSTRAINT fk_pre_prereq   FOREIGN KEY (prereq_id) REFERENCES course(course_id) ON
    ↪ DELETE CASCADE
);

-- 6. Teaching assignments (M:N with semester)
CREATE TABLE teaches (
    instructor_id INT         NOT NULL,
    course_id     INT         NOT NULL,
    semester      VARCHAR(10) NOT NULL,
    CONSTRAINT pk_teaches      PRIMARY KEY (instructor_id, course_id, semester),
    CONSTRAINT fk_teach_inst   FOREIGN KEY (instructor_id) REFERENCES instructor(
```

```
    ↪ instructor_id),
    CONSTRAINT fk_teach_course FOREIGN KEY (course_id) REFERENCES course(course_id)
);

-- 7. Classrooms
CREATE TABLE classroom (
    room_id  VARCHAR(10) NOT NULL,
    building VARCHAR(40) NOT NULL,
    capacity INT         NOT NULL,
    CONSTRAINT pk_classroom PRIMARY KEY (room_id),
    CONSTRAINT ck_room_cap  CHECK (capacity > 0)
);

-- 8. Enrolment (M:N with semester and grade)
CREATE TABLE enrollment (
    enrollment_id INT         NOT NULL AUTO_INCREMENT,
    student_id    CHAR(10)    NOT NULL,
    course_id     INT         NOT NULL,
    semester      VARCHAR(10) NOT NULL,
    grade         CHAR(2)     NULL,
    CONSTRAINT pk_enrollment  PRIMARY KEY (enrollment_id),
    CONSTRAINT uq_enrollment  UNIQUE (student_id, course_id, semester),
    CONSTRAINT ck_enr_grade   CHECK (grade IS NULL OR grade IN
              ('A+','A','A-','B+','B','B-','C+','C','D','F')),
    CONSTRAINT fk_enr_student FOREIGN KEY (student_id) REFERENCES student(student_id)
              ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_enr_course  FOREIGN KEY (course_id) REFERENCES course(course_id)
              ON DELETE RESTRICT
);

-- 9. Examinations
CREATE TABLE examination (
    exam_id   INT         NOT NULL AUTO_INCREMENT,
    course_id INT         NOT NULL,
    room_id   VARCHAR(10) NOT NULL,
    exam_date DATE        NOT NULL,
    exam_type VARCHAR(10) NOT NULL DEFAULT 'FINAL',
    CONSTRAINT pk_examination PRIMARY KEY (exam_id),
    CONSTRAINT ck_exam_type   CHECK (exam_type IN ('MIDTERM','FINAL')),
    CONSTRAINT fk_exam_course FOREIGN KEY (course_id) REFERENCES course(course_id),
    CONSTRAINT fk_exam_room   FOREIGN KEY (room_id)   REFERENCES classroom(room_id)
);

-- 10. Results (M:N STUDENT–EXAMINATION with marks)
CREATE TABLE result (
    result_id  INT          NOT NULL AUTO_INCREMENT,
    exam_id    INT          NOT NULL,
    student_id CHAR(10)     NOT NULL,
    marks      DECIMAL(5,2) NOT NULL,
    CONSTRAINT pk_result       PRIMARY KEY (result_id),
    CONSTRAINT uq_result       UNIQUE (exam_id, student_id),
    CONSTRAINT ck_result_marks CHECK (marks BETWEEN 0 AND 100),
    CONSTRAINT fk_res_exam     FOREIGN KEY (exam_id)    REFERENCES examination(exam_id),
    CONSTRAINT fk_res_student  FOREIGN KEY (student_id) REFERENCES student(student_id)
              ON UPDATE CASCADE
);

-- 11. Fee payments
```

```sql
CREATE TABLE fee_payment (
    payment_id INT           NOT NULL AUTO_INCREMENT,
    student_id CHAR(10)      NOT NULL,
    semester   VARCHAR(10)   NOT NULL,
    amount     DECIMAL(10,2) NOT NULL,
    paid_on    DATE          NOT NULL,
    method     VARCHAR(15)   NOT NULL,
    CONSTRAINT pk_fee        PRIMARY KEY (payment_id),
    CONSTRAINT ck_fee_amount CHECK (amount > 0),
    CONSTRAINT ck_fee_method CHECK (method IN ('BANK','MOBILE','CASH')),
    CONSTRAINT fk_fee_student FOREIGN KEY (student_id) REFERENCES student(student_id)
              ON UPDATE CASCADE ON DELETE RESTRICT
);

-- 12. Grade audit trail
CREATE TABLE grade_audit (
    audit_id      INT         NOT NULL AUTO_INCREMENT,
    enrollment_id INT         NOT NULL,
    old_grade     CHAR(2),
    new_grade     CHAR(2),
    changed_by    VARCHAR(80) NOT NULL,
    changed_at    DATETIME    NOT NULL,
    CONSTRAINT pk_grade_audit PRIMARY KEY (audit_id)
);
```

Two design choices worth noting: `fee_payment` → `student` uses `ON DELETE RESTRICT`, because financial records must never disappear with a student; and `grade_audit` deliberately has **no** foreign key to `enrollment`, so that the audit history survives even if an enrollment is later deleted.

40.7 Sample Data

```sql
INSERT INTO department (dept_id, dept_name, building) VALUES
 ('CSE','Computer Science and Engineering','Academic Building 3'),
 ('EEE','Electrical and Electronic Engineering','Academic Building 2'),
 ('ME', 'Mechanical Engineering','Academic Building 1'),
 ('BBA','Business Administration','Academic Building 4');

INSERT INTO instructor (name, email, designation, salary, dept_id) VALUES
 ('Nasrin Akter','nasrin@nstu.edu.bd','Professor',95000,'CSE'),
 ('Imran Hossain','imran@nstu.edu.bd','Associate Professor',78000,'CSE'),
 ('Sabina Yasmin','sabina@nstu.edu.bd','Assistant Professor',62000,'CSE'),
 ('Tanvir Ahmed','tanvir@nstu.edu.bd','Professor',94000,'EEE'),
 ('Farhana Islam','farhana@nstu.edu.bd','Lecturer',45000,'EEE'),
 ('Mahmudul Hasan','mahmud@nstu.edu.bd','Assistant Professor',60000,'ME'),
 ('Shirin Sultana','shirin@nstu.edu.bd','Associate Professor',75000,'BBA');

UPDATE department SET head_id = 1 WHERE dept_id = 'CSE';
UPDATE department SET head_id = 4 WHERE dept_id = 'EEE';
UPDATE department SET head_id = 6 WHERE dept_id = 'ME';
UPDATE department SET head_id = 7 WHERE dept_id = 'BBA';

INSERT INTO student (student_id, name, email, phone, dob, cgpa, dept_id, advisor_id,
    ↪ admitted_on) VALUES
 ('ASH1901001','Rahim Uddin','rahim@nstu.edu.bd','01712345678','2001-03-14',NULL,'CSE'
    ↪ ,2,'2019-01-10'),
 ('ASH1901002','Karim Mia','karim@nstu.edu.bd','01812345678','2001-07-02',NULL,'EEE',5,'
    ↪ 2019-01-10'),
```

```
 ('ASH1901003','Nila Rahman','nila@nstu.edu.bd','01912345678','2002-01-25',NULL,'CSE',2,
    ↪ '2019-01-10'),
 ('ASH1901004','Sumaiya Akter','sumaiya@nstu.edu.bd','01612345678','2001-11-30',NULL,'
    ↪ CSE',3,'2019-01-10'),
 ('ASH1901005','Jubayer Hossain','jubayer@nstu.edu.bd','01512345678','2000-09-18',NULL,'
    ↪ ME',6,'2019-01-10'),
 ('ASH2001001','Tania Sultana','tania@nstu.edu.bd','01713456789','2002-05-05',NULL,'CSE'
    ↪ ,3,'2020-01-12'),
 ('ASH2001002','Rakib Hasan','rakib@nstu.edu.bd','01813456789','2002-02-19',NULL,'EEE'
    ↪ ,5,'2020-01-12'),
 ('ASH2001003','Mitu Barua','mitu@nstu.edu.bd','01913456789','2002-08-08',NULL,'BBA',7,'
    ↪ 2020-01-12');

INSERT INTO course (course_code, title, credit, dept_id) VALUES
 ('CSE1101','Structured Programming',3.0,'CSE'),     -- 1
 ('CSE2101','Data Structures',3.0,'CSE'),            -- 2
 ('CSE3201','Database Management Systems',3.0,'CSE'),-- 3
 ('CSE3202','Database Systems Laboratory',1.5,'CSE'),-- 4
 ('EEE3101','Digital Electronics',3.0,'EEE'),        -- 5
 ('ME2101', 'Engineering Mechanics',3.0,'ME'),       -- 6
 ('BBA1101','Principles of Management',3.0,'BBA');   -- 7

INSERT INTO prerequisite VALUES (2,1), (3,2), (4,2);    -- DS needs SP; DBMS and lab
    ↪ need DS

INSERT INTO teaches VALUES
 (3,1,'1st'), (2,2,'3rd'), (1,3,'5th'), (3,4,'5th'), (4,5,'5th'), (6,6,'3rd'), (7,7,'1st
    ↪ ');

INSERT INTO classroom VALUES
 ('AB3-401','Academic Building 3',60), ('AB3-402','Academic Building 3',45),
 ('AB2-201','Academic Building 2',80), ('AB4-101','Academic Building 4',120);

INSERT INTO enrollment (student_id, course_id, semester, grade) VALUES
 ('ASH1901001',1,'1st','A'),  ('ASH1901001',2,'3rd','A-'), ('ASH1901001',3,'5th','A'),
 ('ASH1901001',4,'5th','A+'), ('ASH1901003',1,'1st','A+'), ('ASH1901003',2,'3rd','A'),
 ('ASH1901003',3,'5th','A'),  ('ASH1901004',1,'1st','B'),  ('ASH1901004',2,'3rd','C+'),
 ('ASH1901004',3,'5th','B'),  ('ASH2001001',1,'1st','A'),  ('ASH2001001',2,'3rd','A'),
 ('ASH1901002',5,'5th','B+'), ('ASH2001002',5,'5th','C'),  ('ASH1901005',6,'3rd','B-'),
 ('ASH2001003',7,'1st',NULL);

INSERT INTO examination (course_id, room_id, exam_date, exam_type) VALUES
 (3,'AB3-401','2024-06-10','FINAL'), (2,'AB3-402','2023-12-12','FINAL'),
 (5,'AB2-201','2024-06-11','FINAL'), (7,'AB4-101','2024-06-12','FINAL');

INSERT INTO result (exam_id, student_id, marks) VALUES
 (1,'ASH1901001',82.00), (1,'ASH1901003',88.50), (1,'ASH1901004',67.00),
 (2,'ASH1901001',76.00), (2,'ASH1901003',84.00), (2,'ASH1901004',57.00), (2,'ASH2001001'
    ↪ ,86.00),
 (3,'ASH1901002',72.00), (3,'ASH2001002',52.00), (4,'ASH2001003',74.00);

INSERT INTO fee_payment (student_id, semester, amount, paid_on, method) VALUES
 ('ASH1901001','5th',15000,'2024-01-15','BANK'),  ('ASH1901003','5th',15000,'2024-01-16'
    ↪ ,'MOBILE'),
 ('ASH1901004','5th', 7500,'2024-01-20','CASH'),  ('ASH1901002','5th',15000,'2024-01-18'
    ↪ ,'BANK'),
 ('ASH2001001','3rd',15000,'2023-07-10','MOBILE');
```

40.8 Supporting Function: Grade Points

```
DELIMITER $$
CREATE FUNCTION fn_grade_point (p_grade CHAR(2))
RETURNS DECIMAL(3,2)
DETERMINISTIC
BEGIN
    RETURN CASE p_grade
        WHEN 'A+' THEN 4.00  WHEN 'A'  THEN 3.75  WHEN 'A-' THEN 3.50
        WHEN 'B+' THEN 3.25  WHEN 'B'  THEN 3.00  WHEN 'B-' THEN 2.75
        WHEN 'C+' THEN 2.50  WHEN 'C'  THEN 2.25  WHEN 'D'  THEN 2.00
        WHEN 'F'  THEN 0.00  ELSE NULL
    END;
END$$
DELIMITER ;
```

40.9 Stored Procedures

Recompute CGPA (credit-weighted)

```
DELIMITER $$
CREATE PROCEDURE sp_recompute_cgpa (IN p_student CHAR(10))
BEGIN
    UPDATE student s
    SET s.cgpa = (
        SELECT ROUND(SUM(fn_grade_point(e.grade) * c.credit) / SUM(c.credit), 2)
        FROM   enrollment e JOIN course c ON c.course_id = e.course_id
        WHERE  e.student_id = s.student_id AND e.grade IS NOT NULL
    )
    WHERE s.student_id = p_student;
END$$
DELIMITER ;

CALL sp_recompute_cgpa('ASH1901001');
SELECT student_id, name, cgpa FROM student WHERE student_id = 'ASH1901001';
```

Expected output. Rahim's grades are A (3.75), A- (3.50), A (3.75) on 3-credit courses and A+ (4.00) on the 1.5-credit lab:

(3.75×3 + 3.50×3 + 3.75×3 + 4.00×1.5) / 10.5 = 39.00 / 10.5 = **3.71**.

student_id	name	cgpa
ASH1901001	Rahim Uddin	3.71

Real-life interpretation. CGPA is credit-weighted, so a 1.5-credit lab counts half as much as a 3-credit theory course — exactly the calculation printed on a transcript.

Enrol with prerequisite check and transaction

```
DELIMITER $$
CREATE PROCEDURE sp_enrol (
    IN  p_student  CHAR(10),
    IN  p_course   INT,
    IN  p_semester VARCHAR(10),
    OUT p_message  VARCHAR(200)
)
proc: BEGIN
```

```
    DECLARE v_missing INT DEFAULT 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        GET DIAGNOSTICS CONDITION 1 p_message = MESSAGE_TEXT;   -- read the error first
        ROLLBACK;
    END;

    -- BR8: every prerequisite must have been passed (grade present and not F)
    SELECT COUNT(*) INTO v_missing
    FROM   prerequisite p
    WHERE  p.course_id = p_course
      AND  NOT EXISTS (SELECT 1 FROM enrollment e
                       WHERE e.student_id = p_student
                         AND e.course_id  = p.prereq_id
                         AND e.grade IS NOT NULL AND e.grade <> 'F');
    IF v_missing > 0 THEN
        SET p_message = CONCAT('Rejected: ', v_missing, ' prerequisite(s) not passed');
        LEAVE proc;
    END IF;

    START TRANSACTION;
    INSERT INTO enrollment (student_id, course_id, semester) VALUES (p_student, p_course
    ↪ , p_semester);
    COMMIT;
    SET p_message = CONCAT('Enrolled ', p_student, ' in course ', p_course, ' for ',
    ↪ p_semester);
END$$
DELIMITER ;

CALL sp_enrol('ASH2001001', 3, '5th', @m);  SELECT @m;   -- has passed CSE2101: succeeds
CALL sp_enrol('ASH2001002', 3, '5th', @m);  SELECT @m;   -- has not: rejected
```

Expected output: `Enrolled ASH2001001 in course 3 for 5th`, then `Rejected: 1 prerequisite(s) not passed`.

Record a fee payment atomically

```
DELIMITER $$
CREATE PROCEDURE sp_pay_fee (IN p_student CHAR(10), IN p_semester VARCHAR(10),
                             IN p_amount DECIMAL(10,2), IN p_method VARCHAR(15))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    SELECT student_id FROM student WHERE student_id = p_student FOR UPDATE;   -- lock
    ↪ the student row
    INSERT INTO fee_payment (student_id, semester, amount, paid_on, method)
    VALUES (p_student, p_semester, p_amount, CURDATE(), p_method);
    COMMIT;
END$$
DELIMITER ;
```

40.10 Triggers

BR7: at most six courses per semester

```
DELIMITER $$
CREATE TRIGGER trg_enrol_limit
```

```
BEFORE INSERT ON enrollment
FOR EACH ROW
BEGIN
    IF (SELECT COUNT(*) FROM enrollment
        WHERE student_id = NEW.student_id AND semester = NEW.semester) >= 6 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'BR7: a student may take at most six courses per semester
    ↪ ';
    END IF;
END$$
DELIMITER ;
```

BR9: audit every grade change and keep CGPA current

```
DELIMITER $$
CREATE TRIGGER trg_grade_audit
AFTER UPDATE ON enrollment
FOR EACH ROW
BEGIN
    IF NOT (NEW.grade <=> OLD.grade) THEN
        INSERT INTO grade_audit (enrollment_id, old_grade, new_grade, changed_by,
    ↪ changed_at)
        VALUES (OLD.enrollment_id, OLD.grade, NEW.grade, CURRENT_USER(), NOW());
    END IF;
END$$
DELIMITER ;
```

The trigger only records the change. CGPA is recomputed by calling `sp_recompute_cgpa` explicitly inside the grade-correction transaction (Section 40.14) rather than from the trigger. This is a design choice, not a technical necessity: it keeps the trigger short, so every grade update stays fast, and it makes the CGPA recalculation visible in the workflow instead of hidden behind a side effect. (A MySQL trigger may update a *different* table such as `student`; what it may not do is modify the table that fired it.)

```
UPDATE enrollment SET grade = 'B+' WHERE student_id = 'ASH1901004' AND course_id = 3;
SELECT enrollment_id, old_grade, new_grade, changed_by FROM grade_audit;
```

enrollment_id	old_grade	new_grade	changed_by
10	B	B+	controller@localhost

40.11 Views

```
-- transcript view: one row per graded course
CREATE VIEW vw_transcript AS
SELECT s.student_id, s.name, d.dept_name, e.semester, c.course_code, c.title,
       c.credit, e.grade, fn_grade_point(e.grade) AS grade_point
FROM   enrollment e
JOIN   student s    ON s.student_id = e.student_id
JOIN   department d ON d.dept_id    = s.dept_id
JOIN   course c     ON c.course_id  = e.course_id
WHERE  e.grade IS NOT NULL;

-- public directory: no CGPA, phone or date of birth
CREATE VIEW vw_student_directory AS
SELECT s.student_id, s.name, d.dept_name, i.name AS advisor
FROM   student s
JOIN   department d ON d.dept_id = s.dept_id
```

```sql
LEFT   JOIN instructor i ON i.instructor_id = s.advisor_id;

-- fee status for the current semester
CREATE VIEW vw_fee_status AS
SELECT s.student_id, s.name, COALESCE(SUM(f.amount), 0) AS paid,
       15000 - COALESCE(SUM(f.amount), 0)              AS due
FROM   student s
LEFT   JOIN fee_payment f ON f.student_id = s.student_id AND f.semester = '5th'
GROUP  BY s.student_id, s.name;
```

40.12 Indexes

```sql
-- foreign keys used in joins (InnoDB indexes FKs automatically; listed for portability)
CREATE INDEX idx_student_dept    ON student(dept_id);
CREATE INDEX idx_student_advisor ON student(advisor_id);
CREATE INDEX idx_course_dept     ON course(dept_id);
CREATE INDEX idx_exam_course     ON examination(course_id);
CREATE INDEX idx_fee_student_sem ON fee_payment(student_id, semester);

-- covering index for the results portal's hottest query
CREATE INDEX idx_enrol_portal ON enrollment(student_id, semester, course_id, grade);

-- result lookups by student
CREATE INDEX idx_result_student ON result(student_id);
```

`EXPLAIN SELECT course_id, grade FROM enrollment WHERE student_id='ASH1901001' ORDER BY semester;` shows `Using index` — answered from `idx_enrol_portal` alone, with no table access and no sort.

40.13 Queries

Q1. Transcript of one student.

```sql
SELECT semester, course_code, title, credit, grade, grade_point
FROM   vw_transcript WHERE student_id = 'ASH1901001' ORDER BY semester, course_code;
```

semester	course_code	title	credit	grade	grade_point
1st	CSE1101	Structured Programming	3.0	A	3.75
3rd	CSE2101	Data Structures	3.0	A-	3.50
5th	CSE3201	Database Management Systems	3.0	A	3.75
5th	CSE3202	Database Systems Laboratory	1.5	A+	4.00

Q2. Course roster with instructor for the 5th semester (multi-table join).

```sql
SELECT c.course_code, c.title, i.name AS instructor, COUNT(e.enrollment_id) AS enrolled
FROM   teaches t
JOIN   course c     ON c.course_id     = t.course_id
JOIN   instructor i ON i.instructor_id = t.instructor_id
LEFT   JOIN enrollment e ON e.course_id = c.course_id AND e.semester = t.semester
WHERE  t.semester = '5th'
GROUP  BY c.course_code, c.title, i.name
ORDER  BY enrolled DESC;
```

Q3. Departmental performance (aggregation with HAVING).

```
SELECT d.dept_name, COUNT(*) AS graded_courses,
       ROUND(AVG(fn_grade_point(e.grade)), 2) AS avg_gp
FROM   enrollment e
JOIN   student s    ON s.student_id = e.student_id
JOIN   department d ON d.dept_id    = s.dept_id
WHERE  e.grade IS NOT NULL
GROUP  BY d.dept_name
HAVING COUNT(*) >= 2
ORDER  BY avg_gp DESC;
```

Q4. Students eligible for CSE3201 but not yet enrolled (EXISTS / NOT EXISTS).

```
SELECT s.student_id, s.name
FROM   student s
WHERE  s.dept_id = 'CSE'
  AND  NOT EXISTS (SELECT 1 FROM enrollment e WHERE e.student_id = s.student_id AND e.
    ↪ course_id = 3)
  AND  NOT EXISTS (SELECT 1 FROM prerequisite p WHERE p.course_id = 3
                   AND NOT EXISTS (SELECT 1 FROM enrollment e2
                                   WHERE e2.student_id = s.student_id
                                     AND e2.course_id  = p.prereq_id
                                     AND e2.grade IS NOT NULL AND e2.grade <> 'F'));
```

Q5. Students with fees outstanding.

```
SELECT student_id, name, paid, due FROM vw_fee_status WHERE due > 0 ORDER BY due DESC;
```

Q6. Each advisor's advisees and their average CGPA (self-contained join and aggregate).

```
SELECT i.name AS advisor, COUNT(s.student_id) AS advisees, ROUND(AVG(s.cgpa), 2) AS
    ↪ avg_cgpa
FROM   instructor i LEFT JOIN student s ON s.advisor_id = i.instructor_id
GROUP  BY i.instructor_id, i.name
ORDER  BY advisees DESC;
```

Q7. Top performer in each course examination (correlated subquery).

```
SELECT c.course_code, s.name, r.marks
FROM   result r
JOIN   examination x ON x.exam_id    = r.exam_id
JOIN   course c      ON c.course_id  = x.course_id
JOIN   student s     ON s.student_id = r.student_id
WHERE  r.marks = (SELECT MAX(r2.marks) FROM result r2 WHERE r2.exam_id = r.exam_id);
```

Q8. Everyone on campus — students and staff (UNION ALL).

```
SELECT name, email, 'Student' AS role, dept_id FROM student
UNION ALL
SELECT name, email, designation, dept_id FROM instructor
ORDER  BY dept_id, role, name;
```

Q9. Examination room utilization.

```
SELECT x.exam_id, c.course_code, x.room_id, r.capacity,
       COUNT(res.student_id) AS sat,
       ROUND(100 * COUNT(res.student_id) / r.capacity, 1) AS utilization_pct
FROM   examination x
JOIN   course c    ON c.course_id = x.course_id
```

```
JOIN   classroom r ON r.room_id   = x.room_id
LEFT   JOIN result res ON res.exam_id = x.exam_id
GROUP  BY x.exam_id, c.course_code, x.room_id, r.capacity;
```

Q10. Ranking students by CGPA within each department (window function).

```
SELECT dept_id, student_id, name, cgpa,
       RANK() OVER (PARTITION BY dept_id ORDER BY cgpa DESC) AS dept_rank
FROM   student
WHERE  cgpa IS NOT NULL
ORDER  BY dept_id, dept_rank;
```

40.14 Transactions

A grade correction by the examination controller changes the grade, writes the audit row (via the trigger) and recomputes CGPA, all atomically:

```
START TRANSACTION;
UPDATE enrollment SET grade = 'A-' WHERE student_id = 'ASH1901004' AND course_id = 3 AND
    ↪ semester = '5th';
CALL sp_recompute_cgpa('ASH1901004');
COMMIT;
```

If the procedure fails — for instance through a constraint violation — a `ROLLBACK` leaves the grade, the audit row and the CGPA exactly as they were. Result publication itself runs at `READ COMMITTED` so that readers see only committed grades; the controller's batch of corrections runs as one transaction so the portal never shows a half-corrected result sheet.

40.15 Security

```
CREATE ROLE role_controller, role_teacher, role_accounts, role_student_portal,
    ↪ role_report;

-- examination controller: may change grades (audited), cannot touch fees
GRANT SELECT ON ums_db.* TO role_controller;
GRANT UPDATE (grade) ON ums_db.enrollment TO role_controller;
GRANT EXECUTE ON PROCEDURE ums_db.sp_recompute_cgpa TO role_controller;

-- teachers: read transcripts and rosters only
GRANT SELECT ON ums_db.vw_transcript        TO role_teacher;
GRANT SELECT ON ums_db.vw_student_directory TO role_teacher;

-- accounts office: fees only
GRANT SELECT, INSERT ON ums_db.fee_payment TO role_accounts;
GRANT SELECT ON ums_db.vw_fee_status TO role_accounts;
GRANT EXECUTE ON PROCEDURE ums_db.sp_pay_fee TO role_accounts;

-- web portal: stored procedures and views, never base tables
GRANT EXECUTE ON PROCEDURE ums_db.sp_enrol TO role_student_portal;
GRANT SELECT ON ums_db.vw_transcript TO role_student_portal;

-- analysts: read-only
GRANT SELECT ON ums_db.* TO role_report;

CREATE USER 'controller'@'10.0.%.%' IDENTIFIED BY 'Ctrl#Strong2026!' REQUIRE SSL;
GRANT role_controller TO 'controller'@'10.0.%.%';
```

```
SET DEFAULT ROLE ALL TO 'controller'@'10.0.%.%';

CREATE USER 'portal_app'@'10.0.1.%' IDENTIFIED BY 'Portal#Strong2026!' REQUIRE SSL;
GRANT role_student_portal TO 'portal_app'@'10.0.1.%';
SET DEFAULT ROLE ALL TO 'portal_app'@'10.0.1.%';
```

Vendor note: `SET DEFAULT ROLE ALL TO user` is MySQL 8 syntax. MariaDB activates roles per user with `SET DEFAULT ROLE role_controller FOR 'controller'@'10.0.%.%';`. Every other statement in this chapter was executed successfully on a live server while preparing the book.

Concern	Measure in this design
Least privilege	Five roles; no application account can `DROP`, `DELETE` or alter the schema
SQL injection	The portal calls `sp_enrol` through parameterized driver calls only
Data exposure	Teachers and the portal read views that omit sensitive columns
Accountability	Every grade change is written to `grade_audit` with the database user and time
Transport security	`REQUIRE SSL` on all remote accounts
Recoverability	Nightly full backup, continuous binary-log archiving, point-in-time recovery tested monthly

40.16 Deployment and Operations Summary

Area	Decision
DBMS	MySQL 8, InnoDB
Storage	Data on RAID 10, binary logs on RAID 1 (Chapter 24)
High availability	Primary plus two replicas with automatic failover
Publication day	Read traffic to replicas; transcript results cached with a long TTL
Backup	Nightly full + binary logs archived off-site; restore test each month
Monitoring	p95/p99 latency, replication lag, buffer pool hit ratio, deadlocks

Chapter Summary

- The project applied the full database life cycle: requirements and business rules, ER modelling, mapping to relations, normalization, implementation, programming, indexing, transactions and security.
- Business rules were enforced at the strongest possible level: declarative constraints first, then triggers and procedures only where a rule spans rows or tables.
- The schema is in BCNF, with one documented, procedure-maintained denormalization (`cgpa`).
- Views, procedures and roles separate what each user group may see and do.
- A covering index serves the hottest portal query entirely from the index.
- The operational plan connects the design to RAID, replication, caching and backup decisions from earlier chapters.

Key Terms

Database life cycle · Requirements analysis · Business rule · Conceptual design · Logical design · Physical design · Junction table · Recursive relationship · Controlled denormalization · Credit-weighted CGPA · Audit trail · Covering index · Role-based security · Point-in-time recovery

Common Mistakes

1. Designing tables before writing down the business rules.
2. Enforcing in application code rules that the DBMS could enforce declaratively.
3. Cascading deletes into financial or audit tables.
4. Storing derived values such as CGPA without a single, controlled maintenance path.
5. Granting the web application direct access to base tables.
6. Adding indexes without identifying the queries they serve.
7. Planning backups without ever testing a restore.

Solved Problems

Problem 40.1 — Justify why `enrollment` has a surrogate primary key plus a unique natural key, while `teaches` uses its natural key as primary key.

Solution. `enrollment` is referenced by `grade_audit` and is updated frequently; a compact, stable integer key makes references and indexes efficient, while `UNIQUE (student_id, course_id, semester)` still enforces BR6. `teaches` is referenced by nothing and is small; its natural composite key is both unique and meaningful, so a surrogate would add a column and an index with no benefit.

Problem 40.2 — Compute Nila's CGPA from her grades: CSE1101 A+ (3.0 cr), CSE2101 A (3.0 cr), CSE3201 A (3.0 cr).

Solution. (4.00×3 + 3.75×3 + 3.75×3) / 9 = (12.00 + 11.25 + 11.25) / 9 = 34.50 / 9 = **3.83**.

Problem 40.3 — Explain why `grade_audit` has no foreign key to `enrollment`.

Solution. An audit trail must outlive the data it describes. With a foreign key and `ON DELETE CASCADE`, deleting an enrollment would erase its audit history — exactly when an investigator might need it. With `RESTRICT`, legitimate deletions would be blocked forever by old audit rows. Omitting the foreign key keeps the audit table append-only and independent, at the cost of allowing an audit row to reference an enrollment that no longer exists — which is the correct behaviour for an audit log.

Problem 40.4 — Which rules are enforced by constraints, which by triggers, and which by procedures, and why?

Solution. Constraints enforce every single-row or key rule: uniqueness (BR1, BR6), ranges (BR4, BR5, BR10) and referential rules (BR2, BR3). A trigger enforces BR7 because it requires counting other rows of the same table on every insert, from any source. A procedure enforces BR8 because the prerequisite check is part of a controlled workflow, needs a helpful message, and is the only path the portal is permitted to use. BR9 combines privileges (only the controller may update `grade`) with a trigger (every change is audited regardless of path).

Problem 40.5 — The results portal must serve 5,000 concurrent students on publication day. List the design elements from this chapter that make that possible.

Solution. The covering index `idx_enrol_portal` answers the transcript query from the index alone; `vw_transcript` gives the portal one simple, parameterized query; the portal's role reads only views and calls procedures, so no expensive ad-hoc queries run; results are cached after publication because they no longer change; read traffic goes to two replicas; connection pooling bounds database sessions; and monitoring on p99 latency and replication lag gives early warning.

Review Questions

1. List the functional requirements and business rules of the UMS.
2. Identify every entity, its key and its relationships with cardinality and participation.

3. Explain how each relationship in the ER diagram was mapped to tables.
4. Show that the schema is in BCNF.
5. Justify the documented denormalization of `cgpa`.
6. Explain the purpose of each trigger and procedure.
7. Explain the choice of referential actions for fee payments and enrollments.
8. Describe the views and the security role each supports.
9. Explain the covering index and the query it serves.
10. Describe the security and operational plan.

Multiple Choice Questions

1. `prerequisite` implements a (a) 1:N (b) recursive M:N (c) weak entity (d) specialization.
2. `department.head_id` is declared UNIQUE to enforce (a) total participation (b) the 1:1 ratio (c) BCNF (d) indexing.
3. BR7 (six courses) is enforced by a (a) CHECK (b) trigger (c) view (d) index.
4. `fee_payment` → `student` uses (a) CASCADE (b) RESTRICT (c) SET NULL (d) no action.
5. CGPA stored in `student` is a (a) normalization error (b) controlled denormalization (c) foreign key (d) view.
6. The portal account is granted (a) ALL on base tables (b) EXECUTE on procedures and SELECT on views (c) DROP (d) nothing.
7. A credit-weighted average divides by (a) course count (b) total credits (c) 4 (d) semesters.
8. `idx_enrol_portal` enables (a) a full scan (b) an index-only scan (c) a hash join (d) partitioning.
9. The audit table has no foreign key so that (a) inserts are faster (b) history survives deletions (c) it is normalized (d) it is encrypted.
10. The grade correction uses a transaction to guarantee (a) isolation only (b) atomicity of grade, audit and CGPA (c) faster writes (d) no locking.

Answers: 1-(b) 2-(b) 3-(b) 4-(b) 5-(b) 6-(b) 7-(b) 8-(b) 9-(b) 10-(b)

Short Questions

1. What is a business rule?
2. Why is `advisor_id` nullable?
3. What does `ON DELETE SET NULL` do for `head_id`?
4. Why is `course_code` an alternate key?
5. Which table implements SITS?
6. Why does `sp_enrol` use `NOT EXISTS`?
7. What does `<=>` mean in the audit trigger?
8. Why do teachers access a view instead of `enrollment`?
9. Name two measures ensuring recoverability.
10. Why is `REQUIRE SSL` set on remote accounts?

Long / Descriptive Questions

1. Present the complete requirements analysis, ER diagram and relational schema for the UMS, explaining every mapping decision.
2. Prove that the UMS schema is in BCNF and discuss the documented denormalization.
3. Explain the implementation of every business rule, justifying the choice of constraint, trigger or

procedure.

4. Write and explain five complex queries against the UMS covering joins, subqueries, aggregation, set operations and window functions.
5. Describe the security, performance and operational design of the UMS with reference to earlier chapters.

Practical Exercises

1. Run the complete SQL script of this chapter on MySQL 8 and verify the row count of every table.
2. Attempt a seventh enrollment for one student in one semester and record the trigger's error.
3. Change a grade as the controller and verify the audit row and the recomputed CGPA.
4. Extend the schema with an `attendance` table (student, course, date, present) and a view showing attendance percentage per course.
5. Create the five roles and users, log in as each, and verify that every permitted operation succeeds and every other is refused.

APPENDIX A

Database Design Case Studies

Chapter 40 developed the University Management System in full. This appendix applies the same method to four more domains. Each case study follows the sequence a designer uses in practice: requirements → business rules → entities and relationships → ER diagram → relational schema → normalization → SQL implementation → sample queries. All SQL is MySQL 8 and was executed on a live server while preparing this book.

A.1 Banking Management System

Requirements

A bank operates several branches. Customers open accounts at a branch; an account may be held jointly by several customers. Every deposit, withdrawal and transfer is recorded as a transaction against an account. Branches grant loans, which may have several co-borrowers, and each loan is repaid in instalments.

Business rules

#	Rule	Enforcement
B1	Account numbers and customer national IDs are unique	`PRIMARY KEY`, `UNIQUE`
B2	Every account belongs to exactly one branch	`NOT NULL` foreign key
B3	An account has at least one holder	Procedure that opens account and holder together
B4	A savings account balance may never be negative	`CHECK` + transaction logic
B5	Transaction amounts are positive; type is DEPOSIT, WITHDRAW or TRANSFER	`CHECK`
B6	A transfer debits one account and credits another atomically	Stored procedure with a transaction
B7	Financial history is never deleted	`ON DELETE RESTRICT` everywhere

Entities, relationships and ER diagram

Entity	Key	Attributes
BRANCH	`branch_id`	`branch_name`, `city`
CUSTOMER	`customer_id`	`name`, `nid`, `phone`, `address`
ACCOUNT	`acc_no`	`acc_type`, `balance`, `opened_on`, `branch_id`
TRANSACTION (weak)	(`acc_no`, `txn_no`)	`txn_time`, `txn_type`, `amount`, `balance_after`
LOAN	`loan_id`	`amount`, `rate`, `sanctioned_on`, `branch_id`
LOAN_PAYMENT (weak)	(`loan_id`, `instalment_no`)	`paid_on`, `amount`

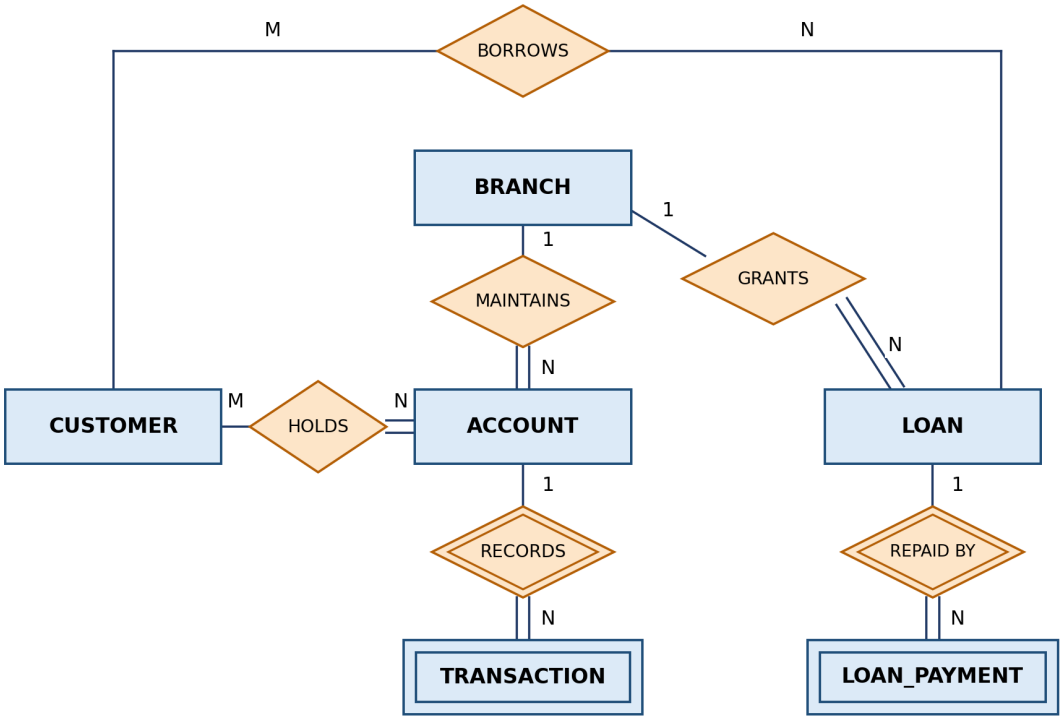


Figure A.1: ER diagram of the Banking Management System

HOLDS and BORROWS are M:N (joint accounts, co-borrowers); TRANSACTION and LOAN_PAYMENT are weak entities identified through their owners.

Relational schema and normalization

```
branch         (branch_id, branch_name, city)
customer       (customer_id, name, nid, phone, address)
account        (acc_no, acc_type, balance, opened_on, branch_id → branch)
account_holder(acc_no → account, customer_id → customer)
bank_txn       (acc_no → account, txn_no, txn_time, txn_type, amount, balance_after)
loan           (loan_id, amount, rate, sanctioned_on, branch_id → branch)
borrower       (loan_id → loan, customer_id → customer)
loan_payment   (loan_id → loan, instalment_no, paid_on, amount)
```

Every non-key attribute depends on the whole key and on nothing else: branch details are held only in `branch`, customer details only in `customer`. The schema is in BCNF. `balance` is technically derivable from the transactions but is stored deliberately and updated only inside the transfer and deposit procedures — the same controlled denormalization used for CGPA in Chapter 40.

SQL implementation

```sql
CREATE DATABASE bank_db;
USE bank_db;

CREATE TABLE branch (
    branch_id   CHAR(4)     PRIMARY KEY,
    branch_name VARCHAR(60) NOT NULL UNIQUE,
    city        VARCHAR(40) NOT NULL
);
CREATE TABLE customer (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
```

```
    nid         CHAR(13)     NOT NULL UNIQUE,
    phone       VARCHAR(20),
    address     VARCHAR(200)
);
CREATE TABLE account (
    acc_no    CHAR(10)      PRIMARY KEY,
    acc_type  VARCHAR(10)   NOT NULL CHECK (acc_type IN ('SAVINGS','CURRENT')),
    balance   DECIMAL(14,2) NOT NULL DEFAULT 0,
    opened_on DATE          NOT NULL,
    branch_id CHAR(4)       NOT NULL,
    CONSTRAINT ck_savings_nonneg CHECK (acc_type <> 'SAVINGS' OR balance >= 0),
    FOREIGN KEY (branch_id) REFERENCES branch(branch_id) ON DELETE RESTRICT
);
CREATE TABLE account_holder (
    acc_no      CHAR(10),
    customer_id INT,
    PRIMARY KEY (acc_no, customer_id),
    FOREIGN KEY (acc_no)      REFERENCES account(acc_no)       ON DELETE RESTRICT,
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id) ON DELETE RESTRICT
);
CREATE TABLE bank_txn (
    acc_no        CHAR(10),
    txn_no        INT,
    txn_time      DATETIME      NOT NULL,
    txn_type      VARCHAR(10)   NOT NULL CHECK (txn_type IN ('DEPOSIT','WITHDRAW','
    ↪ TRANSFER')),
    amount        DECIMAL(14,2) NOT NULL CHECK (amount > 0),
    balance_after DECIMAL(14,2) NOT NULL,
    PRIMARY KEY (acc_no, txn_no),
    FOREIGN KEY (acc_no) REFERENCES account(acc_no) ON DELETE RESTRICT
);
CREATE TABLE loan (
    loan_id       INT AUTO_INCREMENT PRIMARY KEY,
    amount        DECIMAL(14,2) NOT NULL CHECK (amount > 0),
    rate          DECIMAL(5,2)  NOT NULL CHECK (rate BETWEEN 0 AND 25),
    sanctioned_on DATE          NOT NULL,
    branch_id     CHAR(4)       NOT NULL,
    FOREIGN KEY (branch_id) REFERENCES branch(branch_id)
);
CREATE TABLE borrower (
    loan_id INT, customer_id INT,
    PRIMARY KEY (loan_id, customer_id),
    FOREIGN KEY (loan_id)     REFERENCES loan(loan_id),
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);
CREATE TABLE loan_payment (
    loan_id       INT,
    instalment_no INT,
    paid_on       DATE          NOT NULL,
    amount        DECIMAL(14,2) NOT NULL CHECK (amount > 0),
    PRIMARY KEY (loan_id, instalment_no),
    FOREIGN KEY (loan_id) REFERENCES loan(loan_id)
);

INSERT INTO branch VALUES ('DHK1','Dhaka Main','Dhaka'), ('NOA1','Maijdee','Noakhali');
INSERT INTO customer (name, nid, phone, address) VALUES
 ('Rahim Uddin','1990123456789','01712345678','Maijdee, Noakhali'),
 ('Salma Begum','1988123456780','01812345678','Maijdee, Noakhali'),
```

```
 ('Karim Mia',  '1992123456781','01912345678','Dhanmondi, Dhaka');
INSERT INTO account VALUES
 ('NOA1000001','SAVINGS',50000,'2022-01-10','NOA1'),
 ('NOA1000002','SAVINGS',12000,'2023-03-05','NOA1'),
 ('DHK1000001','CURRENT',250000,'2021-07-15','DHK1');
INSERT INTO account_holder VALUES ('NOA1000001',1), ('NOA1000001',2), ('NOA1000002',2),
    ↪ ('DHK1000001',3);
INSERT INTO loan (amount, rate, sanctioned_on, branch_id) VALUES (300000, 9.50, '
    ↪ 2024-01-01', 'NOA1');
INSERT INTO borrower VALUES (1,1), (1,2);
INSERT INTO loan_payment VALUES (1,1,'2024-02-01',15000), (1,2,'2024-03-01',15000);

DELIMITER $$
CREATE PROCEDURE sp_transfer (IN p_from CHAR(10), IN p_to CHAR(10), IN p_amount DECIMAL
    ↪ (14,2))
BEGIN
    DECLARE v_bal DECIMAL(14,2);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    -- lock both rows in a fixed order (by account number) to prevent deadlock
    SELECT balance INTO v_bal FROM account WHERE acc_no = LEAST(p_from, p_to) FOR UPDATE
    ↪ ;
    SELECT balance INTO v_bal FROM account WHERE acc_no = GREATEST(p_from, p_to) FOR
    ↪ UPDATE;
    UPDATE account SET balance = balance - p_amount WHERE acc_no = p_from;
    UPDATE account SET balance = balance + p_amount WHERE acc_no = p_to;
    INSERT INTO bank_txn
      SELECT p_from, COALESCE(MAX(txn_no),0)+1, NOW(), 'TRANSFER', p_amount,
             (SELECT balance FROM account WHERE acc_no = p_from)
      FROM bank_txn WHERE acc_no = p_from;
    INSERT INTO bank_txn
      SELECT p_to, COALESCE(MAX(txn_no),0)+1, NOW(), 'TRANSFER', p_amount,
             (SELECT balance FROM account WHERE acc_no = p_to)
      FROM bank_txn WHERE acc_no = p_to;
    COMMIT;
END$$
DELIMITER ;
```

The procedure combines four ideas from the book: atomicity through a transaction (Chapter 27), consistent lock ordering to prevent deadlock (Chapter 31), the `CHECK` constraint that makes an overdraft on a savings account roll the whole transfer back (Chapter 16), and error propagation with `RESIGNAL`.

Sample queries

```
-- Q1. Joint accounts and their holders
SELECT a.acc_no, GROUP_CONCAT(c.name ORDER BY c.name SEPARATOR ', ') AS holders
FROM   account a
JOIN   account_holder h ON h.acc_no = a.acc_no
JOIN   customer c       ON c.customer_id = h.customer_id
GROUP  BY a.acc_no
HAVING COUNT(*) > 1;

-- Q2. Total deposits held per branch
SELECT b.branch_name, COUNT(a.acc_no) AS accounts, SUM(a.balance) AS total_balance
FROM   branch b LEFT JOIN account a ON a.branch_id = b.branch_id
GROUP  BY b.branch_id, b.branch_name;

-- Q3. Outstanding principal per loan
```

```
SELECT l.loan_id, l.amount, COALESCE(SUM(p.amount),0) AS repaid,
       l.amount - COALESCE(SUM(p.amount),0) AS outstanding
FROM   loan l LEFT JOIN loan_payment p ON p.loan_id = l.loan_id
GROUP  BY l.loan_id, l.amount;

-- Q4. A transfer, then its effect
CALL sp_transfer('NOA1000001', 'NOA1000002', 5000);
SELECT acc_no, balance FROM account WHERE acc_no IN ('NOA1000001','NOA1000002');
```

Expected output of Q4: `NOA1000001` 45,000.00 and `NOA1000002` 17,000.00. Calling `sp_transfer('NOA1000002','NOA1000001', 99999)` fails the savings `CHECK` and leaves both balances unchanged.

A.2 Hospital Management System

Requirements

A hospital has departments staffed by doctors. Patients book appointments with doctors; during an appointment a doctor may prescribe several medicines, each with a dose and duration. Each appointment generates one bill.

Business rules

#	Rule	Enforcement
H1	A doctor belongs to exactly one department	`NOT NULL` foreign key
H2	A doctor cannot have two appointments at the same time	`UNIQUE (doctor_id, appt_time)`
H3	A prescription line exists only within an appointment	Weak entity, `ON DELETE CASCADE`
H4	Each appointment has at most one bill	`UNIQUE (appt_id)` on `bill`
H5	Medical data is visible only to clinical staff	Roles and views (Chapter 33)
H6	Appointment status is BOOKED, COMPLETED or CANCELLED	`CHECK`

Entities, relationships and ER diagram

Entity	Key	Attributes
DEPARTMENT	`dept_id`	`dept_name`, `floor`
DOCTOR	`doctor_id`	`name`, `specialization`, `phone`, `dept_id`
PATIENT	`patient_id`	`name`, `dob`, `gender`, `phone`, `blood_group`
APPOINTMENT	`appt_id`	`patient_id`, `doctor_id`, `appt_time`, `status`
PRESCRIPTION_ITEM (weak)	(`appt_id`, `line_no`)	`medicine_id`, `dose`, `days`
MEDICINE	`medicine_id`	`name`, `strength`, `unit_price`
BILL	`bill_id`	`appt_id`, `amount`, `paid`

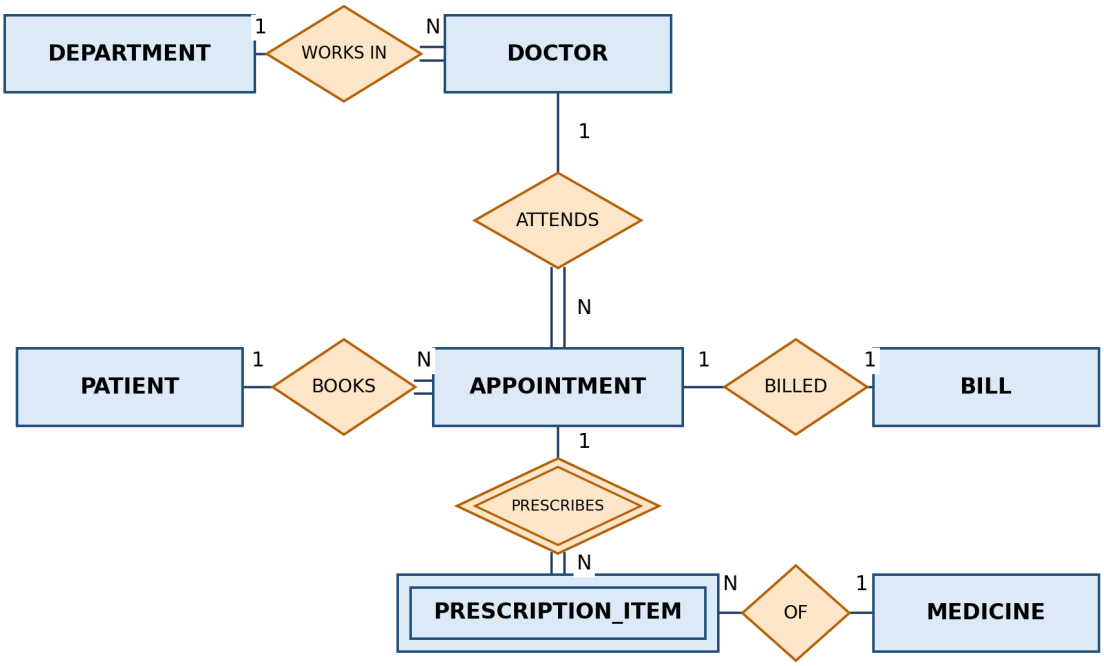


Figure A.2: ER diagram of the Hospital Management System

APPOINTMENT is modelled as an entity rather than a relationship, because prescriptions and bills must refer to it (Problem 7.7).

SQL implementation

```sql
CREATE DATABASE hospital_db;
USE hospital_db;

CREATE TABLE department (
    dept_id   CHAR(4)     PRIMARY KEY,
    dept_name VARCHAR(60) NOT NULL UNIQUE,
    floor     TINYINT
);
CREATE TABLE doctor (
    doctor_id      INT AUTO_INCREMENT PRIMARY KEY,
    name           VARCHAR(100) NOT NULL,
    specialization VARCHAR(60)  NOT NULL,
    phone          VARCHAR(20),
    dept_id        CHAR(4)      NOT NULL,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
CREATE TABLE patient (
    patient_id  INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    dob         DATE         NOT NULL,
    gender      CHAR(1)      CHECK (gender IN ('M','F','O')),
    phone       VARCHAR(20),
    blood_group VARCHAR(3)
);
CREATE TABLE appointment (
    appt_id    INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT      NOT NULL,
    doctor_id  INT      NOT NULL,
    appt_time  DATETIME NOT NULL,
```

```
    status     VARCHAR(10) NOT NULL DEFAULT 'BOOKED'
               CHECK (status IN ('BOOKED','COMPLETED','CANCELLED')),
    UNIQUE (doctor_id, appt_time),
    FOREIGN KEY (patient_id) REFERENCES patient(patient_id),
    FOREIGN KEY (doctor_id)  REFERENCES doctor(doctor_id)
);
CREATE TABLE medicine (
    medicine_id INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(80)  NOT NULL,
    strength    VARCHAR(20)  NOT NULL,
    unit_price  DECIMAL(8,2) NOT NULL CHECK (unit_price >= 0),
    UNIQUE (name, strength)
);
CREATE TABLE prescription_item (
    appt_id     INT,
    line_no     TINYINT,
    medicine_id INT         NOT NULL,
    dose        VARCHAR(30) NOT NULL,
    days        SMALLINT    NOT NULL CHECK (days > 0),
    PRIMARY KEY (appt_id, line_no),
    FOREIGN KEY (appt_id)     REFERENCES appointment(appt_id) ON DELETE CASCADE,
    FOREIGN KEY (medicine_id) REFERENCES medicine(medicine_id)
);
CREATE TABLE bill (
    bill_id INT AUTO_INCREMENT PRIMARY KEY,
    appt_id INT          NOT NULL UNIQUE,
    amount  DECIMAL(10,2) NOT NULL CHECK (amount >= 0),
    paid    BOOLEAN      NOT NULL DEFAULT FALSE,
    FOREIGN KEY (appt_id) REFERENCES appointment(appt_id)
);

INSERT INTO department VALUES ('MED','Medicine',2), ('CARD','Cardiology',3), ('PED','
    ↪ Paediatrics',1);
INSERT INTO doctor (name, specialization, phone, dept_id) VALUES
 ('Dr. Farhana Karim','Internal Medicine','01711000001','MED'),
 ('Dr. Anisur Rahman','Cardiologist','01711000002','CARD'),
 ('Dr. Nusrat Jahan','Paediatrician','01711000003','PED');
INSERT INTO patient (name, dob, gender, phone, blood_group) VALUES
 ('Abdul Karim','1965-04-12','M','01819000001','B+'),
 ('Rina Akter','1990-09-30','F','01819000002','O+'),
 ('Tamim Hasan','2018-02-14','M','01819000003','A+');
INSERT INTO appointment (patient_id, doctor_id, appt_time, status) VALUES
 (1,2,'2025-03-01 10:00','COMPLETED'), (2,1,'2025-03-01 11:00','COMPLETED'),
 (3,3,'2025-03-02 09:30','COMPLETED'), (1,2,'2025-04-01 10:00','BOOKED');
INSERT INTO medicine (name, strength, unit_price) VALUES
 ('Atorvastatin','20 mg',12.00), ('Amlodipine','5 mg',6.50), ('Paracetamol','500 mg'
    ↪ ,1.20);
INSERT INTO prescription_item VALUES
 (1,1,1,'1 tablet at night',30), (1,2,2,'1 tablet daily',30),
 (2,1,3,'1 tablet if fever',5),  (3,1,3,'5 ml syrup thrice',3);
INSERT INTO bill (appt_id, amount, paid) VALUES (1,1500,TRUE), (2,800,TRUE), (3,700,
    ↪ FALSE);
```

Sample queries

```
-- Q1. Today's schedule for each doctor
SELECT d.name AS doctor, a.appt_time, p.name AS patient, a.status
FROM   appointment a
```

```
JOIN   doctor d  ON d.doctor_id  = a.doctor_id
JOIN   patient p ON p.patient_id = a.patient_id
WHERE  DATE(a.appt_time) = '2025-03-01'
ORDER  BY d.name, a.appt_time;

-- Q2. Full prescription for appointment 1 with medicine cost
SELECT pi.line_no, m.name, m.strength, pi.dose, pi.days,
       m.unit_price * pi.days AS est_cost
FROM   prescription_item pi JOIN medicine m ON m.medicine_id = pi.medicine_id
WHERE  pi.appt_id = 1 ORDER BY pi.line_no;

-- Q3. Unpaid bills with patient contact details
SELECT p.name, p.phone, b.amount
FROM   bill b
JOIN   appointment a ON a.appt_id    = b.appt_id
JOIN   patient p     ON p.patient_id = a.patient_id
WHERE  b.paid = FALSE;

-- Q4. Most prescribed medicines
SELECT m.name, COUNT(*) AS times_prescribed
FROM   prescription_item pi JOIN medicine m ON m.medicine_id = pi.medicine_id
GROUP  BY m.medicine_id, m.name ORDER BY times_prescribed DESC;
```

Expected output of Q4: Paracetamol 2, Atorvastatin 1, Amlodipine 1.

Rule H2 is worth testing: inserting a second appointment for doctor 2 at `2025-03-01 10:00` fails with a duplicate-key error — the double booking is prevented by the schema, not by application code.

A.3 E-Commerce System

Requirements

Customers register with one or more delivery addresses and place orders. An order contains several products, each at the price charged at the time of ordering. Products belong to categories and have a stock level. An order may be paid in one or more payments.

Business rules

#	Rule	Enforcement
E1	Stock may never fall below zero	`CHECK (stock >= 0)`
E2	Placing an order and decrementing stock happen atomically	Stored procedure with transaction and row lock
E3	The price on an order line never changes when the catalogue price changes	`unit_price` stored on `order_item`
E4	A product appears at most once per order	`PRIMARY KEY (order_id, product_id)`
E5	Order status follows PENDING → PAID → SHIPPED → DELIVERED, or CANCELLED	`CHECK`

ER diagram

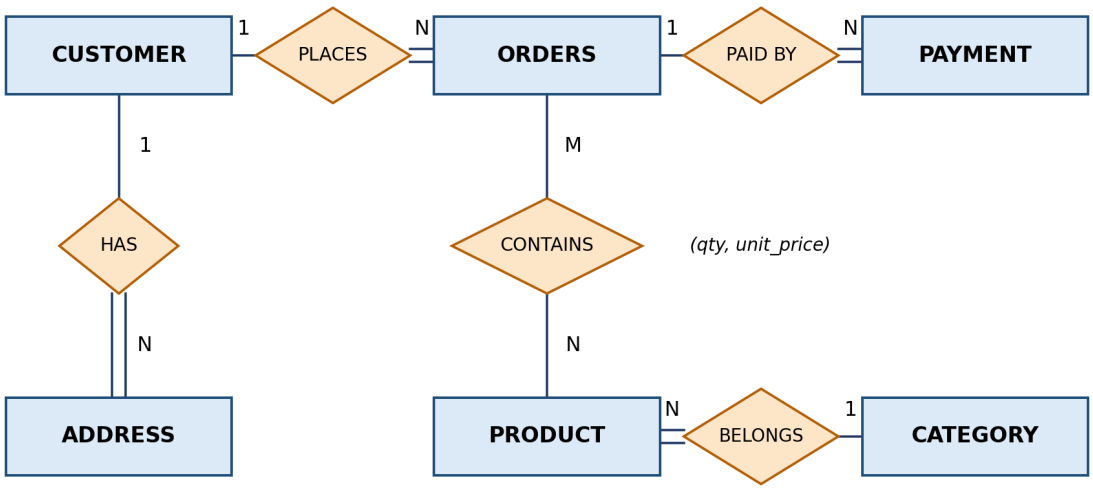


Figure A.3: ER diagram of the E-Commerce System

The M:N relationship CONTAINS carries the descriptive attributes `qty` and `unit_price` and becomes the junction table `order_item`.

SQL implementation

```sql
CREATE DATABASE shop_db;
USE shop_db;

CREATE TABLE customer (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(100) NOT NULL UNIQUE,
    phone       VARCHAR(20)
);
CREATE TABLE address (
    address_id  INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT          NOT NULL,
    line1       VARCHAR(120) NOT NULL,
    city        VARCHAR(40)  NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id) ON DELETE CASCADE
);
CREATE TABLE category (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(60) NOT NULL UNIQUE
);
CREATE TABLE product (
    product_id  INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(120)  NOT NULL,
    price       DECIMAL(10,2) NOT NULL CHECK (price > 0),
    stock       INT           NOT NULL DEFAULT 0 CHECK (stock >= 0),
    category_id INT           NOT NULL,
    FOREIGN KEY (category_id) REFERENCES category(category_id)
);
CREATE TABLE orders (
```

```
    order_id    INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT         NOT NULL,
    address_id  INT         NOT NULL,
    order_date  DATETIME    NOT NULL,
    status      VARCHAR(10) NOT NULL DEFAULT 'PENDING'
                CHECK (status IN ('PENDING','PAID','SHIPPED','DELIVERED','CANCELLED')),
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
    FOREIGN KEY (address_id)  REFERENCES address(address_id)
);
CREATE TABLE order_item (
    order_id   INT,
    product_id INT,
    qty        INT           NOT NULL CHECK (qty > 0),
    unit_price DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id)   REFERENCES orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES product(product_id)
);
CREATE TABLE payment (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id   INT           NOT NULL,
    amount     DECIMAL(10,2) NOT NULL CHECK (amount > 0),
    method     VARCHAR(10)   NOT NULL CHECK (method IN ('CARD','MOBILE','COD')),
    paid_at    DATETIME      NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

INSERT INTO customer (name, email, phone) VALUES
 ('Nila Rahman','nila@example.com','01711111111'), ('Tanvir Ahmed','tanvir@example.com',
    ↪ '01822222222');
INSERT INTO address (customer_id, line1, city) VALUES
 (1,'House 12, Road 3','Noakhali'), (2,'Flat 5B, Lake Road','Dhaka');
INSERT INTO category (name) VALUES ('Electronics'), ('Books');
INSERT INTO product (name, price, stock, category_id) VALUES
 ('USB-C Charger 65W',1800,40,1), ('Wireless Mouse',950,3,1),
 ('Database System Concepts',2400,10,2), ('Clean Code',1600,0,2);

DELIMITER $$
CREATE PROCEDURE sp_place_order (IN p_customer INT, IN p_address INT,
                                 IN p_product INT, IN p_qty INT, OUT p_order INT)
BEGIN
    DECLARE v_price DECIMAL(10,2);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    SELECT price INTO v_price FROM product WHERE product_id = p_product FOR UPDATE;
    UPDATE product SET stock = stock - p_qty WHERE product_id = p_product;   -- CHECK
    ↪ fails if short
    INSERT INTO orders (customer_id, address_id, order_date) VALUES (p_customer,
    ↪ p_address, NOW());
    SET p_order = LAST_INSERT_ID();
    INSERT INTO order_item VALUES (p_order, p_product, p_qty, v_price);
    COMMIT;
END$$
DELIMITER ;
```

`SELECT … FOR UPDATE` locks the product row, so two customers buying the last unit are serialized; the second one's `UPDATE` would drive stock negative, the `CHECK` fails, and the handler rolls back the entire order — no oversold stock and no half-created order (Chapters 28 and 30).

Sample queries

```sql
-- Q1. Place two orders; the second requests more mice than exist and is rolled back
CALL sp_place_order(1, 1, 1, 2, @o1);
CALL sp_place_order(2, 2, 2, 5, @o2);      -- ERROR: check constraint on stock fails

-- Q2. Order totals
SELECT o.order_id, c.name, SUM(oi.qty * oi.unit_price) AS total, o.status
FROM   orders o
JOIN   customer c    ON c.customer_id = o.customer_id
JOIN   order_item oi ON oi.order_id   = o.order_id
GROUP  BY o.order_id, c.name, o.status;

-- Q3. Products that need restocking
SELECT name, stock FROM product WHERE stock < 5 ORDER BY stock;

-- Q4. Revenue by category (including categories with no sales)
SELECT cat.name, COALESCE(SUM(oi.qty * oi.unit_price), 0) AS revenue
FROM   category cat
LEFT   JOIN product p     ON p.category_id = cat.category_id
LEFT   JOIN order_item oi ON oi.product_id = p.product_id
GROUP  BY cat.category_id, cat.name;
```

Expected output: Q2 shows one order for Nila with total 3,600.00; Q3 lists Clean Code (0) and Wireless Mouse (3); Q4 shows Electronics 3,600.00 and Books 0.00.

A.4 Library Management System

Requirements

The library holds book titles, each with one or more authors and one publisher, and several physical copies of each title. Members borrow copies for a fixed period and may reserve a title when no copy is available. Late returns incur a fine of 5 taka per day.

Business rules

#	Rule	Enforcement
L1	A copy can be on loan to at most one member at a time	Trigger checking for an open loan
L2	A member may hold at most five copies at once	Trigger
L3	Loan period is 14 days	`due_on` computed at issue
L4	Fine = 5 × days late, computed at return	Query / view
L5	A copy is identified by title and copy number	Weak entity `copy`

ER diagram

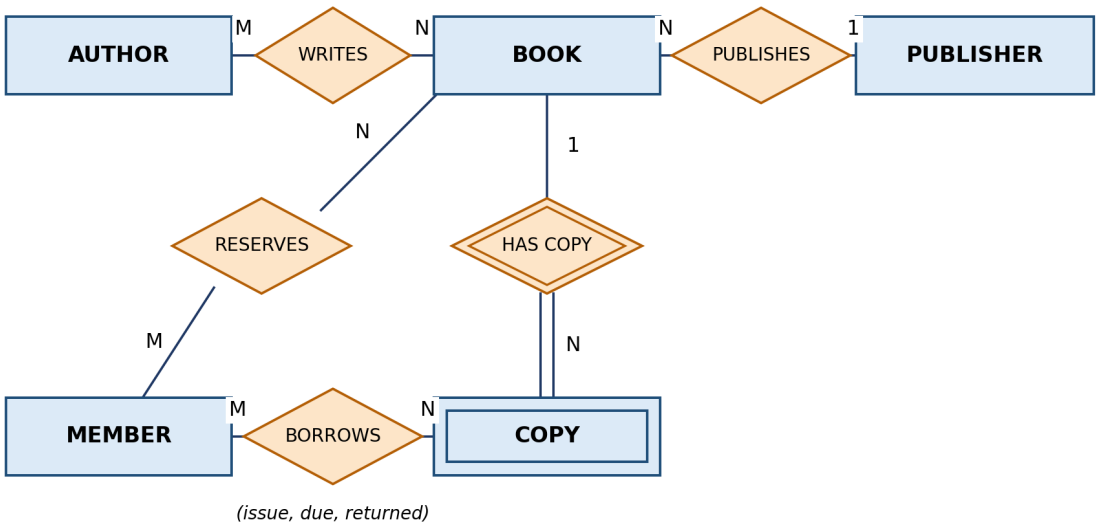


Figure A.4: ER diagram of the Library Management System

`COPY` is weak because "copy 3" means nothing without its title. BORROWS is M:N between MEMBER and COPY over time and carries issue, due and return dates, so it becomes the `loan` table.

SQL implementation

```sql
CREATE DATABASE library_db;
USE library_db;

CREATE TABLE publisher (
    publisher_id INT AUTO_INCREMENT PRIMARY KEY,
    name         VARCHAR(80) NOT NULL UNIQUE
);
CREATE TABLE book (
    isbn         CHAR(13)     PRIMARY KEY,
    title        VARCHAR(200) NOT NULL,
    pub_year     SMALLINT,
    publisher_id INT          NOT NULL,
    FOREIGN KEY (publisher_id) REFERENCES publisher(publisher_id)
);
CREATE TABLE author (
    author_id INT AUTO_INCREMENT PRIMARY KEY,
    name      VARCHAR(100) NOT NULL
);
CREATE TABLE book_author (
    isbn      CHAR(13),
    author_id INT,
    PRIMARY KEY (isbn, author_id),
    FOREIGN KEY (isbn)      REFERENCES book(isbn),
    FOREIGN KEY (author_id) REFERENCES author(author_id)
);
CREATE TABLE copy (
    isbn    CHAR(13),
    copy_no SMALLINT,
    shelf   VARCHAR(10) NOT NULL,
```

```
    PRIMARY KEY (isbn, copy_no),
    FOREIGN KEY (isbn) REFERENCES book(isbn) ON DELETE CASCADE
);
CREATE TABLE member (
    member_id INT AUTO_INCREMENT PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    email     VARCHAR(100) NOT NULL UNIQUE,
    joined_on DATE         NOT NULL
);
CREATE TABLE loan (
    loan_id     INT AUTO_INCREMENT PRIMARY KEY,
    member_id   INT      NOT NULL,
    isbn        CHAR(13) NOT NULL,
    copy_no     SMALLINT NOT NULL,
    issued_on   DATE     NOT NULL,
    due_on      DATE     NOT NULL,
    returned_on DATE     NULL,
    CHECK (due_on > issued_on),
    CHECK (returned_on IS NULL OR returned_on >= issued_on),
    FOREIGN KEY (member_id)     REFERENCES member(member_id),
    FOREIGN KEY (isbn, copy_no) REFERENCES copy(isbn, copy_no)
);
CREATE TABLE reservation (
    member_id   INT,
    isbn        CHAR(13),
    reserved_on DATE NOT NULL,
    PRIMARY KEY (member_id, isbn),
    FOREIGN KEY (member_id) REFERENCES member(member_id),
    FOREIGN KEY (isbn)      REFERENCES book(isbn)
);

DELIMITER $$
CREATE TRIGGER trg_loan_rules
BEFORE INSERT ON loan
FOR EACH ROW
BEGIN
    IF EXISTS (SELECT 1 FROM loan WHERE isbn = NEW.isbn AND copy_no = NEW.copy_no
               AND returned_on IS NULL) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'L1: copy is already on loan';
    END IF;
    IF (SELECT COUNT(*) FROM loan WHERE member_id = NEW.member_id AND returned_on IS
    ↪ NULL) >= 5 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'L2: member already holds five copies
    ↪ ';
    END IF;
END$$
DELIMITER ;

INSERT INTO publisher (name) VALUES ('McGraw-Hill'), ('Pearson');
INSERT INTO book VALUES
 ('9780078022159','Database System Concepts',2019,1),
 ('9780133970777','Fundamentals of Database Systems',2015,2);
INSERT INTO author (name) VALUES ('Abraham Silberschatz'), ('Henry F. Korth'),
 ('S. Sudarshan'), ('Ramez Elmasri'), ('Shamkant B. Navathe');
INSERT INTO book_author VALUES
 ('9780078022159',1), ('9780078022159',2), ('9780078022159',3),
 ('9780133970777',4), ('9780133970777',5);
```

```
INSERT INTO copy VALUES ('9780078022159',1,'D-12'), ('9780078022159',2,'D-12'), ('
    ↪ 9780133970777',1,'D-13');
INSERT INTO member (name, email, joined_on) VALUES
 ('Rahim Uddin','rahim@nstu.edu.bd','2023-01-10'), ('Nila Rahman','nila@nstu.edu.bd','
    ↪ 2023-02-01');
INSERT INTO loan (member_id, isbn, copy_no, issued_on, due_on, returned_on) VALUES
 (1,'9780078022159',1,'2025-02-01','2025-02-15','2025-02-20'),
 (2,'9780133970777',1,'2025-03-01','2025-03-15',NULL);
```

Sample queries

```
-- Q1. Books with their authors
SELECT b.title, GROUP_CONCAT(a.name ORDER BY a.name SEPARATOR '; ') AS authors
FROM   book b
JOIN   book_author ba ON ba.isbn = b.isbn
JOIN   author a       ON a.author_id = ba.author_id
GROUP  BY b.isbn, b.title;

-- Q2. Availability of each title
SELECT b.title, COUNT(c.copy_no) AS copies,
       SUM(EXISTS (SELECT 1 FROM loan l WHERE l.isbn = c.isbn AND l.copy_no = c.copy_no
                   AND l.returned_on IS NULL)) AS on_loan
FROM   book b JOIN copy c ON c.isbn = b.isbn
GROUP  BY b.isbn, b.title;

-- Q3. Fines on returned loans (5 taka per day late)
SELECT m.name, b.title, l.due_on, l.returned_on,
       GREATEST(DATEDIFF(l.returned_on, l.due_on), 0) * 5 AS fine
FROM   loan l
JOIN   member m ON m.member_id = l.member_id
JOIN   book b   ON b.isbn      = l.isbn
WHERE  l.returned_on IS NOT NULL;

-- Q4. Attempt to lend a copy that is already out (fails with L1)
INSERT INTO loan (member_id, isbn, copy_no, issued_on, due_on)
VALUES (1, '9780133970777', 1, '2025-03-05', '2025-03-19');
```

Expected output: Q3 shows Rahim, *Database System Concepts*, returned 5 days late, fine 25; Q4 is rejected with `L1: copy is already on loan`.

A.5 Comparing the Five Case Studies

Aspect	University (Ch. 40)	Banking	Hospital	E-commerce	Library
Central M:N relationship	Enrolment	Account holders	Prescriptions (via appointment)	Order items	Loans over time
Weak entities	—	Transaction, loan payment	Prescription item	—	Copy
Key integrity rule	Prerequisites, course limit	No overdraft on savings	No double booking	No negative stock	One open loan per copy
Mechanism	Procedure + trigger	Transaction + `CHECK`	`UNIQUE`	Row lock + `CHECK`	Trigger
Controlled denormalization	CGPA	Balance	—	Price at order time (historical, not redundant)	—

Aspect	University (Ch. 40)	Banking	Hospital	E-commerce	Library
Dominant concern	Correct results, publication load	Atomicity, audit	Privacy, scheduling	Concurrency on stock	Availability tracking

The comparison shows the recurring lesson of this book: the relational tools are the same in every domain, but which rule matters most — and therefore which mechanism the designer reaches for — is determined by the business.

APPENDIX B

Glossary

Terms are defined as used in this book. The number in brackets gives the chapter where the term is developed.

A

ACID — The four transaction guarantees: atomicity, consistency, isolation, durability. [27]

Aggregate function — A function that summarizes a set of rows into one value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. [12]

Aggregation (EER) — Treating a relationship set as a higher-level entity so that another relationship can refer to it. [8]

Alternate key — A candidate key not chosen as the primary key. [6]

Anomaly — An insertion, update or deletion side effect caused by storing several independent facts in one relation. [18]

ARIES — A recovery algorithm using write-ahead logging, repeating history, and analysis–redo–undo passes. [32]

Armstrong's axioms — Reflexivity, augmentation and transitivity: a sound and complete set of rules for inferring functional dependencies. [17]

Atomicity — A transaction's operations take effect entirely or not at all. [27]

Attribute — A named property of an entity or a column of a relation. [5, 7]

Attribute closure (X^+) — The set of all attributes functionally determined by X. [17]

Audit trail — A tamper-resistant record of who did what to which data, and when. [33]

Authentication — Verifying a user's identity. [33]

Authorization — Determining which operations an authenticated user may perform. [33]

B

B^+ tree — A balanced tree index whose leaves hold all data pointers and are chained in key order. [23]

BASE — Basically available, soft state, eventually consistent: the consistency model of many NoSQL systems. [35]

BCNF (Boyce–Codd normal form) — Every non-trivial functional dependency has a super key as its determinant. [19]

Blocking factor — The number of records that fit in one block: $\lfloor \text{block size} / \text{record size} \rfloor$. [22]

Buffer pool — The region of main memory in which the DBMS caches disk pages. [22, 39]

C

Candidate key — A minimal super key. [6]

CAP theorem — During a network partition, a distributed system must choose between consistency and availability. [35]

Cardinality (relation) — The number of tuples in a relation. [5]

Cardinality ratio — The number of entities that may participate in a relationship: 1:1, 1:N or M:N. [7]

Cascadeless schedule — A schedule in which transactions read only committed data. [29]

Checkpoint — A point recorded in the log from which restart recovery can begin. [32]

Clustered index — An index whose order determines the physical order of the table's rows. [23]

Composite key — A key consisting of two or more attributes. [6]

Conceptual schema — The single, complete logical description of the database in the three-schema architecture. [3]

Concurrency control — The DBMS mechanisms that keep concurrent transactions from interfering. [28–30]

Conflict serializability — Equivalence to a serial schedule in the order of all conflicting operations; tested with a precedence graph. [29]

Consistency — A transaction takes the database from one valid state to another. [27]

Covering index — An index containing every column a query needs, allowing an index-only scan. [23]

CTE (common table expression) — A named subquery introduced with `WITH`. [14]

D

Data — Raw facts without context. [1]

Data dictionary — The system catalog that stores the database's metadata. [1]

Data independence — The ability to change one schema level without changing the level above it; physical or logical. [3]

Data mart — A subject-specific subset of a data warehouse. [36]

Data model — A collection of concepts for describing data, relationships and constraints. [4]

Data warehouse — A subject-oriented, integrated, time-variant, non-volatile store for decision support. [36]

Database — A shared, integrated, persistent collection of logically related data. [1]

DBMS — Software that defines, stores, retrieves, protects and recovers databases. [1]

DDL — Data definition language: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`. [10]

Deadlock — A cycle of transactions each waiting for a lock held by another. [31]

Degree (relation) — The number of attributes of a relation. [5]

Denormalization — Deliberately reintroducing redundancy to improve read performance. [19]

Dense index — An index with one entry per record. [23]

Derived attribute — An attribute whose value is computed from others, such as age from date of birth. [7]

Dimension table — A warehouse table holding descriptive context for facts. [36]

Dirty read — Reading data written by an uncommitted transaction. [28]

Division (÷) — The relational algebra operator for "for all" queries. [20]

DML — Data manipulation language: `INSERT`, `UPDATE`, `DELETE`. [10]

Domain — The set of permitted atomic values of an attribute. [5]

Durability — Committed changes survive any subsequent failure. [27]

E

Entity — A distinguishable object about which data is stored. [7]

Entity integrity — No component of a primary key may be NULL. [5, 16]

Equi join — A join whose condition uses only equality. [13, 20]

ETL — Extract, transform, load: the process that feeds a data warehouse. [36]

Eventual consistency — If updates stop, all replicas eventually converge to the same value. [35]

External schema — A user's or application's view of the database. [3]

F

Fact table — A warehouse table of numeric measures and foreign keys to dimensions. [36]

Foreign key — Attribute(s) whose values must match a primary or unique key in another relation, or be NULL. [6]

Fragmentation — Division of a relation into pieces stored at different sites; horizontal or vertical. [34]

Functional dependency (X → Y) — Any two tuples agreeing on X must agree on Y. [17]

G–H

Generalization — Bottom-up combination of entity sets into a superclass. [8]

Hash index — An index that maps keys to buckets with a hash function; O(1) equality, no ranges. [23]

Heap file — An unordered file organization. [22]

High availability — The ability to remain operational despite component failures. [38]

I

Immediate update — A recovery technique in which changes may reach the database before commit, requiring undo. [32]

Index — An auxiliary structure mapping search-key values to record locations. [23]

Instance — The data in a relation or database at a particular moment (extension). [5]

Intention lock — A lock (IS, IX, SIX) signalling that finer-grained locks are held below a node. [30]

Isolation — Concurrent transactions do not see each other's intermediate states. [27]

Isolation level — READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ or SERIALIZABLE. [28]

J–L

Join — An operation combining rows of two relations on matching values. [13]

Junction table — A table implementing an M:N relationship, keyed on both foreign keys. [9]

Lock — A marker granting a transaction shared or exclusive access to an item. [30]

Log — A sequential record of every change, used for undo and redo. [32]

Lossless join — A decomposition whose natural join reproduces the original relation exactly. [19]

Lost update — One transaction's write overwrites another's, which is lost. [28]

M–N

Materialized view — A view whose result is stored and periodically refreshed. [15]

Metadata — Data describing data. [1]

MVCC — Multiversion concurrency control: writers create versions, readers see snapshots. [30]

Natural join (⋈) — An equi join on all commonly named attributes, keeping one copy of each. [13, 20]

NewSQL / distributed SQL — Systems offering SQL and ACID with horizontal scale via consensus. [38]

Normalization — Decomposing relations to eliminate redundancy and anomalies. [19]

NoSQL — Non-relational data stores: key-value, document, column-family and graph. [35]

NULL — A marker meaning "unknown or not applicable"; comparisons with it yield unknown. [5, 11]

O–P

OLAP — Online analytical processing: interactive multidimensional analysis. [36, 37]

OLTP — Online transaction processing: many short concurrent transactions. [36]

Parameterized query — A statement sent with placeholders and separately bound values; prevents SQL injection. [33]

Partial dependency — A non-prime attribute depending on part of a composite key. [17]

Participation — Whether every entity must take part in a relationship: total or partial. [7]

Partitioning — Dividing a table into physical pieces on one server. [39]

Phantom read — A repeated query returns a different set of rows because of a concurrent insert or delete. [28]

Precedence graph — A graph with an edge $T_i \to T_j$ for each conflict; acyclic iff conflict serializable. [29]

Primary key — The candidate key chosen to identify tuples; unique and non-NULL. [6]

Projection (π) — Relational algebra operator selecting columns and removing duplicates. [20]

Q–R

Query optimizer — The DBMS component that chooses a low-cost execution plan. [26]

RAID — Redundant array of independent disks: striping and/or redundancy across disks. [24]

Recoverable schedule — A schedule in which a transaction commits only after every transaction it read from has committed. [29]

Referential integrity — Every non-NULL foreign key value matches an existing referenced key. [5, 16]

Relation — A set of tuples over a relation schema; presented as a table. [5]

Relational algebra — A procedural query language of operators on relations. [20]

Relational calculus — A non-procedural query language based on predicate logic. [21]

Replication — Maintaining copies of data on several servers. [34, 38]

Role — A named collection of privileges that can be granted to users. [33]

S

Schedule — An ordering of the operations of concurrent transactions. [27, 29]

Schema — The description of a database's structure (intension). [5]

Selection (σ) — Relational algebra operator choosing rows by a predicate. [20]

Semi-join — A join that ships only join-attribute values to reduce distributed transfer. [34]

Serializability — Equivalence of a concurrent schedule to some serial schedule. [29]

Sharding — Partitioning data across servers by a shard key. [38]

Snowflake schema — A warehouse schema with normalized dimension tables. [36]

SQL injection — A vulnerability in which untrusted input alters the structure of an SQL statement. [33]

Star schema — A fact table joined directly to denormalized dimension tables. [36]

Stored procedure — A named program stored in the database and invoked with `CALL`. [15]

Strict two-phase locking — 2PL in which exclusive locks are held until commit. [30]

Subquery — A `SELECT` nested within another statement. [14]

Super key — A set of attributes that uniquely identifies tuples. [6]

Surrogate key — A system-generated identifier with no business meaning. [6]

T–Z

Three-schema architecture — External, conceptual and internal levels with mappings between them. [3]

Timestamp ordering — A concurrency protocol that orders conflicting operations by transaction timestamps. [30]

Transaction — A logical unit of work that executes atomically. [27]

Transitive dependency — A dependency of a non-prime attribute on a key through another non-prime attribute. [17]

Trigger — Code executed automatically in response to `INSERT`, `UPDATE` or `DELETE`. [15]

Tuple — A row of a relation. [5]

Two-phase commit — A protocol ensuring all sites of a distributed transaction commit or all abort. [34]

Two-phase locking — A protocol in which a transaction acquires all its locks before releasing any. [30]

View — A named query presented as a virtual table. [15]

Wait-for graph — A graph of transactions waiting for locks; a cycle indicates deadlock. [31]

Weak entity — An entity without its own key, identified through an owner entity. [7]

Write-ahead logging (WAL) — Log records must reach stable storage before the corresponding data pages and before commit. [32]

APPENDIX C

SQL Cheat Sheet

MySQL 8 syntax unless marked. `[ ]` = optional, `|` = alternatives.

C.1 DDL — Data Definition

```sql
CREATE DATABASE db_name;                    DROP DATABASE [IF EXISTS] db_name;
USE db_name;

CREATE TABLE t (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    code    VARCHAR(10) NOT NULL UNIQUE,
    amount  DECIMAL(10,2) DEFAULT 0 CHECK (amount >= 0),
    ref_id  INT,
    CONSTRAINT fk_t_ref FOREIGN KEY (ref_id) REFERENCES other(id)
        ON DELETE RESTRICT | CASCADE | SET NULL
        ON UPDATE CASCADE
);

ALTER TABLE t ADD COLUMN note VARCHAR(100);
ALTER TABLE t MODIFY COLUMN note VARCHAR(200);          -- PostgreSQL: ALTER COLUMN
    ↪ note TYPE ...
ALTER TABLE t RENAME COLUMN note TO remark;
ALTER TABLE t DROP COLUMN remark;
ALTER TABLE t ADD CONSTRAINT uq_t_code UNIQUE (code);
DROP TABLE [IF EXISTS] t;                               -- structure + data, cannot
    ↪ roll back
TRUNCATE TABLE t;                                       -- all rows, keeps structure,
    ↪ resets counter
```

C.2 DML — Data Manipulation

```sql
INSERT INTO t (code, amount) VALUES ('A1', 100), ('A2', 250);
INSERT INTO t (code, amount) SELECT code, amount FROM staging;
UPDATE t SET amount = amount * 1.10 WHERE code = 'A1';   -- always write WHERE first
DELETE FROM t WHERE amount = 0;
```

C.3 DQL — Querying

```sql
SELECT [DISTINCT] col, expr AS alias
FROM   t [AS a]
[JOIN  u ON u.id = a.ref_id]
[WHERE row_condition]
[GROUP BY col]
[HAVING group_condition]
[ORDER BY col [ASC|DESC]]
[LIMIT n [OFFSET m]];
```

Logical order of evaluation: `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER`

BY → LIMIT.

Predicate	Example
Comparison	`amount >= 100`, `code <> 'A1'`
Range	`amount BETWEEN 100 AND 200` (inclusive)
List	`dept_id IN ('CSE','EEE')`
Pattern	`name LIKE 'R%'` (`%` any run, `_` one char)
NULL test	`cgpa IS NULL`, `cgpa IS NOT NULL`
Existence	`EXISTS (SELECT 1 FROM …)`

C.4 Joins

```
SELECT … FROM a INNER JOIN b ON b.a_id = a.id;          -- matched rows only
SELECT … FROM a LEFT  JOIN b ON b.a_id = a.id;          -- all of a
SELECT … FROM a RIGHT JOIN b ON b.a_id = a.id;          -- all of b
SELECT … FROM a FULL OUTER JOIN b ON …;                 -- PostgreSQL; MySQL: LEFT UNION
    ↪ RIGHT
SELECT … FROM a CROSS JOIN b;                           -- Cartesian product
SELECT … FROM emp e LEFT JOIN emp m ON e.manager_id = m.emp_id;   -- self join
SELECT … FROM a LEFT JOIN b ON b.a_id = a.id WHERE b.a_id IS NULL; -- anti-join
```

C.5 Subqueries and Set Operations

```
WHERE x > (SELECT AVG(x) FROM t)                        -- single-row
WHERE id IN (SELECT a_id FROM b)                        -- multi-row
WHERE x > ALL (SELECT …)    /  > ANY (SELECT …)         -- ALL = MAX, ANY = MIN
WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.a_id = a.id)  -- safe with NULLs
WITH cte AS (SELECT …) SELECT … FROM cte;               -- common table expression
SELECT … UNION [ALL] SELECT …;  INTERSECT;  EXCEPT;     -- same column count and types
```

C.6 Functions

Group	Functions
Aggregate	`COUNT(*)`, `COUNT(col)`, `COUNT(DISTINCT col)`, `SUM`, `AVG`, `MIN`, `MAX`
String	`CONCAT`, `UPPER`, `LOWER`, `LENGTH`, `SUBSTRING`, `TRIM`, `REPLACE`, `LEFT`, `RIGHT`, `LPAD`
Numeric	`ROUND(x,d)`, `CEILING`, `FLOOR`, `TRUNCATE(x,d)`, `ABS`, `MOD`, `POWER`, `SQRT`
Date	`CURDATE()`, `NOW()`, `YEAR(d)`, `DATEDIFF(a,b)`, `TIMESTAMPDIFF(YEAR,a,b)`, `DATE_ADD`, `DATE_FORMAT`
NULL / conditional	`COALESCE(a,b,…)`, `IFNULL(a,b)`, `NULLIF(a,b)`, `CASE WHEN … THEN … ELSE … END`
Window	`RANK() OVER (PARTITION BY … ORDER BY …)`, `ROW_NUMBER()`, `SUM(x) OVER (…)`
OLAP	`GROUP BY a, b WITH ROLLUP` (MySQL); `ROLLUP(…)`, `CUBE(…)` (PostgreSQL)

C.7 Constraints

Constraint	Purpose
`NOT NULL`	Value required
`UNIQUE`	Alternate key; NULLs allowed
`PRIMARY KEY`	Unique + not NULL; one per table

Constraint	Purpose
`FOREIGN KEY … REFERENCES`	Referential integrity
`CHECK (condition)`	Domain rule (enforced in MySQL ≥ 8.0.16)
`DEFAULT value`	Used when the column is omitted

C.8 Views, Triggers, Procedures, Functions

```
CREATE [OR REPLACE] VIEW v AS SELECT … [WITH CHECK OPTION];

DELIMITER $$
CREATE TRIGGER trg_name BEFORE | AFTER  INSERT | UPDATE | DELETE ON t
FOR EACH ROW
BEGIN
    IF NEW.amount < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'amount must be non-negative';
    END IF;
END$$

CREATE PROCEDURE sp_name (IN p_id INT, OUT p_msg VARCHAR(100))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    -- statements
    COMMIT;
    SET p_msg = 'done';
END$$

CREATE FUNCTION fn_name (p_x DECIMAL(5,2)) RETURNS CHAR(2) DETERMINISTIC
BEGIN
    RETURN CASE WHEN p_x >= 80 THEN 'A' ELSE 'B' END;
END$$
DELIMITER ;

CALL sp_name(1, @m);  SELECT @m;
```

C.9 Indexes

```
CREATE INDEX idx_t_code ON t(code);
CREATE UNIQUE INDEX uq_t_email ON t(email);
CREATE INDEX idx_t_multi ON t(a, b, c);        -- serves a | a,b | a,b,c  (leftmost
    ↪ prefix)
DROP INDEX idx_t_code ON t;
EXPLAIN SELECT …;   EXPLAIN ANALYZE SELECT …;
```

C.10 DCL and TCL

```
CREATE USER 'u'@'host' IDENTIFIED BY 'strong-password';
CREATE ROLE r;  GRANT r TO 'u'@'host';  SET DEFAULT ROLE ALL TO 'u'@'host';
GRANT SELECT, INSERT ON db.t TO r;         GRANT UPDATE (col) ON db.t TO r;
REVOKE INSERT ON db.t FROM r;              SHOW GRANTS FOR 'u'@'host';

START TRANSACTION;  …  SAVEPOINT s1;  …  ROLLBACK TO SAVEPOINT s1;  COMMIT | ROLLBACK;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED | REPEATABLE READ | SERIALIZABLE;
SELECT … FOR UPDATE;   SELECT … FOR SHARE;
```

APPENDIX D

Important Formulas and Concepts

D.1 Functional Dependencies

Concept	Statement
FD $X \to Y$	$t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$
Reflexivity	$Y \subseteq X \Rightarrow X \to Y$
Augmentation	$X \to Y \Rightarrow XZ \to YZ$
Transitivity	$X \to Y, Y \to Z \Rightarrow X \to Z$
Union / Decomposition	$X \to Y, X \to Z \Leftrightarrow X \to YZ$ (right side only)
Pseudo-transitivity	$X \to Y, WY \to Z \Rightarrow WX \to Z$
Super key test	$X^+ = R$
FD test	$X \to Y$ holds iff $Y \subseteq X^+$
Candidate-key hints	Left-only and absent attributes are in every key; right-only attributes are in none

D.2 Normalization Rules

Normal form	Condition	Removes
1NF	All values atomic	Repeating groups
2NF	1NF + no partial dependency of a non-prime attribute on a candidate key	Partial dependencies
3NF	For every $X \to A$: X is a super key or A is prime	Transitive dependencies
BCNF	For every non-trivial $X \to Y$: X is a super key	All FD redundancy
4NF	For every non-trivial $X \to\to Y$: X is a super key	Multivalued dependencies
Lossless test	$(R_1 \cap R_2) \to R_1$ or $(R_1 \cap R_2) \to R_2$	—
Guarantee	3NF: lossless + dependency-preserving always; BCNF: lossless always, preserving not always	—

D.3 Relational Algebra

Operator	Symbol	Result
Selection	$\sigma_p(R)$	Rows of R satisfying p
Projection	$\pi_L(R)$	Columns L, duplicates removed
Union / Intersection / Difference	$\cup \cap -$	Set operations on union-compatible relations
Cartesian product	$R \times S$	$\lvert R\rvert \cdot \lvert S\rvert$ tuples, degree deg R + deg S
Theta join	$R \bowtie_\theta S$	$\sigma_\theta(R \times S)$
Division	$R \div S$	Tuples of R associated with all of S
Identities	$R \cap S = R - (R - S)$; $R \div S = \pi(R) - \pi((\pi(R) \times S) - R)$	—

D.4 Storage, Indexing and RAID

Quantity	Formula
Disk access time	seek + rotational latency + transfer
Average rotational latency	½ × 60,000 / rpm ms
Blocking factor	bfr = ⌊block size / record size⌋
Blocks in file	b = ⌈records / bfr⌉
Binary search cost	$\lceil \log_2 b \rceil$ block accesses
B^+ tree height	≈ ⌈log_⌈n/2⌉ N⌉
Index lookup cost	height + 1
RAID 0 / 1 / 5 / 6 / 10 capacity	n / n÷2 (mirror) / n−1 / n−2 / n÷2 disks' worth
RAID fault tolerance	0 / 1 per mirror / 1 / 2 / 1 per mirror pair
RAID 5 / 6 small-write cost	4 / 6 I/Os
Parity	$P = D_0 \oplus D_1 \oplus \ldots$; lost $D_i = P \oplus$ (other D)
Array MTTF without redundancy	disk MTTF ÷ number of disks

D.5 Query Cost

Algorithm	Cost in block transfers
Linear scan	b_R
Nested-loop join	n_R × b_S + b_R
Block nested-loop join	⌈b_R / (M − 2)⌉ × b_S + b_R
Indexed nested-loop join	b_R + n_R × c
Sort-merge join	sort(R) + sort(S) + b_R + b_S
Hash join	≈ 3 (b_R + b_S)
External sort passes	1 + ⌈log_{M−1}(b / M)⌉
Selectivity of A = v	1 / V(A, R)
Join size	n_R n_S / max(V(A,R), V(A,S))

D.6 Transactions and Serializability

Concept	Rule
Conflict	Different transactions, same item, at least one write
Precedence edge	$T_i \to T_j$ if a conflicting operation of T_i precedes one of T_j
Conflict serializable	Precedence graph acyclic; topological order = serial order
Schedule classes	strict ⊂ cascadeless ⊂ recoverable; conflict-serializable ⊂ view-serializable
2PL	No lock acquired after any lock released; serial order = lock-point order
Strict 2PL	Exclusive locks held until commit
Wait-die	Older waits; younger dies
Wound-wait	Older wounds younger; younger waits
Deadlock	Cycle in the wait-for graph
WAL	Log record before data page; commit record before acknowledgement
Restart recovery	Redo committed (forward), then undo incomplete (backward)
Quorum consistency	R + W > N
Availability downtime	(1 − availability) × 8,760 h/year

APPENDIX E

Viva Questions

One hundred questions of the kind asked in laboratory and final vivas, each with the short answer an examiner expects. Chapter numbers point to the full treatment.

Fundamentals (Chapters 1–5)

1. **What is the difference between data and information?** — Data are raw facts; information is data processed and placed in context so that it is meaningful.
2. **What is metadata, and where does a DBMS keep it?** — Data describing data (types, keys, constraints); stored in the data dictionary / system catalog.
3. **Distinguish database, DBMS and database system.** — The database is the stored data; the DBMS is the managing software; the database system is data + DBMS + hardware + procedures + people.
4. **Name four functions of a DBMS.** — Data definition, query processing, transaction management, recovery (also security, concurrency control, integrity).
5. **Give three drawbacks of file processing systems.** — Redundancy leading to inconsistency, no atomicity, no concurrency control (also weak security, difficult ad-hoc access).
6. **What is data redundancy?** — The same fact stored in more than one place.
7. **What is the role of a DBA?** — Security, backup and recovery, performance tuning, storage management, integrity and upgrades.
8. **Explain the three-schema architecture.** — External (user views), conceptual (logical schema), internal (physical storage), linked by two mappings.
9. **Physical versus logical data independence?** — Physical: change storage without changing the logical schema. Logical: change the logical schema without changing user views. Logical is harder.
10. **Two-tier versus three-tier architecture?** — Three-tier adds an application server holding business logic between client and database.
11. **What is a data model?** — A set of concepts for describing structure, operations and constraints of data.
12. **Why did the relational model dominate?** — Simplicity, declarative SQL, mathematical basis enabling optimization, data independence, normalization theory.
13. **Define relation, tuple and attribute.** — A relation is a set of tuples (a table); a tuple is a row; an attribute is a named column.
14. **What is a domain?** — The set of permitted atomic values for an attribute.
15. **Degree versus cardinality of a relation?** — Degree = number of attributes; cardinality = number of tuples.
16. **Schema versus instance?** — Schema is the structure (intension); instance is the data at a moment (extension).
17. **Why can a relation not contain duplicate tuples?** — Because a relation is a mathematical set.
18. **What is entity integrity?** — No part of a primary key may be NULL.
19. **What is referential integrity?** — Every non-NULL foreign key value must match an existing referenced key value.
20. **Does NULL equal zero?** — No. NULL means unknown; `NULL = NULL` evaluates to unknown.

Keys and ER Modelling (Chapters 6–9)

21. **Define super key.** — Any attribute set that uniquely identifies tuples.
22. **Define candidate key.** — A minimal super key.
23. **Primary key versus alternate key?** — The primary key is the chosen candidate key; the others are alternate keys.
24. **What is a composite key?** — A key of two or more attributes.
25. **What is a surrogate key?** — A system-generated identifier with no business meaning.
26. **Primary key versus unique key?** — One primary key per table and no NULLs; many unique keys, NULLs allowed.
27. **What does ON DELETE CASCADE do?** — Deleting a parent row deletes its child rows.
28. **What is an entity set?** — A collection of entities of the same type.
29. **Symbols for entity, relationship and attribute in an ER diagram?** — Rectangle, diamond, ellipse.
30. **What is a weak entity?** — An entity without its own key, identified through an owner entity via an identifying relationship.
31. **What is a partial key?** — The attribute that distinguishes weak entities of the same owner.
32. **Total versus partial participation?** — Total: every entity must participate (double line); partial: may not.
33. **What is a multivalued attribute and how is it mapped?** — An attribute with several values per entity; mapped to a separate table.
34. **What is a derived attribute?** — An attribute computed from others, e.g. age from date of birth.
35. **What is a ternary relationship?** — A relationship involving three entity sets in one indivisible fact.
36. **Specialization versus generalization?** — Specialization is top-down subclassing; generalization is bottom-up superclassing.
37. **Disjoint versus overlapping specialization?** — Disjoint: an entity in at most one subclass; overlapping: possibly several.
38. **What is aggregation in EER?** — Treating a relationship set as an entity so another relationship can involve it.
39. **How is an M:N relationship mapped to tables?** — A junction table containing both foreign keys as a composite key.
40. **Where does the foreign key go in a 1:N relationship?** — On the N (many) side.

SQL (Chapters 10–16)

41. **Name the five SQL command categories.** — DDL, DML, DQL, DCL, TCL.
42. **DELETE versus TRUNCATE versus DROP?** — DELETE removes selected rows (rollback possible); TRUNCATE removes all rows fast; DROP removes the table.
43. **Which of these can be rolled back in MySQL?** — DELETE.
44. **WHERE versus HAVING?** — WHERE filters rows before grouping; HAVING filters groups after grouping and may use aggregates.
45. **COUNT()* versus COUNT(column)?* — *COUNT()* counts rows; COUNT(column) counts non-NULL values.
46. **Why can a SELECT alias not be used in WHERE?** — WHERE is evaluated before SELECT.
47. **What does DISTINCT apply to?** — The entire select list.
48. **INNER JOIN versus LEFT JOIN?** — Inner keeps only matched rows; left keeps all rows of the left table, padding with NULL.
49. **What is a self join?** — A table joined to itself using two aliases.

50. **What is a Cartesian product?** — Every row of one table paired with every row of another.
51. **What is a correlated subquery?** — A subquery that references the outer query and is evaluated per outer row.
52. **Why is NOT IN dangerous?** — It returns no rows if the subquery produces a NULL; use NOT EXISTS.
53. **UNION versus UNION ALL?** — UNION removes duplicates; UNION ALL keeps them and is faster.
54. **What is a view?** — A named stored query acting as a virtual table.
55. **What does WITH CHECK OPTION do?** — Prevents inserts or updates through a view that would move rows out of the view.
56. **What is a trigger?** — Code executed automatically on INSERT, UPDATE or DELETE.
57. **NEW versus OLD in triggers?** — NEW is the incoming row (INSERT/UPDATE); OLD is the previous row (UPDATE/DELETE).
58. **Stored procedure versus function?** — A procedure is called with CALL and may return several OUT values; a function returns one value and can be used in expressions.
59. **Name the four categories of integrity constraint.** — Domain, entity, referential, semantic (business).
60. **Does DEFAULT apply when NULL is inserted explicitly?** — No; only when the column is omitted.

Design Theory and Query Languages (Chapters 17–21)

61. **What is a functional dependency?** — $X \to Y$: tuples equal on X must be equal on Y.
62. **State Armstrong's axioms.** — Reflexivity, augmentation, transitivity.
63. **What is attribute closure used for?** — Testing super keys and FDs, and finding candidate keys.
64. **Partial versus transitive dependency?** — Partial: a non-prime attribute depends on part of a key. Transitive: it depends on the key through another non-prime attribute.
65. **Name the three anomalies.** — Insertion, update, deletion.
66. **Define 1NF, 2NF and 3NF briefly.** — 1NF atomic values; 2NF no partial dependency; 3NF no transitive dependency.
67. **Define BCNF.** — Every determinant of a non-trivial FD is a super key.
68. **3NF versus BCNF?** — 3NF allows $X \to A$ when A is prime; BCNF does not. BCNF may lose dependency preservation.
69. **What is a lossless decomposition?** — One whose join reproduces the original relation without spurious tuples.
70. **What is denormalization?** — Deliberately adding redundancy to speed up reads.
71. **What are the six fundamental relational algebra operators?** — σ, π, ρ, $\cup$, $-$, $\times$.
72. **What does the division operator compute?** — Tuples related to all tuples of another relation ("for all").
73. **Relational algebra versus relational calculus?** — Algebra is procedural; calculus is non-procedural. They are equivalent in power (Codd's theorem).
74. **What is an unsafe calculus expression?** — One that can produce an infinite result.

Storage and Query Processing (Chapters 22–26)

75. **Which component dominates disk access time?** — Seek time.
76. **What is a blocking factor?** — The number of records per block.
77. **Dense versus sparse index?** — Dense: one entry per record; sparse: one per block, requires ordered data.

78. **B-tree versus B$^+$ tree?** — In a B$^+$ tree all data pointers are in chained leaves, giving higher fan-out and efficient range queries.
79. **When is a hash index unsuitable?** — For range queries and ORDER BY.
80. **What is the leftmost-prefix rule?** — A composite index can serve predicates on its leading columns only.
81. **What is a covering index?** — An index containing every column a query needs.
82. **Why does WHERE YEAR(date) = 2025 slow a query?** — The function prevents use of the index on the date column.
83. **RAID 0 versus RAID 1?** — RAID 0 stripes for speed without redundancy; RAID 1 mirrors for redundancy.
84. **Why is RAID 5 poor for write-heavy workloads?** — Each small write costs four I/Os to update parity.
85. **Which RAID level suits OLTP data files?** — RAID 10.
86. **Is RAID a backup?** — No; it protects only against disk failure.
87. **What does the query optimizer do?** — Chooses a low-cost execution plan among equivalent ones.
88. **Why push selections down in a query tree?** — To shrink intermediate results as early as possible.

Transactions and Recovery (Chapters 27–32)

89. **Expand ACID.** — Atomicity, consistency, isolation, durability.
90. **What is a dirty read?** — Reading data written by an uncommitted transaction.
91. **Non-repeatable read versus phantom?** — Non-repeatable: an existing row's value changes; phantom: rows appear or disappear.
92. **Name the four isolation levels.** — Read uncommitted, read committed, repeatable read, serializable.
93. **How is conflict serializability tested?** — By checking the precedence graph for cycles.
94. **What is two-phase locking?** — A growing phase acquiring locks, then a shrinking phase releasing them, with no acquisition after a release.
95. **What is a deadlock and how is it detected?** — A cycle of transactions waiting for each other; detected by a cycle in the wait-for graph.
96. **What is the write-ahead logging rule?** — Log records reach stable storage before the data pages they describe and before commit is acknowledged.
97. **Which transactions are redone and undone after a crash?** — Redo committed transactions; undo those active at the crash.

Modern Systems (Chapters 33–39)

98. **How do parameterized queries prevent SQL injection?** — The statement is parsed before values are bound, so input can never change its structure.
99. **State the CAP theorem.** — During a network partition, a distributed system must sacrifice either consistency or availability.
100. **OLTP versus OLAP?** — OLTP runs many short transactions on current data; OLAP runs large analytical queries on historical data.

APPENDIX F

Interview Questions

Interviewers rarely want a textbook definition alone; they want the definition, a reason, and an example from practice. Each model answer below is written in that shape.

F.1 Basic

1. **What is a DBMS, and why not just use files?** — Software that manages shared data with integrity, concurrency, security and recovery. Files offer none of these: two programs updating one file can corrupt it, and a crash can leave half-finished changes.
2. **What is a primary key? Can it change?** — The chosen unique, non-NULL identifier. It should not change, because every foreign key referencing it would have to change too; that is why surrogate keys are popular.
3. **Primary key versus foreign key?** — A primary key identifies rows in its own table; a foreign key refers to a key in another table and enforces referential integrity.
4. **What is normalization, in one sentence?** — Organizing tables so that each fact is stored once, eliminating update, insert and delete anomalies.
5. **What is a join?** — Combining rows of tables on matching values; it reassembles data that normalization stored separately.
6. **What is an index?** — A sorted auxiliary structure — usually a B^+ tree — that finds rows in logarithmic rather than linear time, at the cost of slower writes.
7. **What is a transaction?** — A unit of work that commits entirely or not at all, such as debiting one account and crediting another.
8. **What is a view, and why use one?** — A stored query. It simplifies complex joins, restricts what users can see, and shields applications from schema changes.
9. **What does NULL mean?** — Unknown or not applicable. It is not zero or an empty string, and any comparison with it is unknown.
10. **DELETE versus TRUNCATE?** — DELETE removes selected rows, is logged per row, fires triggers and can be rolled back; TRUNCATE empties the table almost instantly and resets auto-increment.
11. **What is SQL injection?** — A vulnerability where user input is concatenated into SQL and can alter the statement; prevented by parameterized queries.
12. **What is a constraint?** — A rule the DBMS enforces on every change, such as NOT NULL, UNIQUE, CHECK or FOREIGN KEY.

F.2 Intermediate

13. **Explain 2NF, 3NF and BCNF with one example each.** — 2NF removes `student_id → name` from an enrolment table keyed on (student, course); 3NF removes `dept_id → dept_name` from a student table; BCNF fixes cases like instructor → course where the instructor is not a key.
14. **When would you stop at 3NF instead of BCNF?** — When BCNF decomposition would lose a dependency that must be enforced, such as "a student has one instructor per course".
15. **Clustered versus non-clustered index?** — A clustered index determines the physical row order (InnoDB's primary key); a non-clustered index is a separate structure pointing to rows. A table has at most one clustered index.

16. **Why might the optimizer ignore an index?** — The predicate is unselective, the column is wrapped in a function, types mismatch, the pattern starts with a wildcard, or statistics are stale.
17. **What is a composite index, and does column order matter?** — An index on several columns; yes — it serves only predicates on its leftmost columns, so equality columns should come first.
18. **Explain the four isolation levels.** — Read uncommitted allows dirty reads; read committed prevents them; repeatable read also prevents non-repeatable reads; serializable prevents phantoms and gives full serial equivalence.
19. **What is the difference between a correlated subquery and a join?** — A correlated subquery is conceptually evaluated per outer row; a join combines sets once. Optimizers often convert one into the other, but a join is usually clearer and faster.
20. **What is a covering index?** — An index containing all columns a query needs, so the table is never touched.
21. **What is a deadlock, and how do you prevent one in application code?** — Two transactions waiting on each other's locks. Always lock rows and tables in a consistent order and keep transactions short.
22. **What is the N+1 query problem?** — Fetching a list and then running one query per item; replace with one join or an `IN` query.
23. **How do you find duplicate rows?** — `GROUP BY` the identifying columns with `HAVING COUNT(*) > 1`.
24. **Stored procedures: advantages and disadvantages?** — Centralized logic, fewer round trips and fine-grained permissions, versus vendor lock-in and harder testing and version control.
25. **What is a surrogate key, and when do you still need a natural key constraint?** — An artificial ID; you still need a UNIQUE constraint on the natural key or duplicates slip in.

F.3 Advanced

26. **Explain MVCC.** — Writers create new row versions; readers see a snapshot as of their start, so readers never block writers. Old versions must be garbage-collected (VACUUM, purge).
27. **What is write skew, and which isolation level prevents it?** — Two transactions read overlapping data and update disjoint rows, jointly violating a rule. Snapshot isolation allows it; serializable (SSI in PostgreSQL) prevents it.
28. **Explain the write-ahead logging rule.** — Log records reach stable storage before the data pages they describe, and the commit record before the commit is acknowledged; this makes steal/no-force buffer management safe.
29. **Why do databases use steal/no-force?** — No-force makes commits a cheap sequential log write; steal lets the buffer manager evict any page. The cost — needing both undo and redo — is cheap by comparison.
30. **Explain ARIES in one minute.** — Analysis finds active transactions and dirty pages; redo repeats history from the log; undo rolls back losers, logging compensation records so recovery itself is restartable.
31. **Why is two-phase commit called blocking?** — A participant that voted yes cannot decide alone; if the coordinator fails it must wait, holding locks.
32. **How does conflict serializability relate to 2PL?** — Every 2PL schedule is conflict serializable, equivalent to the serial order of lock points.
33. **What is a phantom and how do databases prevent it?** — A row appearing in a repeated range query; prevented by predicate or index-range (next-key) locking, or serializable snapshot checks.
34. **Explain cost-based optimization.** — The optimizer enumerates equivalent plans and estimates each from statistics — row counts, distinct values, histograms — choosing the cheapest.
35. **Why are cardinality estimates often wrong?** — Stale statistics, correlated columns violating

independence, skew, and errors compounding through joins.

36. **How does a hash join work?** — Build a hash table on the smaller input's join key, then probe it with the larger input; partition both if memory is insufficient.
37. **What is the difference between logical and physical replication?** — Physical ships changed pages or WAL bytes and produces an identical copy; logical ships row-level changes and can target different versions or subsets.
38. **What is split-brain?** — Two nodes both believing they are primary and accepting writes; prevented by quorum and fencing.

F.4 SQL

39. **Find the second-highest salary.** — `SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp);` or `DENSE_RANK()`.
40. **Find the Nth highest value.** — `SELECT DISTINCT salary FROM emp ORDER BY salary DESC LIMIT 1 OFFSET N-1;`
41. **List employees earning more than their manager.** — Self join: `SELECT e.name FROM emp e JOIN emp m ON e.manager_id = m.emp_id WHERE e.salary > m.salary;`
42. **Find customers who never placed an order.** — `SELECT c.* FROM customer c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);`
43. **Top three products per category.** — `ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY sales DESC)` in a CTE, then filter `rn <= 3`.
44. **Running total of daily sales.** — `SUM(amount) OVER (ORDER BY sale_date)`.
45. **Delete duplicate rows keeping one.** — Delete rows whose id is not the `MIN(id)` for their duplicate group: `DELETE t FROM t JOIN t d ON d.email = t.email AND d.id < t.id;`
46. **Pivot rows into columns.** — Conditional aggregation: `SUM(CASE WHEN month = 1 THEN amount END) AS jan, ….`
47. **Difference between WHERE and ON in a LEFT JOIN?** — A right-table condition in WHERE discards NULL-padded rows, turning it into an inner join; put it in ON to keep all left rows.
48. **What does GROUP BY … WITH ROLLUP produce?** — Subtotals per group level plus a grand total, marked with NULLs (distinguish with `GROUPING()`).
49. **How do you write an upsert?** — MySQL: `INSERT … ON DUPLICATE KEY UPDATE`; PostgreSQL: `INSERT … ON CONFLICT (key) DO UPDATE`.
50. **How do you paginate large result sets efficiently?** — Keyset pagination: `WHERE id > :last_id ORDER BY id LIMIT 20` instead of large OFFSETs.
51. **Why is 'SELECT *' discouraged in production code?** — It fetches unneeded columns, breaks when columns change, and prevents covering-index plans.

F.5 Database Design

52. **How would you design a schema for an e-commerce site?** — Customer, address, product, category, orders, order_item (with price at order time), payment, with stock updated transactionally (Appendix A.3).
53. **How do you model a many-to-many relationship with attributes?** — A junction table containing both foreign keys plus the attributes, e.g. enrolment with grade.
54. **How do you model a hierarchy such as an org chart?** — A self-referencing foreign key (adjacency list), queried with recursive CTEs; alternatives are nested sets or closure tables.
55. **How do you store an order's price if the product price changes later?** — Copy the price into the order line; it is a historical fact, not redundancy.
56. **When would you denormalize?** — For measured read bottlenecks on derived data — reporting

tables, counters, warehouse dimensions — with one controlled update path.

57. **How do you model inheritance (e.g. Person → Student, Teacher)?** — Table per class, table per subclass, or single table with a discriminator; choice depends on constraints and query patterns.
58. **UUID or auto-increment primary key?** — Auto-increment is compact and index-friendly; UUIDs suit distributed generation but fragment B^+ tree inserts unless time-ordered (UUIDv7).
59. **How do you handle soft deletes?** — An `is_deleted` or `deleted_at` column, with views or filtered indexes so normal queries exclude deleted rows; beware unique constraints.
60. **How would you store audit history?** — Trigger-written audit tables or system-versioned temporal tables, kept append-only and without cascading foreign keys.
61. **How do you choose data types?** — Smallest type that fits, DECIMAL for money, dedicated DATE/TIME types, and consistent types across join columns.
62. **What is a star schema and why is it denormalized?** — A fact table surrounded by dimension tables; denormalized so analytical queries need few joins.
63. **How would you design for multi-tenancy?** — Shared tables with a tenant_id and row-level security, separate schemas, or separate databases — trading isolation against cost.

F.6 Transactions and Concurrency

64. **Explain ACID with a banking example.** — A transfer either fully happens (A), keeps totals valid (C), is invisible to others until done (I), and survives a crash once confirmed (D).
65. **How do you prevent a lost update?** — Atomic `UPDATE x = x + ?`, `SELECT … FOR UPDATE`, or optimistic locking with a version column.
66. **Optimistic versus pessimistic locking?** — Pessimistic locks before reading (good under contention); optimistic checks a version at write time and retries (good when conflicts are rare).
67. **How would you sell the last concert ticket safely?** — Lock the seat row (`FOR UPDATE`) or use a conditional update `… WHERE seat_id = ? AND status = 'FREE'` and check the affected-row count.
68. **What is MySQL's default isolation level and PostgreSQL's?** — REPEATABLE READ and READ COMMITTED respectively.
69. **What happens to a transaction chosen as a deadlock victim?** — It is rolled back with an error (MySQL 1213); the application should retry it.
70. **What is a savepoint?** — A marker inside a transaction to which you can roll back without abandoning the whole transaction.
71. **What is the difference between a lock timeout and a deadlock?** — A timeout occurs when a lock is held too long by someone else; a deadlock is a cycle that can never resolve without a rollback.
72. **Why should transactions be short?** — Long transactions hold locks, block others, bloat MVCC version storage, and lengthen recovery.
73. **What is a distributed transaction?** — One spanning several databases, coordinated with two-phase commit or a consensus protocol.
74. **How do you guarantee exactly-once processing of a payment callback?** — Store an idempotency key with a UNIQUE constraint and process only if the insert succeeds.
75. **What does `SELECT … FOR UPDATE SKIP LOCKED` enable?** — Job queues in which several workers take different rows without blocking each other.

F.7 Performance

76. **A query suddenly became slow. What do you check?** — The execution plan, estimated versus actual rows, recent data growth, statistics, missing or dropped indexes, and lock waits.
77. **How do you read an EXPLAIN plan?** — Access type per table (ALL is a full scan), chosen key,

estimated rows, and extras such as filesort or temporary.

78. **What causes a filesort?** — ORDER BY or GROUP BY columns that no index can supply in order.
79. **How many indexes are too many?** — When write latency suffers or indexes are unused or redundant; check usage statistics and remove the rest.
80. **What is connection pooling and why does it matter?** — Reusing open connections; opening connections is expensive and too many sessions exhaust server memory.
81. **What is partitioning, and when does it help?** — Splitting a large table by range, list or hash on one server; it helps through partition pruning and fast archiving.
82. **Replication versus sharding for scaling?** — Replicas scale reads; sharding scales writes and storage but complicates queries and transactions.
83. **How do you size the InnoDB buffer pool?** — Large enough for the working set, typically 50–75 % of RAM on a dedicated server; aim for a hit ratio above 99 %.
84. **Why monitor p99 latency rather than average?** — Averages hide the slow tail that users actually experience.
85. **How would you speed up a report scanning millions of rows?** — Covering index, precomputed summary or materialized view, partition pruning, running it on a replica, or moving it to a warehouse.
86. **Why is `OFFSET 100000` slow?** — The database must read and discard all skipped rows; use keyset pagination.
87. **What is caching's biggest risk?** — Stale data from missing or incorrect invalidation.
88. **How do you tune a slow INSERT workload?** — Batch inserts, fewer indexes, larger transactions, sequential primary keys, and adequate log file size.

F.8 Modern Databases

89. **SQL or NoSQL for a new project?** — Default to relational for structured data needing integrity and transactions; choose NoSQL for a specific need — massive scale, flexible documents, key-value caching or graph traversal.
90. **Explain the CAP theorem correctly.** — During a partition, choose consistency or availability; without partitions a system can offer both. PACELC adds a latency-versus-consistency trade-off.
91. **What is eventual consistency? Give a safe use.** — Replicas converge over time; fine for likes, feeds and view counts, not for balances.
92. **What is R + W > N?** — Quorum condition guaranteeing reads see the latest write in replicated stores.
93. **Document database: embed or reference?** — Embed data read together and owned by the parent; reference data shared across many parents.
94. **What is a graph database good at?** — Multi-hop relationship queries such as recommendations and fraud rings.
95. **What is NewSQL / distributed SQL?** — Systems (Spanner, CockroachDB, TiDB) that keep SQL and ACID while scaling horizontally using consensus replication.
96. **Data warehouse versus data lake?** — A warehouse holds cleaned, modelled data (schema-on-write); a lake holds raw data of any format (schema-on-read).
97. **What does "separation of storage and compute" mean in cloud databases?** — Data sits in shared durable storage and compute nodes scale independently.
98. **What are RPO and RTO?** — Maximum acceptable data loss and maximum acceptable recovery time.
99. **What is a vector database used for?** — Storing embeddings and answering nearest-neighbour similarity queries, e.g. for semantic search and retrieval-augmented generation; PostgreSQL offers

it through extensions such as pgvector.

100. **How would you migrate a large production table with zero downtime?** — Add new structures compatibly, backfill in batches, dual-write or use an online schema-change tool, switch reads, then remove the old structure.

APPENDIX G

Final Examination Question Bank

This bank covers the whole book. Answer keys and model solutions follow each section where the answer is objective; descriptive questions reference the chapters that answer them. SQL problems use `university_db` (Chapter 11).

G.1 Multiple Choice Questions (50)

1. The software that manages a database is the (a) schema (b) DBMS (c) data dictionary (d) instance.
2. Metadata is stored in the (a) log (b) buffer (c) data dictionary (d) index.
3. Changing an index without changing applications demonstrates (a) logical (b) physical (c) view (d) external data independence.
4. The number of tuples in a relation is its (a) degree (b) arity (c) cardinality (d) domain.
5. A minimal super key is a (a) foreign key (b) candidate key (c) composite key (d) surrogate key.
6. A relation of degree 4 with one single-attribute candidate key has how many super keys containing it? (a) 4 (b) 7 (c) 8 (d) 16.
7. A weak entity is drawn as a (a) diamond (b) double rectangle (c) dashed ellipse (d) double ellipse.
8. An M:N relationship maps to (a) a foreign key on either side (b) a junction table (c) a view (d) a trigger.
9. Which is DDL? (a) INSERT (b) GRANT (c) TRUNCATE (d) COMMIT.
10. Aggregates are allowed in (a) WHERE (b) HAVING (c) FROM (d) ON.
11. `COUNT(col)` ignores (a) duplicates (b) NULLs (c) zeros (d) negatives.
12. A LEFT JOIN of A (6 rows) and B, with 4 A rows matched once each, returns (a) 4 (b) 6 (c) 10 (d) 24 rows.
13. `NOT IN (…)` returns no rows when the subquery contains (a) zero (b) duplicates (c) NULL (d) strings.
14. A view containing GROUP BY is (a) updatable (b) not updatable (c) materialized (d) indexed.
15. `NEW` is unavailable in a (a) BEFORE INSERT (b) AFTER UPDATE (c) BEFORE DELETE (d) AFTER INSERT trigger.
16. $X \to Y$ holds iff (a) $X \subseteq Y^+$ (b) $Y \subseteq X^+$ (c) $X^+ = Y^+$ (d) $X \cap Y = \varnothing$.
17. A partial dependency violates (a) 1NF (b) 2NF (c) 3NF only (d) 4NF.
18. BCNF requires every determinant to be a (a) prime attribute (b) super key (c) foreign key (d) single attribute.
19. Which is always achievable with dependency preservation? (a) BCNF (b) 3NF (c) 4NF (d) 5NF.
20. Decomposition R(A,B,C) into (A,B) and (B,C) is lossless if (a) $A \to B$ (b) $B \to C$ (c) $C \to A$ (d) always.
21. Which is not a fundamental relational algebra operator? (a) σ (b) π (c) $\bowtie$ (d) $\times$.
22. π in relational algebra (a) sorts (b) removes duplicates (c) joins (d) groups.
23. "For all" queries use (a) union (b) division (c) projection (d) rename.
24. Tuple relational calculus is (a) procedural (b) non-procedural (c) physical (d) navigational.
25. The largest component of disk access time is usually (a) transfer (b) seek (c) bus (d) CPU.
26. A sparse index requires the data file to be (a) hashed (b) ordered (c) heap (d) encrypted.
27. B^+ tree leaves are (a) unordered (b) chained (c) empty (d) in the root.

28. Hash indexes cannot support (a) equality (b) ranges (c) inserts (d) deletes.
29. RAID 5 with 6 disks of 2 TB offers (a) 12 TB (b) 10 TB (c) 8 TB (d) 6 TB.
30. RAID 6 tolerates (a) 0 (b) 1 (c) 2 (d) 3 disk failures.
31. Block nested-loop join should use the (a) larger (b) smaller (c) indexed (d) sorted relation as outer.
32. Hash join cost is approximately (a) b_R × b_S (b) 3(b_R + b_S) (c) n_R × n_S (d) log b_R.
33. The first heuristic of optimization is (a) join early (b) push selections down (c) sort early (d) use products.
34. "All or nothing" is (a) consistency (b) atomicity (c) isolation (d) durability.
35. Reading uncommitted data is a (a) phantom (b) dirty read (c) lost update (d) deadlock.
36. PostgreSQL's default isolation level is (a) READ UNCOMMITTED (b) READ COMMITTED (c) REPEATABLE READ (d) SERIALIZABLE.
37. A schedule is conflict serializable iff its precedence graph is (a) connected (b) acyclic (c) complete (d) a tree.
38. Under 2PL, a transaction may not acquire locks after it has (a) read (b) written (c) released a lock (d) committed.
39. Wound-wait: an older transaction requesting a younger's lock (a) waits (b) dies (c) wounds the younger (d) times out.
40. Deadlock detection uses a (a) precedence graph (b) wait-for graph (c) B^+ tree (d) hash table.
41. WAL requires the log record to be written (a) after (b) before (c) instead of (d) independently of the data page.
42. After a crash, transactions active at the crash are (a) redone (b) undone (c) committed (d) ignored.
43. Authorization determines (a) identity (b) permitted actions (c) encryption keys (d) backups.
44. The primary defence against SQL injection is (a) escaping quotes (b) parameterized queries (c) views (d) long passwords.
45. Splitting a table by rows across sites is (a) vertical fragmentation (b) horizontal fragmentation (c) replication (d) normalization.
46. In CAP, an AP system under partition sacrifices (a) availability (b) consistency (c) partition tolerance (d) durability.
47. Cassandra is a (a) graph (b) document (c) column-family (d) key-value only store.
48. A star schema's dimensions are (a) normalized (b) denormalized (c) absent (d) fact tables.
49. Fixing one dimension of a cube to a single value is (a) dice (b) slice (c) pivot (d) roll-up.
50. Sharding primarily scales (a) reads only (b) writes and storage (c) security (d) backups.

Answer key: 1-b 2-c 3-b 4-c 5-b 6-c 7-b 8-b 9-c 10-b 11-b 12-b 13-c 14-b 15-c 16-b 17-b 18-b 19-b 20-b 21-c 22-b 23-b 24-b 25-b 26-b 27-b 28-b 29-b 30-c 31-b 32-b 33-b 34-b 35-b 36-b 37-b 38-c 39-c 40-b 41-b 42-b 43-b 44-b 45-b 46-b 47-c 48-b 49-b 50-b

Notes on the computational items. Q6: supersets of {A} from the other 3 attributes = $2^3 = 8$. Q12: 4 matched + 2 unmatched = 6. Q20: common attribute B is a key of (B,C) when B → C. Q29: $(6 - 1) \times 2$ = 10 TB.

G.2 Short Questions (50)

1. Define metadata.
2. What is a mini-world?
3. State two limitations of a DBMS.
4. What is data isolation in file systems?
5. What is an external schema?

6. Define logical data independence.
7. Name three data models other than relational.
8. What is an atomic value?
9. State the entity integrity rule.
10. What is a dangling reference?
11. Distinguish super key and candidate key.
12. What is a surrogate key?
13. What does `ON DELETE SET NULL` do?
14. What is total participation?
15. What is a partial key?
16. Define a recursive relationship.
17. What is a shared subclass?
18. How is a multivalued attribute mapped?
19. Name the five SQL command categories.
20. Why use `DECIMAL` for money?
21. What is the logical order of `SELECT` clauses?
22. What does `BETWEEN` include?
23. What is an anti-join?
24. What is a CTE?
25. What does `WITH CHECK OPTION` prevent?
26. When is a trigger preferable to a CHECK constraint?
27. State Armstrong's transitivity axiom.
28. What is a canonical cover?
29. Define a transitive dependency.
30. What is a spurious tuple?
31. What is a multivalued dependency?
32. Express $R \cap S$ using primitive operators.
33. What is Codd's theorem?
34. Define the blocking factor.
35. What is a slotted page?
36. State the leftmost-prefix rule.
37. What is a covering index?
38. What is the RAID 5 small-write penalty?
39. Why is RAID not a backup?
40. What is pipelining in query evaluation?
41. Give the selectivity estimate for `A = v`.
42. Distinguish partially committed from committed.
43. What is a phantom read?
44. What is a cascadeless schedule?
45. What is strict 2PL?
46. What is Thomas' write rule?
47. State the Coffman conditions for deadlock.
48. What is a checkpoint?
49. What is point-in-time recovery?
50. Define eventual consistency.

Answers are in the chapter summaries and Appendix B; for example Q21 — FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT (Chapter 11); Q32 — $R - (R - S)$ (Chapter 20); Q38 — four I/Os per small write (Chapter 24).

G.3 Descriptive Questions (30)

1. Explain the data–information–knowledge–metadata hierarchy and the internal architecture of a DBMS. [1]
2. Compare file processing systems and DBMSs on at least ten criteria, with examples of each file-system problem. [2]
3. Explain the three-schema architecture and both forms of data independence with SQL examples. [3]
4. Compare hierarchical, network, relational, object-oriented and semi-structured data models. [4]
5. Explain the relational model's terminology and three integrity rules with SQL. [5]
6. Explain all kinds of key with examples from one relation and a containment diagram. [6]
7. Draw and explain a complete ER diagram for a hospital, including a weak entity and a ternary relationship. [7]
8. Explain specialization, generalization, constraints, categories and aggregation with an EER example. [8]
9. Describe the ER-to-relational mapping algorithm, applying every step to one example. [9]
10. Compare DELETE, TRUNCATE and DROP, and explain transaction control in SQL. [10]
11. Explain `GROUP BY` and `HAVING`, including NULL behaviour, with worked queries. [11–12]
12. Explain all join types with diagrams, output and the ON-versus-WHERE pitfall. [13]
13. Explain correlated subqueries, `EXISTS` and the `NOT IN` NULL trap. [14]
14. Explain views, triggers and stored procedures with complete examples. [15]
15. Derive candidate keys using attribute closure for a given relation and FD set. [17]
16. Demonstrate the three anomalies on an un-normalized relation and remove them. [18]
17. Explain 1NF–BCNF, lossless join and dependency preservation with examples. [19]
18. Explain relational algebra, including division, and translate five queries to SQL. [20]
19. Explain tuple and domain relational calculus and safety of expressions. [21]
20. Explain disk access time and file organizations with numerical analysis. [22]
21. Explain B^+ trees including insertion with splitting, and compare with hashing. [23]
22. Explain RAID levels 0, 1, 5, 6 and 10 and recommend a database server layout. [24]
23. Explain query processing stages and cost all join algorithms for given relations. [25]
24. Explain heuristic and cost-based optimization with a worked transformation. [26]
25. Explain ACID and the transaction state diagram. [27]
26. Explain concurrency anomalies and SQL isolation levels. [28]
27. Test given schedules for conflict and view serializability and recoverability. [29]
28. Explain 2PL and its variants, timestamp ordering and MVCC. [30]
29. Explain deadlock prevention, detection and recovery. [31]
30. Explain log-based recovery with WAL, checkpoints and ARIES. [32]

G.4 SQL Problems (20) with Solutions

All solutions run against `university_db`.

S1. List the name and CGPA of students in CSE, highest first.

```sql
SELECT name, cgpa FROM student WHERE dept_id = 'CSE' ORDER BY cgpa DESC;
```

S2. Count students per department, including departments with no students.

```sql
SELECT d.dept_id, COUNT(s.student_id) AS students
FROM department d LEFT JOIN student s ON s.dept_id = d.dept_id
GROUP BY d.dept_id;
```

S3. Find students whose name ends in “Hasan”.

```sql
SELECT name FROM student WHERE name LIKE '%Hasan';
```

S4. Show each enrollment with student name, course code and grade.

```sql
SELECT s.name, c.course_code, e.semester, e.grade
FROM enrollment e
JOIN student s ON s.student_id = e.student_id
JOIN course  c ON c.course_id  = e.course_id
ORDER BY s.name, c.course_code;
```

S5. Find courses with no enrollments.

```sql
SELECT course_code FROM course c
WHERE NOT EXISTS (SELECT 1 FROM enrollment e WHERE e.course_id = c.course_id);
```

S6. Average marks per course code, only for courses with at least two results.

```sql
SELECT c.course_code, ROUND(AVG(r.marks), 2) AS avg_marks
FROM result r
JOIN examination x ON x.exam_id  = r.exam_id
JOIN course c      ON c.course_id = x.course_id
GROUP BY c.course_code
HAVING COUNT(*) >= 2;
```

S7. Students with CGPA above their department’s average.

```sql
SELECT s.name, s.dept_id, s.cgpa FROM student s
WHERE s.cgpa > (SELECT AVG(s2.cgpa) FROM student s2 WHERE s2.dept_id = s.dept_id);
```

S8. The instructor(s) with the highest salary.

```sql
SELECT name, salary FROM instructor WHERE salary = (SELECT MAX(salary) FROM instructor);
```

S9. Second-highest CGPA without `LIMIT`.

```sql
SELECT MAX(cgpa) FROM student WHERE cgpa < (SELECT MAX(cgpa) FROM student);
```

S10. Students enrolled in both course 1 and course 2.

```sql
SELECT student_id FROM enrollment WHERE course_id IN (1, 2)
GROUP BY student_id HAVING COUNT(DISTINCT course_id) = 2;
```

S11. Number of graded and ungraded enrollments per semester.

```sql
SELECT semester, COUNT(grade) AS graded, COUNT(*) - COUNT(grade) AS ungraded
FROM enrollment GROUP BY semester ORDER BY semester;
```

S12. Rank students by CGPA within each department.

```sql
SELECT dept_id, name, cgpa,
```

```sql
       RANK() OVER (PARTITION BY dept_id ORDER BY cgpa DESC) AS dept_rank
FROM student WHERE cgpa IS NOT NULL;
```

S13. List everyone (students and instructors) with a role column.

```sql
SELECT name, 'Student' AS role FROM student
UNION ALL
SELECT name, 'Instructor' FROM instructor
ORDER BY role, name;
```

S14. Create a view of CSE students and query it.

```sql
CREATE OR REPLACE VIEW vw_cse AS
SELECT student_id, name, cgpa FROM student WHERE dept_id = 'CSE';
SELECT * FROM vw_cse WHERE cgpa > 3.8;
```

S15. Increase every Lecturer's salary by 5 % in a transaction.

```sql
START TRANSACTION;
UPDATE instructor SET salary = ROUND(salary * 1.05, 2) WHERE designation = 'Lecturer';
COMMIT;
```

S16. Classify students by CGPA band.

```sql
SELECT name,
       CASE WHEN cgpa IS NULL THEN 'Not graded'
            WHEN cgpa >= 3.75 THEN 'Distinction'
            WHEN cgpa >= 3.00 THEN 'First class'
            ELSE 'Pass' END AS band
FROM student;
```

S17. Examination room utilization as a percentage.

```sql
SELECT x.exam_id, r.room_id, r.capacity, COUNT(res.student_id) AS sat,
       ROUND(100 * COUNT(res.student_id) / r.capacity, 1) AS pct
FROM examination x
JOIN classroom r ON r.room_id = x.room_id
LEFT JOIN result res ON res.exam_id = x.exam_id
GROUP BY x.exam_id, r.room_id, r.capacity;
```

S18. Students admitted in 2020 who have at least one grade of A.

```sql
SELECT DISTINCT s.name FROM student s
JOIN enrollment e ON e.student_id = s.student_id
WHERE s.admitted_on BETWEEN '2020-01-01' AND '2020-12-31' AND e.grade = 'A';
```

S19. Department subtotals and grand total of instructor salaries.

```sql
SELECT dept_id, SUM(salary) AS total_salary
FROM instructor GROUP BY dept_id WITH ROLLUP;
```

S20. Create an index supporting "enrollments of a student ordered by semester", and verify it.

```sql
CREATE INDEX idx_enrol_stu_sem ON enrollment(student_id, semester);
EXPLAIN SELECT course_id, grade FROM enrollment
WHERE student_id = 'ASH1901001' ORDER BY semester;
```

G.5 ER Diagram Problems (10)

For each, draw the ER diagram, mark cardinality and participation, identify weak entities, and map to relations.

1. **Library:** titles, copies, members, loans, reservations. *(Model answer: Appendix A.4 — COPY is weak.)*
2. **Hospital:** departments, doctors, patients, appointments, prescriptions. *(Appendix A.2 — APPOINTMENT as an entity; PRESCRIPTION_ITEM weak.)*
3. **Bank:** branches, customers, joint accounts, transactions, loans. *(Appendix A.1.)*
4. **Airline:** airports, flights (flight number, origin, destination), scheduled flight instances by date, passengers, bookings with seat and class. *(Key idea: FLIGHT_INSTANCE is weak, identified by (flight_no, date); BOOKING relates PASSENGER and FLIGHT_INSTANCE with seat; UNIQUE (flight_no, date, seat).)*
5. **Food delivery:** restaurants, menu items, customers, orders, order items, delivery agents. *(MENU_ITEM weak or keyed by restaurant; ORDER 1:N ORDER_ITEM; AGENT 1:N ORDER, partial.)*
6. **University timetable:** courses, sections, rooms, time slots, instructors. *(SECTION weak of COURSE; ternary SCHEDULED(section, room, slot) with UNIQUE (room, slot).)*
7. **E-commerce with reviews:** add REVIEW relating CUSTOMER and PRODUCT with rating, allowed only for purchased products. *(M:N with attributes; purchase rule enforced by trigger or procedure.)*
8. **Government citizen services:** citizens, national IDs, service applications, documents, officers. *(Citizen 1:1 NID; APPLICATION M:N DOCUMENT; officer 1:N application, partial.)*
9. **Vehicle registration:** vehicles owned by persons or companies. *(Category OWNER over PERSON and COMPANY — Chapter 8.)*
10. **Project evaluation:** students work on projects under supervisors; examiners evaluate each student–project pair. *(Aggregation — Chapter 8.)*

G.6 Normalization Problems (10) with Answers

1. R(A,B,C,D), F = {A → B, B → C, C → D}. Key? Highest NF? — **Key A.** B → C and C → D are transitive ⇒ **2NF**. 3NF decomposition: (A,B), (B,C), (C,D).
2. R(A,B,C,D), F = {AB → C, C → D}. — **Key AB.** C → D transitive ⇒ **2NF**. Decompose: (A,B,C), (C,D).
3. R(A,B,C,D), F = {AB → C, B → D}. — **Key AB.** B → D is partial ⇒ **1NF**. Decompose: (A,B,C), (B,D).
4. R(A,B,C), F = {AB → C, C → B}. — **Keys AB and AC.** All attributes prime ⇒ **3NF**; C → B violates BCNF. BCNF: (C,B), (A,C) — lossless, loses AB → C.
5. R(A,B,C,D,E), F = {A → BC, CD → E, B → D, E → A}. — **Keys A, E, CD, BC.** (BC^+ = {B,C,D,E,A} .) Every attribute is prime ⇒ **3NF**; B → D violates BCNF.
6. `STUDENT(sid, name, dept, dept_head)`, sid → name, dept; dept → dept_head. — **2NF** (transitive). Decompose: STUDENT(sid, name, dept), DEPT(dept, dept_head).
7. `ENROL(sid, cid, sname, ctitle, grade)`, key (sid, cid). — **1NF** (partial dependencies). Decompose: STUDENT(sid, sname), COURSE(cid, ctitle), ENROL(sid, cid, grade).
8. Is (A,B), (B,C) a lossless decomposition of R(A,B,C) with F = {A → B}? — $R_1 \cap R_2$ = {B}; B^+ = {B} is a key of neither ⇒ **lossy**.
9. Is (A,B), (A,C) lossless for R(A,B,C) with F = {A → B}? — A^+ = {A,B}, a key of (A,B) ⇒ **lossless**.
10. Find the canonical cover of {A → BC, B → C, A → B, AB → C}. — **{A → B, B → C}** (Chapter 17).

G.7 Relational Algebra Problems (10) with Answers

Using STUDENT(sid, name, dept, cgpa), COURSE(cid, title, dept), ENROLL(sid, cid, grade).

1. Names of CSE students. — π_name(σ_{dept='CSE'}(STUDENT))
2. Names and titles of all enrolments. — π_{name,title}(STUDENT ⋈ ENROLL ⋈ COURSE)
3. Students who took no course. — π_sid(STUDENT) − π_sid(ENROLL)
4. Students who took course C1 or C2. — π_sid(σ_{cid='C1' ∨ cid='C2'}(ENROLL))
5. Students who took both C1 and C2. — π_sid(σ_{cid='C1'}(ENROLL)) ∩ π_sid(σ_{cid='C2'}(ENROLL))
6. Students who took C1 but not C2. — π_sid(σ_{cid='C1'}(ENROLL)) − π_sid(σ_{cid='C2'}(ENROLL))
7. Students who took every CSE course. — π_{sid,cid}(ENROLL) ÷ π_cid(σ_{dept='CSE'}(COURSE))
8. Students with an A in a course of another department. — π_sid(σ_{grade='A' ∧ S.dept ≠ C.dept}(STUDENT ⋈ ENROLL ⋈ COURSE))
9. Highest CGPA. — π_cgpa(STUDENT) − π_{A.cgpa}(σ_{A.cgpa<B.cgpa}(ρ_A(STUDENT) × ρ_B(STUDENT)))
10. Average CGPA per department (extended algebra). — dept $\mathscr{G}$_{AVG(cgpa)}(STUDENT)

G.8 Transaction and Serializability Problems (10) with Answers

1. S: r1(A) r2(A) w1(A) w2(A). Conflict serializable? — Edges T1→T2 (r1 before w2) and T2→T1 (r2 before w1): **cycle — no** (a lost update).
2. S: r1(A) w1(A) r2(A) w2(A) r1(B) w1(B). — Only T1→T2 edges: **yes**, equivalent to T1, T2.
3. S: r1(X) r2(Y) w1(Y) w2(X). — r2(Y)→w1(Y): T2→T1; r1(X)→w2(X): T1→T2: **cycle — no**.
4. S: r1(A) w2(A) w1(A) w3(A). View serializable? — **Yes** (view equivalent to T1, T2, T3), though not conflict serializable (Chapter 29).
5. w1(A) r2(A) c2 a1 — recoverable? — **No**: T2 committed after reading from T1, which aborted.
6. w1(A) r2(A) c1 c2 — classify. — **Recoverable**, not cascadeless.
7. Under wait-die, T1 (ts 5) requests a lock held by T2 (ts 10). — T1 is older ⇒ **waits**.
8. Under wound-wait, T2 (ts 10) requests a lock held by T1 (ts 5). — T2 is younger ⇒ **waits**.
9. Log: <T1 start> <T1,A,10,20> <T2 start> <T2,B,5,15> <T1 commit> **crash**. Actions? — **Redo T1** (A = 20); **undo T2** (B = 5).
10. Is `lock-X(A) unlock(A) lock-X(B) unlock(B)` two-phase? — **No**: a lock is acquired after a release.

G.9 Query Optimization Problems (10) with Answers

1. b_R = 400, b_S = 1,000, M = 22. Block nested-loop cost with R outer? — ⌈400/20⌉ × 1,000 + 400 = **20,400**.
2. Same, with S outer? — ⌈1,000/20⌉ × 400 + 1,000 = **21,000**; R outer is better.
3. Hash join cost for Problem 1? — 3 × (400 + 1,000) = **4,200**.
4. Sort a 1,000-block file with M = 11. Passes? — Pass 0: ⌈1000/11⌉ = 91 runs; merges of 10: 91 → 10 → 1 = 2 merge passes; **3 passes**, ≈ 2 × 1,000 × 3 = 6,000 transfers.
5. n = 20,000 rows, V(dept) = 5. Rows for `dept = 'CSE'`? — **4,000**.
6. Rows for `cgpa > 3.5` with cgpa in [2.0, 4.0]? — 20,000 × 0.5/2.0 = **5,000**.
7. Rows for both conditions assuming independence? — 20,000 × 0.2 × 0.25 = **1,000**.
8. |ENROLL ⋈ STUDENT| with 100,000 enrolments, key–foreign-key join. — **100,000**.

9. Left-deep join orders for 6 relations? — 6! = **720**.
10. Rewrite `WHERE YEAR(d) = 2025` to be index-friendly. — `WHERE d >= '2025-01-01' AND d < '2026-01-01'`.

G.10 RAID Problems (10) with Answers

1. Usable capacity of RAID 5 with 5 × 4 TB disks. — (5 − 1) × 4 = **16 TB**.
2. RAID 6 with 8 × 2 TB. — (8 − 2) × 2 = **12 TB**, tolerating 2 failures.
3. RAID 10 with 8 × 2 TB. — 8 / 2 × 2 = **8 TB**.
4. RAID 0 with 4 × 1 TB, fault tolerance? — **4 TB, none**.
5. Parity of 1010, 0110, 1100. — 1010 ⊕ 0110 = 1100; 1100 ⊕ 1100 = **0000**.
6. Recover D1 given D0 = 1010, D2 = 1100, P = 0000. — 1010 ⊕ 1100 ⊕ 0000 = **0110**.
7. Array of 40 disks, MTTF 800,000 h, no redundancy: time to first failure? — **20,000 h** ≈ 2.3 years.
8. I/Os for one small write on RAID 5 and RAID 6? — **4** and **6**.
9. Choose RAID for (a) transaction log, (b) OLTP data, (c) archive. — (a) **RAID 1**, (b) **RAID 10**, (c) **RAID 6**.
10. Downtime per year at 99.99 % availability. — 0.0001 × 8,760 h = **52.6 minutes**.

APPENDIX H

Practical Lab Exercises

This laboratory section is self-contained: run the setup script once, then work through sixty tasks of increasing difficulty. Every solution below was executed against the setup data while preparing the book. Try each task yourself before reading its solution.

H.1 Setup: Creating and Populating university_db

The script is identical to Chapter 11, so results match the tables printed throughout the book.

```
DROP DATABASE IF EXISTS university_db;
CREATE DATABASE university_db;
USE university_db;

CREATE TABLE department (
    dept_id   CHAR(3)      PRIMARY KEY,
    dept_name VARCHAR(80)  NOT NULL UNIQUE,
    building  VARCHAR(40)
);

CREATE TABLE instructor (
    instructor_id INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    designation   VARCHAR(40)  NOT NULL,
    salary        DECIMAL(10,2) CHECK (salary > 0),
    dept_id       CHAR(3)      NOT NULL,
    CONSTRAINT fk_inst_dept FOREIGN KEY (dept_id)
        REFERENCES department(dept_id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE student (
    student_id  CHAR(10)     PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(100) UNIQUE,
    dob         DATE,
    cgpa        DECIMAL(3,2) CHECK (cgpa BETWEEN 0.00 AND 4.00),
    dept_id     CHAR(3)      NOT NULL,
    admitted_on DATE         NOT NULL,
    CONSTRAINT fk_stu_dept FOREIGN KEY (dept_id)
        REFERENCES department(dept_id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE course (
    course_id   INT AUTO_INCREMENT PRIMARY KEY,
    course_code VARCHAR(10)  NOT NULL UNIQUE,
    title       VARCHAR(120) NOT NULL,
    credit      DECIMAL(2,1) NOT NULL DEFAULT 3.0 CHECK (credit BETWEEN 0.5 AND 4.0),
    dept_id     CHAR(3)      NOT NULL,
    CONSTRAINT fk_course_dept FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

CREATE TABLE classroom (
```

```
    room_id  VARCHAR(10) PRIMARY KEY,
    building VARCHAR(40) NOT NULL,
    capacity INT         NOT NULL CHECK (capacity > 0)
);

CREATE TABLE enrollment (
    enrollment_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id    CHAR(10) NOT NULL,
    course_id     INT      NOT NULL,
    semester      VARCHAR(10) NOT NULL,
    grade         CHAR(2)  NULL,
    CONSTRAINT uq_enrol UNIQUE (student_id, course_id, semester),
    CONSTRAINT fk_enr_stu FOREIGN KEY (student_id)
        REFERENCES student(student_id) ON DELETE CASCADE,
    CONSTRAINT fk_enr_course FOREIGN KEY (course_id)
        REFERENCES course(course_id) ON DELETE RESTRICT
);

CREATE TABLE examination (
    exam_id   INT AUTO_INCREMENT PRIMARY KEY,
    course_id INT         NOT NULL,
    exam_date DATE        NOT NULL,
    room_id   VARCHAR(10) NOT NULL,
    CONSTRAINT fk_exam_course FOREIGN KEY (course_id) REFERENCES course(course_id),
    CONSTRAINT fk_exam_room   FOREIGN KEY (room_id)   REFERENCES classroom(room_id)
);

CREATE TABLE result (
    result_id  INT AUTO_INCREMENT PRIMARY KEY,
    exam_id    INT      NOT NULL,
    student_id CHAR(10) NOT NULL,
    marks      DECIMAL(5,2) CHECK (marks BETWEEN 0 AND 100),
    CONSTRAINT uq_result UNIQUE (exam_id, student_id),
    CONSTRAINT fk_res_exam FOREIGN KEY (exam_id)    REFERENCES examination(exam_id),
    CONSTRAINT fk_res_stu  FOREIGN KEY (student_id) REFERENCES student(student_id)
);
```

```
INSERT INTO department VALUES
 ('CSE','Computer Science and Engineering','Academic Building 3'),
 ('EEE','Electrical and Electronic Engineering','Academic Building 2'),
 ('ME', 'Mechanical Engineering','Academic Building 1'),
 ('BBA','Business Administration','Academic Building 4');

INSERT INTO instructor (name, designation, salary, dept_id) VALUES
 ('Nasrin Akter','Professor',95000,'CSE'),
 ('Imran Hossain','Associate Professor',78000,'CSE'),
 ('Sabina Yasmin','Assistant Professor',62000,'CSE'),
 ('Tanvir Ahmed','Professor',94000,'EEE'),
 ('Farhana Islam','Lecturer',45000,'EEE'),
 ('Mahmudul Hasan','Assistant Professor',60000,'ME'),
 ('Shirin Sultana','Associate Professor',75000,'BBA');

INSERT INTO student VALUES
 ('ASH1901001','Rahim Uddin','rahim@nstu.edu.bd','2001-03-14',3.75,'CSE','2019-01-10'),
 ('ASH1901002','Karim Mia','karim@nstu.edu.bd','2001-07-02',3.60,'EEE','2019-01-10'),
 ('ASH1901003','Nila Rahman','nila@nstu.edu.bd','2002-01-25',3.90,'CSE','2019-01-10'),
 ('ASH1901004','Sumaiya Akter','sumaiya@nstu.edu.bd','2001-11-30',3.20,'CSE','2019-01-10
    ↪ '),
```

```
 ('ASH1901005','Jubayer Hossain','jubayer@nstu.edu.bd','2000-09-18',2.85,'ME','
    ↪ 2019-01-10'),
 ('ASH2001001','Tania Sultana','tania@nstu.edu.bd','2002-05-05',3.95,'CSE','2020-01-12')
    ↪ ,
 ('ASH2001002','Rakib Hasan','rakib@nstu.edu.bd','2002-02-19',3.10,'EEE','2020-01-12'),
 ('ASH2001003','Mitu Barua','mitu@nstu.edu.bd','2002-08-08',NULL,'BBA','2020-01-12'),
 ('ASH2001004','Shakib Al Hasan','shakib@nstu.edu.bd','2001-12-01',3.45,'BBA','
    ↪ 2020-01-12'),
 ('ASH2001005','Fahmida Nabi','fahmida@nstu.edu.bd','2002-04-21',3.70,'ME','2020-01-12')
    ↪ ;

INSERT INTO course (course_code, title, credit, dept_id) VALUES
 ('CSE3201','Database Management Systems',3.0,'CSE'),
 ('CSE3203','Operating Systems',3.0,'CSE'),
 ('CSE2101','Data Structures',3.0,'CSE'),
 ('CSE4101','Machine Learning',3.0,'CSE'),
 ('EEE3101','Digital Electronics',3.0,'EEE'),
 ('EEE3103','Signals and Systems',3.0,'EEE'),
 ('ME2101', 'Engineering Mechanics',3.0,'ME'),
 ('BBA1101','Principles of Management',3.0,'BBA');

INSERT INTO classroom VALUES
 ('AB3-401','Academic Building 3',60),
 ('AB3-402','Academic Building 3',45),
 ('AB2-201','Academic Building 2',80),
 ('AB4-101','Academic Building 4',120);

INSERT INTO enrollment (student_id, course_id, semester, grade) VALUES
 ('ASH1901001',1,'5th','A'),   ('ASH1901001',2,'5th','A-'),
 ('ASH1901001',3,'3rd','B+'),  ('ASH1901003',1,'5th','A'),
 ('ASH1901003',2,'5th','A'),   ('ASH1901003',4,'7th','A-'),
 ('ASH1901004',1,'5th','B'),   ('ASH1901004',3,'3rd','C+'),
 ('ASH2001001',1,'5th','A'),   ('ASH2001001',3,'3rd','A'),
 ('ASH1901002',5,'5th','B+'),  ('ASH1901002',6,'5th','B'),
 ('ASH2001002',5,'5th','C'),   ('ASH1901005',7,'3rd','B-'),
 ('ASH2001005',7,'3rd','A-'),  ('ASH2001004',8,'1st','B+'),
 ('ASH2001003',8,'1st',NULL);

INSERT INTO examination (course_id, exam_date, room_id) VALUES
 (1,'2024-06-10','AB3-401'), (2,'2024-06-13','AB3-401'),
 (3,'2024-06-16','AB3-402'), (5,'2024-06-11','AB2-201'),
 (8,'2024-06-12','AB4-101');

INSERT INTO result (exam_id, student_id, marks) VALUES
 (1,'ASH1901001',82.00), (1,'ASH1901003',88.50), (1,'ASH1901004',67.00),
 (1,'ASH2001001',91.00), (2,'ASH1901001',78.00), (2,'ASH1901003',85.00),
 (3,'ASH1901001',71.50), (3,'ASH1901004',58.00), (3,'ASH2001001',89.00),
 (4,'ASH1901002',74.00), (4,'ASH2001002',52.00), (5,'ASH2001004',76.00);
```

After running it, verify the load:

```
SELECT 'department' AS tbl, COUNT(*) AS n FROM department UNION ALL
SELECT 'instructor', COUNT(*) FROM instructor UNION ALL
SELECT 'student',    COUNT(*) FROM student    UNION ALL
SELECT 'course',     COUNT(*) FROM course     UNION ALL
SELECT 'classroom',  COUNT(*) FROM classroom  UNION ALL
SELECT 'enrollment', COUNT(*) FROM enrollment UNION ALL
SELECT 'examination',COUNT(*) FROM examination UNION ALL
```

```sql
SELECT 'result',     COUNT(*) FROM result;
```

Expected: 4, 7, 10, 8, 4, 17, 5, 12 rows respectively.

H.2 Basic Queries (1–20)

Skills: SELECT, WHERE, ORDER BY, DISTINCT, LIKE, BETWEEN, IN, IS NULL, LIMIT, simple aggregates.

L1. Display all columns of every department.

```sql
SELECT * FROM department;
```

L2. List student names and CGPAs.

```sql
SELECT name, cgpa FROM student;
```

L3. List students of the EEE department.

```sql
SELECT student_id, name FROM student WHERE dept_id = 'EEE';
```

L4. List students with CGPA of at least 3.50, highest first.

```sql
SELECT name, cgpa FROM student WHERE cgpa >= 3.50 ORDER BY cgpa DESC;
```

L5. List the distinct designations of instructors.

```sql
SELECT DISTINCT designation FROM instructor ORDER BY designation;
```

L6. Find students whose name starts with “S”.

```sql
SELECT name FROM student WHERE name LIKE 'S%';
```

L7. Find instructors earning between 60,000 and 80,000.

```sql
SELECT name, salary FROM instructor WHERE salary BETWEEN 60000 AND 80000;
```

L8. List courses of CSE or ME.

```sql
SELECT course_code, title FROM course WHERE dept_id IN ('CSE', 'ME');
```

L9. Find students whose CGPA has not been recorded.

```sql
SELECT student_id, name FROM student WHERE cgpa IS NULL;
```

L10. Show the three youngest students.

```sql
SELECT name, dob FROM student ORDER BY dob DESC LIMIT 3;
```

L11. Count all students.

```sql
SELECT COUNT(*) AS total_students FROM student;
```

L12. Show the highest, lowest and average CGPA.

```sql
SELECT MAX(cgpa) AS highest, MIN(cgpa) AS lowest, ROUND(AVG(cgpa), 2) AS average FROM
    ↪ student;
```

L13. Total monthly salary bill of all instructors.

```sql
SELECT SUM(salary) AS salary_bill FROM instructor;
```

L14. Show each student's CGPA as a percentage, aliased.

```sql
SELECT name, ROUND(cgpa * 25, 1) AS percentage FROM student WHERE cgpa IS NOT NULL;
```

L15. List classrooms with capacity above 50, largest first.

```sql
SELECT room_id, capacity FROM classroom WHERE capacity > 50 ORDER BY capacity DESC;
```

L16. List enrollments without a grade.

```sql
SELECT student_id, course_id, semester FROM enrollment WHERE grade IS NULL;
```

L17. Students admitted in 2019.

```sql
SELECT name, admitted_on FROM student
WHERE admitted_on >= '2019-01-01' AND admitted_on < '2020-01-01';
```

L18. Courses whose title contains "System".

```sql
SELECT course_code, title FROM course WHERE title LIKE '%System%';
```

L19. Examinations held in June 2024, in date order.

```sql
SELECT exam_id, course_id, exam_date FROM examination
WHERE exam_date BETWEEN '2024-06-01' AND '2024-06-30' ORDER BY exam_date;
```

L20. Results with marks below 60.

```sql
SELECT exam_id, student_id, marks FROM result WHERE marks < 60 ORDER BY marks;
```

H.3 Intermediate Queries (21–40)

Skills: GROUP BY, HAVING, joins, outer joins, self joins, subqueries, EXISTS, UNION, CASE, date functions.

L21. Number of students per department.

```sql
SELECT dept_id, COUNT(*) AS students FROM student GROUP BY dept_id;
```

L22. Departments whose average CGPA exceeds 3.40.

```sql
SELECT dept_id, ROUND(AVG(cgpa), 2) AS avg_cgpa
FROM student GROUP BY dept_id HAVING AVG(cgpa) > 3.40;
```

L23. Each student with their department name.

```sql
SELECT s.name, d.dept_name
FROM student s JOIN department d ON d.dept_id = s.dept_id;
```

L24. Each instructor with their department's building.

```sql
SELECT i.name, i.designation, d.building
FROM instructor i JOIN department d ON d.dept_id = i.dept_id;
```

L25. All courses with their enrollment counts, including zero.

```sql
SELECT c.course_code, COUNT(e.enrollment_id) AS enrolled
FROM course c LEFT JOIN enrollment e ON e.course_id = c.course_id
GROUP BY c.course_id, c.course_code ORDER BY enrolled DESC;
```

L26. Student name, course title and grade for every graded enrollment.

```sql
SELECT s.name, c.title, e.grade
FROM enrollment e
JOIN student s ON s.student_id = e.student_id
JOIN course  c ON c.course_id  = e.course_id
WHERE e.grade IS NOT NULL;
```

L27. Average marks per examination with the course code.

```sql
SELECT x.exam_id, c.course_code, ROUND(AVG(r.marks), 2) AS avg_marks
FROM examination x
JOIN course c ON c.course_id = x.course_id
JOIN result r ON r.exam_id = x.exam_id
GROUP BY x.exam_id, c.course_code;
```

L28. Students with CGPA above the university average.

```sql
SELECT name, cgpa FROM student WHERE cgpa > (SELECT AVG(cgpa) FROM student);
```

L29. Students who have scored an A in any course (using EXISTS).

```sql
SELECT s.name FROM student s
WHERE EXISTS (SELECT 1 FROM enrollment e WHERE e.student_id = s.student_id AND e.grade =
    ↪ 'A');
```

L30. Courses that no student has taken (using NOT EXISTS).

```sql
SELECT course_code FROM course c
WHERE NOT EXISTS (SELECT 1 FROM enrollment e WHERE e.course_id = c.course_id);
```

L31. Pairs of students in the same department (each pair once).

```sql
SELECT a.name AS student_1, b.name AS student_2, a.dept_id
FROM student a JOIN student b ON a.dept_id = b.dept_id AND a.student_id < b.student_id;
```

L32. A combined directory of students and instructors.

```sql
SELECT name, 'Student' AS role FROM student
UNION ALL
SELECT name, 'Instructor' FROM instructor ORDER BY role, name;
```

L33. Instructors earning more than every Lecturer.

```sql
SELECT name, salary FROM instructor
WHERE salary > ALL (SELECT salary FROM instructor WHERE designation = 'Lecturer');
```

L34. Label each result as Pass (≥ 40) or Fail.

```sql
SELECT student_id, exam_id, marks,
       CASE WHEN marks >= 40 THEN 'Pass' ELSE 'Fail' END AS status
FROM result;
```

L35. Each student's age in completed years.

```sql
SELECT name, TIMESTAMPDIFF(YEAR, dob, CURDATE()) AS age FROM student ORDER BY age DESC;
```

L36. Number of enrollments per semester, ungraded counted separately.

```sql
SELECT semester, COUNT(*) AS total, COUNT(*) - COUNT(grade) AS ungraded
FROM enrollment GROUP BY semester ORDER BY semester;
```

L37. Departments with more than one instructor.

```sql
SELECT dept_id, COUNT(*) AS staff FROM instructor GROUP BY dept_id HAVING COUNT(*) > 1;
```

L38. Highest mark in each examination together with the student who scored it.

```sql
SELECT r.exam_id, s.name, r.marks
FROM result r JOIN student s ON s.student_id = r.student_id
WHERE r.marks = (SELECT MAX(r2.marks) FROM result r2 WHERE r2.exam_id = r.exam_id);
```

L39. Departments that have courses but no instructors earning below 50,000.

```sql
SELECT DISTINCT c.dept_id FROM course c
WHERE NOT EXISTS (SELECT 1 FROM instructor i WHERE i.dept_id = c.dept_id AND i.salary <
    ↪ 50000);
```

L40. Number of courses offered by each department, including departments with none.

```sql
SELECT d.dept_name, COUNT(c.course_id) AS courses
FROM department d LEFT JOIN course c ON c.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```

H.4 Advanced Queries (41–60)

Skills: CTEs, window functions, ROLLUP, views, triggers, stored procedures, functions, indexes, transactions.

L41. Rank students by CGPA within each department.

```sql
SELECT dept_id, name, cgpa,
       RANK() OVER (PARTITION BY dept_id ORDER BY cgpa DESC) AS dept_rank
FROM student WHERE cgpa IS NOT NULL;
```

L42. Departments whose average CGPA is above the average of departmental averages (CTE).

```sql
WITH dept_avg AS (
    SELECT dept_id, AVG(cgpa) AS avg_cgpa FROM student GROUP BY dept_id
)
SELECT dept_id, ROUND(avg_cgpa, 2) AS avg_cgpa FROM dept_avg
WHERE avg_cgpa > (SELECT AVG(avg_cgpa) FROM dept_avg);
```

L43. Running total of instructor salaries in descending salary order.

```sql
SELECT name, salary, SUM(salary) OVER (ORDER BY salary DESC, name) AS running_total
FROM instructor;
```

L44. Salary subtotals per department and grand total.

```sql
SELECT dept_id, SUM(salary) AS total FROM instructor GROUP BY dept_id WITH ROLLUP;
```

L45. Students enrolled in every CSE course that has at least one enrollment (division).

```sql
SELECT e.student_id FROM enrollment e JOIN course c ON c.course_id = e.course_id
WHERE c.dept_id = 'CSE'
GROUP BY e.student_id
HAVING COUNT(DISTINCT e.course_id) =
       (SELECT COUNT(DISTINCT e2.course_id) FROM enrollment e2
        JOIN course c2 ON c2.course_id = e2.course_id WHERE c2.dept_id = 'CSE');
```

L46. Pivot: number of enrollments per semester as columns for each department.

```sql
SELECT s.dept_id,
       SUM(e.semester = '1st') AS sem1, SUM(e.semester = '3rd') AS sem3,
       SUM(e.semester = '5th') AS sem5, SUM(e.semester = '7th') AS sem7
FROM enrollment e JOIN student s ON s.student_id = e.student_id
GROUP BY s.dept_id;
```

L47. Create a view of the full result sheet and query it for one student.

```sql
CREATE OR REPLACE VIEW vw_lab_result_sheet AS
SELECT s.student_id, s.name, c.course_code, e.semester, e.grade, r.marks
FROM enrollment e
JOIN student s ON s.student_id = e.student_id
JOIN course  c ON c.course_id  = e.course_id
LEFT JOIN examination x ON x.course_id = c.course_id
LEFT JOIN result r ON r.exam_id = x.exam_id AND r.student_id = s.student_id;

SELECT course_code, semester, grade, marks FROM vw_lab_result_sheet
WHERE student_id = 'ASH1901001' ORDER BY semester;
```

L48. Create a security view hiding CGPA and date of birth.

```sql
CREATE OR REPLACE VIEW vw_lab_directory AS
SELECT s.student_id, s.name, d.dept_name
FROM student s JOIN department d ON d.dept_id = s.dept_id;
SELECT * FROM vw_lab_directory ORDER BY dept_name, name;
```

L49. A function converting marks to a letter grade, used in a query.

```sql
DROP FUNCTION IF EXISTS fn_lab_letter;
DELIMITER $$
CREATE FUNCTION fn_lab_letter (p_marks DECIMAL(5,2)) RETURNS CHAR(2) DETERMINISTIC
BEGIN
    RETURN CASE WHEN p_marks >= 80 THEN 'A' WHEN p_marks >= 70 THEN 'B'
                WHEN p_marks >= 60 THEN 'C' WHEN p_marks >= 40 THEN 'D' ELSE 'F' END;
END$$
DELIMITER ;
SELECT student_id, marks, fn_lab_letter(marks) AS letter FROM result ORDER BY marks DESC
    ↪ ;
```

L50. A stored procedure returning the number of students in a given department.

```sql
DROP PROCEDURE IF EXISTS sp_lab_dept_count;
DELIMITER $$
CREATE PROCEDURE sp_lab_dept_count (IN p_dept CHAR(3), OUT p_count INT)
BEGIN
    SELECT COUNT(*) INTO p_count FROM student WHERE dept_id = p_dept;
END$$
```

```
DELIMITER ;
CALL sp_lab_dept_count('CSE', @n);
SELECT @n AS cse_students;
```

L51. A trigger rejecting marks outside 0–100 on insert.

```
DROP TRIGGER IF EXISTS trg_lab_marks;
DELIMITER $$
CREATE TRIGGER trg_lab_marks BEFORE INSERT ON result
FOR EACH ROW
BEGIN
    IF NEW.marks < 0 OR NEW.marks > 100 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Marks must be between 0 and 100';
    END IF;
END$$
DELIMITER ;
```

L52. An audit table and trigger recording every grade change.

```
CREATE TABLE IF NOT EXISTS lab_grade_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY, enrollment_id INT,
    old_grade CHAR(2), new_grade CHAR(2), changed_at DATETIME
);
DROP TRIGGER IF EXISTS trg_lab_grade_audit;
DELIMITER $$
CREATE TRIGGER trg_lab_grade_audit AFTER UPDATE ON enrollment
FOR EACH ROW
BEGIN
    IF NOT (NEW.grade <=> OLD.grade) THEN
        INSERT INTO lab_grade_audit (enrollment_id, old_grade, new_grade, changed_at)
        VALUES (OLD.enrollment_id, OLD.grade, NEW.grade, NOW());
    END IF;
END$$
DELIMITER ;
UPDATE enrollment SET grade = 'B+' WHERE enrollment_id = 8;
SELECT enrollment_id, old_grade, new_grade FROM lab_grade_audit;
```

L53. A transaction that transfers a student to another department and rolls back on demand.

```
START TRANSACTION;
UPDATE student SET dept_id = 'EEE' WHERE student_id = 'ASH1901004';
SELECT student_id, dept_id FROM student WHERE student_id = 'ASH1901004';   -- EEE inside
    ↪ the txn
ROLLBACK;
SELECT student_id, dept_id FROM student WHERE student_id = 'ASH1901004';   -- CSE again
```

L54. Use a savepoint to keep one change and discard another.

```
START TRANSACTION;
UPDATE instructor SET salary = salary + 1000 WHERE instructor_id = 5;
SAVEPOINT after_raise;
UPDATE instructor SET salary = salary * 0.5 WHERE instructor_id = 5;      -- a mistaken
    ↪ cut
ROLLBACK TO SAVEPOINT after_raise;
COMMIT;
SELECT name, salary FROM instructor WHERE instructor_id = 5;
```

Expected: Farhana Islam, 46000.00 — the raise is kept, the mistaken cut is discarded.

L55. Create an index for looking up enrollments by student and show that it is used.

```
CREATE INDEX idx_lab_enrol_student ON enrollment(student_id, semester);
EXPLAIN SELECT course_id, grade FROM enrollment WHERE student_id = 'ASH1901001';
```

L56. Create a covering index for a department CGPA report and inspect the plan.

```
CREATE INDEX idx_lab_student_cover ON student(dept_id, cgpa, name);
EXPLAIN SELECT name, cgpa FROM student WHERE dept_id = 'CSE' ORDER BY cgpa DESC;
```

L57. Top two students per department (window function in a CTE).

```
WITH ranked AS (
    SELECT dept_id, name, cgpa,
           ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY cgpa DESC) AS rn
    FROM student WHERE cgpa IS NOT NULL
)
SELECT dept_id, name, cgpa FROM ranked WHERE rn <= 2 ORDER BY dept_id, rn;
```

L58. Each student's marks compared with the average of the same examination.

```
SELECT r.exam_id, r.student_id, r.marks,
       ROUND(AVG(r.marks) OVER (PARTITION BY r.exam_id), 2) AS exam_avg,
       ROUND(r.marks - AVG(r.marks) OVER (PARTITION BY r.exam_id), 2) AS difference
FROM result r ORDER BY r.exam_id, r.marks DESC;
```

L59. A recursive CTE listing the semester sequence 1 to 8.

```
WITH RECURSIVE sem(n) AS (
    SELECT 1
    UNION ALL
    SELECT n + 1 FROM sem WHERE n < 8
)
SELECT n AS semester_no FROM sem;
```

L60. A procedure that raises all salaries in a department by a percentage inside a transaction and reports the rows changed.

```
DROP PROCEDURE IF EXISTS sp_lab_raise;
DELIMITER $$
CREATE PROCEDURE sp_lab_raise (IN p_dept CHAR(3), IN p_pct DECIMAL(5,2), OUT p_rows INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    UPDATE instructor SET salary = ROUND(salary * (1 + p_pct / 100), 2) WHERE dept_id =
    ↪ p_dept;
    SET p_rows = ROW_COUNT();
    COMMIT;
END$$
DELIMITER ;
CALL sp_lab_raise('BBA', 5, @rows);
SELECT @rows AS rows_changed;
```

H.5 Lab Assessment Checklist

Skill	Tasks	Achieved when you can…
Filtering and sorting	L1–L20	Write correct predicates including NULL tests and ranges
Aggregation	L11–L13, L21–L22, L36–L37	Choose correctly between WHERE and HAVING
Joins	L23–L27, L31, L40	Choose inner versus outer joins deliberately
Subqueries	L28–L30, L33, L38–L39, L45	Use EXISTS / NOT EXISTS safely
Advanced SQL	L41–L46, L57–L59	Use CTEs, window functions and ROLLUP
Database objects	L47–L52, L60	Create views, functions, procedures and triggers
Transactions and indexes	L53–L56	Control commits and read execution plans

References

Textbooks

1. Silberschatz, A., Korth, H. F., and Sudarshan, S. *Database System Concepts*, 7th ed. McGraw-Hill, 2019.
2. Elmasri, R., and Navathe, S. B. *Fundamentals of Database Systems*, 7th ed. Pearson, 2015.
3. Ramakrishnan, R., and Gehrke, J. *Database Management Systems*, 3rd ed. McGraw-Hill, 2003.
4. Date, C. J. *An Introduction to Database Systems*, 8th ed. Addison-Wesley, 2003.
5. Garcia-Molina, H., Ullman, J. D., and Widom, J. *Database Systems: The Complete Book*, 2nd ed. Pearson, 2008.
6. Connolly, T., and Begg, C. *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Pearson, 2015.
7. Gray, J., and Reuter, A. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.
8. Kimball, R., and Ross, M. *The Data Warehouse Toolkit*, 3rd ed. Wiley, 2013.
9. Inmon, W. H. *Building the Data Warehouse*, 4th ed. Wiley, 2005.
10. Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly, 2017.
11. Sadalage, P. J., and Fowler, M. *NoSQL Distilled*. Addison-Wesley, 2012.

Foundational Papers

12. Codd, E. F. "A Relational Model of Data for Large Shared Data Banks." *Communications of the ACM*, 13(6), 1970.
13. Chen, P. P. "The Entity-Relationship Model — Toward a Unified View of Data." *ACM Transactions on Database Systems*, 1(1), 1976.
14. Bayer, R., and McCreight, E. "Organization and Maintenance of Large Ordered Indexes." *Acta Informatica*, 1(3), 1972.
15. Eswaran, K. P., Gray, J. N., Lorie, R. A., and Traiger, I. L. "The Notions of Consistency and Predicate Locks in a Database System." *Communications of the ACM*, 19(11), 1976.
16. Selinger, P. G., et al. "Access Path Selection in a Relational Database Management System." *Proceedings of ACM SIGMOD*, 1979.
17. Patterson, D. A., Gibson, G., and Katz, R. H. "A Case for Redundant Arrays of Inexpensive Disks (RAID)." *Proceedings of ACM SIGMOD*, 1988.
18. Mohan, C., et al. "ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging." *ACM Transactions on Database Systems*, 17(1), 1992.
19. Berenson, H., et al. "A Critique of ANSI SQL Isolation Levels." *Proceedings of ACM SIGMOD*, 1995.
20. Gilbert, S., and Lynch, N. "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services." *ACM SIGACT News*, 33(2), 2002.
21. Chang, F., et al. "Bigtable: A Distributed Storage System for Structured Data." *Proceedings of OSDI*, 2006.
22. DeCandia, G., et al. "Dynamo: Amazon's Highly Available Key-Value Store." *Proceedings of SOSP*, 2007.
23. Cahill, M. J., Röhm, U., and Fekete, A. D. "Serializable Isolation for Snapshot Databases." *Pro-*

ceedings of ACM SIGMOD, 2008.

24. Corbett, J. C., et al. "Spanner: Google's Globally-Distributed Database." *Proceedings of OSDI*, 2012.
25. Ongaro, D., and Ousterhout, J. "In Search of an Understandable Consensus Algorithm (Raft)." *Proceedings of USENIX ATC*, 2014.

Standards and Official Documentation

26. ISO/IEC 9075. *Information Technology — Database Languages — SQL* (SQL standard, multiple parts and editions).
27. Oracle Corporation. *MySQL 8.0 Reference Manual*. `https://dev.mysql.com/doc/refman/8.0/en/`
28. The PostgreSQL Global Development Group. *PostgreSQL Documentation*. `https://www.postgresql.org/docs/`
29. MariaDB Foundation. *MariaDB Server Documentation*. `https://mariadb.com/kb/en/documentation/`
30. OWASP Foundation. *SQL Injection Prevention Cheat Sheet*. `https://cheatsheetseries.owasp.org/`
31. MongoDB, Inc. *MongoDB Manual*. `https://www.mongodb.com/docs/manual/`